基于R应用的统计学丛书

多元统计分析

——R与Python的实现

Multivariate Statistical Analysis with R and Python

吴喜之 编著

k-Means Clustering of 2 Colours

k-Means Clustering of 4 Colours

k-Means Clustering of 16 Colours

k-Means Clustering of 32 Colours

中国人民大学出版社

基于R应用的统计学丛书

	统计学——基于R（第二版）	贾俊平
●	多元统计分析——R与Python的实现	吴喜之
	多元统计分析——基于R	费　宇
	时间序列分析——基于R	王　燕
	应用回归及分类——基于R	吴喜之
	非参数统计——基于R	王　星

本书特色

◎ 本书基于数据驱动的思维，以R和Python作为编程工具对大量实际案例做了分析。介绍了回归、分类及各种多元分析方法（包括主成分分析、因子分析、聚类分析、典型相关分析、二元及多元对应分析、多维尺度变换）的概念、应用及可能的误区。

◎ 本书并不要求读者有数理统计的预备知识。本书在一开始为读者提供了理解概念所必需的代数知识，并附有学习编程语言的练习。读者可以通过本书学到机器学习的最重要内容，同时学习编程语言，打下数据科学的基础。

多元统计分析

——R与Python的实现

Multivariate Statistical Analysis with R and Python

策划编辑　王伟娟
责任编辑　胡志敏　刘　璐
封面设计　贾俊平　拾光书坊

中国人民大学出版社　www.crup.com.cn
了解图书出版信息　下载教学辅助资料

ISBN 978-7-300-26655-8
9 787300 266558 >
定价：39.80元

基于R应用的统计学丛书

多元统计分析

——R与Python的实现

Multivariate Statistical Analysis with R and Python

吴喜之 编著

中国人民大学出版社
·北京·

图书在版编目 (CIP) 数据

多元统计分析: R 与 Python 的实现 / 吴喜之编著. —北京: 中国人民大学出版社, 2019.1
(基于 R 应用的统计学丛书)
ISBN 978-7-300-26655-8

I. ①多 … II. ①吴 … III. 多元分析 – 统计分析 IV.①O212.4

中国版本图书馆 CIP 数据核字 (2019) 第 016562 号

基于 R 应用的统计学丛书

多元统计分析——R 与 Python 的实现

吴喜之　编著

Duoyuan Tongji Fenxi——R yu Python de Shixian

出版发行	中国人民大学出版社		
社　　址	北京中关村大街 31 号	**邮政编码**	100080
电　　话	010-62511242(总编室)	010-62511398(质管部)	
	010-82501766(邮购部)	010-62514148(门市部)	
	010-62515195(发行公司)	010-62515275(盗版举报)	
网　　址	http://www.crup.com.cn		
	http://www.ttrnet.com (人大教研网)		
经　　销	新华书店		
印　　刷	北京鑫丰华彩印有限公司		
规　　格	185mm× 260mm　16 开本	**版　　次**	2019 年 1 月第 1 版
印　　张	20.75 插页 1	**印　　次**	2019 年 1 月第 1 次印刷
字　　数	484 000	**定　　价**	39.80 元

版权所有　侵权必究　　印装差错　负责调换

前 言

目前, 多数国内多元分析教材有下面几个特点: (1) 着重于数学理论和推导. 这往往使得读者忘记了分析数据的本来目的, 把数据科学当成数学来学. (2) 关于多元正态分布的性质占据很大篇幅. 这实际上过分强调了对数据的狭隘数学假定, 而忽略了真实数据的复杂性和多样性. (3) 对真实的数据分析强调得不够. 一些典型的教材数据往往和现实世界差得很远, 对读者具有误导性. (4) 在所涉及的统计软件应用方面, 大多只限于商业软件. 笔者认为, 使用盗版国外软件不仅犯法, 而且最终会形成对这些盗版软件的依赖性而不能自拔. 使用 "傻瓜式" 商业软件对于学习编程没有任何好处, 反而会阻碍人们获得编程能力, 而是否会编程将如是否文盲一样影响人们的未来发展.

本书力图用简单通俗的语言阐述有关的基本概念, 使得非数学背景的读者亦可以较容易地理解. 考虑到许多读者的主要目标是应用, 本书通过案例来帮助理解各种方法的概念及计算. 本书也包括了传统多元分析课本所具有的数学公式, 目的在于方便读者查询.

本书基于实际数据分析, 在正文中通过 R 软件解释方法及计算输出, 同时在每一章都给出用 Python 3 计算这一章数据例子的代码.

本书包括传统多元分析教材通常包括的内容: 主成分分析、因子分析、聚类分析、判别分析、典型相关分析以及非传统的对应分析方法. 一些多元分析教材还包括完整的多重回归内容, 但这远远超出一个学期的课程内容. 本书对多元回归予以回顾, 介绍了部分机器学习回归方法, 判别分析则安排在 "分类" 一章, 并且介绍了部分机器学习分类方法. 关于回归和分类的更多内容及细节, 请读者参看笔者编写的由中国人民大学出版社出版的《应用回归及分类——基于 R》一书.

应用统计方法的学习, 应该是拉动式学习, 也就是遵从问题驱动或者数据驱动模式: 从数据出发, 需要什么就学什么. 如此, 所有学习都是目标明确的, 学的都是有用的知识和能力. 有的教师喜欢以 "打基础" 的名义给学生一大堆书去读, 这样的教学方法事倍功半.

和其他自然科学一样, 统计需要从数学课程中获得严密的逻辑思维能力, 还需要计算机编程能力. 这不仅需要学会针对一两个统计软件的编程, 还需要获得能迅速掌握任何开源软件的**泛型编程能力**. 有了来自数学的思维能力、想象力和逻辑, 以及计算机编程能力, 再加上学习自然科学所必需的批判性思维及对所面对实际问题的理解, 你就拥有了研究统计最重要的必要条件.

本书的排版是笔者用 LaTeX 软件实现的, 一切错误由笔者负责.

吴喜之

目 录

前 言

第 1 章 引 言

1.1 数据科学 …… 1
1.1.1 统计是数据科学吗? …… 1
1.1.2 计算机学科在数据科学中的地位 2
1.1.3 问题驱动应成为数据科学的基本思维方式 …… 2
1.2 多元分析的对象 …… 2
1.3 需要的工具 …… 3
1.4 各章的安排 …… 3
1.5 软件和编程 …… 4
1.6 如何教学 …… 4

第 2 章 矩阵代数回顾

2.1 矩 阵 …… 6
2.1.1 基本定义 …… 6
2.1.2 基本矩阵运算 …… 7
2.1.3 行列式 …… 8
2.1.4 矩阵的逆 …… 9
2.1.5 矩阵的广义逆 …… 9
2.1.6 Kronecker 积 …… 10
2.1.7 幂等矩阵 …… 10
2.1.8 向量空间 …… 11
2.1.9 正交性 …… 11
2.1.10 矩阵的秩 …… 11
2.1.11 矩阵的迹 …… 12
2.1.12 特征值 …… 12
2.1.13 广义特征值 …… 13
2.1.14 分块矩阵 …… 13
2.2 矩阵的分解 …… 15
2.2.1 矩阵的特征值分解 …… 15
2.2.2 奇异值分解及广义奇异值分解 16
2.2.3 QR 分解 …… 18
2.2.4 Cholesky 分解 …… 19
2.3 二次型 …… 20
2.3.1 定义 …… 20
2.3.2 二次型和矩阵的定性 …… 20
2.3.3 椭球 …… 21
2.4 矩阵的导数 …… 22
2.4.1 向量关于数量的偏导数 …… 22
2.4.2 数量关于向量的偏导数 …… 22
2.4.3 向量关于向量的偏导数 …… 22
2.4.4 矩阵关于数量的偏导数 …… 22
2.4.5 数量关于矩阵的偏导数 …… 23
2.4.6 有关内积、二次型的导数 …… 23
2.4.7 函数的偏导数 …… 23
2.5 本章矩阵简单运算的 R 和 Python 代码 …… 24
2.6 习 题 …… 27

第 3 章 回 归

3.1 经典回归模型基本要素 …… 29
3.1.1 描述数据 …… 30
3.1.2 线性回归模型 …… 31
3.1.3 最小二乘回归 …… 32
3.1.4 最小二乘回归参数的估计 …… 34
3.2 交叉验证 …… 36
3.3 经典线性回归的基本假定及根据假定所得到的结论 …… 38
3.3.1 经典线性回归的数学假定 …… 38
3.3.2 最小二乘回归参数估计量的一些性质及有关检验 …… 39
3.3.3 例 3.1 波士顿住房数据的最小二乘回归 …… 42
3.3.4 最小二乘回归结果的解释 …… 43
3.3.5 有关拟合的 R^2 及 AIC …… 46
3.4 自变量有分类变量 (定性变量) 的情况 * …… 48
3.5 机器学习回归简介及案例 …… 52
3.5.1 例子和机器学习方法的优势 … 52
3.5.2 决策树回归 …… 54
3.5.3 bagging 回归 …… 57

3.5.4 随机森林回归 ····· 58
3.6 经典线性回归与各种机器学习回归的比较 ····· 62
3.6.1 对例 3.1 波士顿住房数据的九种方法的比较 ····· 62
3.6.2 对例 3.1 波士顿住房数据使用 caret 包比较各种方法 * ····· 64
3.7 多元多重回归回顾 * ····· 69
3.7.1 模型 ····· 69
3.7.2 最小二乘估计及对估计量的检验 70
3.7.3 多元回归检验例子 ····· 71
3.8 本章计算的 Python 代码 ····· 76
3.8.1 例 3.1 波士顿住房数据的线性回归 ····· 76
3.8.2 例 3.1 波士顿住房数据的决策树回归、bagging 回归和随机森林回归 ····· 79
3.8.3 把分类变量转换成哑元变量 ····· 84
3.9 习 题 ····· 86

第 4 章 分 类

4.1 目的和基本概念 ····· 89
4.1.1 目的和例子 ····· 89
4.1.2 分类和聚类的区别 ····· 90
4.2 经典判别分析 ····· 90
4.2.1 线性判别分析和二次判别分析 ····· 90
4.2.2 Fisher 判别分析 ····· 92
4.3 例 4.1 数字笔迹识别的判别分析实践 ····· 93
4.3.1 例 4.1 数字笔迹识别的线性判别分析的计算 ····· 93
4.3.2 例 4.1 数字笔迹识别的二次判别分析的计算 ····· 94
4.4 判别分析的一些概念和某些数学细节的补充 * ····· 95
4.4.1 数据总体分布已知时的判别 ····· 95
4.4.2 线性判别分析的某些细节 ····· 97
4.4.3 Fisher 线性判别分析的某些细节 ····· 98
4.5 二分类因变量的 Logistic 回归 ····· 99
4.5.1 Logistic 回归模型 ····· 99
4.5.2 Logistic 回归模型对实际数据的拟合 ····· 101
4.5.3 Logistic 回归对例 4.2 献血数据的分类 ····· 102
4.5.4 ROC 等描述性曲线 ····· 103
4.5.5 三种分类方法对例 4.2 献血数据的交叉验证比较 ····· 106
4.6 机器学习模型简介及案例 ····· 106
4.6.1 决策树分类模型 ····· 107
4.6.2 随机森林分类模型 ····· 109
4.6.3 adaboost 分类模型 ····· 114
4.6.4 adaboost 拟合例 4.1 数字笔迹识别全部数据 ····· 115
4.6.5 adaboost 的变量重要性 ····· 115
4.6.6 adaboost 的交叉验证 ····· 115
4.6.7 各种分类模型的比较 ····· 116
4.7 本章计算的 Python 代码 ····· 118
4.7.1 产生 Z 折交叉验证集的代码 118
4.7.2 线性判别分析及交叉验证 ····· 120
4.7.3 二次判别分析 ····· 121
4.7.4 Logistic 回归 ····· 122
4.7.5 对例 4.2 献血数据做 3 个经典分类方法的交叉验证 ····· 124
4.7.6 随机森林分类 ····· 125
4.7.7 对例 4.1 数字笔迹识别数据做 5 个机器学习方法分类的交叉验证比较 ····· 125
4.8 习 题 ····· 128

第 5 章 主成分分析

5.1 基本思想和例子 ····· 129
5.1.1 数据中变量之间的关系与降维的可能性 ····· 129
5.1.2 从例子中产生的问题 ····· 130
5.2 问题和计算 ····· 133
5.2.1 求协方差矩阵及样本相关阵的特征值和特征向量 ····· 134
5.2.2 求各个成分的贡献率和累积贡献率 ····· 135
5.2.3 根据贡献率选取少数重要成分 138
5.2.4 主成分和原始变量的相关系数计算与载荷图 ····· 138
5.2.5 主成分得分 ····· 139
5.3 主成分分析在图像处理中的应用 142
5.3.1 图像压缩案例 (例 5.2 江景图片) ····· 142
5.3.2 主成分分析运用于图像识别 ····· 145
5.4 案例: 例 5.3 数据的主成分分析 145
5.5 主成分分析的一些数学知识 * ····· 148
5.5.1 目标 ····· 148
5.5.2 过程 ····· 148
5.5.3 累积贡献率 ····· 150
5.5.4 总体主成分的一些性质 ····· 151
5.5.5 样本主成分的一些性质 ····· 151
5.5.6 主成分的解释 ····· 152
5.5.7 标准化主成分分析 ····· 152

5.6 本章计算的 Python 代码 153
5.6.1 例 5.1 教师数据 153
5.6.2 图像压缩例子 158
5.7 习 题 159

第 6 章 因子分析

6.1 基本内容 161
6.1.1 概述 161
6.1.2 模型 163
6.1.3 性质 164
6.1.4 旋转 165
6.1.5 因子得分 165
6.1.6 计算及分析步骤 166
6.2 例子和计算 166
6.2.1 例 6.1WHO 数据分析 166
6.2.2 例 6.2 豌豆数据的因子分析 172
6.2.3 例 5.1 教师数据的因子分析 174
6.3 因子分析计算基于的原理 * 176
6.3.1 主成分方法 176
6.3.2 主因子方法 177
6.3.3 最大似然法 177
6.3.4 因子个数的选择 178
6.3.5 旋转 178
6.3.6 因子得分 179
6.3.7 建议和评论 180
6.4 本章计算的 Python 代码 180
6.5 习 题 182

第 7 章 聚类分析

7.1 目的及基本概念 184
7.1.1 目的 184
7.1.2 聚类和分类方法没有必然联系 184
7.1.3 距离 186
7.1.4 两个数据例子 191
7.1.5 集群倾向的度量 193
7.2 分层聚类 195
7.2.1 对观测值分类: 例 7.2 美国各州数据 195
7.2.2 对观测值聚类和对变量聚类: 例 7.3 花卉数据 198
7.3 K 均值聚类和基于密度的聚类 200
7.3.1 K 均值聚类的基本思想 200
7.3.2 K 均值聚类中类别数目的确定 201
7.3.3 基于密度聚类的思想 * 203
7.4 基于模型的聚类 * 205
7.4.1 直观描述 205
7.4.2 E-M 算法 206
7.4.3 基于模型聚类的计算实现 208
7.5 聚类数目的选择 211
7.5.1 Gap 方法 * 211
7.5.2 轮廓法 * 212
7.5.3 计算聚类数目的不同软件 215
7.6 实例的计算 220
7.6.1 种子数据 220
7.6.2 胎心宫缩监护数据 225
7.6.3 例 5.1 教师数据的聚类 228
7.7 图像色彩的聚类 230
7.7.1 对图像色彩聚类的例子 230
7.7.2 颜色的十六进制表示形式 * 233
7.8 本章计算的 Python 代码 234
7.8.1 例 7.2 美国各州数据的聚类 235
7.8.2 例 7.6 种子数据的聚类 238
7.8.3 图像色彩的聚类 241
7.9 习 题 243

第 8 章 典型相关分析

8.1 基本内容 245
8.1.1 问题的描述和例子 245
8.1.2 典型相关及特征值问题 247
8.1.3 载荷 252
8.1.4 典型变量的选择 253
8.2 实例计算: 例 5.1 教师数据 257
8.3 典型相关分析的不同角度推导 * 262
8.3.1 完全利用特征值问题的性质 262
8.3.2 利用 Cauchy-Schwarz 不等式 263
8.3.3 利用奇异值分解 264
8.3.4 利用广义特征值问题及 Cholesky 分解 265
8.3.5 各种公式的总结 265
8.4 本章计算的 Python 代码 266
8.5 习 题 267

第 9 章 对应分析

9.1 基本内容和应用 268
9.2 二元对应分析 268
9.2.1 二元对应分析的数学原理 271
9.2.2 对应分析行列相关的显著性 * 272
9.2.3 两个解释图形 273

9.3 多元及联合对应分析 ····· 274
9.3.1 例 9.1 收入数据多元及联合对应分析实践 ····· 274
9.3.2 多元对应分析方法的数学 * ····· 277
9.3.3 基于指标矩阵和 Burt 矩阵的方法 ····· 277
9.3.4 从 Burt 矩阵到联合对应分析 280
9.4 本章计算的 Python 代码 ····· 280
9.4.1 二元对应分析 ····· 280
9.4.2 多元对应分析 ····· 282
9.5 习 题 ····· 282
第 10 章 多维尺度变换
10.1 目 的 ····· 284
10.2 多维尺度变换分类 ····· 285
10.3 距离函数 ····· 285
10.4 mMDS 的目的 ····· 286
10.5 mMDS 方法的原理 ····· 287
10.6 nMDS 的基本思维 ····· 288
10.7 本章计算的 Python 代码 ····· 290
10.8 习 题 ····· 292
附录 1 R 简介——通过运行来领悟 293
附录 2 Python 简介——通过运行来领悟 305
参考文献 319

第 1 章 引 言

1.1 数据科学

1.1.1 统计是数据科学吗?

按照《不列颠百科全书》所述, **统计是收集、分析、展示及解释数据的科学** (Statistics, the science of collecting, analyzing, presenting, and interpreting data).[1] 也就是说, 统计是**数据科学** (the science of data). 早在 20 年前, 华裔统计学家吴建福 (C.F. Jeff Wu) 就多次说过 "统计应该称为数据科学".

但是, 无论口头如何说, 实际上统计界许多人是在搞数学, 并冠之以 "理论统计" 或 "数理统计" 的名字. 这是有历史原因的. 计算机出现之前, 人们只能处理少量数据, 为了计算方便和弥补信息量的不足, 以数学家为主体的学者创造了很多基于主观假定的数学模型. 统计因此被数学所主导, 统计方面的研究文章大部分是数学研究. 《中国大百科全书》明确定义, 数理统计是 "数学的一个分支学科".[2]

事实上, 在前计算机时代, 数据分析一直依赖于这些数学家所创造的数学模型. 因此, 模型驱动而非问题驱动一直是统计界的主流思维方式, 模型驱动思维就是数学式的演绎思维. 一直到 20 世纪 90 年代, 计算机还仅仅是各个领域数据分析者的工具, 是手和笔的延伸. 如果在那个时候, 统计学家认为自己是数据科学家, 而把计算机视为自己不可缺少的工具包括在数据科学之内, 统计学的进程就完全不一样了. 可是, 以数学模型为主导的统计界没有这么做. 当今, 统计界无法和真正的数学界比数学之美, 也无法和计算机群体比处理数据问题的效率和能力, 统计失去了很多热门领域和优秀人才.

机器学习或者基于机器学习方法的人工智能目前发展迅速, 但有多少人会把这些发展和本应是 "数据科学" 的传统数理统计相联系呢? 尽管现在一部分 "大数据中心"(或 "研究院""学院" 等) 的领导者都是专注于数学模型的学者, 但那些计算机领域的人会臣服吗?

从深度、难度以及美学的角度来看, 统计所用的数学和纯粹数学根本无法相比, 与数学家在一起可能会令某些统计学人感到不舒服. 其实, 如果这些统计学人展示的不是数学而是分析数据的能力, 就不会有这种感觉了, 问题是这些统计学人究竟有没有数据处理的能力. 类似地, 部分统计学人和搞计算机的同事在一起时也会有不快的感觉. 如果他们不会编程计算却又专注于一些没有实践背景的数学模型, 自然会感到不爽.

对于整个社会来说, 传统统计的边缘化并不见得是个坏事情. 对于统计学科本身, 危机感能够促进变化, 把统计界从封闭的小圈子里解放出来.

---

[1] 不列颠百科全书. https://www.britannica.com/science/statistics.

[2] 中国大百科全书总编辑委员会. 中国大百科全书: 数学卷. 北京: 中国大百科全书出版社, 1988: 593.

1.1.2 计算机学科在数据科学中的地位

由于统计界的主流实际上不愿意采取问题驱动或数据驱动的思维，而计算机学科借助数据源、数据清理、数据处理及算法改进等方面的优势，已经成为数据科学的主要实体，而且占有越来越重要的地位.

实际上，主要的机器学习方法的创造者大都不是沉溺于数学模型的“统计学家”，也不是计算机界的学者. 他们多是有数学背景或其他领域背景的科学家. 他们依赖的并不是某些专业课程知识，而是自己对实际问题的理解、对随机性的理解、抽象思维能力和想象力，而这些思维能力很多是从严格的纯粹数学训练中获得的.

在当今世界，人们离不开计算机，计算机是数据科学最重要的基石之一，也是工作量最大的部分，没有计算机，所有的机器学习、人工智能、大数据分析就都没有意义. 但是必须清醒地认识到，**单纯的“软件 + 硬件”能力不会自动产生新的算法，工程师式的工作代替不了科学家的创造**.

1.1.3 问题驱动应成为数据科学的基本思维方式

笔者觉得目前的统计学科、数学或计算机学科没有一个能够单独代表数据科学. 数据科学本身包括了所有可能的知识积累和人们的才智. **知识不是由学科划分的**，当前科技进步的进程不断突破人们有限的想象力，试图当老大或者试图把学科标签固化的思维曾经阻碍了一些学科的发展，一些故步自封的学科将来也会消亡，但社会总是在不断前进.

很多人认为，学完了统计、数学、计算机及各领域的一些课程就可以解决数据科学的问题. 实际上，课程仅仅是历史的一些记录，人们可以从中得到一些经验、启发及知识，而不可能得到能力. 有人觉得，根据分析某些“教科书式的数据”的经验，按照现成模型可能会找到所谓“标准答案”，但是，**在实际问题中不可能存在“标准答案”**，针对一个问题可能有很多不同的答案，而每种答案都可能具有某种合理性. 试图寻求一个“标准答案”本身就限制了人们的想象力、能力及对实际问题复杂性的判断. 更为大量的实际问题是无法限定于一种或若干种方法 (或组合) 的，认为只要选择合适的模型和参数就应该得到“正确”或“精准”答案的想法是把世界简单化，真实世界比围棋机器人所面对的围棋世界要复杂得多.

因此，面对复杂的真实问题，我们对问题本身的意义及有关数据的理解、知识积累、想象力和处理复杂问题的能力就是关键. 能力是从不断解决实际问题中获得的，是从无数失败中悟出来的，而不是从书本上学来的.

1.2 多元分析的对象

本书介绍的方法主要适用于规范的横截面数据，观测值及变量可以按照行列排成方阵. 这里讨论的变量都是相关的，而且每个观测值都是同时度量的. 根据机器学习术语，回归和分类属于**有指导 (或有监督) 学习** (supervised learning)，也就是有因变量作为目标变量，主要目的是建立可以预测的模型，有指导学习的回归和分类是机器学习的核心，也是人工智能的基本组成部分；其他方法都属于**无指导 (或无监督) 学习** (unsupervised learning)，其中没有目标变量，目的是揭示相关变量所提供的潜在重叠信息所代表数据的背景结构，因此，简化、降维、汇总是其主要特点. 这部分多元统计分析基本上是探索性的数据分析，它可能提出一些假设，但检验往往并不是其主要目的.

从名称上看, 只要多于一个变量的数据分析都应该属于多元分析. 这实际上包括了除收集数据之外的几乎整个数据科学的内容. 但是, 传统的多元分析要狭窄得多, 除了往往放在回归课程中讲授的回归分析之外, 多元分析传统上是由无指导学习中的一组方法所构成的. 这包括主成分分析、因子分析、聚类分析、典型相关分析和判别分析等, 这些方法传统上多假定多元正态分布, 但由于大多是描述性的, 在方法及结果的解释方面并不完全依赖分布假定. 再拓宽一些, 还可能包括本书没有包括在内的多元方差分析及结构方程模型的协方差方法等, 它们严重依赖于对数据的各种假定.

除了可以应用于所有领域并成为人工智能基础的有指导学习之外, 属于无指导学习的经典多元分析技术的很大一部分最早应用于行为科学和生物科学, 而现在已经扩展到各个领域, 包括物理、化学、生物、医学、心理学、社会学、经济学、教育学、法律、商学、文学、传媒学、护理学、语言学、地质学等等.

读者可以通过本书理解各种多元分析的方法、目的、适用范围、局限性及数据分析的具体计算机实现, 以及各种方法的关联及区别. 在动手操作方面, 期望读者能够根据具体数据选择合适的方法并且解释多元分析方法的计算机输出. 希望读者能够通过编程来熟悉本书的概念, 同时通过分析数据来熟悉编程.

1.3 需要的工具

学习本书, 需要掌握一定的数学知识, 主要是线性代数或矩阵理论等数学, 这些知识在本书的第 2 章专门进行了回顾, 可以自给自足; 微积分用得不多 (只有个别地方应用了导数的概念). 读者有一些初等统计知识就够了. 本书强调对模型和方法的理解, 而不是数学证明和推理的细节, 大多数被展示的结果未给出证明, 因为直观的解释更加有说服力. 本书所使用的 R 或者 Python 等编程语言完全可以在使用中学习. 学习软件最快的方法就是使用, 有问题可从网络上寻求答案, 不必专门在课堂上学习.

1.4 各章的安排

本书从第 2 章开始的各章内容安排如下: 第 2 章 "矩阵代数回顾", 提供本书所需要的一些代数知识, 供需要时查阅, 这些内容一般不用在课堂上专门介绍 (由于后续各章多少涉及该章内容, 所以没有打星号). 第 3、4 章的内容为有指导学习: 第 3 章 "回归", 回顾经典回归分析的内容, 同时简单介绍少数机器学习的回归方法, 这一章可以自学, 也可以在课堂上做介绍, 该章的多元多重回归一节仅供参考之用; 第 4 章 "分类", 介绍经典判别分析, 同时介绍少数机器学习分类方法. 从第 5 章开始介绍无指导学习的概念、方法、目的及案例分析, 包括第 5 章 "主成分分析"、第 6 章 "因子分析"、第 7 章 "聚类分析"、第 8 章 "典型相关分析"、第 9 章 "对应分析"、第 10 章 "多维尺度变换". 本书的最后设置了关于 R 和 Python 语言的附录.

此外, 本书在正文中使用 R 语言来实现数据分析的计算并解释其输出的结果, 从第 2 章开始的每一章都提供了例题计算的 Python 代码, 以满足学习或使用 Python 语言读者的需要.

除了第 1 章, 每章后面都提供了习题, 帮助读者掌握各章的内容或计算.

1.5 软件和编程

在现代社会，一个人是否会编程密切关系到其生存状况，笔者认为，编程不限于一个软件的语言，而是要培养明白编程要点的泛型编程能力.

编程必须在应用中学，上专门的软件理论课学习编程远远不如针对具体目的来完成一个数据分析任务效果好. 在编程中要不怕犯错误，不断纠正错误是熟悉软件的最快途径.

R 软件的程序大多是统计学家编写的，对搞统计分析的人很友好. R 网站上不断更新并持续增加的 1 万多个程序包[3]展示了各种最新的数据分析方法. 绝大多数新方法的计算机实现均首先出现在 R 网站上. 这些程序包的程序都是透明的，可供审视和改写，这是任何商业软件无法比拟的，透明是对抗错误的最好方式. R 软件的优点是容易编写，帮助系统很优秀，而且对各种统计方法有比较细致的描述，输出结果丰富；缺点是软件本身处理较大数据的速度较慢，但目前许多快速编程软件都有针对 R 的接口，而且还有一些针对 R 包装的软件，使得速度慢不会成为其发展及受欢迎的障碍.

不像 R 软件主要用于数据分析，Python 是通用软件，可以完成任何计算机能够做的任务. 由于 Python 比 C++ 更易编写，有限的速度差距又已经被一些包装软件所克服，Python 很快成为 C++ 最强有力的替代. Python 的编程非常灵活，虽然其帮助系统不如 R 软件，但由于它是开源软件，网上帮助很好用. Python 不是专门的统计软件，其统计方法资源远不如 R 软件多，而且对于一些模型，它的输出结果也不如 R 软件那么丰富.

人们通常想知道应该学习什么编程软件，笔者认为，学习任何编程软件都有益处，如果学会第一个编程语言花了一个星期，学会第二个可能只需三四天的时间. 这里所说的“学会”是指能够完成你手中的任务，而不是指精通. 软件是工具，我们能够使用它完成任务就行了，没有必要完全精通. 学习编程的关键是遇到问题时知道去哪里寻求帮助，网络上的信息很全面，显然任何教师都无法独自掌握如此海量信息，在网上寻求帮助是编程者的惯例. 软件在不断发展，我们必须跟上时代的潮流.

鉴于 R 和 Python 的区别，以及各种模型的选项和程序包或模块的编写过程及意图的不同，本书中使用两种软件处理同一个数据的输出结果可能不同. 我们对数据和程序包或模块采用了一些比较随意的选择方式，希望这不会对可能有“洁癖”或“强迫症”心理的人造成不适.

1.6 如何教学

人都是在不断学习中成长的，而且绝大多数能力是自学的. 回想一下，你有哪一能力是课堂上当场学会的 (不用做作业)? 教师应该引导学生产生兴趣及疑问、展开独立思考并理解世界，让学生关注世上的最新事物，还是应该告诉学生世上有什么? 学生难道不会自己查阅资料和自学吗? 离开学校就是知识获取过程的终结吗? 教学是否等于播放陈旧的录音? 不看讲义而能不错一个符号地把书上的内容抄写在黑板上是好老师的标准吗? 教师自己有必要不断学习吗? 学生挑战教师好不好? 没有学生挑战说明什么? 教师应如何面对学生 (在许多方面) 比自己强这样的现实? 不带疑问和怀疑去上课的学生会有收获吗? 课后学生没有疑问的课堂教学正常吗?

[3]在 2009 年底，R 网站上的程序包不到 1000 个.

思考了上面一些问题之后，关于本书的教学方式也就十分清楚了．最主要的是弄明白每个模型的直观含义及拟合数据的过程和原理．至于有关的数学公式，只需要介绍其目的及意义，而不必做过度的数学推导，这些数学公式大都是有百年历史的既定结论．实际上，解释这些模型的原理并不比推导数学公式更容易．本书中标有 * 号的部分可作为参考材料，教师可以根据需要来确定讲哪些内容．但是，部分没有标 * 号的内容也可以用于自学或作为参考材料．**教科书仅仅是教学的参考和工具，教科书的使用应该服从于获取知识和能力的需要，教科书不应是限制师生思维的教条和框框**．

我相信，任何读者只要愿意努力，就有可能成为一个名副其实的数据科学家．

第 2 章　矩阵代数回顾

本章的内容是非常经典的数学知识, 在几乎所有的高等代数书中都可以找到. 在本章安排的这些内容, 主要是方便读者查阅, 它们只是工具, 不是必须要在课堂上讲授. 由于后续各章多少会涉及本章内容, 所以本章内容没有标星号 (*).

2.1　矩　阵

2.1.1 基本定义

矩阵是数值或数值变量的一个矩形数字阵, m 行 n 列 (亦称 $m \times n$ 维或 $m \times n$ 阶) 的矩阵 $\boldsymbol{A}$ 的形式为:

$$\boldsymbol{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

$\boldsymbol{A}$ 的表示方法或记号有很多, 比如显示维数的 $\boldsymbol{A}_{(m\times n)} = \{a_{ij}\}$ 或者显示元素的 $\boldsymbol{A} = \{a_{ij}\}$. 此外, 根据不同的需要及方便起见, 元素下标的次序并没有统一规定, 比如, 元素 a_{ij} 的两个下标不一定必须是 i 代表行及 j 代表列, 也可能相反.

前述矩阵 $\boldsymbol{A}$ 的**转置**记为 $\boldsymbol{A}^\top$(这是本书采用的记号) 或 $\boldsymbol{A}'$, 定义为:

$$\boldsymbol{A}^\top = \begin{bmatrix} a_{11} & a_{21} & \cdots & a_{m1} \\ a_{12} & a_{22} & \cdots & a_{m2} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1n} & a_{2n} & \cdots & a_{mn} \end{bmatrix}.$$

一个 $m \times 1$ 的矩阵称为**列向量**, 而一个 $1 \times n$ 的矩阵称为**行向量**. 比如 $1 \times m$ 矩阵 $\boldsymbol{a} = [a_1, a_2, \ldots, a_m]$ 是一个行向量, 也记为 $\boldsymbol{a} = (a_1, a_2, \ldots, a_m)$, 其元素为 $a_1, a_2, \ldots, a_m$, 而其转置为列向量, 即

$$\boldsymbol{a}^\top = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_m \end{bmatrix}.$$

也可以用列向量或行向量表示矩阵, 比如, 前述矩阵 $\boldsymbol{A}$ 的 n 个列向量记为 $\boldsymbol{a}_j = (a_{1j}, a_{2j}, \ldots, a_{mj})^\top$ $(j = 1, 2, \ldots, n)$, 那么矩阵 $\boldsymbol{A}$ 可表示为 $\boldsymbol{A} = [\boldsymbol{a}_1, \boldsymbol{a}_2, \ldots, \boldsymbol{a}_n]$; 如果

前述矩阵 $\boldsymbol{A}$ 的 m 个行向量记为 $\boldsymbol{a}_i = (a_{i1}, a_{i2}, \ldots, a_{in})^\top$ $(i = 1, 2, \ldots, m)$, 那么, 矩阵 $\boldsymbol{A}$ 可表示为 $\boldsymbol{A}^\top = [\boldsymbol{a}_1, \boldsymbol{a}_2, \ldots, \boldsymbol{a}_m]$.

对于一个方阵 $\boldsymbol{A} = \{a_{ij}\}$, 如果 $a_{ij} = a_{ji}$, $\forall i, j$ [1], 则称该矩阵为**对称矩阵**.

如果一个方阵 $\boldsymbol{U} = \{u_{ij}\}$ 的元素满足 $u_{ij} = 0$, $\forall j < i$, 则称该矩阵为**上三角矩阵**; 如果一个方阵 $\boldsymbol{L} = \{l_{ij}\}$ 的元素满足 $l_{ij} = 0$, $\forall j > i$, 则称该矩阵为**下三角矩阵**.

如果一个方阵 $\boldsymbol{D} = \{d_{ij}\}$ 的元素满足 $d_{ij} = 0$, $\forall j \neq i$, 则称该矩阵为**对角矩阵**, 这时可记 d_{ii} 为 d_i, 以及 $\boldsymbol{D} = \mathrm{diag}(d_1, d_2, \ldots, d_m)$. 对角线元素全部为 1 的对角矩阵 $\boldsymbol{I} = \mathrm{diag}(1, 1, \ldots, 1)$ 称为**单位矩阵**; 如果其维度为 n, 往往记为 $\boldsymbol{I}_n$ 或 $\boldsymbol{I}_{(n\times n)}$. 元素全部是 1 的向量记为 $\mathbf{1} = (1, 1, \ldots, 1)^\top$.

2.1.2 基本矩阵运算

两个同样维数的矩阵 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 之和定义为一个新矩阵 $\boldsymbol{C} = \boldsymbol{A} + \boldsymbol{B}$, 其元素等于 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 相应元素之和, 即 $\boldsymbol{C} = \{c_{ij}\} = \{a_{ij} + b_{ij}\}$. 同样定义两个矩阵之差. 一个数量 c 和一个矩阵 $\boldsymbol{A} = \{a_{ij}\}$ 相乘意味着逐项相乘, 即 $\boldsymbol{B} = c\boldsymbol{A} = \{ca_{ij}\}$. $-\boldsymbol{A}$ 定义为 $\boldsymbol{A}$ 的每个元素乘 -1. 显然有下面的性质:

$$
\begin{aligned}
&\boldsymbol{A} + \boldsymbol{B} = \boldsymbol{B} + \boldsymbol{A}, \quad \boldsymbol{A} + (\boldsymbol{B} + \boldsymbol{C}) = (\boldsymbol{A} + \boldsymbol{B}) + \boldsymbol{C},\\
&\boldsymbol{A} - \boldsymbol{B} = \boldsymbol{A} + (-\boldsymbol{B}) = -(\boldsymbol{B} - \boldsymbol{A}), \quad \boldsymbol{A} - \boldsymbol{A} = \mathbf{0},\\
&(\boldsymbol{A} + \boldsymbol{B})^\top = \boldsymbol{A}^\top + \boldsymbol{B}^\top, \quad (b + c)\boldsymbol{A} = b\boldsymbol{A} + c\boldsymbol{A},\\
&\boldsymbol{A} + \mathbf{0} = \boldsymbol{A}, \quad c\boldsymbol{A} = \boldsymbol{A}c, \quad -(-\boldsymbol{A}) = \boldsymbol{A},\\
&c(\boldsymbol{A} + \boldsymbol{B}) = c\boldsymbol{A} + c\boldsymbol{B}, \quad 0\boldsymbol{A} = \mathbf{0}, \quad 1\boldsymbol{A} = \boldsymbol{A}.
\end{aligned}
$$

如果一个矩阵的列数和另一矩阵的行数相同 (称为可相乘的), 比如 $\boldsymbol{A}_{(m\times n)}$ 和 $\boldsymbol{B}_{(n\times p)}$, 则它们的积定义为一个 $m \times p$ 矩阵 $\boldsymbol{C} = \boldsymbol{A}\boldsymbol{B} = \{c_{ij}\}$, 这里

$$
c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.
$$

两个相同维列向量 $\boldsymbol{a} = (a_1, a_2, \ldots, a_n)^\top$ 和 $\boldsymbol{b} = (b_1, b_2, \ldots, b_n)^\top$ 的内积 (又称点积或点乘)$\boldsymbol{a}^\top\boldsymbol{b}$, 也记为 $\boldsymbol{a} \cdot \boldsymbol{b}$ 或者 $\langle \boldsymbol{a}, \boldsymbol{b} \rangle$, 定义为数量

$$
\boldsymbol{a}^\top \boldsymbol{b} = \sum_{i=1}^{n} a_i b_i.
$$

[1] 符号 "$\forall$" 是 "对于所有"(for all) 的简写.

关于矩阵乘法有下列性质:

$$
\begin{aligned}
&(\boldsymbol{A}+\boldsymbol{B})\boldsymbol{C}=\boldsymbol{A}\boldsymbol{C}+\boldsymbol{B}\boldsymbol{C},\quad \boldsymbol{A}(\boldsymbol{B}+\boldsymbol{C})=\boldsymbol{A}\boldsymbol{B}+\boldsymbol{A}\boldsymbol{C},\\
&\boldsymbol{A}_{(m\times n)}\boldsymbol{I}_n=\boldsymbol{I}_m\boldsymbol{A}_{(m\times n)},\quad \boldsymbol{A}_{(m\times n)}\boldsymbol{0}_{(n\times p)}=\boldsymbol{0}_{(m\times p)},\\
&\boldsymbol{0}_{(q\times m)}\boldsymbol{A}_{(m\times n)}=\boldsymbol{0}_{(q\times n)},\quad \boldsymbol{A}\boldsymbol{B}^{\top}=(\boldsymbol{B}\boldsymbol{A}^{\top})^{\top}.
\end{aligned}
$$

分块矩阵 (见2.1.14) 的运算和矩阵运算类似, 只不过把块 (子矩阵) 当成矩阵元素, 比如, 对分块矩阵

$$
\boldsymbol{A}=\begin{bmatrix}\boldsymbol{M}_{11} & \boldsymbol{M}_{12}\\ \boldsymbol{M}_{21} & \boldsymbol{M}_{22}\end{bmatrix}=\left[\begin{array}{ccc|cc}a_{11} & a_{12} & a_{13} & b_{11} & b_{12}\\ a_{21} & a_{22} & a_{23} & b_{21} & b_{22}\\ \hline c_{11} & c_{12} & c_{13} & d_{11} & d_{12}\\ c_{21} & c_{22} & c_{23} & d_{21} & d_{22}\\ c_{31} & c_{32} & c_{33} & d_{31} & d_{32}\end{array}\right],
$$

有

$$
\boldsymbol{A}^2=\begin{bmatrix}\boldsymbol{M}_{11} & \boldsymbol{M}_{12}\\ \boldsymbol{M}_{21} & \boldsymbol{M}_{22}\end{bmatrix}\begin{bmatrix}\boldsymbol{M}_{11} & \boldsymbol{M}_{12}\\ \boldsymbol{M}_{21} & \boldsymbol{M}_{22}\end{bmatrix}=\begin{bmatrix}\boldsymbol{M}_{11}^2+\boldsymbol{M}_{12}\boldsymbol{M}_{21} & \boldsymbol{M}_{11}\boldsymbol{M}_{12}+\boldsymbol{M}_{12}\boldsymbol{M}_{22}\\ \boldsymbol{M}_{21}\boldsymbol{M}_{11}+\boldsymbol{M}_{22}\boldsymbol{M}_{21} & \boldsymbol{M}_{21}\boldsymbol{M}_{12}+\boldsymbol{M}_{22}^2\end{bmatrix}.
$$

2.1.3 行列式

一个 2×2 矩阵

$$
\boldsymbol{A}=\begin{bmatrix}a_{11} & a_{12}\\ a_{21} & a_{22}\end{bmatrix}
$$

的行列式定义为

$$
|\boldsymbol{A}|=\det(\boldsymbol{A})=\begin{vmatrix}a_{11} & a_{12}\\ a_{21} & a_{22}\end{vmatrix}=a_{11}a_{22}-a_{21}a_{12},
$$

而一个 3×3 矩阵

$$
\boldsymbol{A}=\begin{bmatrix}a_{11} & a_{12} & a_{13}\\ a_{21} & a_{22} & a_{23}\\ a_{31} & a_{32} & a_{33}\end{bmatrix}
$$

的行列式定义为:

$$
|\boldsymbol{A}|=\det(\boldsymbol{A})=a_{11}\begin{vmatrix}a_{22} & a_{23}\\ a_{32} & a_{33}\end{vmatrix}-a_{12}\begin{vmatrix}a_{21} & a_{23}\\ a_{31} & a_{33}\end{vmatrix}+a_{13}\begin{vmatrix}a_{21} & a_{22}\\ a_{31} & a_{32}\end{vmatrix}.
$$

一般的 $n \times n$ 矩阵 $\boldsymbol{A} = \{a_{ij}\}$ 的行列式定义为:

$$|\boldsymbol{A}| = \det(\boldsymbol{A}) = a_{k1}\alpha_{k1} + a_{k2}\alpha_{k2} + \cdots + a_{kn}\alpha_{kn} = \sum_{j=1}^{n} a_{kj}\alpha_{kj}, \ \forall\, k$$

$$= \sum_{i=1}^{n} a_{ik}\alpha_{ik}, \ \forall\, k,$$

这里

$$\alpha_{ij} = (-1)^{i+j}\beta_{ij},$$

而 β_{ij} 为 $\boldsymbol{A}$ 中没有第 i 行及第 j 列元素的 $(n-1) \times (n-1)$ 维子矩阵的行列式. 这里的 α_{ij} 称为 a_{ij} 的**余子式**.

对于 n 阶方阵 $\boldsymbol{A}$ 和 $\boldsymbol{B}$, 以及 $c \in \mathbb{R}$, 有下列性质:

$$|c\boldsymbol{A}| = c^n|\boldsymbol{A}|, \ \ |\boldsymbol{AB}| = |\boldsymbol{BA}| = |\boldsymbol{A}||\boldsymbol{B}|.$$

2.1.4 矩阵的逆

一个 n 阶方阵 $\boldsymbol{A}$ 的**逆**也是 n 阶方阵, 记为 $\boldsymbol{A}^{-1}$, 满足 $\boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{I}_n$. 当然, 矩阵的逆不一定存在, 如果 $\boldsymbol{A}$ 的逆存在, 则称 $\boldsymbol{A}$ 为**非奇异的**, 否则称它为**奇异的**. 方阵 $\boldsymbol{A}$ 存在逆的充分必要条件是其行列式 $|\boldsymbol{A}| \neq 0$.

关于矩阵的逆, 有下列性质:

$$\begin{aligned}
&\boldsymbol{I}^{-1} = \boldsymbol{I}, \ \ (\boldsymbol{A}^{-1})^{-1} = \boldsymbol{A}, \ (\boldsymbol{A}^{\top})^{-1} = (\boldsymbol{A}^{-1})^{\top} \\
&(\boldsymbol{AB})^{-1} = \boldsymbol{B}^{-1}\boldsymbol{A}^{-1}, \ \ (c\boldsymbol{A})^{-1} = c^{-1}\boldsymbol{A}^{-1} \\
&\mathrm{diag}^{-1}(d_1, d_2, \ldots, d_n) = \mathrm{diag}(1/d_1, 1/d_2, \ldots, 1/d_n),
\end{aligned}$$

这里, $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 为非奇异方阵, c 为数量.

2.1.5 矩阵的广义逆

如果 $\boldsymbol{A}$ 为 $n \times m$ 矩阵, $\boldsymbol{y} \in \mathbb{R}^n$ 及 $\boldsymbol{x} \in \mathbb{R}^m$, 考虑方程组

$$\boldsymbol{A}\boldsymbol{x} = \boldsymbol{y}, \tag{2.1.1}$$

则在 $m = n$ 时, 如果 $\boldsymbol{A}$ 的逆存在, 则方程组 (2.1.1) 有解 $\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{y}$; 如果 $\boldsymbol{A}$ 不可逆或者 $m \neq n$, 则无解或者解不唯一. 这时可以定义 $\boldsymbol{A}$ 的广义逆.

矩阵 $\boldsymbol{A}$ 的**广义逆** $\boldsymbol{A}^{-}$ 满足下列条件:

$$\boldsymbol{A}\boldsymbol{A}^{-}\boldsymbol{A} = \boldsymbol{A}. \tag{2.1.2}$$

容易验证, 如果方阵 $\boldsymbol{A}$ 的逆 $\boldsymbol{A}^{-1}$ 存在, 则 $\boldsymbol{A}^{-1} = \boldsymbol{A}^{-}$. 这种情况之外的广义逆并不唯一.

可以证明, 方程组 (2.1.1) 有解 $\boldsymbol{x} \in \mathbb{R}^m$ 的充分必要条件是 $\boldsymbol{A}\boldsymbol{A}^-\boldsymbol{y} = \boldsymbol{y}$. 此外, 还可以证明, 如果方程组 (2.1.1) 有解, 而 $\boldsymbol{x} \in \mathbb{R}^m$ 是一个解的充分必要条件是对某 $\boldsymbol{z} \in \mathbb{R}^m$, 有

$$\boldsymbol{x} = \boldsymbol{A}^-\boldsymbol{y} + (\boldsymbol{I} - \boldsymbol{A}^-\boldsymbol{A})\boldsymbol{z}.$$

2.1.6 Kronecker 积

如果 $\boldsymbol{A}$ 为 $m \times n$ 矩阵, 而 $\boldsymbol{B}$ 为 $p \times q$ 矩阵, 则它们的 Kronecker 积记为 $\boldsymbol{A} \otimes \boldsymbol{B}$, 定义为一个 $mp \times nq$ 矩阵:

$$\boldsymbol{A} \otimes \boldsymbol{B} = \begin{bmatrix} a_{11}\boldsymbol{B} & a_{12}\boldsymbol{B} & \cdots & a_{1n}\boldsymbol{B} \\ a_{21}\boldsymbol{B} & a_{22}\boldsymbol{B} & \cdots & a_{2n}\boldsymbol{B} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}\boldsymbol{B} & a_{m2}\boldsymbol{B} & \cdots & a_{mn}\boldsymbol{B} \end{bmatrix}.$$

关于矩阵的 Kronecker 积有下列性质 (这些公式中的矩阵乘积相关的矩阵的维数必须匹配, 而求逆的矩阵必须非奇异):

$$\begin{aligned}
&(\boldsymbol{A} + \boldsymbol{B}) \otimes \boldsymbol{C} = \boldsymbol{A} \otimes \boldsymbol{C} + \boldsymbol{B} \otimes \boldsymbol{C}, \\
&\boldsymbol{A} \otimes (\boldsymbol{B} + \boldsymbol{C}) = \boldsymbol{A} \otimes \boldsymbol{B} + \boldsymbol{A} \otimes \boldsymbol{C}, \\
&(\boldsymbol{A} \otimes \boldsymbol{B}) \otimes \boldsymbol{C} = \boldsymbol{A} \otimes (\boldsymbol{B} \otimes \boldsymbol{C}), \\
&c(\boldsymbol{A} \otimes \boldsymbol{B}) = (c\boldsymbol{A}) \otimes \boldsymbol{B} = \boldsymbol{A} \otimes (c\boldsymbol{B}), \\
&(\boldsymbol{A} \otimes \boldsymbol{B}) \otimes (\boldsymbol{C} \otimes \boldsymbol{D}) = \boldsymbol{A}\boldsymbol{C} \otimes \boldsymbol{B}\boldsymbol{D}, \\
&(\boldsymbol{A} \otimes \boldsymbol{B})^{-1} = \boldsymbol{A}^{-1} \otimes \boldsymbol{B}^{-1}, \\
&(\boldsymbol{A} \otimes \boldsymbol{B})^{\top} = \boldsymbol{A}^{\top} \otimes \boldsymbol{B}^{\top}.
\end{aligned}$$

如果 $\boldsymbol{A}$ 为 $m \times m$ 矩阵, 而 $\boldsymbol{B}$ 为 $n \times n$ 矩阵, 则

$$\det(\boldsymbol{A} \otimes \boldsymbol{B}) = \det(\boldsymbol{B} \otimes \boldsymbol{A}) = \det(\boldsymbol{A})^n \det(\boldsymbol{B})^m.$$

2.1.7 幂等矩阵

如果方阵 $\boldsymbol{M} = \boldsymbol{M}\boldsymbol{M}$, 则称 $\boldsymbol{M}$ 为**幂等矩阵**. 如果 $\boldsymbol{M}$ 非奇异, 有下列性质:

$$\boldsymbol{M} = \boldsymbol{I}\boldsymbol{M} = \boldsymbol{M}^{-1}\boldsymbol{M}\boldsymbol{M} = \boldsymbol{M}^{-1}\boldsymbol{M} = \boldsymbol{I}.$$

方阵 $\boldsymbol{M}$ 是幂等矩阵的充分必要条件为: 对于任意整数 n, 有 $\boldsymbol{M}^n = \boldsymbol{M}$.

幂等矩阵的一个例子是线性模型 (见3.1.2)

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

的最小二乘回归预测矩阵 (亦称帽子矩阵, 有时用 $\boldsymbol{H}$ 表示) $\boldsymbol{P}=\boldsymbol{X}(\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top$, 容易验证, 预测矩阵 $\boldsymbol{P}$ 为幂等矩阵.

2.1.8 向量空间

一个 n 维**向量空间**为所有向量 $\boldsymbol{x}=(x_1,x_2,\ldots,x_n)$ 的无穷集合. n 维向量空间的一个**子空间**为一组向量 $\{\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_k\}$ 所生成的所有线性组合的集合:

$$\boldsymbol{A}=a_1\boldsymbol{x}_1+a_2\boldsymbol{x}_2+\cdots+a_k\boldsymbol{x}_k,\ \forall(a_1,a_2,\ldots,a_k),$$

这个子空间称为由该组向量 $\{\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_k\}$ **张成的**.

如果一组向量 $\{\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_k\}$ 中的任何一个都不能表示成其他向量的线性组合, 则称该组向量**线性独立**; 等价地, 线性独立向量组不可能有 $\sum_{i=1}^k b_i\boldsymbol{x}_i=\boldsymbol{0}$, 除非所有 b_i 全部等于 0.

一组线性独立的向量可以形成其所张成的子空间的基, 如果该子空间的一个向量 $\boldsymbol{y}$ 可以表示成 $\boldsymbol{y}=\sum_{i=1}^k c_i\boldsymbol{x}_i$, 则 $\{c_i\}$ 为 $\boldsymbol{y}$ 关于这组基的坐标.

2.1.9 正交性

如果两个向量的内积为 0, 则称它们**正交**. 一个满足 $\boldsymbol{X}^\top\boldsymbol{X}=\boldsymbol{I}$ 的矩阵 $\boldsymbol{X}$ 称为**正交矩阵**.

记 $\boldsymbol{X}_{(n\times k)}\equiv[\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_k]$. 一个向量 $\boldsymbol{y}$ 到由 $\{\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_k\}$ 张成的子空间的**正交投影**定义为向量

$$\hat{\boldsymbol{y}}=\boldsymbol{X}\boldsymbol{b}=\sum_{i=1}^k b_i\boldsymbol{x}_i,$$

这里的 $\boldsymbol{b}=(b_1,b_2,\ldots,b_k)^\top$ 满足条件 $\boldsymbol{X}^\top(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{b})=\boldsymbol{0}$. 可以解出: $\boldsymbol{b}=(\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{y}$. 注意, 只要 $\{\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_k\}$ 线性独立, $\boldsymbol{X}^\top\boldsymbol{X}$ 就是非奇异的, 因此其逆有意义.

一个**正交变换**是在实内积空间 $\mathbb{V}$ 上保持内积不变的一个线性变换: $T:\mathbb{V}\mapsto\mathbb{V}$. 也就是说, 对于每对 $\boldsymbol{u},\boldsymbol{v}\in\mathbb{V}$, 有

$$\boldsymbol{u}^\top\boldsymbol{v}=(\boldsymbol{T}\boldsymbol{u})^\top(\boldsymbol{T}\boldsymbol{v}).$$

2.1.10 矩阵的秩

一个矩阵 $\boldsymbol{A}$ 的**秩**定义为其线性独立列向量的个数 (称为**列秩**), 它总是等于其线性独立行向量的个数 (称为**行秩**). 矩阵 $\boldsymbol{A}$ 的秩记为 $\mathrm{rank}(\boldsymbol{A})$.

对于 $\boldsymbol{A}_{(n\times p)}$ 和 $\boldsymbol{B}_{(p\times n)}$, 有下列性质:

$$\begin{aligned}
&\mathrm{rank}(\boldsymbol{A})\leqslant\min(n,p),\quad \mathrm{rank}(\boldsymbol{A})\geqslant 0,\\
&\mathrm{rank}(\boldsymbol{A})=\mathrm{rank}(\boldsymbol{A}^\top),\quad \mathrm{rank}(\boldsymbol{A}^\top\boldsymbol{A})=\mathrm{rank}(\boldsymbol{A}),\\
&\mathrm{rank}(\boldsymbol{A}+\boldsymbol{B})\leqslant\mathrm{rank}(\boldsymbol{A})+\mathrm{rank}(\boldsymbol{B}),\\
&\mathrm{rank}(\boldsymbol{A}\boldsymbol{B})\leqslant\min(\mathrm{rank}(\boldsymbol{A}),\mathrm{rank}(\boldsymbol{B})).
\end{aligned}$$

对于矩阵 $\boldsymbol{A}_{(n\times p)}, \boldsymbol{B}_{(p\times q)}, \boldsymbol{C}_{(q\times n)}$, 如果 $\boldsymbol{A}$ 和 $\boldsymbol{C}$ 非奇异, 则

$$\operatorname{rank}(\boldsymbol{ABC}) = \operatorname{rank}(\boldsymbol{B}).$$

对于非奇异 $\boldsymbol{A}_{(p\times p)}$

$$|\boldsymbol{A}^{-1}| = |\boldsymbol{A}|^{-1},\ \operatorname{rank}(\boldsymbol{A}) = p.$$

2.1.11 矩阵的迹

一个 $m\times m$ 矩阵 (称为**方阵**、m **维方阵**或m **阶方阵**) $\boldsymbol{A} = \{a_{ij}\}$ 的对角线元素之和称为**迹**, 记为 $\operatorname{trace}(\boldsymbol{A}) = \sum_{i=1}^{m} a_{ii}$. $\operatorname{trace}(\boldsymbol{A})$ 也记为 $\operatorname{tr}(\boldsymbol{A})$. 对于 m 维方阵 $\boldsymbol{B} = \{b_{ij}\}, \boldsymbol{C} = \{c_{ij}\}, \boldsymbol{D} = \{d_{ij}\}$, 迹有下列性质 (这里的 c 为常数):

$$\begin{gathered}
\operatorname{tr}(\boldsymbol{A}+\boldsymbol{B}) = \operatorname{tr}(\boldsymbol{A}) + \operatorname{tr}(\boldsymbol{B}),\ \ \operatorname{tr}(c\boldsymbol{A}) = c\operatorname{tr}(\boldsymbol{A}),\ \ \operatorname{tr}(\boldsymbol{A}) = \operatorname{tr}(\boldsymbol{A}^{\top}),\\
\operatorname{tr}(\boldsymbol{A}^{\top}\boldsymbol{B}) = \operatorname{tr}(\boldsymbol{A}\boldsymbol{B}^{\top}) = \operatorname{tr}(\boldsymbol{B}^{\top}\boldsymbol{A}) = \operatorname{tr}(\boldsymbol{B}\boldsymbol{A}^{\top}) = \sum_{i,j} a_{ij}b_{ij},\\
\operatorname{tr}(\boldsymbol{ABCD}) = \operatorname{tr}(\boldsymbol{DABC}) = \operatorname{tr}(\boldsymbol{CDAB}) = \operatorname{tr}(\boldsymbol{BCDA}),\\
\operatorname{tr}(\boldsymbol{A}) = \operatorname{tr}(\boldsymbol{P}^{-1}\boldsymbol{AP}),\ \ \operatorname{tr}(\boldsymbol{A}\otimes\boldsymbol{B}) = \operatorname{tr}(\boldsymbol{A})\operatorname{tr}(\boldsymbol{B}).
\end{gathered}$$

2.1.12 特征值

考虑 $p\times p$ 矩阵 $\boldsymbol{A}$, 如果存在数量 λ 和向量 $\boldsymbol{v}$ 使得

$$\boldsymbol{Av} = \lambda\boldsymbol{v},$$

则称 λ 为一个**特征值**或**特征根**, $\boldsymbol{v}$ 为一个特征向量.

能够证明, 特征值 λ 为 p 阶多项式 $|\boldsymbol{A} - \lambda\boldsymbol{I}_p| = 0$ 的一个根. 因此, $\boldsymbol{A}$ 有 p 个特征值 $\lambda_1, \lambda_2, \ldots, \lambda_p$. 对每个特征值 λ_j 相应于一个特征向量 $\boldsymbol{v}_j$, 令 $\boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1, \lambda_2, \ldots, \lambda_p)$, 即对角线元素为特征值的对角矩阵.

行列式 $|\boldsymbol{A}|$ 和迹 $\operatorname{tr}(\boldsymbol{A})$ 有下列关系:

$$\begin{aligned}
|\boldsymbol{A}| =& |\boldsymbol{\Lambda}| = \prod_{j=1}^{p}\lambda_j,\\
\operatorname{tr}(\boldsymbol{A}) =& \operatorname{tr}(\boldsymbol{\Lambda}) = \sum_{j=1}^{p}\lambda_j,\ \text{或更一般地, } \operatorname{tr}(\boldsymbol{A}^k) = \sum_{i}\lambda_i^k.
\end{aligned}$$

对于幂等矩阵 $\boldsymbol{M}$, 仅能够有 $\{0,1\}$ 中的特征值, 即特征值 $\lambda = \lambda^2$, 因此, $\operatorname{tr}(\boldsymbol{M}) = \operatorname{rank}(\boldsymbol{M})$ = 非零特征值的个数.

下面是关于特征值问题的两个后面章节会用到的性质:

- 对于 $n\times p$ 矩阵 $\boldsymbol{A}$ 及 $p\times n$ 矩阵 $\boldsymbol{B}$, $\boldsymbol{AB}$ 和 $\boldsymbol{BA}$ 的非零特征值相同, 并有同样的重数. 如果 $p > n$, 那么 $\boldsymbol{BA}$ 有额外的零特征值. 如果 $\boldsymbol{x}$ 是 $\boldsymbol{AB}$ 的一个非平凡特征向量 (相应于非零特征值), 那么 $\boldsymbol{y} = \boldsymbol{Bx}$ 为 $\boldsymbol{BA}$ 的一个非平凡特征向量.

- 对于 $n \times p$ 矩阵 $\boldsymbol{A}$, $q \times n$ 矩阵 $\boldsymbol{B}$, $p \times 1$ 维向量 $\boldsymbol{a}$, $q \times 1$ 维向量 $\boldsymbol{b}$, 矩阵 $\boldsymbol{A}\boldsymbol{a}\boldsymbol{b}^{\top}\boldsymbol{B}$ 的秩最多为 1, 其非零特征值如果存在, 则等于 $\boldsymbol{b}^{\top}\boldsymbol{B}\boldsymbol{A}\boldsymbol{a}$, 相应的特征向量为 $\boldsymbol{A}\boldsymbol{a}$.

2.1.13 广义特征值

考虑 $p \times p$ 矩阵 $\boldsymbol{A}$ 和 $\boldsymbol{B}$, **广义特征值问题**是寻求满足

$$\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{B}\boldsymbol{v}$$

的数量 λ 和向量 $\boldsymbol{v}$. 显然, 如果 $\boldsymbol{B}^{-1}$ 存在, 则上述问题转换成

$$\boldsymbol{B}^{-1}\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v},$$

这可归纳到上一小节所讨论的特征值问题.

如果 $\boldsymbol{B}$ 为 Hermitian 正定矩阵 (见2.3.2), 则可以用后面要介绍的 Cholesky 分解 (见2.2.4)

$$\boldsymbol{B} = \boldsymbol{L}\boldsymbol{L}^{\boldsymbol{H}},$$

这里, $\boldsymbol{L}$ 是一个下三角矩阵, 而 $\boldsymbol{L}^{H}$ 是 $\boldsymbol{L}$ 的共轭转置矩阵[2], 于是有

$$\left[\boldsymbol{L}^{-1}\boldsymbol{A}(\boldsymbol{L}^{H})^{-1}\right]\left[\boldsymbol{L}^{H}\boldsymbol{v}\right] = \lambda\left[\boldsymbol{L}^{H}\boldsymbol{v}\right],$$

令 $\boldsymbol{C} = \left[\boldsymbol{L}^{-1}\boldsymbol{A}(\boldsymbol{L}^{H})^{-1}\right]$, $\boldsymbol{u} = \left[\boldsymbol{L}^{H}\boldsymbol{v}\right]$, 就回到了最初的特征值问题

$$\boldsymbol{C}\boldsymbol{u} = \lambda \boldsymbol{u}.$$

2.1.14 分块矩阵

一个矩阵可以分成几个子矩阵, 例如在矩阵

$$\boldsymbol{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \end{bmatrix}$$

中定义**子矩阵 (块)**

$$\boldsymbol{A}_{11} = \begin{bmatrix} a_{11} & a_{12} \end{bmatrix}, \quad \boldsymbol{A}_{12} = \begin{bmatrix} a_{13} & a_{14} \end{bmatrix},$$
$$\boldsymbol{A}_{21} = \begin{bmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}, \quad \boldsymbol{A}_{22} = \begin{bmatrix} a_{23} & a_{24} \\ a_{33} & a_{34} \end{bmatrix},$$

于是 $\boldsymbol{A}$ 可以写成下面的**分块矩阵**形式:

[2]每个 Hermitian 正定矩阵 (包括每个实质对称正定矩阵) 都有唯一的 Cholesky 分解. 对于实对称矩阵 $\boldsymbol{B}$, $\boldsymbol{L}^{H} = \boldsymbol{L}^{\top}$.

$$\boldsymbol{A}=\left[\begin{array}{cc|cc} a_{11} & a_{12} & a_{13} & a_{14} \\ \hline a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \end{array}\right]=\begin{bmatrix} \boldsymbol{A}_{11} & \boldsymbol{A}_{12} \\ \boldsymbol{A}_{21} & \boldsymbol{A}_{22} \end{bmatrix}.$$

对于一般维矩阵 $\boldsymbol{A}_{(n\times p)}$ 的 2×2 分块为:

$$\boldsymbol{A}=\begin{bmatrix} \boldsymbol{A}_{11} & \boldsymbol{A}_{12} \\ \boldsymbol{A}_{21} & \boldsymbol{A}_{22} \end{bmatrix},$$

这里, $\boldsymbol{A}_{ij}$ 的维数为 $(n_i\times p_j)$, $i,j=1,2, n_1+n_2=n$, $p_1+p_2=p$.

如果 $\boldsymbol{B}_{(n\times p)}$ 有和 $\boldsymbol{A}_{(n\times p)}$ 类似的分块, 则

$$\begin{aligned} \boldsymbol{A}+\boldsymbol{B}&=\begin{bmatrix} \boldsymbol{A}_{11}+\boldsymbol{B}_{11} & \boldsymbol{A}_{12}+\boldsymbol{B}_{12} \\ \boldsymbol{A}_{21}+\boldsymbol{B}_{21} & \boldsymbol{A}_{22}+\boldsymbol{B}_{22} \end{bmatrix}, \\ \boldsymbol{B}^{\top}&=\begin{bmatrix} \boldsymbol{B}_{11}^{\top} & \boldsymbol{B}_{21}^{\top} \\ \boldsymbol{B}_{12}^{\top} & \boldsymbol{B}_{22}^{\top} \end{bmatrix}, \\ \boldsymbol{A}\boldsymbol{B}^{\top}&=\begin{bmatrix} \boldsymbol{A}_{11}\boldsymbol{B}_{11}^{\top}+\boldsymbol{A}_{12}\boldsymbol{B}_{12}^{\top} & \boldsymbol{A}_{11}\boldsymbol{B}_{21}^{\top}+\boldsymbol{A}_{12}\boldsymbol{B}_{22}^{\top} \\ \boldsymbol{A}_{21}\boldsymbol{B}_{11}^{\top}+\boldsymbol{A}_{22}\boldsymbol{B}_{12}^{\top} & \boldsymbol{A}_{21}\boldsymbol{B}_{21}^{\top}+\boldsymbol{A}_{22}\boldsymbol{B}_{22}^{\top} \end{bmatrix}. \end{aligned}$$

一个重要的特殊情况是方阵 $\boldsymbol{A}$ $(p\times p)$ 划分成方形子矩阵 $\boldsymbol{A}_{11}$ 和 $\boldsymbol{A}_{22}$, 即 $n_j=p_j, j=1,2$ 的情况. 能够验证, 当 $\boldsymbol{A}$ 非奇异 $(\boldsymbol{A}\boldsymbol{A}^{-1}=I_p)$ 时, 则有

$$\boldsymbol{A}^{-1}=\begin{bmatrix} \boldsymbol{A}^{11} & \boldsymbol{A}^{12} \\ \boldsymbol{A}^{21} & \boldsymbol{A}^{22} \end{bmatrix},$$

其中 (符号 "$\equiv$" 代表 "定义为")

$$\begin{aligned} \boldsymbol{A}^{11}&=(\boldsymbol{A}_{11}-\boldsymbol{A}_{12}\boldsymbol{A}_{22}^{-1}\boldsymbol{A}_{21})^{-1}\equiv(\boldsymbol{A}_{11.2})^{-1} \\ \boldsymbol{A}^{12}&=-(\boldsymbol{A}_{11.2})^{-1}\boldsymbol{A}_{12}\boldsymbol{A}_{22}^{-1}, \\ \boldsymbol{A}^{21}&=-\boldsymbol{A}_{22}^{-1}\boldsymbol{A}_{21}(\boldsymbol{A}_{11.2})^{-1} \\ \boldsymbol{A}^{22}&=\boldsymbol{A}_{22}^{-1}+\boldsymbol{A}_{22}^{-1}\boldsymbol{A}_{21}(\boldsymbol{A}_{11.2})^{-1}\boldsymbol{A}_{12}\boldsymbol{A}_{22}^{-1}. \end{aligned}$$

另一种表示是在原始矩阵中把 $\boldsymbol{A}_{11}$ 和 $\boldsymbol{A}_{22}$ 的位置对换而得到. 如果 $\boldsymbol{A}_{11}$ 是非奇异的, 则有

$$|\boldsymbol{A}|=|\boldsymbol{A}_{11}||\boldsymbol{A}_{22}-\boldsymbol{A}_{21}\boldsymbol{A}_{11}^{-1}\boldsymbol{A}_{12}|=|\boldsymbol{A}_{11}||\boldsymbol{A}_{22.1}|;$$

如果 $\boldsymbol{A}_{22}$ 是非奇异的, 则有

$$|\boldsymbol{A}|=|\boldsymbol{A}_{22}||\boldsymbol{A}_{11}-\boldsymbol{A}_{12}\boldsymbol{A}_{22}^{-1}\boldsymbol{A}_{21}|=|\boldsymbol{A}_{22}||\boldsymbol{A}_{11.2}|.$$

基于逆及行列式的表示是一个常用的公式, 比如, 令

$$\boldsymbol{B}=\begin{bmatrix}1 & \boldsymbol{b}^{\top}\\ \boldsymbol{a} & \boldsymbol{A}\end{bmatrix},$$

这里, $\boldsymbol{a}$ 和 $\boldsymbol{b}$ 均为 $(p\times 1)$ 向量, 而 $\boldsymbol{A}$ 非奇异. 于是有

$$|\boldsymbol{B}|=|\boldsymbol{A}||1-\boldsymbol{b}^{\top}\boldsymbol{A}^{-1}\boldsymbol{a}|.$$

让两个关于 $\boldsymbol{B}^{22}$ 的表达式 (下标 1 和 2 对换得到的) 相等, 得到

$$(\boldsymbol{A}-\boldsymbol{a}\boldsymbol{b}^{\top})^{-1}=\boldsymbol{A}^{-1}+\frac{\boldsymbol{A}^{-1}\boldsymbol{a}\boldsymbol{b}^{\top}\boldsymbol{A}^{-1}}{1-\boldsymbol{b}^{\top}\boldsymbol{A}^{-1}\boldsymbol{a}}.$$

由前面关于 $|\boldsymbol{A}|$ 的两个公式可以得到前面2.1.12节已经列出的结果: 对于 $\boldsymbol{A}_{(n\times p)}$ 和 $\boldsymbol{B}_{(p\times n)}$, $\boldsymbol{AB}$ 和 $\boldsymbol{BA}$ 的非零特征值相同, 而且有相同的重数; 如果 $\boldsymbol{x}$ 是 $\boldsymbol{AB}$ 的关于特征值 $\lambda\neq 0$ 的一个特征向量, 那么 $\boldsymbol{y}=\boldsymbol{B}\boldsymbol{x}$ 是 $\boldsymbol{BA}$ 的一个特征向量.

因此, 还可以得到: 对于 $\boldsymbol{A}_{(n\times p)}$ 和 $\boldsymbol{B}_{(q\times n)}$, $\boldsymbol{a}_{(p\times 1)}$ 及 $\boldsymbol{b}_{(q\times 1)}$, 有

$$\operatorname{rank}(\boldsymbol{A}\boldsymbol{a}\boldsymbol{b}^{\top}\boldsymbol{B})\leqslant 1.$$

2.2 矩阵的分解

2.2.1 矩阵的特征值分解

特征值分解 (eigenvalue decomposition) 也称为**谱分解** (spectral decomposition) 为从空间 $\mathbb{R}^p$ 到其本身通过 $p\times p$ 方阵 $\boldsymbol{A}$ 的映射, $\boldsymbol{A}:\mathbb{R}^p\mapsto\mathbb{R}^p$. 如果秩为 r 的矩阵 $\boldsymbol{A}$ 的非零特征值和相应的特征向量矩阵分别为 $\lambda_1,\lambda_2,\ldots,\lambda_r$ 和 $\boldsymbol{\Gamma}=[\boldsymbol{\gamma}_1,\boldsymbol{\gamma}_2,\ldots,\boldsymbol{\gamma}_r]$, 则有

$$\boldsymbol{A}\boldsymbol{\Gamma}=\boldsymbol{\Gamma}\boldsymbol{\Lambda},$$

这里, $\Lambda=\operatorname{diag}(\lambda_1,\lambda_2,\ldots,\lambda_r)$. 如果 $\boldsymbol{A}$ 满秩, 则有特征值分解

$$\boldsymbol{A}=\boldsymbol{\Gamma}\boldsymbol{\Lambda}\boldsymbol{\Gamma}^{-1}.$$

每个实对称半正定矩阵[3] $\boldsymbol{A}(p\times p)$ 能够写成

$$\boldsymbol{A}=\boldsymbol{\Gamma}\boldsymbol{\Lambda}\boldsymbol{\Gamma}^{\top},$$

这里, $\boldsymbol{\Lambda}=\operatorname{diag}(\lambda_1,\lambda_2,\ldots,\lambda_p)$ 为 $\boldsymbol{A}$ 的特征值 $(\lambda_1,\lambda_2,\ldots,\lambda_p)$ 所组成的对角矩阵, 而且 $\boldsymbol{\Gamma}=[\boldsymbol{\gamma}_1,\boldsymbol{\gamma}_2,\ldots,\boldsymbol{\gamma}_p]$ 为 $\boldsymbol{A}$ 的特征值 λ_j $(j=1,2,\ldots,p)$ 所对应的特征向量 $\boldsymbol{\gamma}_j$ $(j=1,\ldots,p)$

[3] 关于矩阵的定性 (如正定、非负定等等, 请看2.3.2节).

作为列向量组成的一个正交矩阵. 换言之, 即有 $\boldsymbol{A}\boldsymbol{\gamma}_i = \lambda_i\boldsymbol{\gamma}_i$, $\boldsymbol{\gamma}_i^\top\boldsymbol{\gamma}_j = \delta_{ij}$, 这里

$$\delta_{ij} = \begin{cases} 1, & i = j; \\ 0, & i \neq j. \end{cases}$$

对于投影 $\boldsymbol{x} \mapsto \boldsymbol{A}\boldsymbol{x}$, 该分解的解释为: 先通过 $\boldsymbol{\Gamma}^\top$ 实行旋转: $\boldsymbol{\Gamma}^\top\boldsymbol{x}$, 再通过 $\boldsymbol{\Lambda}$ 做尺度调整: $\boldsymbol{\Lambda}\boldsymbol{\Gamma}^\top\boldsymbol{x}$, 最后通过 $\boldsymbol{\Gamma}$ 做旋转: $\boldsymbol{\Gamma}\boldsymbol{\Lambda}\boldsymbol{\Gamma}^\top\boldsymbol{x}$.

作为应用例子, 根据该结果可以确定对称矩阵 $\boldsymbol{A}$ 的指数幂, 如果 $\alpha \in \mathbb{R}$, 有

$$\boldsymbol{A}^\alpha = \boldsymbol{\Gamma}\boldsymbol{\Lambda}^\alpha\boldsymbol{\Gamma}^\top,$$

式中, $\boldsymbol{\Lambda}^\alpha = \mathrm{diag}(\lambda_1^\alpha, \lambda_2^\alpha, \ldots, \lambda_p^\alpha)$. 作为特例, 当 $\alpha = -1$ 及 $\alpha = -\frac{1}{2}$ 时, $\boldsymbol{A}^{-1} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}^{-1}\boldsymbol{\Gamma}^\top$, $\boldsymbol{A}^{-\frac{1}{2}} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}^{-\frac{1}{2}}\boldsymbol{\Gamma}^\top$,

当且仅当 $\boldsymbol{A}$ 是对称正定矩阵时, 这个特征值分解和下一小节 (2.2.2节) 奇异值分解相同. 对于非方阵 $\boldsymbol{A}$, 2.2.2节的奇异值分解 $\boldsymbol{A} = \boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^\top$(见式 (2.2.2)), 有矩阵分解

$$\boldsymbol{A}^\top\boldsymbol{A} = \boldsymbol{V}\boldsymbol{D}^2\boldsymbol{V}^\top;$$
$$\boldsymbol{A}\boldsymbol{A}^\top = \boldsymbol{U}\boldsymbol{D}^2\boldsymbol{U}^\top.$$

由于这两个矩阵是对称正定的, 因此该分解为特征值分解, $\boldsymbol{\Lambda} = \boldsymbol{D}^2$.

2.2.2 奇异值分解及广义奇异值分解

奇异值分解

奇异值分解 (singular value decomposition, SVD) 是上述特征值分解的推广: 每个具有秩 r 的矩阵 $\boldsymbol{A}(n \times p)$ 能够分解成

$$\boldsymbol{A} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^H, \tag{2.2.1}$$

这里矩阵 $\boldsymbol{U}_{n\times n}$ 和 $\boldsymbol{V}_{p\times p}$ 都是酉矩阵 (对于复数矩阵 $\boldsymbol{A}$)[4]或列正交 (对于实矩阵 $\boldsymbol{A}$) 的. 矩阵 $\boldsymbol{\Sigma}$ 为非负实元素组成的 $n \times p$ 对角矩阵. 矩阵 $\boldsymbol{\Sigma}$ 的非零对角线元素称为 $\boldsymbol{A}$ 的**非零奇异值**, 这些奇异值为 $\boldsymbol{A}\boldsymbol{A}^H$ 和 $\boldsymbol{A}^H\boldsymbol{A}$ 的共同非零特征值的平方根.[5]

注意: 由于矩阵 $\boldsymbol{A}$ 可能不对称, 而且即使对称, 也不一定会满秩, 因此奇异值分解式 (2.2.1) 在实矩阵 $\boldsymbol{A}$ 的情况往往简写为 (特别是在计算机程序的输出中):

$$\boldsymbol{A} = \boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^\top, \tag{2.2.2}$$

[4]矩阵 $\boldsymbol{U}$ 为酉矩阵 (unitary matrix, 也称幺矩阵) 的条件是其共轭转置 (亦称 Hermitian 转置)$\boldsymbol{U}^H$ 是其逆矩阵, 即 $\boldsymbol{U}^H\boldsymbol{U} = \boldsymbol{U}\boldsymbol{U}^H = \boldsymbol{I}$.

[5]矩阵 $\boldsymbol{A}\boldsymbol{A}^H$ 和 $\boldsymbol{A}^H\boldsymbol{A}$ 的正交特征向量分别称为 $\boldsymbol{A}$ 的**左奇异向量** (left-singular vectors) 和**右奇异向量** (right-singular vectors).

这里的 $\boldsymbol{U}_{n\times r}$ 和 $\boldsymbol{V}_{p\times r}$ 分别是式 (2.2.1) 中的 $\boldsymbol{U}$ 和 $\boldsymbol{V}$ 的前 r 列, 满足 $\boldsymbol{U}^{\top}\boldsymbol{U}=\boldsymbol{V}^{\top}\boldsymbol{V}=\boldsymbol{I}_r$, 而且, 作为式 (2.2.1) 中 $\boldsymbol{\Sigma}$ 非零部分子矩阵的 $\boldsymbol{D}_{r\times r}$ 为 $\boldsymbol{A}\boldsymbol{A}^{\top}$ 及 $\boldsymbol{A}^{\top}\boldsymbol{A}$ 的 r 个共同非零特征值[6]$(\lambda_1,\lambda_2,\ldots,\lambda_r)$ 的**平方根**[7]所组成的对角矩阵: $\boldsymbol{D}=\operatorname{diag}(\lambda_1^{1/2},\lambda_2^{1/2},\ldots,\lambda_r^{1/2}),\lambda_j>0,\ j=1,2,\ldots,r$. $\boldsymbol{U}$ 的列包含了 $\boldsymbol{A}\boldsymbol{A}^{\top}$ 的特征向量, 矩阵 $\boldsymbol{V}$ 的列包含了 $\boldsymbol{A}^{\top}\boldsymbol{A}$ 的特征向量.

投影 $\boldsymbol{x}\mapsto\boldsymbol{A}\boldsymbol{x}$ 是从 $\mathbb{R}^p$ 变换到 $\mathbb{R}^n$, 因此, 在这两个值域分别采用 $\boldsymbol{V}$ 和 $\boldsymbol{U}$ 的列向量为基. 奇异值分解使得这种分解尽可能简单. 可以利用前面对称矩阵的特征值分解 $\boldsymbol{A}^{\top}\boldsymbol{A}=\boldsymbol{V}\boldsymbol{D}\boldsymbol{V}^{\top}$, 这里的 $\boldsymbol{D}$ 为非降特征值 $\{\lambda_i\}$ 的对角阵, 矩阵 $\boldsymbol{V}=[\boldsymbol{v}_1,\boldsymbol{v}_2,\ldots,\boldsymbol{v}_p]$ 的列 (即 $\boldsymbol{A}^{\top}\boldsymbol{A}$ 的特征向量) 为正交的. 因此

$$(\boldsymbol{A}\boldsymbol{v}_i)^{\top}(\boldsymbol{A}\boldsymbol{v}_j)=\boldsymbol{v}_i^{\top}\boldsymbol{A}^{\top}\boldsymbol{A}\boldsymbol{v}_j=\boldsymbol{v}_i^{\top}(\lambda_j\boldsymbol{v}_j)=\lambda_j\boldsymbol{v}_i^{\top}\boldsymbol{v}_j.$$

这意味着 $\{\boldsymbol{A}\boldsymbol{v}_1,\boldsymbol{A}\boldsymbol{v}_2,\ldots,\boldsymbol{A}\boldsymbol{v}_p\}$ 的非零向量形成正交基. 对于 $i=j$, $|\boldsymbol{A}\boldsymbol{v}_i|^2=\lambda_i$. 由于 $\operatorname{rank}(\boldsymbol{A})=r$, 则 $\lambda_1\geqslant\lambda_2\geqslant\cdots\geqslant\lambda_r>0$, $\lambda_i=0,\forall i>r$, 对此范围的正交基可定义为:

$$\boldsymbol{u}_i=\frac{\boldsymbol{A}\boldsymbol{v}_i}{|\boldsymbol{A}\boldsymbol{v}_i|}=\frac{1}{\sqrt{\lambda_i}}\boldsymbol{A}\boldsymbol{v}_i,\quad 1\leqslant i\leqslant r.$$

如果 $r<n$, 则 $\{\boldsymbol{u}_i\}$ 形成 $\mathbb{R}^n$ 的正交基. 因为 $\boldsymbol{A}\boldsymbol{v}_i=\sqrt{\lambda_i}\boldsymbol{u}_i$, 即有前面的 $\boldsymbol{A}\boldsymbol{V}=\boldsymbol{U}\boldsymbol{D}$, 或者 $\boldsymbol{A}=\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^{\top}$.[8]

作为一个应用, 可以利用对于对称矩阵 $\boldsymbol{A}$ 的奇异值分解 $\boldsymbol{A}=\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^{\top}$ 来找到定义为

$$\boldsymbol{A}\boldsymbol{A}^{-}\boldsymbol{A}=\boldsymbol{A}$$

的 Moore-Penrose 广义逆 $\boldsymbol{A}^{-}$. 令 $\boldsymbol{A}^{-}=\boldsymbol{V}\boldsymbol{D}^{-1}\boldsymbol{U}^{\top}$, 于是

$$\boldsymbol{A}\boldsymbol{A}^{-}\boldsymbol{A}=\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^{\top}\boldsymbol{V}\boldsymbol{D}^{-1}\boldsymbol{U}^{\top}\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^{\top}=\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^{\top}=\boldsymbol{A}.$$

注: 广义逆 $\boldsymbol{A}^{-}$ 不唯一, 而且满足下列性质:

$$\begin{aligned}&\boldsymbol{A}^{-}\boldsymbol{A}\boldsymbol{A}^{-}=\boldsymbol{A}^{-},\\&(\boldsymbol{A}\boldsymbol{A}^{-})^{\top}=\boldsymbol{A}\boldsymbol{A}^{-},\\&(\boldsymbol{A}^{-}\boldsymbol{A})^{\top}=\boldsymbol{A}^{-}\boldsymbol{A}.\end{aligned}$$

广义奇异值分解 (定义 1)

下面的**广义奇异值分解** (generalized singular value decomposition, GSVD) 是前面奇

[6]一般来说, $\boldsymbol{A}\boldsymbol{A}^{\top}$ 及 $\boldsymbol{A}^{\top}\boldsymbol{A}$ 的非零特征值相同.

[7]注意: 在特征值分解中的对角矩阵 $\boldsymbol{\Lambda}$ 是由 $\boldsymbol{A}$ 的特征值组成的对角矩阵, 而不是特征值的平方根.

[8]注意: $\boldsymbol{A}^{\top}\boldsymbol{A}=\boldsymbol{V}\boldsymbol{D}^{\top}\boldsymbol{D}\boldsymbol{V}^{\top}$ 和 $\boldsymbol{A}\boldsymbol{A}^{\top}=\boldsymbol{U}\boldsymbol{D}\boldsymbol{D}^{\top}\boldsymbol{U}^{\top}$ 中的 $\boldsymbol{D}^{\top}\boldsymbol{D}$ 或 $\boldsymbol{D}\boldsymbol{D}^{\top}$ 的头 r 个对角线元素都是非零的 $\{\lambda_i\}$, 其余为 0.

异值分解的带约束的推广. 假定矩阵 $\boldsymbol{A}_{n\times p}$ 的奇异值分解为:

$$\boldsymbol{A}=\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^{H},$$

这里的约束为

$$\boldsymbol{U}^{H}\boldsymbol{W}_{u}\boldsymbol{U}=\boldsymbol{V}^{H}\boldsymbol{W}_{v}\boldsymbol{V}=\boldsymbol{I}.$$

这意味着, 给定约束 $\boldsymbol{W}_u$ 和 $\boldsymbol{W}_v$, 矩阵 $\boldsymbol{U}$ 和 $\boldsymbol{V}$ 为正交的. $\boldsymbol{W}_u$ 和 $\boldsymbol{W}_v$ 往往为正定的, 甚至是正定对角矩阵.

广义奇异值分解 (定义 2)

还有一种广义奇异值分解: 对于 $m\times n$ 矩阵 $\boldsymbol{A}$ 和 $p\times n$ 矩阵 $\boldsymbol{B}$, 它们的**广义奇异值分解**由下面一对因子化定义:

$$\begin{aligned}\boldsymbol{A}&=\boldsymbol{U}\boldsymbol{\Sigma}_1[\boldsymbol{0},\boldsymbol{R}]\boldsymbol{Q}^{\top};\\ \boldsymbol{B}&=\boldsymbol{V}\boldsymbol{\Sigma}_2[\boldsymbol{0},\boldsymbol{R}]\boldsymbol{Q}^{\top}.\end{aligned}$$

这里的矩阵有下面的性质:

- $\boldsymbol{U}$ 是 $m\times m$ 阶, $\boldsymbol{V}$ 是 $p\times p$ 阶, $\boldsymbol{Q}$ 是 $n\times n$ 阶, 而且这三个矩阵均为正交矩阵. 如果 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 是复数矩阵, 则它们是酉矩阵, 这时, 因子化公式中的 $Q^{\top}$ 应该换成 Q^{H}.
- $\boldsymbol{R}$ 是 $r\times r$ 阶的上三角非奇异矩阵. $[\boldsymbol{0},\boldsymbol{R}]$ 是 $r\times n$ 矩阵, $\boldsymbol{0}$ 是 $r\times(n-r)$ 零矩阵, 这里的 $r\leqslant n$ 为 $\begin{bmatrix}\boldsymbol{A}\\ \boldsymbol{B}\end{bmatrix}$ 的秩.
- $\boldsymbol{\Sigma}_1$ 是 $m\times r$ 阶, $\boldsymbol{\Sigma}_2$ 是 $p\times r$ 阶, 它们均为非负实对角矩阵, 而且 $\boldsymbol{\Sigma}_1^{\top}\boldsymbol{\Sigma}_1+\boldsymbol{\Sigma}_2^{\top}\boldsymbol{\Sigma}_2=\boldsymbol{I}$. $\boldsymbol{\Sigma}_1^{\top}\boldsymbol{\Sigma}_1=\mathrm{diag}(\alpha_1^2,\alpha_2^2,\ldots,\alpha_r^2)$, $\boldsymbol{\Sigma}_2^{\top}\boldsymbol{\Sigma}_2=\mathrm{diag}(\beta_1^2,\beta_2^2,\ldots,\beta_r^2)$, 这里, $\alpha_i,\beta_i\in(0,1)$ $(i=1,2,\ldots,r)$. $\alpha_1/\beta_1,\alpha_2/\beta_2,\ldots,\alpha_r/\beta_r$ 称为 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 矩阵对的广义特征值. 如果 $\beta_i=0$, 则 $\alpha_i/\beta_i=\infty$.

2.2.3 QR 分解

一个 $n\times n$ 实方阵 $\boldsymbol{A}$ 可以分解为

$$\boldsymbol{A}=\boldsymbol{Q}\boldsymbol{R},$$

这里的 $\boldsymbol{Q}$ 是一个 $n\times n$ 正交矩阵, 即 $\boldsymbol{Q}^{\top}\boldsymbol{Q}=\boldsymbol{I}$, 而 $\boldsymbol{R}$ 是一个 $n\times n$ 上三角矩阵. 如果 $\boldsymbol{A}$ 可逆, 则该分解是唯一的. 如果矩阵 $\boldsymbol{A}$ 的列线性独立, 则 $\boldsymbol{Q}$ 的列形成 $\boldsymbol{A}$ 的列空间的正交基. 对于复数矩阵, 上述分解中的 $\boldsymbol{Q}$ 为酉矩阵, 即 $\boldsymbol{Q}^{H}\boldsymbol{Q}=\boldsymbol{I}$, 其中 $\boldsymbol{Q}^{H}$ 为 $\boldsymbol{Q}$ 的共轭转置.

对于 $m\times n$ $(m>n)$ 方阵 $\boldsymbol{A}$, 上述分解 $\boldsymbol{A}=\boldsymbol{Q}\boldsymbol{R}$ 中的 $\boldsymbol{Q}$ 为 $m\times m$ 酉矩阵, 而 $\boldsymbol{R}$ 为 $m\times n$ 上三角矩阵, 其中, 下面 $m-n$ 行全部为 0. 因此, 该分解可以分块为:

$$\boldsymbol{A}=\boldsymbol{Q}\boldsymbol{R}=\boldsymbol{Q}\begin{bmatrix}\boldsymbol{R}_1\\ \boldsymbol{0}\end{bmatrix}=\begin{bmatrix}\boldsymbol{Q}_1 & \boldsymbol{Q}_2\end{bmatrix}\begin{bmatrix}\boldsymbol{R}_1\\ \boldsymbol{0}\end{bmatrix}=\boldsymbol{Q}_1\boldsymbol{R}_1,$$

这里的 $\boldsymbol{R}_1$ 为 $n \times n$ 上三角矩阵, $\boldsymbol{0}$ 为 $(m-n) \times n$ 零矩阵, $\boldsymbol{Q}_1$ 和 $\boldsymbol{Q}_2$ 分别为 $m \times n$ 及 $m \times (m-n)$ 矩阵, 皆为正交矩阵.

2.2.4 Cholesky 分解

如果 $\boldsymbol{A}$ 为实质对称正定矩阵, Cholesky 分解为:

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^{\top},$$

这里的 $\boldsymbol{L}$ 是一个下三角矩阵. 如果 $\boldsymbol{A}$ 为更一般的 Hermitian 正定矩阵 (见2.3.2), Cholesky 分解为:

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^{H}.$$

这里 $\boldsymbol{L}$ 是一个下三角矩阵, 而 $\boldsymbol{L}^H$ 是 $\boldsymbol{L}$ 的共轭转置矩阵, 对于实对称正定矩阵 (及更一般的 Hermitian 正定矩阵), 都有唯一的 Cholesky 分解.

Cholesky 分解有很多应用, 比如解方程 (这里的 $\boldsymbol{A}$ 为实对称正定矩阵)

$$\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b},$$

则变成解

$$\boldsymbol{L}\boldsymbol{L}^{\top}\boldsymbol{x} = \boldsymbol{b}.$$

令 $\boldsymbol{L}^{\top}\boldsymbol{x} = \boldsymbol{y}$, 有 $\boldsymbol{b} = \boldsymbol{L}\boldsymbol{L}^{\top}\boldsymbol{x} = \boldsymbol{L}\boldsymbol{y}$. 这是两个方程, 其中方程

$$\boldsymbol{L}\boldsymbol{y} = \boldsymbol{b}$$

的系数是下三角阵, 可以用向前替代法 (forward substitution) 解出 $\boldsymbol{y}$. 而另一个方程

$$\boldsymbol{L}^{\top}\boldsymbol{x} = \boldsymbol{y}$$

的系数是上三角阵, 可以用向后替代法 (backward substitution) 解出 $\boldsymbol{x}$.

当然也可以用

$$\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$$

来直接解出 $\boldsymbol{x}$, 但这所需的计算量大约比上面通过 Cholesky 分解的计算量多 3 倍.

显然, 可以用 Cholesky 分解 $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^{\top}$ 来求正定矩阵 $\boldsymbol{A}$ 的逆. 如果 $\boldsymbol{L}$ 是可逆的, 即其对角线元素为非零, 容易验证 $\boldsymbol{B} = (\boldsymbol{L}^{\top})^{-1}\boldsymbol{L}^{-1}$ 既为 $\boldsymbol{A}$ 的右逆矩阵, 即 $\boldsymbol{A}\boldsymbol{B} = \boldsymbol{I}$, 也是 $\boldsymbol{A}$ 的左逆矩阵, 即 $\boldsymbol{B}\boldsymbol{A} = \boldsymbol{I}$. 所以, 如果 $\boldsymbol{A}$ 是可逆的, 则 $\boldsymbol{B} = (\boldsymbol{L}^{\top})^{-1}\boldsymbol{L}^{-1}$ 为 $\boldsymbol{A}$ 的逆矩阵, 即

$$\boldsymbol{A}^{-1} = (\boldsymbol{L}^{\top})^{-1}\boldsymbol{L}^{-1}.$$

2.3 二次型

2.3.1 定义

二次型 $Q(\boldsymbol{x})$ 是一个 $\mathbb{R}^p \mapsto \mathbb{R}$ 的投影, 由对称矩阵 $\boldsymbol{A}_{p\times p} = \{a_{ij}\}$ $(a_{ij} = a_{ji})$ 和向量 $\boldsymbol{x} = (x_1, x_2, \ldots, x_p) \in \mathbb{R}^p$ 组成:

$$Q(x) = \boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} = \sum_{i=1}^{p}\sum_{j=1}^{p} a_{ij}x_i x_j.$$

二次型具有唯一性, 如果对于对称的 $\boldsymbol{A}$ 和 $\boldsymbol{B}$, $\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} = \boldsymbol{x}^\top \boldsymbol{B}\boldsymbol{x}, \forall \boldsymbol{x} \in \mathbb{R}^p$, 那么 $\boldsymbol{A} = \boldsymbol{B}$.

对于常数 a, 满足 $Q(\boldsymbol{x}) = a$ 的 $\boldsymbol{x}$ (记为 $\{\boldsymbol{x}|Q(\boldsymbol{x}) = a\}$) 形成的曲面称为二次曲面, 而 $\{\boldsymbol{x}|Q(\boldsymbol{x}) \leqslant a\}$ 称为二次区域.

根据对称矩阵的特征值分解 $\boldsymbol{A} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}\boldsymbol{\Gamma}^\top$, 有

$$\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} = \boldsymbol{x}^\top \boldsymbol{\Gamma}\boldsymbol{\Lambda}\boldsymbol{\Gamma}^\top \boldsymbol{x} = (\boldsymbol{\Gamma}^\top \boldsymbol{x})^\top \boldsymbol{\Lambda}(\boldsymbol{\Gamma}^\top \boldsymbol{x}) = \sum_{i=1}^{p} \lambda_i (\boldsymbol{\gamma}_i^\top \boldsymbol{x})^2$$

$$\leqslant \lambda_1 \sum_{i=1}^{p} \lambda_i (\boldsymbol{\gamma}_i^\top \boldsymbol{x})^2 = \lambda_1 \|\boldsymbol{x}\|^2,$$

即 $\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} \leqslant \lambda_1 \boldsymbol{x}^\top \boldsymbol{x}$.

上述结果还可以叙述为: 如果 $\boldsymbol{A}$ 为对称的, 而且 $Q(x) = \boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x}$ 为相应的二次型, 那么存在一个正交变换 $\boldsymbol{x} \mapsto \boldsymbol{\Gamma}^\top \boldsymbol{x} = \boldsymbol{y}$ 使得

$$\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} = \sum_{i=1}^{p} \lambda_i y_i^2,$$

这里的 λ_i 为 $\boldsymbol{A}$ 的特征值.

而且, 如果 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 为对称的, 且 $\boldsymbol{B} > 0$, 那么在约束 $\boldsymbol{x}^\top \boldsymbol{B}\boldsymbol{x} = 1$ 下, $\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x}$ 的最大值为 $\boldsymbol{B}^{-1}\boldsymbol{A}$ 的最大特征值. 更一般地, 有

$$\max_{\{\boldsymbol{x}:\ \boldsymbol{x}^\top \boldsymbol{B}\boldsymbol{x}=1\}} \boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} = \lambda_1 \geqslant \lambda_2 \geqslant \cdots \geqslant \lambda_p = \min_{\{\boldsymbol{x}:\ \boldsymbol{x}^\top \boldsymbol{B}\boldsymbol{x}=1\}} \boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x},$$

这里的 λ_i 为 $\boldsymbol{B}^{-1}\boldsymbol{A}$ 的特征值. 在约束 $\boldsymbol{x}^\top \boldsymbol{B}\boldsymbol{x} = 1$ 下, 使 $\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x}$ 最大 (最小) 的向量为 $\boldsymbol{B}^{-1}\boldsymbol{A}$ 的相应于最大 (最小) 特征值的特征向量.

2.3.2 二次型和矩阵的定性

考虑对称矩阵 $\boldsymbol{A} \in \mathbb{R}^{p\times p}$. 如果 $Q(x) > 0$, $\forall x \neq 0$, 称 $\boldsymbol{A}$ 为**正定**的, 并记为 $\boldsymbol{A} > 0$. $\boldsymbol{A} > 0$ 的充分必要条件为其最小特征值大于零. 如果 $Q(x) \geqslant 0$, $\forall x \neq 0$, 称 $\boldsymbol{A}$ 为**半正定**的, 并记为 $\boldsymbol{A} \geqslant 0$. $\boldsymbol{A} \geqslant 0$ 的充分必要条件为其特征值均为非负. 类似地, 可以定义**负定**和**半负定**.

可以用不等式表示同阶对称矩阵的关系, 例如, 对于对称矩阵 $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{R}^{p\times p}$, 如果 $\boldsymbol{A} -$

$\boldsymbol{B} \geqslant 0$, 则记 $\boldsymbol{A} \geqslant \boldsymbol{B}$. 如果 $\boldsymbol{B}-\boldsymbol{A} < 0$, 则记 $\boldsymbol{A} > \boldsymbol{B}$. $\boldsymbol{A} > \boldsymbol{B}$ 意味着 $\boldsymbol{x}^\top \boldsymbol{A}x > \boldsymbol{x}^\top \boldsymbol{B}\boldsymbol{x}$, $\forall \boldsymbol{x} \neq \boldsymbol{0}$.

更一般的为 Hermitian 正定矩阵. 一个 Hermitian 矩阵等于其共轭转置, 也称为自伴随矩阵 (self-adjoint matrix). 一个 $n \times n$ 阶的 Hermitian 矩阵 $\boldsymbol{A}$ 的第 ij 元素 a_{ij} 满足

$$a_{ij} = \overline{a_{ji}} \text{ 或者 } \boldsymbol{A} = \overline{\boldsymbol{A}^\top}.$$

所以, Hermitian 矩阵是实对称矩阵的推广. Hermitian 正定矩阵满足 $\boldsymbol{z}^H \boldsymbol{A}\boldsymbol{z}$ 为正实数, 这里的向量 $\boldsymbol{z}^H$ 表示向量 $\boldsymbol{z}$ 的共轭转置. 类似地, 可以定义 Hermitian 半正定、负定和半负定.

2.3.3 椭球

令 $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^p$, 而 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 之间的**距离**定义为下面的函数 d.

$$d : \mathbb{R}^{2p} \mapsto \mathbb{R}_+ \text{ 满足} \begin{cases} d(\boldsymbol{x}, \boldsymbol{y}) > 0, & \forall \boldsymbol{x} \neq \boldsymbol{y}; \\ d(\boldsymbol{x}, \boldsymbol{y}) = 0, & \text{当且仅当 } \boldsymbol{x} = \boldsymbol{y}; \\ d(\boldsymbol{x}, \boldsymbol{y}) \leqslant d(\boldsymbol{x}, \boldsymbol{z}) + d(\boldsymbol{z}, \boldsymbol{y}), & \forall \boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z} \ (\boldsymbol{z} \in \mathbb{R}^p). \end{cases}$$

$\boldsymbol{x}$ 和 $\boldsymbol{y}$ 之间的欧式距离 d 定义为:

$$d^2(\boldsymbol{x}, \boldsymbol{y}) = (\boldsymbol{x} - \boldsymbol{y})^\top \boldsymbol{A}(\boldsymbol{x} - \boldsymbol{y}),$$

这里的 $\boldsymbol{A}$ 为正定矩阵 ($\boldsymbol{A} > 0$). $\boldsymbol{A}$ 称为一个度量, 特例为 $\boldsymbol{A} = \boldsymbol{I}_p$.

集合 $E_d = \{x \in \mathbb{R}^p | (\boldsymbol{x} - \boldsymbol{x}_0)^\top (\boldsymbol{x} - \boldsymbol{x}_0) = d^2\}$, 即以 $\boldsymbol{x}_0$ 为中心、d 为半径的球面.

更一般的情况为:

$$E_d = \{x \in \mathbb{R}^p | (\boldsymbol{x} - \boldsymbol{x}_0)^\top \boldsymbol{A}(\boldsymbol{x} - \boldsymbol{x}_0) = d^2\}.$$

它是以 $\boldsymbol{x}_0$ 为中心的椭球面. 如果这一等式中的等号改成小于等于号 ($\leqslant d^2$), 则为以 $\boldsymbol{x}_0$ 为中心的椭球.

令 $\boldsymbol{\gamma}_1, \boldsymbol{\gamma}_2, \ldots, \boldsymbol{\gamma}_p$ 为 $\boldsymbol{A}$ 的相应于特征值 $\lambda_1, \lambda_2, \ldots, \lambda_p$ 的正交特征向量, 有

(i) E_d 的主轴方向为 $\boldsymbol{\gamma}_i, i = 1, 2, \ldots, p$;

(ii) 半轴长度为 $\sqrt{d^2/\lambda_i}, i = 1, 2, \ldots, p$;

(iii) 围绕 E_d 的矩形由下列不等式定义:

$$x_{0i} - \sqrt{d^2 a^{ii}} \leqslant x_i \leqslant x_{0i} + \sqrt{d^2 a^{ii}}, \quad i = 1, 2, \ldots, p,$$

这里 a^{ii} 为 $\boldsymbol{A}^{-1}$ 的 (i, i) 元素. 围绕 E_d 的矩形的边平行于坐标轴. 作为特例, 集合 $E = \{\boldsymbol{x} \in \mathbb{R}^p | \boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} \leqslant 1\}$ 形成一个中心为原点 $\boldsymbol{0}$ 的椭球. 该椭球有 $r = \operatorname{rank}(\boldsymbol{A})$ 个互相正交的轴, 第 i 个轴的半轴长度为 $s_i = 1/\sqrt{\lambda_i}$, 这里的 λ_i 为 $\boldsymbol{A}$ 的第 i 个特征值, 也就是说特征值确定轴的长度. 相应的特征向量方向确定相应轴的方向. 最大最小特征值平方根之比

$\sqrt{\lambda_{\max}/\lambda_{\min}}$ 给出了最大的离心率.

如果对称矩阵 $B>0$ 定义的椭球为 $E^*=\{\boldsymbol{x}\in\mathbb{R}^p|\boldsymbol{x}^\top\boldsymbol{B}\boldsymbol{x}\leqslant 1\}$, 则 $E^*\supseteq E$ 的充分必要条件是 $\boldsymbol{B}\geqslant\boldsymbol{A}$.

2.4 矩阵的导数

2.4.1 向量关于数量的偏导数

对于向量 $\boldsymbol{y}=[y_1,y_2,\ldots,y_m]^\top$ 及数量 x, 定义

$$\frac{\partial\boldsymbol{y}}{\partial x}=\left[\frac{\partial y_1}{\partial x},\frac{\partial y_2}{\partial x},\ldots,\frac{\partial y_m}{\partial x}\right]^\top=\begin{bmatrix}\frac{\partial y_1}{\partial x}\\ \frac{\partial y_2}{\partial x}\\ \vdots\\ \frac{\partial y_m}{\partial x}\end{bmatrix},$$

这是投影 $\mathbb{R}\mapsto\mathbb{R}^m$.

2.4.2 数量关于向量的偏导数

对于数量 y 及向量 $\boldsymbol{x}=[x_1,x_2,\ldots,x_n]^\top$, 定义

$$\frac{\partial y}{\partial\boldsymbol{x}}=\left[\frac{\partial y}{\partial x_1},\frac{\partial y}{\partial x_2},\ldots,\frac{\partial y}{\partial x_n}\right].$$

2.4.3 向量关于向量的偏导数

对于向量 $\boldsymbol{y}=[y_1,y_2,\ldots,y_m]^\top$ 及向量 $\boldsymbol{x}=[x_1,x_2,\ldots,x_n]^\top$, 定义

$$\frac{\partial\boldsymbol{y}}{\partial\boldsymbol{x}}=\begin{bmatrix}\frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} & \cdots & \frac{\partial y_1}{\partial x_n}\\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} & \cdots & \frac{\partial y_2}{\partial x_n}\\ \vdots & \vdots & \ddots & \vdots\\ \frac{\partial y_m}{\partial x_1} & \frac{\partial y_m}{\partial x_2} & \cdots & \frac{\partial y_m}{\partial x_n}\end{bmatrix}.$$

2.4.4 矩阵关于数量的偏导数

对于 $m\times n$ 矩阵 $\boldsymbol{y}=\{y_{ij}\}$ 及数量 x, 定义

$$\frac{\partial\boldsymbol{y}}{\partial x}=\begin{bmatrix}\frac{\partial y_{11}}{\partial x} & \frac{\partial y_{12}}{\partial x} & \cdots & \frac{\partial y_{1n}}{\partial x}\\ \frac{\partial y_{21}}{\partial x} & \frac{\partial y_{22}}{\partial x} & \cdots & \frac{\partial y_{2n}}{\partial x}\\ \vdots & \vdots & \ddots & \vdots\\ \frac{\partial y_{m1}}{\partial x} & \frac{\partial y_{m2}}{\partial x} & \cdots & \frac{\partial y_{mn}}{\partial x}\end{bmatrix}.$$

2.4.5 数量关于矩阵的偏导数

对于数量 y 及 $p\times q$ 矩阵 $\boldsymbol{X}=\{x_{ij}\}$, 定义

$$\frac{\partial y}{\partial \boldsymbol{X}}=\begin{bmatrix}\frac{\partial y}{\partial x_{11}} & \frac{\partial y}{\partial x_{21}} & \cdots & \frac{\partial y}{\partial x_{p1}}\\ \frac{\partial y}{\partial x_{12}} & \frac{\partial y}{\partial x_{22}} & \cdots & \frac{\partial y}{\partial x_{p2}}\\ \vdots & \vdots & \ddots & \vdots\\ \frac{\partial y}{\partial x_{1q}} & \frac{\partial y}{\partial x_{2q}} & \cdots & \frac{\partial y}{\partial x_{pq}}\end{bmatrix}.$$

2.4.6 有关内积、二次型的导数

考虑向量 $\boldsymbol{x}=[x_1,x_2,\ldots,x_n]^\top$. 此外, 向量 $\boldsymbol{a},\boldsymbol{b}$ 及矩阵 $\boldsymbol{A}$ 不是 $\boldsymbol{x}$ 的函数. 有

$$\frac{\partial \boldsymbol{a}^\top\boldsymbol{x}}{\partial \boldsymbol{x}}=\frac{\partial \boldsymbol{x}\boldsymbol{a}^\top}{\partial \boldsymbol{x}}=\boldsymbol{a}\text{ 或 }\boldsymbol{a}^\top;$$

$$\frac{\partial \boldsymbol{b}^\top\boldsymbol{A}\boldsymbol{x}}{\partial x}=\boldsymbol{b}^\top\boldsymbol{A}\text{ 或 }\boldsymbol{A}^\top\boldsymbol{b};$$

$$\begin{aligned}\frac{\partial \boldsymbol{x}^\top\boldsymbol{A}\boldsymbol{x}}{\partial \boldsymbol{x}}&=\boldsymbol{x}^\top(\boldsymbol{A}+\boldsymbol{A}^\top)\text{ 或 }(\boldsymbol{A}+\boldsymbol{A}^\top)\boldsymbol{x};\\ &=2\boldsymbol{x}^\top\boldsymbol{A}\text{ 或 }2\boldsymbol{A}\boldsymbol{x}\ \text{(当 }\boldsymbol{A}\text{ 对称时)};\end{aligned}$$

$$\frac{\partial \boldsymbol{x}^\top\boldsymbol{x}}{\partial \boldsymbol{x}}=2\boldsymbol{x}^\top\text{ 或 }2\boldsymbol{x};$$

$$\frac{\partial \boldsymbol{a}^\top\boldsymbol{x}\boldsymbol{x}^\top\boldsymbol{b}}{\partial \boldsymbol{x}}=\boldsymbol{x}^\top(\boldsymbol{a}\boldsymbol{b}^\top+\boldsymbol{b}\boldsymbol{a}^\top)\text{ 或 }(\boldsymbol{a}\boldsymbol{b}^\top+\boldsymbol{b}\boldsymbol{a}^\top)\boldsymbol{x}.$$

2.4.7 函数的偏导数

考虑 $f:\mathbb{R}^p\mapsto\mathbb{R}$ 和 $p\times1$ 向量 $\boldsymbol{x}$, 那么 $\partial f(\boldsymbol{x})/\partial \boldsymbol{x}$ 为一个偏导数的列向量 $\{\partial f(\boldsymbol{x})/\partial x_j\}(j=1,\ldots,p)$, 而 $\partial f(\boldsymbol{x})/\partial \boldsymbol{x}^\top$ 为同一偏导数的行向量. $\partial f(\boldsymbol{x})/\partial \boldsymbol{x}$ 称为 f 的**梯度**.

$\partial^2 f(\boldsymbol{x})/\partial \boldsymbol{x}\partial \boldsymbol{x}^\top$ 为二阶导数, 是 $p\times p$ 矩阵, 元素为 $\left\{\frac{\partial^2 f(\boldsymbol{x})}{\partial x_i\partial x_j}\right\}$ $(i=1,2,\ldots,p;\ j=1,2,\ldots,p)$. $\partial^2 f(\boldsymbol{x})/\partial \boldsymbol{x}\partial \boldsymbol{x}^\top$ 称为 f 的 Hessian 矩阵.

二次型 $Q(\boldsymbol{x})=\boldsymbol{x}^\top\boldsymbol{A}\boldsymbol{x}$ 的 Hessian 矩阵为:

$$\frac{\partial \boldsymbol{x}^\top\boldsymbol{A}\boldsymbol{x}}{\partial \boldsymbol{x}\partial \boldsymbol{x}^\top}=2\boldsymbol{A}$$

例 2.1. 导数在线性模型最小二乘估计中的应用 为了求使得 $(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\beta})^\top(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\beta})$ 最小的 $\boldsymbol{\beta}$, 即

$$\min_{\boldsymbol{\beta}}\{(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\beta})^\top(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\beta})\}=\min_{\boldsymbol{\beta}}\{\boldsymbol{y}^\top\boldsymbol{y}-2\boldsymbol{y}^\top\boldsymbol{X}\boldsymbol{\beta}+\boldsymbol{\beta}^\top\boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{\beta}\},$$

对残差平方和求导, 令其等于 0, 问题归结为解方程

$$\frac{\partial[(\boldsymbol{y}-\boldsymbol{X}\beta)^{\top}(\boldsymbol{y}-\boldsymbol{X}\beta)]}{\partial\beta}=\frac{\partial(\boldsymbol{y}^{\top}\boldsymbol{y}-2\boldsymbol{y}^{\top}\boldsymbol{X}\beta+\beta^{\top}\boldsymbol{X}^{\top}\boldsymbol{X}\beta)}{\partial\beta}=-2\boldsymbol{X}^{\top}\boldsymbol{y}+2\boldsymbol{X}^{\top}\boldsymbol{X}\beta=0$$

得到

$$\beta=(\boldsymbol{X}^{\top}\boldsymbol{X})^{-1}\boldsymbol{X}^{\top}\boldsymbol{y}.$$

2.5 本章矩阵简单运算的 R 和 Python 代码

以下代码做如下几种运算:

- 生成由正态分布 (分别为 $N(-1,1)$ 和 $N(1,1)$) 随机数组成的两个矩阵 $\boldsymbol{A}_{(3\times4)}$ 和 $\boldsymbol{B}_{(4\times3)}$, 代码中确定随机种子是为了在重复运行同样代码时产生一致的结果, 注意: 不同的软件或程序, 即便设定相同数目的随机种子, 产生的 (伪) 随机数也可能不一样.
- 计算 $2\boldsymbol{A}+3\boldsymbol{B}^{\top}$.
- 计算乘积 $\boldsymbol{C}=\boldsymbol{A}\boldsymbol{B}$.

R

```
set.seed(1010)
(A=matrix(rnorm(12,-1,1),3,4)) #赋值表达式加括号意味着打印结果
(B=matrix(rnorm(12,1,1),4,3))
(2*A+3*t(B))
(C=A%*%B)#%*%是矩阵乘号
```

Python

```
import numpy as np
np.random.seed(1010)
A=np.random.normal(-1,1,size=12).reshape(3,4)
B=np.random.normal(1,1,size=12).reshape(4,3)
2*A+3*B.T
C=np.dot(A,B)#或者C=A.dot(B)
print(A,"\n",B,"\n",C)
```

以下代码做如下几种运算:

- 计算行列式 $|\boldsymbol{C}|$ 的值.
- 输出 $\boldsymbol{C}^{-1}\boldsymbol{C}$ 的结果, 它应该等于单位矩阵, 但由于浮点运算, 结果不大可能刚好是 0 和 1 组成的单位矩阵, 我们将它四舍五入到小数点后 14 位.
- 通过 Kronecker 乘积 $\boldsymbol{K}=\boldsymbol{A}\otimes\boldsymbol{B}$ 的计算, 得到 $\boldsymbol{K}$ 的维数为 12×12.
- 产生幂等矩阵 $\boldsymbol{M}=\boldsymbol{B}(\boldsymbol{B}^{\top}\boldsymbol{B})^{-1}\boldsymbol{B}^{\top}$, 并验证 $\boldsymbol{M}\boldsymbol{M}-\boldsymbol{M}$ 等于全部是 0 的矩阵 (也将其四舍五入到小数点后 14 位).

注意: 当 Python 代码一行太长时, 可以加上反斜杠 "\" (其后面不能有空格) 来表示这一行命令没有完; 在 R 代码中, 如果一行太长, 可以在头一行明显没有独立意义时换行, 比如语句 `rnorm(5)[3]`, 不可以在 `rnorm(5)` 之后换行, 但可以在 `rnorm(5)[3` 之后或者 `rnorm(`之后换行.

R

```
det(C)#行列式
CI=solve(C)#逆矩阵
CI%*%C
round(CI%*%C,14)
K=A%x%B #%x%为Kronecker乘号
dim(K) #维数
(M=B%*%solve(t(B)%*%B)%*%t(B))#幂等阵
round(M%*%M-M,14)
```

Python

```
print (np.linalg.det(C))#行列式
CI=np.linalg.inv(C)#逆矩阵
print (CI.dot(C))
print (np.round(CI.dot(C),14))
K=np.kron(A,B)#Kronecker乘积
K.shape #维数
M=np.dot(np.dot(B,np.linalg.inv(\
np.dot(B.T,B))),B.T)
print(M,"\n",np.round(np.dot(M,M)-M,14))
```

下面代码随机产生 25×4 矩阵 $\boldsymbol{X}$; 计算 $\boldsymbol{X}$ 的相关阵; 解 $\boldsymbol{X}$ 的相关阵的特征值问题; 验证特征向量组成正交矩阵; 验证 $\boldsymbol{X}$ 相关阵的对角线元素的和 (迹) 等于其特征根的和.

R

```
set.seed(1010)
X=matrix(rnorm(100),25,4)
XC=cor(X)
a=eigen(XC)
a$values #特征值, 可简写为a$va
a$vectors #特征向量, 可简写为a$ve
round(t(a$vectors)%*%a$vectors,14)
```

```
(tr=sum(a$va));sum(diag(XC))
```

Python

```
np.random.seed(1010)
X=np.random.normal(25,4,size=100)
X.shape=25,4
XC=np.corrcoef(X.T)
w,v=np.linalg.eig(XC)#特征值及向量
np.round(v.T.dot(v),14)
print(w,"\n",v)
print (np.sum(w))
print (np.sum(np.diag(XC)))
```

以下代码做上面产生的矩阵 $\boldsymbol{X}$ 的奇异值分解 $\boldsymbol{X}=\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^{\top}$, 这里的 $\boldsymbol{U}$ 为 25×4 维, $\boldsymbol{U}$ 为 4×4 维; 查看 $\boldsymbol{U}$ 和 $\boldsymbol{V}$ 的正交性; 验证分解的正确性.

R

```
s=svd(X)
s$u
s$d #D的对角线元素
s$v
round(t(s$u)%*%s$u,14)
round(t(s$v)%*%s$v,14)
round(s$u%*%diag(s$d)%*%t(s$v)-X,10)
sqrt(eigen(t(X)%*%X)$va)#等于s$d
```

Python

```
u,d,v=np.linalg.svd(X,full_matrices=False)
print (u,"\n",d,"\n",v)
np.round(np.dot(u.T,u),14)
np.round(np.dot(v.T,v),14)
np.round(np.dot(np.dot(u,np.diag(d)),v)-X)
eva,eve=np.linalg.eig(np.dot(X.T,X))
print(np.sqrt(eva),d)
```

以下代码为 QR 分解和 Cholesky 分解:

- 对 4×3 矩阵 $\boldsymbol{B}$ 的 QR 分解 (在 R 中可以通过代码 `b=qr(B)` 实行, 但这样得到的是紧密形式, 还要用以下所述函数展开, 在 Python 中没有这个问题).
- 计算矩阵 $\boldsymbol{B}$ 的秩 (等于 3).
- 计算 Cholesky 分解, 我们用 $\boldsymbol{X}$ 的相关阵作为分解目标的对称矩阵, 即 $\mathrm{Cor}(\boldsymbol{X}) = \boldsymbol{L}\boldsymbol{L}^{\top}$. R 软件得到的是 $\boldsymbol{L}^{\top}$, 而 Python 得到的是 $\boldsymbol{L}$, 可以验证 $\boldsymbol{L}\boldsymbol{L}^{\top} = \mathrm{Cov}(\boldsymbol{X})$.

R

```
b=qr(B)
qr.Q(b)#Q矩阵
qr.R(b)#R矩阵
b$rank #B的秩
CXC=chol(XC)
round(t(CXC)%*%CXC-XC,14)
```

Python

```
Q,R=np.linalg.qr(B)
print(Q)
print(R)
np.linalg.matrix_rank(B)#B的秩
CXC=np.linalg.cholesky(XC)
print(np.round(np.dot(CXC,CXC.T)-XC,15))
```

2.6 习 题

1. 用 R 或 Python 代码随机产生一个 4×5 矩阵 $\boldsymbol{A}$, 并且使它在第二行及第二列处分块:

$$\boldsymbol{A} = \begin{bmatrix} \boldsymbol{M}_{11} & \boldsymbol{M}_{12} \\ \boldsymbol{M}_{21} & \boldsymbol{M}_{22} \end{bmatrix}.$$

(1) 从数值上验证:

$$\boldsymbol{A}^2 = \begin{bmatrix} \boldsymbol{M}_{11} & \boldsymbol{M}_{12} \\ \boldsymbol{M}_{21} & \boldsymbol{M}_{22} \end{bmatrix} \begin{bmatrix} \boldsymbol{M}_{11} & \boldsymbol{M}_{12} \\ \boldsymbol{M}_{21} & \boldsymbol{M}_{22} \end{bmatrix} = \begin{bmatrix} \boldsymbol{M}_{11}^2 + \boldsymbol{M}_{12}\boldsymbol{M}_{21} & \boldsymbol{M}_{11}\boldsymbol{M}_{12} + \boldsymbol{M}_{12}\boldsymbol{M}_{22} \\ \boldsymbol{M}_{21}\boldsymbol{M}_{11} + \boldsymbol{M}_{22}\boldsymbol{M}_{21} & \boldsymbol{M}_{21}\boldsymbol{M}_{12} + \boldsymbol{M}_{22}^2 \end{bmatrix}.$$

(2) 求 $\boldsymbol{A}$ 的广义逆.

(3) 求 $\boldsymbol{A}\boldsymbol{A}^{\top}$ 的行列式.

(4) 求 $\boldsymbol{A}^{\top}\boldsymbol{A}$ 的特征值和特征向量. 有几个非零特征值? 在计算机浮点运算结果中, 什么样的数值被认为是零?

(5) 求 $\boldsymbol{A}$ 的奇异值分解. 奇异值和上面的特征值有什么关系? 奇异值分解的两个向量矩阵和上面的特征向量有什么关系?

2. 用 R 或 Python 代码随机产生两个 2×2 矩阵 $\boldsymbol{A}$ 和 $\boldsymbol{B}$, 求 $\boldsymbol{A} \otimes \boldsymbol{B}$.
3. 用 R 或 Python 代码随机产生一个 20×5 矩阵 $\boldsymbol{X}$, 数值上验证矩阵 $\boldsymbol{P} = \boldsymbol{X}(\boldsymbol{X}^\top \boldsymbol{X})^{-1}\boldsymbol{X}^\top$ 为幂等矩阵.
4. 用 R 或 Python 代码随机产生一个 20×1 向量 $\boldsymbol{y}$, 把 $\boldsymbol{y}$ 分解成两个正交的向量, 并数值验证正交性. (提示: 随机产生一个 20×5 矩阵 $\boldsymbol{X}$, 形成投影矩阵 $\boldsymbol{P} = \boldsymbol{X}(\boldsymbol{X}^\top \boldsymbol{X})^{-1}\boldsymbol{X}^\top$, 验证 $\boldsymbol{P}\boldsymbol{y}$ 和 $(\boldsymbol{I} - \boldsymbol{P})\boldsymbol{y}$ 的正交性. 单位阵可用代码 `diag(20)` 产生.)
5. 用 R 或 Python 代码随机产生一个 20×5 矩阵 $\boldsymbol{X}$, 求 $\boldsymbol{X}^\top \boldsymbol{X}$ 的 Cholesky 分解.
6. 用 R 或 Python 代码随机产生一个 20×5 矩阵 $\boldsymbol{X}$, 求 $\boldsymbol{X}^\top \boldsymbol{X}$ 的 QR 分解.
7. 构造一个实对称正定矩阵 $\boldsymbol{A}$, 通过特征值分解或奇异值分解求 $\boldsymbol{A}^{-1/2}$.

第 3 章　回 归

回归分析属于统计学习中的有指导 (或有监督) 学习. 有指导学习名称中的 "指导" 意味着有一个变量 (即因变量) 作为目标, 使得训练数据集有一个指导性的目标. 然后, 用训练集的自变量和因变量 "学习"(或训练) 得出一个模型, 并且用该模型对新数据或者测试集中的因变量的值做预测. 当因变量是数量变量时, 称为**回归** (regression), 而当因变量是分类变量 (定性变量) 时, 称为**分类** (classification),

基于最小二乘法的线性回归分析及其延伸和扩展是经典统计中应用最广的一个方向. 最小二乘法出现于 100 多年前, 而且一直在发展, 但目前一些经典线性回归课程有其致命的弱点. 类似于经典数理统计的其他方向, 线性回归的主题内容是数学家在前计算机时代发展的. 由于数据量小, 计算能力缺乏 (全部靠手工计算), 为了评价回归结果的性质, 经典线性回归教程中充满了对数据分布及数据结构的主观数学假定. 其论述完全是数学式的, 包含大量的定义、引理、定理、推论等各种证明推导过程. 这种思维方式是**模型驱动的** (model driven), 和数据所代表的具体问题基本无关.

模型驱动的思维方式在纯粹数学领域内是无可非议的, 但对于旨在解决各个应用领域问题的统计来说, 模型驱动会造成误导. 作为数据科学的统计应该是问题驱动或数据驱动 (problem driven 或 data driven), 而不是模型驱动. 以模型驱动为主导的 "统计研究" 没有前途.

除了基于数学公式的经典回归之外, 有大量机器学习回归方法, 其中有许多方法的预测精度及可解释性都大大超过经典回归. 由于各种回归分析一般会在单独的课程中讲述, 这里仅简单介绍一下经典回归的基本概念, 并且通过例子和几个机器学习回归方法做一比较. 至于包括诸如机器学习在内的各种回归方法的细节, 请参看吴喜之的《应用回归及分类——基于 R》(2016).

3.1　经典回归模型基本要素

这里, 我们通过例子对线性回归做一回顾. 先看一个经典例子.

例 3.1. 波士顿住房数据 (BostonHousing2.csv)　这是一个非常经典的数据, 为 1970 年人口普查的 506 个人口普查区住房数据.[1]　该数据包含在程序包 `mlbench`[2] 中, 名为 `BostonHousing`, 包含 Harrison 和 Rubinfeld(1979) 的原始数据, 而带有额外空间信息的数据 `BostonHousing2` 是校正版本.

[1] Newman, D.J. & Hettich, S. & Blake, C.L. & Merz, C.J. (1998). UCI Repository of machine learning databases [http://www.ics.uci.edu/~mlearn/MLRepository.html]. Irvine, CA: University of California, Department of Information and Computer Science.

[2] Friedrich Leisch & Evgenia Dimitriadou (2010). mlbench: Machine Learning Benchmark Problems. R package version 2.1-1.

原始数据有 14 个变量的 506 个观察值, 其中, medv(自住房屋房价中位数, 单位: 千美元) 是原始的目标变量, 其他变量包括: crim (城镇的人均犯罪率)、zn (占地面积超过 25000 平方英尺的住宅用地的比例)、indus (每个镇的非零售业务比例, 单位: 英亩)、chas (有关查尔斯河的虚拟变量, 如果挨着河为 1, 否则为 0)、nox (一氧化氮浓度, 单位: ppm)、rm (平均每间住房的房间数量)、age (1940 年以前建成的自住单位的房龄比例)、dis (五个波士顿就业中心的加权距离)、rad (高速公路的可达性指数)、tax (每万美元全价物业值的财产税率)、ptratio (城镇学生与教师的比例)、b ($= 1000(B-0.63)^2$, 其中的 B 是城镇黑人的比例)、lstat (低收入人口比例); 更正过的数据集有以下附加变量: cmedv (修正了的自住房价中位数, 单位: 千美元)、town (镇名称)、tract (人口普查区)、lon (人口普查区的经度)、lat (人口普查区的纬度).

我们将用 cmedv (修正了的自住房屋房价中位数) 作为因变量, 舍弃原来的 medv, 而其他多数变量作为自变量 (除了 town(镇名称), tract(人口普查区), lon(人口普查区的经度), lat(人口普查区的纬度) 等变量).

该数据有一个用哑元 (0-1) 表示的定性变量 (chas), 在 R 中, 哑元表示的定性变量在处理时需要用函数 `factor` 做标记 ("因子化"). 比如, 在做回归之前使用下面的语句来确定, 否则会被计算机当成数量.

```
w=read.csv("BostonHousing2.csv")[,-c(1:5)]
w$chas=factor(w$chas)
```

但是, 由于变量 chas 只有两个水平, 因此对回归没有影响, 我们不做处理. 但对于有超过两个水平的定性自变量的回归, 则在 R 中一定要因子化, 在 Python 中一定要转换成与水平数目相适应的若干 0-1 哑元变量, 并施加约束条件.

3.1.1 描述数据

对于例3.1波士顿住房数据, 需要对数据做出直观描述, 以指导我们的建模. 最简单的描述是计算线性相关系数及产生各对变量的散点图. 图3.1表示该数据除定性变量 chas 之外的变量之间的散点图 (下三角)、变量本身的密度曲线 (对角线) 及线性相关系数 (上三角).

产生该图的代码为:

```
w=read.csv("BostonHousing2.csv")[,-c(1:5)]
library(GGally)
ggpairs(w[,-5])
```

还可以输出各个变量的汇总, 下面是代码及输出:

```
> summary(w)
   cmedv            crim                zn              indus        chas
Min.   : 5.00   Min.   : 0.00632   Min.   :  0.00   Min.   : 0.46   0:471
1st Qu.:17.02   1st Qu.: 0.08204   1st Qu.:  0.00   1st Qu.: 5.19   1: 35
Median :21.20   Median : 0.25651   Median :  0.00   Median : 9.69
Mean   :22.53   Mean   : 3.61352   Mean   : 11.36   Mean   :11.14
3rd Qu.:25.00   3rd Qu.: 3.67708   3rd Qu.: 12.50   3rd Qu.:18.10
```

```
Max.   :50.00   Max.   :88.97620   Max.   :100.00   Max.   :27.74
     nox               rm               age              dis              rad
Min.   :0.3850   Min.   :3.561   Min.   :  2.90   Min.   : 1.130   Min.   : 1.000
1st Qu.:0.4490   1st Qu.:5.886   1st Qu.: 45.02   1st Qu.: 2.100   1st Qu.: 4.000
Median :0.5380   Median :6.208   Median : 77.50   Median : 3.207   Median : 5.000
Mean   :0.5547   Mean   :6.285   Mean   : 68.57   Mean   : 3.795   Mean   : 9.549
3rd Qu.:0.6240   3rd Qu.:6.623   3rd Qu.: 94.08   3rd Qu.: 5.188   3rd Qu.:24.000
Max.   :0.8710   Max.   :8.780   Max.   :100.00   Max.   :12.127   Max.   :24.000
      tax           ptratio            b               lstat
Min.   :187.0   Min.   :12.60   Min.   :  0.32   Min.   : 1.73
1st Qu.:279.0   1st Qu.:17.40   1st Qu.:375.38   1st Qu.: 6.95
Median :330.0   Median :19.05   Median :391.44   Median :11.36
Mean   :408.2   Mean   :18.46   Mean   :356.67   Mean   :12.65
3rd Qu.:666.0   3rd Qu.:20.20   3rd Qu.:396.23   3rd Qu.:16.95
Max.   :711.0   Max.   :22.00   Max.   :396.90   Max.   :37.97
```

由于在做这个汇总之前对定性变量 chas 做了因子化, 输出中就只有其各个水平的数目, 而不像其他变量那样被当成连续变量而给出均值、极大极小值、中位数及四分位点.

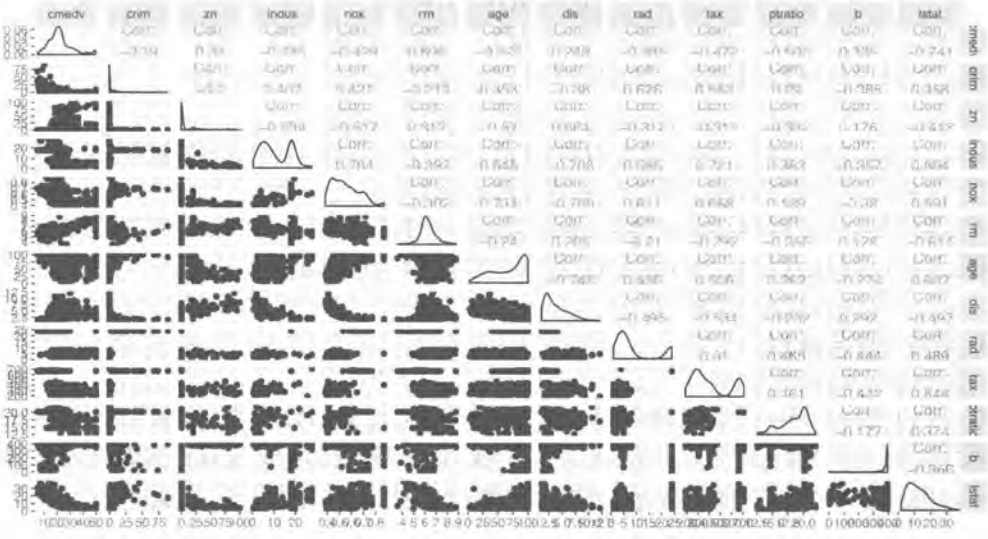

图 3.1 例3.1波士顿住房数据的变量相关图

3.1.2 线性回归模型

回归模型表现了因变量和自变量之间的关系, 所谓线性回归就是把因变量**假定**为自变量的线性组合形式, 也称为线性模型. 多个自变量的回归称为**多重回归**. 如果自变量有 p 个 (对于例3.1波士顿住房数据, $p = 13$), 线性回归模型的形式为:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \epsilon, \tag{3.1.1}$$

这里, ϵ 为误差项, 凡是因变量和自变量之间的线性关系解释不了的部分都属于误差. 如果有 n 个样本 (对于例3.1, $n=506$), 则线性模型的样本形式为:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \epsilon_i, \quad i = 1, 2, \ldots, n. \tag{3.1.2}$$

如果记

$$\boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}; \ \boldsymbol{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}; \ \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}; \ \boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix},$$

则矩阵形式的式 (3.1.2) 为:

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}. \tag{3.1.3}$$

实际上, 一般的回归模型可以写成

$$\boldsymbol{y} = \mu(\boldsymbol{X}, \boldsymbol{\beta}, \boldsymbol{\epsilon});$$

如果误差项假定是可加的, 则上述模型可写为:

$$\boldsymbol{y} = \mu(\boldsymbol{X}, \boldsymbol{\beta}) + \boldsymbol{\epsilon}.$$

再假定线性模型, 有 $\mu(\boldsymbol{X}, \boldsymbol{\beta}) = \boldsymbol{X}\boldsymbol{\beta}$, 就是式 (3.1.3) 的形式. 人们选取线性模型的原因主要是最简单直观, 而且容易计算. 其实, 现实问题很少是线性的.

3.1.3 最小二乘回归

有了数据 $\boldsymbol{y}$ 和 $\boldsymbol{X}$, 希望得到参数 $\boldsymbol{\beta}$ 的估计值 $\hat{\boldsymbol{\beta}}$. 只要估计了参数, 模型形式就确定了. 但是用什么标准来做估计呢? 人们希望估计值 $\hat{\boldsymbol{\beta}}$ 能够使得用模型得到的因变量的**拟合值** (fitted value) $\hat{\boldsymbol{y}} = \boldsymbol{X}\hat{\boldsymbol{\beta}}$ 与数据中原始的因变量值 $\boldsymbol{y}$ 尽可能地接近. 什么叫作 "接近" 呢? 这自然是一个类似于距离的概念. 从另一个角度来说, $\hat{\boldsymbol{y}}$ 和 $\boldsymbol{y}$ 离得越远, 损失就越大. 关于如何度量 "损失", 不同的人有不同的看法, 这就是如何定义**损失函数** (loss function) 的问题了. 用何种损失函数不是数学推导出来的, 而是主观选择的结果. 选择不同的损失函数会导致不同的估计值及不同的模型.

损失函数

理论上, 损失函数有无穷多种. 为了方便描述, 我们先定义回归中的**残差** (residual) 为因变量值及其拟合值之间的差: $\boldsymbol{e} = \boldsymbol{y} - \hat{\boldsymbol{y}}$ 或 $e_i = y_i - \hat{y}_i$ $(i = 1, 2, \ldots, n)$.

在第 i 个观测值处的损失函数可以定义为某个残差的函数 $\rho(e_i) = \rho(y_i - \hat{y}_i)$. 当然, 在

独立样本的**假定**下, 我们可以**主观选择**总的损失为单独观测值损失之和:

$$\sum_{i=1}^{n} \rho(y_i - \hat{y}_i) = \sum_{i=1}^{n} \rho(e_i).$$

下面的问题就是选取 $\rho()$ 的形式, 这不是数学问题, 也不是科学问题, 而是工程问题. 你可以选择绝对值, 也可以选择绝对值的某次方, 比如对于线性回归问题式 (3.1.2) 来说:

- 当 ρ 为绝对值函数 $\rho(u) = |u|$ 时, 回归称**最小一乘回归**, 损失为:

$$\sum_{i=1}^{n} e_i = \sum_{i=1}^{n} |y_i - \hat{y}_i|.$$

- 当 ρ 为平方函数 $\rho(u) = u^2$ 时, 回归称**最小二乘回归**, 损失为:

$$\sum_{i=1}^{n} e_i^2 = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2;$$

- 当 ρ 为函数 $\rho(u) = u(\tau - I(u < 0))$ 时 (这里 $I(u < 0)$ 当 $u < 0$ 时等于 1, 否则为 0, 而 τ 为在 $(0, 1)$ 区间的实数), 回归称为**分位数回归**, 损失为:

$$\sum_{i=1}^{n} e_i(\tau - I(e_i < 0)) = \sum_{i=1}^{n} (y_i - \hat{y}_i)\left[\tau - I\left(y_i - \hat{y}_i < 0\right)\right],$$

图3.2为三种损失函数图: 绝对值 (左)、二次函数 (中)、$\tau = 0.3$ 的分位数损失函数 (右).

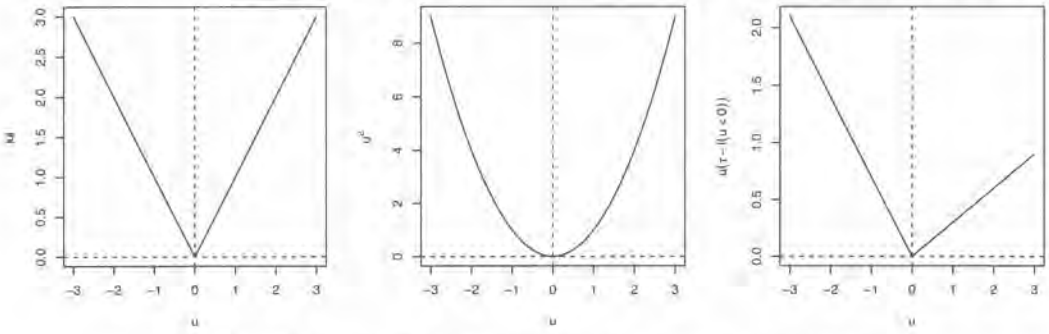

图 3.2 三种损失函数: 绝对值 (左)、二次函数 (中)、分位数损失函数 (右)

图3.2中前两种回归是对称的损失函数, 而最后一种是非对称的.

注意

实际生活中的损失有很多是非对称的, 比如一个木制房梁, 做完后长了一厘米和短了一厘米的损失很不一样, 买东西称重时多称了和少称了的损失也不是对称的, 把癌症病人误诊成没病和把健康的人误诊为有病的损失也不一样. 但人们为什么主要讨论对称的, 而且是最小二乘回归呢? 这主要是因为二次函数的推导 (比如求导数等) 及计算相对简单, 而且有一些 "漂亮" 的数学结论. 在一百年前数据量不大时, 用手工计算也可以得出最小二乘回归的结果, 而其他两种回归只有在计算机时代才能广泛应用. 损失函数不是推导出来的, 而是根据目标及回归的可行性主观选择的.

除了上面的损失函数，最小二乘回归还有很多延伸，对模型 (3.1.1) 最小二乘回归的参数估计满足

$$\hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j \right)^2 ,$$

公式中的 arg 是英文 **argument**(变元) 的缩写. 而符号 $\arg\min_k f(k)$ 为 "使得函数 $f(k)$ 达到最小值的变元 k 的值", 类似地有符号 $\arg\max_k f(k)$.

下面的岭回归和 lasso 回归 (本书不做详细介绍) 就是针对自变量有些相关时 (称为多重共线性) 所使用的带有惩罚项的损失函数:

- 岭回归的参数估计满足

$$\hat{\beta}^{(\mathbf{ridge})} = \arg\min_{\beta} \sum_{i=1}^{n} \left[\left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j \right)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \right] ,$$

- lasso 回归的参数估计满足

$$\hat{\beta}^{(\mathbf{lasso})} = \arg\min_{\beta} \sum_{i=1}^{n} \left[\left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j \right)^2 + \lambda \sum_{j=1}^{p} |\beta_j| \right] .$$

3.1.4 最小二乘回归参数的估计

最小二乘回归往往用图3.3来描述, 那里的 $\boldsymbol{e} = \boldsymbol{y} - \hat{\boldsymbol{y}}$ 就是原始的 $\boldsymbol{y}$ 投影到 $\boldsymbol{X}$ 张成的空间 (图中画成 (超) 平面的样子) 上的 $\hat{\boldsymbol{y}}$ 的残差, 该图能够使得下面的过程更容易理解.

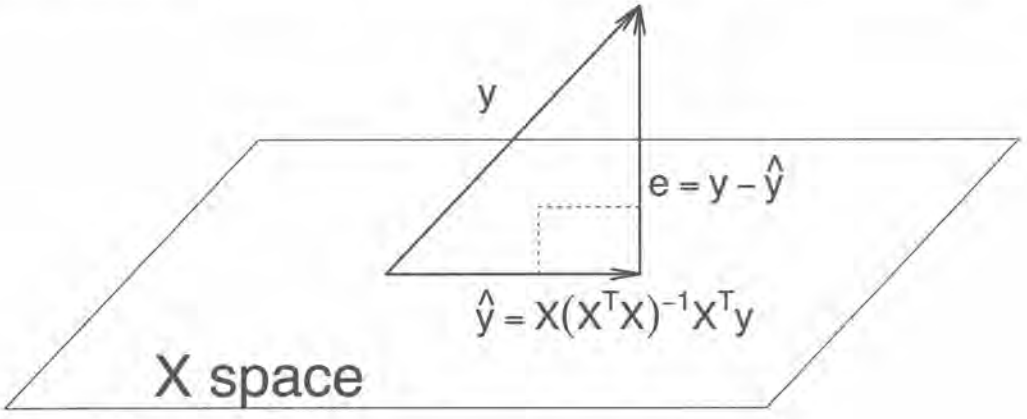

图 3.3 回归投影示意图: $\boldsymbol{y} = \hat{\boldsymbol{y}} + \boldsymbol{e}$ 及 $\boldsymbol{e} \perp \boldsymbol{y}$

最小二乘估计就是找到使得残差平方和最小的参数, 即

$$\hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j \right)^2 = \arg\min_{\beta} \left[(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta})^{\top} (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}) \right] .$$

关于 $\boldsymbol{\beta}$ 求偏导数, 解方程

$$\frac{\partial\left[(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\beta})^{\top}(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\beta})\right]}{\partial\boldsymbol{\beta}}=\frac{\partial(\boldsymbol{y}^{\top}\boldsymbol{y}-2\boldsymbol{y}^{\top}\boldsymbol{X}\boldsymbol{\beta}+\boldsymbol{\beta}^{\top}\boldsymbol{X}^{\top}\boldsymbol{X}\boldsymbol{\beta})}{\partial\boldsymbol{\beta}}=-2\boldsymbol{X}^{\top}\boldsymbol{y}+2\boldsymbol{X}^{\top}\boldsymbol{X}\boldsymbol{\beta}=0$$

可以得到 (见例2.1)

$$\hat{\boldsymbol{\beta}}=(\boldsymbol{X}^{\top}\boldsymbol{X})^{-1}\boldsymbol{X}^{\top}\boldsymbol{y}.$$

因变量的拟合值为:

$$\hat{\boldsymbol{y}}=\boldsymbol{X}\hat{\boldsymbol{\beta}}=\boldsymbol{X}(\boldsymbol{X}^{\top}\boldsymbol{X})^{-1}\boldsymbol{X}^{\top}\boldsymbol{y}, \tag{3.1.4}$$

这里, 矩阵 $\boldsymbol{P}\equiv\boldsymbol{X}(\boldsymbol{X}^{\top}\boldsymbol{X})^{-1}\boldsymbol{X}^{\top}$ 称为**预测矩阵** (prediction matrix) 或者**帽子矩阵** (hat matrix), 称为后者是因为它作用到 $\boldsymbol{y}$, 使其 "戴了帽子"(变成 $\hat{\boldsymbol{y}}$).[3] 预测矩阵 $\boldsymbol{P}$ 把任何向量, 诸如 $\boldsymbol{y}$, 投影到由 $\boldsymbol{X}$ 张成的空间之中, 因此有 $\boldsymbol{P}\boldsymbol{y}=\hat{\boldsymbol{y}}$, 显然,

$$\begin{aligned}\boldsymbol{P}\boldsymbol{e}=&\boldsymbol{P}(\boldsymbol{y}-\hat{\boldsymbol{y}})\\=&\boldsymbol{P}(\boldsymbol{y}-\boldsymbol{X}(\boldsymbol{X}^{\top}\boldsymbol{X})^{-1}\boldsymbol{X}^{\top}\boldsymbol{y})\\=&\boldsymbol{P}\boldsymbol{y}-\boldsymbol{X}(\boldsymbol{X}^{\top}\boldsymbol{X})^{-1}\boldsymbol{X}^{\top}\boldsymbol{X}(\boldsymbol{X}^{\top}\boldsymbol{X})^{-1}\boldsymbol{X}^{\top}\boldsymbol{y}\\=&\boldsymbol{0},\end{aligned}$$

也就是说, $\boldsymbol{e}$ 到 $\boldsymbol{X}$ 张成的平面的投影为 $\boldsymbol{0}$, 即残差 $\boldsymbol{e}$ 和 $\boldsymbol{X}$ 张成的平面垂直, 当然也有 $\boldsymbol{e}\perp\boldsymbol{y}$. 图3.3说明了这些关系 ($\boldsymbol{y}=\hat{\boldsymbol{y}}+\boldsymbol{e}$ 及 $\boldsymbol{e}\perp\boldsymbol{y}$), 图中的 $\boldsymbol{y}$ 是因变量向量, $\hat{\boldsymbol{y}}$ 是因变量 $\boldsymbol{y}$ 向 $\boldsymbol{X}$ 所张成的空间的投影 ($\hat{\boldsymbol{y}}=\boldsymbol{P}\boldsymbol{y}$), 而 $\boldsymbol{e}=(\boldsymbol{I}-\boldsymbol{P})\boldsymbol{y}$ 为残差, 它和 $\boldsymbol{X}$ 所张成的空间正交, 因而也和 $\hat{\boldsymbol{y}}$ 正交.

例3.1波士顿住房数据的最小二乘拟合的系数估计及拟合的模型

得到例3.1的线性最小二乘回归系数估计的代码及结果如下:

```
> a=lm(cmedv~.,w);a$coefficients
(Intercept)    crim        zn     indus     chas1       nox        rm
     0.3637 -0.1062   0.04772   0.02325     2.692   -0.1774     3.789
      age      dis       rad       tax   ptratio         b     lstat
0.0005749  -1.502    0.3038  -0.01270   -0.9239  0.009228   -0.5307
```

根据上面计算的输出, 该线性模型

$$\hat{y}=\hat{\beta}_0+\hat{\beta}_1x_1+\cdots+\hat{\beta}_{13}x_{13}$$

[3] $\boldsymbol{P}$ 也称为投影矩阵, 除了记号 "$\boldsymbol{P}$", 它也被各种文献记为 "$\boldsymbol{H}$" 或 "$\boldsymbol{M}$" 等.

的线性最小二乘回归系数估计为:

$$\hat{\beta}_0 = 0.3637;\ \hat{\beta}_1 = -0.1062;$$
$$\hat{\beta}_2 = 0.04772;\ \hat{\beta}_3 = 0.02325;$$
$$\hat{\beta}_4 = 2.692;\ \hat{\beta}_5 = -0.1774;$$
$$\hat{\beta}_6 = 3.789;\ \hat{\beta}_7 = 0.0005749;$$
$$\hat{\beta}_8 = -1.502;\ \hat{\beta}_9 = 0.3038;$$
$$\hat{\beta}_{10} = -0.01270;\ \hat{\beta}_{11} = -0.9239;$$
$$\hat{\beta}_{12} = 0.009228;\ \hat{\beta}_{13} = -0.5307.$$

根据这个模型, 人们可以从任何自变量 $x_1, x_2, \ldots, x_{13}$ 的观测值得到因变量 y 的预测值.

到此为止, 我们已经完整地介绍了最小二乘回归的全过程: 从假定的线性模型和选择的二次损失函数到具体的参数估计. 但是, 似乎还少了些什么, 这就是如何评价最小二乘回归结果的好坏. 不同的视角产生不同的方法及结论. 下面首先介绍在计算机时代才能实现的基于预测精度的交叉验证方法.

3.2 交叉验证

我们用一个数据建立了模型, 该模型可能对这部分数据很合适, 但它是不是适合其他数据, 则必须由没有参加建模的数据来验证, 这就是交叉验证的思想. 具体地说, 所谓交叉验证就是用一部分数据建立模型, 这部分数据集合称为**训练集** (training set); 用另一部分数据检验该模型对这部分数据的预测精度, 这部分数据集合称为**测试集** (testing set). 交叉验证中预测精度不好的模型肯定不是好模型.

训练集和测试集的确定方法有很多. 常用的一种就是$\boldsymbol{m}$ **折交叉验证** (m-fold cross validation), 它把数据随机分成 m 份 (折), 各折的观测值数目尽量相等. 然后做 m 次验证, 每次轮流用 1 份数据作为测试集, 其余 $m-1$ 份作为训练集, 用训练集建模, 再用根据训练集所建立的模型拟合测试集. 这样, 在 m 次验证之后得到 m 次预测误差, 最终得到交叉验证的平均误差.

回归交叉验证的误差通常基于残差平方和 (SSE) 或均方误差 (MSE). 记训练集的自变量矩阵为 $\boldsymbol{X}_{train}$, 训练集的因变量为 $\boldsymbol{y}_{train}$, 记测试集的自变量矩阵为 $\boldsymbol{X}_{test}$, 测试集的因变量为 $\boldsymbol{y}_{test}$. 由训练集拟合的系数估计为:

$$\hat{\beta} = (\boldsymbol{X}_{train}^{\top}\boldsymbol{X}_{train})^{-1}\boldsymbol{X}_{train}^{\top}\boldsymbol{y}_{train},$$

这时, 基于测试集 $\boldsymbol{X}_{test}$ 的预测值为:

$$\hat{\boldsymbol{y}}_{test} = \boldsymbol{X}_{test}\hat{\beta},$$

这时, 交叉验证的残差平方和为 (下面假定进入测试集的观测值总数为 n[4]):

$$SSE_{cv} = \sum_{i=1}^{n} (y_{test\,i} - \hat{y}_{test\,i})^2 ,$$

而交叉验证的均方误差为:

$$MSE_{cv} = \frac{1}{n} SSE_{cv} = \frac{1}{n} \sum_{i}^{n} (y_{test\,i} - \hat{y}_{test\,i})^2 .$$

如果没有模型, 最朴素的预测就是均值, 那时, 可用均值 $\bar{\boldsymbol{y}}_{test}$ 来代替上面 SSE_{cv} 或 MSE_{cv} 的 $\hat{\boldsymbol{y}}_{test}$(分别记为 SST_{cv} 和 MST_{cv}).

我们可以用 SST_{cv} 或 MST_{cv} 作为除数来将 SSE_{cv} 或 MSE_{cv} 标准化, 得到**标准化均方误差** (normalized mean square error, NMSE)

$$NMSE_{cv} = \frac{SSE_{cv}}{SST_{cv}} = \frac{MSE_{cv}}{\mathrm{MST}_{cv}} = \frac{\sum_{i=1}^{n} (y_{test\,i} - \hat{y}_{test\,i})^2}{\sum_{i=1}^{n} (y_{test\,i} - \bar{y}_{test})^2},$$

同样, 也可以得到另一个等价指标 R^2_{cv}:

$$R^2_{cv} = 1 - NMSE_{cv} = 1 - \frac{\sum_{i=1}^{n} (y_{test\,i} - \hat{y}_{test\,i})^2}{\sum_{i=1}^{n} (y_{test\,i} - \bar{y}_{test})^2},$$

该指标在经典回归中 (即没有测试集或训练集就是测试集时) 称为**可决系数** (coefficient of determination)(按照机器学习术语也称为记分或得分 (score)).

显然, 如果标准化均方误差 $NMSE_{cv} > 1$ 或者等价地 $R^2_{cv} < 0$, 说明有模型还不如没有模型 (用朴素的均值作为预测值). 显然, 交叉验证的这两个等价度量不仅仅适用于最小二乘回归, 而且适用于任何模型, 可以用于不同模型预测精度的比较.

对例3.1波士顿住房数据最小二乘线性回归的交叉验证

求标准化均方误差及记分, 我们做 $Z = 10$ 折交叉验证 (对于例3.1的数据, 每份 (折) 数据约 50 个观测值). 用以下函数把数据行下标随机打乱, 输出 Z 个下标集:

```
CV=function(data=w,Z=10,seed=1010){
  n=nrow(data);N=1:n;Z=10
  set.seed(seed)
```

[4]如果原数据样本量为 n, 进行 K 折交叉验证, 那么每个观测值都会轮流作为测试集成员出现一次, 这时测试集的总样本量也是 n. 如果把数据仅仅分成一个训练集和一个测试集, 那么原数据样本量和测试集的样本量就不一样了.

```
  mm=sample(rep(1:Z,ceiling(n/Z))[N])
  K=list();for(i in 1:Z)K[[i]]=N[mm==i]
  return(K)
}
```

然后用这个函数得到的 10 个下标集组成 10 个数据集, 轮流每次用 9 个数据集合并作为训练集, 再用一个作为测试集, 做十次拟合, 得到标准化均方误差 $NMSE_{cv}$ 及等价的 R^2_{cv}:

```
Z=10;mm=CV(w)
y_test=NULL->y_pred
for(i in 1:Z){
  at=lm(cmedv~.,w[-mm[[i]],])
  y_test=c(y_test,w$cmedv[mm[[i]]])
  y_pred=c(y_pred,predict(at,w[mm[[i]],]))
}
sse=sum((y_test-y_pred)^2)
sst=sum((y_test-mean(y_test))^2)
nmse=sse/sst;r2=1-nmse; print(c(nmse,r2))
```

得到 $NMSE_{cv} = 0.2739$ 及 $R^2_{cv} = 0.7261$. 如果不用交叉验证, 即全部数据既是训练集又是测试集, 则得到 $NMSE = 0.256$ 及 $R^2 = 0.744$.

注意

由于 m 折交叉验证的结果有随机性, 在观测值少的时候, 应该尽可能使得各折中诸定性自变量各个水平的搭配比较平衡, 以及使得诸定量自变量的各个分布段的搭配比较平衡. 但一般现成的 m 折交叉验证程序不保证这一点, 这是由于大多数数据是没有事先设计过的观测数据, 很难保证能够实现较完美的平衡. 观测数据不可能被人类的愿望所左右, 这就是真实的世界.

但在实践中, 人们在选择各折数据时, 至少可以使得定性因变量的各个水平尽可能平衡. 但这是分类的实践, 在回归实践中, 人们一般不按照因变量的分布来平衡数据.

3.3 经典线性回归的基本假定及根据假定所得到的结论

3.3.1 经典线性回归的数学假定

上面提到最小二乘线性回归的基本运算已经完成, 但是经典线性最小二乘回归不用交叉验证, 而是用许多假设检验和区间估计来评价模型. 但除交叉验证之外的任何经典的统计推断都离不开对数据的主观数学假定. 下面是经典线性回归的一些主观假定:

(1) 模型对于参数的**线性形式**为:

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\beta} + \boldsymbol{\epsilon} \text{ 或 } y_i = \beta_0 + \beta_1 X_{i1} + \cdots + \beta_p X_{ip} + \epsilon_i.\ i = 1, 2, \ldots, n;$$

(2) $\boldsymbol{y}$ 的所有值都是**互相独立**的, 如果把 $\boldsymbol{X}$ 看成是固定的, 则等价于 $\boldsymbol{\epsilon}$ 的所有值都是互相独立的, 这意味着没有自相关;

(3) 对于每个 $\boldsymbol{X}$ 的值, $\boldsymbol{y}$ 的分布是**正态分布**的, 如果把 $\boldsymbol{X}$ 看成是固定的, 则等价于 $\boldsymbol{\epsilon}$ 的分布是正态分布;

(4) 条件期望 $E(\boldsymbol{y}|\boldsymbol{X}) = \boldsymbol{X}\boldsymbol{\beta}$ 或 $E(\boldsymbol{\epsilon}|\boldsymbol{X}) = \boldsymbol{0}$, 协方差阵 $\mathrm{Cov}(\boldsymbol{y}|\boldsymbol{X}) = \mathrm{Cov}(\boldsymbol{\epsilon}|\boldsymbol{X}) = \sigma^2\boldsymbol{I}$, 这意味着所有观测值独立同方差 (homoscedasticity), 对于正态分布意味着所有**观测值都独立同分布** (当然误差项也独立同分布);

(5) 自变量和 $\boldsymbol{\epsilon}$ **独立**;

(6) 没有多重共线性 (multicollinearity), 即自变量相互独立, 如果把自变量看成是固定的 (非随机的), 自变量相互独立的条件应该改成$\boldsymbol{X}$ **矩阵的列向量是正交的**.

注意: 如果假定自变量是弱外生的 (weak exogeneity), 可以近似地把 $\boldsymbol{X}$ 看成是固定的, 上面的概率就不是条件概率了, 自变量和 $\boldsymbol{\epsilon}$ 独立的条件就用不着了.

当自变量被看成是固定的, 人们所说的最小二乘线性回归的**基本假定**就是一句话: **观测值独立同正态分布**. 或者用公式表示为:

$$\boldsymbol{y} \sim N\left(\boldsymbol{X}\boldsymbol{\beta}, \sigma^2\boldsymbol{I}\right), \quad \boldsymbol{\epsilon} \sim N\left(\boldsymbol{0}, \sigma^2\boldsymbol{I}\right).$$

注意

(1) 所有上述假定都无法证明满足, 只能够证伪.

(2) 把因变量和自变量的关系假定为线性关系的主要原因是当年人们的能力只能勉强应对线性模型所带来的数学及计算等一些问题.

(3) 把样本假定为独立同正态分布, 这大多不合乎实际, 但基于与上一款同样的理由, 这种假定在数学上是比较方便的. 这种分布假定和最小二乘回归方法本身无关, 仅仅和与回归相关的各种推断有关, 比如 t 检验及 F 检验统计量的分布、各种估计量的分布及性质等等.

没有假定则无法做任何经典统计推断, 只有基于计算机算法的机器学习存在, 才有可能摆脱主观假定的束缚.

当前面一些假定不成立时, 人们发展了一些经典方法来处理. 比如, 在独立同方差不能满足的情况下, 可用加权最小二乘回归; 在具有多重共线性时, 人们用岭回归、lasso 回归及偏最小二乘回归等来处理; 对于非线性关系, 有非线性回归模型.

3.3.2 最小二乘回归参数估计量的一些性质及有关检验

如果不做任何数学假定, 也不做任何诸如假设检验或区间估计一类的推断, 而用较客观的交叉验证来审视模型, 那么这一小节的内容在实践中完全不会用到. 虽然过分专注于这些结论的推导和证明可能会对读者产生误导, 但这些内容反映了传统最小二乘线性回归所关注的一些问题.

$\hat{\boldsymbol{\beta}}$ 的性质

在关于数据独立同正态分布的基本假定下, 线性回归系数的最小二乘估计 $\hat{\boldsymbol{\beta}}$ 等于最大似然估计, 有下列性质:

- $E(\hat{\boldsymbol{\beta}}) = \boldsymbol{\beta}$;

- $\mathrm{Var}(\hat{\boldsymbol{\beta}}) = \sigma^2(\boldsymbol{X'X})^{-1}$;
- $\hat{\boldsymbol{\beta}} \sim N(\boldsymbol{\beta}, \sigma^2(\boldsymbol{X'X})^{-1})$, 如果没有前面的基本假设, 基于中心极限定理, 当样本量趋于无穷时, 这个正态分布是极限分布.

σ^2 的最大似然估计

记 $RSS = RSS(\hat{\boldsymbol{\beta}}) = (\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{\beta}})^\top(\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{\beta}}) = \boldsymbol{y}^\top\boldsymbol{y} - \hat{\boldsymbol{\beta}}\boldsymbol{X}^\top\boldsymbol{Y}$. 在关于数据独立同正态分布的基本假定下, 有

- σ^2 的最大似然估计为:

$$\hat{\sigma}^{*2} = \frac{RSS(\hat{\boldsymbol{\beta}})}{n},$$

这是有偏估计;

- 为使得其 σ^2 的估计无偏, 把 n 换成 $n-(p+1)$ 得到其无偏估计量

$$\hat{\sigma}^2 = \frac{RSS(\hat{\boldsymbol{\beta}})}{n-(p+1)}.$$

$\hat{\boldsymbol{\beta}}$ 的方差和标准误差的估计

方差 $\mathrm{Var}(\hat{\boldsymbol{\beta}})$ 的一个估计为 $\widehat{\mathrm{Var}}(\hat{\boldsymbol{\beta}}) = \hat{\sigma}^2(\boldsymbol{X'X})^{-1}$. $\widehat{\mathrm{Var}}(\hat{\boldsymbol{\beta}})$ 的第 i 个对角线元素的平方根为 $\hat{\beta}_i$ 的标准误差, 记为 $se(\hat{\beta}_i)$. 作为特例, 在只有一个自变量回归时, 斜率 β_1 的拟合值的方差估计及其标准误差分别为:

$$\widehat{\mathrm{Var}}(\hat{\beta}_1) = \frac{\hat{\sigma}^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2}; \qquad se(\hat{\beta}_1) = \frac{\hat{\sigma}}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}}.$$

关于 $H_0 : \beta_i = 0 \Leftrightarrow H_1 : \beta_i \neq 0$ 的 t 检验

在基本假定之下, 检验 $H_0 : \beta_i = \beta_i^* \Leftrightarrow H_1 : \beta_i \neq \beta_i^*$ 的检验统计量

$$t = \frac{\hat{\beta}_i - \beta_i^*}{se(\hat{\beta}_i)} \tag{3.3.1}$$

在 H_0 下具有 $n-p-1$ 个自由度的 t 分布, 由此可以得到 p 值. 人们主要关心的是 $\beta_i^* = 0$ 的情况, 即检验 $H_0 : \beta_i = 0 \Leftrightarrow H_1 : \beta_i \neq 0$ 的情况, 软件自动输出的也是这个检验的 p 值. 这时的检验统计量为:

$$t = \frac{\hat{\beta}_i}{se(\hat{\beta}_i)},$$

注意

提到检验, 必须明白下面的问题:

- **诸如式 (3.3.1) 那样的检验统计量不是推导出来的, 而是根据目的为了找到其分布而构造出来的 (或者发现的). 根据同样目的构造出来的统计量可能有多个, 无所谓好坏, 它们基于不同的**

视角，而且结果也不尽相同. 检验统计量的分布是根据对数据的分布假定推导出来的.

- 根据数学假定求出 p 值的过程是纯粹的数学过程，但之后判断是否显著这样的结论则是人为主观决策.
- 检验是否 "显著" 依赖于所谓 p 值的大小，只有 p 值很小时 (相应于零假设下的小概率事件)，才算是显著，才能够拒绝零假设. 但是，p 值小于多少才能说明 "小概率事件发生了"，以至于拒绝零假设呢？这完全没有定论. 通常所说的 "$p = 0.05$ 就算显著" 完全是误导，难道 100 个亲子鉴定中有 5 个鉴定错了是小概率事件吗？难道 100 人出门有 5 人被车撞死是小概率事件吗？如果不动脑筋，则会接受任何荒唐的事情. p 值多大算是显著必须完全依赖于问题本身的性质，这种主观决策往往会造成误导.
- 许多教科书中说，"不能拒绝零假设就接受零假设". 这种说法极其不负责任. 在拒绝零假设时，至少还给出个 p 值作为风险度量 (虽然不完美)，而 "接受零假设" 的说法则给不出任何风险 (贝叶斯统计会给出采取任何假设的犯错误概率，但也是基于数学上假定的模型). 实际上，在不能拒绝零假设时，只能说 "证据不足以拒绝零假设".
- 检验统计量的分布都是基于对数据的正态分布假定或大样本假定，而这些假定都是无法核对或证明的，设立假定本身也是人为的主观决策.
- 检验和 p 值本身的意义必须与实际问题相结合，而且 p 值也仅仅依赖于一个数据，没有人能够根据 p 值来确定模型是否具有普遍意义.

关于多自变量系数复合假设的 F 检验及方差分析表

在对数据的基本假定下，关于多自变量系数的检验为 (这里的 p 是自变量个数，不要和 p 值混淆):

$$H_0 : \beta_1 = \beta_2 = \cdots = \beta_p = 0 \Leftrightarrow H_1 : \beta_i \text{不全为 } 0, \tag{3.3.2}$$

注意，这个检验**不等于**一连串 p 个下列检验 $H_0^{(i)} : \beta_i = 0 \Leftrightarrow H_1^{(i)} : \beta_i \neq 0$. 假定对于单独一个检验 $H_0^{(i)}$ 的犯第一类错误概率为 $\alpha = P(\text{拒绝} H_0^{(i)} | H_0^{(i)} \text{为真})$，那么当所有 $H_0^{(i)}$ 都为真时，在 p 个这样的检验中至少有一个零假设被拒绝的概率为 $1 - (1-\alpha)^p$. 这就是上面复合检验的犯第一类错误的概率，它随着 p 的增加而增加. 因此，不能用个别系数的检验代替式 (3.3.2).

为了实行检验式 (3.3.2)，可以利用下面的分解:

$$\sum_{i=1}^{n}(y_i - \bar{y})^2 = \sum_{i=1}^{n}(\hat{y}_i - \bar{y})^2 + \sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \tag{3.3.3}$$

我们把式 (3.3.3) 左边称为总变差平方和 (记为 SST)，右边第一项称为源于回归的平方和 (记为 SSR)，右边第二项称为源于无法被模型解释的误差的平方和 (记为 SSE 或 RSS). 式 (3.3.3) 可以写成

$$SST = SSR + SSE,$$

由于在各种假定之下，可以表明

$$\sum_{i=1}^{n}(y_i - \bar{y})^2/\sigma^2 = SSE/\sigma^2 \sim \chi^2_{(n-p-1)} \quad \text{及} \quad \sum_{i=1}^{n}(\hat{y}_i - \bar{y})^2/\sigma^2 = SSR/\sigma^2 \sim \chi^2_p,$$

加上 SST 和 SSE 的独立性, 根据概率论, 在基本假定和复合假设式 (3.3.2)H_0 之下有

$$F = \frac{SSR/p}{SSE/(n-p-1)} \sim F_{p,n-p-1}. \tag{3.3.4}$$

因此, 可以用这个 F 统计量来对假设式 (3.3.2) 的 H_0 做检验 (获得 p 值). 当 F 统计量很大时, 则可以拒绝零假设.

这些平方和的关系能够用所谓的方差分析 (analysis of variance, ANOVA) 表 (见下表) 来汇总.

变差来源	自由度 (d.f)	平方和	均方	F 值	p 值
回归	p	SSR	$MSR = SSR/p$	$F = MSR/MSE$	$P(F > f)$
残差	$n-p-1$	SSE	$MSE = SSE/(n-p-1)$		
总变差	$n-1$	SST			

由于最后一行前两列是前面两行的总和, 在 R 的 ANOVA 输出中省略了. 当有多个变量时, R 的 ANOVA 输出中把 SSR 进一步按照自变量分解, 因此显示出这些自变量的平方和对 SSR 的贡献, 其数学细节在这里就不介绍了.

3.3.3 例3.1波士顿住房数据的最小二乘回归

对于例3.1数据做最小二乘估计, 我们完全可以用式 (3.1.4) 来写程序计算, 当然, 用下面的现成程序更方便, 一行就够了, 所有的结果都在 "a" 中 (代码中的 "cmedv~." 意味着因变量 cmedv 对所有自变量的回归).

```
a=lm(cmedv~.,w)
```

可以输出什么结果呢? 可以用 summary 函数来看汇总结果, 这种输出在各种软件中都是标准的:

```
> summary(a)
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  3.64e+01   5.06e+00    7.19  2.4e-12
crim        -1.06e-01   3.26e-02   -3.26  0.00119
zn           4.77e-02   1.36e-02    3.51  0.00049
indus        2.33e-02   6.09e-02    0.38  0.70297
chas1        2.69e+00   8.54e-01    3.15  0.00172
nox         -1.77e+01   3.79e+00   -4.69  3.6e-06
rm           3.79e+00   4.14e-01    9.15  < 2e-16
age          5.75e-04   1.31e-02    0.04  0.96499
dis         -1.50e+00   1.98e-01   -7.60  1.5e-13
rad          3.04e-01   6.58e-02    4.62  4.9e-06
tax         -1.27e-02   3.73e-03   -3.41  0.00071
ptratio     -9.24e-01   1.30e-01   -7.13  3.7e-12
b            9.23e-03   2.66e-03    3.47  0.00057
```

```
lstat       -5.31e-01   5.03e-02  -10.56  < 2e-16

Residual standard error: 4.7 on 492 degrees of freedom
Multiple R-squared:  0.744,Adjusted R-squared:  0.738
F-statistic:  110 on 13 and 492 DF,  p-value: <2e-16
```

上面的结果给出的主要内容为:

- Estimate 一列给出了参数 β_i 的估计值 $\hat{\beta}_i$.
- Std. Error 一列给出了 $\hat{\beta}_i$ 的标准误差的估计值 $se(\hat{\beta}_i)$.
- 第三列 t value 给出了前面式 (3.3.1) 提到的 t 检验统计量的实现值;
- 最后一列 Pr(>|t|) 为 p 值.

在结果中的最下面部分, 除了 R^2 和 $\bar{R}^2$, 还有一个 F 检验, 相应于检验式 (3.3.2),

上面输出的对于回归系数逐个 t 检验的结果显示, 最小的 p 值小于 2×10^{-16}(打印输出为: "< 2e-16"), 这几乎是 0, F 检验的 p 值也小于 2×10^{-16}, 说明可能有些回归系数的确显著. 关于拟合的 $R^2=0.744$, 说明拟合得还可以. 再看看方差分析表 (通过代码 anova(a)):

```
> anova(a)
Analysis of Variance Table

Response: cmedv
           Df Sum Sq Mean Sq F value  Pr(>F)
crim        1   6462    6462  292.20 < 2e-16
zn          1   3535    3535  159.84 < 2e-16
indus       1   2560    2560  115.77 < 2e-16
chas        1   1531    1531   69.23 8.7e-16
nox         1     83      83    3.76   0.053
rm          1  10930   10930  494.21 < 2e-16
age         1     90      90    4.05   0.045
dis         1   1829    1829   82.72 < 2e-16
rad         1     41      41    1.84   0.176
tax         1    344     344   15.54 9.3e-05
ptratio     1   1236    1236   55.91 3.5e-13
b           1    590     590   26.69 3.5e-07
lstat       1   2465    2465  111.48 < 2e-16
Residuals 492  10881      22
```

该方差分析表也显示出大多数变量相应的 p 值都很小 (很多几乎是 0).

3.3.4 最小二乘回归结果的解释

残差的正态性检验

另外, 前面的各种检验基于对样本和模型的各种假定, 很难核对和证明这些假定的合理性. 但似乎至少可以利用对残差的正态性检验来看是不是可以拒绝残差的正态性 (一些人认为残差是模型中误差项 ϵ 的实现值, 但这只有在真模型和假定模型一样时才对, 而且没

有人能保证这一点). 较常用的正态性检验是 Shapiro-Wilk 正态性检验, 对于例3.1波士顿住房数据回归的残差的 Shapiro-Wilk 正态性检验的结果为:

```
> shapiro.test(a$residuals)

Shapiro-Wilk normality test

data:  a$residuals
W = 0.9, p-value <2e-16
```

这次 p 值小于 2×10^{-16}, 几乎等于 0, 这说明正态性不满足. 因此, 前面的 t 检验和 F 检验都失去了理论基础, 不可信.

回归系数的解释

多重回归系数就应该按照模型本身来解释: **它们就是假定的线性模型的系数.** 任何人对这些都不会产生什么误解. 但是, 很多回归教科书却画蛇添足, 说多重回归的某一个变量系数的意义为:

在其他变量不变时, 一个变量系数为该变量增加一个单位对因变量的贡献.

这一说法实际上完全依赖于3.3.1节中的 "没有多重共线性" 的假定. 也就是说对数据加以下面的数学假定: **其他变量不变时, 一个变量可以单独变化**. 因此, **该说法是假定出来的, 和实际毫无关系, 而且在有多重共线性时是完全错误的**. 数学可以任意做出假定, 但在应用中不应该做这种假定. 人们喜欢使用该说法的可能原因是:

- 试图识别哪些变量重要 (对因变量贡献大).
- 试图说明 "线性回归的可解释性".

该说法意味着对现实世界的无知及对回归的数学原理一知半解. 对此, 人们应该思考几个问题:

(1) 多重回归的自变量或多或少相关, 有没有可能让别的变量不变, 而只有某一个自变量变化?

(2) 如果我们采用不同的变量组合, 那些系数的估计值会大相径庭, 请问那些相信该说法的人, 我们应该采取哪一个系数来解释呢? 在该说法的信徒中有多少人能够回答这个问题呢? 而且, 回归系数必定会因为加入一些新变量或减少一些变量而改变.

(3) 如果多重回归的**自变量正交 (自变量矩阵为正交矩阵), 而且回归模型不带有常数项时**, 上面的说法还有些道理. 但是, 我们首先要问, 有多少回归的自变量矩阵是正交矩阵呢? 其次, 在这种情况下, 还有没有必要做多重回归?

(4) 许多最小二乘回归的延伸, 特别是在多重共线性存在时发展的诸如前面提到的岭回归、lasso 回归等等都使得回归系数的 "可解释性" 说法完全站不住脚.

下面就例3.1的波士顿住房数据来表明该说法的荒谬.

(1) 首先这些变量都是相关的 (请看图3.4及前面的图3.1), 不可能让其他变量不变而只有一个变量自己单独变.

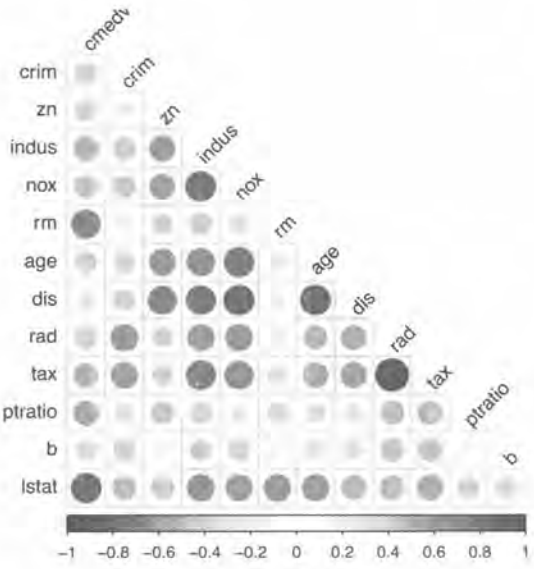

图 3.4 例3.1波士顿住房数据变量的线性相关系数展示

(2) 我们来比较一下每一个自变量单独回归时的系数估计与多重回归时该变量系数估计的不同. 图3.5中的条形图展示了每个系数的两种回归的估计值, 这里的两种回归模型均未包含常数项. 结果表明对全部自变量和单个自变量回归时系数估计之间的巨大差异(很多正负号都相反).

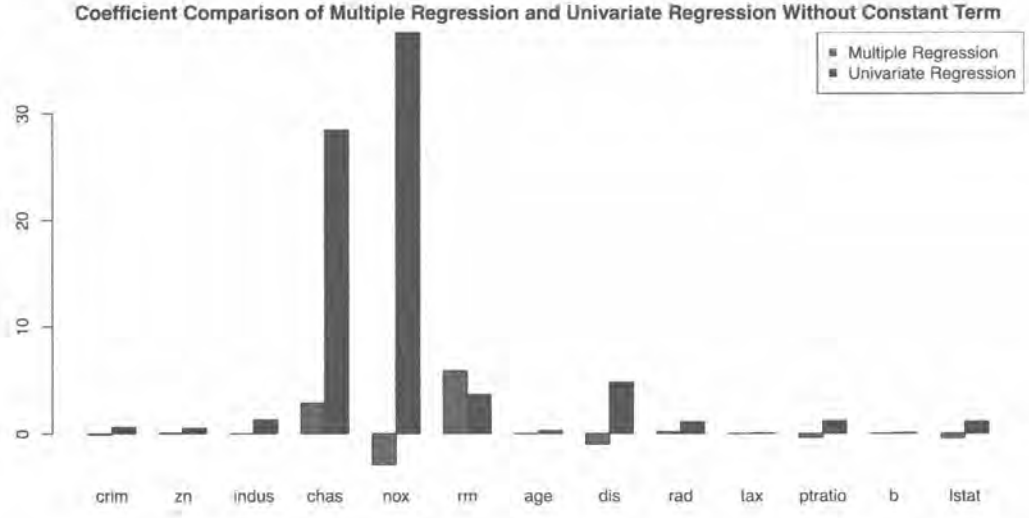

图 3.5 对例3.1波士顿住房数据做每个自变量的简单回归和多重回归系数估计的变化

人们可能会说, 图3.5的两个不那么显著的变量 chas 和 nox 的值很大, 其他变量之间可能不会有什么差距. 为此, 我们去掉这两个变量, 重新画条形图 (见图3.6), 结果显示出其他变量系数在两种回归中同样有很大差异, 而且很多正负号相反.

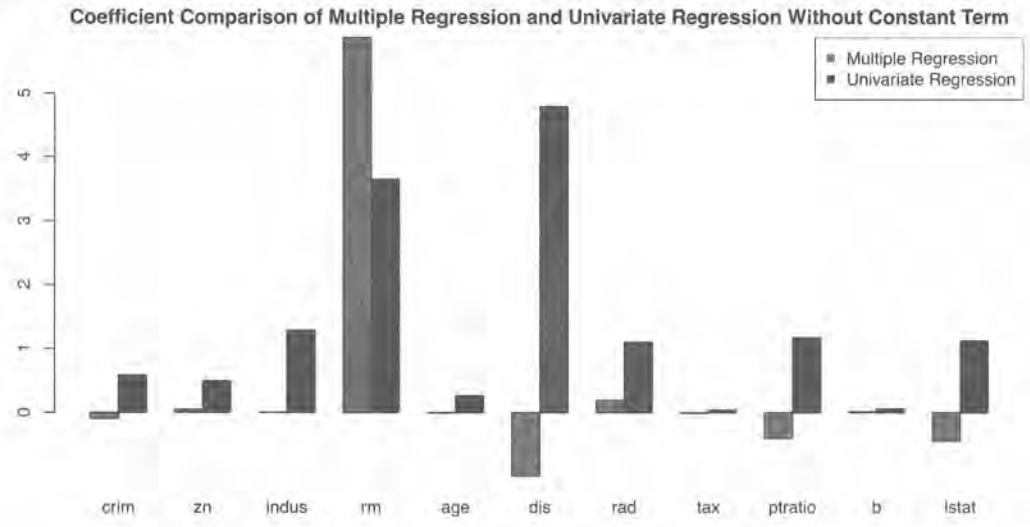

图 3.6 对例3.1的波士顿住房数据去掉变量 chas 和 nox 后每个自变量的简单回归和多重回归系数估计的变化

注意

经典多重回归模型的所谓“可解释性”，是那些不情愿接受机器学习方法的人的一个最常见的借口. 实际上，这种“可解释性”仅仅是一件皇帝的新衣而已. “在其他变量不变时，一个变量系数为该变量增加一个单位对因变量的贡献”的说法是多重回归“可解释性”的最主要的依据. 由于历史原因，目前在国内尚有一些市场，但在整个统计界，很少有人这么认识了.

对于多重回归，当自变量具有多重共线性时，单独一个变量的作用掩盖在其他变量的共同作用之中，其系数大小没有任何解释意义，根本不值得探讨.

许多机器学习回归方法的可解释性相当好，而且不对数据做任何假定.

3.3.5 有关拟合的 R^2 及 AIC

R^2 的解释

这里所说的可决系数是没有交叉验证的，即训练集和测试集就是原始数据，包含在软件的标准输出之中. 有人认为 R^2 越大模型越好，其实应该说“R^2 越接近 1，拟合得越好”，但拟合好不能说明模型就好，因为可能出现**过拟合** (overfitting). 下面举一个简单例子 (利用 R 自带的数据 cars，用停车距离对车速做回归 (dist~speed)). 我们用两个回归，一个是最小二乘回归，另一个是用连接所有点的折线作为回归曲线 (你不能说这不是回归，因为用铅笔画的一条线也可以称为回归). 前者的 $R^2 = 0.651$，而折线的 $R^2 = 1$，你能够说折线 (回归) 比最小二乘回归更有道理吗？这两个回归的拟合图见图3.7.

显然，折线回归是过拟合，它只适合于训练集，没有普遍意义. 为了避免过拟合现象，人们发明了带有惩罚项的判别准则，诸如下面的 AIC 和 BIC.

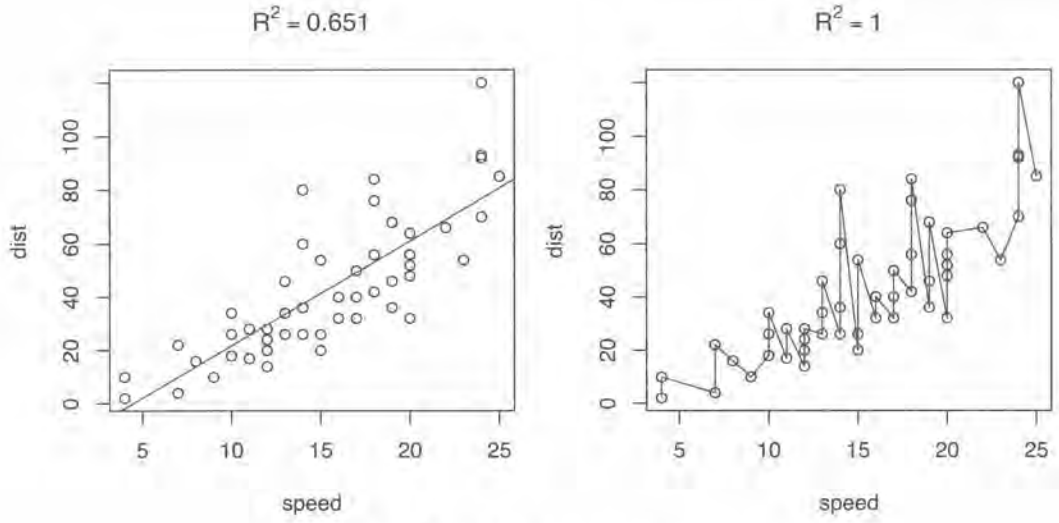

图 3.7 最小二乘回归和折线回归

AIC 和 BIC

R^2 是基于残差平方和的, 如果残差平方和为 0, 则有 $R^2 = 1$. 典型的带有惩罚项的判别准则为 AIC(Akaike information criterion) 和 BIC(Bayesian information criterion), 对线性模型 AIC 和 BIC 的定义为:

$$\begin{aligned} \text{AIC} &= 2p - 2\ln(\hat{L}); \\ \text{BIC} &= \ln(n)p - 2\ln(\hat{L}), \end{aligned}$$

式中的 p 和 n 分别是模型待估计参数的个数及样本量, 而 $\hat{L}$ 是误差项分布的 (作为参数函数的) 似然函数的最大值. 对于误差有正态分布假定的线性回归模型, 有

$$\begin{aligned} \text{AIC} =& n + n\log 2\pi + n\log(SSE/n) + 2(p+1); \\ \text{BIC} =& n + n\log 2\pi + n\log(SSE/n) + (\log n)(p+1), \end{aligned}$$

式中的 SSE 表示残差平方和, p 为待估计的参数个数. 它们的值越小越好, 对于小的 SSE 值有奖励, 对于太大参数数目 (p) 给以惩罚. 显然, 这些准则的确定存在一定的任意性和主观性, 在没有假定分布时, 这些准则很难计算出来. 因为这些准则都是人们主观创造出来的工程师式的结果, 只对某些假定的数据结构有一定的意义.

过拟合意味着模型没有普遍意义, 除了用于建模的数据, 不能适用于其他数据. 显然, 那个折线回归根本不能推广, 而诸如 R^2、AIC 和 BIC 等准则都有局限性. 前面介绍的**交叉验证**可以较客观地比较不同的模型及判断模型的好坏.

3.4 自变量有分类变量 (定性变量) 的情况 *

这部分的内容可以做参考, 也可以讲一下要义. 对于 Python 的大多数回归程序, 如果遇到字符串表示的变量水平时, 则必须转换成哑元.

分类自变量的表示方法

前面所有的回归理论涉及的都是数量变量. 我们也说过, 对于只有两个水平的哑元 (如 0-1 表示的) 分类变量, 可以不做任何处理, 不会影响结果. 但是, 对于有多个水平的定性自变量, 无论是用哑元表示还是用字符表示, 都必须予以注意.

下面用一个很小的人造数据来说明. 这个数据有两种形式, 分别存在文件 Artif0.csv 及文件 Artif1.csv 中, 其中:

(1) Artif0.csv 有三个分类变量: Age (水平为 Middle, Old, Young), Edu (水平为: Coll, Ele, Grad, Hs) 和 Sex (水平为: M,F), 还有一个可以当成因变量的数量变量 Income.

(2) Artif1.csv 和 Artif0.csv 一样, 只不过三个分类变量的水平用哑元 (自然数) 表示.

下面是读入并显示数据输出, 对于数据 Artif0.csv:

```
> (w=read.csv("Artif0.csv"))
      Age  Edu Income Sex
1     Old Grad 520000   F
2  Middle Coll  51000   F
3   Young   Hs   1200   F
4     Old  Ele   1500   F
5  Middle Grad 200000   M
6   Young Coll  15000   M
7     Old   Hs   2000   M
8  Middle  Ele   1100   F
9   Young Grad 150000   M
10    Old Coll  10000   F
11 Middle   Hs   3000   F
12  Young  Ele    500   M
```

对于数据 Artif1.csv:

```
> (w1=read.csv("Artif1.csv"))
   Age  Edu Income Sex
1    1   1 520000   0
2    2   2  51000   0
3    3   3   1200   0
4    1   4   1500   0
5    2   1 200000   1
6    3   2  15000   1
7    1   3   2000   1
8    2   4   1100   0
```

```
9    3   1 150000   1
10   1   2  10000   0
11   2   3   3000   0
12   3   4    500   1
```

显然, 在回归估计公式 $\hat{\boldsymbol{y}} = \boldsymbol{X}(\boldsymbol{X}^\top \boldsymbol{X})^{-1}\boldsymbol{X}^\top \boldsymbol{y}$ 中的 $\boldsymbol{X}$ 不能直接用这个数据的自变量, 因为一个是字符, 而另一个虽然是数字, 但没有数量意义.

哑元在 R 中回归时的情况

对于 R 中的诸如 lm、glm 等许多回归或分类函数都自动识别如 Artif0.csv 数据那样的字符表示的分类变量, 并加以处理, 不用额外处理, 比如代码

```
a=lm(Income~.,w) #或者a=lm(Income~Age+Edu+Sex,w)
```

而对于如 Artif1.csv 数据那样的用数字表示的分类变量, 必须用函数 factor 标明, 比如用

```
a1=lm(Income~factor(Age)+factor(Edu)+factor(Sex),w1)
```

或者用类似于下面的代码:

```
for (i in c(1,2,4)) w1[,i]=factor(w[,i])
a1=lm(Income~.,w1)
```

如果用汇总输出 (对第二个数据使用了先对分类变量逐项用 factor 的方式), 则得到对数据 Artif0.csv 的输出 (summary(a)):

```
> summary(a)
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)    23256      71440    0.33    0.758
AgeOld         69600      70468    0.99    0.369
AgeYoung       19167      78786    0.24    0.817
EduEle        -24300      81369   -0.30    0.777
EduGrad       292178      84692    3.45    0.018 *
EduHs         -23267      81369   -0.29    0.786
SexM          -82533      70468   -1.17    0.294

Residual standard error: 99700 on 5 degrees of freedom
Multiple R-squared:  0.809,Adjusted R-squared:  0.579
F-statistic: 3.53 on 6 and 5 DF,  p-value: 0.094
```

也得到对数据 Artif1.csv 的输出 (summary(a1)):

```
> summary(a1)
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   385033      78786    4.89   0.0045 **
Age2          -69600      70468   -0.99   0.3687
Age3          -50433      78786   -0.64   0.5503
Edu2         -292178      84692   -3.45   0.0182 *
Edu3         -315444      84692   -3.72   0.0136 *
Edu4         -316478      84692   -3.74   0.0135 *
Sex1          -82533      70468   -1.17   0.2943

Residual standard error: 99700 on 5 degrees of freedom
Multiple R-squared:  0.809,Adjusted R-squared:  0.579
F-statistic: 3.53 on 6 and 5 DF,  p-value: 0.094
```

这两个输出并非完全相同, 每个变量都少了一个水平, 但少得不一样, 每个都少了按照字符大小顺序来看最小的那个水平:

- 变量 Age 在 `summary(a)` 输出的估计没有 `AgeMiddle`(相当于 `summary(a1)` 输出的 `Age2`), Age 在 `summary(a1)` 输出的估计没有 `Age1`(相当于 `summary(a)` 的 `AgeOld`), 缺乏的水平作为约束条件自动设为 0.
- Edu 在 `summary(a)` 输出的估计没有 `EduColl`(相当于 `summary(a1)` 输出的 `Edu2`), Edu 在右边的估计没有 `Edu1` (相当于左边的 `EduGrad`), 缺乏的水平作为约束条件自动设为 0.
- 在 `summary(a)` 输出中的 Sex 没有 `SexF`(相当于 `summary(a1)` 输出中的 `Sex0`), Sex 在 `summary(a1)` 输出中没有 `Sex0`(相当于 `summary(a)` 输出中的 `SexF`), 缺乏的水平作为约束条件自动设为 0.

对于定性变量, 各个水平是不可估计的 (或者是需要约束的), 这里输出的估计中每个变量都少了一个水平: 这些缺少的 "估计" 都被软件自动设定为 0(当然你也可以设置其他类型的约束). **虽然上面两个回归的约束不一样, 但两个模型完全等价, 它们的拟合值也完全一样.** 输出中对每个水平的估计 (`Estimate`) 实际上是相对于那个设定为 0 的水平的差值. 因此, **这里对定性变量水平的 t 检验是没有意义的**, 因为你不能说 "某个水平不显著 (因为等于 0), 而另外的水平显著" 之类的话, **定性变量只能整个变量显著或整个变量不显著, 这反映了水平差异导致的因变量的差异, 绝对不可能有 "某个水平显著" 这种结论. 因此, 定性变量必须用方差分析来看其显著性.**

这个数据的拟合模型为:

$$\begin{aligned}\text{Income} =& 23256 + 69600\ I(\text{Age=Old}) + 19167\ I(\text{Age=Young}) - 24300\ I(\text{Edu=Ele})\\ &+ 292178\ I(\text{Edu=Grad}) - 23267\ I(\text{Edu=Hs}) - 82533\ I(\text{Sex=M}),\end{aligned}$$

式中的 $I()$ 是示性函数, 当变元为 TRUE 时函数值等于 1, 否则为 0. 3 个 (分别有 3、4、2 个水平的) 自变量一共有 $3\times 4\times 2 = 24$ 种组合, 该模型实际上有 24 个截距或水平线 (24

个值).

那么, 在回归估计公式 $\hat{\boldsymbol{y}} = \boldsymbol{X}(\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{y}$ 中的 $\boldsymbol{X}$ 对于这个例子是什么样的呢? 下面我们把数据 Artif0.csv 中的每一个水平都作为一个 0-1 哑元变量, 若观测值是这个水平, 则为 1, 否则为 0. 原始数据的自变量于是转换成下列形式:

	AgeMiddle	AgeOld	AgeYoung	EduColl	EduEle	EduGrad	EduHs	SexF	SexM
1	0	1	0	0	0	1	0	1	0
2	1	0	0	1	0	0	0	1	0
3	0	0	1	0	0	0	1	1	0
4	0	1	0	0	1	0	0	1	0
5	1	0	0	0	0	1	0	0	1
6	0	0	1	1	0	0	0	0	1
7	0	1	0	0	0	0	1	0	1
8	1	0	0	0	1	0	0	1	0
9	0	0	1	0	0	1	0	0	1
10	0	1	0	1	0	0	0	1	0
11	1	0	0	0	0	0	1	1	0
12	0	0	1	0	1	0	0	0	1

显然, 包括 Age 的三列、包括 Edu 的四列以及包括 Sex 的两列相加就变成全部是 1 的列, 也就是说, 这个矩阵中的自变量线性相关. 于是, 对每一个定性变量选出一列删除 (相当于约束是设定其为 0), 再加上一列代表截距的 1. 这就可以使用估计公式 $\hat{\boldsymbol{y}} = \boldsymbol{X}(\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{y}$ 了, 如果删除上面矩阵的第 1、4、8 列, 再加上一列 1, 可得到下面的矩阵, 这就是公式中的 $\boldsymbol{X}$ 矩阵:

	Const	AgeOld	AgeYoung	EduEle	EduGrad	EduHs	SexM
1	1	1	0	0	1	0	0
2	1	0	0	0	0	0	0
3	1	0	1	0	0	1	0
4	1	1	0	1	0	0	0
5	1	0	0	0	1	0	1
6	1	0	1	0	0	0	1
7	1	1	0	0	0	1	1
8	1	0	0	1	0	0	0
9	1	0	1	0	1	0	1
10	1	1	0	0	0	0	0
11	1	0	0	0	0	1	0
12	1	0	1	1	0	0	1

使用这个自变量矩阵做回归会产生等价的结果.

许多软件或者函数不能自动转换数据里的文字形式的定性变量, 强行使用会出现报错信息. 这时就需要在编程中事先转换.

方差分析表对两个数据完全一样. 前面输出的 `summary` 之所以不一样是因为约束不同, 但模型是相同的, 数学表述虽不相同但等价. 下面是使用函数 `anova` 得到的方差分析表. 对于数据 Artif0.csv 的方差分析表 (`anova(a)`) 为:

```
> anova(a)
Analysis of Variance Table

Response: Income
          Df   Sum Sq  Mean Sq F value Pr(>F)
Age        2 1.83e+10 9.16e+09    0.92  0.456
Edu        3 1.78e+11 5.94e+10    5.98  0.041 *
Sex        1 1.36e+10 1.36e+10    1.37  0.294
Residuals  5 4.97e+10 9.93e+09
```

对于数据 Artif1.csv 的方差分析表 (anova(a1)) 为:

```
> anova(a1)
Analysis of Variance Table

Response: Income
          Df   Sum Sq  Mean Sq F value Pr(>F)
Age        2 1.83e+10 9.16e+09    0.92  0.456
Edu        3 1.78e+11 5.94e+10    5.98  0.041 *
Sex        1 1.36e+10 1.36e+10    1.37  0.294
Residuals  5 4.97e+10 9.93e+09
```

3.5 机器学习回归简介及案例

3.5.1 例子和机器学习方法的优势

由于本书主要介绍经典的回归分析, 在这里使用一个例子来比较包括经典回归在内的各种回归方法.

各种机器学习方法对于例3.1波士顿住房数据回归的结果

首先在图3.8中展示包括经典线性回归在内的 16 种方法预测精度 (NMSE) 的 10 折交叉验证结果. 这些计算过程会在本节最后一小节介绍, 最终结果的数字打印输出为:

```
                                  Predictor   NMSE Rsquare    RMSE    MAE   time
                          Linear Regression 0.2769  0.7231  108.58   1703  0.720
                                Bagged CART 0.1932  0.8068   90.70   1373  1.936
                                Bagged MARS 0.2288  0.7712  170.94   4448 16.080
                  Bagged MARS using gCV Pruning 0.1480  0.8520   79.39   1218  6.413
                               Boosted Tree 0.1821  0.8179  264.17  12286 11.877
 Conditional Inference Random Forest 0.2333  0.7667  172.63   4241 27.185
                   Conditional Inference Tree 0.2204  0.7796  167.80   4466  1.197
                    eXtreme Gradient Boosting 0.2297  0.7703 1027.75 149935 17.422
                               Random Forest 0.1297  0.8703  182.05   6667  7.045
                 Stochastic Gradient Boosting 0.1446  0.8554  235.43  10821  2.113
                       Parallel Random Forest 0.1212  0.8788  124.44   3264 20.879
                       Random Forest (ranger) 0.1297  0.8703  182.05   6667  6.935
```

```
                                  Random Forest(Rborist) 0.1102  0.8898  118.65   3252 16.295
                                        Random Forest(rf) 0.1210  0.8790  124.31   3256 20.393
Bayesian Regularized Neural Networks 0.1849  0.8151  153.69   4113  2.343
                                Stochastic Gradient Boosting 0.1446  0.8554  235.43  10821  1.969
                                   k-Nearest Neighbors 0.1862  0.8138  154.20   3675  1.114
```

其中的 Predictor 是预测方法名称, 而 NMSE(标准化均方误差 $NMSE_{cv}$)、Rsquare(记分 R^2_{cv})、RMSE(均方误差平方根 $RMSE_{cv}$)、MAE(误差绝对值之均值 MAE_{cv}) 是关于各种误差的度量 (有些前面介绍过). 这些都是用拟合训练集所得到的模型来对测试集做预测所得到的误差, 是交叉验证的结果. 下面是各个度量的定义, 其中 $y_{test\,i}$ 是测试集的因变量第 i 个观测值, 而 $\hat{y}_{test\,i}$ 是用训练集得到的模型来对测试集 $y_{test\,i}$ 所做的预测, $\bar{y}_{test}$ 是测试集因变量的均值. 其中前 3 个等价, 都基于均方误差 $MSE_{cv}=\frac{1}{n}\sum_{i=1}^{n}(y_{test\,i}-\hat{y}_{test\,i})^2$. 这里的公式 (和前面3.2节的公式) 中都加了 "test" 或 "cv" 等下标以强调用测试集来度量训练集产生的模型的预测精度. 其实, 在明白了是交叉验证的结果时, 不是必须有这些下标.

$$NMSE_{cv}=\frac{\sum_{i=1}^{n}(y_{test\,i}-\hat{y}_{test\,i})^2}{\sum_{i=1}^{n}(y_{test\,i}-\bar{y}_{test})^2};$$

$$R^2_{cv}=1-NMSE_{cv}=1-\frac{\sum_{i=1}^{n}(y_{test\,i}-\hat{y}_{test\,i})^2}{\sum_{i=1}^{n}(y_{test\,i}-\bar{y}_{test})^2};$$

$$RMSE_{cv}=\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_{test\,i}-\hat{y}_{test\,i})^2};$$

$$MAE_{cv}=\frac{1}{n}\sum_{i=1}^{n}|y_{test\,i}-\hat{y}_{test\,i}|.$$

上面输出中的 time 是在笔者计算机上计算的耗时 (秒). 具体计算程序将在后面介绍.

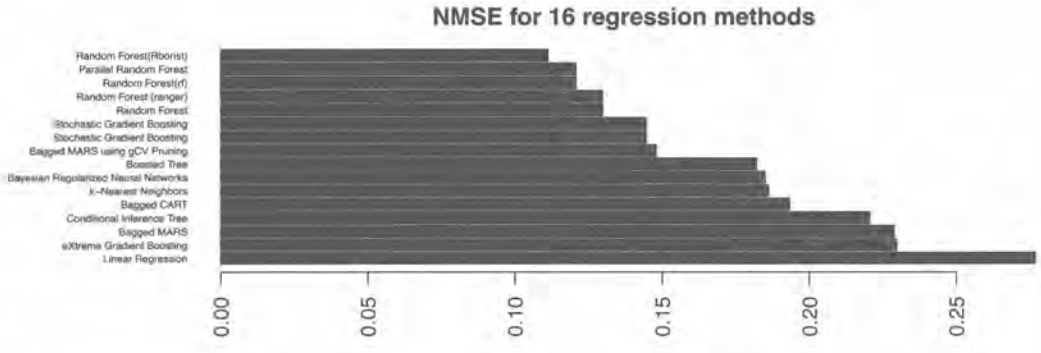

图 3.8 对于例3.1波士顿住房数据的 16 种回归方法预测精度 (NMSE) 的 10 折交叉验证结果

从这些交叉验证结果可以看出最好的是随机森林, 它比最差的线性回归精确两倍多. 因此, 在回归分析的课程中只讲经典线性回归而不介绍各种机器学习方法是不负责任的. 本书没有详细介绍各种机器学习的回归方法, 下面仅仅直观地介绍三种机器学习回归方法: 作为基石的**决策树** (decision tree) 回归 (回归树 (regression tree)) 和决策树的两种组合方法 —— **bagging** 回归及**随机森林** (random forest) 回归.

3.5.2 决策树回归

决策树对例3.1波士顿住房数据的回归

首先, 我们来查看对例3.1数据做决策树回归的结果, 这里用的是程序包 `rpart`[5] 里面的 `rpart` 函数, 通过代码可得到数值输出和图3.9, 图很直观, 但产生图的数值输出所包含的信息量更多.

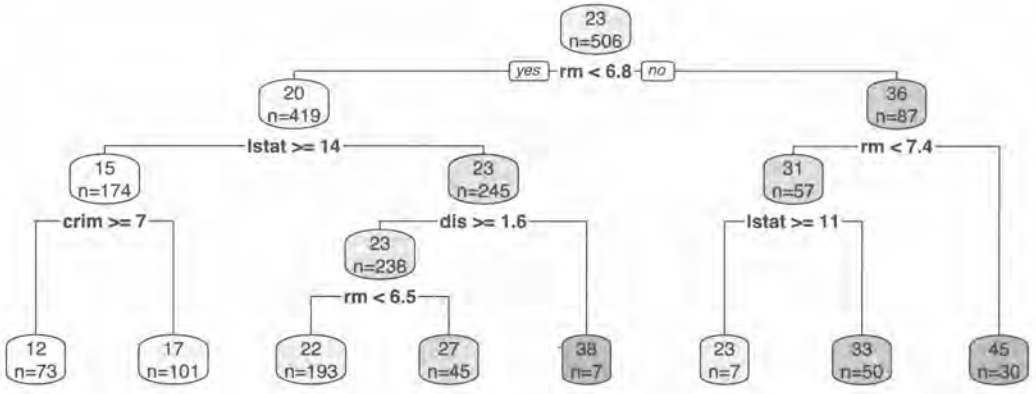

图 3.9 对例3.1波士顿住房数据做决策树回归的结果

决策树的使用

先不做更多解释, 如果我们有了根据例3.1波士顿住房数据做的决策树图3.9, 对于一个新观测值该如何预测其房价呢? 从图的最上方, 我们看到是否 `rm<6.8` 的标识, 如果为是 (`yes`) 走向左边, 如果为否 (`no`) 则走向右边. 无论什么结果, 只能走向一个叉, 然后又有新的变量及问题, 比如左边问是否 `lstat>=14`, 右边问是否 `rm<7.4`. 如此走下去, 我们最终只能到达 8 个终点之一, 然后把那里的值取为预测房价.

下面是得到图3.9及数值输出的代码:

```
w=read.csv("BostonHousing2.csv")[,-c(1:5)]
library(rpart.plot);(a=rpart(cmedv~.,w))
rpart.plot(a,extra = 1)
```

数值输出为:

[5]Terry Therneau, Beth Atkinson and Brian Ripley (2017). rpart: Recursive Partitioning and Regression Trees. R package version 4.1-11. https://CRAN.R-project.org/package=rpart.

```
n= 506

node), split, n, deviance, yval
    * denotes terminal node

1) root 506 42580.0 22.53
  2) rm< 6.838 419 16420.0 19.71
    4) lstat>=14.4 174  3112.0 14.83
      8) crim>=6.992 73   842.1 11.75 *
      9) crim< 6.992 101  1079.0 17.05 *
    5) lstat< 14.4 245  6205.0 23.18
      10) dis>=1.551 238  3195.0 22.75
        20) rm< 6.543 193  1499.0 21.69 *
        21) rm>=6.543 45   555.0 27.28 *
      11) dis< 1.551 7  1429.0 38.00 *
  3) rm>=6.838 87  6851.0 36.09
    6) rm< 7.437 57  2033.0 31.34
      12) lstat>=11.32 7   390.5 22.60 *
      13) lstat< 11.32 50  1033.0 32.57 *
    7) rm>=7.437 30  1099.0 45.10 *
```

决策树的细节

图3.9是由 15 个**节点** (node) 组成的决策树, 它像一棵倒长的树, 其中最上面的是**根节点** (root), 最下面的是**叶节点**或**终节点** (terminal node). 根节点是初始节点, 包含了全部数据 (图上有 n=506 以及这些观测值因变量均值 23), 其余的每个节点代表一个数据子集 (在数据输出和图中都标明该节点的观测值数目 n=.. 及在该节点观测值因变量的均值). 下面介绍决策树建立的步骤 (这里只考虑每次分两叉的决策树, 也只考虑残差平方和这一种误差度量):

(1) 首先, 我们看到在输出 (见图3.9) 中的根节点 (n=506, 均值 (yval) 为 22.53, 残差平方和 $\sum(y_i-\bar{y})^2$ 为 deviance=42580) 处有一个变量 rm, 当 rm<6.838 时, 向左分叉, 而当 rm>=6.838 时, 向右分叉, 这样就把原始数据分成两个数据. 它们的样本量分别为 n=419 和 n=87, 均值分别为 19.71 和 36.09, 而 deviance 分别为 16420 和 6851(其和为 23271). 我们有下列问题:

(a) 为什么选择变量 rm (称为**拆分变量**) 在 6.838 的值作为把全部数据拆分的准则? 这是因为, 相对于 rm 的其他值, 6.838 能够使得总 deviance 最小 ($16420+6851=23271$). 这里的 deviance 是在每个节点的所有因变量的值减去均值 yval 的平方和. 如果在这一点不再继续下去了, 就用均值代表在该节点的拟合值.

(b) 为什么选择变量 rm 作为拆分变量, 而不是其他变量? 其实, 对每个变量都试过了 (所有变量都作为候选变量来竞争拆分变量这个位置), 而变量 rm 使得拆分后的残差平方和最小, 因此被选中了.

(c) 如果变量是分类变量, 比如红、黄、黑三种颜色, 怎么办呢? 对于数量变量, 比如

12、24、90, 则有两种分割: 在 12 和 24 之间及 24 和 90 之间; 但对于红、黄、黑三种颜色, 则有 "红黄一组, 黑的一组""红黑一组, 黄的一组""红黑一组, 黄的一组" 等三种分割. 还是在各种分割上选择误差最小的分割.

(2) 在每个节点都重复类似于在根节点处的对拆分变量的竞争以及对数据的拆分, 一直进行下去.

(3) 在某些准则下 (比如不出现过拟合, 或到达预期深度等), 决策树停止生长, 得到一些终节点 (在输出中有 * 号的节点), 终节点的各个均值 (yval) 就是拟合值. 对于线性回归, 有和数据量一样多的拟合值, 而在决策树中只有和终节点数目相同的拟合值 (每个终节点的观测值共享一个拟合值).

(4) 每个新数据必定满足沿着一条路径走到某一个终节点, 而该终节点的拟合值就是该数据的预测值这一条件.

决策树和线性回归的简单比较

人们可能会问: 决策树只有几个拟合值, 能和线性回归比较吗? 我们比较一下它们的残差图 (见图3.10), 并计算各自的残差平方和. 代码为:

```
b=lm(cmedv~.,w); a=rpart(cmedv~.,w);resid=w$cmedv-predict(a,w)
b.mse=mean((b$res)^2);a.mse=mean(resid^2)
par(mfrow=c(1,2))
plot(b$res,ylab="residuals",main="Linear Regression: MSE=21.50")
plot(resid,ylab="residuals",main="Decision Tree: MSE=15.66")
```

仅仅比较图3.10 就可以看出, 线性回归和决策树的残差紧凑度稍有不同, 就残差平方和来说, 回归的为 21.50, 决策树的只有 15.66.

注意: 对于单独一棵决策树, 预测效果不一定比线性回归更好, 这取决于数据, 但许多决策树构成的诸如随机森林那样的组合方法会大大提高预测精度.

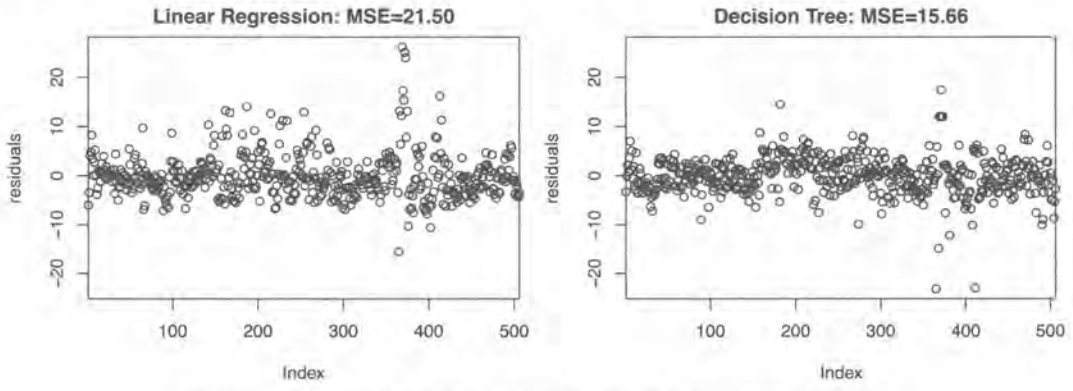

图 3.10 例3.1波士顿住房数据的线性回归 (左) 和决策树回归的残差图 (右)

决策树和线性回归的交叉验证比较

下面的程序是把全部数据随机划分成 Z 折交叉验证集的程序 (这里默认 $Z = 10$), 输出为各个折的下标集.

```
CV=function(data=w,Z=10,seed=1010){
  n=nrow(data);N=1:n;Z=10
  set.seed(seed)
  mm=sample(rep(1:Z,ceiling(n/Z))[N])
  K=list();for(i in 1:Z)K[[i]]=N[mm==i]
  return(K)
}
```

下面是对例3.1波士顿住房数据用决策树和线性回归分别做 10 折交叉验证的代码:

```
n=nrow(w);N=1:n;Z=10;mm=CV()
rs=NULL;
for(i in 1:Z){
  at=rpart(cmedv~.,w[-mm[[i]],])
  pred=predict(at,w[mm[[i]],])
  rs=rbind(rs,data.frame(y=w[mm[[i]],1],pred=pred))
}
mse=mean((apply(rs,1,diff))^2)
mst=mean((rs[,1]-mean(rs[,2]))^2)
mse/mst #0.264
################
rs2=NULL;
for(i in 1:Z){
  al=lm(cmedv~.,w[-mm[[i]],])
  pred=predict(al,w[mm[[i]],])
  rs2=rbind(rs2,data.frame(y=w[mm[[i]],1],pred=pred))
}
mse2=mean((apply(rs2,1,diff))^2)
mst2=mean((rs2[,1]-mean(rs2[,2]))^2)
mse2/mst2 #0.2739
```

得到的交叉验证的输出表明, 两个回归的标准化均方误差相差不大, 其中决策树为 0.264, 而线性回归为 0.2739, 稍微高一点.

3.5.3 bagging 回归

一棵决策树的效果可能很平常, 但是借助对数据多次**放回抽样** (即**自助法抽样** (bootstrap sampling)) 所得到的许多棵决策树却会得到非常好的模型, 这就是**组合模型** (ensemble model), 组合模型中包括 bagging、boosting 和随机森林等非常优秀的模型.

bagging(bootstrap aggregating) 是由 Breiman 提出的一个简单的组合模型, 它对原始数据集做很多次放回抽样, 每次抽取和样本量同样多的观测值, 放回抽样使得每次都有大约 30% 多的观测值没有被抽到, 另一些观测值则会被重复抽到. 如此, 得到很多不同的数据集, 然后针对每个数据集建立一个决策树, 因此产生大量决策树. 对于回归来说, 一个新的观测值通过如此多的决策树得到很多预测值, 最终结果为这些预测值的简单平均.

在抽样过程中, 由于是放回抽样, 每次大约有百分之三四十的观测值不会被抽到, 称为"袋外"(out of bag, OOB) 观测值, 它们被 bagging 用来当作天然的测试集以确定交叉验证的预测精度.

对例3.1波士顿住房数据做和前面一样 (同样的 10 折数据) 的 bagging 回归的 10 折交叉验证, 可以用类似的代码, 这里用到程序包 ipred.[6]

```
set.seed(1010)
rsb=NULL;
for(i in 1:Z){
  ab=bagging(cmedv~.,w[-mm[[i]],])
  pred=predict(ab,w[mm[[i]],])
  rsb=rbind(rsb,data.frame(y=w[mm[[i]],1],pred=pred))
}
mseb=mean((apply(rsb,1,diff))^2)
mstb=mean((rsb[,1]-mean(rsb[,2]))^2)
mseb/mstb #0.1789336
```

得到标准化均方误差为 0.1789336, 只有最小二乘线性回归的 65.3%.

3.5.4 随机森林回归

随机森林是由 Breiman 提出的, 该方法把分类和回归的精度提到了前所未有的高度. 随机森林不会过拟合, 不怕维数诅咒, 变量多多益善, 对数据没有限制和假定. Breiman (2001) 用 "随机森林在基因芯片淋巴瘤数据集上运行, 有三个类别, 样本量为 81, 有 4682 个变量 (基因), 没有任何变量选择". 他指出, "从科学观点来看, 有趣的是估计了 4682 个基因表达的重要性". 对于如此少的样本量和如此多的变量, 经典统计是无能为力的. 图3.11是 Breiman 文章这一段的原文截图.

> Random forests was run on a microarray lymphoma data set with three classes, sample size of 81 and 4,682 variables (genes) without any variable selection [for more information about this data set, see Dudoit, Fridlyand and Speed, (2000)]. The error rate was low. What was also interesting from a scientific viewpoint was an estimate of the importance of each of the 4,682 gene expressions.

图 3.11 Breiman(2001) 文章部分截图

随机森林的原理

随机森林的原理并不复杂, 和 bagging 类似, 它对原始数据集做很多次放回抽样, 每次抽取和样本量同样多的观测值, 由于是放回抽样, 每次都有一些观测值没有被抽到, 有一些观测值会被重复抽到. 如此, 会得到很多不同的数据集, 然后针对每个数据集建立一棵决策

[6] Andrea Peters and Torsten Hothorn (2017). ipred: Improved Predictors. R package version 0.9-6. https://CRAN.R-project.org/package=ipred.

树, 因此产生大量决策树. 和 bagging 不同的是, 在随机森林每棵树的每个节点, 拆分变量不是由所有变量竞争, 而是由**随机挑选**的少数变量竞争, 而且每棵树都长到底. 拆分变量候选者的数目限制可以避免由于强势变量主宰而忽略的数据关系中的细节, 因而大大提高了模型对数据的代表性. 随机森林的最终结果是所有树的结果的平均, 也就是说, 一个新的观测值通过许多棵树 (比如 n 棵) 得到 n 个预测值, 最终用这 n 个预测值的平均作为最终结果.

我们使用随机森林程序包 randomForest[7]的同名函数 randomForest 来做随机森林回归, 其默认树的棵数是 ntree=500. 和 bagging 一样, 在抽样过程中, 由于是放回抽样, 每次不会被抽到的观测值称为 "袋外"(OOB) 观测值, 它们被随机森林用来做 OOB 交叉验证的测试集以确定预测精度.

用随机森林拟合例3.1波士顿住房数据

首先拟合所有数据:

```
w=read.csv("BostonHousing2.csv")[,-c(1:5)]
a.rf=randomForest(cmedv~.,w,importance=TRUE, localImp=TRUE,proximity=TRUE)
```

变量重要性、局部重要性、部分依赖性、离群点的图

我们可以得到很多结果, 首先是由代码 a.rf$importance 得到的随机森林的变量重要性, 用图 (见图3.12) 表示, 左图是用 OOB 数据作为训练集所做的交叉验证得到的度量, 意义为删除某个自变量后均方误差增加的程度 (增加越多, 变量越重要), 右图为综合了每个自变量在各个节点的表现 (使得数据变 "纯", 即使均方误差变小的效率) 而产生的重要性度量. 作图代码如下:

```
layout(t(1:2))
for (i in 1:2)
barplot(t(a.rf$importance[,i]),names.arg=row.names(a.rf$importance),
cex.names = 0.7,horiz=TRUE,col=4,las=1,xlab=colnames(a.rf$importance)[i])
```

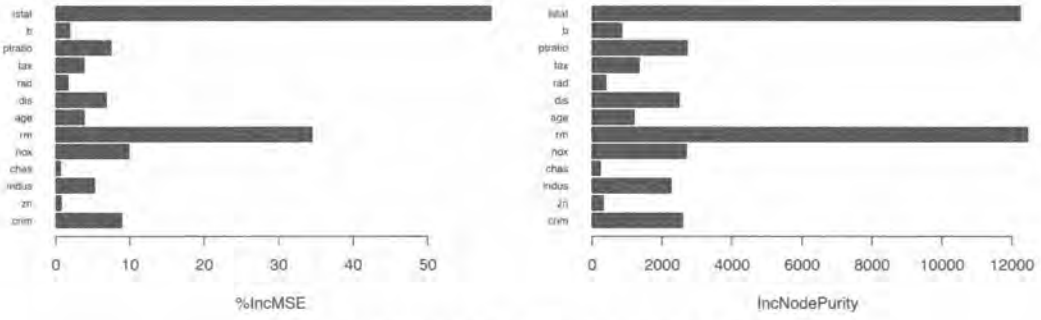

图 3.12 随机森林拟合例3.1波士顿住房数据的变量重要性

[7]A. Liaw and M. Wiener (2002). Classification and Regression by randomForest. R News 2(3), 18–22.

除了总体的变量重要性之外, 还有局部变量重要性, 这是每个自变量对每一个观测值的影响 (rf$local), 对于例3.1的 13 个变量和 506 个观测值, 它是一个 13×506 矩阵, 图形 (见图3.13) 的每一条线代表一个观测值 (一共 506 个), 而横坐标为变量名, 纵坐标为局部重要性的度量.

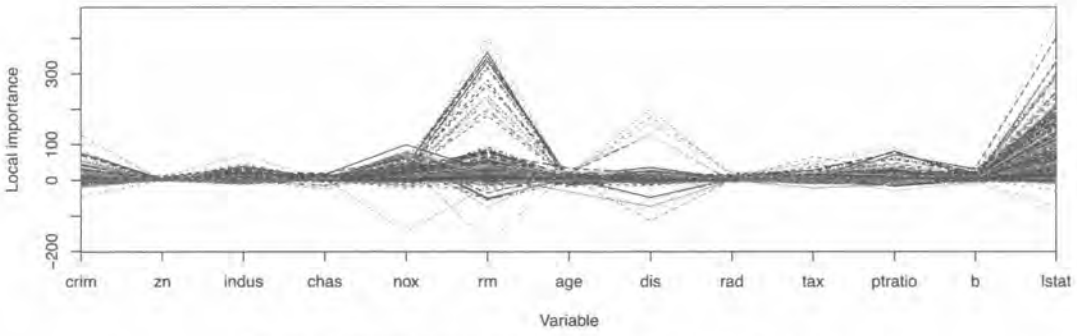

图 3.13 随机森林拟合例3.1波士顿住房数据的局部变量重要性图

产生图3.13的代码如下:

```
matplot(1:13,a.rf$local,type = "l",xaxt="n",
  xlab="Variable",ylab = "Local importance")
axis(side = 1,at = 1:13,labels = rownames(a.rf$local),las=0,cex.axis=1)
```

除此之外, 还有每个自变量对因变量的部分依赖性图, 我们把重要的两个变量 lstat、rm 和不那么重要的 zn 点图 (见图3.14), 代码为:

```
par(mfrow=c(1,3))
partialPlot(a.rf, pred.data=w, x.var=lstat)
partialPlot(a.rf, pred.data=w, x.var=rm)
partialPlot(a.rf, pred.data=w, x.var=zn)
```

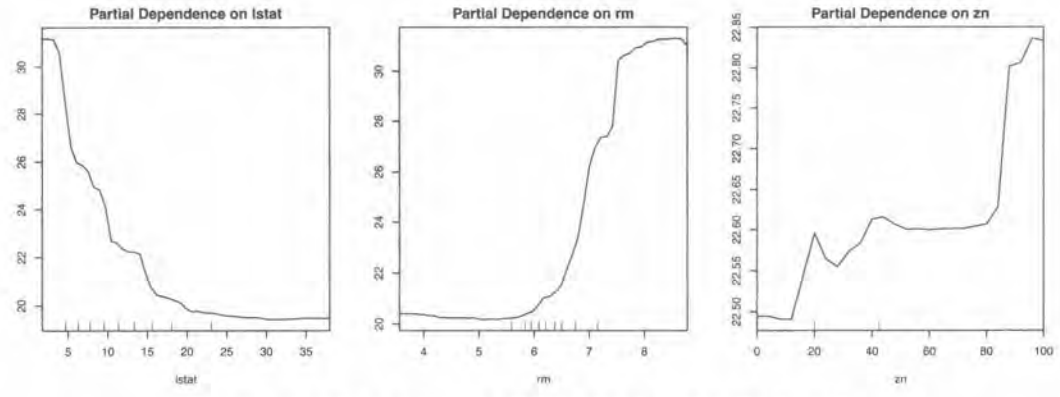

图 3.14 随机森林拟合例3.1波士顿住房数据对三个变量的部分依赖性图

图3.14左边及中间两个重要变量可以造成因变量幅度 10 (从 20 到 30 多) 的变化 (纵坐标是变化的一种度量), 而右边不重要的变量 zn 对因变量的影响幅度仅仅有 0.35 (从 22.50 到 22.85). 该图还说明因变量对第一个变量大体上是递减关系, 而对后两个大体上是递增关系.

随机森林还根据是否经常出现在同样的节点给出了观测值之间的亲疏程度, 根据这个度量 (a.rf$proximity) 可以得到离群点图 (见图3.15), 代码为:

```
plot(outlier(a.rf$proximity), type="h",ylab="Outlying measure")
```

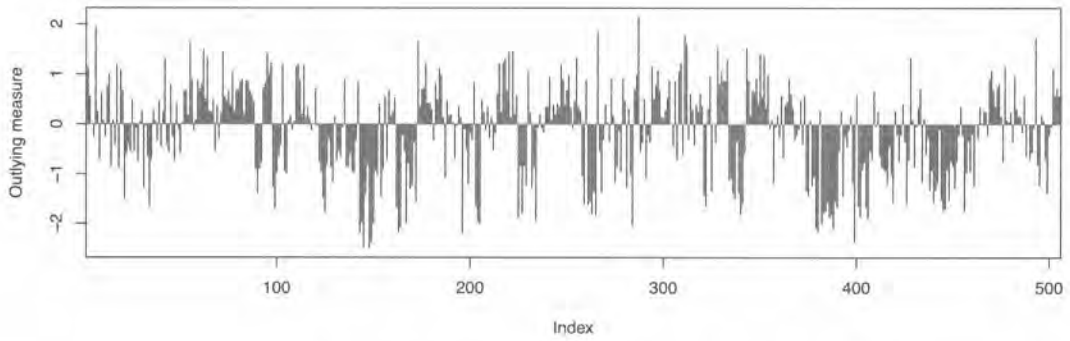

图 3.15 随机森林拟合例3.1波士顿住房数据的离群点图

随机森林拟合例3.1波士顿住房数据预测精度的交叉验证

还是用前面对线性回归、决策树及 bagging 的同样 10 折数据对随机森林做交叉验证, 代码为:

```
r3=NULL;
for(i in 1:Z){
  a3=randomForest(cmedv~.,w[-mm[[i]],])
  pred=predict(a3,w[mm[[i]],])
  r3=rbind(r3,data.frame(y=w[mm[[i]],1],pred=pred))
}
mse3=mean((apply(r3,1,diff))^2)
mst3=mean((r3[,1]-mean(r3[,2]))^2)
mse3/mst3 #0.1202
```

得到的标准化均方误差为 0.1202, 比前面决策树的 0.264, bagging 的 0.1789 及线性回归的 0.2739 都低, 这说明随机森林基于决策树, 但大大优于决策树. 关于随机森林的想法十几年来已经扩展了很多, 出现了很多衍生模型. 下面一节会在方法比较的计算中使用到其中的一些.

3.6 经典线性回归与各种机器学习回归的比较

3.6.1 对例3.1波士顿住房数据的九种方法的比较

这里我们列举了几种机器学习的模型, 并对例3.1的数据进行拟合, 通过 10 折交叉验证比较它们的预测精度.

需要的程序包和方法

我们需要载入需要的程序包:

```
library(ipred);library(mboost);library(party);library(kernlab)
library(ranger);library(kknn);library(randomForest);library(partykit)
```

这里要比较的方法所用的函数包括 lm(线性回归)、bagging (bagging 回归)、blackboost (梯度自助回归)、cforest (条件随机森林)、ctree (条件推断树)、ranger (快速随机森林)、kknn (k 最近邻法)、randomForest (随机森林)、ksvm (支持向量机).

输入数据分折的程序

```
CV=function(data=w,Z=10,seed=1010){
  n=nrow(data);N=1:n;Z=10
  set.seed(seed)
  mm=sample(rep(1:Z,ceiling(n/Z))[N])
  K=list();for(i in 1:Z)K[[i]]=N[mm==i]
  return(K)
}
```

计算

```
Z=10;D=1;mm=CV(w);ff=cmedv ~ .
sst=sum((w[,D]-mean(w[,D]))^2);sse=vector()
J=1;set.seed(1010);sse.a=0
for(i in 1:Z) {
  m=mm[[i]]
  a=lm(ff,data =w[-m,])
  sse.a=sse.a+sum((w[m,D]-predict(a,w[m,]))^2)
}
sse[J]=sse.a
J=J+1;set.seed(1010);sse.a=0
for(i in 1:Z) {
  m=mm[[i]]
  a=bagging(ff,data =w[-m,])
  sse.a=sse.a+sum((w[m,D]-predict(a,w[m,]))^2)
}
```

```
sse[J]=sse.a
J=J+1;set.seed(1010);sse.a=0
for(i in 1:Z) {
  m=mm[[i]]
  a=blackboost(ff,data =w[-m,])
  sse.a=sse.a+sum((w[m,D]-predict(a,w[m,]))^2)
}
sse[J]=sse.a
J=J+1;set.seed(1010);sse.a=0
for(i in 1:Z) {
  m=mm[[i]]
  a=cforest(ff,data =w[-m,])
  sse.a=sse.a+sum((w[m,D]-predict(a,w[m,]))^2)
}
sse[J]=sse.a
J=J+1;set.seed(1010);sse.a=0
for(i in 1:Z) {
m=mm[[i]]
a=ctree(ff,data =w[-m,])
sse.a=sse.a+sum((w[m,D]-predict(a,w[m,]))^2)
}
sse[J]=sse.a
J=J+1;set.seed(1010);sse.a=0
for(i in 1:Z) {
  m=mm[[i]]
  a=ranger(ff,data =w[-m,])
  sse.a=sse.a+sum((w[m,D]-predict(a,w[m,])$predictions)^2)
}
sse[J]=sse.a
J=J+1;set.seed(1010);sse.a=0
for(i in 1:Z) {
  m=mm[[i]]
  yp=kknn(formula=ff,train=w[-m,],test=w[m,])$fitted
  sse.a=sse.a+sum((w[m,D]-yp)^2)
}
sse[J]=sse.a
J=J+1;set.seed(1010);sse.a=0
for(i in 1:Z) {
  m=mm[[i]]
  a=randomForest(ff,data =w[-m,])
  sse.a=sse.a+sum((w[m,D]-predict(a,w[m,]))^2)
}
sse[J]=sse.a
J=J+1;set.seed(1010);sse.a=0
```

```
for(i in 1:Z) {
  m=mm[[i]]
  a=ksvm(as.matrix(w[-m,-D]),w[-m,D])
  sse.a=sse.a+sum((w[m,D]-predict(a,w[m,-D]))^2)
}
sse[J]=sse.a
nmse=sse/sst
r2=1-nmse
```

下表为各个回归方法拟合例3.1波士顿住房数据所得到的交叉验证的 NMSE, 图3.16是根据这个输出生成的. 相信读者很容易看出各个方法对这个数据的预测精度的不同.

方法	randomForest	ranger	blackboost	bagging	cforest	ksvm	ctree	kknn	lm
NMSE	0.1188	0.1285	0.1362	0.1789	0.1807	0.1873	0.2170	0.2224	0.2739

九种方法交叉验证的标准化均方误差图 (见图3.16) 的作图代码为:

```
reg.names=c('lm','bagging','blackboost','cforest','ctree','ranger',
'kknn','randomForest','ksvm')
barplot(nmse[order(nmse)],horiz = TRUE, names.arg=reg.names[order(nmse)],
   col=4,las=2)
title("NMSE for 9 regression methods")
```

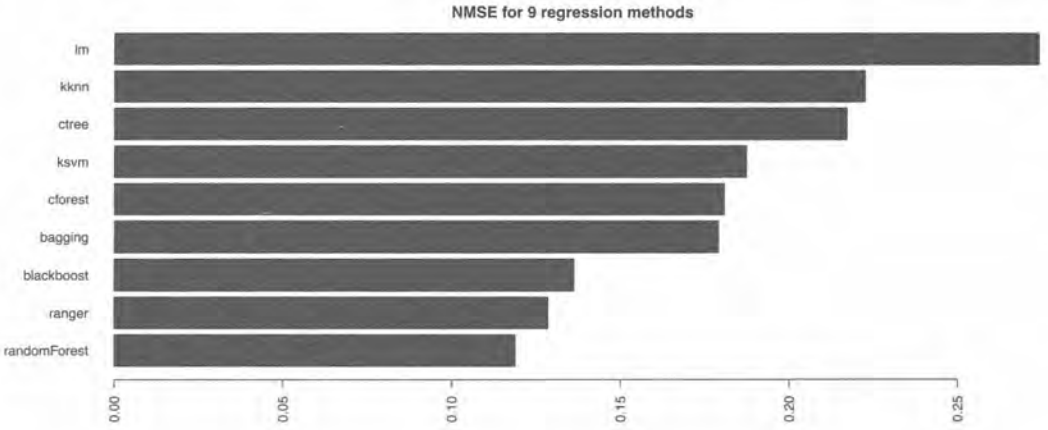

图 3.16 九种方法拟合例3.1波士顿住房数据的 NMSE 图

3.6.2 对例3.1波士顿住房数据使用 caret 包比较各种方法 *

相信读者已经掌握前一小节的代码. 但在这里为了介绍一个新程序包 caret 而改变编程的方法. 该程序包看上去有些啰唆, 但对于大量模型的处理似乎要方便一些.

这里, 我们使用的程序包 caret[8]本身不包括算法, 它是对各个独立程序包的一个包装, 你可以使用该程序包运行 200 多个模型, 而且不需要使用 library 函数来手动引用那些程

[8]Max Kuhn. Contributions from Jed Wing, Steve Weston, Andre Williams, Chris Keefer, Allan Engelhardt,

序包, 只要你的电脑中已经下载那些程序包, caret 会自动引用. 实际运算还是那些程序包各自的函数, 但代码基本统一在 caret 的类似代码中了.

以随机森林拟合例3.1波士顿住房数据为例介绍 caret 程序包

最简单的以随机森林拟合例3.1数据的程序是 (通过选项 method="rf" 确定用随机森林模型, 只要 method= 所选的标签存在, 就可做拟合):

```
library(caret)
rf=train(cmedv~., data = w, method = "rf")
rf
```

给出下面结果, 实际上是三个结果, 是根据内置的重抽样做了三次, 选择了不同的参数 mtry, 并且根据误差度量 RMSE 得到最优的 mtry=7:

```
> rf
Random Forest

506 samples
13 predictor

No pre-processing
Resampling: Bootstrapped (25 reps)
Summary of sample sizes: 506, 506, 506, 506, 506, 506, ...
Resampling results across tuning parameters:

 mtry  RMSE   Rsquared  MAE
  2    3.583  0.8543    2.385
  7    3.287  0.8646    2.202
 13    3.527  0.8407    2.307

RMSE was used to select the optimal model using the smallest value.
The final value used for the model was mtry = 7.
```

这时, 可以输出最后选中的模型:

```
> rf$finalModel

Call:
randomForest(x = x, y = y, mtry = param$mtry)
Type of random forest: regression
Number of trees: 500
No. of variables tried at each split: 7
```

Tony Cooper, Zachary Mayer, Brenton Kenkel, the R Core Team, Michael Benesty, Reynald Lescarbeau, Andrew Ziem, Luca Scrucca, Yuan Tang, Can Candan and Tyler Hunt. (2017). caret: Classification and Regression Training. R package version 6.0-78. https://CRAN.R-project.org/package=caret.

```
Mean of squared residuals: 9.229
% Var explained: 89.03
```

对拟合的控制和对参数的调整

对拟合的控制可以通过对函数 train 的选项 trControl 得到, 比如下面代码就要求做 10 折交叉验证, 并且重复 4 次.

```
tr=trainControl(method="repeatedcv",number=10,repeats=4)
rf1=train(cmedv~., data = w, method = "rf", trControl = tr )
```

下面是在上面控制的基础上, 增加调控选项 (tuneGrid), 对随机森林的选项做 mtry 分别等于数目 2、4、6、8、10 的 5 次试验:

```
tg=data.frame(mtry = seq(2, 10, by =2))#mtry 只有随机森林有
rf2=train(cmedv~.,data=w, method="rf",trControl=tr,tuneGrid = tg)
```

可以得到如下的结论 (有些重复), 其中第一个 (通过代码 rf2) 给出调试结果, 并选中 mtry=6, 第二个 (通过代码 rf2$results) 则给出了调试中更多的误差度量值.

```
> rf2
Random Forest

506 samples
13 predictor

No pre-processing
Resampling: Cross-Validated (10 fold, repeated 4 times)
Summary of sample sizes: 457, 455, 455, 457, 456, 455, ...
Resampling results across tuning parameters:

 mtry  RMSE   Rsquared  MAE
  2    3.375  0.8800    2.287
  4    3.021  0.8980    2.086
  6    3.007  0.8951    2.080
  8    3.012  0.8930    2.093
 10    3.036  0.8899    2.103

RMSE was used to select the optimal model using the smallest value.
The final value used for the model was mtry = 6.
> rf2$results
  mtry  RMSE Rsquared   MAE RMSESD RsquaredSD  MAESD
1    2 3.375   0.8800 2.287 0.9092    0.06380 0.3836
2    4 3.021   0.8980 2.086 0.7274    0.04472 0.3487
3    6 3.007   0.8951 2.080 0.6825    0.04788 0.3448
```

```
4    8 3.012   0.8930 2.093 0.6632    0.05045 0.3517
5   10 3.036   0.8899 2.103 0.6927    0.05715 0.3612
```

注意: 调控选项 (tuneGrid) 必须和原始程序包的函数一致, 比如这里的 mtry 只有随机森林才有, 用在其他模型中就不一定合适了.

下面的代码就是做 10 折交叉验证 (不再重复), 并且记录交叉验证的预测值 (选项的名称为: savePredictions).

```
tr3=trainControl(method = "cv", number = 10, savePredictions = TRUE)
rf3=train(cmedv~., data = w, method = "rf", trControl = tr3)
```

16 种回归模型的 10 折交叉验证

首先, 下面是一个整理各个方法结果的函数, 该函数计算每一种方法的 MSE、RMSE、R^{*2}、NMSE、MAE 等指标, 并保存到最后:

```
Res = function(model,model.label,tm , grid = NULL) {
MM=sum((model$pred[,2]-mean(model$pred[,2]))^2)
MSE=sum(apply(model$pred[,1:2],1,diff)^2)
rmse=sqrt(MSE)
nmse=sum(apply(model$pred[,1:2],1,diff)^2)/MM
rsquare=1-nmse
mae=sum(abs(apply(model$pred[,1:2],1,diff)))
perf.grid = NULL
if (is.null(grid)) {
perf.grid = data.frame(Predictor = c(model.label),
    NMSE=nmse,Rsquare=rsquare,
    RMSE =rmse, MAE=mae, time = c(tm[[3]]))} else {
.grid = data.frame(Predictor = c(model.label),
    NMSE=nmse,Rsquare=rsquare,
    RMSE =rmse, MAE=mae, time = c(tm[[3]]))
perf.grid = rbind(grid, .grid)
}
perf.grid
}
```

下面是主要代码使用的程序包 (只要你的计算机下载了, caret 会自动引用, 不用人工使用 library 引用):

```
M.pack=c("earth","ipred", "plyr", "e1071","mboost", "plyr", "import",
  "party", "xgboost", 'ranger','gbm','randomForest',  'foreach',
'quantregForest','RRF','inTrees', 'brnn','h2o')
```

主要代码如下:

```
tc_cv<- trainControl(method="cv", number=10, savePredictions = TRUE)
M.label=c('lm','treebag','bagEarth','bagEarthGCV','blackboost',
  'ctree','xgbTree','ranger','gbm','parRF','ranger','Rborist','rf','brnn',
  'gbm','kknn')

M.name=c("Linear Regression","Bagged CART","Bagged MARS",
  "Bagged MARS using gCV Pruning",
  "Boosted Tree", "Conditional Inference Tree",
  "eXtreme Gradient Boosting","Random Forest",
  "Stochastic Gradient Boosting", "Parallel Random Forest",
  "Random Forest (ranger)", "Random Forest(Rborist)", "Random Forest(rf)",
  "Bayesian Regularized Neural Networks", "Stochastic Gradient Boosting",
  "k-Nearest Neighbors")
length(M.name);length(M.label)
Model=list()
for(i in (1:length(M.label))){
  set.seed(669); ptm <- proc.time()
  Model[[i]]=train(cmedv~.,data = w,method=M.label[i],trControl=tc_cv)
  tm = proc.time() - ptm
  if(i==1)
    perf.grid = Res (Model[[i]],M.name[i],tm , grid = NULL)
  else
    perf.grid = Res (Model[[i]],M.name[i],tm , grid = perf.grid)
}
perf.grid
```

最终的结果输出为:

```
> (RR=perf.grid[order(perf.grid[,2],decreasing = T),])
                            Predictor      NMSE   Rsquare       RMSE        MAE   time
1                     Linear Regression 0.2769006 0.7230994  108.58086   1703.192  0.708
7             eXtreme Gradient Boosting 0.2297022 0.7702978 1027.74592 149934.582 14.510
3                          Bagged MARS 0.2287623 0.7712377  170.94019   4448.486 14.787
6            Conditional Inference Tree 0.2204393 0.7795607  167.80172   4465.628  1.419
2                          Bagged CART 0.1932147 0.8067853   90.70086   1372.664  1.741
16                 k-Nearest Neighbors 0.1861523 0.8138477  154.20062   3675.379  1.065
14 Bayesian Regularized Neural Networks 0.1849227 0.8150773  153.69049   4112.808  2.306
5                          Boosted Tree 0.1821114 0.8178886  264.16854  12286.469 10.231
4         Bagged MARS using gCV Pruning 0.1480328 0.8519672   79.39081   1218.099  5.911
9          Stochastic Gradient Boosting 0.1446443 0.8553557  235.43079  10821.341  1.951
15         Stochastic Gradient Boosting 0.1446443 0.8553557  235.43079  10821.341  1.900
8                        Random Forest 0.1297286 0.8702714  182.04751   6667.209  7.433
11               Random Forest (ranger) 0.1297286 0.8702714  182.04751   6667.209  7.252
13                   Random Forest(rf) 0.1207560 0.8792440  124.19564   3261.894 20.848
10               Parallel Random Forest 0.1207127 0.8792873  124.17336   3254.063 21.745
12               Random Forest(Rborist) 0.1111774 0.8888226  119.16812   3265.148 15.691
```

以上结果已经在本节开始的图3.8中展示. 产生图3.8的代码为:

```
RR=perf.grid[order(perf.grid[,2],decreasing = T),]
reg.names=RR[,1]
par(mar=c(4,9,2,1))
barplot(t(RR[,2]),cex.names=.5,horiz=TRUE,
  names.arg=reg.names, col=4,las=2)
title(paste0("NMSE for ", nrow(RR)," regression methods"))
```

3.7 多元多重回归回顾 *

多元多重回归并不是常用的回归教科书的重点, 这里仅仅给出一些类似于词典词条似的回顾. 仅供参考, 不必在课堂介绍.

名称问题

首先, 对各种可能造成混淆的回归名称做一说明. 假定我们有自变量 $X_1,\ldots,X_k$ 和因变量 $Y_1,\ldots,Y_m$, 那么:

- 当 $k>1$ 时的回归称为**多重回归** (multiple regression), 即使只有一个因变量及多个自变量时也这么称呼, 国内外也经常称其为 (和下面的混淆) **多元回归** (multi-variable regression; multivariate regression).
- 当 $m>1$ 时的回归称为**多元回归**, 指多个因变量回归.
- 当 $m>1, k>1$ 时的回归称为**多元多重回归** (multivariate multiple regression), 意味着多个因变量对多个自变量的回归.

多元回归的主要内容是在正态假定之下做各种检验, 特别是对涉及实验设计的回归结果的检验. 对于预测, 多元回归并不比一元回归更加优秀, 更不能与机器学习的回归比较预测精度. 因此, 我们不会过多回顾预测内容.

3.7.1 模型

对多个因变量的数据做多元多重线性回归所得到的各个自变量的系数与单独对每个因变量做通常的最小二乘回归所得到的系数是完全相同的, 而且它们的标准误差及 t 检验的 p 值也都相同, 然而单独的回归不会产生某些多元结果及某些系数检验.

考虑模型

$$\boldsymbol{Y}=\boldsymbol{X}\boldsymbol{B}+\boldsymbol{E}, \tag{3·7·1}$$

这里, $\boldsymbol{Y}(n\times m)$ 为有 n 个观测值及 m 个因变量的矩阵, 记 $\boldsymbol{y}_{(i)}$ 为 $\boldsymbol{Y}$ 的第 i 个列向量; $\boldsymbol{X}(n\times k)$ 为已知的矩阵, 代表 k 或 (在通常第一列包含常数 1 的情况下) $k-1$ 个自变量; $\boldsymbol{E}(n\times m)$ 为误差矩阵, 代表了模型所不能表达的部分, 记 $\boldsymbol{\epsilon}_{(i)}$ 为 $\boldsymbol{E}$ 的第 i 个列向量, 或记 $\boldsymbol{E}=(\boldsymbol{\epsilon}_1^\top,\boldsymbol{\epsilon}_2^\top,\ldots,\boldsymbol{\epsilon}_n^\top)^\top$, 即 $\boldsymbol{\epsilon}_i^\top$ 为 $\boldsymbol{E}$ 的第 i 行, 并假定 $\boldsymbol{\epsilon}_i^\top\sim N_m(\boldsymbol{0},\boldsymbol{\Sigma})$. 如果记 $\mathrm{vec}(\boldsymbol{E})=(\boldsymbol{\epsilon}_1^\top,\boldsymbol{\epsilon}_2^\top,\ldots,\boldsymbol{\epsilon}_n^\top)$, 则 $\mathrm{vec}(\boldsymbol{E})\sim N_{nm}(\boldsymbol{0},\boldsymbol{I}\otimes\boldsymbol{\Sigma})$, 这里 "$\otimes$" 为 Kronecker 乘积算子.

我们还有 $E(\boldsymbol{y}_{(i)}) = \boldsymbol{X}\boldsymbol{\beta}_{(i)}$. 式 (3.7.1) 也可以写成下面形式:

$$\begin{aligned}
&\begin{bmatrix} y_{11} & y_{12} & \cdots & y_{1m} \\ y_{21} & y_{22} & \cdots & y_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ y_{n1} & y_{n2} & \cdots & y_{nm} \end{bmatrix} = [\boldsymbol{y}_{(1)}, \boldsymbol{y}_{(2)}, \ldots, \boldsymbol{y}_{(m)}] \\
&= \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1k} \\ x_{21} & x_{22} & \cdots & x_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nk} \end{bmatrix} \begin{bmatrix} \beta_{11} & \beta_{12} & \cdots & \beta_{1m} \\ \beta_{21} & \beta_{22} & \cdots & \beta_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ \beta_{k1} & x_{k2} & \cdots & x_{km} \end{bmatrix} + \begin{bmatrix} \epsilon_{11} & \epsilon_{12} & \cdots & \epsilon_{1m} \\ \epsilon_{21} & \epsilon_{22} & \cdots & \epsilon_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ \epsilon_{n1} & \epsilon_{n2} & \cdots & \epsilon_{nm} \end{bmatrix} \\
&= [\boldsymbol{X}\boldsymbol{\beta}_{(1)}, \boldsymbol{X}\boldsymbol{\beta}_{(2)}, \ldots, \boldsymbol{X}\boldsymbol{\beta}_{(m)}] + [\boldsymbol{\epsilon}_{(1)}, \boldsymbol{\epsilon}_{(2)}, \ldots, \boldsymbol{\epsilon}_{(m)}] \\
&= E([\boldsymbol{y}_{(1)}, \boldsymbol{y}_{(2)}, \ldots, \boldsymbol{y}_{(m)}]) + [\boldsymbol{\epsilon}_{(1)}, \boldsymbol{\epsilon}_{(2)}, \ldots, \boldsymbol{\epsilon}_{(m)}].
\end{aligned}$$

3.7.2 最小二乘估计及对估计量的检验

估计量

记 $n \times n$ 矩阵 $\boldsymbol{P} = \boldsymbol{X}(\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top$, 参数矩阵 $\boldsymbol{B}$ 及 $\boldsymbol{\Sigma}$ 的最小二乘估计 (或者正态分布假定下的最大似然估计) 为:

$$\begin{aligned}
\hat{\boldsymbol{B}} &= (\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{Y}, \\
\hat{\boldsymbol{\Sigma}} &= n^{-1}\boldsymbol{Y}^\top(\boldsymbol{I} - \boldsymbol{P})\boldsymbol{Y},
\end{aligned}$$

而 $\boldsymbol{\Sigma}$ 的无偏估计为:

$$(n-k)^{-1}\boldsymbol{Y}^\top(\boldsymbol{I} - \boldsymbol{P})\boldsymbol{Y},$$

显然, $\hat{\boldsymbol{B}} = (\hat{\boldsymbol{\beta}}_1, \hat{\boldsymbol{\beta}}_2, \ldots, \hat{\boldsymbol{\beta}}_m)$ 的每一列都和单独的一元多重回归相同, 即

$$\hat{\boldsymbol{\beta}}_i = (\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{y}_{(i)},\ i = 1, 2, \ldots, m.$$

关于这些估计量, 有以下结论:

- $\hat{\boldsymbol{B}}$ 对 $\boldsymbol{B}$ 是无偏的.
- $\hat{\boldsymbol{E}} = (\boldsymbol{I} - \boldsymbol{P})\boldsymbol{Y}$, 而且 $E(\hat{\boldsymbol{E}}) = \boldsymbol{0}$.
- $\hat{\boldsymbol{B}}$ 和 $\hat{\boldsymbol{E}}$ 有多元正态分布.
- $\hat{\boldsymbol{B}}$ 独立于 $\hat{\boldsymbol{E}}$ 及 $\hat{\boldsymbol{\Sigma}}$.
- $\mathrm{Cov}(\hat{\beta}_i, \hat{\beta}_j) = \sigma_{ij}(\boldsymbol{X}^\top\boldsymbol{X})^{-1},\ i, j = 1, 2, \ldots, k.$
- $n\hat{\boldsymbol{\Sigma}} \sim \boldsymbol{W}_m(\boldsymbol{\Sigma}, n-k)$.

式中的 $\boldsymbol{W}_m$ 代表 Wishart 分布.[9]

与一元回归对总平方和分解为回归平方和与残差平方和类似, 多元线性回归也可以把

[9]如果矩阵 $\boldsymbol{M} = \boldsymbol{X}^\top\boldsymbol{X}$, 而 $\boldsymbol{X}(m \times k) \sim N_k(\boldsymbol{0}, \boldsymbol{\Sigma})$, 则 $\boldsymbol{M}(k \times k)$ 称为有尺度矩阵 $\boldsymbol{\Sigma}$ 及自由度 m 的 Wishart 矩阵. 当 $\boldsymbol{\Sigma} = \boldsymbol{I}_k$ 时称为标准 Wishart 分布.

总平方和及交叉乘积 (total sum-of-squares-and-cross-products, SSP) 矩阵 $SSP_T(m \times m)$ 分解成残差平方和 SSP_R 及回归平方和 SSP_{Reg}:

$$SSP_T = \boldsymbol{Y}^\top \boldsymbol{Y} - n\bar{\boldsymbol{y}}\bar{\boldsymbol{y}}^\top = \hat{\boldsymbol{E}}\hat{\boldsymbol{E}}^\top + (\hat{\boldsymbol{Y}}^\top \hat{\boldsymbol{Y}} - n\bar{\boldsymbol{y}}\bar{\boldsymbol{y}}^\top) = SSP_R + SSP_{Reg}.$$

关于估计量的检验

在一元回归中, 往往有诸如 $H_0 : \boldsymbol{L}^\top \boldsymbol{\beta} = C$ 或 $H_0 : \boldsymbol{L}\boldsymbol{\beta} - C = \boldsymbol{0}$ 的检验, 比如 $\boldsymbol{\beta} = (\beta_1, \beta_2, \beta_3)^\top$, $\boldsymbol{L} = (2, -1, -1)^\top$, $C = 0$, 这意味着检验 $H_0 : 2\beta_1 = \beta_2 + \beta_3$. 我们有方差分析表中利用各种平方和的 F 检验. 在多元回归中也类似, 比如要检验:

$$H_0 : \boldsymbol{L}\boldsymbol{B} = \boldsymbol{C}, \tag{3.7.2}$$

这里, $\boldsymbol{L}$ 为 $q \times k$ 阶, $\boldsymbol{B}$ 为 $k \times m$ 阶, $\boldsymbol{C}$ 为 $q \times m$ 阶. 那么用于检验的 SSP_H 为:

$$SSP_H = (\hat{\boldsymbol{B}}^\top \boldsymbol{L}^\top - \boldsymbol{C}^\top) \left[\boldsymbol{L}(\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{L}^\top\right]^{-1} (\boldsymbol{L}\hat{\boldsymbol{B}} - \boldsymbol{C})$$

检验统计量为 $SSP_H SSP_{Reg}^{-1}$.

对于一般的 SSP 增量假设 SSP_H 来说, F 检验基于矩阵 $SSP_H SSP_{Reg}^{-1}$ 的 m 个特征值 $\lambda_1, \lambda_2, \ldots, \lambda_m$. 常用的统计量为:

- Pillai-Bartlett 迹检验统计量:

$$T_{PB} = \sum_{j=1}^{m} \frac{\lambda_j}{1 - \lambda_j}.$$

- Hotelling-Lawley 迹检验统计量

$$T_{HL} = \sum_{j=1}^{m} \lambda_j.$$

- Wilks's Lambda 检验统计量:

$$\Lambda = \prod_{j=1}^{m} \frac{1}{1 + \lambda_j}.$$

- Roy 最大根检验统计量: λ_1.

3.7.3 多元回归检验例子

我们不想涉及太多的数学细节, 但上面一小节的内容必须能够在数据分析中实现, 本小节仅仅涉及少数检验例子, 旨在让读者熟悉 R 软件可以做的事情. 相信读者可以在任何标准的回归分析教科书中找到这些检验 (或其延伸) 的各种公式和理论细节.

这里, 我们使用例8.1的橄榄油数据, 该数据由通过一组传感器获得的关于 16 种橄榄油的 5 个属性以及 6 个物理化学质量参数的 11 个变量组成, 由传感器获得的 5 个变量为 Acidity、Peroxide、K232、K270、DK, 而 6 个物理化学质量参数变量为 yellow、green、

brown、glossy、transp、syrup. 这 16 种橄榄油中的前 5 种产自希腊 (Greece), 中间 5 种产自意大利 (Italy), 最后 6 种产自西班牙 (Spain). 我们试图用 K232、DK 和 brown 作为因变量, nation 作为自变量进行多元回归.

以 nation 作为标识的 K232、DK 和 brown 的散点图展示在图3.17中. 产生图3.17使用了程序包 car[10], 代码 (包括读取数据及增加产地变量 nation) 为:

```
w=read.csv("oliveoil.csv")
w$nation=factor(c(rep("Greece",5),rep("Italy",5),rep("Spain",6)))
library(car)
scatterplotMatrix(~K232+brown+DK|nation,data=w, smooth=FALSE,
reg.line=FALSE, ellipse=TRUE, by.groups=TRUE, diagonal="none")
```

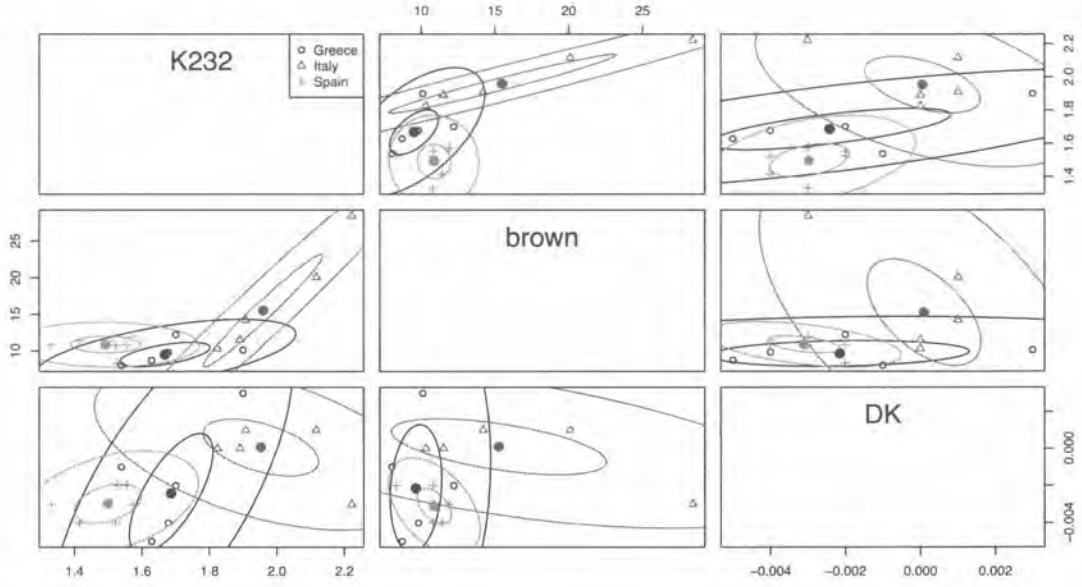

图 3.17 例8.1三个变量 K232、DK 和 brown 的散点图 (以 nation 为标识)

下面用 K232、DK 和 brown 作为因变量, nation 作为自变量进行多元回归:

```
a=lm(cbind(K232, DK, brown)~nation,data=w)
summary(a)
```

输出为三个单独的一元回归结果:

```
Response K232 :

Call:
lm(formula = K232 ~ nation, data = w)
```

[10]John Fox and Sanford Weisberg (2011). *An R Companion to Applied Regression*, Second Edition. Thousand Oaks CA: Sage. URL: http://socserv.socsci.mcmaster.ca/jfox/Books/Companion.

```
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.68940    0.05939  28.447 4.29e-13 ***
nationItaly  0.30300    0.08399   3.608  0.00318 **
nationSpain -0.20223    0.08041  -2.515  0.02585 *

Residual standard error: 0.1328 on 13 degrees of freedom
Multiple R-squared:  0.753,Adjusted R-squared:  0.715
F-statistic: 19.81 on 2 and 13 DF,  p-value: 0.000113

Response DK :

Call:
lm(formula = DK ~ nation, data = w)

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.0018000  0.0009081  -1.982    0.069 .
nationItaly  0.0016000  0.0012842   1.246    0.235
nationSpain -0.0012000  0.0012296  -0.976    0.347

Residual standard error: 0.002031 on 13 degrees of freedom
Multiple R-squared:  0.2853,Adjusted R-squared:  0.1754
F-statistic: 2.595 on 2 and 13 DF,  p-value: 0.1126

Response brown :

Call:
lm(formula = brown ~ nation, data = w)

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   9.7600     1.9236   5.074 0.000213 ***
nationItaly   7.1400     2.7203   2.625 0.020998 *
nationSpain   0.9067     2.6045   0.348 0.733327

Residual standard error: 4.301 on 13 degrees of freedom
Multiple R-squared:  0.3904,Adjusted R-squared:  0.2967
F-statistic: 4.163 on 2 and 13 DF,  p-value: 0.04005
```

下面做零假设为三个因变量均值相等的方差分析 (F 检验):

```
> (manova.w=Anova(a))

Type II MANOVA Tests: Pillai test statistic
       Df test stat approx F num Df den Df   Pr(>F)
nation  2    1.0915   4.8059      6     24 0.002375 **
```

做汇总输出:

```
> summary((manova.w))

Type II MANOVA Tests:

Sum of squares and products for error:
           K232         DK      brown
K232  0.2292552  0.0006030   5.305713
DK    0.0006030  0.0000536  -0.034060
brown 5.3057133 -0.0340600 240.505333

------------------------------------------

Term: nation

Sum of squares and products for the hypothesis:
           K232        DK      brown
K232  0.6987498 0.0038650   8.941462
DK    0.0038650 0.0000214   0.048535
brown 8.9414617 0.0485350 154.049042

Multivariate Tests: nation
                 Df test stat  approx F num Df den Df     Pr(>F)
Pillai            2  1.091523  4.805942      6     24 0.00237523 **
Wilks             2  0.145627  5.941708      6     22 0.00082583 ***
Hotelling-Lawley  2  4.238370  7.063950      6     20 0.00038061 ***
Roy               2  3.811070 15.244280      3     12 0.00021436 ***
```

下面检验零假设: Spain 和 Italy 的加权平均 (权重 0.4、0.6) 等于 Greece.

```
> linearHypothesis(a,"0.4*nationSpain+0.6*nationItaly",verbose=TRUE)

Hypothesis matrix:
                                (Intercept) nationItaly nationSpain
0.4*nationSpain+0.6*nationItaly           0         0.6         0.4

Right-hand-side matrix:
```

```
                                K232 DK brown
0.4*nationSpain+0.6*nationItaly    0  0     0

Estimated linear function (hypothesis.matrix %*% coef - rhs):
     K232        DK     brown
0.1009067 0.0004800 4.6466667


Sum of squares and products for the hypothesis:
              K232           DK        brown
K232   0.0340920381 1.621714e-04  1.569909524
DK     0.0001621714 7.714286e-07  0.007467857
brown  1.5699095238 7.467857e-03 72.293005952

Sum of squares and products for error:
           K232         DK      brown
K232   0.2292552  0.0006030   5.305713
DK     0.0006030  0.0000536  -0.034060
brown  5.3057133 -0.0340600 240.505333

Multivariate Tests:
                 Df test stat approx F num Df den Df  Pr(>F)
Pillai            1 0.3050595 1.609565      3     11 0.24337
Wilks             1 0.6949405 1.609565      3     11 0.24337
Hotelling-Lawley  1 0.4389722 1.609565      3     11 0.24337
Roy               1 0.4389722 1.609565      3     11 0.24337
```

还可以做零假设为 Spain 和 Italy 相等的检验:

```
> linearHypothesis(a,"*nationSpain=nationItaly",verbose=TRUE)

Hypothesis matrix:
                        (Intercept) nationItaly nationSpain
*nationSpain=nationItaly           0          -1           1

Right-hand-side matrix:
                        K232 DK brown
*nationSpain=nationItaly    0  0     0

Estimated linear function (hypothesis.matrix %*% coef - rhs):
      K232         DK      brown
-0.5052333 -0.0028000 -6.2333333
```

```
Sum of squares and products for the hypothesis:
             K232           DK      brown
K232   0.696165603 3.858145e-03   8.588967
DK     0.003858145 2.138182e-05   0.047600
brown  8.588966667 4.760000e-02 105.966667

Sum of squares and products for error:
           K232         DK       brown
K232   0.2292552  0.0006030   5.305713
DK     0.0006030  0.0000536  -0.034060
brown  5.3057133 -0.0340600 240.505333

Multivariate Tests:
                 Df test stat approx F num Df den Df     Pr(>F)
Pillai            1  0.790586 13.84249      3     11 0.00047173 ***
Wilks             1  0.209414 13.84249      3     11 0.00047173 ***
Hotelling-Lawley  1  3.775226 13.84249      3     11 0.00047173 ***
Roy               1  3.775226 13.84249      3     11 0.00047173 ***
```

3.8 本章计算的 Python 代码

首先, 输入需要的一些基本模块:

```
import numpy as np
import matplotlib
matplotlib.use('TkAgg')#使用Windows不需要这个命令
import matplotlib.pyplot as plt
%matplotlib inline
import scipy.stats as stats
import pandas as pd
```

设定工作目录:

```
import os
os.getcwd()
os.chdir('/work/')
```

3.8.1 例3.1波士顿住房数据的线性回归

输入数据并画出成对散点图 (见图3.18):

```
w=pd.read_csv('BostonHousing2.csv')
wn0=w.columns
```

```
wn=wn0[5:] #不选前面不参与建模的5个变量
f=plt.figure(figsize=(16, 8))
k=0
for i in range(len(wn)):
    for j in range(len(wn)):
        k=k+1
        if i!=j:
            f.add_subplot(len(wn),len(wn),k)
            plt.scatter(w[wn[i]],w[wn[j]])
            #plt.axis('off') #不显示坐标轴和边框
        else:
            f.add_subplot(len(wn),len(wn),k)
            plt.scatter([0,1],[0,1])
            plt.text(.5,.5, wn[i], \
            ha='center', va='center',size=10) #打印变量名字
```

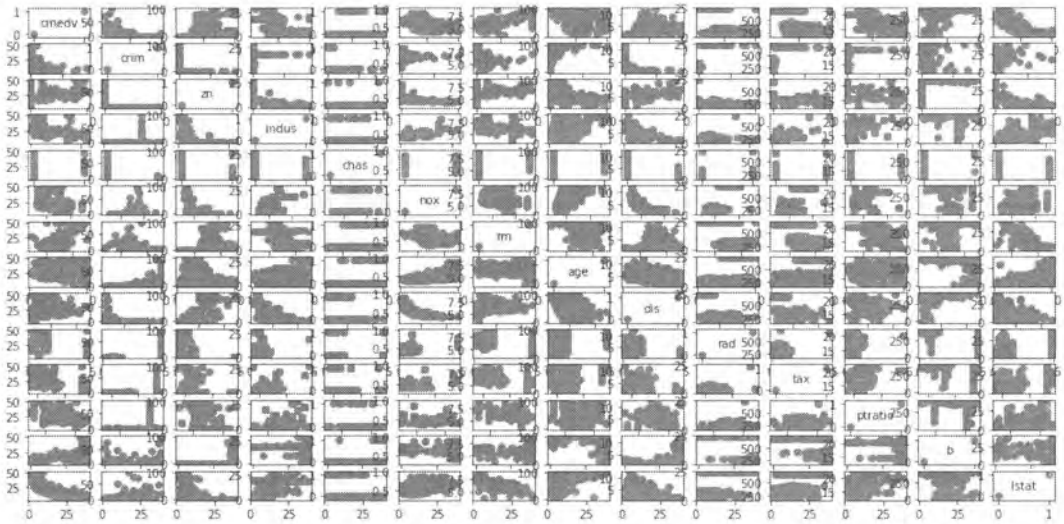

图 3.18 例3.1波士顿住房数据的成对散点图

确定因变量和自变量, 并使用模块 statsmodels.api, 用最小二乘回归拟合数据:

```
y=np.array(w[wn[0]])[:,np.newaxis] #转换成列向量
X=np.array(w[wn[1:]])
import statsmodels.api as sm
mod=sm.OLS(y,X)
res=mod.fit()
print (res.summary())
```

回归结果汇总打印输出为:

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.960
Model:                            OLS   Adj. R-squared:                  0.959
Method:                 Least Squares   F-statistic:                     906.3
Date:                Thu, 17 May 2018   Prob (F-statistic):               0.00
Time:                        10:08:15   Log-Likelihood:                -1519.5
No. Observations:                 506   AIC:                             3065.
Df Residuals:                     493   BIC:                             3120.
Df Model:                          13
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
x1            -0.0911      0.034     -2.670      0.008      -0.158      -0.024
x2             0.0500      0.014      3.502      0.001       0.022       0.078
x3            -0.0013      0.064     -0.020      0.984      -0.127       0.124
x4             2.8586      0.896      3.189      0.002       1.097       4.620
x5            -2.8802      3.331     -0.865      0.388      -9.424       3.664
x6             5.9026      0.307     19.256      0.000       5.300       6.505
x7            -0.0074      0.014     -0.538      0.591      -0.034       0.020
x8            -0.9960      0.194     -5.134      0.000      -1.377      -0.615
x9             0.1692      0.066      2.556      0.011       0.039       0.299
x10           -0.0098      0.004     -2.512      0.012      -0.017      -0.002
x11           -0.3647      0.109     -3.347      0.001      -0.579      -0.151
x12            0.0148      0.003      5.538      0.000       0.010       0.020
x13           -0.4225      0.050     -8.389      0.000      -0.521      -0.324
==============================================================================
Omnibus:                      210.398   Durbin-Watson:                   1.005
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             1456.932
Skew:                           1.660   Prob(JB):                         0.00
Kurtosis:                      10.621   Cond. No.                     8.50e+03
==============================================================================

Warnings:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 8.5e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
```

下面用另一个模块 (sklearn) 做上述回归 (不用截距项), 并画出残差对拟合值图和残差的 Q-Q 图 (见图3.19):

```
from sklearn import linear_model
regr = linear_model.LinearRegression(fit_intercept=False) #不用常数项
regr.fit(X, y)
print(regr.coef_) #输出估计的系数
#yhat=X.dot(regr.coef_.reshape(10,1)) #直接计算拟合值
#resid=y-yhat #直接计算残差
res=y-regr.predict(X) #计算残差
import scipy.stats as stats
import pylab
res.shape=res.shape[0] #样本量
```

```
f=plt.figure(figsize=(12,5))
f.add_subplot(121)
plt.scatter(regr.predict(X),res) #残差对拟合值图
plt.plot(regr.predict(X),np.ones(len(y)))
plt.xlabel('Fitted values')
plt.ylabel('Residuals')
f.add_subplot(122)
stats.probplot(res, dist="norm", plot=pylab) #残差的正态Q-Q图
```

输出的系数 (没有截距, 因此验证了上面模块回归的输出) 估计为:

```
[[ -9.11218348e-02   5.00126673e-02  -1.30708588e-03   2.85858964e+00
   -2.88024218e+00   5.90258783e+00  -7.36751413e-03  -9.95959986e-01
    1.69186398e-01  -9.77330608e-03  -3.64702252e-01   1.48089321e-02
   -4.22468608e-01]]
```

注: 如果需要截距项 (常数项), 则可以通过在自变量矩阵 X 前面加上一列 1(形成新的自变量矩阵 X1) 来实现有截距项的回归 (代码相同):

```
X1=np.insert(X,0,1, axis=1)#在0列(axis=1意味着列)加上1
```

图3.19为例3.1波士顿住房数据的残差对拟合值图 (左) 及残差的正态 Q-Q 图 (右). 该图表明残差显示极大地偏离正态分布, 因此所有输出的检验都没有多少意义.

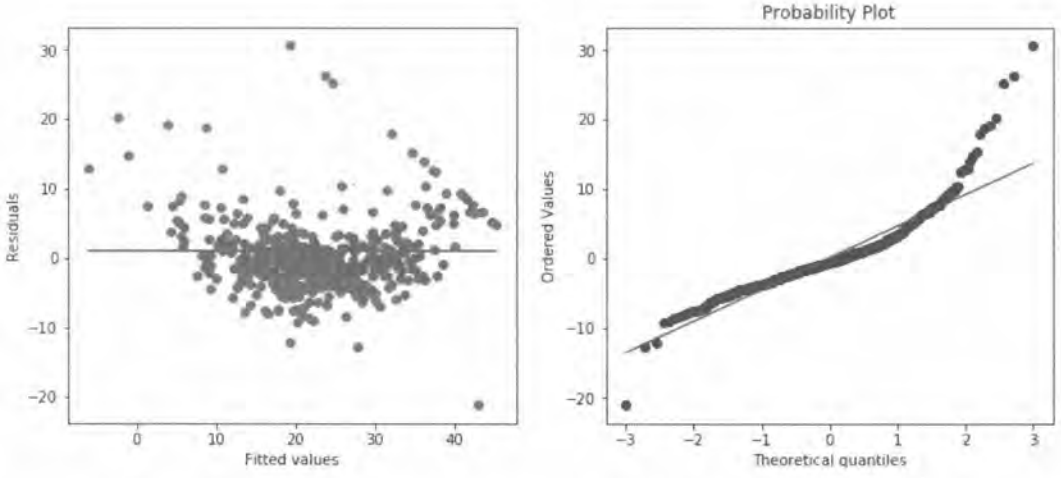

图 3.19 例3.1波士顿住房数据回归的残差对拟合值图 (左) 及残差的正态 Q-Q 图 (右)

3.8.2 例3.1波士顿住房数据的决策树回归、bagging 回归和随机森林回归

决策树回归

输入数据:

```
u=pd.read_csv('BostonHousing2.csv')
un0=u.columns
un=un0[5:] #不选前面不参与建模的5个变量
y=np.array(u[un[0]])[:,np.newaxis]
X=np.array(u[un[1:]])
print(un[0]);print(un[1:])
```

输入需要的模块:

```
from sklearn.tree import DecisionTreeRegressor
from sklearn import tree
from sklearn.model_selection import cross_val_score
import graphviz #为画图
```

做决策树并画图 (见图3.20):

```
regr = DecisionTreeRegressor(max_depth=4,random_state=1010)
regr=regr.fit(X,y)
dot_data = tree.export_graphviz(regr,feature_names=un[1:],out_file=None)
graph = graphviz.Source(dot_data)
graph.render("Bostonpy0") #输出决策树图文件:Bostonpy0.pdf
graph
```

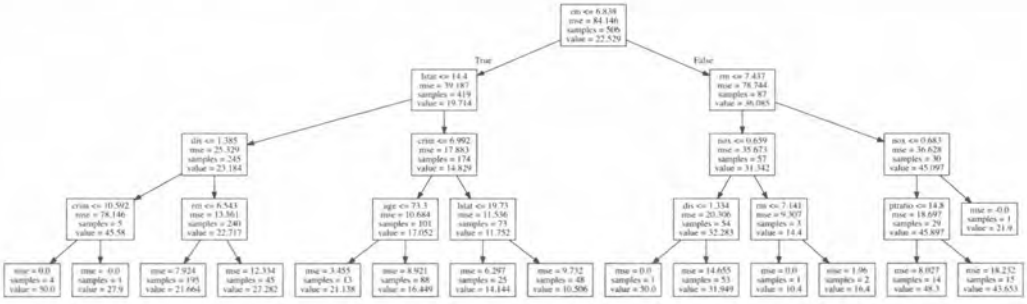

图 3.20 例3.1波士顿住房数据回归决策树图

生成决策树变量重要性图 (见图3.21):

```
f=plt.figure(figsize=(12,5))
f.add_subplot(111)
height = regr.feature_importances_
bars = un[1:]
y_pos = np.arange(len(bars))
plt.bar(y_pos, height)
plt.xticks(y_pos, bars, color='green')#变量名字
plt.yticks(color='green')
#plt.show()
```

当然, 还可以得到其他结果, 比如得分 (score)(R^2), 计算全部数据集 R^2 的代码为:

```
reg.score(X,y)
```

得到 $R^2 = 0.8690$. 这种得分不是交叉验证的, 如果 `X_test` 和 `y_test` 是测试集, 那么产生交叉验证 R^2 的代码为 `reg.score(X_test,y_test)`.

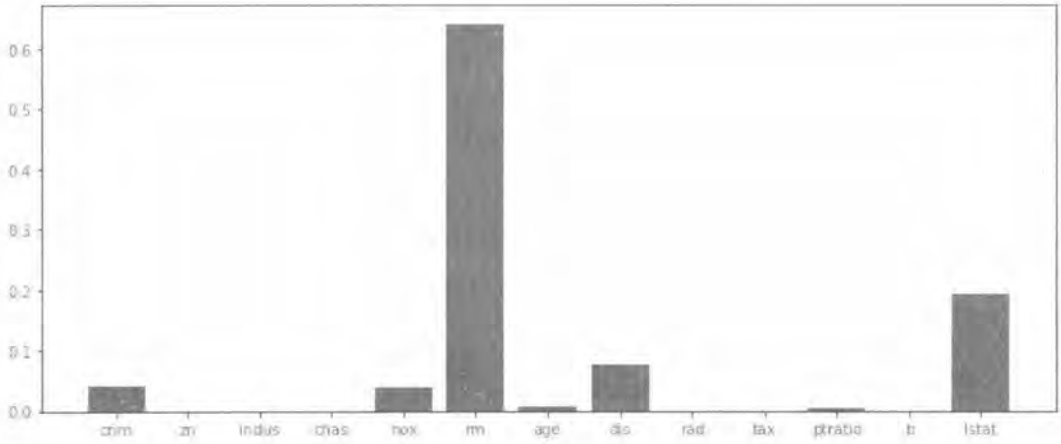

图 3.21 例3.1波士顿住房数据决策树回归变量重要性图

bagging 回归

bagging 回归代码和决策树的代码类似:

```
from sklearn.ensemble import BaggingRegressor
regr = BaggingRegressor(n_estimators=100,oob_score=True,random_state=1010)
regr.fit(X, y)
print("Score:\n",regr.score(X,y))
```

输出的得分 (即 R^2) 为 0.98385, 这种得分不是交叉验证的. 交叉验证得分要比这个小, OOB 交叉验证得分 (R^{*2}) 可以用代码 `regr.oob_score_` 得到, 等于 0.88222.

随机森林回归

随机森林回归代码和决策树很相似:

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.datasets import make_regression
regr = RandomForestRegressor(oob_score=True,random_state=1010)
regr.fit(X, y)
print("Variable importance:\n",regr.feature_importances_)
print("Score:\n",regr.oob_score_)
```

输出的变量重要性及 OOB 交叉验证得分 R^2 为:

```
Variable importance:
[ 0.03251241  0.00122295  0.00667047  0.00342961  0.02106578  0.27725915
  0.01432973  0.0697463   0.00251973  0.01098063  0.01442163  0.01158948
  0.53425212]
```

```
Score:
0.771396153829
```

画出变量重要性图 (使用和前面决策树点图完全相同的代码)(见图3.22).

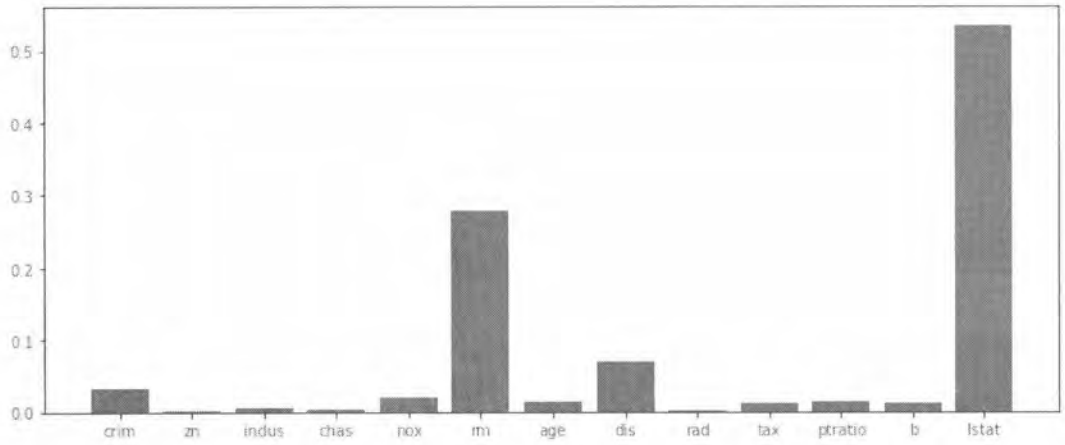

图 3.22 例3.1波士顿住房数据随机森林回归变量重要性图

多种回归的交叉验证

输入必要的模块:

```
import numpy as np
import matplotlib
matplotlib.use('TkAgg')#使用Windows不需要这个命令
import matplotlib.pyplot as plt
%matplotlib inline
import scipy.stats as stats
import pandas as pd
```

构造一个函数以方便计算:

```
def FCV(x,y,regr, cv=10,seed=1010):
    np.random.seed(seed)
    ind=np.arange(len(y))
    np.random.shuffle(ind) #随机化下标
    X_folds = np.array_split(x[ind], cv)
    y_folds = np.array_split(y[ind], cv)
    X2=np.empty((0,X.shape[1]), float)
    y2=np.empty((0,y.shape[0]), float)
    yp=np.empty((0,y.shape[0]), float)
    for k in range(cv):
        X_train = list(X_folds) #X_fold实际上已经是list
        X_test  = X_train.pop(k) #只有list才能pop,从中取出第k份
```

```
        X_train = np.concatenate(X_train) #合并剩下的cv-1份
        y_train = list(y_folds)
        y_test  = y_train.pop(k)
        y_train = np.concatenate(y_train)
        regr.fit(X_train,y_train.ravel()) #拟合选中的regr模型
        y2=np.append(y2,y_test)
        X2=np.append(X2,X_test)
        yp=np.append(yp,regr.predict(X_test))
    nmse=np.sum((y2-yp)**2)/np.sum((y2-np.mean(y2))**2)
    r2=1-nmse
    return(np.array([nmse, r2]))
```

输入所需的另外一些模块以及计算的代码:

```
from sklearn.neighbors import   KNeighborsRegressor
from sklearn.linear_model import  LassoLars, Lars,Lasso,\
LinearRegression
from sklearn.ensemble import AdaBoostRegressor, \
RandomForestRegressor,BaggingRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVR
```

我们这里使用了多种回归方法, 包括最近邻方法、lasso lars、lars、lasso、线性回归、决策树、随机森林、adaboost、bagging、支持向量机 (SVM), 用这些方法做交叉验证并且画图 (见图3.23) 的代码为:

```
names = ["N-Neighbors",
    "Lasso Lars", "Lars",  "Lasso",
    "Linear Regression", "Decision Tree",
    "Random Forest", "AdaBoost", "bagging", "SVR"]
regressors = [
    KNeighborsRegressor(),
    LassoLars(), Lars(), Lasso(), LinearRegression(),
    DecisionTreeRegressor(),
    RandomForestRegressor(n_estimators=500, oob_score=True),
    AdaBoostRegressor(DecisionTreeRegressor(max_depth=4),\
        n_estimators=300),
    BaggingRegressor(), SVR(C=1.0, epsilon=0.2)]

# 在各种回归方法上迭代
A=np.empty((0,2), float)
for reg in regressors:
    tt=np.array(FCV(X,y,reg,10));tt.shape=(1,2)
    A=np.append(A,tt,0) #把各种方法的回归结果合并
```

```
fig=plt.figure(figsize=(20,10))
ax=fig.add_subplot(121)
ax.bar(np.arange(np.array(A).shape[0]), np.array(A)[:,0])
ax.set_xticklabels(names) #标注回归方法
fig.autofmt_xdate() #回归方法标注斜放
ax.set_ylabel('NMSE')
ax.set_title('NMSE')
ax.set_xticks(np.arange(np.array(A).shape[0])+ 0.35)
bx=fig.add_subplot(122)
bx.bar(np.arange(np.array(A).shape[0]), np.array(A)[:,1])
bx.set_xticklabels(names)
fig.autofmt_xdate()
bx.set_ylabel('Score')
bx.set_title('Score')
bx.set_xticks(np.arange(np.array(A).shape[0])+ 0.35)
```

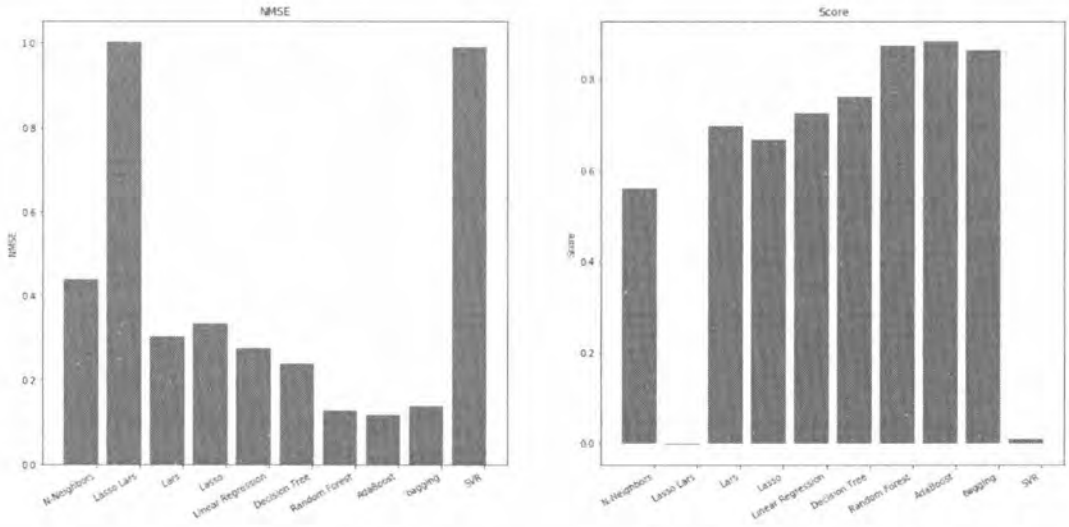

图 3.23 例3.1波士顿住房数据 10 折交叉验证图, 左图为标准化均方误差 (NMSE), 右图为 R^2 (Score)

3.8.3 把分类变量转换成哑元变量

Python 的回归和分类函数大都不会自动处理分类变量, 必须由用户转换分类变量为哑元变量. 这里介绍如何用 Python 做这种转换, 使用的是前面用过的数据 Artif0.csv.

读入数据及展示原数据

读入数据及展示原数据的代码:

```
import pandas as pd
v=pd.read_csv('Artif0.csv')
print(v)
```

原数据为:

```
       Age   Edu    Income Sex
0      Old  Grad  520000.0   F
1   Middle  Coll   51000.0   F
2    Young    Hs    1200.0   F
3      Old   Ele    1500.0   F
4   Middle  Grad  200000.0   M
5    Young  Coll   15000.0   M
6      Old    Hs    2000.0   M
7   Middle   Ele    1100.0   F
8    Young  Grad  150000.0   M
9      Old  Coll   10000.0   F
10  Middle    Hs    3000.0   F
11   Young   Ele     500.0   M
```

转换成哑元的完整结果

转换成哑元的代码如下, 这里每个变量的转换结果都是共线的 (相加得到全是 1 的向量):

```
v1=pd.get_dummies(v, drop_first=False)
print(v1)
```

转换后的数据为:

```
      Income  Age_Middle  Age_Old  Age_Young  Edu_Coll  Edu_Ele  Edu_Grad  \
0   520000.0           0        1          0         0        0         1
1    51000.0           1        0          0         1        0         0
2     1200.0           0        0          1         0        0         0
3     1500.0           0        1          0         0        1         0
4   200000.0           1        0          0         0        0         1
5    15000.0           0        0          1         1        0         0
6     2000.0           0        1          0         0        0         0
7     1100.0           1        0          0         0        1         0
8   150000.0           0        0          1         0        0         1
9    10000.0           0        1          0         1        0         0
10    3000.0           1        0          0         0        0         0
11     500.0           0        0          1         0        1         0

    Edu_Hs  Sex_F  Sex_M
0        0      1      0
1        0      1      0
2        1      1      0
3        0      1      0
```

```
4       0       0       1
5       0       0       1
6       1       0       1
7       0       1       0
8       0       0       1
9       0       1       0
10      1       1       0
11      0       0       1
```

转换成哑元并舍弃每个自变量的第一个水平的结果

由于上面得到的数据 (v) 是共线的, 对于诸如经典线性回归不能运算, 必须增加约束条件. 最简单的约束条件是每个自变量舍弃一个哑元变量, 转换成哑元并舍弃每个自变量第一个哑元变量的代码, 仅仅在前面代码加上一个选项 `drop_first=True`(默认值为 `drop_first=False`).

```
v2=pd.get_dummies(v, drop_first=True)
print(v2)
```

转换后的数据为:

```
      Income  Age_Old  Age_Young  Edu_Ele  Edu_Grad  Edu_Hs  Sex_M
0   520000.0        1          0        0         1       0      0
1    51000.0        0          0        0         0       0      0
2     1200.0        0          1        0         0       1      0
3     1500.0        1          0        1         0       0      0
4   200000.0        0          0        0         1       0      1
5    15000.0        0          1        0         0       0      1
6     2000.0        1          0        0         0       1      1
7     1100.0        0          0        1         0       0      0
8   150000.0        0          1        0         1       0      1
9    10000.0        1          0        0         0       0      0
10    3000.0        0          0        0         0       1      0
11     500.0        0          1        1         0       0      1
```

3.9 习 题

1. 考虑线性回归模型 (3.1.3) 的最小二乘回归. 请问:
   (1) 残差是误差项 ϵ 的估计值吗?
   (2) 误差 ϵ 和 $\boldsymbol{X}$ 张成的平面正交吗?
   (3) 残差和 $\boldsymbol{X}$ 张成的平面正交吗?
   (4) 残差可以看成是误差项 ϵ 的实现值吗?
   (5) 有截距项和没有截距项回归的 ($\boldsymbol{y}$ 的) 拟合值 $\hat{\boldsymbol{y}}$ 是否相同?

(6) 在误差独立同正态分布的假定下, 有截距项和没有截距项的 (非截距) 系数大小不同, 但它们相应 t 检验的 p 值是否相同?

(7) 没有误差分布假定时, 如何判断该回归的拟合好坏? 能不能使用 t 检验或 F 检验?

(8) 是否假定误差分布对回归的拟合结果或系数估计有影响? 如果否, 误差分布假定是出于什么目的而设定的?

2. 考虑下面的问题:

(1) 检验统计量是从数学 (包括概率论) 假定推导出来的, 还是根据目的构造的?

(2) 确定了检验统计量, 在对数据的某些假定下, 根据数据得到的 p 值是推导出来的数学结果吗?

(3) 由算出的 p 值导致的是否 "显著" 的结论是数学结论还是主观决策?

(4) 损失函数是推导出来的数学结论还是人们创造的函数?

(5) 线性回归的 "线性" 形式是数学推导出来的吗?

3. 对于回归模型 (3.1.2) 岭回归在约束条件 $\sum_{j=1}^{p}\beta_j^2 \leqslant s$ 下, 系数满足下面条件:

$$\hat{\boldsymbol{\beta}}^{(ridge)} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j\right)^2.$$

而 lasso 回归在约束条件 $\sum_{j=1}^{p}|\beta_j| \leqslant s$ 下, 系数需要满足下面的条件:

$$\hat{\boldsymbol{\beta}}^{(lasso)} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j\right)^2.$$

请问, 这样得到的系数估计和没有约束条件的通常最小二乘回归的系数估计一样吗?

4. 用下面代码产生的因变量 y 和自变量 x 做最小二乘回归:

```
set.seed(1010);x=rnorm(50);y=rnorm(50);x[51]=50;y[51]=50
a=lm(y~x);summary(a)
```

根据输出的结果做数据的散点图和拟合直线. 请问:

(1) 得到的对斜率的 t 检验是否显著? 为什么?

(2) 这个回归有意义吗? 为什么?

(3) 对残差做正态性检验 (用 `shapiro.test(a$residuals)`), 你能得出结论吗? 你能得出什么结论?

(4) 讨论对残差做的正态 Q-Q 图 (`qqnorm(a$residuals)`).

(5) 对这个问题做个总结.

5. 一些人喜欢用国家统计局公布的 31 个省市自治区数据的一些汇总变量做回归, 每个省市自治区的数据值是一个观测值, 因此, 样本量为 31, 而且假定独立同正态分布. 请问:

(1) 31 个省市自治区的各个变量的数值是不是来自某个总体? 如果是, 总体是什么? 还可以抽样吗?

(2) 31 个省市自治区的数据是不是独立的? 是不是同分布的? 是不是正态分布的? 它们是不是样本?

(3) 讨论整个问题.

6. 考虑一个自变量的简单回归模型 $y=\alpha+\beta x+\epsilon$, 其斜率的 t 检验 ($H_0:\beta=0\Leftrightarrow\beta\neq 0$) 和相关系数检验 ($H_0:r=0\Leftrightarrow r\neq 0$) 完全一样. 后者也是 t 检验, 称为 Pearson 乘积矩相关检验 (Pearson's product-moment correlation), 可以用代码 `cor.test(x,y)` 实行.
   (1) 请核对上述斜率的 t 检验和相关系数为 0 的 t 检验等价.
   (2) 如果 x 和 y 的 Pearson 乘积矩相关检验显著, 则认为相关系数 $r\neq 0$. 如果 $r=0.01$, 当然应该拒绝零假设, 但你认为 x 和 y 相关吗? 这个相关系数是否为 0 的检验和是否相关等价吗?
   (3) 如果回归中得到关于斜率的 t 检验显著 (及 $\beta\neq 0$), 你能说明回归有意义吗?
   (4) 讨论整个问题, 包括确定 p 值多小算是显著的依据问题.

7. 考虑下面关于定性 (分类) 自变量的问题:
   (1) 为什么定性自变量的各个水平的效应不可估计?
   (2) 在约束条件下得到的各个水平的 "效应" 的 "估计" 说明什么?
   (3) 对于定性变量各个水平的 t 检验有意义吗?
   (4) 如果改变约束条件 (比如改变等于 0 的水平), 这些 t 检验的 p 值相同吗?
   (5) 方差分析表中对于定性自变量的 F 检验如果显著, 它说明了什么?

8. 为什么最小二乘回归的观测值数目必须多于自变量的个数?

9. 下载**最小一乘回归** (minimum absolute residual regression; L1 regression) 程序包 `L1pack`, 考虑 R 自带的数据 `stackloss`(用 `stack.loss` 作为因变量, `stack.x` 作为自变量).
   (1) 用程序包 `L1pack` 的函数 `l1fit` 拟合数据 `stackloss`, 这是最小一乘回归.
   (2) 对数据 `stackloss` 做最小二乘回归 (用函数 `lm`).
   (3) 比较这两个回归的残差平方和及两个回归的绝对值平方和, 并做出评论.
   (4) 对上面的两组残差做正态性检验 (使用函数 `shapiro.test`), 你能得到任何结论吗 (参考检验 `shapiro.test(1:21)` 的结果)?
   (5) 删除数据中的第 1、3、4、21 个观测值后, 再重复上面的工作, 并把两组残差画在一张图中.

10. 对例3.1波士顿住房数据做线性回归, 算出其 10 折交叉验证的 MSE 和 R^{*2}, 并且和没有测试集 (原数据本身既是训练集又是测试集) 的 MSE 和 R^2 做比较.

11. 对 R 自带的 `mtcars` 数据做线性回归、决策树回归及随机森林回归.
   (1) 通过 10 折交叉验证比较三个回归的预测精度.
   (2) 画出后两个回归的变量重要性图, 它们有什么不同?
   (3) 画出随机森林回归的局部变量重要性图.
   (4) 画出随机森林回归的部分依赖性图.

第 4 章　分　类

4.1　目的和基本概念

4.1.1　目的和例子

和回归分析类似, 分类是统计学习中的有指导 (或有监督) 学习之一. 有指导学习名称中的 "指导" 意味着有一个因变量作为目标, 使得训练数据集有一个指导性的目标. 然后用训练集的自变量和因变量 "学习"(或训练) 得出一个模型, 并且用该模型对新数据或者测试集中的因变量的值做预测. 当因变量是数量变量时, 称为回归, 而当因变量是分类变量 (定性变量) 时, 称为分类, 经典统计中, 分类被称为**判别分析** (discriminant analysis). 注意第7章介绍的聚类是无指导学习, 不包括任何因变量.

和回归问题相比, 同样重要的分类问题在经典统计中却相对被冷落, 这不是因为分类不重要, 而是现有的数学工具对分类问题力不从心. 因此, 在模型驱动的思维模式下, 经典统计中对分类问题的研究基本上还停留在 80 年前的水平, 只有在 "多元分析" 之下的判别分析是关于多水平因变量分类的, 对于两水平分类, 经典统计主要用 **Logistic 回归**方法, 但在传统统计教科书中的 Logistic 回归内容主要强调的是数学模型及参数性质等方面, 对其分类的应用介绍得不多. 目前, 虽然从总体上来说, 计算机时代发展的机器学习方法无论在数量上还是预测精度上都超过经典统计方法, 但是, 对于某些数据, 特别是在自变量基本是数量变量的情况下, 经典的判别分析方法在分类问题上仍然是一类重要的方法, 而 Logistic 回归方法对两水平因变量的分类仍是相当不错的.

本章不但对经典判别分析及 Logistic 回归做了较详细的介绍, 也简单介绍了一些机器学习分类方法, 并且通过案例与经典判别分析做比较.

例 4.1. 数字笔迹识别 (pendigits.csv) 该数据有 10992 个观测值和 17 个变量. 原始数据有大量缺失值, 这里给出的数据文件为用 `missForest()` 函数弥补缺失值后的数据. 变量中的第 17 个变量 (V17) 为有 10 个水平的因变量, 这 10 个水平为 $0, 1, \ldots, 9$ 等 10 个阿拉伯数字, 而其余变量都是数量变量.

如果要用原始数据 (从网上下载), 请注意数据格式的转换和缺失值的非正规标识方法.[1]

例4.1就是一个分类例子, 试图根据自变量 V1~V16 及目标变量 (因变量)V17 的观测值建立分类 (判别) 模型, 以用于未知目标变量类型的分类.

---

[1]原数据的网址之一为: `http://www.csie.ntu.edu.tw/$~\sim$cjlin/libsvmtools/datasets/multiclass.html#news20`, 数据名为 pendigits(训练集) 和 pendigits.t(测试集), 都属于 LIBSVM 格式. 网站`https://archive.ics.uci.edu/ml/datasets/Pen-Based+Recognition+of+Handwritten+Digits`也提供该数据, 但其中的缺失值都以字符 "空格 +0" 表示 (但说明中显示无缺失值, 这是不对的). 第二个网址给出了数据的细节. 数据来源于 E. Alpaydin, Fevzi. Alimoglu, Department of Computer Engineering, Bogazici University, 80815 Istanbul Turkey, alpaydinboun.edu.tr.

图4.1为例4.1的用两种不同方法分类后投影到两个变量的效果, 其中左图为线性判别分析, 右图为朴素贝叶斯方法. 可以看出, 由于因变量的类别很多, 自变量维数很大, 很难在二维图上展示分类结果. 产生该图的代码为:

```
w=read.csv("pendigits.csv");w$V17=factor(w$V17)
library(klaR)
partimat(V17~V16+V14,data=w,method="lda")
partimat(V17~V16+V14,data=w,method="naiveBayes")
```

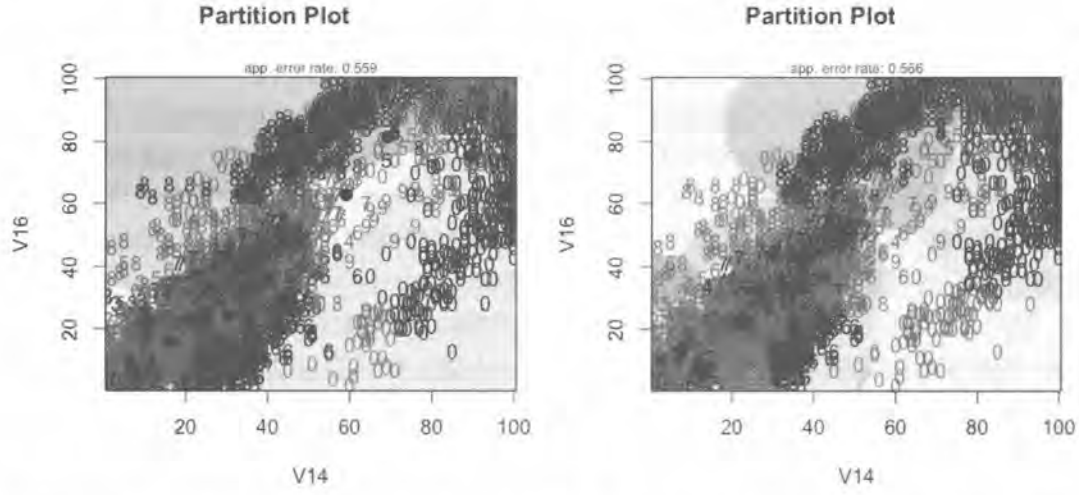

图 4.1 例4.1的用两种不同方法分类后投影到两个变量的效果, 其中左图为线性判别分析, 右图为朴素贝叶斯方法

4.1.2 分类和聚类的区别

分类 (或判别) 和聚类看上去很像, 都是把观测值分成若干群, 希望每一群内部的观测值共性较多 (比如距离较近), 而不同群之间的差异则较大. 但聚类是无指导学习, 也就是说, 训练集数据本身的观测值并没有属于某一群的标签, 也就是说没有因变量, 完全根据所有变量的性质来聚类; 而分类时, 存在一个因变量, 也就是说, 训练集数据的每个观测值都有一个属于某一类的已知标签, 因此在建模时有一个指导或者对照. 由于聚类是所有变量都起作用, 而分类是只有和因变量各类的特点有关的变量起作用, 因此**分类和聚类的结果不应该有必然的关联, 将两种方法的结果做比较必定是具有误导性的**.

4.2 经典判别分析

4.2.1 线性判别分析和二次判别分析

线性判别分析 (linear discriminant analysis, LDA) 的目的是找到原始变量的线性组合 (例4.1为 16 个自变量), 使数据集中因变量的各水平 (例4.1中的 10 个数字形成因变量的 10 个水平) 有最佳的分离. 线性判别分析也称为**典型判别分析** (canonical discriminant analysis), 或简称**判别分析** (discriminant analysis).

由于线性判别分析是用若干个超平面来分割观测值, 因此最终结果无论到哪个平面的投影的分割都是直线, 如图4.1左图所示. 下面叙述确定这些用于分割的超平面的公式.

假定观测值一共有 K 类: $\Pi_1, \Pi_2, \ldots, \Pi_K$. 也就是说因变量有 K 个水平. 自变量 $\boldsymbol{x}$ 所属类号用 $G(\boldsymbol{x})$ 表示, 其值域为 $\mathbb{K} = \{1, 2, \ldots, K\}$, 即 $G(\boldsymbol{x}) \in \mathbb{K}$, 观测值 $\boldsymbol{x} \in \Pi_k$ 等价于 $G(\boldsymbol{x}) = k$. 可以假定一个对象属于第 k 类的 (先验) 概率为 $\pi_k = P(G(\boldsymbol{x}) = k) = P(\boldsymbol{x} \in \Pi_k)$, 而 $\sum_{k=1}^{K} \pi_k = 1$. 通常 π_k 由频率

$$\hat{\pi}_k = \frac{\text{第 } k \text{ 类的样本个数}}{\text{总样本个数}}$$

来估计. 如果观测值中的自变量有 p 个, 一共有 n 个观测值, 其中对于第 k 类有 n_k 个观测值, 用 $f_k(\boldsymbol{x})$ 表示属于第 k 类的观测值的分布密度.

根据贝叶斯最大后验分布估计 (maximum a posteriori estimation, MAP), 点 $\boldsymbol{x}$ 属于某一群的估计为使得 $MAP = P(G = k|\boldsymbol{x}) = f_k(\boldsymbol{x})\pi_k$ 最大的 k:

$$\hat{G}(\boldsymbol{x}) = \arg\max_k P(G = k|\boldsymbol{x}) = \arg\max_k \left[f_k(\boldsymbol{x})\pi_k\right], \tag{4.2.1}$$

公式中的 arg 是英文 argument(变元) 的缩写, 而符号 $\arg\max_k f(k)$ 为 "使得函数 $f(k)$ 达到最大值的变元 k 的值".

正态分布假定下, 协方差阵 Σ_k 相同的情况 ($\Sigma_k = \Sigma$, $\forall k$)

如果属于第 k 类的观测值属于正态分布 $N(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$, 而且 $\boldsymbol{\Sigma}_k = \Sigma$ ($\forall k$), 那么, 式 (4.2.1) 为:

$$\hat{G}(\boldsymbol{x}) = \arg\max_k \left[\boldsymbol{x}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k - \frac{1}{2}\boldsymbol{\mu}_k^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k + \ln(\pi_k)\right],$$

定义线性判别函数 (linear discriminant function) 为:

$$\delta_k(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k - \frac{1}{2}\boldsymbol{\mu}_k^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k + \ln(\pi_k),$$

因此, 如果在所有 $\delta_k(\boldsymbol{x})$ 中 ($k \in \mathbb{K}$), 观测值 $\boldsymbol{x}$ 使得某一个 $\delta_k(\boldsymbol{x})$ 最大, 那么 $\boldsymbol{x}$ 属于该类, 即

$$\hat{G}(\boldsymbol{x}) = \arg\max_k \delta_k(\boldsymbol{x}). \tag{4.2.2}$$

当然, 在实践中, 公式中的参数 π_k、$\boldsymbol{\mu}$ 和 $\boldsymbol{\Sigma}$ 未知, 参数 π_k 可用上面的 $\hat{\pi}_k$ 替代, $\boldsymbol{\mu}$ 和 $\boldsymbol{\Sigma}$ 可用相应的样本均值和样本协方差阵替代.

线性判别函数实际上是观测值的 K 种线性组合 $\boldsymbol{w}^\top\boldsymbol{x}$, 并且依照其大小来决定 $\boldsymbol{x}$ 属于哪一类.

正态分布假定下, 协方差阵 Σ_k 不相同的情况

如果对于不同的 k, $\boldsymbol{\Sigma}_k$ 不相等, 则判别函数为:

$$\delta_k^{(2)}(\boldsymbol{x}) = -\frac{1}{2}\ln|\boldsymbol{\Sigma}_k| - \frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_k) + \ln(\pi_k),$$

它称为**二次判别函数** (quadratic discriminant function). 而

$$\hat{G}(\boldsymbol{x}) = \arg\max_k \delta_k^{(2)}(\boldsymbol{x}),$$

这里的判别函数就是二次多项式了.

4.2.2 Fisher 判别分析

Fisher 线性判别并未要求事先知道总体分布类型, 其工作原理就是对原数据进行降维投影 $\boldsymbol{a}^\top\boldsymbol{x}$, 寻求能够将各个类尽可能分开的方向, 也就是寻求使得各类均值到总均值的距离 (类间离差平方和) 最大, 而总方差 (组内离差平方和) 尽可能小. 这就导致了一个广义特征值问题, 即求使得下式即所谓类间平方和 (between-group-sum of squares)$\boldsymbol{a}^\top\boldsymbol{B}\boldsymbol{a}$ 与类内平方和 (within-group-sum of squares)$\boldsymbol{a}^\top\boldsymbol{W}\boldsymbol{a}$ 之比最大的 $\boldsymbol{a}$:

$$\frac{\boldsymbol{a}^\top\boldsymbol{B}\boldsymbol{a}}{\boldsymbol{a}^\top\boldsymbol{W}\boldsymbol{a}}, \tag{4.2.3}$$

这里

$$\boldsymbol{B} = \hat{\boldsymbol{B}}_\mu = \sum_{k=1}^{K} n_k(\bar{\boldsymbol{x}}_k - \bar{\boldsymbol{x}})(\bar{\boldsymbol{x}}_k - \bar{\boldsymbol{x}})^\top,$$

$$\boldsymbol{W} = \sum_{k=1}^{K}\sum_{j=1}^{n_k}(\bar{\boldsymbol{x}}_{kj} - \bar{\boldsymbol{x}}_k)(\bar{\boldsymbol{x}}_{kj} - \bar{\boldsymbol{x}}_k)^\top,$$

其中, $\bar{\boldsymbol{x}}$ 及 $\bar{\boldsymbol{x}}_k$ 分别为总样本均值及类内均值 (均为 p 维向量). 其解为 $\boldsymbol{W}^{-1}\boldsymbol{B}$ 的非零特征根 $\lambda_1, \lambda_2, \cdots, \lambda_s$ ($\lambda_1 > \lambda_2 > \cdots > \lambda_s$), $s \leqslant \min(K-1, p)$ (p 为自变量个数), 而相应的特征向量为 $\boldsymbol{a}_1, a_2, \ldots, \boldsymbol{a}_s$. 通常把特征向量单位化, 使得 $\boldsymbol{a}_i = \boldsymbol{e}_i$, 即 $\boldsymbol{e}_i^\top\hat{\boldsymbol{\Sigma}}\boldsymbol{e}_i = 1$. 根据这些特征向量的投影, $\boldsymbol{a}_1^\top\boldsymbol{x}$ 称为第一判别量, $\boldsymbol{a}_2^\top\boldsymbol{x}$ 称为第二判别量, 等等. 一般选择前 $r(\leqslant s)$ 个判别量. 这时, Fisher 判别法为:

$$\text{如果 } \forall i \neq k\text{: } \sum_{j=1}^{r}[\boldsymbol{a}_j^\top(\boldsymbol{x}-\bar{\boldsymbol{x}}_k)]^2 \leqslant \sum_{j=1}^{r}[\boldsymbol{a}_j^\top(\boldsymbol{x}-\bar{\boldsymbol{x}}_i)]^2, \text{ 则判 } \boldsymbol{x} \text{ 为第 } k \text{ 类.}$$

注意

从公式可以看出, 这里介绍的一次或二次判别分析的方法要求数据不但是连续变量, 而且要服从多元正态分布, 但数据是否服从某种分布是无法核对或证明的. 尽管如此, 在实践中只要不做任何假设检验, 对任何数据都可以试着做判别, 只不过对于偏离正态分布的数据效果可能不会那么好. 经

验表明, 二次判别一般要优于线性判别方法, 更优于 Fisher 判别.

4.3 例4.1数字笔迹识别的判别分析实践

下面使用程序包 MASS[2]的 lda 及 qda 函数来做线性判别分析和二次判别分析. 输入数据, 使用训练集获得模型, 并对测试集做交叉验证.

4.3.1 例4.1数字笔迹识别的线性判别分析的计算

下面对例4.1做线性判别分析, 并且用已给的训练集和测试集做交叉验证. 首先输入一个把数据随机分成 D 折的函数 Fold, 它把因变量各个水平尽量平均分配到各个水平.

```
Fold=function(Z=5,w,D,seed=7777){
  n=nrow(w);d=1:n
  e=levels(w[,D]);Y=length(e)#因变量Y类
  set.seed(seed)
  dd=lapply(1:Y,function(i){
    d0=d[w[,D]==e[i]];j=length(d0);ZT=rep(1:Z,ceiling(j/Z))[1:j]
    id=cbind(sample(ZT,length(ZT)),d0);id})
#上面每个dd[[i]]是随机1:Z及i类的下标集组成的矩阵
  mm=lapply(1:Z, function(i){
    u=NULL;for(j in 1:Y) u=c(u,dd[[j]][dd[[j]][,1]==i,2]);u})
  return(mm)
}
```

下面是对数据做线性判别, 并且做 10 折交叉验证的代码, 输出交叉验证的混淆矩阵和总误差率.

```
library(MASS)
w=read.csv("pendigits.csv")#读入数据
w[,17]=factor(w[,17]) #因子化哑元变量
Z=10;D=17
mm=Fold(Z=10,w,D=17,1010) #10折交叉验证集的确定
y=NULL->yh
for(i in 1:Z){
  m=mm[[i]]
  a=lda(V17~.,w[-m,])
  y=c(y,w[m,D])
  yh=c(yh,predict(a,w[m,])$class)
}
table(y,yh)
(er=sum(y!=yh)/dim(w)[1])
```

[2]Venables, W. N. & Ripley, B. D. (2002) *Modern Applied Statistics with S*. Fourth Edition. Springer, New York. ISBN 0-387-95457-0.

输出为:

```
> table(y,yh)
    yh
y       1    2    3    4    5    6    7    8    9   10
  1  1013   12    0    0   14    1    5    0   90    8
  2     0  771  209   26    1   70    5   35    1   25
  3     0   19 1113    0    0    0    0   12    0    0
  4     0   13    1 1029    0    0    0    8    0    4
  5     0    0    1    0 1108    9    4    1    0   21
  6     0   12    0   39    1  717   28    4    7  247
  7     9    0    0    0    2    1 1028    0   16    0
  8     0   47   29   58    2    5    0  977    4   20
  9    61   37    0   25    0   72    6    4  834   16
  10    1   38    0    9    7   27    8    2    0  963
> (er=sum(y!=yh)/dim(w)[1])
[1] 0.1309134
```

上面输出的矩阵称为混淆矩阵 (confusion matrix), 行代表真实的类, 列代表用训练集得到的模型来拟合测试集所预测的类. 在 10992 个观测值中, 有 1439 个被判错 (上面输出的对角线外的数目之和), 判错率为 0.1309134, 如果有 6 个数字组成的手写数码每个数字有这么大的出错率, 那么每个完整数码被正确识别的概率只有大约 43.1%!

4.3.2 例4.1数字笔迹识别的二次判别分析的计算

下面对例4.1做二次判别分析, 并且用前面通过 `Fold` 函数得到的训练集和测试集做交叉验证, 这里的代码和前面线性判别分析的代码几乎雷同:

```
y=NULL->yh
for(i in 1:Z){
  m=mm[[i]]
  a=qda(V17~.,w[-m,])
  y=c(y,w[m,D])
  yh=c(yh,predict(a,w[m,])$class)
}
table(y,yh)
(er=sum(y!=yh)/dim(w)[1])
```

输出为:

```
> table(y,yh)
   yh
y       1    2    3    4    5    6    7    8    9   10
  1  1120    0    0    0    0    0    2    0   17    4
  2     0 1101   17    3    0    6    1    0    3   12
```

```
   3     0   21 1111    0    0    0    0   11    1    0
   4     0    5    1 1033    0    2    0    7    2    5
   5     0    0    1    0 1130    1    0    0    0   12
   6     0    3    0    1    1 1025    0    0    5   20
   7     0    1    0    0    0    2 1045    0    8    0
   8     0    5    0    0    0    0    0 1127    3    7
   9     0    0    0    0    0    0    0    2 1050    3
   10    2    3    0    0    0    0    0    1    1 1048
> (er=sum(y!=yh)/dim(w)[1])
[1] 0.018377
```

在测试集的 10992 个观测值中, 有 202 个被判错 (上面输出的对角线外的数目之和), 判错率为 0.018377, 错判率只有线性判别分析的 14%.

4.4 判别分析的一些概念和某些数学细节的补充 *

因为4.2节的内容比较简略, 为了满足读者可能的需要, 本节对一些经典判别的概念及4.2节的某些数学细节给出一些补充, 仅供参考之用.

假定数据为 p 维的, 也就是说, 有 p 个自变量, 每个观测值 $\boldsymbol{x}$ 有 p 个坐标, 即 $\boldsymbol{x} \in \mathbb{R}^p$. 考虑 K 个总体或者群 $\Pi_1, \Pi_2, \ldots, \Pi_K$, 第 i 个总体在空间 $\mathbb{R}^p$ 有密度函数 $f_i(\boldsymbol{x})$, 为强调似然函数意义, 也记为 $f_i(\boldsymbol{x}) = L_i(\boldsymbol{x})$. 一个**判别规则**相应于空间 $\mathbb{R}^p$ 的一个划分: $R_1, R_2, \ldots, R_K$, 使得 $\cup R_i = \mathbb{R}^p$, 如果 $\boldsymbol{x} \in R_i$, 则该规则把 $\boldsymbol{x}$ 判为属于 Π_i. 注意: $\boldsymbol{x} \in R_i$ 不一定保证 $G(\boldsymbol{x}) = i$ (即 $\boldsymbol{x} \in \Pi_i$), 因为 R_i 仅仅是 Π_i 的估计, 但两者不一定相等.

4.4.1 数据总体分布已知时的判别

最大似然判别准则

如果在各个似然函数中, $L_j(\boldsymbol{x})$ 在所有的 $\{L_i\}$ 中最大, 则**最大似然判别准则**把 $\boldsymbol{x}$ 判别属于 Π_j. 即 $\hat{G}(\boldsymbol{x}) = j$. 最大似然判别准则可以写成:

$$\hat{G}(\boldsymbol{x}) = \arg\max_i L_i(\boldsymbol{x}).$$

经典判别分析往往假定数据有正态分布, 如果总体 $\Pi_i \sim N_p(\boldsymbol{\mu}_i, \boldsymbol{\Sigma})$ $(i = 1, 2, \ldots, k)$; $\boldsymbol{\Sigma} > 0$, 似然函数为:

$$L_i(\boldsymbol{x}) = \frac{1}{(2\pi)^{p/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_i)\right\},$$

那么, 最大似然判别准则把 $\boldsymbol{x}$ 判为使得 $\{(\boldsymbol{x}-\boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_i)\}$ 最小的那一群:

$$\hat{G}(\boldsymbol{x}) = \arg\min_i (\boldsymbol{x}-\boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_i).$$

在 $K = 2$ 的特殊情况下, 如果记 $\boldsymbol{\alpha} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$ 及 $\boldsymbol{\mu} = \frac{1}{2}(\boldsymbol{\mu}_1 + \boldsymbol{\mu}_2)$, 则在 $\boldsymbol{\alpha}^\top(\boldsymbol{x}-\boldsymbol{\mu}) > 0$

时, $\hat{G}(\boldsymbol{x})=1$, 否则 $\hat{G}(\boldsymbol{x})=2$.

在 $K=2$ 的特殊情况下, 令 $\delta(\boldsymbol{x})=\log_1(\boldsymbol{x})-\log_2(\boldsymbol{x})$, 称其为**判别函数**, 则如果 $\delta(\boldsymbol{x})>0$, 那么把 $\boldsymbol{x}$ 判归 Π_1, 或 $\hat{G}(\boldsymbol{x})=1$, 否则 $\hat{G}(\boldsymbol{x})=2$.

贝叶斯判别准则

假定 K 个总体的先验概率记为 $\boldsymbol{\pi}=(\pi_1,\pi_2,\ldots,\pi_K)$, 则**贝叶斯判别准则**为:

$$\hat{G}(\boldsymbol{x})=\arg\max_i \pi_i L_i(\boldsymbol{x}).$$

在 $K=2$ 的特例中, 还是用判别函数 $\delta(\boldsymbol{x})=\log_1(\boldsymbol{x})-\log_2(\boldsymbol{x})$, 但是这时如果 $\delta(\boldsymbol{x})>\log(\pi_2/\pi_1)$, 那么 $\hat{G}(\boldsymbol{x})=1$, 否则 $\hat{G}(\boldsymbol{x})=2$.

最优性质

人们往往定义随机化判别准则, 把 $\boldsymbol{x}$ 判归 Π_i 的概率记为 $\phi_i(\boldsymbol{x})$, 定义在 $\mathbb{R}^p$, 而且对所有 $\boldsymbol{x}$, $\sum\phi_i(\boldsymbol{x})=1$. 这时, 如果 $\boldsymbol{x}\in R_i$, 则 $\phi_i(\boldsymbol{x})=1$, 否则为 0.

比如贝叶斯准则为

$$\phi_i(\boldsymbol{x})=\begin{cases}1, & \text{当}\phi_i L_i(\boldsymbol{x})=\max_i \pi_i L_i(\boldsymbol{x});\\ 0, & \text{当}\phi_i L_i(\boldsymbol{x})\neq\max_i \pi_i L_i(\boldsymbol{x}).\end{cases}$$

一个属于 Π_j 的观测值被判为属于 Π_i 的概率为:

$$p_{ij}=\int \phi_i(\boldsymbol{x})L_j(\boldsymbol{x})\mathrm{d}\boldsymbol{x},$$

因此, 它被判对的概率为 p_{ii}, 错判的概率为 $1-p_{ii}$. 因此概率 $p_{11},p_{22},\ldots,p_{KK}$ 为判别准则好坏的标准.

如果一个判别准则 d 的概率 $\{p_{ii}\}$ 和其他具有概率 $\{p'_{ii}\}$ 的准则 d' 满足

$$p_{ii}\geqslant p'_{ii},\quad \forall i=1,2,\ldots,K,$$

则说明准则 d 和准则 d' 一样好. 如果上述不等式有一个是严格不等式, 则称 d 是较好的, 如果没有比 d 更好的准则, 则称 d 是**可允许的** (admissible).

可以证明, 所有贝叶斯判别准则 (包括最大似然准则) 都是可允许的. 如果 $\{\Pi_i\}$ 有先验分布 $\{\pi_i\}$, 那么, 在关于该先验分布的所有判别准则中, 贝叶斯准则有最大的正确判别的后验概率.

损失函数和风险

令损失函数为

$$K(i,j)=\begin{cases}0, & \text{当}i=j;\\ c_{ij}, & \text{当}i\neq j.\end{cases}$$

则风险函数定义为:

$$R(d,j)=E(K(d(\boldsymbol{x}),j)|\Pi_i)=\sum_i K(i,j)\int \phi_i(\boldsymbol{x})L_j(\boldsymbol{x})\mathrm{d}\boldsymbol{x}=\sum_i c_{ij}p_{ij}.$$

对于先验分布 $\boldsymbol{\pi}=(\pi_1,\pi_2,\ldots,\pi_K)^\top$, **贝叶斯风险**定义为:

$$r(d,\boldsymbol{\pi})=\sum_i \pi_i R(d,j).$$

样本的情况

在样本情况下, 各种参数必须用数据来估计, 比如对各个群的均值 $\boldsymbol{\mu}_1,\boldsymbol{\mu}_2,\ldots,\boldsymbol{\mu}_K$ 的无偏估计为各类的样本均值 $\bar{\boldsymbol{x}}_1,\bar{\boldsymbol{x}}_2,\ldots,\bar{\boldsymbol{x}}_K$, 而 $\boldsymbol{\Sigma}$ 的无偏估计为 $\boldsymbol{S}=\sum n_i\boldsymbol{S}_i/(n-K)$, 这里的 $\boldsymbol{S}_i$ 为第 i 类的样本协方差矩阵.

4.4.2 线性判别分析的某些细节

假定观测值服从多元正态分布, 密度函数为:

$$f_k(\boldsymbol{x})=\frac{1}{(2\pi)^{p/2}|\boldsymbol{\Sigma}_k|^{1/2}}\exp\left\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_k)^\top\boldsymbol{\Sigma}_k^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_k)\right\},$$

对于线性判别分析, 还假定 $\boldsymbol{\Sigma}_k=\boldsymbol{\Sigma}$ $(\forall k)$. 在这个假定下, 不同类的分布仅仅按照正态分布的位置 (参数) 来区别. 这时, 最优的分类为:

$$\begin{aligned}\hat{G}(\boldsymbol{x})&=\arg\max_k\left[f_k(\boldsymbol{x})\pi_k\right]=\arg\max_k\left[\ln\left(f_k(\boldsymbol{x})\pi_k\right)\right]\\&=\arg\max_k\left\{-\ln\left[(2\pi)^{p/2}|\boldsymbol{\Sigma}|^{1/2}\right]-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_k)^\top\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_k)+\ln(\pi_k)\right\},\end{aligned}$$

由此得到

$$\hat{G}(\boldsymbol{x})=\arg\max_k\left[\boldsymbol{x}^\top\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k-\frac{1}{2}\boldsymbol{\mu}_k^\top\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k+\ln(\pi_k)\right].$$

定义线性判别函数为:

$$\delta_k(\boldsymbol{x})=\boldsymbol{x}^\top\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k-\frac{1}{2}\boldsymbol{\mu}_k^\top\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k+\ln(\pi_k),$$

则

$$\hat{G}(\boldsymbol{x})=\arg\max_k\delta_k(\boldsymbol{x}).$$

显然, 在第 k 类和第 ℓ 类之间的决策边界为点集:

$$\{\boldsymbol{x}:\delta_k(\boldsymbol{x})=\delta_\ell(\boldsymbol{x})\}$$

或者

$$\left\{\boldsymbol{x}:\ \boldsymbol{x}^\top\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_k-\boldsymbol{\mu}_\ell)-\frac{1}{2}(\boldsymbol{\mu}_k+\boldsymbol{\mu}_\ell)^\top\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_k-\boldsymbol{\mu}_\ell)+\ln\left(\frac{\pi_k}{\pi_\ell}\right)=0\right\}. \tag{4.4.1}$$

在实践中, 上面的参数 $\boldsymbol{\mu}_k,\pi_k,\boldsymbol{\Sigma}$ 都未知, 必须做出估计 (记 x_{ki} 为 X_{ki} 的实现值):

$$\begin{aligned}\hat{\pi}_k&=n_k/n;\\ \hat{\boldsymbol{\mu}}_k&=\tfrac{1}{n_k}\textstyle\sum_{i=1}^{n_k}\boldsymbol{x}_{ki};\\ \hat{\boldsymbol{\Sigma}}&=\tfrac{1}{n-K}\textstyle\sum_{k=1}^{K}(n_k-1)\hat{\boldsymbol{\Sigma}}_k;\\ \hat{\boldsymbol{\Sigma}}_k&=\tfrac{1}{n_k-1}\textstyle\sum_{j=1}^{n_k}(\boldsymbol{x}_{kj}-\bar{\boldsymbol{x}}_k)(\boldsymbol{x}_{kj}-\bar{\boldsymbol{x}}_k)^\top.\end{aligned}$$

注: 我们前面用的最大后验概率 (MAP) 方法是诸多方法中的一种. 更一般地, 如果记 $c(\ell|k)$ 为把第 k 类判为第 ℓ 类的损失, R_k 为 $\boldsymbol{x}$ 被判为第 k 类的区域 ($R_i\cap R_j=\emptyset$, $\forall\ell\neq k$ 以及 $\cup_i R_i=\Omega$), $P(\ell|k)$ 为本来属于第 k 类的个体分到第 ℓ 类的概率, 即 $P(\ell|k)=\int_{R_\ell}f_k(\boldsymbol{x})\mathrm{d}\boldsymbol{x}$, 则来自第 k 类的个体 $\boldsymbol{x}$ 的期望误判损失 (expected cost of misclassification, ECM) 为:

$$ECM(k)=\sum_{j=1}^{K}P(j|k)c(j|k),$$

这里, $c(k|k)=0$ 且 $P(k|k)=1-\sum_{j\neq k}P(j|k)$. 于是总的损失为:

$$ECM=\sum_{k=1}^{K}\pi_k\left[\sum_{j=1,j\neq k}^{K}P(j|k)c(j|k)\right].$$

因此, 使来自第 k 类的 $\boldsymbol{x}$ 的期望误判损失最小的判别区域为使得下式最小的区域:

$$\sum_{j=1,j\neq k}^{K}\pi_k f_k(\boldsymbol{x})c(j|k).$$

对于 $K=2$ 的正态分布情况, 上式导致决策边界点为:

$$\left\{\boldsymbol{x}:\ \boldsymbol{x}^\top\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1-\boldsymbol{\mu}_2)-\frac{1}{2}(\boldsymbol{\mu}_1+\boldsymbol{\mu}_2)^\top\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1-\boldsymbol{\mu}_2)+\ln\left[\frac{c(2|1)\pi_1}{c(1|2)\pi_2}\right]=0\right\}. \tag{4.4.2}$$

显然, 式 (4.4.1) 对应于误判损失函数相等的特殊情况, 也就是说, MAP 是在损失函数都相等时的判别方法.

4.4.3 Fisher 线性判别分析的某些细节

Fisher 判别法是对 (有 K 个总体的) 观测值做少数线性组合, 也就是把观测值投影到较低维的空间, 从而形成维数较小 (比如二维、三维) 的总体, 这时可用目视或图形表示来做进一步分析. Fisher 判别法不用假定正态分布, 但假定 K 个总体协方差矩阵满秩且相同:

$\boldsymbol{\Sigma}_i = \boldsymbol{\Sigma}$ $(\forall i)$.

令 $\boldsymbol{\mu} = \frac{1}{K}\sum_{k=1}^{K}\boldsymbol{\mu}_k$, 并记

$$\boldsymbol{B}_{\boldsymbol{\mu}} = \sum_{k=1}^{K}(\boldsymbol{\mu}_k - \boldsymbol{\mu})(\boldsymbol{\mu}_k - \boldsymbol{\mu})^{\top},$$

令线性组合 $\boldsymbol{a}^{\top}\boldsymbol{x}$ 对总体 k 的期望为 $\boldsymbol{a}^{\top}\boldsymbol{\mu}_k$, 对所有总体的方差为 $\boldsymbol{a}^{\top}\boldsymbol{\Sigma}\boldsymbol{a}$, 总均值 $\mu_Y = \frac{1}{K}\sum_{k=1}^{K}\boldsymbol{a}^{\top}\boldsymbol{\mu}_k = \boldsymbol{a}^{\top}\boldsymbol{\mu}$. 我们希望投影使得各类的均值到总均值的距离最大, 这等价于使得下式最大:

$$\sum_{k=1}^{K}(\boldsymbol{a}^{\top}\boldsymbol{\mu}_k - \boldsymbol{a}^{\top}\boldsymbol{\mu})^2 = \boldsymbol{a}^{\top}\left[\sum_{k=1}^{K}(\boldsymbol{\mu}_k - \boldsymbol{\mu})(\boldsymbol{\mu}_k - \boldsymbol{\mu})^{\top}\right]\boldsymbol{a} = \boldsymbol{a}^{\top}\boldsymbol{B}_{\boldsymbol{\mu}}\boldsymbol{a},$$

同时, 我们希望总方差 $\boldsymbol{a}^{\top}\boldsymbol{\Sigma}\boldsymbol{a}$ 尽可能小, 这就归结为使

$$\frac{\boldsymbol{a}^{\top}\boldsymbol{B}_{\boldsymbol{\mu}}\boldsymbol{a}}{\boldsymbol{a}^{\top}\boldsymbol{\Sigma}\boldsymbol{a}}$$

最大化的问题. 由于矩阵 $\boldsymbol{B}_{\boldsymbol{\mu}}$ 和 $\boldsymbol{\Sigma}$ 未知, 我们用训练样本来估计:

$$\hat{\boldsymbol{B}}_{\mu} = \textstyle\sum_{k=1}^{K} n_k(\bar{\boldsymbol{x}}_k - \bar{\boldsymbol{x}})(\bar{\boldsymbol{x}}_k - \bar{\boldsymbol{x}})^{\top},$$
$$\hat{\boldsymbol{\Sigma}} = \tfrac{1}{n-K}\left\{\textstyle\sum_{k=1}^{K}\sum_{j=1}^{n_k}(\bar{\boldsymbol{x}}_{kj} - \bar{\boldsymbol{x}}_k)(\bar{\boldsymbol{x}}_{kj} - \bar{\boldsymbol{x}}_k)^{\top}\right\},$$

这里,

$$\bar{\boldsymbol{x}}_k = \frac{1}{n_k}\sum_{j=1}^{n_k}\boldsymbol{x}_{kj}, \quad \bar{\boldsymbol{x}} = \frac{\displaystyle\sum_{k=1}^{K}\sum_{j=1}^{n_k}\boldsymbol{x}_{kj}}{\displaystyle\sum_{i=1}^{K} n_i}.$$

4.5 二分类因变量的 Logistic 回归

4.5.1 Logistic 回归模型

Logistic 回归之所以称为 "回归", 是因为人们曾试图用线性回归的方式处理因变量仅有两个不同值的情况 (假定因变量为 y, 自变量为 $x_1, x_2, \ldots, x_p$). 在线性回归中, 如果因变量 y 只有 0 和 1 两个值, 即使用线性回归训练出模型, 其预测值不会在 0 和 1 之间, 更不会仅限于 0 和 1 两个值. 此外, 诸如正态分布等假定对于两个值的变量更不合适. 因此人们把因变量取两个值考虑成 Bernoulli 试验 $Bernoulli(p)$, 等价于 $n = 1$ 时的二项分布 $Bin(1, p)$, 这里 Bernoulli 试验的 "成功" 的概率为 p(比如 $p = P(y = 1)$). 如果把 p 的**优势** (odd) $p/(1-p)$ 的对数 $\ln(p/(1-p))$(称为**优势比** (odds ratio)) 作为自变量的线性函数,

这就是 Logistic 回归模型:

$$\ln\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p = \boldsymbol{X}\boldsymbol{\beta} \tag{4.5.1}$$

或者

$$p = \frac{e^{\boldsymbol{X}\boldsymbol{\beta}}}{1 + e^{\boldsymbol{X}\boldsymbol{\beta}}}. \tag{4.5.2}$$

实际上, Logistic 回归是广义线性模型的一个特例. 广义线性模型适用于因变量为指数分布族的情况, 其一般的模型形式为**因变量分布的期望 $E(y)$ 的某个函数 $g(E(y))$ 为自变量的线性组合**:

$$g(E(y)) = \boldsymbol{X}\boldsymbol{\beta},$$

这里的函数 g 称为**连接函数** (link function). Logistic 回归中 Bernoulli 分布的期望为 p, 连接函数为对数优势 $g(p) = \ln(p/(1-p))$.

注意, 在式 (4.5.1) 中的左边不是因变量, 而是因变量的期望, 它是参数的函数, 是不可观测的. 因为左边不是变量的观测值, 所以右边不会像线性回归那样有误差项. 由于给定了分布, Logistic 回归的所有参数可以用最大似然法来解出.

Logistic 回归的名字来源于式 (4.5.1) 中右边的所谓 **Logistic 函数**:

$$y = \frac{e^x}{1+e^x} = \frac{1}{1+e^{-x}} = \frac{1}{2} + \frac{1}{2}\tanh\left(\frac{1}{x}\right).$$

显然, Logistic 函数的逆函数有下列形式:

$$y = \ln\frac{x}{1-x},$$

它称为 **Logit 函数**. 这两个函数的图形显示在图4.2中.

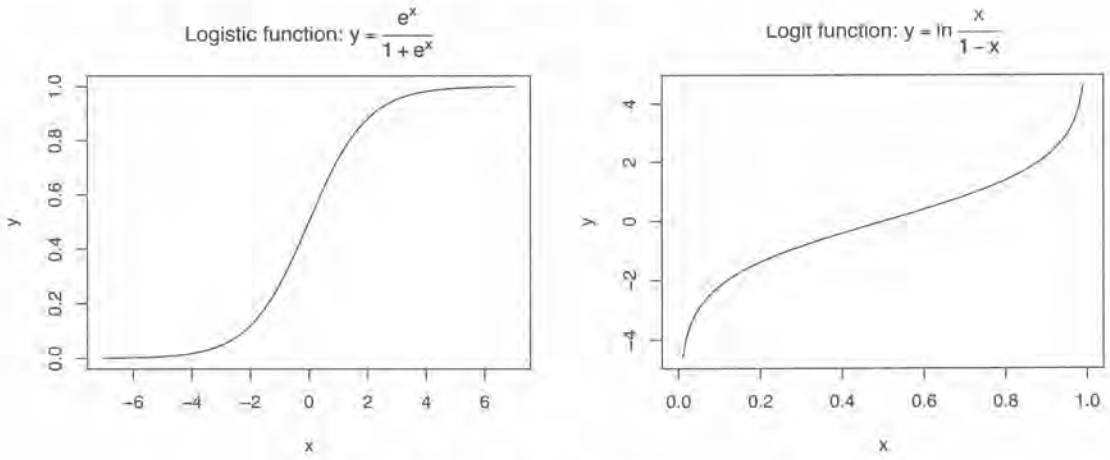

图 4.2 Logistic 函数 (左) 和 logit 函数 (右)

4.5.2 Logistic 回归模型对实际数据的拟合

下面我们用一个例子来描述 Logistic 回归的拟合 (本小节) 及分类 (下一小节).

例 4.2. 献血数据 (Trans.csv) 这组数据[3]来自新竹市输血服务中心的记录, 变量有 Recency(上次献血距离研究时的月份), Frequency(总献血次数), Time(第一次献血是多少个月之前), Donate(是否将在 2007 年 3 月再献血, 1 为会, 0 为不会).

我们将用 Logistic 回归来拟合这组数据, 以二分类变量 Donate 为因变量. 此外, 我们的数据比原始数据少一个变量 (Monetary: 总献血量, 单位: 毫升), 这是因为每次献血的数量是固定的 (250 毫升), 所以它和变量 Frequency 严格共线, 没有必要在数据中保留两个共线变量.

用 Logistic 模型拟合例4.2中数据的代码如下:

```
w=read.csv("Trans.csv")
a=glm(Donate~.,w,family=binomial)
summary(a)
```

部分输出为:

```
Call:
glm(formula = Donate ~ ., family = binomial, data = w)

Deviance Residuals:
Min       1Q   Median       3Q      Max
-2.4875  -0.7933  -0.4997  -0.1701   2.6450

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.449540   0.180349  -2.493 0.012681
Recency     -0.098584   0.017317  -5.693 1.25e-08
Frequency    0.135390   0.025672   5.274 1.34e-07
Time        -0.023092   0.005964  -3.872 0.000108
```

为简单起见, 用 x_k $(k=1,2,3)$ 分别代表变量 Recency、Frequency 和 Time; 用 β_k $(k=1,2,3)$ 分别代表这些变量的系数; β_0 代表截距项.

$$\ln\left(\frac{p}{1-p}\right)=\beta_0+\sum_{k=1}^{3}\beta_k x_k.$$

[3]Yeh, I-Cheng, Yang, King-Jang, and Ting, Tao-Ming, "Knowledge discovery on RFM model using Bernoulli sequence," Expert Systems with Applications, 2008. http://archive.ics.uci.edu/ml/datasets/Blood+Transfusion+Service+Center.

根据输出, 得到各个参数的估计为:

$$\hat{\beta}_0 = -0.449540, \quad \hat{\beta}_1 = -0.098584,$$
$$\hat{\beta}_2 = 0.135390, \quad \hat{\beta}_3 = -0.023092.$$

本书引入 Logistic 回归的主要目的在于分类, 对拟合的参数性质等结果不做深入的讨论.

4.5.3 Logistic 回归对例4.2献血数据的分类

我们在得到参数 $\boldsymbol{\beta}$ 的估计值 $\hat{\boldsymbol{\beta}}$ 之后, 可以根据数据 $\boldsymbol{X}^\top = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n]$ 对 p 做预测:

$$\ln\left(\frac{\hat{p}_i}{1-\hat{p}_i}\right) = \boldsymbol{x}_i^\top \hat{\boldsymbol{\beta}}, \ i = 1, 2, \ldots, n. \tag{4.5.3}$$

式 (4.5.3) 还可以表示成:

$$\hat{p}_i = \frac{e^{\boldsymbol{x}_i^\top \hat{\boldsymbol{\beta}}}}{1 + e^{\boldsymbol{x}_i^\top \hat{\boldsymbol{\beta}}}}, \ i = 1, 2, \ldots, n. \tag{4.5.4}$$

很显然, 对于每个观测值 (比如观测值 i) 我们预测的是 p_i, 而不是第 i 个观测值的具体水平 (诸如是 "0" 还是 "1", 是 "yes" 还是 "no", 等等). 这时就需要有一个阈值 p_t, 使得当 $\hat{p}_i > p_t$ 时判定第 i 个观测值属于某一水平; 否则, 判定为另一个水平.

下面先取 $p_t = 0.5$ 做分类. 基于前一小节对例4.2献血数据的拟合结果 (存在对象 a 中), 得到混淆矩阵和预测精度:

```
> table(w$Donate,(pred>0.5)*1 )

    0   1
0 555  15
1 156  22
> sum(w$Donate!=(pred>0.5)*1)/nrow(w)
[1] 0.2286096
```

这个误差很大, 约 22.9%.

注: 在广义线性模型函数 glm 的程序中, 预测的概率 $\hat{p}_i$ 是指 (第 i 个观测值) 水平名字按照字符大小顺序水平较大的概率. 比如, "0" 和 "1" 为水平的名字, 预测的概率为水平 1 的概率; 再如, 对于水平 Male 及 Female, 预测的概率是为 Male 的, 因为按照字母顺序, Male 较大 (可以试试代码 "Male">"Female", 看得到什么结果).

显然, 用 $p_t = 0.5$ 作为阈值不一定合适, 下面程序自动选择阈值, 使得训练集误判率最小 (D 是因变量位置, w 是数据名, ff 代表诸如前面 " Donate ~ ." 那样的回归公式).

```
BI=function(D,w,ff,fm="binomial"){
  a=glm(ff,w,family=fm)
  z=predict(a,w,type="response")
  L=max(levels(w[,D]))
  ee=NULL
  for(p in seq(.01,.99,.01)){
    u=rep(L,nrow(w));u[!(z>p)]=min(levels(w[,D]))
    e=sum(u!=w[,D])/nrow(w);ee=rbind(ee,c(p,e))}
    I=which(ee[,2]==min(ee[,2]))
  return(ee[min(I),])
}
```

该程序自动从 0.01 到 0.99 以间隔 0.01 的步长搜索使得误判率最小的 p_t, 而输出为 p_t 和对训练集的误判率. 如下具体运行该代码:

```
ff=Donate~. #公式
w$Donate=factor(w$Donate)#该函数要求把因变量因子化
BI(D=4,w,ff)  #调用程序
```

得到:

```
> BI(D=4,w,ff)
[1] 0.440 0.207
```

这意味着在阈值为 0.44 时, 对训练集的误判率为 0.207, 结果比阈值 0.5 时的误判率 0.229 要好些.

4.5.4 ROC 等描述性曲线

上面提到的阈值 p_t 的选择对于分类结果有很重要的影响, 如果选择极端的 0 或 1, 结果全部判断为 1 或者全部判断为 0, 而如果取中间的一些值, 则判断为 0 或 1 的比例会发生变化, 相应的误差也会发生变化. 在临床实验中, 把一个有病的人判断为有病称为真阳性 (true positive), 真阳性计数简记为 TP, 其所占的比率称为真阳性比率 (true positive rate), 简记为 TPR; 把无病的人判断为有病的, 称为假阳性 (false positive rate), 假阳性计数简记为 FP, 其所占的比率称为假阳性比率 (false positive rate), 简称为 FPR. 类似地, 有真阴性计数 TN、假阴性计数 FN、真阴性比率 TNR、假阴性比率 FN. 此外, 记阳性样本计数为 P, 阴性样本为 N.

下面是基于上面术语的一些度量:

- 灵敏度 (sensitivity), 定义为 $TP/(TP+FN)$.
- 回顾 (recall) 为 TPR, 定义为 TP/P, 由于 $P=TP+FN$, 它和灵敏度等价.
- 精确性 (precision) 或阳性预测值 (positive predictive value, PPV), 定义为 $TP/(TP+FP)$.
- 特异性 (specificity), 即 TNR, 定义为 TN/N 或 $TN/(TN+FP)$.

- 提升 (lift), 定义为 $TPR/(TP+FP)$.
- 正确预测率 (rate of positive predictions, RPP), 定义为 $(TP+FP)/(TP+FP+TN+FN)$.

由这些度量可以画出很多图, 下面是其中几个:

1. ROC 曲线 (receiver operating characteristic curve), 定义为 TPR 对 FPR 的点图, 曲线下的面积越大 (或曲线越靠近左上角), 说明在 FPR 很小时 TPR 就很大.
2. 精确度/回顾图或精确度/灵敏度图 (precision/recall graph), 定义为精确度对灵敏度的点图.
3. 灵敏度/特异性图 (sensitivity/specificity plot), 定义为灵敏度对特异性点图.
4. 提升图 (lift chart), 定义为提升对 RPP 点图

下表有助于理解上面的一些概念.

		疾病		
		+	-	
检验	+	真阳性 (TP)	假阳性 (FP)	$PPV=TP/(TP+FP)$
	-	假阴性 (FN)	真阴性 (TN)	
		$P=TP+FN$	$N=FP+TN$	
		灵敏度 $TP/(TP+FN)$	特异性 $TN/(TN+FP)$	

图4.3为例4.2献血数据根据一个训练集和一个测试集得到的 (自左至右)ROC 曲线、精确度/灵敏度图、灵敏度/特异性曲线、提升图. 关于这些图的解释请读者自己琢磨, 相信读者完全能够理解这些图的意义.

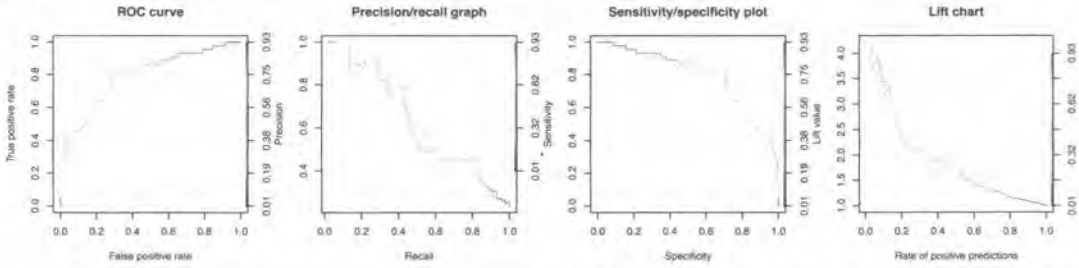

图 4.3 例4.2献血数据根据一个训练集和一个测试集得到的 (自左至右)ROC 曲线、精确度/灵敏度图、灵敏度/特异性曲线、提升图

产生图4.3的代码为:

```
w=read.csv("Trans.csv");D=4
library(caTools)
set.seed(1010)
split <- sample.split(w[,D], SplitRatio = 0.75)#训练集占75%
#得到一个训练集和一个测试集
w_train <- subset(w, split == TRUE)
w_test <- subset(w, split == FALSE)
#Logistic 回归
a <- glm (Donate ~ ., data = w_train, family = binomial)
pred <- predict(a,w_test, type = 'response')
```

```
#四个点图:
library(ROCR)
par(mfrow=c(1,4),mar=c(4,4,4,2))
ROCRpred <- prediction(pred, w_test$Donate)
ROCRperf <- performance(ROCRpred, 'tpr','fpr')
plot(ROCRperf, colorize = TRUE, text.adj = c(-0.2,1.7))
title("ROC curve")
Precperf <- performance(ROCRpred, 'prec','rec')
plot(Precperf, colorize = TRUE, text.adj = c(-0.2,1.7))
title("Precision/recall graph")
Sensperf <- performance(ROCRpred, 'sens','spec')
plot(Sensperf, colorize = TRUE, text.adj = c(-0.2,1.7))
title("Sensitivity/specificity plot")
Liftperf <- performance(ROCRpred, 'lift','rpp')
plot(Liftperf, colorize = TRUE, text.adj = c(-0.2,1.7))
title("Lift chart")
```

也可以得到 ROC 曲线下面的面积 (area under the curve, AUC; area under the receiver operating characteristic curve, AUROC):

```
pr <- prediction(pred, w_test$Donate)
auc <- performance(ROCRpred, measure = "auc")
auc <- auc@y.values[[1]]
auc
```

得到 $AUC = 0.793774$, 即 ROC 曲线下面的面积 (总面积 =1), AUC 代表了一个随机抽取的病人相对于随机抽取的无病的人被正确地判别 (有较前的排序) 为有病的概率, 等于 Wilcoxon-Mann-Whitney 检验统计量的值.

因此, AUC 有下面几个等价的解释:

- 随机抽取的阳性病人排在随机抽取的阴性病人前面的概率.
- 阳性病人排在随机抽取的阴性病人前面的比例的期望值.
- 排序停止于随机抽取一个阴性病人之前的期望 TPR.
- 阴性病人排在随机抽取的阳性病人后面的比例的期望值.
- 排序停止于随机抽取一个阳性病人之后的期望 FPR.

传统上, 在诊断问题上, 有人如同成绩一样把 AUC 粗略地按照所属的区间分成五等:

(A) $AUC \in (0.9, 1)$ 意味着优秀 (excellent).
(B) $AUC \in (0.8, 0.9)$ 意味着良好 (good).
(C) $AUC \in (0.7, 0.8)$ 意味着尚可 (fair).
(D) $AUC \in (0.6, 0.7)$ 意味着不好 (poor).
(F) $AUC \in (0.5, 0.6)$ 意味着失败 (fail).

4.5.5 三种分类方法对例4.2献血数据的交叉验证比较

我们用 Logistic 回归、线性判别分析及二次判别分析对例4.2数据分类做 10 折交叉验证. 我们还是用前面的 `Fold` 函数来把数据分成 10 份, 再对每个方法做交叉验证 (这里 Logistic 回归的阈值取 $p_t = 0.44$). 代码为:

```
D=4;Z=10;n=nrow(w);
mm=Fold(Z,w,D,8888)
ff=Donate ~ .

E=vector()
J=1;E[J]=0
for(i in 1:Z){
  m=mm[[i]]
  a=glm(ff,w[-m,],family="binomial")
  z=(predict(a,w[m,],type="response")>0.44)
  u=rep(levels(w[m,D])[2],nrow(w[m,]));u[!z]=levels(w[m,D])[1]
  E[J]=E[J]+sum(w[m,D]!=u)
}
J=J+1;E[J]=0
for(i in 1:Z){
  m=mm[[i]]
  a=lda(ff,w[-m,])
  u=predict(a,w[m,])$class
  E[J]=E[J]+sum(w[m,D]!=u)
}
J=J+1;E[J]=0
for(i in 1:Z){
  m=mm[[i]]
  a=qda(ff,w[-m,])
  u=predict(a,w[m,])$class
  E[J]=E[J]+sum(w[m,D]!=u)
}
E/n
```

得到三种方法的 10 折交叉验证误差分别为 0.2126(Logistic 回归), 0.2313(线性判别分析), 0.2233(二次判别分析).

4.6 机器学习模型简介及案例

除了上面提到的线性和二次判别分析, 还有其他许多分类方法, 比如 adaboost, 随机森林 (random forest)、决策树 (decision tree)、支持向量机 (support vector machine, SVM)、k 最近邻方法 (k nearest neighbors) 等等. 这些方法都属于机器学习方法, 完全可以写成一本厚厚的教科书, 本书主要介绍经典的多元分析教程, 不可能包含所有其他分类的内容. 下面仅就决策树分类 (亦称分类树)、随机森林及 adaboost 分类做简单介绍.

4.6.1 决策树分类模型

用于分类的决策树和前面提到的用于回归的决策树只存在很少的差别, 下面用例4.1数字笔迹识别来做一简单的说明, 用下面代码来做决策树的拟合, 并输出结果和图形 (见图4.4):

```
w=read.csv("pendigits.csv");w[,17]=factor(w[,17])
library(rpart.plot)
(a=rpart(V17~.,w))
rpart.plot(a,type=1,extra=2)
```

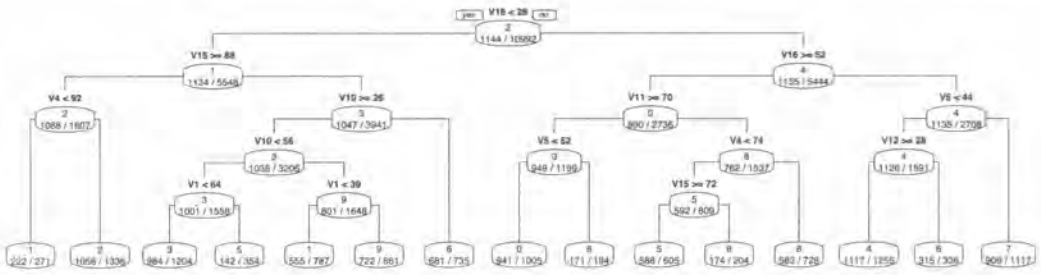

图 4.4 对例4.1数字笔迹识别做决策树分类图

从计算结果中可以看出每个节点中的因变量哪个数字 (从 0 到 9 的 10 个数字) 占多数, 比如在根节点, "2" 占多数, 图中的 1144/10992 意味着在 10992 个观测值中, "2" 有 1144 个, 是最多的, 然后, 根据拆分变量 V16 是否小于 28, 把数据分成两部分. 左边部分有 5548 个观测值, 那里因变量的值中 "1" 占多数, 有 1134 个; 右边有 5444 个观测值, 其因变量值中 "4" 占多数, 有 1135 个. 与前面回归一样有选择拆分变量的问题, 比如, 为什么在根节点选择 V16 在 28 的分割呢? 这里不能用均方误差, 而要用一些纯度的度量, 决策树分类在每个节点选择使某种不纯度量变小最快的变量分割. 假定一共有 J 类, 类别 $i \in \{1, 2, \ldots, J\}$. 令 p_i 为第 i 类在一个节点数据中的比例, 则两种最常用的不纯度量 (impurity measures) 为:

- Gini 不纯度 (Gini impurity)[4]:

$$Gini = 1 - \sum_{i=1}^{J} {p_i}^2.$$

- 熵 (entropy):

$$Entropy = -\sum_{i=1}^{J} p_i \log_2 p_i.$$

显然, 当一个节点只有一类时, 这两个度量都为 0, 而各个节点的类和整个数据比例一样时, 不纯度量最大. 使得熵变纯的方法叫信息增益法. 不同的软件或选项会用不同的方式表示纯度.

[4]也称 Gini 度量、Gini 指数、Gini 系数, 不要和经济领域的 Gini 指数弄混了.

直观来说, 如果某个节点只有因变量的一个水平 (比如这个例子中的某一个数字), 那么该节点就是最纯的, 不用再拆分了. 如果各个水平完全均匀地存在于一个节点, 那就是最不纯的. 因此, 拆分的过程就是把不纯的数据集不断拆分成越来越纯的一些子集. **在任何节点的首选拆分变量是相对于其他变量最能够纯化数据的变量**. 人们可以看到, 这棵决策树的终节点没有一个是纯的.

和图4.4平行的结果输出为:

```
n= 10992

node), split, n, loss, yval, (yprob)
* denotes terminal node

1) root 10992 9848 2 (0.1 0.1 0.1 0.096 0.1 0.096 0.096 0.1 0.096 0.096)
  2) V16< 27.98 5548 4414 1 (0.0088 0.2 0.2 0.19 0.0016 0.076 0.12 0.018 0.0041 0.17)
    4) V15>=87.5 1607  519 2 (0 0.27 0.68 0.00062 0.00062 0 0 0.042 0 0.015)
      8) V4< 92.5 271   49 1 (0 0.82 0.11 0 0.0037 0 0 0.0074 0 0.059) *
      9) V4>=92.5 1336  278 2 (0 0.15 0.79 0.00075 0 0 0 0.049 0 0.006) *
    5) V15< 87.5 3941 2894 3 (0.012 0.18 0.011 0.27 0.002 0.11 0.17 0.0084 0.0058 0.23)
     10) V10>=25.75 3206 2170 3 (0.015 0.21 0.011 0.32 0.0022 0.13 0.0019 0.0066 0.0072 0.29)
       20) V10< 56.5 1558  557 3 (0.0058 0.075 0.022 0.64 0.0026 0.16 0.0039 0.0077 0.0071 0.076)
         40) V1< 64.17 1204  220 3 (0.005 0.026 0.027 0.82 0.00083 0.086 0 0.01 0.0083 0.02) *
         41) V1>=64.17 354  212 5 (0.0085 0.24 0.0028 0.048 0.0085 0.4 0.017 0 0.0028 0.27) *
       21) V10>=56.5 1648  847 9 (0.024 0.34 0.00061 0.021 0.0018 0.11 0 0.0055 0.0073 0.49)
         42) V1< 38.97 787  232 1 (0.033 0.71 0.0013 0.041 0.0013 0.091 0 0.011 0.015 0.1) *
         43) V1>=38.97 861  139 9 (0.016 0.015 0 0.0035 0.0023 0.12 0 0 0 0.84) *
     11) V10< 25.75 735   54 6 (0 0.031 0.0095 0.015 0.0014 0 0.93 0.016 0 0) *
   3) V16>=27.98 5444 4309 4 (0.2 0.0017 0.0026 0.0013 0.21 0.12 0.068 0.19 0.19 0.02)
     6) V16>=51.5 2736 1746 0 (0.36 0 0 0 0 0.23 0.0091 0.036 0.36 0.0084)
      12) V11>=70.5 1199  250 0 (0.79 0 0 0 0 0 0.02 0.0058 0.18 0.0067)
        24) V5< 52.5 1005   64 0 (0.94 0 0 0 0 0 0.024 0 0.04 0) *
        25) V5>=52.5 194   23 8 (0.041 0 0 0 0 0 0 0.036 0.88 0.041) *
      13) V11< 70.5 1537  775 8 (0.027 0 0 0 0 0.41 0.00065 0.059 0.5 0.0098)
        26) V4< 73.5 809  217 5 (0.027 0 0 0 0 0.73 0.0012 0 0.22 0.019)
          52) V15>=71.5 605   17 5 (0 0 0 0 0 0.97 0.0017 0 0.0083 0.018) *
          53) V15< 71.5 204   30 8 (0.11 0 0 0 0 0.02 0 0 0.85 0.02) *
        27) V4>=73.5 728  145 8 (0.026 0 0 0 0 0.048 0 0.12 0.8 0) *
     7) V16< 51.5 2708 1573 4 (0.038 0.0033 0.0052 0.0026 0.42 0.0015 0.13 0.35 0.022 0.032)
      14) V5< 44.5 1591  465 4 (0.05 0 0.00063 0 0.71 0.0025 0.21 0.022 0.0031 0.0057)
        28) V12>=27.5 1255  138 4 (0.064 0 0 0 0.89 0 0.013 0.027 0.0008 0.0056) *
        29) V12< 27.5 336   21 6 (0 0 0.003 0 0.027 0.012 0.94 0.003 0.012 0.006) *
      15) V5>=44.5 1117  208 7 (0.021 0.0081 0.012 0.0063 0.0081 0 0.012 0.81 0.048 0.071) *
```

可以看出决策树的分类并不是很理想, 还没有做交叉验证, 误判率就很高, 利用下面的代码:

```
sum(w$V17!=predict(a,w,type="class"))#1830
sum(w$V17!=predict(a,w,type="class"))/nrow(w)#0.1665
```

可以得到误判的观测值有 1830 个, 而误判率为 16.65%. 下面进行 10 折交叉验证 (和前面线性判别和二次判别相同的训练集和测试集):

```
library(rpart.plot)
y=NULL->yh
for(i in 1:Z){
  m=mm[[i]]
  a=rpart(V17~.,w[-m,])
  y=c(y,w[m,D])
  yh=c(yh,predict(a,w[m,],type="class"))
}
table(y,yh)
(er=sum(y!=yh)/dim(w)[1])
```

输出的混淆矩阵为:

```
> table(y,yh)
    yh
y       1    2    3    4    5    6    7    8    9   10
  1   917   28    5    2   86   10    0   25   60   10
  2     0  831  186   54    0   27   23    8    0   14
  3     0   77 1012   30    0    4    8   13    0    0
  4     0   45    2  973    0   17    9    6    0    3
  5     0    6    0    0 1110    4   12    9    0    3
  6     0   72    1   97    0  710    3    0   47  125
  7    23    5   25    0   18   10  962   13    0    0
  8     0   34   38    8   36    0   30  906   90    0
  9    40   15    0   10    1    8    6   54  921    0
  10    0   91    5   55    7   73    2   76    9  737
> (er=sum(y!=yh)/dim(w)[1])
[1] 0.1740357
```

交叉验证误差率为 0.1740357, 和不进行交叉验证类似, 结果不如经典判别分析.

注意

这里使用的决策树是一个基本的分类器, 它本身的结果不一定好, 也不一定稳定. 但是, 如果把决策树作为一个基石, 形成组合算法 (ensemble algorithm), 则会大大改进结果. 下面要介绍的基于决策树的组合方法是随机森林.

4.6.2 随机森林分类模型

随机森林分类的原理和随机森林回归的原理类似, 也对原始数据集做很多次放回抽样, 得到很多不同的数据集, 然后针对每个数据集建立一个决策树, 在每个节点, 拆分变量不是由所有变量竞争, 而是由**随机挑选**的少数变量竞争, 而且每棵树都长到底. 随机森林分类的最终结果是所有树等权投票, 也就是说, 一个新的观测值通过许多棵树得到许多结果, 就以

例4.1数字笔迹识别来说, 新观测值可能被某棵树判断为 "1", 被另一棵树判断为 "7" 或其他数字等等, 最终结果为多数决策树所得到的结果, 相当于等权投票.

随机森林分类的代码和随机森林回归的代码类似:

```
set.seed(1010)
RF=randomForest(V17~.,w,importance=TRUE,proximity=TRUE,localImp=TRUE)
sum(w$V17!=predict(RF,w))/nrow(w)
```

对于全部数据来说, 不做交叉验证, 误判率为 0. 比单独一棵决策树好多了. 再用代码 RF, 得到利用 OOB 数据[5]做的交叉验证及所谓**混淆矩阵**. 交叉验证结果表明误判率为 0.0076 或 0.76%. 混淆矩阵对角线之外的数目是误分的观测值个数. 可以看出把 "1" 分错的最多 (错分率为 0.0244969379).

```
Call:
randomForest(formula = V17 ~ ., data = w, importance = TRUE,
proximity = TRUE, localImp = TRUE)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 4

        OOB estimate of  error rate: 0.76%
Confusion matrix:
     0    1    2    3    4    5    6    7    8    9  class.error
0 1133    0    0    0    3    0    2    0    4    1 0.0087489064
1    0 1115   19    7    0    1    0    0    0    1 0.0244969379
2    0    8 1134    0    0    0    0    2    0    0 0.0087412587
3    0    1    1 1046    0    3    0    2    0    2 0.0085308057
4    1    0    0    0 1143    0    0    0    0    0 0.0008741259
5    0    0    0    4    0 1046    1    0    1    3 0.0085308057
6    0    0    1    0    0    1 1054    0    0    0 0.0018939394
7    0    1    5    1    0    0    0 1135    0    0 0.0061295972
8    0    0    0    0    0    1    0    1 1052    1 0.0028436019
9    1    2    0    0    0    0    0    0    1 1051 0.0037914692
```

随机森林的变量重要性、局部重要性、部分依赖性、离群点的图

首先介绍随机森林的变量重要性, 这可由 RF$importance 得到, 我们把变量重要性用图4.5来展示. 按照从左到右、从上到下的次序, 图4.5的前面 10 个图是从 0 到 9 的各个水平 (类) 的各变量重要性, 倒数第二个图 (右二图) 是用 OOB 数据作为训练集所做的交叉验证得到的度量, 意义为删除某个变量后精确度降低的程度 (降低越多, 变量越重要), 最后一个图 (右下图) 为每个变量在各个节点的表现 (使得数据变 "纯" 的能力, 这里用 Gini 不纯度量). 作图代码如下:

[5]OOB 数据为每次有放回再抽样时没有抽到的观测值数据, 可以用作天然的交叉验证测试集.

```
par(mfrow=c(3,4))
for (i in 1:10)
barplot(t(RF$importance[,i]),names.arg=row.names(RF$importance),
cex.names = 0.4,col=4,las=0,xlab=paste("digit:",colnames(RF$importance)[i]))
for (i in 11:12)
barplot(t(RF$importance[,i]),names.arg=row.names(RF$importance),
cex.names = 0.4,col=2,las=0,xlab=colnames(RF$importance)[i])
```

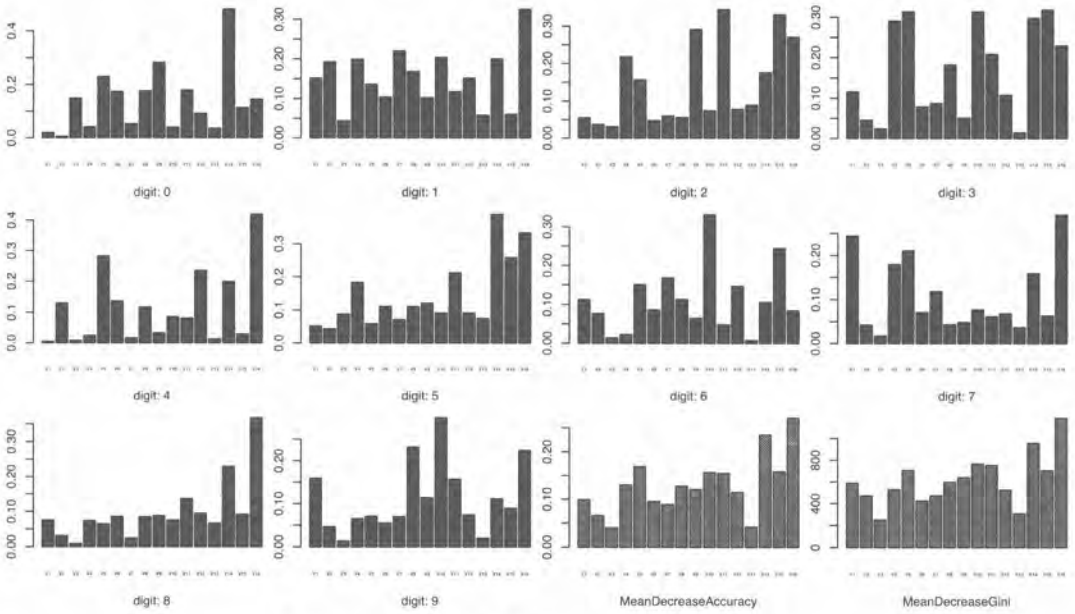

图 4.5 随机森林拟合例4.1数字笔迹识别数据的变量重要性

除了总体的变量重要性, 还有局部变量重要性, 这是每个变量对每一个观测值的影响 (RF$local), 对于例4.1, 它是一个 16×10992 矩阵, 图形 (见图4.6) 的每一条线代表一个观测值 (一共 10992 个), 而横坐标为变量名, 纵坐标为局部重要性的度量. 产生图4.6的代码如下:

```
matplot(1:16,RF$local,type = "l",xaxt="n",
  xlab="Variable",ylab = "Local importance")
axis(side = 1,at = 1:16,labels = rownames(RF$local),las=0,cex.axis=1)
```

除此之外, 还有每个自变量对因变量的部分依赖性图, 我们把重要的两个变量 V16 和 V6 点图 (见图4.7), 代码为:

```
par(mfrow=c(1,2))
partialPlot(RF, pred.data=w, x.var=V16)
partialPlot(RF, pred.data=w, x.var=V6)
```

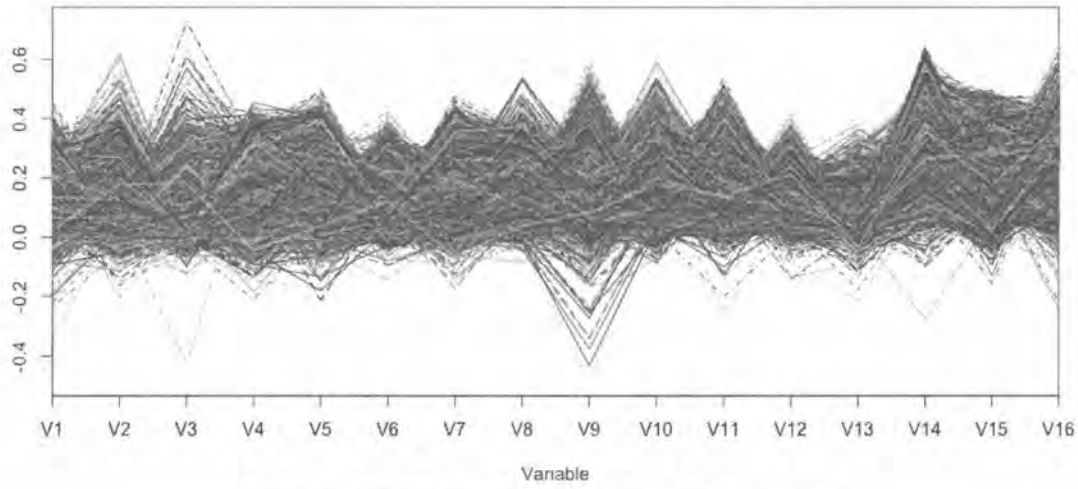

图 4.6 随机森林拟合例4.1数字笔迹识别数据的局部变量重要性图

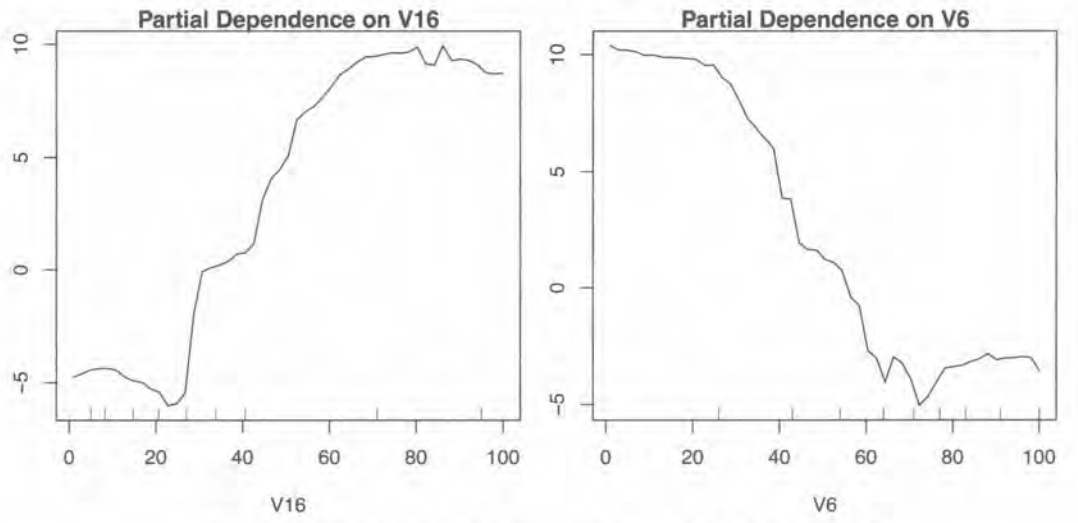

图 4.7 随机森林拟合例4.1数字笔迹识别数据对 2 个变量的部分依赖性图

图4.7还说明因变量对第一个自变量大体上是递增关系, 而对第二个自变量大体上是递减关系.

类似于随机森林回归, 随机森林分类还根据是否经常出现在同样节点给出了观测值之间的亲疏程度 (RF$proximity), 根据这个度量可以得到离群点图 (见图4.8), 代码为:

```
plot(outlier(RF$proximity), type="h",ylab="Outlying measure")
```

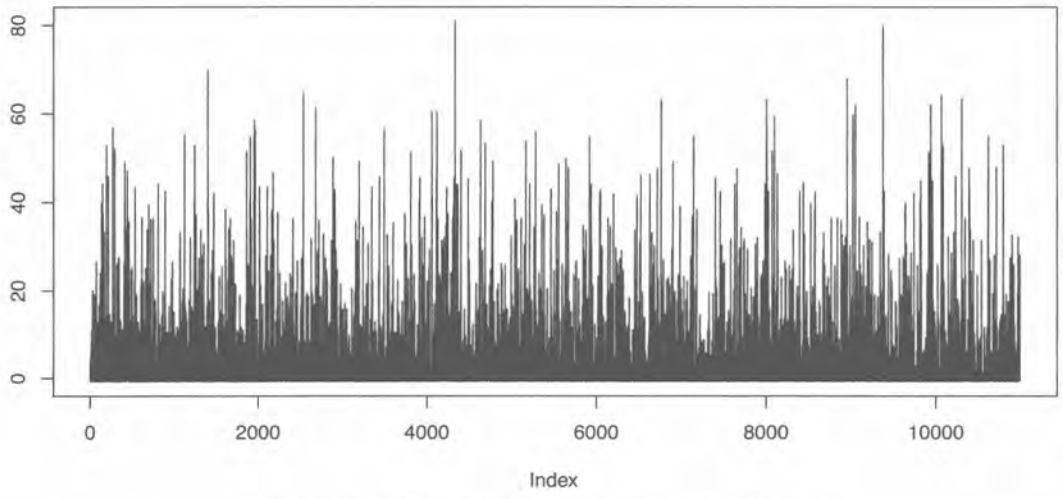

图 4.8 随机森林拟合例4.1数字笔迹识别数据的离群点图

随机森林预测的交叉验证

用与前面线性判别和二次判别同样的 10 折训练集和测试集做随机森林拟合的交叉验证:

```
library(randomForest)
y=NULL->yh
for(i in 1:Z){
  m=mm[[i]]
  a=randomForest(V17~.,w[-m,])
  y=c(y,w[m,D])
  yh=c(yh,predict(a,w[m,]))
}
table(y,yh)
(er=sum(y!=yh)/dim(w)[1])
```

得到的混淆矩阵为:

```
> table(y,yh)
   yh
y       1    2    3    4    5    6    7    8    9   10
  1  1133    0    0    0    3    0    2    0    4    1
  2     0 1112   22    6    0    1    0    1    0    1
  3     0    7 1134    1    0    0    0    2    0    0
  4     0    2    1 1047    0    1    0    2    0    2
  5     1    0    0    0 1143    0    0    0    0    0
  6     0    0    0    4    0 1047    0    0    1    3
  7     0    0    0    0    0    1 1055    0    0    0
  8     0    2    5    1    0    0    0 1134    0    0
```

```
  9     1    0    0    0    0    1    0    1 1051    1
  10    1    2    0    0    0    0    0    0    1 1051
> (er=sum(y!=yh)/dim(w)[1])
[1] 0.007732897
```

交叉验证结果表明误判率为 0.0077, 和 OOB 误差相似, 该误差只有前面二次判别的 42%.

4.6.3 adaboost 分类模型

adaboost(通常写成 AdaBoost, 是 adaptive boosting 的缩写) 是一种非常优秀的基于决策树的组合方法. 其原理很简单, 和随机森林类似, 要建立很多决策树. adaboost 每次用自助法抽样来构建树时, 都根据前一棵树的结果对于误判的观测值增加抽样权重, 使得下一棵树能够对误判的观测值有更多的代表性. 最终的结果由所有的树加权投票得到, 权重根据各棵树的精确度来确定. 我们要用程序包adabag[6]中的函数boosting(), 该函数通过自助法加权抽样构建的决策树的默认棵数为 100.

数学语言描述二元因变量时的 adaboost 分类 *

假定样本为 $(\boldsymbol{X}_1, y_1), (\boldsymbol{X}_2, y_2), \ldots, (\boldsymbol{X}_n, y_n)$, 为简化描述, 假定因变量为二分类变量, $y \in \{0, 1\}$, 即只有两类. adaboost 的具体步骤为:

(1) 对观测值点选择初始的自助法抽样权重 $w_i^{[0]} = 1/n$ $(i = 1, 2, \ldots, n)$, 设 $m = 0$.

(2) 把 m 增加 1. 用基本方法 (这里是分类树) 拟合加权抽样的数据, 权重为 $\boldsymbol{w}^{[m-1]}$, 产生分类器 $\hat{g}^{[m]}(\cdot)$.

(3) 计算样本内的加权误判率:

$$\mathrm{err}^{[m]} = \sum_{i=1}^n w_i^{[m-1]} I(Y_i \neq \hat{g}^{[m]}(\boldsymbol{X}_i)) / \sum_{i=1}^n w_i^{[m-1]},$$
$$\alpha^{[m]} = \ln\left(\frac{1-\mathrm{err}^{[m]}}{\mathrm{err}^{[m]}}\right),$$

然后更新权重:

$$\tilde{w}_i = w_i^{[m-1]} \exp(\alpha^{[m]} I(Y_i \neq \hat{g}^{[m]}(\boldsymbol{X}_i))),$$
$$w_i^{[m]} = \tilde{w}_i / \sum_{j=1}^n \tilde{w}_j.$$

(4) 重复第 (2) 步和第 (3) 步, 直到预先设定的步数 $m = m_{\mathrm{stop}}$. 这样就建立了根据加权投票来分类的组合分类器:

$$\hat{f}_{\mathrm{adaboost}}(\boldsymbol{x}) = \arg\min_{y \in \{0,1\}} \sum_{m=1}^{m_{\mathrm{stop}}} \alpha^{[m]} I(\hat{g}^{[m]}(\boldsymbol{x}) = y).$$

[6] Alfaro, E., Gamez, M. Garcia, N.(2013). adabag: An R Package for Classification with Boosting and Bagging. *Journal of Statistical Software*, 54(2), 1-35. URL http://www.jstatsoft.org/v54/i02/.

4.6.4 adaboost 拟合例4.1数字笔迹识别全部数据

我们使用以下代码:

```
w=read.csv("pendigits.csv")#读入数据
w[,17]=factor(w[,17]) #因子化哑元变量
library(adabag)
set.seed(1010)
a=boosting(class~.,w)
table(w$V17,predict(a,w)$class)
```

得到的混淆矩阵为:

```
     0    1    2    3    4    5    6    7    8    9
0 1138    0    0    0    0    0    0    0    4    1
1    0 1117   20    6    0    0    0    0    0    0
2    0    3 1137    0    0    0    0    4    0    0
3    0    2    1 1047    0    1    0    3    0    1
4    1    0    0    0 1143    0    0    0    0    0
5    0    1    0    1    0 1051    0    0    0    2
6    0    0    0    0    0    1 1054    1    0    0
7    0    4    2    0    0    0    0 1136    0    0
8    3    0    0    0    0    1    0    1 1050    0
9    0    2    0    0    0    1    0    0    1 1051
```

对角线外的元素占的比率为 0.006186, 也就是误判率. 这里的行是真实的类, 列是预测的类.

4.6.5 adaboost 的变量重要性

使用代码 `a$importance` 可以得到变量重要性, 我们把变量重要性展示在图4.9中.

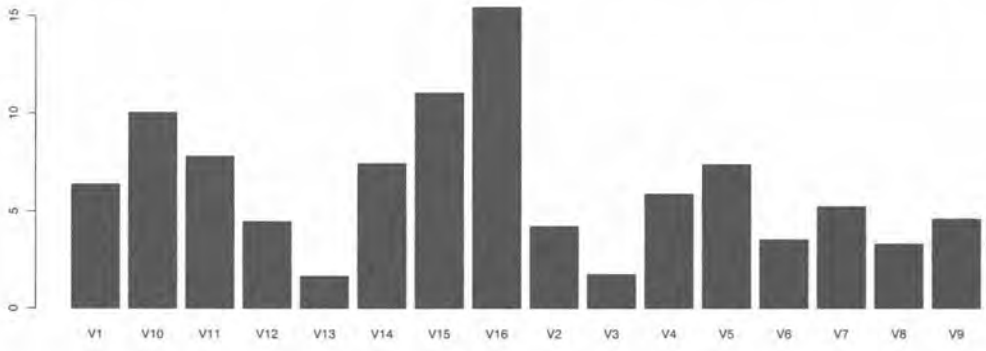

图 4.9 adaboost 拟合例4.1数字笔迹识别数据的变量重要性

4.6.6 adaboost 的交叉验证

我们用程序包adabag 自带的程序boosting.cv 做 10 折交叉验证.

```
set.seed(1010)
pen.cv=boosting.cv(V17~.,v=10, data=w)
pen.cv$confusion #混淆矩阵
pen.cv$error #误判率
```

得到的误判率为 0.01491994178, 而混淆矩阵为 (这里的行是预测的类, 列是真实的类):

```
                Observed Class
Predicted Class    0    1    2    3    4    5    6    7    8    9
              0 1133    0    0    0    2    0    1    0    4    1
              1    0 1096    8    6    0    2    0    5    0    3
              2    0   31 1125    1    1    0    0    5    0    0
              3    0   12    0 1039    0    6    0    4    0    1
              4    2    0    0    0 1138    1    2    0    1    0
              5    0    1    0    2    0 1036    2    0    1    4
              6    0    1    0    0    0    2 1050    1    0    0
              7    0    2   11    5    3    0    1 1122    1    1
              8    6    0    0    0    0    4    0    4 1045    1
              9    2    0    0    2    0    4    0    1    3 1044
```

4.6.7 各种分类模型的比较

下面仅就例4.1数字笔迹识别数据做几种分类方法 (随机森林、adaboost、支持向量机、k 最近邻方法、线性判别分析、二次判别分析) 的比较, 其中一些方法本书没有介绍, 详情请参看吴喜之所著的《应用回归及分类——基于 R》(2016). 这里的测试集和训练集由前面程序确定得到的误判率显示在图4.10之中. 计算过程的 R 代码如下:

```
w=read.csv("pendigits.csv")#读入数据
w[,17]=factor(w[,17]) #因子化哑元变量
Z=10;D=17
mm=Fold(Z=10,w,D=17,1010) #10折交叉验证集的确定

library(MASS);library(kknn)
library(randomForest);library(kernlab);library(adabag)
D=17;n=nrow(w)
ff=paste(names(w)[D],"~.");ff=as.formula(ff)
###########
E=vector()

#随机森林分类
J=1;error=NULL;set.seed(1010)
for(i in 1:Z){
  m=mm[[i]]
  a=randomForest(ff,data=w[-m,])
```

```
  e=sum(w[m,D]!=predict(a,w[m,]))
  error=c(error,e)
}
E[J]=sum(error)
#adaboost分类
J=J+1;error=NULL;set.seed(8888)
for(i in 1:Z){
  m=mm[[i]]
  a=boosting(ff,w[-m,])
  e=sum(w[m,D]!=predict(a,w[m,])$class)
  error=c(error,e)
}
E[J]=sum(error)
#支持向量机分类
J=J+1;error=NULL;set.seed(1010)
for(i in 1:Z){
  m=mm[[i]]
  a=ksvm(ff,w[-m,])
  e=sum(w[m,D]!=predict(a,w[m,]))
  error=c(error,e)
}
E[J]=sum(error)
#k最近邻方法分类
J=J+1;error=NULL;set.seed(1010)
for(i in 1:Z){
  m=mm[[i]]
  e=sum(w[m,D]!=kknn(ff, train= w[-m,],test=w[m,])$fit)
  error=c(error,e)
}
E[J]=sum(error)
#线性判别分析分类
J=J+1;error=NULL;set.seed(1010)
for(i in 1:Z){
  m=mm[[i]]
  a=lda(ff,w[-m,])
  e=sum(w[m,D]!=predict(a,w[m,])$class)
  error=c(error,e)
}
E[J]=sum(error)
#二次判别分析分类
J=J+1;error=NULL;set.seed(1010)
for(i in 1:Z){
  m=mm[[i]]
  a=qda(ff,w[-m,])
```

```
  e=sum(w[m,D]!=predict(a,w[m,])$class)
  error=c(error,e)
}
E[J]=sum(error)
print(E/n)
barplot(E/n,names.arg = c("Random Forest",
  "Adaboost","SVM","KNN","LDA","QDA"),col=4)
```

输出的 5 个误差为:

方法	Random Forest	AdaBoost	SVM	KNN	LDA	QDA
误差	0.008006	0.016103	0.005913	0.006004	0.130913	0.018377

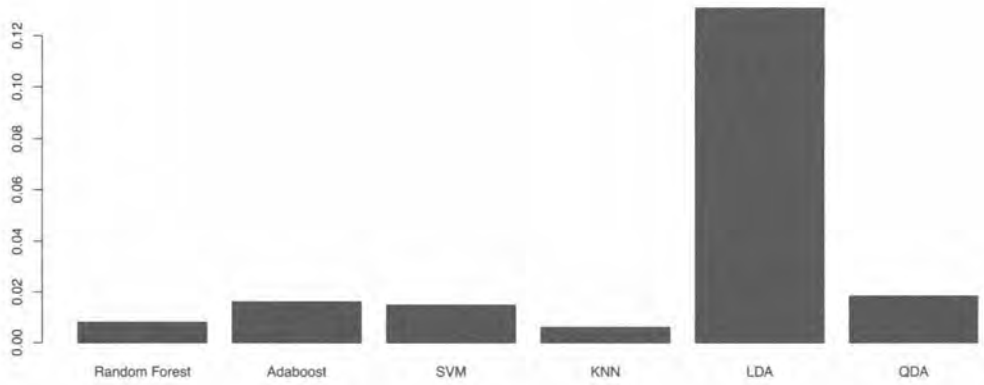

图 4.10 对例4.1数字笔迹识别数据分类的 6 种方法的测试集预测误判率

4.7 本章计算的 Python 代码

输入模块的代码为:

```
import numpy as np
import pandas as pd
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.metrics import confusion_matrix
```

4.7.1 产生 Z 折交叉验证集的代码

随机产生 Z 折交叉验证集对于分类来说, 最好能够把因变量每个水平都比较均匀地分为 Z 份. 因此, 这里的程序比回归稍微复杂一些.

```
def Fold(y,X,Z,seed=8888):
    id0=np.arange(len(y))
    part=[]
    for i in np.unique(y):
        part.append(id0[y==i]) #part is index of y=i
    Zid=[];Xn=[];yn=[]
    np.random.seed(seed)
    for k in part:
        np.random.shuffle(k)#for each y=i shuffle index
        yn.extend(y[k])
        Xn.extend(X[k])
        Zid.extend((list(range(Z))*int(len(k)/Z+1))[:len(k)])
    return Zid, yn, Xn
```

输入数据的代码为:

```
v= pd.read_csv('pendigits.csv',index_col=False)
X=np.array(v[v.columns[:16]])#自变量
y=np.array(v[v.columns[16]])#因变量
```

产生 10 折交叉验证集的要素:

```
Zid, yn, Xn=Fold(y=y,X=X,Z=10,seed=1010)
yn=np.array(yn)
Xn=np.array(Xn)
```

上面代码中的 yn 和 Xn 是重新随机排序的因变量和自变量, Zid 为对应重新排序变量的各折的代号 (由 $0, 1, \ldots, (Z-1)$ 组成).

做交叉验证分类的函数代码为:

```
def CCV(clf,Zid, Xn, yn):
    y_pred=[];yN=[]
    for j in np.unique(Zid): #j has Z kinds of values
        clf.fit(Xn[Zid!=j],yn[Zid!=j])
        yN.extend(yn[Zid==j])
        y_pred.extend(clf.predict(Xn[Zid==j]))
    y_pred=np.array(y_pred)
    yN=np.array(yN)
    error=np.sum(yN!=y_pred)/len(y)
    cmatrix=confusion_matrix(yN,y_pred)
    return(error,cmatrix)
```

4.7.2 线性判别分析及交叉验证

不做交叉验证

线性判别分析拟合代码 (不做交叉验证) 为:

```
lda=LinearDiscriminantAnalysis()
lda.fit(X,y)
y_pred=lda.predict(X)
y_score=lda.score(X,y)
print (confusion_matrix(y,y_pred))
print ('Misclassification rate=', 1-y_score)
```

输出的所谓混淆矩阵 (行代表正确的类, 列代表预测的类, 对角线为各个类正确划分的计数) 和误判率 (12.9%) 为:

```
[[1015   13    0    0   13    1    5    0   89    7]
 [   0  770  211   26    0   73    6   35    1   21]
 [   0   19 1113    0    0    0    0   12    0    0]
 [   0   13    1 1029    0    0    0    8    0    4]
 [   0    0    1    0 1108    9    4    1    0   21]
 [   0   11    0   38    1  718   27    6    7  247]
 [   9    0    0    0    2    1 1029    0   15    0]
 [   0   46   29   57    2    5    0  979    4   20]
 [  59   37    0   25    0   70    6    4  839   15]
 [   1   34    0    8    7   26    7    2    0  970]]
Misclassification rate= 0.129366812227
```

10 折交叉验证

代码为:

```
lda=LinearDiscriminantAnalysis()
er,cm=CCV(lda)
print("Error rate=",er)
print (cm)
```

输出的混淆矩阵和误判率 (13.0%) 为:

```
Error rate= 0.130458515284
[[1013   13    0    0   13    2    5    0   88    9]
 [   0  773  209   27    1   71    5   34    1   22]
 [   0   20 1111    0    0    0    0   13    0    0]
 [   0   12    1 1029    0    0    0    9    0    4]
 [   0    0    1    0 1108   10    3    1    0   21]
 [   0   12    0   38    1  717   26    4    7  250]
```

```
[   9    0    0    0    2    1 1027    0   17    0]
[   0   47   30   56    3    5    0  978    4   19]
[  59   39    0   26    0   70    6    3  837   15]
[   1   34    0   10    8   28    7    2    0  965]]
```

4.7.3 二次判别分析

不做交叉验证

二次判别分析不做交叉验证的代码和线性判别分析几乎一样:

```
qda=QuadraticDiscriminantAnalysis()
qda.fit(X,y)
y_pred2=qda.predict(X)
y_score2=qda.score(X,y)
print( confusion_matrix(y,y_pred2))
print( 'Misclassification rate=', 1-y_score2)
```

输出的混淆矩阵和误判率 (1.50%) 为:

```
[[1127    1    0    0    0    0    1    0   11    3]
 [   0 1107   17    3    0    5    1    0    2    8]
 [   0   18 1117    0    0    0    0    8    1    0]
 [   0    4    1 1037    0    1    0    5    2    5]
 [   0    0    1    0 1130    1    0    0    0   12]
 [   0    3    0    1    1 1028    0    0    4   18]
 [   0    1    0    0    0    2 1046    0    7    0]
 [   0    2    0    0    0    0    0 1134    1    5]
 [   0    0    0    0    0    0    0    2 1051    2]
 [   0    2    0    0    0    0    0    2    1 1050]]
Misclassification rate= 0.0150109170306
```

10 折交叉验证

代码和线性判别分析类似:

```
qda=QuadraticDiscriminantAnalysis()
er,cm=CCV(qda)
print("Error rate=",er)
print (cm)
```

下面是输出的误判率 (1.79%) 和混淆矩阵:

```
Error rate= 0.017922125182
[[1125    1    0    0    0    0    2    0   12    3]
 [   0 1101   17    3    0    7    1    0    3   11]
 [   0   22 1110    0    0    0    0   10    2    0]
 [   0    6    1 1034    0    1    0    6    2    5]
 [   0    0    1    0 1130    0    0    0    0   13]
 [   0    3    0    1    1 1026    0    0    5   19]
 [   0    1    0    0    0    2 1044    0    9    0]
 [   0    5    0    0    0    0    0 1125    3    9]
 [   0    0    0    0    0    0    0    2 1051    2]
 [   1    2    0    0    0    0    0    2    1 1049]]
```

4.7.4 Logistic 回归

Logistic 回归对例4.2献血数据的拟合

首先输入需要的模块:

```
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import roc_curve, auc, confusion_matrix
```

输入数据, 并随机分成 80%的训练集和 20%的测试集:

```
w = pd.read_csv("trans.csv")
w['intercept'] = 1.0 #加上截距
X_cols=w.columns[[4,0,1,2]];y_col=w.columns[3]
X=w[X_cols];y=w[y_col]
X=np.array(X);y=np.array(y)
Zid=np.ones(len(y));Zid[:int(len(y)*0.20)]=0
X_train=X[Zid==1,:];y_train=y[Zid==1]
X_test=X[Zid==0,:];y_test=y[Zid==0]
```

做 Logistic 回归:

```
result = sm.Logit(y_train, X_train).fit()
print (result.summary())
```

得到下面的输出:

```
Optimization terminated successfully.
        Current function value: 0.425548
        Iterations 6
                          Logit Regression Results
```

```
==============================================================================
Dep. Variable:                      y   No. Observations:                  599
Model:                          Logit   Df Residuals:                      595
Method:                           MLE   Df Model:                            3
Date:                Thu, 26 Apr 2018   Pseudo R-squ.:                  0.1122
Time:                        15:30:40   Log-Likelihood:                -254.90
converged:                       True   LL-Null:                       -287.13
LLR p-value:                6.567e-14
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -0.6778      0.221     -3.073      0.002      -1.110      -0.246
x1            -0.0901      0.019     -4.758      0.000      -0.127      -0.053
x2             0.1200      0.029      4.140      0.000       0.063       0.177
x3            -0.0179      0.007     -2.671      0.008      -0.031      -0.005
==============================================================================
```

对例4.2献血数据 Logistic 回归拟合的 ROC 曲线

求对测试集的预测值, 计算 ROC 曲线及曲线下的面积:

```
y_pred= result.predict(X_test)
fpr, tpr, thresholds =roc_curve(y_test, y_pred)
roc_auc = auc(fpr, tpr)
print("Area under the ROC curve : %f" % roc_auc)
```

得到 $AUC = 0.643156$. 在一张图 (见图4.11) 中画出 ROC 曲线 (TPR 对 FPR) 及曲线 (TPR 对 $1-FPR$):

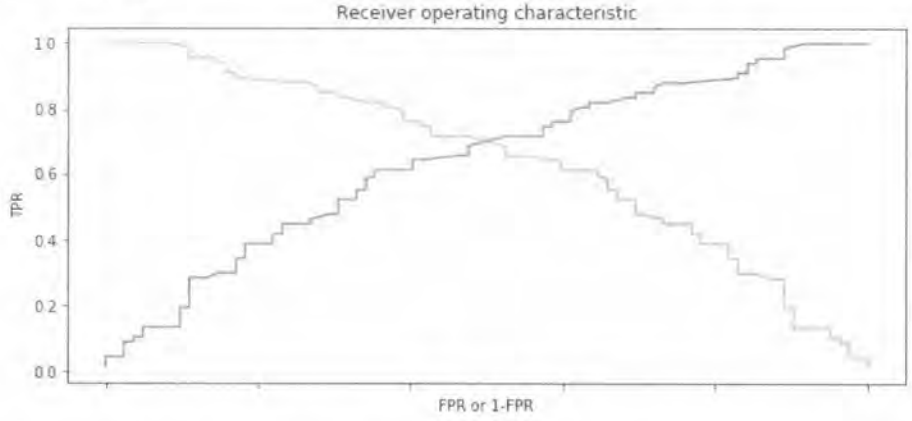

图 4.11 例4.2献血数据 Logistic 回归的 ROC 曲线 (TPR 对 FPR) 及曲线 (TPR 对 $1-FPR$)

根据图4.11, 最优的阈值是曲线中 TRP 高而 FPR 低的位置, 可以用 $TPR-(1-FPR)$ 等于 0 的位置的阈值. 这可以用下面代码得到, 等于 0.3421.

```
thresholds[np.argmin(np.abs(tpr-(1-fpr)))]
```

测试集的混淆矩阵及误判率 (利用上面的阈值) 可由下面代码得到, 误判率为 0.3758.

```
print(confusion_matrix(y_test, 1*(y_pred>0.3421)))
print(np.sum(y_test!=1*(y_pred>0.3421))/len(y_test))
```

绘制图4.11的代码如下:

```
i = np.arange(len(tpr)) # index for df
roc = pd.DataFrame({'fpr' : pd.Series(fpr, index=i),
                    'tpr' : pd.Series(tpr, index = i),
                    '1-fpr' : pd.Series(1-fpr, index = i),
                    'tf' : pd.Series(tpr - (1-fpr), index = i),
                    'thresholds' : pd.Series(thresholds, index = i)})
roc.ix[(roc.tf-0).abs().argsort()[:1]]

# Plot tpr vs 1-fpr
fig, ax = plt.subplots(figsize=(10,4.4))
plt.plot(roc['fpr'],roc['tpr'])
plt.plot(roc['1-fpr'],roc['tpr'])
plt.xlabel('FPR or 1-FPR')
plt.ylabel('TPR')
plt.title('Receiver operating characteristic')
ax.set_xticklabels([])
```

4.7.5 对例4.2献血数据做 3 个经典分类方法的交叉验证

利用先前的函数 Fold 把数据分成 10 折, 输入需要的模块:

```
X=np.array(X);y=np.array(y)
Zid, yn, Xn=Fold(y=y,X=X,Z=10,seed=1010)
yn=np.array(yn)
Xn=np.array(Xn)
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
```

利用先前的函数 CCV 做交叉验证:

```
names = ["Linear Discriminant Analysis", "Quadratic Discriminant
Analysis","Logistic Regression"]
classifiers = [
    LinearDiscriminantAnalysis(),
    QuadraticDiscriminantAnalysis(),LogisticRegression()]
A=list()
for clf in classifiers:
    np.random.seed(1010)
```

```
    er,cm=CCV(clf)
    A.append(er)
print(A)
```

得到三种方法的误差分别为 0.2313(线性判别分析)、0.2380(二次判别分析)、0.2313(Logistic 回归).

4.7.6 随机森林分类

我们直接做随机森林分类的交叉验证:

```
from sklearn.ensemble import RandomForestClassifier
RF=RandomForestClassifier(n_estimators=500)
np.random.seed(1010)
er,cm=CCV(RF)
print("Error rate=",er)
print (cm)
```

输出的混淆矩阵和误判率 (0.80%) 为:

```
Error rate= 0.0080058224163
[[1133    0    0    0    3    0    2    0    3    2]
 [   0 1113   22    6    0    1    0    0    0    1]
 [   0    9 1133    0    0    0    0    2    0    0]
 [   0    3    1 1047    0    1    0    1    0    2]
 [   1    0    0    0 1143    0    0    0    0    0]
 [   0    0    0    4    0 1047    0    0    1    3]
 [   1    0    0    0    0    1 1053    1    0    0]
 [   0    2    5    1    0    0    0 1134    0    0]
 [   0    0    0    0    0    1    0    3 1050    1]
 [   1    2    0    0    0    0    0    0    1 1051]]
```

按照交叉验证, 前面二次判别的误差率是随机森林误差率的 2.2 倍.

4.7.7 对例4.1数字笔迹识别数据做 5 个机器学习方法分类的交叉验证比较

最后, 用下面代码计算几种分类方法的交叉验证的误判率, 这些方法包括最近邻方法、线性支持向量机 (SVM)、决策树、随机森林、高斯朴素贝叶斯方法.

首先载入必要的模块:

```
import numpy as np
import matplotlib
matplotlib.use('TkAgg')#使用Windows不需要这个命令
import matplotlib.pyplot as plt
%matplotlib inline
import scipy.stats as stats
```

```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_moons, make_circles, make_classification
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
```

载入各种分类方法和名称:

```
names = ["Nearest Neighbors", "Linear SVM", "Decision Tree",
         "Random Forest", "Naive Bayes"]
classifiers = [
    KNeighborsClassifier(3),
    SVC(kernel="linear", C=0.025),
    DecisionTreeClassifier(max_depth=5),
    RandomForestClassifier(n_estimators=500),
    GaussianNB()]
```

进行计算:

```
A=list()
for clf in classifiers:
    np.random.seed(1010)
    er,cm=CCV(clf)
    A.append(er)
```

打印 5 个误判率条形图 (见图4.12) 的代码:

```
plt.bar(range(len(A)),A,)
plt.ylabel('Error rate')
plt.title('Error rate')
plt.xticks(np.arange(len(names)),names,rotation=90)
```

相应于图4.12的 5 种机器学习方法分类 10 折交叉验证的误差率的数值输出 (`print(A)`):

```
[0.0067321688500727806,
 0.025927947598253277,
 0.13100436681222707,
 0.0080058224163027658,
 0.13991994177583697]
```

可以看出, 对于这个数据, 在这几种分类方法中, k 最近邻方法误差率最小, 随机森林次之, 然后是支持向量机, 剩下两种和前面三种有一定距离.

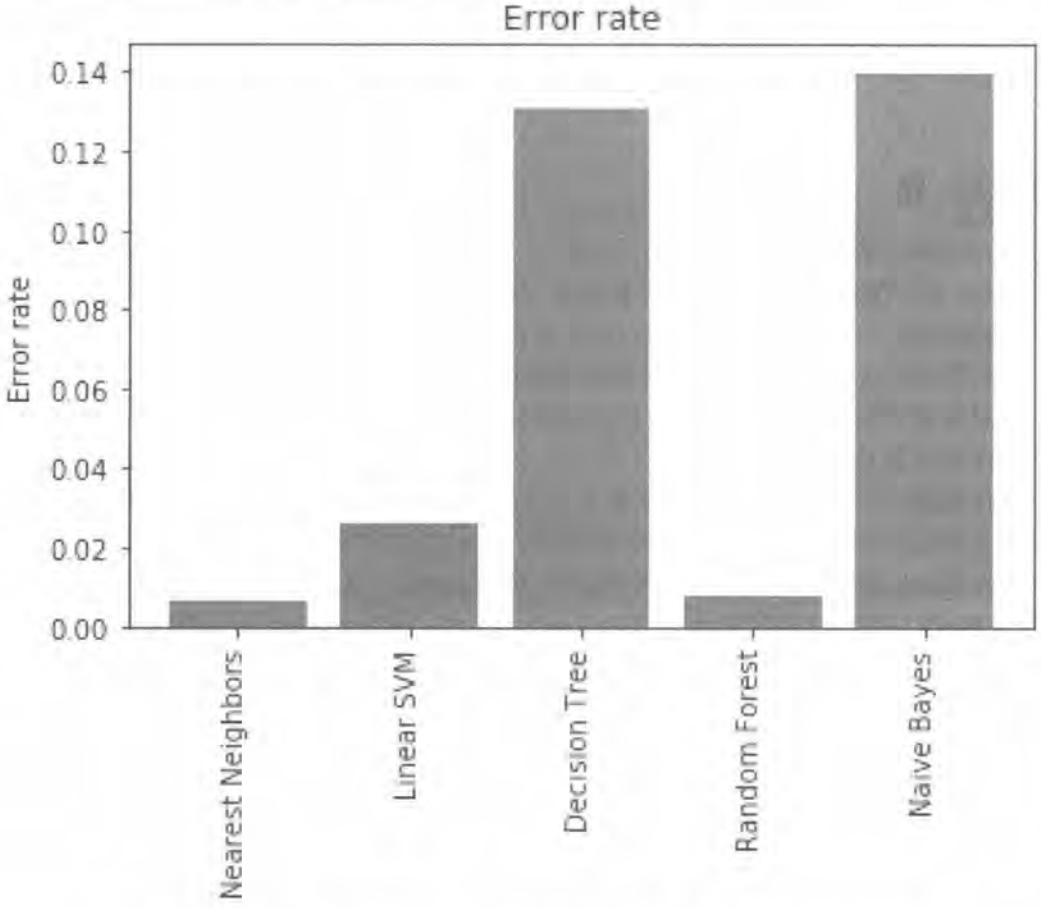

图 4.12 例4.1数字笔迹识别数据的 5 种机器学习方法分类 10 折交叉验证的误差率

注意

请注意下面几点:

- 对于任何方法, 都很难说绝对好或者绝对不好, 每一种方法都可能适合于某些数据, 而不适合另外一些数据.
- 由于每一种软件都有众多的选项, 使用不同的选项也会使得一个方法的效果存在很大差异.
- 许多机器学习方法有随机性, 因此, 如果不选定随机种子, 重复做同一个数据的拟合可能得到不相同的结果, 当然差别不会很大.
- 由于不同的软件编写不同, 不同人设计的同一个名字下的软件很可能在运算速度、结果的精度等方面存在区别 (有时区别还很大).
- 交叉验证时, 如果训练集和测试集都是随机选择的, 那么结果也不会相同. 如果数据量大, 差别不会很大.
- 进行 D 折交叉验证时, 对于定性变量各个水平的在各折中的平衡很重要, 否则各折的验证结果会有较大起伏. 在例4.1数字笔迹识别数据的拟合的各折数据集的确定中, 我们只考虑了唯一的定性变量 (因变量) 的平衡 (通过函数 `Fold`). 但如果自变量也有定性变量, 则应该尽可

能平衡，这在数据量小的时候比较困难，有如不完全设计问题，在现成的软件程序中还找不到普遍的方法.

- 实际上，因变量和自变量交叉验证各个折的平衡虽然对于定量变量很重要，但实践起来会遇到棘手的理论问题及编程问题.

4.8 习 题

1. 指出回归和分类的基本概念的唯一区别.
2. 请用例7.6的数据，以变量 ID 为目标变量，用线性判别和随机森林方法数据做分类. 请做到以下几点:
   (1) 对两种方法不做交叉验证，得到混淆矩阵及误判率.
   (2) 对两种方法做出 5 折交叉验证的混淆矩阵，求出交叉验证时的误判率，并且和不用交叉验证的结果做比较.
   (3) 找到随机森林方法 (没有刻意做交叉验证时) 的 OOB 误判率，这个误判率和上面交叉验证的结果是否有本质上的不同?
   (4) 找到随机森林方法产生的变量重要性，并且点图.
3. 关于 Logistic 回归:
   (1) 对于癌症诊断，假阳性 (FP) 误差和假阴性 (FN) 误差所造成的后果一样吗? 应该根据什么确定阈值?
   (2) 请解释图4.3中各个图的含义，如何根据这些图对模型做出描述或决定阈值?
   (3) 模型 (4.5.1) 中为什么没有误差项? 它和线性模型 (3.1.1) 相比有什么异同?
   (4) 为什么不能用和线性模型类似的最小二乘法解 Logistic 模型?
   (5) 有人认为 ROC 曲线比误判率更能说明问题，你觉得呢?
4. 请用例7.7胎心宫缩监护数据，以变量 NSP 为目标变量，用决策树、adaboost 和随机森林方法对数据做分类.
   (1) 对三种方法不做交叉验证，得到混淆矩阵及误判率.
   (2) 画出随机森林的部分依赖图，根据这个图找出重要变量以及每个变量如何影响因变量.
   (3) 画出随机森林方法产生的变量重要性图，并且与根据部分依赖性图所得到的变量是否重要的结论做比较.
   (4) 画出 adaboost 的变量重要性图，并且与随机森林的变量重要性图做比较.
   (5) 对三种方法做分类的 10 折交叉验证，比较结果.
5. 看懂 (无论是 R 还是 Python 版本的) 函数 `Fold` 的编写逻辑，理解和改进程序.

第 5 章　主成分分析

包括本章在内的以后各章都属于统计学习中的无指导 (或无监督) 学习, 也就是说, 这里没有因变量, 各变量之间都或多或少相关. 在具体问题中, 随着研究目的的变化, 各个变量的作用不尽相同, 也可能会很不相同, 各个变量在分析中具体起什么作用, 得让数据本身说话. 实际上, 在数据分析中, 最重要的是明白原始数据中应该包含何种变量的数据, 选择变量往往比分析方法本身更重要.

5.1　基本思想和例子

5.1.1 数据中变量之间的关系与降维的可能性

在多元数据的研究中, 很多人探讨**降维**的可能性. 降维就是用根据原始变量构造的少数变量来代替多数变量, 这样会简化计算过程, 提高分析的效率并使得结果容易解释. 本章的**主成分分析** (principal component analysis, PCA) 就是一种降维的方法. 然而, 并不是所有多元数据分析都能够降维. 数据的特征或者信息是由各个变量描述的, 在不同的数据中, 变量之间的关系也不同, 这些关系确定了降维的可能性.

(1) **数据各变量不相关的情况.** 如果一个数据的变量互相独立, 或者不相关, 那么, 每个变量对于这个数据的整体特征的贡献都是独立的, 不与其他变量混杂. 比如对各个国家进行描述时, 面积、气候带、人均收入这三个变量基本不相关, 它们不可互相替代, 每个变量都有自己的意义, 都对国家的描述有独立贡献. 这样的不相关变量组成的数据就不可能降维, 对其做主成分分析没有意义.

(2) **数据各变量相关的情况.** 关于上面关于国家描述的问题, 如果还包含了人均病床数、人均收入、新生儿死亡率、文盲比例、外汇储备、发明专利数、进出口贸易额等变量, 这些变量不但不独立, 反而可能十分相关. 那么, 由于变量之间的关系, 谈论单独某个变量对国家特征的单独贡献就没有多大的意义, 这些相关的变量对一个国家的描述有着 "集体贡献", 总的贡献绝对不是单个变量的贡献之和. 在这种情况下, 就有降维的可能.

(3) **主成分分析是在变量相关时寻找不相关的线性组合.** 在存在相关变量的情况下, 人们可以由相关的变量形成若干变量的线性组合, 而这些线性组合不相关. 这样, 每种组合对问题的贡献就和其他组合无关了, 这时谈论每个组合对问题的单独贡献就有意义了. 这些不相关的组合实际上形成了新的变量, 在主成分分析中, 这些组合称为**成分**, 而在因子分析中称为**因子**. 最终的成分 (或因子) 中, 如果只有少数具有主要贡献 (这又产生什么是 "主要贡献" 的问题), 人们就可以用这少数的成分 (或因子) 来代替众多的原始变量, 这就实现了降维.

主成分分析的主要目的及其与下一章的因子分析的区别主要不在于数据或研究目的, 而在

于数学模型的差异. 一般来说, 因子分析降维的效果可能优于主成分分析. 主成分分析的优点是原理简单、计算容易. 因子分析需要很强的关于模型的数学假定. 另外, 主成分分析所选取的少数主成分只能代表原数据的部分信息, 而因子分析中的少数因子 (按照数学假定) 代表全部数据的信息.

在做主成分分析及因子分析时, 最主观、对结果影响最大的决策就是选择哪些变量参与建模, 这方面怎么强调都不为过. 比如, 在对学校进行评价时, 如果只考虑建筑面积、教学设施状态、规章的种类、教师人数、领导的级别等, 而不考虑科研成果的价值和历届学生的出路, 那么, 无论用什么数学模型都没有意义. 此外, 较强的关于数据的假定往往使得模型与现实的距离更远. 由于结果中的很大部分源于主观的假定, 所做的分析不易使人信服.

5.1.2 从例子中产生的问题

例 5.1. 教师数据 (full.aaup.csv) 该数据来自美国大学教授协会 (AAUP) 年度普查, 包括分别按照正教授和副教授的平均工资和补贴 (1994 年) 统计的数据.[1]一共有 17 个变量, 其中有 4 个定性变量: FICE (联邦 ID 代码)、College (学校名称)、State (州)、Type (类型: I、IIA, 或 IIB). 还有 13 个数量变量: ASF (正教授平均工资)、ASA1 (副教授平均工资)、ASA2 (助理教授平均工资)、ASALL (所有级别平均工资)、ACF (正教授平均补贴)、ACA1 (副教授平均补贴)、ACA2 (助理教授平均补贴)、ACALL (所有级别平均补贴)、NF (正教授人数)、NA1 (副教授人数)、NA2 (助理教授人数)、NIN (助教人数)、NALL (所有级别教授数目). 该数据涉及 1161 个学校. 其中只有 13 个是定量变量, 而经典主成分分析只能处理定量变量. 因此, 虽然数据为 1161×17 矩阵的形式, 但我们只使用其中的 13 列数据, 即 1161×13 矩阵的形式. 这里的数据是对原始数据通过程序包 `missForest`[2]的函数 `missForest()` 弥补缺失值后的数据.

这个数据包含了许多变量, 如果这些变量互相独立, 也就是说它们都有自己的意义, 我们没有必要把它们放在一起来分析研究. 然而, 例5.1中的 13 个变量都是相关的变量, 每个变量不能代替另一个变量, 但一个变量的变化会导致其他变量的变化. 这样, 我们就希望并且有可能找到这些变量的若干代表, 这些代表是以这些变量的函数 (线性组合) 形式出现, 而且每个代表有自己的特点, 各个代表不相关.

下面的代码和结果显示了例5.1中的 13 个变量的线性相关系数矩阵:

```
> w=read.csv("full.aaup.csv")[,-(1:4)]#读入除前4个之外的13个变量
> cor(w)
        ASF ASA1 ASA2 ASALL  ACF ACA1 ACA2 ACALL   NF  NA1  NA2   NIN NALL
ASF   1.00 0.95 0.92 0.970 0.99 0.93 0.90 0.959 0.58 0.56 0.55 0.151 0.58
ASA1  0.95 1.00 0.94 0.945 0.94 0.98 0.93 0.940 0.49 0.50 0.48 0.137 0.50
ASA2  0.92 0.94 1.00 0.932 0.91 0.93 0.97 0.924 0.53 0.53 0.51 0.150 0.54
ASALL 0.97 0.94 0.93 1.000 0.96 0.93 0.92 0.989 0.61 0.54 0.51 0.078 0.57
ACF   0.99 0.94 0.91 0.965 1.00 0.95 0.92 0.972 0.55 0.54 0.52 0.131 0.55
```

[1]来自http://www.umass.edu/statdata/statdata/stat-rmult.html.

[2]Daniel J. Stekhoven (2013). missForest: Nonparametric Missing Value Imputation using Random Forest. R package version 1.4. Stekhoven D. J., & Buehlmann, P. (2012). MissForest—non-parametric missing value imputation for mixed-type data. *Bioinformatics*, 28(1), 112-118.

```
ACA1  0.93 0.98 0.93 0.932 0.95 1.00 0.95 0.952 0.47 0.47 0.46 0.114 0.48
ACA2  0.90 0.93 0.97 0.918 0.92 0.95 1.00 0.938 0.52 0.52 0.50 0.138 0.52
ACALL 0.96 0.94 0.92 0.989 0.97 0.95 0.94 1.000 0.58 0.52 0.49 0.067 0.55
NF    0.58 0.49 0.53 0.606 0.55 0.47 0.52 0.583 1.00 0.89 0.86 0.356 0.96
NA1   0.56 0.50 0.53 0.543 0.54 0.47 0.52 0.524 0.89 1.00 0.92 0.453 0.96
NA2   0.55 0.48 0.51 0.509 0.52 0.46 0.50 0.487 0.86 0.92 1.00 0.536 0.95
NIN   0.15 0.14 0.15 0.078 0.13 0.11 0.14 0.067 0.36 0.45 0.54 1.000 0.48
NALL  0.58 0.50 0.54 0.570 0.55 0.48 0.52 0.548 0.96 0.96 0.95 0.482 1.00
```

上面输出表明, 以字母 "A" 开头的和收入有关的 8 个变量之间的相关系数比较大, 都在 0.9 以上; 以 "N" 开头的和人数有关的后面 5 个变量之间的相关系数 (除了变量 NIN 有些 "另类" 之外) 也不小, 但不如前 8 个变量那么相关; 而前 8 个变量与后 5 个变量之间的相关系数最大只有 0.606, 其余则小于 0.6.

这个线性相关系数矩阵可以用更直观的方法展示 (见图5.1中的左、右两图). 产生图5.1的代码为:

```
library(corrplot)
corrplot(cor(w), type = "lower", diag=FALSE, tl.col = "black", tl.srt = 45)
library(lattice)
rgb.palette <- colorRampPalette(c("white", "black"), space = "rgb")
levelplot(cor(w), las=1,main="Correlation matrix", xlab="", ylab="",
   col.regions=rgb.palette(120), cuts=100, at=seq(0,1,0.01))
```

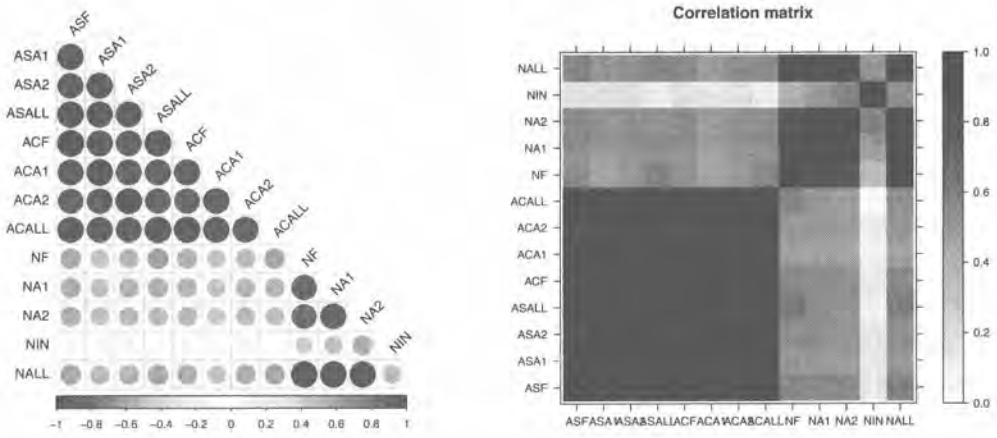

图 5.1 例5.1教师数据的线性相关系数两种直观展示

我们将使这 13 个定量变量形成 13 个不相关的线性组合 (称为成分), 并选取其最有代表性的少数一两个组合来描述这个数据. 我们将要考虑的问题是:

1. 如何选择变量的代表, 即**成分**.
2. 选择多少成分, 以及这些成分代表了多少原来数据的信息.
3. 这些成分的选择有没有意义.
4. 是否有可能尝试解释这些成分.

例 5.2. 江景 (XMY.jpg) 这个例子的数据很简单, 就是一张照片 (见图5.2), 当图片像素很大时, 传输和存储都不太方便, 这时需要降低像素, 主成分分析的降维功能可以用来做这件事情.

图 5.2 例5.2江景照片

例5.2的照片是彩色照片 (注意, 本书为黑白印刷, 彩色无法显示), 其数据由 3 个 3264×2448 矩阵组成, 矩阵中的每个数目取值在区间 $[0,1]$, 表示像素大小 (越接近 0 越深). 这三个矩阵分别代表红色、绿色和蓝色, 也就是说, 照片有 $3264\times 2448=7990272$ 个点, 每个点由三种颜色混合而成. 和例5.1教师数据不同, 这里不需要解释各个成分的意义, 但希望在降维之后尽量保持图片的可识别性.

例 5.3. 洛杉矶街区 (LA.Neighborhoods.csv) 该数据为美国人口普查局 2000 年数据.[3] 一共有 110 个街区、15 个变量, 变量情况见下表 (表中的 API 为涉及学生成绩的 Academic Performance Index 缩写).

[3]数据网站: `http://wiki.stat.ucla.edu/socr/index.php/SOCR_{}Data_{}LA_{}Neighborhoods_{}Data`.

变量名	描述	性质	变量名	描述	性质
LA.Nbhd	街区名字	分类	Black	黑人比例	数量
Income	收入中位数	数量	Latino	拉美裔比例	数量
Schools	公立学校 API 成绩中位数	数量	White	白人比例	数量
Diversity	种族多样性 (0 ~ 10 分)	分类/定序	Population	人口	数量
Age	年龄中位数	数量	Area	面积	数量
Homes	有房家庭比例	数量	Longitude	经度	数量
Vets	复员军人比例	数量	Latitude	纬度	数量
Asian	亚裔比例	数量			

这个例子我们将在后面作为一个案例来分析.

5.2 问题和计算

例5.1教师数据有 13 个定量变量, 因此对应 13 维空间, 数据为 13 维空间的点云. 如何降维呢? 由于 13 维空间无法直观观测, 下面以 2 维空间为例, 看关于 2 维空间如何降维到 1 维空间的直观说明. 图5.3给出了两种 2 维变量点云的示例.

在图5.3的两个图中, 左图是两个相关变量的点云, 而由于两个变量相关, 图形呈现出一个椭圆形状, 其中沿着椭圆长轴的方向有较多的信息, 即**方差最大的方向**, 而沿着椭圆短轴方向的方差较小. 如果椭圆短轴的长度缩短到 0, 则该 2 维点云实际上是 1 维点云, 已经降低维度. 因此, 左图所示的数据应该可以降维. 图5.3的右图是两个不相关变量的点云, 呈现出一个长短轴没有明显差别的近似于圆的 "椭圆", 对该点云做降维没有什么意义, 或者说, 把右图的点朝任何方向投影都不会有多大的区别.

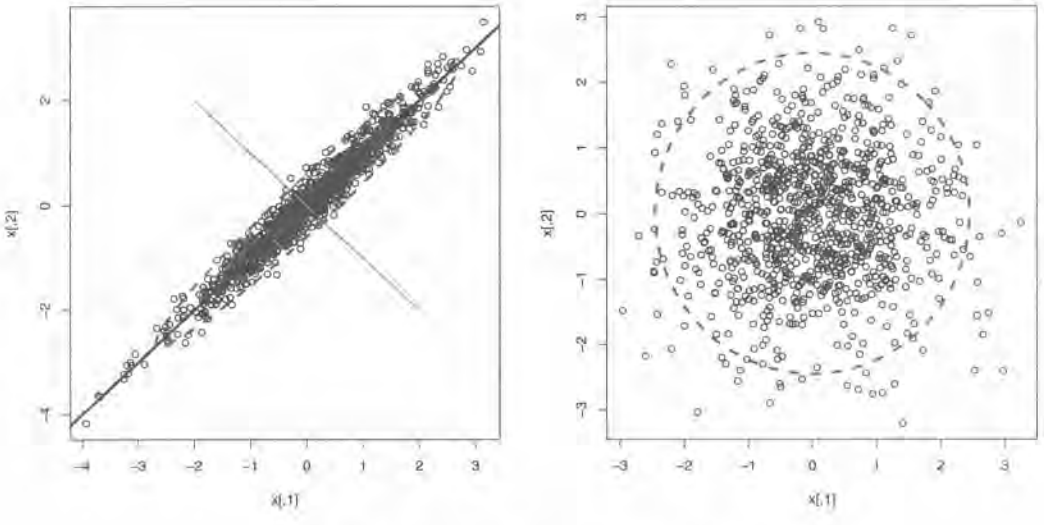

图 5.3 相关变量点云 (左) 及无关变量点云 (右)

注意

图5.3的比较 "完美" 的椭圆形状会在变量近似地来自二元正态分布总体时出现. 但在实际应用中, 数据很难真正来自多元正态分布总体, 其点云可能完全不像一个椭圆或空间椭球. 虽然人们往往主观假定数据服从多元正态分布, 但这最多只是为了方便而做的无法核对的假定, 因为我们永远无法证明一个数据真正来自正态分布 (或任何分布).

5.2.1 求协方差矩阵及样本相关阵的特征值和特征向量

一般来说, 如果数据具有以向量 $\boldsymbol{x}=(x_1,x_2,\ldots,x_p)^\top$ 表示的 p 个变量, 我们要在这些变量组成的 p 维空间中找到方差最大的方向. 由于任何方向都是原始变量的线性组合, 我们的目的是寻找变量线性组合 $\boldsymbol{\gamma}^\top\boldsymbol{x}=(\gamma_1x_1,\gamma_2x_2,\ldots,\gamma_px_p,)$ 的系数 $\boldsymbol{\gamma}=(\gamma_1,\gamma_2,\ldots,\gamma_p)^\top$, 使得 $\mathrm{Var}(\boldsymbol{\gamma}^\top\boldsymbol{x})=\boldsymbol{\gamma}^\top\boldsymbol{\Sigma}\boldsymbol{\gamma}$ 最大. 这个最大化过程可以用数学语言表达为:

$$\boldsymbol{\gamma}=\arg\max_{\boldsymbol{\gamma}}\mathrm{Var}(\boldsymbol{\gamma}^\top\boldsymbol{x})=\arg\max_{\boldsymbol{\gamma}}\boldsymbol{\gamma}^\top\boldsymbol{\Sigma}\boldsymbol{\gamma},$$

式中, $\boldsymbol{\Sigma}$ 为这 p 个变量的协方差矩阵. 在线性代数中, 这等价于求 $\boldsymbol{\Sigma}$ 的最大特征值所对应的特征向量, 记该方向的单位特征向量为 $\boldsymbol{\gamma}_1$, 称这个向量所代表的线性组合 $\boldsymbol{\gamma}_1^\top\boldsymbol{x}$ 为第一主成分. 类似地, 在与 $\boldsymbol{\gamma}_1$ 正交的空间, 再找使得方差最大的单位向量 $\boldsymbol{\gamma}_2$, 如此下去, 可以找到 p 个主成分 (即 p 个特征向量) $\boldsymbol{\gamma}_1,\boldsymbol{\gamma}_2,\ldots,\boldsymbol{\gamma}_p$, 以及对应这些特征向量的 ($\boldsymbol{\Sigma}$ 的) p 个特征值 $\lambda_1,\lambda_2,\ldots,\lambda_p$ ($\lambda_1\geqslant\lambda_2\geqslant\cdots\geqslant\lambda_p$). 为叙述方便, 不妨假定这些 $\boldsymbol{\gamma}_i$ 为单位向量.

在记号上, 对于总体协方差阵 $\boldsymbol{\Sigma}$, 其特征值和特征向量分别表示为:

$$\boldsymbol{\lambda}=(\lambda_1,\lambda_2,\ldots,\lambda_p)^\top\quad\text{和}\quad\boldsymbol{\Gamma}=[\boldsymbol{\gamma}_1,\boldsymbol{\gamma}_2,\ldots,\boldsymbol{\gamma}_p]=\begin{bmatrix}\gamma_{11}&\gamma_{21}&\cdots&\gamma_{p1}\\\gamma_{12}&\gamma_{22}&\cdots&\gamma_{p2}\\\vdots&\vdots&\ddots&\vdots\\\gamma_{1p}&\gamma_{2p}&\cdots&\gamma_{pp}\end{bmatrix},$$

式中, 列向量 $\boldsymbol{\gamma}_i=(\gamma_{i1},\gamma_{i2},\ldots,\gamma_{ip})^\top$ $(i=1,2,\ldots,p)$.

在实际问题中, 由于变量的协方差阵 $\boldsymbol{\Sigma}$ 未知, 可用样本协方差阵代替. 基于数据, 上述求 p 个特征值及对应的 p 个正交向量的过程是一步完成的. 为了避免结果受变量不同量纲的影响, 通常用样本相关阵 $\boldsymbol{R}$ 代替样本协方差阵, 这等价于用标准化数据的协方差阵. 在本章中, 我们用样本相关阵, 而不是样本协方差阵.

在记号上, 对于**样本相关阵$\boldsymbol{R}$**, 其特征值和特征向量分别表示为:

$$\boldsymbol{\ell}=(\boldsymbol{\ell}_1,\boldsymbol{\ell}_2,\ldots,\boldsymbol{\ell}_p)^\top\quad\text{和}\quad\boldsymbol{A}=[\boldsymbol{a}_1,\boldsymbol{a}_2,\ldots,\boldsymbol{a}_p]=\begin{bmatrix}a_{11}&a_{21}&\cdots&a_{p1}\\a_{12}&a_{22}&\cdots&a_{p2}\\\vdots&\vdots&\ddots&\vdots\\a_{1p}&a_{2p}&\cdots&a_{pp}\end{bmatrix},$$

式中, 列向量 $\boldsymbol{a}_i=(a_{i1},a_{i2},\ldots,a_{ip})^\top$ $(i=1,2,\ldots,p)$.

对于例5.1教师数据, 读取数据、对原始数据的定性变量仅保存学校名称、求数据 (不包括校名变量) 相关阵的特征值和特征向量的 R 代码为:

```
w=read.csv("full.aaup.csv")#读取数据
u=w[,-c(1,3,4)] #去掉无关定性变量(仅保存学校名称)
a=eigen(cor(u[,-1]))#解(不包括校名变量的)相关矩阵的特征值问题
a$va #输出特征值
data.frame(a$ve,row.names=names(u)[-1])[,1:4]#带有变量名的4个特征向量
```

输出的 13 个特征值及相应于 4 个最大特征值的 4 个特征向量为:

```
> a$value
$values
 [1] 9.30528828 2.43991068 0.70609232 0.18019578
 [5] 0.13667776 0.07624507 0.06672505 0.05213008
 [9] 0.02702074 0.00498025 0.00256042 0.00190026
[13] 0.00027331

> data.frame(a$vectors,row.names=names(u)[-1])[,1:4]
               X1         X2          X3          X4
ASF   -0.31273824 -0.1352035  0.02804996 -0.30274173
ASA1  -0.30502296 -0.1891519  0.11642372  0.11576999
ASA2  -0.30568127 -0.1559534  0.08826369  0.41070451
ASALL -0.31187011 -0.1573095 -0.07045888 -0.30599752
ACF   -0.31098143 -0.1579576  0.03176836 -0.27621024
ACA1  -0.30214696 -0.2076846  0.11074421  0.13769700
ACA2  -0.30382362 -0.1683954  0.08791390  0.42882607
ACALL -0.31028804 -0.1751978 -0.05822265 -0.25605919
NF    -0.24334906  0.3569358 -0.33116826 -0.33087167
NA1   -0.24106277  0.3936994 -0.17830881  0.26108295
NA2   -0.23529148  0.4123763 -0.04912807  0.26515144
NIN   -0.08890082  0.3897086  0.87898802 -0.19914972
NALL  -0.24752096  0.4067060 -0.17891781 -0.01775209
```

上面 (R 代码 `$values` 下面) 的输出数值是按照降序排列的 13 个特征值 (可用 R 代码 `a$value` 单独得到), 而后面是相应于上面 13 个特征值的 13 个单位列向量 (这里只显示其中的前 4 个, 全部向量可用 R 代码 `a$vector` 单独得到), 如果这 13 个列向量组成的矩阵用 $\boldsymbol{A}=[\boldsymbol{a}_1,\boldsymbol{a}_2,\ldots,\boldsymbol{a}_p]$ 表示 (R 代码 `a$vector`), 那么 $\boldsymbol{A}^{\top}\boldsymbol{A}=\boldsymbol{I}$, 也就是说这些向量正交. 这可以用 R 代码 `round(t(a$ve)%*%a$ve,14)` 验证 (这里的 `round(,14)` 是保持小数点后面 14 位的四舍五入函数. 在浮点运算中, 一般不大可能得到纯粹的整数 0 或 1, 小数点后十几位才出现非零数字的数目可认为是 0).

5.2.2 求各个成分的贡献率和累积贡献率

通常, 一个成分的重要性用其方差占总方差的比例来度量, 比如, 对于第 i 个成分, 其方差为特征值 λ_i, 而总方差为 $\sum_{j=1}^{p}\lambda_j$, 因此该成分占总方差的比例为 $\lambda_i/\sum_{j=1}^{p}\lambda_j$, 即该特

征值占各个成分特征值之和的比率. 对于例5.1教师数据, 从样本相关阵 $\boldsymbol{R}$ 计算出来的第 i 个主成分的贡献率为 $\ell_i/\sum_{j=1}^{p}\ell_j$, 前 k 个主成分的累积贡献率为 $\sum_{j=1}^{k}\ell_i/\sum_{j=1}^{p}\ell_j$.

上面通过 R 计算出来的特征向量 $\boldsymbol{a}_i$ $(i=1,2,\ldots,p)$ 都是单位向量, 这时**第 i 个主成分与第 j 个原始变量之间的相关系数为** $\sqrt{\ell_i}a_{ij}$, 也就是说, 第 i 个特征向量 $\boldsymbol{a}_i$ 与相应特征值的平方根 $\sqrt{\ell_i}$ 的乘积 $\sqrt{\ell_i}\boldsymbol{a}_i$ 的各个分量为第 i 个成分与各个变量之间的相关系数(向量). 由于一个特征向量和任何实数的乘积仍然是相应特征值的特征向量, 单位特征向量 $\boldsymbol{a}_i$ (矩阵 $\boldsymbol{A}$ 的第 i 列) 和向量 $\sqrt{\ell_i}\boldsymbol{a}_i$ 均为同方向的特征向量. 在一些软件的输出中 (比如 SPSS) 称 $\sqrt{\ell_i}\boldsymbol{a}_i$ 为**载荷** (loading), 但在另一些软件 (比如 R 函数 `princomp()` 及 SAS 相应模块) 的输出中, 称单位特征向量 $\boldsymbol{a}_i$ 为载荷. 这仅仅是名称不同, 只要明白其是如何定义的就行了.

计算贡献率 $\ell_i/\sum_{j=1}^{p}\ell_j$ $(i=1,2,\ldots,p)$ 和累积贡献率 $\sum_{j=1}^{k}\ell_i/\sum_{j=1}^{p}\ell_j$ $(k=1,2,\ldots,p)$ 的 R 代码为:

```
(cca=(a$va)/sum(a$va))#贡献率
(ca=cumsum(a$va)/sum(a$va))#累积贡献率
```

相应的输出为:

```
> (cca=(a$va)/sum(a$va))#贡献率
 [1] 7.15791e-01 1.87685e-01 5.43148e-02 1.38612e-02
 [5] 1.05137e-02 5.86501e-03 5.13270e-03 4.01001e-03
 [9] 2.07852e-03 3.83096e-04 1.96955e-04 1.46174e-04
[13] 2.10239e-05
> (ca=cumsum(a$va)/sum(a$va))#累积贡献率
 [1] 0.715791 0.903477 0.957792 0.971653 0.982167
 [6] 0.988032 0.993164 0.997174 0.999253 0.999636
[11] 0.999833 0.999979 1.000000
```

输出表明, 贡献率最大的前两个主成分的贡献率之和 (即累积贡献率) 已经超过 90% 了, 这直观地显示在下面的崖底碎石图中.

所谓**崖底碎石图** (scree plot) 是特征值 ℓ_i 对成分编号 $1,2,\ldots,p$ 的点图, 它描述了各个成分贡献的变化. 该图和贡献率图 ($\ell_i/\sum_i\ell_i$ 对成分编号 $1,2,\ldots,p$ 的点图) 形状相同, 因此用特征值和贡献率都可以得到形状相同 (差一个因子) 的崖底碎石图. 产生崖底碎石图及累积贡献率图 (见图5.4) 的 R 代码如下:

```
#崖底碎石图及累积贡献率图
par(mfrow=c(1,2))
plot(1:(ncol(u)-1),a$va,type="o",pch=17,col=4,main="Scree Plot",
    xlab="Component Number",ylab="Eigen Value")
plot(1:(ncol(u)-1),ca,type="o",pch=17,col=4,
    main="Cumulative Contribution",
    xlab="Component Number",ylab="Cumulative Contribution")
```

这里之所以称特征值的特征值或贡献率图为崖底碎石图, 是因为适合主成分分析的数

据的少数几个大特征值的贡献占了很大的比例, 这样, 图形下降得很陡, 就像悬崖一样, 剩下的特征值都很小, 如碎石躺在陡峭的悬崖下面. 如果数据不那么相关, 各个特征值大小差不多, 就不会有这样的陡峭图形.

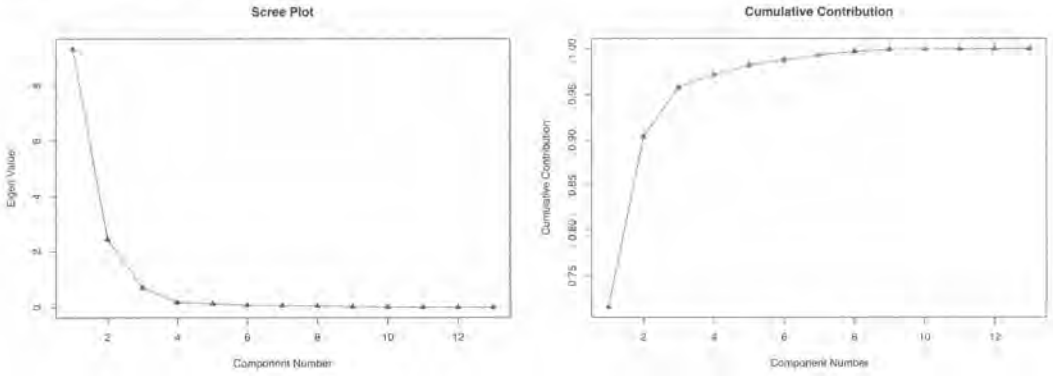

图 5.4 例5.1教师数据的贡献率图 (崖底碎石图)(左) 及累积贡献率图 (右)

不相关变量的说明性例子. 假定一个数据 (设 $p = 5$) 各个变量完全不相关, 那么其相关阵为单位矩阵 (R 代码为 `diag(5)`). 其特征值全部为 1, 而没有 "悬崖" 的崖底碎石图则成为 "平地图". 下面是这个说明性例子 ($p = 5$) 的代码、输出以及 "平面的" 悬崖碎石图 (见图5.5).

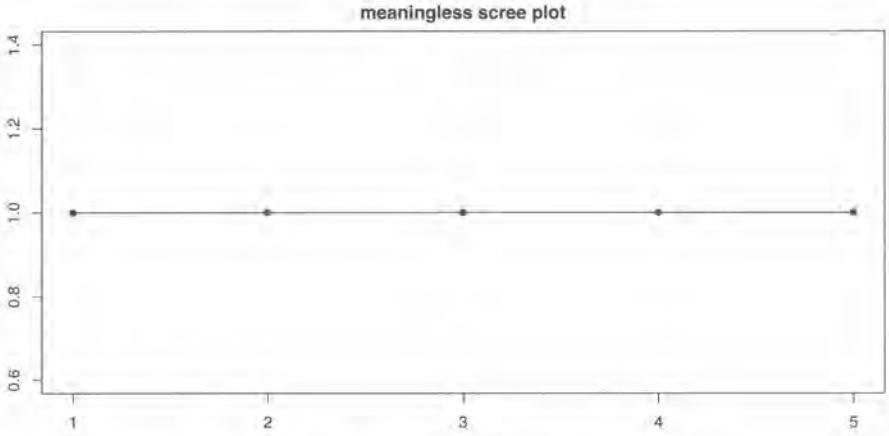

图 5.5 没有意义的主成分分析的崖底碎石图

```
> (b=eigen(diag(5)))
$values
 [1] 1 1 1 1 1

$vectors
     [,1] [,2] [,3] [,4] [,5]
[1,]    0    0    0    0    1
[2,]    0    0    0    1    0
[3,]    0    0    1    0    0
```

```
[4,]    0    1    0    0    0
[5,]    1    0    0    0    0

> plot(1:5,b$values,type="o",pch=16,main ="meaningless scree plot")
```

5.2.3 根据贡献率选取少数重要成分

下表为例5.1教师数据的主成分分析的特征值、贡献率、累积贡献率表. 而前面图5.4的左右两图则是根据该表中的数据绘出的, 可以看出, 第一主成分的贡献率较大, 占 72%, 而第二个占 19%, 它们合计贡献率为 90%, 而其他主成分的贡献率都不大. 因此, 选取头两个主成分就可以解释 90%的方差.

例5.1教师数据主成分分析的特征值、贡献率、累积贡献率

成分	1	2	3	4	5	6	7	8	9	10	11	12	13
特征值	9.31	2.44	0.71	0.18	0.14	0.08	0.07	0.05	0.03	0.00	0.00	0.00	0.00
贡献率	0.72	0.19	0.05	0.01	0.01	0.01	0.01	0.00	0.00	0.00	0.00	0.00	0.00
累积贡献率	0.72	0.90	0.96	0.97	0.98	0.99	0.99	1.00	1.00	1.00	1.00	1.00	1.00

注意

对于在主成分分析中应该选取多少成分, 说法不一. 有的说被选取的成分累积贡献率应达到 70%以上, 有的说必须达到 75%、80%或 85%以上. 实际上, 单纯按照百分数来选择主成分个数是不合适的. 如果各个变量不相关, 那么所有变量贡献率相同, 其崖底碎石图基本上是一条直线, 如图5.5所示, 这时, 按照累积贡献率来选择成分就意味着选百分之几的变量, 这显然是没有意义的. 对于这种数据, 人们不应该也没有必要做主成分分析. 只有在变量相关的情况下, 崖底碎石图才会很陡, 这时, 很小比例的变量会有很大比例的贡献率.

笔者曾在一次研讨会上见到有人极其荒唐地对由主成分分析得到的各个不相关成分做第二次主成分分析, 令人哭笑不得.

5.2.4 主成分和原始变量的相关系数计算与载荷图

前面提及第 i 个主成分和第 j 个原始变量的相关系数 (载荷) 为 $\sqrt{\ell_i}a_{ij}$, 这来自每个单位特征向量 (a$vec) 乘以相应的特征值的平方根 (sqrt(a$value)) 的结果. 下面的代码计算 (特征值都大于 1 的) 前两个主成分和原始变量的相关系数 (载荷), 其中第一行就是计算相关系数的语句, 而后面几行语句得出前两个主成分与原始变量的相关系数表 (见下表), 并绘出载荷图 (见图5.6).

```
(b=sweep(a$vec[,a$va>0],2,sqrt(a$value[a$va>0]),"*"))
pct=paste("(",round(a$va/sum(a$va)*100),"%)",sep="")
tt=NULL;for(i in 1:4)tt=c(tt,paste("Component", i,pct[i]))
plot(b[,1:2], type="n", xlab=tt[1], ylab=tt[2],main="Loadings",
    xlim=c(-1,0.05))
text(b[,1],b[,2],names(u[,-1]));abline(h=0);abline(v=0)
```

例5.1教师数据的两个个主成分与原始变量的相关系数

原始变量	ASF	ASA1	ASA2	ASALL	ACF	ACA1	ACA2	ACALL	NF	NA1	NA2	NIN	NALL
成分 1	-0.954	-0.930	-0.932	-0.951	-0.949	-0.922	-0.927	-0.947	-0.742	-0.735	-0.718	-0.271	-0.755
成分 2	-0.211	-0.295	-0.244	-0.246	-0.247	-0.324	-0.263	-0.274	0.558	0.615	0.644	0.609	0.635

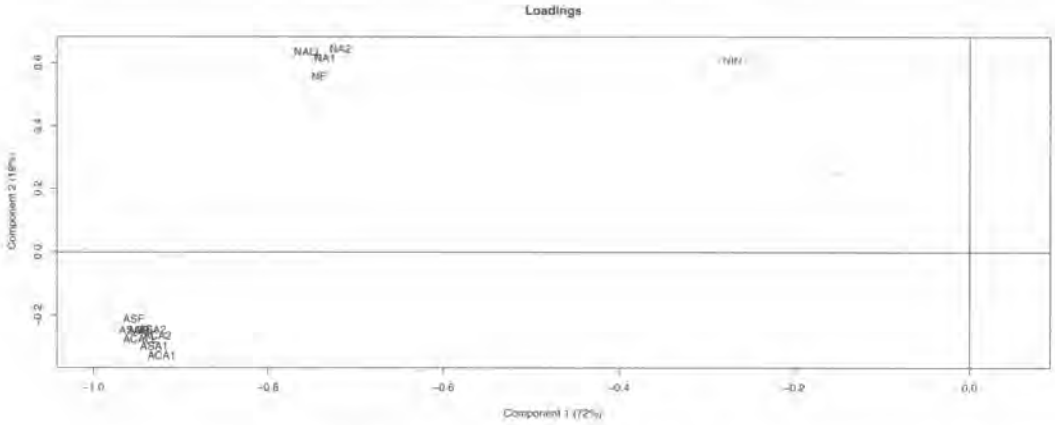

图 5.6 例5.1教师数据的主成分分析载荷图

载荷图 (见图5.6) 的每一个点代表例5.1的一个原始变量, 并以其名字标识 (比如变量 ASF 或 NIN). **载荷图每一点的横坐标为相应变量与第一主成分的线性相关系数, 而其纵坐标为该变量与第二主成分的线性相关系数**. 图5.6各个点的坐标对应例5.1的方差最大的 2 个主成分与原始变量的相关系数, 即上表的两行数目相应于图5.6中点的横、纵坐标. 可以看出, 第一主成分绝对值大 (这里是负数) 的点既对应和收入有关的变量 (字母 "A" 开头的变量), 也对应和人数有关的变量 (字母 "N" 开头的变量), 但变量 NIN 是例外; 而第二主成分绝对值大 (这里是正数) 的点主要对应和人数有关的变量 (字母 "N" 开头的变量).

每一个选中的主成分所代表的特征可以用来给这些成分起名字, 但并不一定都合理. 给例5.1的前两个主成分起名字就不那么容易. 第一主成分可以勉强称为 "总体成分"(代表人数和收入二者), 而第二主成分可称为 "人力资源成分"(代表人数).

由于任何单位特征值乘以 -1 之后仍然是单位特征值, 载荷在乘以 -1 之后仍然是载荷. 因此, 上面的载荷图相对于任何坐标轴的镜面映射仍然是载荷图. 另外, 有些软件所产生的载荷图仅仅基于一对 (未乘以相应特征值的平方根的) 单位特征向量. 载荷图的种种变化并不影响对结果的解读. 因为对于一个成分而言, 我们仅仅关注各个变量与该成分相关程度的相对大小, 比如, 在例5.1的载荷图 (见图5.6) 中, 变量 ASF 比 NIN 与第一主成分更加相关 (无论正负), 但变量 NIN 比 ASF 与第二主成分更加相关, 而且由于符号不同 (无所谓哪个为正, 哪个为负), 它们与第二主成分的相关方向相反 (如果一个为正相关, 则另一个为负相关, 而且程度不同).

5.2.5 主成分得分

对于每个观测值都可以得到它们关于每个主成分的得分, 也就是观测值在各个成分上的投影. 数据 $\boldsymbol{X}$ 在主成分上的投影记为 $\boldsymbol{X}' \equiv \boldsymbol{X}\boldsymbol{A}$(注意: $\boldsymbol{X}'$ 不是矩阵 $\boldsymbol{X}$ 的转置, 本书把 $\boldsymbol{X}^{\top}$ 记为矩阵 $\boldsymbol{X}$ 的转置), 这里的 $\boldsymbol{A}$ 是前面引入的由主成分列向量组成的矩阵, 其元素

a_{ij} 是第 i 个成分相应于第 j 个变量的值, 而 $\boldsymbol{X}=\{x_{ij}\}$ 是原始变量的标准化形式, 其元素 x_{ij} 是第 i 个观测值对应于第 j 个变量的 (标准化后的) 值. 这里所说的标准化, 是指每个变量的观测值减去该变量所有观测值的均值再除以该变量所有观测值的标准差之后的结果. 假定原始数据 $n\times p$ 矩阵为:

$$\boldsymbol{X}^{(0)}=\begin{bmatrix} x_{11}^{(0)} & x_{12}^{(0)} & \cdots & x_{1p}^{(0)} \\ x_{21}^{(0)} & x_{22}^{(0)} & \cdots & x_{2p}^{(0)} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1}^{(0)} & x_{n2}^{(0)} & \cdots & x_{np}^{(0)} \end{bmatrix},$$

则标准化后的数据矩阵为:

$$\boldsymbol{X}=\begin{bmatrix} (x_{11}^{(0)}-\bar{x}_1^{(0)})/s_1^{(0)} & (x_{12}^{(0)}-\bar{x}_2^{(0)})/s_2^{(0)} & \cdots & (x_{1p}^{(0)}-\bar{x}_p^{(0)})/s_p^{(0)} \\ (x_{21}^{(0)}-\bar{x}_1^{(0)})/s_1^{(0)} & (x_{22}^{(0)}-\bar{x}_2^{(0)})/s_2^{(0)} & \cdots & (x_{2p}^{(0)}-\bar{x}_p^{(0)})/s_p^{(0)} \\ \vdots & \vdots & \ddots & \vdots \\ (x_{n1}^{(0)}-\bar{x}_1^{(0)})/s_1^{(0)} & (x_{n2}^{(0)}-\bar{x}_2^{(0)})/s_2^{(0)} & \cdots & (x_{np}^{(0)}-\bar{x}_p^{(0)})/s_p^{(0)} \end{bmatrix},$$

这里的 $\bar{x}_i^{(0)}$ 和 $s_i^{(0)}$ 分别为 $x_{1j}^{(0)},x_{2j}^{(0)},\ldots,x_{nj}^{(0)}$ $(j=1,2,\ldots,p)$ 的均值和标准差.

对于例5.1教师数据, 我们选中两个主成分, 第 i 个观测值 $(i=1,2,\ldots,1161)$ 对于这两个主成分的得分为:

$$x'_{ik}=\sum_{j=1}^{13}x_{ij}a_{kj},\quad i=1,2,\ldots,1161;\ k=1,2,$$

于是, 每个观测值在以这两个主成分为坐标轴的平面上代表一个点.

下面的 R 语句计算了前两个主成分的得分, 即用标准化的原始数据矩阵乘特征向量矩阵, 并把这些得分展示在图5.7(左) 中.

```
par(mfrow=c(1,2))
sc=scale(u[,-1])%*%a$ve[,1:2] #计算得分
plot(sc[,1],sc[,2],type="n",ylim=c(-6,10),xlim=c(-11,7),
    main="Sample Principal Components", xlab="Component 1",
    ylab="Component 2")
text(sc[,1],sc[,2],u[,1],cex=.4)
abline(v=0,col=2);abline(h=0,col=2)
CA=(w$State=="CA")
plot(sc[CA,1],sc[CA,2],type="n",ylim=c(-6,4),xlim=c(-12,5),
    main="Sample Principal Components for California Schools",
    xlab="Component 1", ylab="Component 2")
text(sc[CA,1],sc[CA,2],u[CA,1],cex=1)
abline(v=0,col=2);abline(h=0,col=2)
par(mfrow=c(1,1))
```

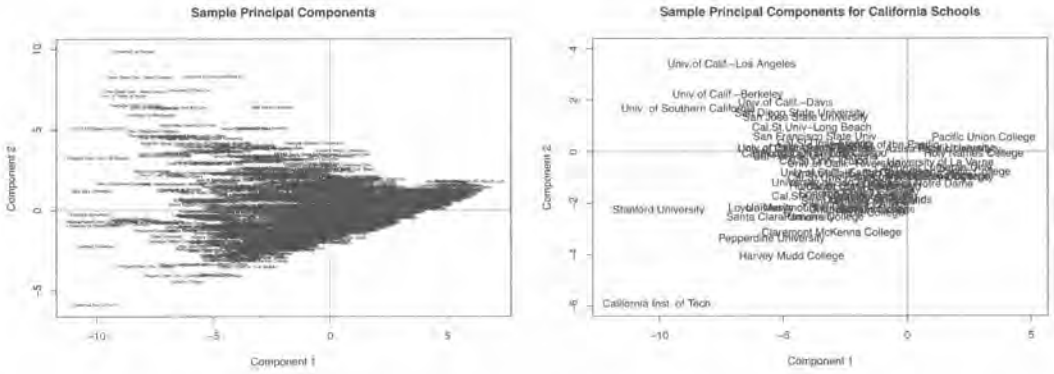

图 5.7 例5.1教师数据的样本主成分得分，左图是全部观测值，右图仅为加利福尼亚州高校观测值

图5.7中各个点是用学校名字代表的，虽然有些拥挤 (特别是左图)，但还是可以看出一些特别突出的学校名字. 图5.7中右图为加利福尼亚州学校的得分图，没有左图那么拥挤. 从右图的横坐标 (相应于第一主成分) 可以看出加利福尼亚州哪些学校的平均收入和教师人数综合得分较高 (负值越大)，比如加利福尼亚理工学院、斯坦福大学、南加利福尼亚大学、加利福尼亚大学伯克利分校和洛杉矶分校等最高. 从右图的纵坐标 (相应于第二主成分) 可以看出加利福尼亚州哪些学校的教师人数得分较高，这里加利福尼亚大学洛杉矶分校、加利福尼亚大学伯克利分校、加利福尼亚大学戴维斯分校、南加利福尼亚大学、圣迭戈州立大学等得分最高，但由于第二主成分和变量的相关性远不如第一主成分强，因此对数据总体的解释性也弱.

得到上面两个成分的前 5 名学校名称的代码为 (读者可以自行运行来查看输出):

```
u[CA,1][order(sc[CA,1],decreasing =F)][1:5]
u[CA,1][order(sc[CA,1],decreasing =T)][1:5]
```

注意

综合评价的悖论. 有些人把主成分分析和因子分析中的成分或因子作为产生 "综合指标" 的所谓 "综合评价" 的基础，比如，某个成分为 "教育成分"，某个成分为 "经济发展成分"，等等，然后用这些成分的平均或者它们 (按照主成分贡献比例) 的加权平均作为 "政绩" 的综合评价标准，并用来排序. 但是，如果分析时不仅仅局限于经济变量，还包括廉政、环境、公平、效率等其他变量，结果就不会那么简单了. 这时，"经济发展成分" 和腐败及污染很可能十分相关，这时如何称呼、如何进行综合评价都会成为问题. 比如，公平和效率也是负相关的，反映效率的成分难道可以称为 "不公平成分" 吗?

变量的不同选择决定 "综合指标" 的不同结果. 关于按照主成分贡献比例对各个成分做加权平均来产生 "综合指标" 的做法完全依赖于变量的选择，不同的变量选择得到不同的主成分和不同的 "权重". 根据某种目的去人为地选择变量是不可取的. 进行主成分分析或因子分析最重要的是客观地、毫无偏见地选择变量.

排名的争议. 根据各种变量形成 "综合指标" 不可能有唯一的方案，以学校排名为例，人们可以从很多指标中挑选部分来做，比如电脑网络设备、教师学生比例、论文引用总数、得奖者人数、项目数量、资金数量、人均论文引用数量、博士毕业生的学术成就等等. 采用不同的变量，结果也大不一

样. 举例来说, 如果强调论文引用总数, 教师少的学校 (比如普林斯顿大学) 就吃亏了, 但如果强调人均论文引用数量, 诸如普林斯顿大学那样的学校就会名列前茅. 实际上, 没有任何一个机构对学校的排名能够令人满意.

简单化会损失信息. 用少数指标对一个事物做描述肯定要损失信息. 比如, 人的体能本来可以用持久性、爆发力、速度等多方面来描述, 并且给人以非常明确的印象. 但是, 如果一定要用一个 "综合指标" 来对不同人的体能做描述, 则会把本来明确的印象弄模糊. 诸如 "哪个学校第一""哪个公司最好""哪个教授最优秀" 之类的问题都是试图把复杂事物简单化的幼稚做法. 我们都明白, 以 "非黑即白" 的观点看问题已成为历史.

5.3 主成分分析在图像处理中的应用

主成分分析在图像处理中有很多应用, 最典型的是图像压缩和人脸、植物、文字、飞行器、船只等各种图像的识别, 而每一种应用都有不同的方法. 为节省篇幅, 这里仅仅做些简单的介绍, 不做更多展开.

5.3.1 图像压缩案例 (例5.2江景图片)

例5.2的照片 (见图5.2) 像素较高, 我们试着对其通过主成分分析实现降维, 看看效果如何. 首先用下面使用了程序包 jpeg[4]的函数代码读入数据:

```
library(jpeg)
xmy=readJPEG('XMY.jpg')
```

然后用下面代码对各个颜色做 3 次主成分分析 (分别选取的主成分个数为 3、10、100), 并且把图像输出到 3 张图中 (文件名为 compXMY3.jpg, compXMY10.jpg, compXMY100.jpg), 这 3 张图 (以及原始图片) 显示在图5.8中.

```
xmypca=list()
for(i in 1:3)xmypca[[i]]=prcomp(xmy[,,i], center = FALSE)
imge=list()
for (i in c(3,10,100)) {
  imge[[i]] <- sapply(xmypca, function(y) {
  comimge <- y$x[,1:i] %*% t(y$rotation[,1:i])
  }, simplify = 'array')
  writeJPEG(imge[[i]], paste('compXMY', round(i,0), '.jpg', sep = ''))
}
```

上面代码使用了主成分分析的函数 prcomp(), 其输入值为原始数据矩阵, 输出中的 $x 为原始数据的旋转 (原数据乘以输出中的 $rotation), 而 $rotation 为特征向量 (注意, prcomp() 和另一个常用的主成分分析函数 princomp() 的输出不同). 可以看出, 主成分个数为 3、10 的图不那么清楚, 而最后一个主成分个数为 100 的图已经相当清楚. 前 3 个

[4]Simon Urbanek (2014). jpeg: Read and write JPEG images. R package version 0.1-8. https://CRAN.R-project.org/package=jpeg.

图文件相对于原图文件的大小比例分别为 0.12、0.15、0.22. 即使是最大一个压缩的图文件的大小也只有原图文件的 22%. 这些比例的计算代码及输出如下:

```
> FS=file.info('XMY.jpg')$size
> z=NULL
> for(i in c(3,10,100))
> z=c(z,file.info(paste("compXMY",i,".jpg",sep=""))$size/FS)
> z
0.1165802 0.1482081 0.2239737
```

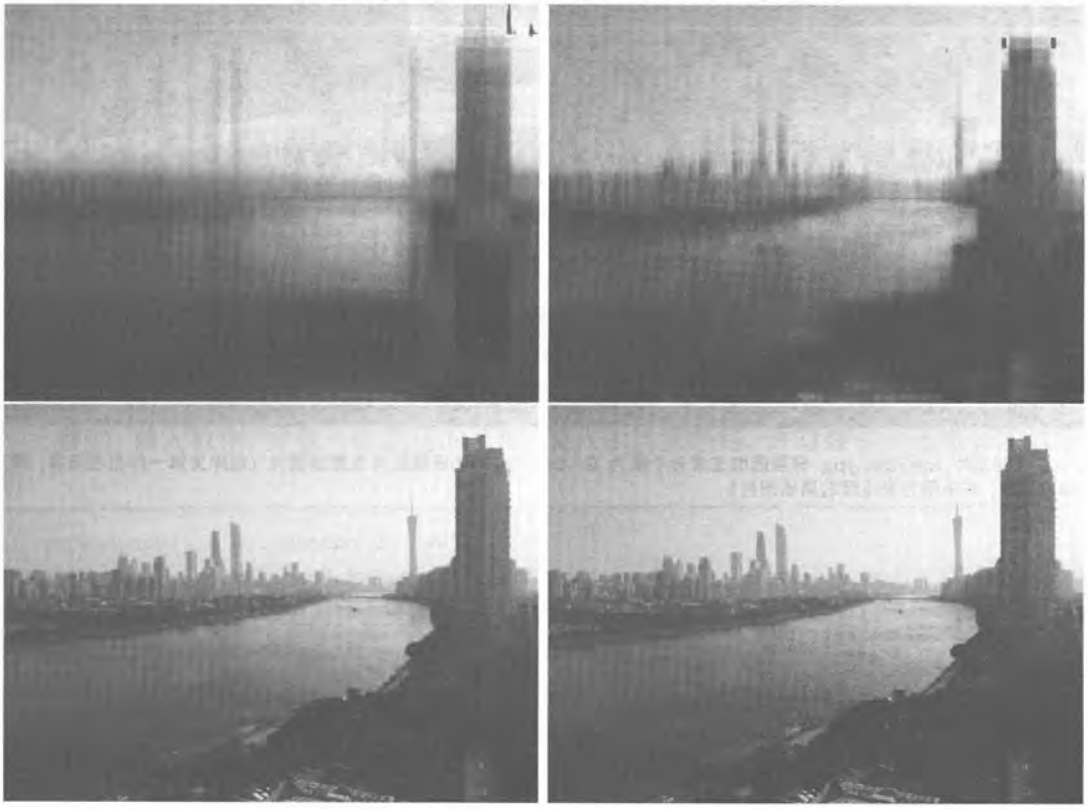

图 5.8 例5.2江景照片分别选取的主成分个数为 3、10、100 的压缩照片以及原始图片 (自左上到右下逐行排列)

前面代码所做的实际上是对形成图片的 (红、绿、蓝) 三个颜色 (注意: 本书为黑白印刷, 彩色无法显示) 矩阵分布实行奇异值分解 $\boldsymbol{X}_{,,k} = \boldsymbol{U}_k\boldsymbol{D}_k\boldsymbol{V}_k^\top$ $(k = 1, 2, 3)$, 而选取 i 个主成分压缩的图片三色 $(k = 1, 2, 3)$ 的矩阵为 (这里的 $\boldsymbol{U}[, 1 : i]_k$ 表示 $\boldsymbol{U}_k$ 的前 i 列, $\boldsymbol{V}[, 1 : i]_k$ 类似):

$$\boldsymbol{Y}_{,,k} = \boldsymbol{U}[, 1 : i]_k\boldsymbol{D}_k\boldsymbol{V}[, 1 : i]_k^\top, \quad k = 1, 2, 3.$$

例 5.4. 市场图片 (market.jpg) 这个例子也是一张照片 (见图5.9中右下图).

对于例5.4, 直接计算该奇异值分解并压缩图片 (文件: market.jpg) 的代码如下 (取 5、20、100 个主成分之后的压缩图片及原始图片显示在图5.9中):

图 5.9 对图片 market.jpg 分别选取主成分个数为 5、20、100 的压缩照片及原始图片 (顺序为第一行自左至右，第二行自左至右，右下图为未压缩的原始图片)

```
mk=readJPEG('market.jpg')
mksvd=list()
for(i in 1:3)mksvd[[i]]=svd(mk[,,i])
commk=mk;cmk=list()
FS=file.info('market.jpg')$size
for (i in c(5, 20, 100)){
  for(k in 1:3){
    commk[,,k]=mksvd[[k]]$u[,1:i] %*% diag(mksvd[[k]]$d[1:i]) %*%
      t(mksvd[[k]]$v[,1:i])
    writeJPEG(commk, paste('commk', round(i,0), '.jpg', sep = ''))
  }
  cmk[[i]]=commk
}
z=NULL;for(i in c(5,20, 100))
z=c(z,file.info(paste("commk",i,".jpg",sep=""))$size/FS);z
```

计算出来的 3 个压缩后的图文件相对于原图文件的大小比例分别为 0.18、0.26、0.38.

5.3.2 主成分分析运用于图像识别

假定我们想要知道一个人脸是不是属于已知一群人中的一个, 我们需要把该人的照片和那一群人的照片 (假定一共有 N 张照片) 的数据放到一个矩阵中. 当然, 这要求每张照片都有同样的大小 (比如 $m \times n = M$ 个像素), 所有的头像都在照片中间, 有类似的角度, 等等. 把每张照片的数据矩阵 "拉长" 成 (长度为 M) 行向量, 所有 N 个行向量形成一个 $N \times M$ 矩阵 $\boldsymbol{X}$. 然后, 对这个矩阵通过主成分分析降维, 于是可以在主成分得分中看所关注的照片是否和其他照片中的某个接近. 由于 $M = m \times n$ 很大, 解维数为 $M \times M$ 的样本协方差阵 $\boldsymbol{X}^\top \boldsymbol{X}$ 的特征值问题有些不方便, 但我们可以解维数为 $N \times N$ 的样本协方差阵 $\boldsymbol{X}\boldsymbol{X}^\top$ 的特征值问题. 如果 $\boldsymbol{X}\boldsymbol{X}^\top$ 的特征向量为 $\boldsymbol{u}$, 那么 $\boldsymbol{X}^\top \boldsymbol{X}$ 的特征向量为 $\boldsymbol{X}^\top \boldsymbol{u}$, 由于它们的特征值相同 (假定为 λ), 则有:

$$\boldsymbol{X}\boldsymbol{X}^\top \boldsymbol{u} = \lambda \boldsymbol{u} \Leftrightarrow \boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{X}^\top \boldsymbol{u} = X^\top \lambda \boldsymbol{u} \Leftrightarrow \boldsymbol{X}^\top \boldsymbol{X}(\boldsymbol{X}^\top \boldsymbol{u}) = \lambda(\boldsymbol{X}^\top \boldsymbol{u}).$$

对于主成分分析在图像识别中的应用, 在这里就不举例子了.

5.4 案例: 例5.3数据的主成分分析

前面的例子演示夹杂着公式和解释, 这里我们一气呵成地做例5.3洛杉矶街区数据的常规主成分分析. 为了方便, 我们不用公式, 直接使用程序包 `factoextra`[5]中的可视化函数及程序包 `FactoMineR`[6] 中的基本运算函数.

首先, 读入数据, 并做一些适当的调整, 装入上述程序包, 并直接引用函数做主成分分析:

```
w=read.csv("LA.Neighborhoods.csv")
w$density=w$Population/w$Area #增加密度
u=w[,-c(12:15)]# 去掉人口, 面积, 经纬度
library("FactoMineR");library("factoextra")
row.names(u)=u[,1]
u.r=PCA(u[,-1], scale.unit=TRUE,graph=FALSE)#主程序
```

可以用代码 `print(u.r)` 列出上面函数 PCA 所得到的各种结果, 请读者自己实现. 下面代码打印出特征值, 包括各个特征值所占总方差的比例及累积比例, 并产生崖底碎石图 (见图5.10).

```
> fviz_eig(u.r, addlabels = TRUE, ylim = c(0, 50))
> (u.eigen <- get_eigenvalue(u.r))#值
       eigenvalue variance.percent cumulative.variance.percent
Dim.1  4.6930331513      42.66393774                    42.66394
Dim.2  1.8176421789      16.52401981                    59.18796
```

[5]Alboukadel Kassambara and Fabian Mundt (2017). factoextra: Extract and Visualize the Results of Multivariate Data Analyses. R package version 1.0.5. https://CRAN.R-project.org/package=factoextra.

[6]Sebastien Le, Julie Josse, Francois Husson (2008). FactoMineR: An R Package for Multivariate Analysis. *Journal of Statistical Software*, 25(1), 1-18. 10.18637/jss.v025.i01.

```
Dim.3  1.1687855390   10.62532308   69.81328
Dim.4  1.0269743864    9.33613079   79.14941
Dim.5  0.8062945744    7.32995068   86.47936
Dim.6  0.5577210494    5.07019136   91.54955
Dim.7  0.4099288177    3.72662562   95.27618
Dim.8  0.2437073056    2.21552096   97.49170
Dim.9  0.1870581845    1.70052895   99.19223
Dim.10 0.0882870577    0.80260962   99.99484
Dim.11 0.0005677551    0.00516141  100.00000
```

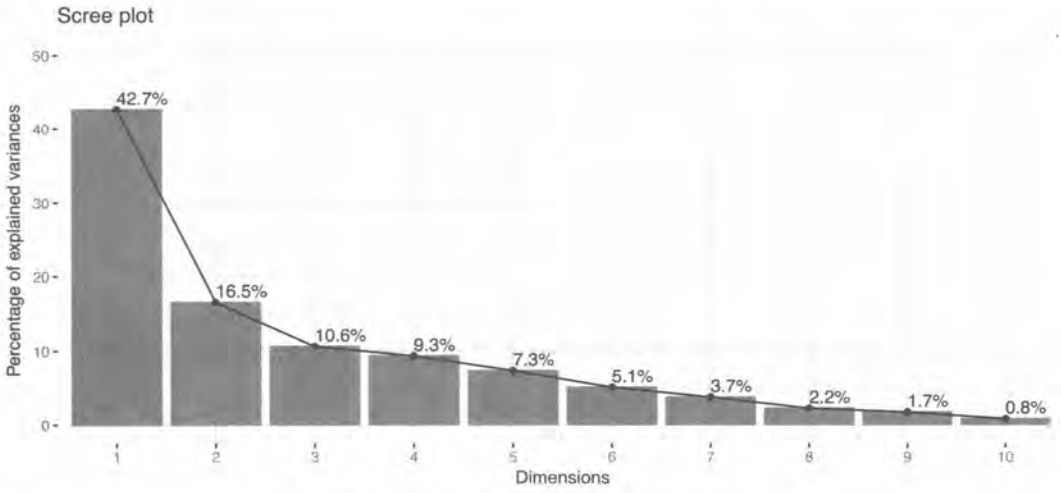

图 5.10 例5.3主成分分析的崖底碎石图

各个变量对前两个主成分的贡献 (百分比) 图 (见图5.11) 可以用下面代码产生:

```
fviz_contrib(u.r, choice="var", axes=1)
fviz_contrib(u.r, choice="var", axes=2)
```

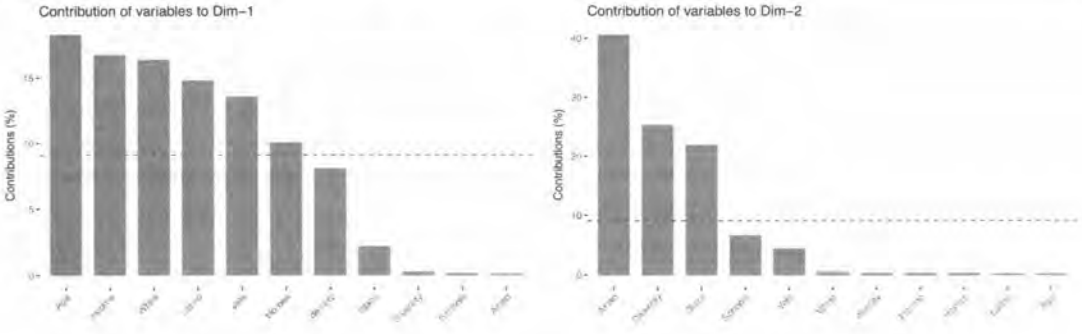

图 5.11 例5.3主成分分析各个变量对第一 (左) 及第二 (右) 主成分的贡献 (百分比) 图

描述变量和主成分相关系数的载荷图 (见图5.12) 是由下面代码产生的, 图5.12中左图是单纯的载荷图, 右图用不同颜色 (注意: 本书为黑白印刷, 彩色无法显示) 展示了各个变量的平方坐标, 称为 cos2. 载荷图中的半径为 1 的圆称为相关圆, 所有的点都不会超过它.

```
fviz_pca_var(u.r)
fviz_pca_var(u.r, col.var = "cos2",
  gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"), repel = TRUE)
```

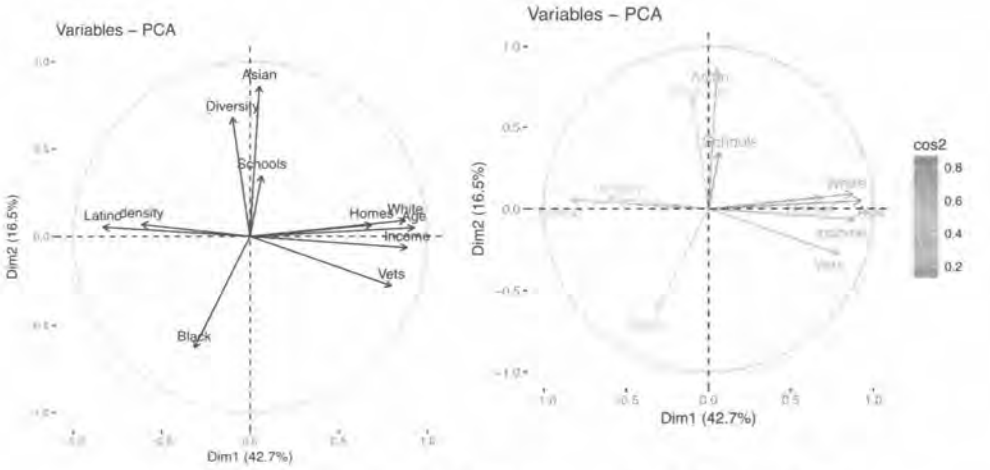

图 5.12 例5.3主成分分析的载荷图 (左) 及加上 cos2 信息后的载荷图 (右)

图5.12的坐标和 cos2 的值的名字可以用代码 (var=get_pca_var(u.r)) 输出, 比如, 其中有 "$cos2", 那么用 var$cos2 即可输出相应的数值 (这里不一一演示).

可以用下面代码得到观测值在前两个主成分的记分点图 (见图5.13), 点的大小表示其 cos2 的值. 由于各个街区数目多, 名字也很长, 显得拥挤.

```
fviz_pca_ind(u.r,pointsize="cos2",pointshape=21,fill="#E7B800",repel=F)
```

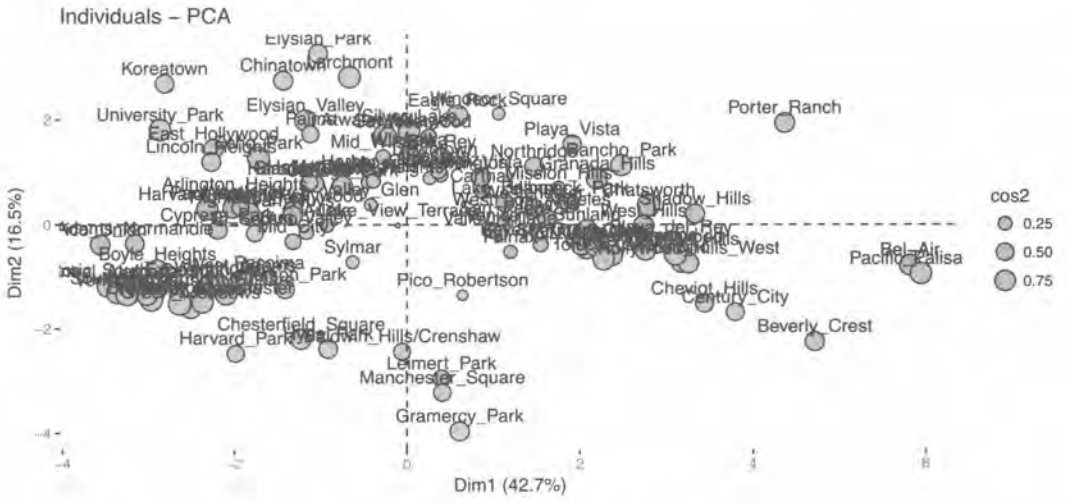

图 5.13 例5.3主成分分析的记分点图

5.5 主成分分析的一些数学知识 *

本节主要供读者参考之用.

5.5.1 目标

令 $\boldsymbol{X}=(X_1,X_2,\ldots,X_p)^\top$ 为 p 维随机向量. 我们的目标是在 p 维空间 $\mathbb{R}^p$ 中找到使得线性组合 $\boldsymbol{\gamma}_1^\top\boldsymbol{X}$ 具有最大方差的 p 维向量 $\boldsymbol{\gamma}_1$, 然后寻找和 $\boldsymbol{\gamma}_1$ 不相关的使得线性组合 $\boldsymbol{\gamma}_2^\top\boldsymbol{X}$ 具有最大方差的 $\boldsymbol{\gamma}_2$. 如此下去, 可重复 p 次, 找到 p 个不相关的 $\boldsymbol{\gamma}_k$ $(k=1,2,\ldots,p)$. 这些由向量 $\{\boldsymbol{\gamma}_k\}$ 代表的新随机变量 $\{Y_k\}=\{\boldsymbol{\gamma}_k^\top\boldsymbol{X}\}$ 称为主成分. 为了达到降维的目的, 我们希望找到 m $(m\ll p)$ 个主成分, 使得它们能够解释大部分方差.

假定 $\boldsymbol{\Sigma}$ 是变量 $\boldsymbol{X}$ 的协方差矩阵. 我们已经知道的结论是: 第 k 个主成分为 $\boldsymbol{\gamma}_k^\top\boldsymbol{X}$, 其中 $\boldsymbol{\gamma}_k$ 为 $\boldsymbol{\Sigma}$ 的相应于第 k 大的特征值 λ_k 的一个特征向量. 如果 $\boldsymbol{\gamma}_k$ 为单位向量 $(\boldsymbol{\gamma}_k^\top\boldsymbol{\gamma}_k=1)$, 则 $\mathrm{Var}(Y_k)=\mathrm{Var}(\boldsymbol{\gamma}_k^\top\boldsymbol{X})=\lambda_k$. 在 $\boldsymbol{\gamma}_k^\top\boldsymbol{\gamma}_k=1$ 时, 线性组合 $\boldsymbol{\gamma}_k^\top\boldsymbol{X}$ 称为标准化线性组合. 实际应用中, $\boldsymbol{\Sigma}$ 往往未知, 可以用样本协方差矩阵代替, 但更通常的做法是用样本相关矩阵 $\boldsymbol{R}$ 代替 (如本章的做法), 这等价于 (为了去除变量量纲的影响) 将数据标准化之后的样本协方差矩阵.

5.5.2 过程

从随机变量角度的推导

令 $\boldsymbol{\gamma}\in\mathbb{R}^p$, 我们在 $\boldsymbol{\gamma}^\top\boldsymbol{\gamma}=1$ 的约束下求使得 $\mathrm{Var}(\boldsymbol{\gamma}^\top\boldsymbol{X})=\boldsymbol{\gamma}^\top\boldsymbol{\Sigma}\boldsymbol{\gamma}$ 最大的单位向量 $\boldsymbol{\gamma}$. 根据 Lagrange 乘数法, 要求:

$$\arg\max_{\boldsymbol{\gamma},\lambda}\{\boldsymbol{\gamma}^\top\boldsymbol{\Sigma}\boldsymbol{\gamma}-\lambda(\boldsymbol{\gamma}^\top\boldsymbol{\gamma}-1)\},$$

关于 $\boldsymbol{\gamma}$ 求微分, 得到方程:

$$\begin{aligned}\frac{d}{d\boldsymbol{\gamma}}\{\boldsymbol{\gamma}^\top\boldsymbol{\Sigma}\boldsymbol{\gamma}-\lambda(\boldsymbol{\gamma}^\top\boldsymbol{\gamma}-1)\}=0\ &\Rightarrow\\ \boldsymbol{\Sigma}\boldsymbol{\gamma}-\lambda\boldsymbol{\gamma}=0\ &\Rightarrow\\ \boldsymbol{\Sigma}\boldsymbol{\gamma}=\lambda\boldsymbol{\gamma}.&\end{aligned}$$

上边最后一个式子表明, $\boldsymbol{\gamma}$ 是 $\boldsymbol{\Sigma}$ 的一个特征向量, 而 λ 是相应的特征值. 由于 $\boldsymbol{\Sigma}$ 是 $p\times p$ 的对称矩阵, 根据线性代数, 它有 p 个特征值, 而且其特征向量都是互相正交的. 由于 $\boldsymbol{\Sigma}$ 是一个非负定矩阵, 其特征值都是非负的.

令 $\boldsymbol{\Sigma}$ 的特征值按大小排列为 $\lambda_1\geqslant\lambda_2\geqslant\cdots\geqslant\lambda_p$, 则相应于各个特征值的代表各个主成分的特征向量为 $\boldsymbol{\gamma}_1,\boldsymbol{\gamma}_2,\ldots,\boldsymbol{\gamma}_p$, 它们满足 $\boldsymbol{\Sigma}\boldsymbol{\gamma}_k=\lambda_k\boldsymbol{\gamma}_k$ 及 $\mathrm{Var}(\boldsymbol{\gamma}_k^\top\boldsymbol{X})=\lambda_k$ $(k=1,2,\ldots,p)$.

数据矩阵角度的推导

目的: 对于 $n\times p$ 维数据矩阵 $\boldsymbol{X}$(假定已经中心化), 寻找一个线性变换 (通过 $p\times p$ 变

换矩阵 $\boldsymbol{A}$) $\boldsymbol{Y}=\boldsymbol{X}\boldsymbol{A}$, 这里的 $\boldsymbol{Y}$ 为 $n\times p$ 矩阵, 而且满足:

1. $\boldsymbol{A}$ 的列不相关, 即 $\boldsymbol{a}_i^{\top}\boldsymbol{a}_j=0\ (\forall i\neq j)$;
2. $\boldsymbol{Y}$ 的样本协方差矩阵 $\boldsymbol{S}_{\boldsymbol{Y}}$ 为对角型

$$\boldsymbol{S}_{\boldsymbol{Y}}=\boldsymbol{Y}^{\top}\boldsymbol{Y}/n=\boldsymbol{L},$$

而且这里的 $\boldsymbol{L}$ 为原数据样本相关阵 $\boldsymbol{R}_{\boldsymbol{X}}=\boldsymbol{X}^{\top}\boldsymbol{X}/n$ 的特征值 $\ell_1,\ell_2,\ldots,\ell_p$ 组成的对角线矩阵, 即 $\boldsymbol{L}=\mathrm{diag}(\ell_1,\ell_2,\ldots,\ell_p)$.

细节: 记 $n\times p$ 维数据矩阵 $\boldsymbol{X}=[\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_p]$, 这里的 $\boldsymbol{x}_i=(x_{1j},x_{2j},\ldots,x_{nj})^{\top}$ $(j=1,2,\ldots,p)$, 即

$$\boldsymbol{X}=[\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_p]=\begin{bmatrix}x_{11}&x_{12}&\cdots&x_{1p}\\x_{21}&x_{22}&\cdots&x_{2p}\\\vdots&\vdots&\ddots&\vdots\\x_{n1}&x_{n2}&\cdots&x_{np}\end{bmatrix}.$$

由于中心化 (等价于空间中的平移) 不影响我们的结果, 可以假定矩阵 $\boldsymbol{X}$ 是中心化的, 即其每一列的样本均值 (或者和) 为零: $\sum_{j=1}^{n}x_{ij}=0\ (i=1,2,\ldots,p)$. 我们的目标是希望找到 $p\times p$ 维矩阵 $\boldsymbol{A}=[\boldsymbol{a}_1,\boldsymbol{a}_2,\ldots,\boldsymbol{a}_p]$, 这里的 $\boldsymbol{a}_i=(a_{i1},a_{i2},\ldots,a_{ip})^{\top}\ (i=1,2,\ldots,p)$, 即

$$\boldsymbol{A}=[\boldsymbol{a}_1,\boldsymbol{a}_2,\ldots,\boldsymbol{a}_p]=\begin{bmatrix}a_{11}&a_{21}&\cdots&a_{p1}\\a_{12}&a_{22}&\cdots&a_{p2}\\\vdots&\vdots&\ddots&\vdots\\a_{1p}&a_{2p}&\cdots&a_{pp}\end{bmatrix},$$

而且满足 $\boldsymbol{A}$ 的列不相关, 变换则为:

$$\boldsymbol{Y}=\begin{bmatrix}y_{11}&y_{21}&\cdots&y_{p1}\\y_{12}&y_{22}&\cdots&y_{p2}\\\vdots&\vdots&\ddots&\vdots\\y_{1n}&y_{2n}&\cdots&y_{pn}\end{bmatrix}=\boldsymbol{X}\boldsymbol{A}=\begin{bmatrix}x_{11}&x_{12}&\cdots&x_{1p}\\x_{21}&x_{22}&\cdots&x_{2p}\\\vdots&\vdots&\ddots&\vdots\\x_{n1}&x_{n2}&\cdots&x_{np}\end{bmatrix}\begin{bmatrix}a_{11}&a_{21}&\cdots&a_{p1}\\a_{12}&a_{22}&\cdots&a_{p2}\\\vdots&\vdots&\ddots&\vdots\\a_{1p}&a_{2p}&\cdots&a_{pp}\end{bmatrix}$$

$$=\begin{bmatrix}\sum_{i=1}^{p}a_{1i}x_{1i}&\sum_{i=1}^{p}a_{2i}x_{1i}&\cdots&\sum_{i=1}^{p}a_{pi}x_{1i}\\\sum_{i=1}^{p}a_{1i}x_{2i}&\sum_{i=1}^{p}a_{2i}x_{2i}&\cdots&\sum_{i=1}^{p}a_{pi}x_{2i}\\\vdots&\vdots&\ddots&\vdots\\\sum_{i=1}^{p}a_{1i}x_{ni}&\sum_{i=1}^{p}a_{2i}x_{ni}&\cdots&\sum_{i=1}^{p}a_{pi}x_{ni}\end{bmatrix}.$$

协方差矩阵等于 $\boldsymbol{L}$ 的条件则要求:

$$\begin{aligned}\boldsymbol{S_Y} =&(\boldsymbol{Y}^\top\boldsymbol{Y})/n = ((\boldsymbol{XA})^\top\boldsymbol{XA})/n\\ =&\boldsymbol{A}^\top(\boldsymbol{X}^\top\boldsymbol{X})\boldsymbol{A}/n = \boldsymbol{A}^\top\boldsymbol{S_X}\boldsymbol{A}\\ =&\boldsymbol{L},\end{aligned}$$

这里的 $\boldsymbol{S_X} = \boldsymbol{X}^\top\boldsymbol{X}/n$ 是原数据的样本协方差矩阵. 根据2.2.1节的特征值分解, 对称矩阵 $\boldsymbol{S_X}$ $(p\times p)$ 能够写成:

$$\boldsymbol{S_X} = \boldsymbol{ALA}^\top,$$

这里的 $\boldsymbol{L} = \mathrm{diag}(\ell_1, \ell_2, \ldots, \ell_p)$, 为 $\boldsymbol{S_X}$ 的特征值所组成的对角矩阵, 而且

$$\boldsymbol{A} = [\boldsymbol{a}_1, \boldsymbol{a}_2, \ldots, \boldsymbol{a}_p]$$

为 $\boldsymbol{S}$ 的特征值 ℓ_j $(j = 1, 2, \ldots, p)$ 所对应的特征向量 $\boldsymbol{a}_j$ $(j = 1, 2, \ldots, p)$ 作为列向量组成的一个正交矩阵.

我们问题中所求的 $\boldsymbol{A}$ 正是由上面特征值分解得到的 $\boldsymbol{A}$, 我们目标的两个条件都能够得到满足, 即

$$\boldsymbol{a}_i^\top\boldsymbol{a}_j = 0\ (\forall i \neq j)\ \text{ 及 }\ \boldsymbol{S_Y} = \boldsymbol{A}^\top\boldsymbol{S_X}\boldsymbol{A} = \boldsymbol{A}^\top\boldsymbol{ALA}^\top\boldsymbol{A} = \boldsymbol{L}.$$

奇异值分解的应用

奇异值分解常用来解主成分分析, 下面一段基本上重复2.2.2节的论述.

假定秩为 r 的 $n\times p$ 数据矩阵为 $\boldsymbol{X}$. 主成分分析中求 $\boldsymbol{X}^\top\boldsymbol{X}$ 或 $\boldsymbol{XX}^\top$ 的特征值问题可以归结为奇异值分解

$$\boldsymbol{X} = \boldsymbol{UDV}^\top,$$

这里的矩阵 $\boldsymbol{U}_{n\times r}$ 和 $\boldsymbol{V}_{p\times r}$ 都是列正交的, 即 $\boldsymbol{U}^\top\boldsymbol{U} = \boldsymbol{V}^\top\boldsymbol{V} = \boldsymbol{I}_r$, 而且 $\boldsymbol{D}$ 为 $\boldsymbol{XX}^\top$ 及 $\boldsymbol{X}^\top\boldsymbol{X}$ 的 r 个共同非零特征值 $(\lambda_1, \lambda_2, \ldots, \lambda_r)$ 的平方根所组成的对角矩阵:

$$\boldsymbol{D} = \mathrm{diag}(\lambda_1^{1/2}, \lambda_2^{1/2}, \ldots, \lambda_r^{1/2})\ (\lambda_j > 0).$$

$\boldsymbol{U}$ 的列包含了 $\boldsymbol{XX}^\top$ 的特征向量, $\boldsymbol{V}$ 的列包含了 $\boldsymbol{X}^\top\boldsymbol{X}$ 的特征向量.

奇异值分解求特征值比较快, 许多软件在解特征值问题时使用奇异值分解.

5.5.3 累积贡献率

考虑随机向量 $\boldsymbol{X} = (X_1, X_2, \ldots, X_p)^\top$, 为了降维, 如从 p 降到 q $(1 \leqslant q < p)$ 维, 取相应于最大的 q $(< p)$ 个特征值 $(\lambda_1 \geqslant \lambda_2 \geqslant \cdots \geqslant \lambda_q)$ 的 $p\times p$ 特征向量矩阵 $\boldsymbol{\Gamma}$ 的前 q 列组成的 $p\times q$ 矩阵 $\boldsymbol{\Gamma_q}$, 则相应的正交线性变换为:

$$\boldsymbol{Y} = \boldsymbol{\Gamma}_q^\top\boldsymbol{X},$$

这里变换后的 $\boldsymbol{Y}$ 为 q 维向量. 该变换使得在 q 维的限制下 $\boldsymbol{Y}$ 的方差最大. 由于这些投影是正交的, 保留 q 个主成分所得到的方差的百分比为:

$$100 \times \sum_{k=1}^{q} \lambda_k \Big/ \sum_{k=1}^{p} \lambda_k .$$

这就是前面例5.1教师数据计算中的累积贡献率.

5.5.4 总体主成分的一些性质

这里考虑主成分的总体 (不是样本) 性质. 首先, 考虑 p 维椭球面 $\boldsymbol{x}^{\top}\boldsymbol{\Sigma}^{-1}\boldsymbol{x} = \text{const}$, 这里 $\boldsymbol{\Sigma}$ 是 p 维随机变量 $\boldsymbol{X}$ 的协方差矩阵, "const" 代表一个常数. 我们有分解 $\boldsymbol{\Sigma} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}\boldsymbol{\Gamma}^{\top}$, 由于主成分 $\boldsymbol{Y} = \boldsymbol{\Gamma}^{\top}\boldsymbol{X}$, 上述的 p 维椭球面在主成分的坐标中为 $\boldsymbol{y}^{\top}\boldsymbol{\Lambda}^{-1}\boldsymbol{y} = \text{const}$. 这可以写成

$$\sum_{k=1}^{p} \frac{y_k^2}{\lambda_k} = \text{const},$$

这意味着椭球主轴的半长度和 $\sqrt{\lambda_i}$ $(i = 1, 2, \ldots, p)$ 成比例. 因此, 如果 $\boldsymbol{X}$ 服从多元正态分布, 椭球面 $\boldsymbol{x}^{\top}\boldsymbol{\Sigma}^{-1}\boldsymbol{x} = \text{const}$ 定义了相等的概率, 而互相正交的主轴按照特征值的大小确定了相应方向上方差的大小.

实际上, 主成分分析是一个描述性统计方法. 一般来说, 特别是在应用中, 并不要求对变量的正态性分布, 也无法验证这一要求.

主成分还有下面的性质:

$$\sum_{j=1}^{p} \text{Var}(Y_j) = \sum_{j=1}^{p} \lambda_j = \text{trace}(\boldsymbol{\Sigma}), \quad \prod_{j=1}^{p} \text{Var}(Y_j) = \prod_{j=1}^{p} \lambda_j = |\boldsymbol{\Sigma}|.$$

另外还可以表明, 对两个独立并有相同分布的 p 维随机向量 $\boldsymbol{X}_1$ 和 $\boldsymbol{X}_2$ 做相同的变换 $\boldsymbol{Y}_i = \boldsymbol{B}^{\top}\boldsymbol{X}_i$ $(i = 1, 2)$, 这里, $\boldsymbol{B}_{p\times q}$ 有正交的列使得 $E[(\boldsymbol{Y}_1 - \boldsymbol{Y}_2)^{\top}(\boldsymbol{Y}_1 - \boldsymbol{Y}_2)]$ 达到最大, 那么必然有 $\boldsymbol{B} = \boldsymbol{\Gamma}_q$, 这里的 $\boldsymbol{\Gamma}_q$ 为 $\boldsymbol{\Sigma}$ 的前面 q 个最大特征向量组成的矩阵. 这意味着两个同样分布的 p 维向量在 q 维子空间中的期望欧氏距离在前 q 个主成分定义的子空间中达到最大.

5.5.5 样本主成分的一些性质

如果有中心化数据阵 $\boldsymbol{X}_{n\times p}$, 其第 ij 个元素为 $x_{ij} - \bar{x}_j$. 也就是说, $\boldsymbol{X}$ 为零均值设计阵. 作为协方差矩阵的无偏估计的样本协方差矩阵为:

$$\boldsymbol{S} = \frac{1}{n-1}\boldsymbol{X}^{\top}\boldsymbol{X}.$$

如果 $\boldsymbol{S}$ 的特征值分解为 $\boldsymbol{A}\boldsymbol{L}\boldsymbol{A}^{\top}$, 定义主成分得分为:

$$\boldsymbol{Y} = \boldsymbol{X}\boldsymbol{A}.$$

这时主成分 $\boldsymbol{Y}_i$ 的方差为特征值 ℓ_i, 而对角线矩阵 $\boldsymbol{L}$ 是用 ℓ_i $(i=1,2,\ldots,p)$ 作为对角线元素.

这时, 椭球面 $(x-\bar{\boldsymbol{x}})^{\top}\boldsymbol{S}^{-1}(x-\bar{\boldsymbol{x}})=\text{const}$ 被解释为到样本均值距离 $\bar{\boldsymbol{x}}$ 有相等 Mahalanobis 距离的面. 因此, 主成分分析也被解释为寻找数据距离一个包含所有数据的超球面有最小 Mahalanobis 距离的正交方向.

类似于总体情况, 对观测值 $\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_n$ 做相同的变换 $\boldsymbol{y}_i=\boldsymbol{B}^{\top}\boldsymbol{x}_i$ $(i=1,2,\ldots,n)$, 这里, $\boldsymbol{B}_{p\times q}$ 有正交的列使得 $\sum_{i=1}^{n}\sum_{j=1}^{n}(\boldsymbol{y}_i-\boldsymbol{y}_j)^{\top}(\boldsymbol{y}_i-\boldsymbol{y}_j)$ 达到最大, 那么必然有 $\boldsymbol{B}=\boldsymbol{A}_q$, 这里的 $\boldsymbol{A}_q$ 为 $\boldsymbol{S}$ 前面 q 个最大特征向量组成的矩阵. 这意味着这些向量在 q 维子空间中的平方欧氏距离之和在前 q 个主成分定义的子空间中达到最大.

5.5.6 主成分的解释

主成分 $\boldsymbol{Y}=\boldsymbol{\Gamma}^{\top}\boldsymbol{X}$ 和原始变量 $\boldsymbol{X}$(记 $\boldsymbol{\mu}=E(\boldsymbol{X})$) 的协方差为:

$$\begin{aligned}\operatorname{Cov}(\boldsymbol{X},\boldsymbol{Y}) =&E(\boldsymbol{X},\boldsymbol{Y}^{\top})-E\boldsymbol{X}E\boldsymbol{Y}^{\top}\\ =&E(\boldsymbol{X}\boldsymbol{X}^{\top}\boldsymbol{\Gamma})-\boldsymbol{\mu}\boldsymbol{\mu}^{\top}\boldsymbol{\Gamma}=\operatorname{Cov}(\boldsymbol{X})\boldsymbol{\Gamma}\\ =&\boldsymbol{\Sigma}\boldsymbol{\Gamma}=\boldsymbol{\Gamma}\boldsymbol{\Lambda}\boldsymbol{\Gamma}^{\top}\boldsymbol{\Gamma}\\ =&\boldsymbol{\Gamma}\boldsymbol{\Lambda}.\end{aligned}$$

因此, 变量 X_i 和成分 Y_j 的相关系数 $\rho_{X_iY_j}$ 为:

$$\rho_{X_iY_j}=\frac{\Gamma_{ij}\lambda_j}{(\Sigma_{X_iX_i}\lambda_j)^{1/2}}=\gamma_{ij}\left(\frac{\lambda_j}{\Sigma_{X_iX_i}}\right)^{1/2}.$$

对于实际数据为:

$$r_{X_iY_j}=a_{ij}\left(\frac{\ell_j}{s_{X_iX_i}}\right)^{1/2}.$$

对于标准化的 X, $r_{X_iY_j}=a_{ij}\sqrt{\ell_j}$. 注意,

$$\sum_{j=1}^{p}r_{X_iY_j}^2=\frac{\sum\limits_{j=1}^{p}\ell_j a_{ij}^2}{s_{X_iX_i}}=\frac{s_{X_iX_i}}{s_{X_iX_i}}=1.$$

实际上, $\sum_{j=1}^{p}\ell_j A_{ij}^2=\boldsymbol{a}_i^{\top}\boldsymbol{L}\boldsymbol{a}_i$ 是矩阵 $\boldsymbol{A}\boldsymbol{L}\boldsymbol{A}^{\top}=\boldsymbol{S}$ 的第 (i,i) 个元素, 因此 $r_{X_iY_j}^2$ 可以看成 X_i 的方差被 Y_j 所解释的比例.

5.5.7 标准化主成分分析

对于异方差情况, 数据 $\boldsymbol{X}_{n\times p}=(\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_p)$ 需要标准化, 即使得每个元素减去列均值 (中心化) 并除以列的样本标准差而成为 $(x_{ij}-\bar{x}_i)/s_i$ $(i=1,2,\ldots,p)$, 这里的 $\{s_i\}$ 代表各列的样本标准差. 标准化之后的数据阵 $\boldsymbol{X}_S$ 的列均值为 0, 列标准差为 1. 这种标准化

可以用下面矩阵符号表示:

$$\boldsymbol{X}_S = \boldsymbol{H}\boldsymbol{X}\boldsymbol{D}^{-1/2},$$

这里的 $\boldsymbol{H} = \boldsymbol{I}_n - \frac{1}{n}\mathbf{1}_n\mathbf{1}_n^\top$ 为中心化矩阵, 即 $\boldsymbol{H}\boldsymbol{X}$ 的第 ij 个元素为 $x_ij - \bar{x}_j$; 而 $\boldsymbol{D} = \mathrm{diag}(s_1^2, s_2^2, \ldots, s_p^2)$ 是对角线元素为样本方差的对角矩阵. $\boldsymbol{X}_S$ 的主成分变换称为标准化主成分. 记 $\boldsymbol{X}$ 的相关阵为 $\boldsymbol{S}_{\boldsymbol{X}_S} = \boldsymbol{R}$, 它等于 $\boldsymbol{X}_S$ 的协方差阵 (注意, 列变量无论是否标准化, 相关阵不变). $\boldsymbol{R}$ 的谱分解为:

$$\boldsymbol{R} = \boldsymbol{A}_R\boldsymbol{L}_R\boldsymbol{A}_R^\top,$$

这里 $\boldsymbol{L}_R = \mathrm{diag}(\ell_1^{\boldsymbol{R}}, \ell_2^{\boldsymbol{R}}, \ldots, \ell_p^{\boldsymbol{R}})$ 及 $\ell_1^{\boldsymbol{R}} \geqslant \ell_2^{\boldsymbol{R}} \geqslant \cdots \geqslant \ell_p^{\boldsymbol{R}}$ 为 $\boldsymbol{R}$ 的相应于特征向量 $a_1^{\boldsymbol{R}}, a_2^{\boldsymbol{R}}, \ldots, a_p^{\boldsymbol{R}}$ 的特征值, 而且 $\sum_{j=1}^p \ell_j^{\boldsymbol{R}} = \mathrm{tr}(\boldsymbol{R}) = p$.

前面例5.1教师数据的主成分计算就是标准化主成分计算. 在那里, 将数据 `u[,-1]` 标准化的代码为 `scale(u[,-1])`, 而解其特征值问题的代码为 `a=eigen(cor(u[-1]))`.

标准化主成分提供了每个个体的得分:

$$\boldsymbol{Z} = \boldsymbol{X}_S\boldsymbol{A}_R.$$

前面例5.1教师数据个体的主成分得分计算代码为 `scale(u[,-1])%*%a$ve`. 对于得分 $\boldsymbol{Z}$, 记其协方差阵 (不是相关阵) 为 $\boldsymbol{S}_Z$, 有

$$\boldsymbol{S}_Z = \boldsymbol{A}_R^\top\boldsymbol{S}_{\boldsymbol{X}_S}\boldsymbol{A}_R = \boldsymbol{A}_R^\top\boldsymbol{R}\boldsymbol{A}_R = \boldsymbol{L}_R.$$

5.6 本章计算的 Python 代码

首先要输入一些模块, 这些模块在本节会用到.

```
import pandas as pd
import numpy as np
matplotlib.use('TkAgg')#使用Windows不需要这个命令
import matplotlib.pyplot as plt
%matplotlib inline
```

5.6.1 例5.1教师数据

求协方差矩阵及样本相关阵的特征值和特征向量

读入数据, 并求数据相关阵的特征值和特征向量 (只输出 4 列特征向量):

```
w=pd.read_csv("full.aaup.csv")
u=w[w.columns[4:]]
ava,ave=np.linalg.eig(np.corrcoef(u.T))
print("eigenvalues=\n",ava,"4 eigen vectors=\n",ave[:,:4])
```

输出为:

```
eigenvalues=
 [  9.30528828e+00   2.43991068e+00   7.06092317e-01   1.80195778e-01
   1.36677759e-01   7.62450750e-02   6.67250540e-02   5.21300766e-02
   2.70207415e-02   4.98024612e-03   2.73310426e-04   1.90026170e-03
   2.56042138e-03] 4 eigen vectors=
 [[ 0.31273824  0.13520345 -0.02804996 -0.30274173]
 [ 0.30502296  0.18915188 -0.11642372  0.11576999]
 [ 0.30568127  0.15595337 -0.08826369  0.41070451]
 [ 0.31187011  0.15730953  0.07045888 -0.30599752]
 [ 0.31098143  0.15795763 -0.03176836 -0.27621024]
 [ 0.30214696  0.20768459 -0.11074421  0.137697  ]
 [ 0.30382362  0.16839536 -0.0879139   0.42882607]
 [ 0.31028804  0.17519777  0.05822265 -0.25605919]
 [ 0.24334906 -0.35693583  0.33116826 -0.33087167]
 [ 0.24106277 -0.39369935  0.17830881  0.26108295]
 [ 0.23529148 -0.41237627  0.04912807  0.26515144]
 [ 0.08890082 -0.3897086  -0.87898802 -0.19914972]
 [ 0.24752096 -0.40670597  0.17891781 -0.01775209]]
```

求各个成分的贡献率和累积贡献率, 并产生崖底碎石图

求各个成分的贡献率和累积贡献率:

```
cca=ava/np.sum(ava)#贡献率
ca=np.cumsum(ava)/np.sum(ava)#累积贡献率
```

产生崖底碎石图 (见图5.14):

```
fig=plt.figure(figsize=(10,4))
fig.add_subplot(121)
plt.scatter(range(1,14),ava)
plt.plot(range(1,14),ava)
plt.title('Scree Plot')
plt.xlabel('Component Number')
plt.ylabel('Eigen Value')
fig.add_subplot(122)
plt.scatter(range(1,14),ca)
plt.plot(range(1,14),ca)
plt.title('Cumulative Contribution')
plt.xlabel('Component Number')
plt.ylabel('Cumulative Contribution')
#plt.show()
```

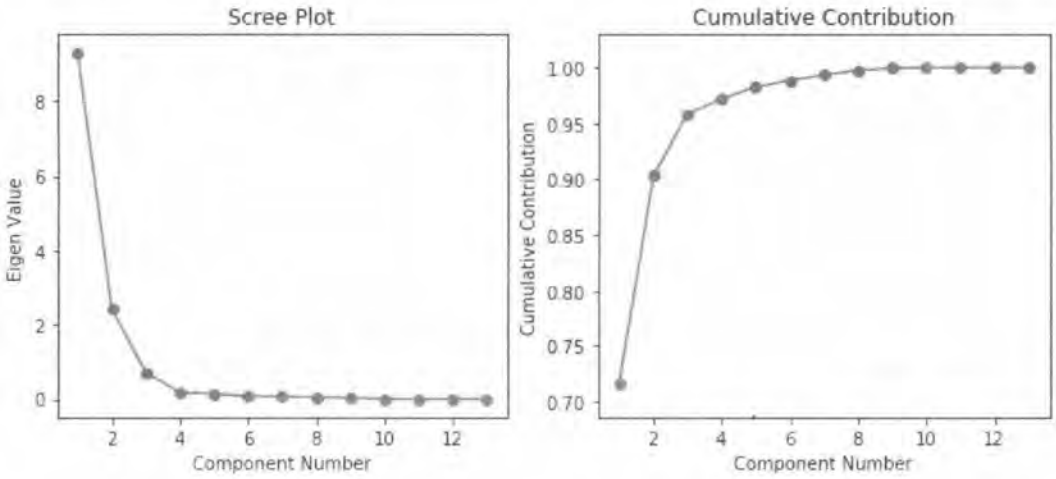

图 5.14 例5.1教师数据主成分分析的崖底碎石图

主成分和原始变量的相关系数与载荷图

下面第一行代码得到各个变量 (比如第 j 个) 和诸成分 (比如第 i 个) 的相关系数 ($\sqrt{\ell_i}a_{ij}$), 后面的代码产生载荷图 (见图5.15).

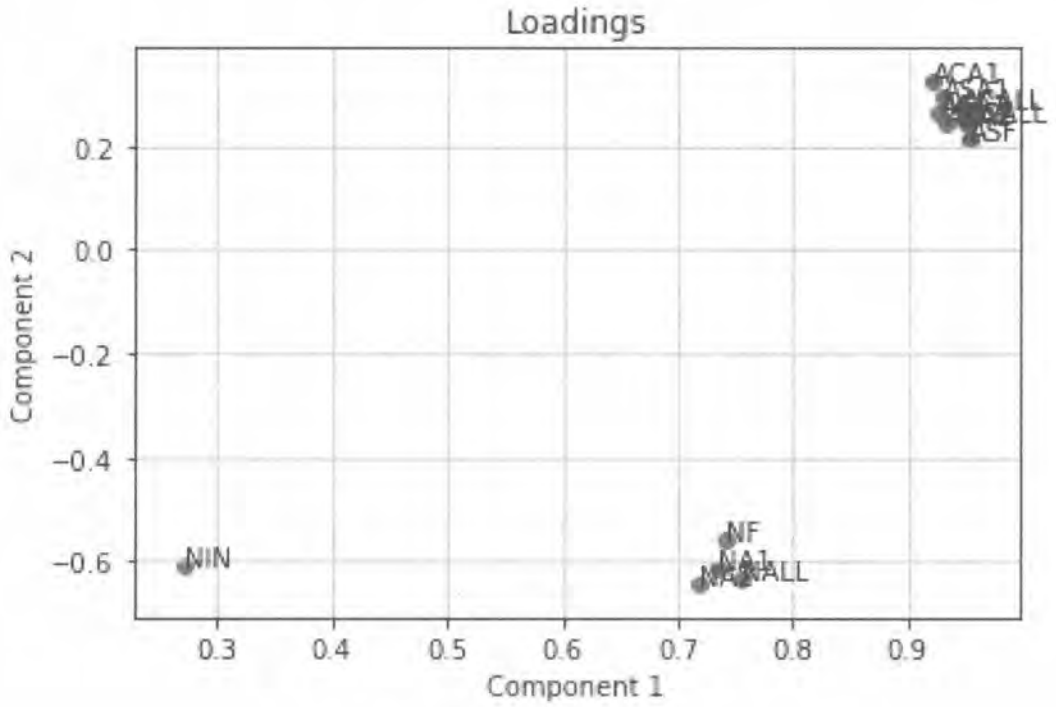

图 5.15 例5.1教师数据主成分分析的载荷图

```
loadings=np.sqrt(ava)*ave

plt.scatter(loadings[:,0],loadings[:,1])
for i in range(13):
    plt.text(loadings[i,0],loadings[i,1],w.columns[4:][i])
plt.title('Loadings')
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.grid(True)
plt.show()
```

主成分得分

下面代码前 3 行将数据标准化和计算观测值得分, 中间三行把加利福尼亚州的数据抽出来. 最后, 分别画出对所有学校和加利福尼亚州学校的主成分得分图 (见图5.16).

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
sc=scaler.fit_transform(u)su.dot(ave)

CA=np.array(w[w.columns[2]]=='CA')
name_CA=np.array(w[w.columns[1]][CA])
sc1=sc[CA,:]

fig=plt.figure(figsize=(10,8))
fig.add_subplot(121)
plt.scatter(sc[:,0],sc[:,1])
for i in range(u.shape[0]):
    plt.text(sc[i,0],sc[i,1],w[w.columns[1]][i])
plt.title('Sample Principal Components')
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.grid(True)
fig.add_subplot(122)
plt.scatter(sc1[:,0],sc1[:,1])
for i in range(sc1.shape[0]):
    plt.text(sc1[i,0],sc1[i,1],name_CA[i])
plt.title('Sample Principal Components for California Schools')
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.grid(True)
plt.show()
```

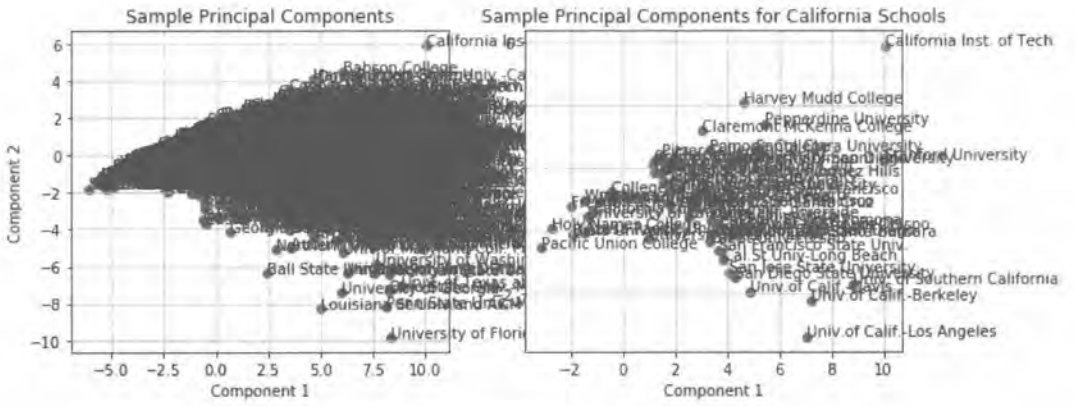

图 5.16 例5.1教师数据主成分分析的得分图

直接利用函数 sklearn.decomposition.PCA 做主成分分析

前面是用代数公式得到主成分分析的结果, 这里直接利用模块 sklearn 中的函数做主成分分析. 我们只输出载荷图 (见图5.17).

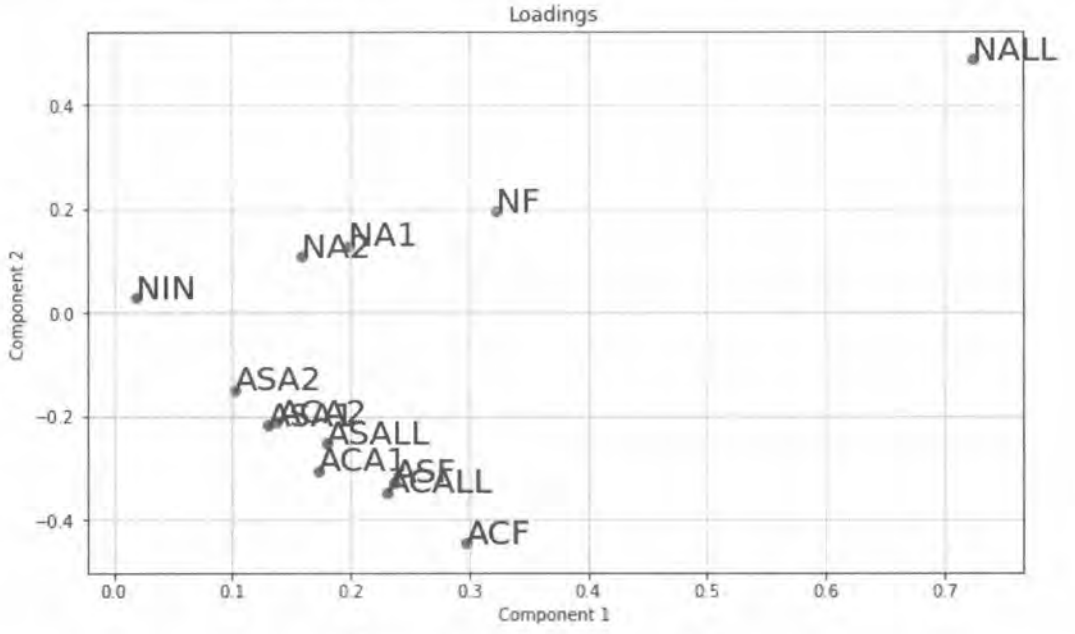

图 5.17 例5.1教师数据主成分分析的载荷图

首先读入数据:

```
import pandas as pd
w=pd.read_csv("full.aaup.csv")
u=w[w.columns[4:]];X=np.array(u)
```

拟合数据:

```
import numpy as np
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
Y=pca.fit_transform(X)#主成分得分
print(pca.explained_variance_ratio_) #方差百分比
```

打印出两个主成分被解释的方差百分比 [0.77370134 0.19963845].

画出载荷图 (见图5.17):

```
plt.figure(figsize=(10,7))
plt.scatter(pca.components_[0],pca.components_[1])
for i in range(13):
    plt.text(pca.components_[0,i],pca.components_[1,i],w.columns[4:][i],
    fontsize=20)
plt.title('Loadings')
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.grid(True)
plt.show()
```

5.6.2 图像压缩例子

例 5.5. 牌坊图片 (PF.jpg) 这是一个牌坊的照片 (见图5.18的右图), 我们用它做图像压缩.

首先输入一些必要的模块:

```
import scipy
import scipy.ndimage
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
from pylab import imread,subplot,imshow,title,gray,figure,show,NullLocator
```

定义图像压缩主成分分析函数:

```
def comp_2d(image_2d,numpc=100):
    cov_mat = (image_2d.T - np.mean(image_2d , axis = 1)).T
    eig_val, eig_vec = np.linalg.eigh(np.cov(cov_mat))
    p = np.size(eig_vec, axis =1)
    idx = np.argsort(eig_val)
    idx = idx[::-1]
    eig_vec = eig_vec[:,idx]
    eig_val = eig_val[idx]
    if numpc <p or numpc >0:
```

```
        eig_vec = eig_vec[:, range(numpc)]
    score = np.dot(eig_vec.T, cov_mat)
    recon = (np.dot(eig_vec, score).T + np.mean(image_2d, axis = 1)).T
    recon_img_mat = np.uint8(np.absolute(recon))
    return recon_img_mat
```

定义使用上述函数和画图函数:

```
def pltPCA(a,numpc):
    a_np = np.array(a)
    a_r = a_np[:,:,0]
    a_g = a_np[:,:,1]
    a_b = a_np[:,:,2]
    a_r_recon, a_g_recon, a_b_recon = comp_2d(a_r,numpc),\
    comp_2d(a_g,numpc), comp_2d(a_b,numpc)
    recon_color_img = np.dstack((a_r_recon, a_g_recon, a_b_recon))
    recon_color_img = Image.fromarray(recon_color_img)
    fig = plt.figure(figsize=(4, 3), dpi=72)
    ax = fig.add_axes([0.0, 0.0, 1.0, 1.0], frameon=False, aspect=1)
    ax.set_xticks([])
    ax.set_yticks([])
    imshow(recon_color_img)
```

具体画图 (见图5.18) 代码 (分别取 10 和 100 个主成分) 为:

```
a = scipy.ndimage.imread("PF.jpg")
pltPCA(a,10);pltPCA(a,100)
```

图 5.18 左图和中图为对图片 PF.jpg 的两种压缩 (分别取 10 和 100 个主成分), 右图为原图

5.7 习 题

1. 回答下面问题:
   (1) 主成分分析完全是数学推导和统计方法的结果吗? 有没有主观决策的成分?
   (2) 主成分分析变量的选择, 成分个数的选择、距离度量的选择会对结果有什么影响? 请一一做出解释.

2. 请举例说明根据主成分分析得到的有关“政绩”的“综合指数”的荒诞之处.
3. 能够根据主成分分析做出确定性结论吗? 请说明.
4. 以例5.1教师数据的主成分得分为新的数据阵, 再做一次主成分分析,
   (1) 特征值是多少? 请画出载荷图.
   (2) 验证得分组成的新数据阵正交.
   (3) 你会得到什么结论? 请对你的结果做出解释.
5. 随便取一张图片, 利用主成分分析做图片的压缩 (分别采用不同数量的主成分), 并产生压缩后的图片.
6. 对例6.1WHO 数据做完整的主成分分析.
   (1) 求出特征值和特征向量;
   (2) 选择主成分的数目, 并作载荷图.
   (3) 尝试给各个主成分命名.
   (4) 作观测值的主成分得分图.
7. 对例6.2豌豆数据做完整的主成分分析.
   (1) 求出特征值和特征向量.
   (2) 选择主成分的数目, 并作载荷图.
   (3) 尝试给各个主成分命名.
   (4) 作观测值的主成分得分图.

第 6 章　因子分析

6.1　基本内容

6.1.1 概述

主成分分析主要是为了降维, 本章的因子分析也可以用于降维. 但除此之外, 因子分析还有其他的用途, 在理论上也很不一样.

在很多领域中, 我们可以观测到许多变量, 比如, 可以观测到汽车的电器功能、对驾驶员操作的反应速度、油耗、舒适程度、加速性能、外观、性价比、安全性、折旧率、尺寸等等. 这些变量很可能隐含着可靠性、经济适用性等无法直接观测的变量 (称为隐变量或潜变量), 而人们希望用可观测变量来得到隐变量的信息. 这些隐变量就是因子分析中的**因子**.

回归或者分类方法可以用来研究因变量和自变量之间的关系. 因子分析则用来研究许多因变量之间的关系, 目的是找到影响这些因变量的不可观测的独立变量, 而由因子分析得到的结果大多是假设性的或实验性的, 这是由于人们不能观测到这些自变量.

因子分析可能针对下面一些问题:

(1) 需要多少因子来解释可观测变量之间的变化.

(2) 如何认识这些因子的性质和意义.

(3) 这些假设的因子是否很好地解释了那些可观测变量.

除了前面的例5.1教师数据之外, 在本章我们还考虑以下例子.

例 6.1. WHO 数据 (who1.csv) 该数据是从世界卫生组织 (WHO) 数据中节选出来的. 由 185 个国家和 29 个变量组成, 第一个变量 country 是国家名字, 后面变量为 PT (总人口, 单位: 千人)、PGR (人口增长率, 单位: %)、PUP (城市人口比例, 单位: %)、P60P (60 岁以上人口比例, 单位: %)、P15P (15 岁以下人口比例, 单位: %)、TFR (妇女怀孕率)、IMR (一岁童 MCV 接种率, 单位: %)、I3R (一岁童 3 次乙肝接种率, 单位: %)、HER (卫生的外部资源占卫生总支出的百分比, 单位: %)、HGR (一般政府卫生支出占卫生总支出的百分比, 单位: %)、HGGR (一般政府卫生支出占政府总支出的百分比, 单位: %)、NDP (人均牙医人数)、NNMP (人均护士助产士人数)、NPP (人均内科医生人数)、NMP (每万人护士助产士人数)、HPP (自付开支占私人健康支出的百分比, 单位: %)、HPHP (私人健康支出占健康总支出的百分比, 单位: %)、NMPP (护士助产士与内科医生的比例, 单位: %)、HGDP (卫生总支出占国内生产总值的百分比, 单位: %)、DR1 (每 10 万人口癌症死亡数)、DR2 (每 10 万人口心血管死亡数)、DR3 (每 10 万人口因伤死亡数)、DR4 (每 10 万人口非传染病因死亡数)、TBN (每 10 万人口结核病数)、NMN (每千名新生儿死亡数)、YCP (传染病损失年龄比例, 单位: %)、YIP (受伤损失年龄比例, 单位: %)、YNCP (非传

染病损失年龄比例, 单位: %). 各个变量的相关系数显示在图6.1中.

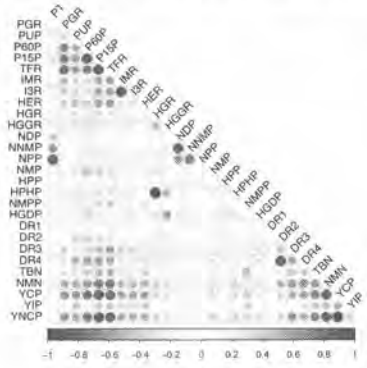

图 6.1 例6.1WHO 数据各个变量的相关系数图

图6.1是由下面代码实现的:

```
w=read.csv("who1.csv")
library(corrplot)
corrplot(cor(w[,-c(1)]),type="lower",diag=FALSE,
   tl.col="black",tl.srt=45)
```

例 6.2. 豌豆数据 (peas2.csv)[1] 该数据是关于 60 个分别属于 27 种豌豆的味道打分数据. 这些豌豆在煮熟之后快速冷冻, 储存三个月后由评委打分. 变量有 Tender(嫩度)、DryM(干物质, 单位: %)、DryMA(冻后干物质, 单位: %)、Sucrose(蔗糖, 单位: %)、Glucose1(葡萄糖度量 1)、Glucose2(葡萄糖度量 2)、Flavour(风味)、Sweet(甜味)、Fruity(果味)、Off.flavour(异味)、Mealiness(绵软性)、Hardness(硬度)、Whiteness(白色度)、Colour1(颜色 1 读数)、Colour2(颜色 2 读数)、Colour3(颜色 3 读数)、Skin(外观). 该数据维数为 60×17. 该数据各个变量之间的相关系数点图显示在图6.2中, 该图是用完全类似于产生图6.1的代码实现的.

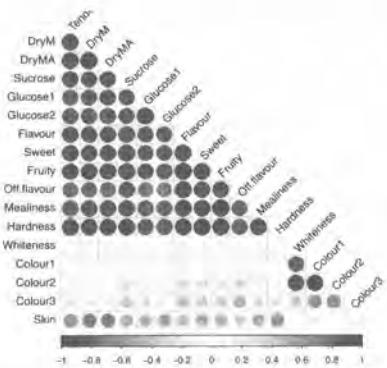

图 6.2 例6.2豌豆数据各个变量的相关系数图

[1]来自网站 http://openmv.net/info/peas, 参看 Kjølstad, L., Isaksson, T., Hans J. Rosenfeld (2006) Prediction of sensory quality by near infrared reflectance analysis of frozen and freeze dried green peas (Pisum sativum) *Journal of the Science of Food and Agriculture*, 51,2.

6.1.2 模型

假定数据有 p 个变量, 在主成分分析中, 人们形成 p 个正交的主成分, 并且选取其中少数成分来代表大部分数据所包含的信息. 这里没有误差结构, 也没有随机性的度量. 无论选择几个成分, 主成分分析中的成分都是从原始变量或者原始数据中完全确定的, 不会因为后续选取成分的个数而变. 在主成分分析中, 希望选择的少数成分能够尽可能解释大部分方差.

在降维方面, 因子分析有些类似于主成分分析, 不同的是, 因子分析有一个随机模型, 它把原始的 p 个变量转换成少数不相关的 m 个因子, 因子数量 m 比原始变量的数量 p 少很多 (或表示为 $m \ll p$). 因子个数 m 必须事先确定, 不同的 m 导致不同的结果. 在模型上, 每个原始变量都表示为所有不可观测因子的线性组合加上随机误差项. 这里的因子是不可观测的假定变量, 称为**隐变量**或**潜变量** (latent variable). 在计算中, 对因子的确定必须在关于变量的许多假定之下才能实现, 这使得因子分析比主成分分析复杂得多. 因子分析通过考察可观测变量之间的相关模式来进行. 高度相关的可观测变量 (无论是正相关还是负相关) 有可能被同一个因子所影响, 相对不那么相关的可观测变量则可能被不同的因子所影响.

在许多研究问题中, 人们通过定性分析猜想出可能有几个潜在的因子, 而且每个因子可能影响某一些可观测变量. 最常见的是心理学的例子, 在那里人们可观测到许多心理度量, 并且假定它们可以被少数几个假设的隐变量影响, 但这样做可能有误, 因此需要通过多次分析来调整原先的模型, 增加或减少一些隐变量. 在很多情况下, 人们事先并不知道需要多少因子, 这就需要通过数据本身来判断.

下面我们叙述因子分析模型. 注意, 这个模型完全是假设的, 合不合适必须由数据来决定. 假定原始变量 $\boldsymbol{x} = (x_1, x_2, \ldots, x_p)^\top$ 有均值 $\boldsymbol{\mu} = (\mu_1, \ldots, \mu_p)^\top$ 及协方差矩阵 $\boldsymbol{\Sigma}$. 如果 $\boldsymbol{f} = (f_1, f_2, \ldots, f_m)^\top$ 为需要找到的因子, 那么它们和原始变量 $\boldsymbol{x}$ 之间的关系为:

$$
\begin{aligned}
x_1 - \mu_1 &= q_{11} f_1 + q_{12} f_2 + \cdots + q_{1m} f_m + \epsilon_1 \\
x_2 - \mu_2 &= q_{21} f_1 + q_{22} f_2 + \cdots + q_{2m} f_m + \epsilon_2 \\
&\vdots \\
x_p - \mu_p &= q_{p1} f_1 + q_{p2} f_2 + \cdots + q_{pm} f_m + \epsilon_p,
\end{aligned}
$$

或者以矩阵形式表示为:

$$
\boldsymbol{x} - \boldsymbol{\mu} = \boldsymbol{Q}\boldsymbol{f} + \boldsymbol{\epsilon}, \tag{6.1.1}
$$

这里 $\boldsymbol{Q}_{p\times m} = \{q_{ij}\}$, $\boldsymbol{\epsilon} = (\epsilon_1, \epsilon_2, \ldots, \epsilon_p)^\top$. 其中, μ_i 为变量 x_i 的均值 ($E(x_i) = \mu_i$); ϵ_i 称为第 i 个**特殊因子**, 只与 x_i 有关; 而 f_i 称为第 i 个**公共因子**, 与所有变量都有关; q_{ij} 为第 i 个变量在第 j 个因子上的**载荷**, 它描述第 i 个可观测变量在多大程度上依赖第 j 个因子. 对于**标准化**的 $\boldsymbol{x}$, q_{ij} 就是第 i 个变量与第 j 个公共因子的相关系数. 图6.3是 5 个可观察变量 x_1, x_2, x_3, x_4, x_5 与 2 个隐变量 f_1, f_2 关系的示意图.

单凭上面的关系式无法算出因子 $\boldsymbol{f}$. 为了计算方便, 必须做出各种假定, 下面就是通常

的假定.

通常假定不可观测的隐变量满足下列条件: $\boldsymbol{f}$ 和 $\boldsymbol{\epsilon}$ 独立, $E(\boldsymbol{f})=\boldsymbol{0}$, $\mathrm{Cov}(\boldsymbol{f})=\boldsymbol{I}_m$, $E(\boldsymbol{\epsilon})=\boldsymbol{0}$, $\mathrm{Cov}(\boldsymbol{\epsilon})=\boldsymbol{\Psi}$, $\boldsymbol{\Psi}=\mathrm{diag}(\psi_1,\psi_2,\ldots,\psi_p)$. 这些条件确保了模型数学上的合理性, 以及各个特殊因子 ϵ_i 之间及各个公共因子 f_i 之间不相关.

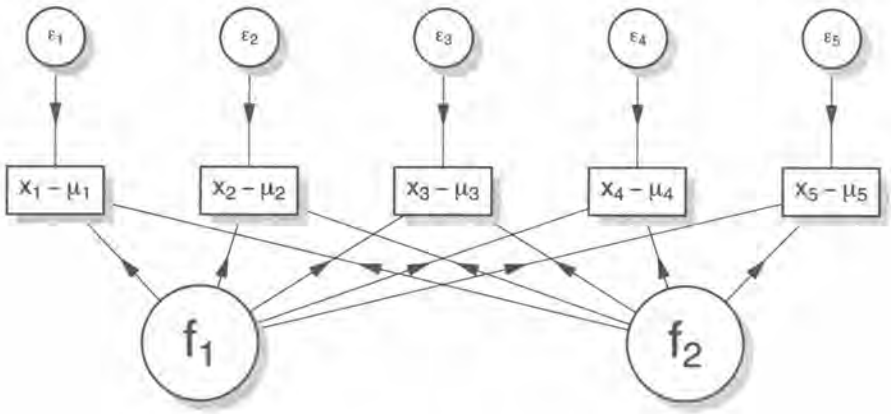

图 6.3 因子分析示意图

6.1.3 性质

对于第 i 个可观测变量 x_i, 它与各个因子的载荷的平方和 $h_i^2\equiv\sum_{k=1}^m q_{ik}^2$ 称为**共性方差**, 也称**公共方差**或**变量共同度** (common variances、commonalities), 变量共同度刻画全部公共因子对变量 x_i 的总方差所做的贡献. 而 $\boldsymbol{\Psi}$ 称为**特殊方差** (specific variance). 显然,

$$\mathrm{Var}(x_i)=\sum_{j=1}^m q_{ij}^2+\psi_i=h_i^2+\psi_i=\text{共同度}+\text{特殊方差},$$

以及

$$\begin{aligned}\boldsymbol{\Sigma}&=\mathrm{Cov}(\boldsymbol{x})=\mathrm{Cov}(\boldsymbol{Qf}+\boldsymbol{\epsilon})\\&=\mathrm{Cov}(\boldsymbol{Qf})+\mathrm{Cov}(\boldsymbol{\epsilon})=\boldsymbol{Q}\mathrm{Cov}(\boldsymbol{f})\boldsymbol{Q}^{\top}+\boldsymbol{\Psi}\\&=\boldsymbol{QIQ}^{\top}+\boldsymbol{\Psi}=\boldsymbol{QQ}^{\top}+\boldsymbol{\Psi}.\end{aligned}$$

由于 $\boldsymbol{\Sigma}=\boldsymbol{QQ}^{\top}+\boldsymbol{\Psi}$, 并注意到 $\boldsymbol{\Psi}$ 是对角矩阵, 则对于 $i\neq j$,

$$\mathrm{Cov}(\boldsymbol{x}_i,\boldsymbol{x}_j)=\sigma_{ij}=\sum_{k=1}^m q_{ik}q_{jk},$$

这意味着如果 $\boldsymbol{x}_i$ 和 $\boldsymbol{x}_j$ 很接近, 那么它们有类似的载荷于 f_k $(k=1,2,\ldots,m)$. 也就是说,

那些 $q_{ik}q_{jk}$ 会比较高; 反之它们会较低.

关于 x_i 和 f_j 的相关, 有

$$\begin{aligned}\mathrm{Cov}(x_i, f_j) &= E[(x_i - \mu_i)(f_j - E(f_j))] \\ &= E[(\sum_{k=1}^{m} q_{ik}f_k)f_j] = \sum_{k=1}^{m} q_{ik}\mathrm{Cov}(f_k, f_j) \\ &= q_{ij}, \quad i = 1, 2, \ldots, p, j = 1, 2, \ldots, m,\end{aligned}$$

用矩阵符号, 有

$$\mathrm{Cov}(\boldsymbol{x}, \boldsymbol{f}) = \boldsymbol{Q}.$$

6.1.4 旋转

模型

$$\boldsymbol{x} - \boldsymbol{\mu} = \boldsymbol{Q}\boldsymbol{f} + \boldsymbol{\epsilon}$$

的解 $\boldsymbol{Q}$ 和 $\boldsymbol{f}$ 并不唯一, 假定 $\boldsymbol{T}$ 为任意的满足 $\boldsymbol{T}\boldsymbol{T}^{\top} = \boldsymbol{I}$ 的正交矩阵. 作为算子的 $\boldsymbol{T}$ 作用于 $\boldsymbol{Q}$ 相当于使 $\boldsymbol{Q}$ 旋转. 那么, 令 $\boldsymbol{Q}^* = \boldsymbol{Q}\boldsymbol{T}, \boldsymbol{f}^* = \boldsymbol{T}^{\top}\boldsymbol{f}$, 有

$$\boldsymbol{x} - \boldsymbol{\mu} = \boldsymbol{Q}\boldsymbol{f} + \boldsymbol{\epsilon} = \boldsymbol{Q}\boldsymbol{T}\boldsymbol{T}^{\top}\boldsymbol{f} + \boldsymbol{\epsilon} = \boldsymbol{Q}^*\boldsymbol{f}^* + \boldsymbol{\epsilon},$$

也就是说, 经过正交矩阵旋转后的解 $\boldsymbol{Q}^*$ 和 $\boldsymbol{f}^*$ 仍然满足模型. 这时,

$$\boldsymbol{\Sigma} = \boldsymbol{Q}^*\boldsymbol{Q}^{*\top} + \boldsymbol{\Psi} = \boldsymbol{Q}\boldsymbol{Q}^{\top} + \boldsymbol{\Psi}.$$

最常用的旋转是使得 $\boldsymbol{Q}^*$ 各列的方差最大的旋转.

6.1.5 因子得分

因子分析的模型是把变量表示成因子带有误差项的线性组合 $\boldsymbol{x} - \boldsymbol{\mu} = \boldsymbol{Q}\boldsymbol{f} + \boldsymbol{\epsilon}$. 对于具体观测值的 $n \times p$ 矩阵 $\boldsymbol{X} = (\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n)^{\top}$ 来说 (这里的 $\boldsymbol{x}_i$ 是 p 维列向量), 该模型可以写成

$$\boldsymbol{x}_i - \boldsymbol{\mu} = \boldsymbol{Q}\boldsymbol{f}_i + \boldsymbol{\epsilon}_i, \ i = 1, 2, \ldots, n,$$

这里的 $\boldsymbol{Q}$ 的列向量为计算得到的因子载荷. 我们希望对每个观测值 i $(i = 1, 2, \ldots, n)$ 估计**因子得分**向量

$$(\{f_{i1}\}, \{f_{i2}\}, \ldots, \{f_{im}\})^{\top}, \ i = 1, 2, \ldots, n,$$

也就是说, 要用变量来表示因子. 把 $\boldsymbol{f}$ 反解出来的方法不止一种, 通常有回归和加权最小二乘法. 由此得到估计的模型

$$\hat{\boldsymbol{f}} = \hat{\boldsymbol{B}}(x - \bar{\boldsymbol{\mu}}),$$

式中的 $\hat{\boldsymbol{B}}_{m\times p}=\{\hat{\beta}_{ij}\}$, 或者

$$\begin{aligned}\hat{f}_1&=\hat{\beta}_{11}x_1+\hat{\beta}_{12}x_2+\cdots+\hat{\beta}_{1p}x_p\\ \hat{f}_2&=\hat{\beta}_{21}x_1+\hat{\beta}_{22}x_2+\cdots+\hat{\beta}_{2p}x_p\\ &\vdots\\ \hat{f}_m&=\hat{\beta}_{m1}x_1+\hat{\beta}_{m2}x_2+\cdots+\hat{\beta}_{mp}x_p,\end{aligned}$$

式中, $m\ll p$. 这里用回归等方法算出的 $\hat{\beta}_{ij}$ 不再是相关系数. 在读任何软件输出时都要注意这一点, 以免与主成分分析的输出格式混淆.

把实际数据 $\boldsymbol{X}=(\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_n)^\top$ 的第 i 个观测值带入上式, 得到该观测值对应的因子得分

$$\begin{aligned}\hat{f}_{i1}&=\hat{\beta}_{11}x_{i1}+\hat{\beta}_{12}x_{i2}+\cdots+\hat{\beta}_{1p}x_{ip}\\ \hat{f}_{i2}&=\hat{\beta}_{21}x_{i1}+\hat{\beta}_{22}x_{i2}+\cdots+\hat{\beta}_{2p}x_{ip}\\ &\vdots\\ \hat{f}_{im}&=\hat{\beta}_{m1}x_{i1}+\hat{\beta}_{m2}x_{i2}+\cdots+\hat{\beta}_{mp}x_{ip}.\end{aligned}$$

6.1.6 计算及分析步骤

综上所述, 我们需要做的计算及分析步骤为:

(1) 首先需要基于对式 (6.1.1) 的假定及可观测变量 $\boldsymbol{x}$ 的观测值估计出式 (6.1.1) 中的 $\boldsymbol{Q}$ 和 $\boldsymbol{\Psi}$ 的值.

(2) 然后再选择适合我们目的的旋转 $\boldsymbol{T}$, 以确定最终的 $\boldsymbol{Q}$ 的估计值.

(3) 通过考察 $\boldsymbol{Q}$ 的估计值 $\hat{\boldsymbol{Q}}$ 可以了解各个因子与原始变量之间的关系, 并以此确定各个因子的意义.

(4) 如果对结果不满意, 还可以试着用不同的因子数目 (即 m 值) 重复上面的过程, 直到满意或放弃为止.

(5) 最后得到因子得分的估计 $\hat{\boldsymbol{f}}$.

与主成分分析不同, 实现这些步骤的方法绝非唯一, 结果也不完全相同. 感兴趣的读者可参考6.3节.

6.2 例子和计算

6.2.1 例6.1WHO 数据分析

初步因子分析, 不进行旋转

对于例6.1的数据, 首先不进行旋转, 暂时选取 7 个因子, 读入数据和实行因子分析并输出载荷的 R 代码为:

```
w=read.csv("who1.csv")
a=factanal(scale(w[,-1]),factors=7,rotation ="none")
a$loadings
```

得到的输出为:

```
Loadings:
    Factor1 Factor2 Factor3 Factor4 Factor5 Factor6 Factor7
PT            0.414   0.494   0.161
PGR   0.633  -0.129           0.154  -0.302          -0.290
PUP  -0.675   0.134                  -0.154
P60P -0.770   0.176          -0.322   0.110  -0.131   0.240
P15P  0.886  -0.242           0.145  -0.166          -0.308
TFR   0.824  -0.257   0.113   0.108  -0.125          -0.268
IMR  -0.565   0.104  -0.109           0.144   0.715
I3R  -0.561   0.124  -0.117           0.135   0.743
HER   0.527  -0.413   0.169
HGR  -0.639  -0.738   0.168   0.130
HGGR -0.455  -0.296   0.192
NDP  -0.202   0.438   0.712   0.157                  -0.109
NNMP -0.232   0.482   0.824   0.172
NPP  -0.160   0.431   0.586   0.201
NMP  -0.426                  -0.190
HPP   0.117          -0.204           0.218
HPHP  0.638   0.738  -0.168  -0.130
NMPP  0.363  -0.366   0.213
HGDP -0.394           0.235  -0.151           0.104  -0.147
DR1   0.103                                  -0.152   0.215
DR2   0.526          -0.101   0.396   0.666
DR3   0.596                   0.486          -0.154   0.189
DR4   0.719  -0.130           0.384   0.558
TBN   0.648  -0.252   0.202                          0.309
NMN   0.875  -0.150   0.116                          0.100
YCP   0.932  -0.236   0.154          -0.202
YIP  -0.549   0.332  -0.299   0.661  -0.240
YNCP -0.912   0.186          -0.179   0.281

                Factor1 Factor2 Factor3 Factor4 Factor5 Factor6 Factor7
SS loadings       9.796   2.798   2.290   1.416   1.228   1.184   0.583
Proportion Var    0.350   0.100   0.082   0.051   0.044   0.042   0.021
Cumulative Var    0.350   0.450   0.532   0.582   0.626   0.668   0.689
```

上面的载荷 (Loadings) 为估计的矩阵 $\hat{Q}$, 其中有些空白, 不代表没有值, 而是值由于比较小 (小于 0.1) 而被忽略. 要想得到全部的数值, 可用代码 `a$loadings[,1:7]`. 上面输出的

下面最后三行中:

- SS loadings 是载荷的每一列的平方和, 即特征值, 或对应因子的方差.
- Proportion Var 是 SS loadings 除以变量数目得到的结果, 即各个因子所占方差的比例.
- Cumulative Var 是 Proportion Var 的累积和.

下面是计算这三行的代码和输出 (当然和上面的相同):

```
> SS=apply(a$loadings^2,2,sum);PV=SS/ncol(w[,-1]);CV=cumsum(SS/ncol(w[,-1]))
> rbind(SS,PV,CV)
   Factor1  Factor2  Factor3  Factor4  Factor5  Factor6  Factor7
SS 9.79605 2.797912 2.290272 1.416081 1.227919 1.183534 0.583347
PV 0.34986 0.099925 0.081795 0.050574 0.043854 0.042269 0.020834
CV 0.34986 0.449784 0.531580 0.582154 0.626008 0.668277 0.689111
```

我们的因子个数选择 (选了 7 个) 有些任意, 一些人凭经验认为, 上面输出中的特征值 SS loadings 大于 1 的就是重要的因子. 我们这里有 6 个因子的特征值大于 1 (分别为 9.796、2.798、2.290、1.416、1.228、1.184), 按照这个规则, 我们可以选取 6 个因子. 我们的结果中有变量的**独特性** (uniquenesses), 可以直接用 a$uniquenesses 查看. 独特性为 1 减去共同度后得到的结果, 即 1 减去载荷矩阵各行的平方和后得到的结果, 可以用代码 1-apply(a$loadings^2,1,sum) 做验证计算. 如果所有因子联合解释某变量的大部分方差, 则该变量具有高共同度, 因而具有较低的独特性. 独特性的输出为:

```
> a$uniquenesses
       PT       PGR       PUP       P60P       P15P       TFR       IMR
0.5543082 0.3822383 0.4900374 0.1788032 0.0080127 0.1381221 0.1203097
      I3R       HER       HGR       HGGR       NDP      NNMP       NPP
0.0814451 0.5201604 0.0050000 0.6525816 0.2187396 0.0050000 0.4022083
      NMP       HPP      HPHP       NMPP      HGDP       DR1       DR2
0.7658980 0.8823500 0.0050000 0.6668617 0.7182895 0.9100380 0.1019443
      DR3       DR4       TBN       NMN       YCP       YIP      YNCP
0.3469229 0.0050000 0.3664546 0.1741203 0.0050000 0.0050000 0.0050000
```

我们例子的变量中, 最独特的前 8 名变量为 DR1、HPP、NMP、HGDP、NMPP、HGGR、PT、HER, 它们的独特性度量都超过了 0.5.

选择因子数目 m, 旋转并做载荷图

我们按照方差大于 1 的原则, 先选取 6 个因子做因子分析并旋转, 发现最后一个因子和任何一个变量的相关系数 (绝对值) 中最大的只有 0.28. 因此最终只选择 5 个因子并做旋转 (采用最大方差准则来旋转, R 选项为 rotation="varimax"), 并输出旋转后的载荷:

```
> a5=factanal(scale(w[,-1]),factors=5,rotation ="varimax",
+             scores="regression")
> a5$loadings
```

```
Loadings:
     Factor1 Factor2 Factor3 Factor4 Factor5
PT             0.651  -0.127
PGR    0.655          -0.181           0.146
PUP   -0.584           0.182  -0.303   0.198
P60P  -0.722   0.146   0.200  -0.280  -0.217
P15P   0.867  -0.124  -0.199   0.180
TFR    0.825          -0.142   0.177
IMR   -0.557           0.165
I3R   -0.562           0.141
HER    0.637  -0.107   0.129   0.135  -0.155
HGR   -0.190  -0.126   0.966  -0.102
HGGR  -0.199           0.494  -0.231
NDP   -0.105   0.862          -0.113
NNMP  -0.128   0.985
NPP   -0.108   0.763
NMP   -0.359           0.210  -0.147  -0.157
HPP           -0.143  -0.140   0.255
HPHP   0.190   0.126  -0.966   0.102
NMPP   0.492           0.165          -0.225
HGDP  -0.281   0.154   0.310  -0.112  -0.213
DR1    0.118
DR2    0.265          -0.149   0.890
DR3    0.556          -0.211   0.341   0.371
DR4    0.499          -0.154   0.848
TBN    0.734                          -0.134
NMN    0.812          -0.239   0.310
YCP    0.967          -0.211
YIP   -0.511   0.115                   0.847
YNCP  -0.959           0.230          -0.118

                Factor1 Factor2 Factor3 Factor4 Factor5
SS loadings       8.317   2.896   2.796   2.198   1.182
Proportion Var    0.297   0.103   0.100   0.078   0.042
Cumulative Var    0.297   0.400   0.500   0.579   0.621
```

而独特性为:

```
> a5$uniquenesses
       PT       PGR       PUP       P60P       P15P       TFR       IMR
0.5573100 0.5116940 0.4925244 0.2924215 0.1595452 0.2596223 0.6546994
      I3R       HER       HGR       HGGR       NDP      NNMP       NPP
0.6568066 0.5246529 0.0050000 0.6544077 0.2330395 0.0050000 0.4041565
      NMP       HPP       HPHP       NMPP       HGDP       DR1       DR2
```

```
0.7799399 0.8942449 0.0050000 0.6751870 0.7429410 0.9769226 0.1138973
      DR3       DR4       TBN       NMN       YCP       YIP      YNCP
0.3876693 0.0050000 0.4352173 0.1793108 0.0050000 0.0050000 0.0050000
```

独特性最大的 5 个变量为: DR1、HPP、NMP、HGDP、NMPP.

从载荷中可以看出:

- 与第 1 因子的相关系数 (绝对值) 大于 0.5 的变量包括 YCP (传染病损失年龄比例)、YNCP (非传染病损失年龄比例)、P15P (15 岁以下人口比例)、TFR (妇女怀孕率)、NMN (每千名新生儿死亡数)、TBN (每 10 万人口结核病数)、PGR (人口增长率)、HER (卫生的外部资源占卫生总支出的百分比)、DR3 (每 10 万人口因伤死亡数)、PUP (城市人口比例)、IMR (一岁童 MCV 接种率)、P60P (60 岁以上人口比例)、I3R (一岁童 3 次乙肝接种率)、YIP (受伤损失年龄比例). 由于第 1 因子和各方面的卫生健康及人口等信息有关, 我们给第 1 因子取名为 "综合卫生因子".
- 与第 2 因子的相关系数 (绝对值) 大于 0.5 的变量包括 NNMP (人均护士助产士人数)、NDP (人均牙医人数)、NPP (人均内科医生人数)、PT (总人口). 由于第 2 因子和医务人员资源有关, 可以取名为 "医务人员资源因子".
- 与第 3 因子的相关系数 (绝对值) 大于 0.5 的变量只有 HGR (一般政府卫生支出占卫生总支出的百分比) 和 HPHP (私人健康支出占健康总支出的百分比), 由于该因子仅和卫生开支有关, 可以取名为 "卫生支出因子".
- 与第 4 因子的相关系数 (绝对值) 大于 0.5 的变量包括 DR2 (每 10 万人心血管死亡数)、DR4 (每 10 万人非传染病因死亡数), 由于第 4 因子和两种死亡原因有关, 取名为 "非传染病死亡因子".
- 与第 5 因子的相关系数 (绝对值) 大于 0.5 的变量只有 YIP (受伤损失年龄比例), 由于该因子仅和意外死亡有关, 取名为 "意外死亡因子".

可以看出, 前面独特性较强的 5 个变量没有一个在这些和因子相关的变量中, 因此, 这些因子不代表那些独特的变量.

审视了载荷之后, 实际上没有必要画出载荷图, 载荷图是为了直观展示. 由于很难画出一维载荷图, 因此, 一般都用二维图, 下面我们画出对于因子对 1 和 2、因子对 3 和 4、因子对 4 和 5 的三个载荷图, 作为对照, 画出未经旋转因子分析的相应因子对的三个载荷图及主成分分析相应主成分对的三个载荷图 (见图6.4). 下面是作图代码:

```
a=factanal(scale(w[,-1]),factors=5,rotation ="none")
b=princomp(scale(w[,-1]))

B=matrix(c(1:4,4,5),3,2,b=T)
TT=c("FA with rotation","FA without rotation","PCA")
A=list(a5$loadings,a$loadings,b$loadings)
for(k in 1:3)
  for (i in 1:3){
    plot(A[[k]][,B[i,]],xlim=c(-1,1),ylim = c(-1,1),type="n")
```

```
    text(A[[k]][,B[i,]],names(w[,-1]),cex=.6)
    abline(h=0,lty=2);abline(v=0,lty=2);title(TT[k])
}
```

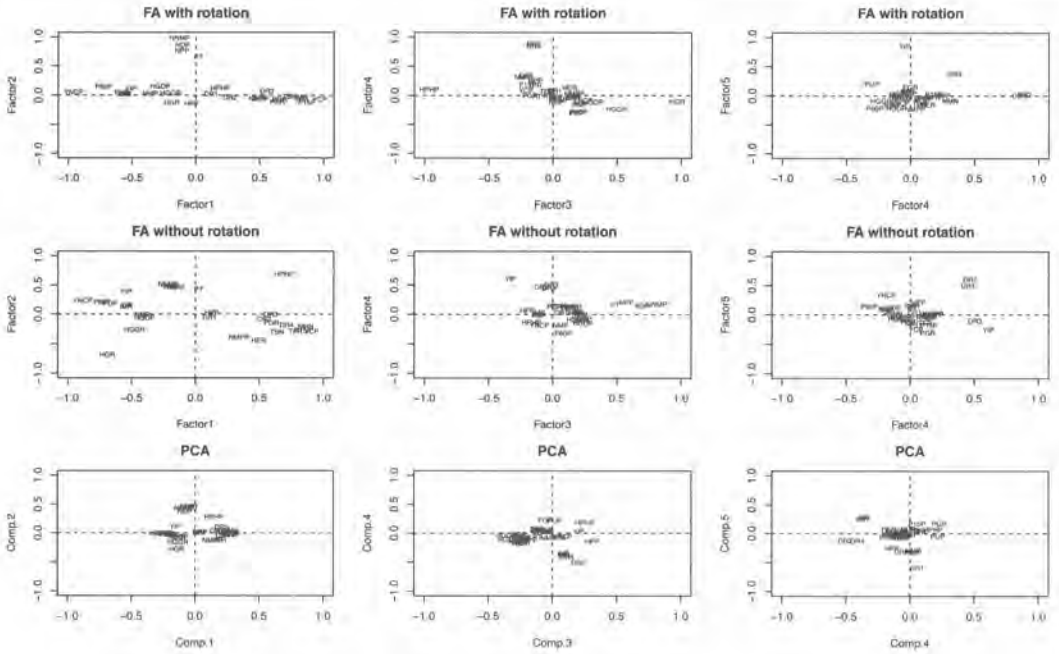

图 6.4 例6.1WHO 数据的因子分析 (第一行是旋转后的, 第二行是不旋转的) 和主成分分析 (第三行) 的载荷图, 每一行的三个图分别是关于因子 (成分) 对 1 和 2、3 和 4、4 和 5 的载荷图

由于图6.4的尺度是一样的, 集中于原点附近的点 (变量) 和相应的因子相关性小, 而远离原点的点 (变量) 和相应的因子相关性大. 第 1 因子和第 2 因子的载荷图中的点最分散, 说明它们所代表的变量更多.

计算因子得分, 并做出因子得分图

前面计算中已经隐含因子得分 (上面程序代码中函数 `factanal` 的选项表明这里用 `scores="regression"` 来得到因子得分), 由于国家太多, 我们不输出因子得分数值, 但可以知道下面的结果:

- 第 1 因子即所谓 "综合卫生因子" 得分最高 (最落后) 的五个国家为 Botswana、Mozambique、Swaziland、Angola、Lesotho, 而得分最低 (最先进) 的五个国家为 Georgia、Bulgaria、Singapore、Armenia、Bosnia 和 Herzegovina.
- 第 2 因子即所谓 "医务人员资源因子" 得分最高 (医务人员比例高) 的五个国家为 United States of America、India、China、Russia、Japan, 而得分最低的的五个国家为 San Marino、Malta、Andorra、Monaco、Iceland.
- 第 3 因子即所谓 "卫生支出因子" 得分最高 (卫生支出高) 的五个国家为 Marshall Islands、Tuvalu、Solomon Islands、Angola、Kiribati, 而得分最低 (卫生支出低) 的五个国家为 Guinea、Georgia、Myanmar、Pakistan、Lao People's Democratic Republic.
- 第 4 因子即所谓 "非传染病死亡因子" 得分最高 (死亡比例高) 的五个国家为 Nauru、

Turkmenistan、Afghanistan、Kazakhstan、Russia, 而得分最低 (死亡比例低) 的五个国家为 Japan、Panama、Monaco、Singapore、Switzerland.

- 第 5 因子即所谓 "意外死亡因子" 得分最高 (死亡比例高) 的五个国家为 Colombia、Venezuela、Iran (Islamic Republic of)、United Arab Emirates、Lithuania, 而得分最低 (死亡比例低) 的五个国家为 Georgia、Jamaica、Kiribati、Azerbaijan、Netherlands.

我们也可以画出因子得分图, 但因为国家太多, 不一定能看得清. 作为示意, 这里仅仅画出相对于因子 1 和 2、3 和 4 的两个因子得分图 (见图6.5), 代码如下:

```
layout(t(1:2))
for (i in 1:2){
  plot(a5$scores[,B[i,]],type="n",ylim=range(a5$scores[,B[i,2]]),
  xlim=range(a5$scores[,B[i,1]]+c(-.3,.3)), main="Factor Scores")
  abline(h=0);abline(v=0)
  text(a5$scores[,B[i,]],labels=w[,1],cex=.5)
}
```

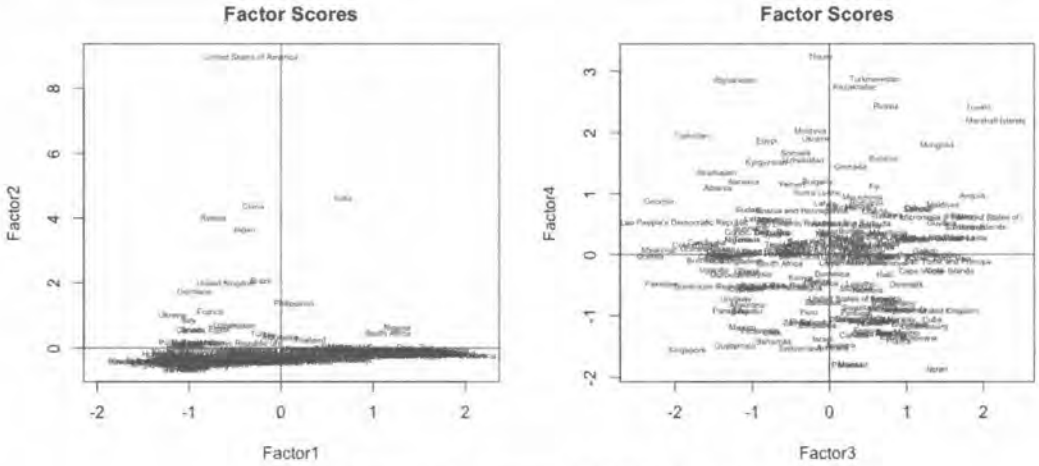

图 6.5 例6.1WHO 数据的因子得分图

6.2.2 例6.2豌豆数据的因子分析

初步因子分析, 不进行旋转

对于例6.2豌豆数据, 首先不进行旋转, 暂时选取 4 个因子, 读入数据和实行因子分析并输出载荷的 R 代码为:

```
w=read.csv("peas2.csv")
a=factanal(scale(w),factors=4,rotation ="none" , scores="regression")
a$loading
```

得到各个变量的特征值 (SS loadings) 为 11.071、3.148、0.625、0.384, 其中只有两个大于 1, 因此我们取两个因子做因子分析, 并且旋转.

选择两个因子, 旋转并做载荷图

下面选择两个因子, 旋转并做两个因子的载荷图 (见图6.6):

```
a2=factanal(scale(w),factors=2,rotation ="varimax",
   scores="regression")
a2$loadings
plot(a2$loadings[,1:2],type="n",xlim=c(-1.1,1.1))
text(a2$loadings[,1:2],labels=names(w),cex=.6)
abline(h=0,lty=2);abline(v=0,lty=2)
```

图6.6显示出变量 Skin 和 Colour3 有些 "不合群"(最靠近原点), 实际上, 变量 Skin 和 Colour3 的独特性是最大的 (大于 0.5 的变量仅有两个).

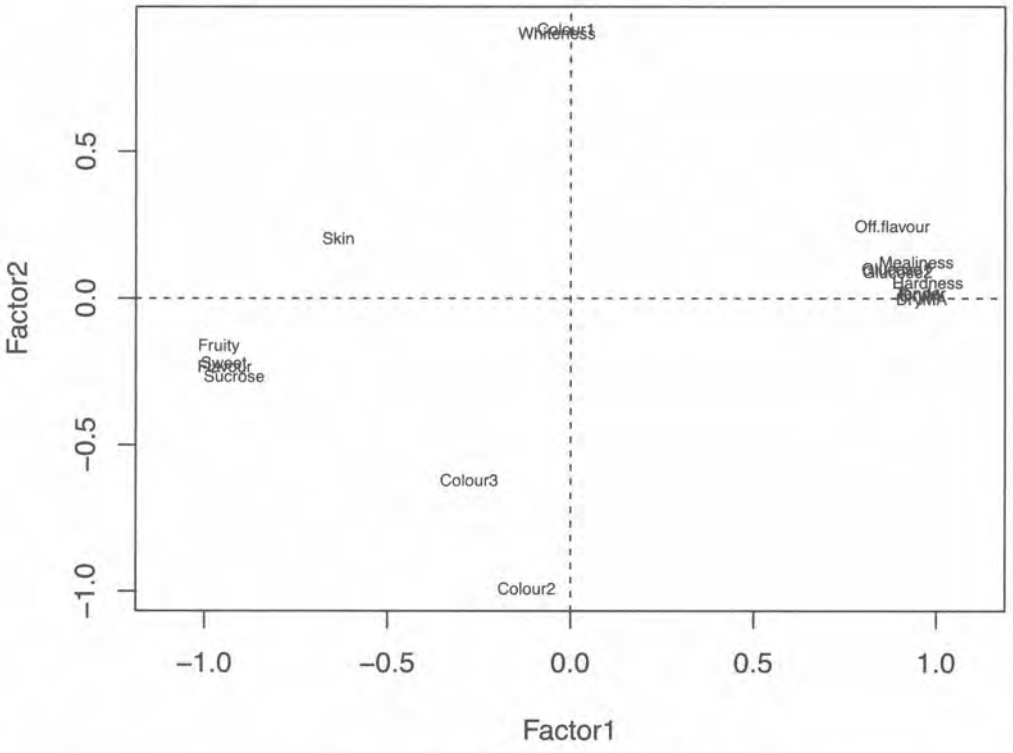

图 6.6 例6.2豌豆数据的因子分析的载荷图

输出的载荷为:

```
Loadings:
        Factor1 Factor2
Tender    0.956
DryM      0.955
DryMA     0.959
```

```
Sucrose      -0.919  -0.265
Glucose1      0.891
Glucose2      0.893
Flavour      -0.945  -0.233
Sweet        -0.948  -0.219
Fruity       -0.961  -0.165
Off.flavour   0.879   0.246
Mealiness     0.945   0.123
Hardness      0.976
Whiteness             0.904
Colour1               0.919
Colour2      -0.120  -0.990
Colour3      -0.277  -0.620
Skin         -0.634   0.204

                Factor1 Factor2
SS loadings      11.010   3.365
Proportion Var    0.648   0.198
Cumulative Var    0.648   0.846
```

除了三个颜色变量 (Whiteness、Colour2 和 Colour3) 之外, 大部分变量和第 1 因子的相关系数大于 0.85 (例外为 Skin 只有 -0.634), 而三个颜色变量 Whiteness、Colour2 和 Colour3 与第 2 因子的相关系数分别为 0.904, -0.990 和 -0.620. 因此, 可以称第 1 因子为 "综合性质因子", 而称第二因子为 "颜色因子".

6.2.3 例5.1教师数据的因子分析

对于例5.1的数据, 读入数据并随意选择 6 个因子做不旋转的因子分析, R 代码为:

```
w=read.csv("full.aaup.csv");u=w[,-c(1,3,4)]
a=factanal(scale(u[,-1]),factors=6,rotation = "none")
a$loadings
```

从输出中可以看出, 只有两个因子的特征值 (`SS loadings`) 大于 1, 其余很小 (小于 0.22). 我们于是做 2 个因子的因子分析并输出载荷, 代码如下:

```
a2=factanal(scale(u[,-1]),factors=2,rotation ="varimax",
   scores="regression")
a2$loadings
```

得到输出为:

```
Loadings:
     Factor1 Factor2
ASF    0.938   0.290
```

```
ASA1   0.947   0.201
ASA2   0.913   0.254
ASALL  0.946   0.276
ACF    0.950   0.257
ACA1   0.954   0.174
ACA2   0.918   0.236
ACALL  0.956   0.249
NF     0.353   0.895
NA1    0.319   0.909
NA2    0.296   0.904
NIN            0.515
NALL   0.328   0.942

                Factor1 Factor2
SS loadings       7.496   4.074
Proportion Var    0.577   0.313
Cumulative Var    0.577   0.890
```

从输出中可以看出, 和工资有关的变量 (全部以 "A" 开头) 仅和第 1 因子相关, 而凡是和人数有关的变量 (均以 "N" 开头) 仅和第 2 因子相关. 因此, 可以把第 1 因子称为 "工资因子", 而第 2 因子称为 "人数因子".

除此之外, 可以用代码 **a2$scores** 得到每个观测值在这两个因子坐标中的得分. 由于太长, 这里未提供输出. 但我们可以画出载荷图和加利福尼亚州学校的得分 (为了不至于太密, 仅选加利福尼亚州学校), 这两个图显示在图6.7中.

从图6.7(左图) 中可以看出, 与主成分分析中的第 1 主成分主导的情况不同, 在因子分析中, 第 1 因子 (横坐标) 越大, 各种平均工资越高, 而第 2 因子 (纵坐标) 越大, 各类人数越多. 从图6.7(右图) 中可以看出, 加利福尼亚大学洛杉矶分校人数最多, 而加利福尼亚理工学院人数最少, 但工资最高.

图6.7是用下面 R 代码产生的:

```
layout(t(1:2))
plot(a2$loadings[,1:2],type="n",ylim=c(-1.1,1.1),
xlim=c(-1.1,1.2),main = "FA loadings")
abline(h=0,lty=2);abline(v=0,lty=2)
text(a2$loadings[,1:2], labels=names(u[,-1]),cex=.7)
plot(a2$scores[w[,3]=="CA",1:2],type="n",
ylim=c(-1.9,4),xlim=c(-2,5.5), main="Factor Scores")
abline(h=0,lty=2);abline(v=0,lty=2)
text(a2$scores[w[,3]=="CA",1:2],labels=u[w[,3]=="CA",1],cex=.7)
```

变量 NIN 和第 1 因子不相关, 与第 2 因子相关系数最小 (0.515), 在载荷图 (见图6.7(左图)) 中, NIN 有些 "不合群". 实际上, 通过 **a2$uniquenesses** 可算出各个变量的独特性, 而变量 NIN 的独特性 (**a2$uniquenesses**) 最大, 比位居第二的 ACA2 大 7 倍多.

还可以把图6.7与该例的主成分分析的载荷图 (见图5.6) 做比较, 可以看出因子分析可以通过前两个因子把两类变量更加明确地分开.

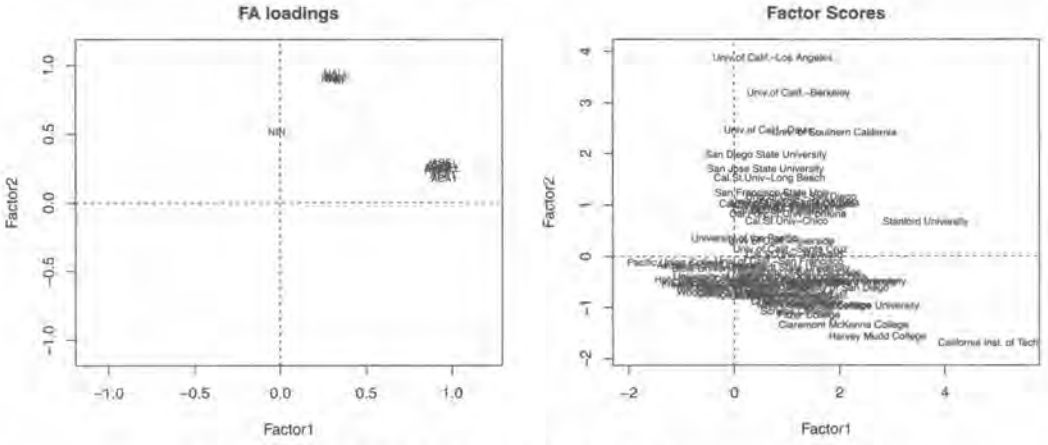

图 6.7 例5.1教师数据的因子分析的载荷图 (左) 及加利福尼亚州学校的得分 (右)

6.3 因子分析计算基于的原理 *

本节仅供感兴趣的读者参考.

在因子分析中, 我们需要得到 $\boldsymbol{Q}$ 和 $\boldsymbol{\Psi}$ 的估计 $\hat{\boldsymbol{Q}}$ 和 $\hat{\boldsymbol{\Psi}}$. 下面介绍几种估计方法.

6.3.1 主成分方法

对于实际数据, 样本协方差矩阵应该有

$$\boldsymbol{S} \approx \hat{\boldsymbol{Q}}\hat{\boldsymbol{Q}}^{\top} + \hat{\boldsymbol{\Psi}}.$$

按照主成分方法, 忽略 $\hat{\boldsymbol{\Psi}}$, 做谱分解

$$\boldsymbol{S} = \boldsymbol{C}\boldsymbol{D}\boldsymbol{C}^{\top} = \boldsymbol{C}\boldsymbol{D}^{1/2}\boldsymbol{D}^{1/2}\boldsymbol{C}^{\top} = (\boldsymbol{C}\boldsymbol{D}^{1/2})(\boldsymbol{C}\boldsymbol{D}^{1/2})^{\top},$$

然后令 $\hat{\boldsymbol{Q}}_{p\times m}$ 等于 $\boldsymbol{C}\boldsymbol{D}^{1/2}$ 的前 m 列, 即

$$\hat{\boldsymbol{Q}} = (\sqrt{d_1}\boldsymbol{c}_1, \sqrt{d_2}\boldsymbol{c}_2, \ldots, \sqrt{d_m}\boldsymbol{c}_m),$$

这里的 d_i 是对角矩阵 $\boldsymbol{D}$ 的对角线上第 i 个元素, $\boldsymbol{c}_k$ 是 $\boldsymbol{C}$ 的第 k 列. 此外, 令

$$\hat{\psi}_i = s_{ii} - \sum_{k=1}^{m} \hat{q}_{ik}^2,$$

这样就完成了估计. 如果基于相关阵 $\boldsymbol{R}$, 一切类似, 只不过要把 s_{ij} 换成 r_{ij} 而已.

按照假定, $\boldsymbol{S}$ 的对角线元素等于 $\hat{\boldsymbol{Q}}\hat{\boldsymbol{Q}}^{\top} + \hat{\boldsymbol{\Psi}}$ 的对角线元素. 对角线外的元素不必估计.

这个近似好不好呢? 考虑由主成分分解得的残差矩阵

$$\boldsymbol{S}-(\hat{\boldsymbol{Q}}\hat{\boldsymbol{Q}}^{\top}+\hat{\boldsymbol{\Psi}}).$$

解析上, 有

$$\sum_{ij}(\boldsymbol{S}-\hat{\boldsymbol{Q}}\hat{\boldsymbol{Q}}^{\top}-\hat{\boldsymbol{\Psi}})_{ij}^{2}\leqslant d_{m+1}^{2}+d_{m+2}^{2}+\cdots+d_{p}^{2},$$

这意味着被忽略的特征值越小, 造成的近似误差也越小. 选择因子数目的一个直观方式是考虑第 j 个因子占总样本方差的比例. 这个量一般等于 $d_j/\sum_{j=1}^{p}s_{jj}$(对 $\boldsymbol{S}$ 的因子分析) 或 d_j/p(对 $\boldsymbol{R}$ 的因子分析).

6.3.2 主因子方法

主因子方法又称主轴法, 主因子方法着重于相关矩阵 $\boldsymbol{R}$ 或协方差矩阵 $\boldsymbol{S}$ 的分解, 即考虑利用样本协方差和样本相关系数两种情况的估计:

$$\boldsymbol{S}-\hat{\Psi}\approx\hat{\boldsymbol{Q}}\hat{\boldsymbol{Q}}^{\top}\ \text{ 或者 }\ \boldsymbol{R}-\hat{\boldsymbol{\Psi}}\approx\hat{\boldsymbol{Q}}\hat{\boldsymbol{Q}}^{\top},$$

前面提到过, $\boldsymbol{R}$ 和 $\boldsymbol{S}$ 的谱分解产生不同的结果, 因而由主因子方法可能得到不同的估计. 这里的估计通常使用迭代来计算, 就上面左边使用样本相关矩阵 $\boldsymbol{R}$ 的情况来说, 首先给出一个 $\hat{h}_i^2$ 的初始值, 然后通过 $\hat{\psi}_i=r_{ii}-\hat{h}_i$ 得到 $\hat{\psi}_i$ 的值, 再利用前面主成分法通过谱分解 $(\boldsymbol{R}=\boldsymbol{C}\boldsymbol{D}\boldsymbol{C}^{\top}=(\boldsymbol{C}\boldsymbol{D}^{1/2})(\boldsymbol{C}\boldsymbol{D}^{1/2})^{\top})$ 得到的 $\boldsymbol{C}\boldsymbol{D}^{1/2}$ 的前 m 列作为 $\hat{\boldsymbol{Q}}$, 然后会得到新的 $\hat{h}_i^2$ 值, 如此下去直至收敛. 对于 $\boldsymbol{S}$ 或 $\boldsymbol{R}$, 通常 $\hat{h}_i^2$ 的初始值分别取

$$\hat{h}_i^2=1-\frac{1}{s^{ii}}\ \text{ 或者 }\ \hat{h}_i^2=1-\frac{1}{r^{ii}},$$

这里的 s^{ii} 和 r^{ii} 分别为 $\boldsymbol{S}^{-1}$ 和 $\boldsymbol{R}^{-1}$ 的第 i 个对角线元素.

当然, 利用这个方法有时不一定得到合适的收敛结果, 比如 $\hat{h}_i^2>1$, 这导致不准确的 $\hat{\psi}_i<0$. 这可能与程序的编写或者对数据很强的数学假定不合适有关.

由于主因子方法的计算依赖于共同度 h_j^2 的初始值 $\tilde{h}_j^2$, 建议两个传统的初始值为:

- $\tilde{h}_j^2$ 定义为 $\boldsymbol{X}_j$ 和不包含第 j 列的数据 $\boldsymbol{X}_{(-j)}$ 的多重相关系数的平方, 即 $\rho^2(\boldsymbol{X}_j,\boldsymbol{X}_{(-j)}\tilde{\beta})$, 而 $\hat{\beta}$ 为 $\boldsymbol{X}_j$ 对 $\boldsymbol{X}_{(-j)}$ 最小二乘回归的系数估计.
- $\tilde{h}_j^2=\max_{\ell\neq j}|r_{j\ell}|$.

6.3.3 最大似然法

最大似然法必然要假定样本的分布, 即 $\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_n$ 为来自

$$N_p(\boldsymbol{\mu},\boldsymbol{\Sigma})=N_p(\boldsymbol{\mu},\boldsymbol{Q}\boldsymbol{Q}^{\top}+\boldsymbol{\Psi})$$

的随机样本. 显然, 最大似然法可以得到估计 $\hat{\boldsymbol{Q}}$ 和 $\hat{\boldsymbol{\Psi}}$. 下面是解法梗概.

观测数据矩阵 $\boldsymbol{X}$ 的对数似然函数 ℓ 为

$$\begin{aligned}\ell(\boldsymbol{X};\boldsymbol{\mu},\boldsymbol{\Sigma}) &= -\frac{n}{2}\log|2\pi\boldsymbol{\Sigma}| - \frac{1}{2}\sum_{i=1}^{n}(x_i-\mu)\boldsymbol{\Sigma}^{-1}(x_i-\mu)^{\top}\\ &= -\frac{n}{2}\log|2\pi\boldsymbol{\Sigma}| - \frac{n}{2}\mathrm{tr}(\boldsymbol{\Sigma}^{-1}\boldsymbol{S}) - \frac{n}{2}(\bar{x}-\mu)\boldsymbol{\Sigma}^{-1}(\bar{x}-\mu)^{\top},\end{aligned}$$

用 $\hat{\boldsymbol{\mu}}=\bar{\boldsymbol{x}}$ 代替 $\boldsymbol{\mu}$, 则有

$$\ell(\boldsymbol{X};\hat{\boldsymbol{\mu}},\boldsymbol{\Sigma}) = -\frac{n}{2}\{\log|2\pi\boldsymbol{\Sigma}| + \mathrm{tr}(\boldsymbol{\Sigma}^{-1}\boldsymbol{S})\}.$$

用 $\boldsymbol{Q}\boldsymbol{Q}^{\top}+\boldsymbol{\Psi}$ 代替 $\boldsymbol{\Sigma}$, 得到

$$\ell(\boldsymbol{X};\hat{\boldsymbol{\mu}},\boldsymbol{Q},\boldsymbol{\Psi}) = -\frac{n}{2}\{\log|2\pi(\boldsymbol{Q}\boldsymbol{Q}^{\top}+\boldsymbol{\Psi})| + \mathrm{tr}((\boldsymbol{Q}\boldsymbol{Q}^{\top}+\boldsymbol{\Psi})^{-1}\boldsymbol{S})\},$$

由此可以求出需要的最大似然估计. 当然, 这个方程比较复杂, 必须用迭代算法.

6.3.4 因子个数的选择

有几种因子个数 m 的选择方法. 记 $\theta_j=\sum_{k=1}^{p}\hat{q}_{kj}^2$, 它代表了第 j 个因子对总样本方差的贡献. 因此, 特别是对主成分法估计, 选择 m 使得

$$\sum_{j=1}^{m}\theta_j/\mathrm{trace}(\boldsymbol{S}) \quad \text{或者} \quad \sum_{j=1}^{m}\theta_j/\mathrm{trace}(\boldsymbol{R})$$

大于某个认可的百分比 (诸如 80%、85% 等等).

另一种方法是把大于平均特征值的数目定为 m. 对于 $\boldsymbol{R}$, 平均特征值为 1, 而对于 $\boldsymbol{S}$, 平均特征值为 $\sum_{j=1}^{p}\theta_j/p$.

此外还有基于悬崖碎石图是否 "陡峭" 的几何检验和对 m 大小的检验 (下面的 $\boldsymbol{Q}$ 是 $p\times m$ 矩阵):

$$H_0:\boldsymbol{\Sigma}=\boldsymbol{Q}\boldsymbol{Q}^{\top}+\boldsymbol{\Psi} \Leftrightarrow \boldsymbol{\Sigma}\neq\boldsymbol{Q}\boldsymbol{Q}^{\top}+\boldsymbol{\Psi}.$$

几何检验由于与直观较接近, 结果较好. 直接根据崖底碎石图来得到结论可能更有道理, 但遗憾的是, 有些人更相信 p 值及主观选择的显著性水平, 而不是直观的图形. 对 m 大小的检验往往提供 m 的一个上界, 因而其通常得到多于可以接受的 m 值.

6.3.5 旋转

根据假定、代数的知识及计算方法, 可以得到对 q_{ij} 的矩阵 $\boldsymbol{Q}$ 的解, 但这个解不唯一, 解之间相差一个正交变换或刚体旋转. 因此, 方差分析还需要在这些解中寻找一个最能够解释数据的解. 这意味着因子分析多了旋转这个步骤. 在正交变换 (刚体旋转) $\boldsymbol{T}$ 下可以得到不同的解 $\boldsymbol{Q}^*=\boldsymbol{Q}\boldsymbol{T}$. 因此, 在实践中使用的旋转大多是正交旋转.

在 $m=2$ 时, 最常用的正交旋转是最大方差旋转, 它使得 $\hat{\boldsymbol{Q}}^*$ 的每一列的平方载荷的方差最大. 最大方差方法的思想就是寻求 $\boldsymbol{T}$, 使得载荷 $\hat{q}_{ij}^*$ 的方差和在 $\hat{\boldsymbol{Q}}^*$ 每一列中最大.

更确切地说, 定义 $\tilde{q}_{j\ell}^{*}=\hat{q}_{j\ell}^{*}/\hat{h}_{j}^{*}$, 方差最大准则选择 θ 使得

$$V=\frac{1}{p}\sum_{\ell}^{k}\left\{\sum_{j=1}^{p}(\tilde{q}_{j\ell}^{*})^{4}-\left[\frac{1}{p}\sum_{j=1}^{p}(\tilde{q}_{j\ell}^{*})^{2}\right]^{2}\right\}$$

最大.

6.3.6 因子得分

因子分析往往集中在估计载荷及共同度上. 逻辑上, 这是因为因子 $\boldsymbol{f}$ 被认为是信息的标准化随机源泉, 并且作为非特殊的共同因子被显示. 因子得分是对每个个体 $\boldsymbol{x}_i$ ($i=1,2,\ldots,n$) 的不可观测的随机向量 $\boldsymbol{f}_i$, 的估计. 实践上, 有结果非常类似的三个方法, 下面予以简单介绍.

通常最小二乘法

$$\boldsymbol{x_i}-\boldsymbol{\mu}=\boldsymbol{Q}\boldsymbol{f}_i+\boldsymbol{\epsilon}_i,\ i=1,2,\ldots n,$$

使得 (这里用 $\bar{\boldsymbol{x}}$ 代替 $\boldsymbol{\mu}$)

$$(\boldsymbol{x_i}-\bar{\boldsymbol{x}}-\boldsymbol{Q}\boldsymbol{f}_i)^{\top}(\boldsymbol{x_i}-\bar{\boldsymbol{x}}-\boldsymbol{Q}\boldsymbol{f}_i)$$

最小, 得到

$$\hat{\boldsymbol{f}}_i=\left(\hat{\boldsymbol{Q}}^{\top}\hat{\boldsymbol{Q}}\right)^{-1}\hat{\boldsymbol{Q}}^{\top}(\boldsymbol{x}_i-\bar{\boldsymbol{x}}).$$

加权最小二乘法

类似于通常最小二乘法, 只不过使得

$$(\boldsymbol{x_i}-\bar{\boldsymbol{x}}-\boldsymbol{Q}\boldsymbol{f}_i)^{\top}\boldsymbol{\Psi}^{-1}(\boldsymbol{x_i}-\bar{\boldsymbol{x}}-\boldsymbol{Q}\boldsymbol{f}_i)$$

最小, 得到

$$\hat{\boldsymbol{f}}_i=\left(\hat{\boldsymbol{Q}}^{\top}\boldsymbol{\Psi}^{-1}\hat{\boldsymbol{Q}}\right)^{-1}\hat{\boldsymbol{Q}}^{\top}\boldsymbol{\Psi}^{-1}(\boldsymbol{x}_i-\bar{\boldsymbol{x}}).$$

用最大似然估计来计算载荷时的回归方法

令 $\boldsymbol{x}_i$ 和 $\boldsymbol{f}_i$ 的联合分布为:

$$\begin{bmatrix}\boldsymbol{x}_i\\ \boldsymbol{f}_i\end{bmatrix}\sim N\left(\begin{bmatrix}\boldsymbol{\mu}\\ \boldsymbol{0}\end{bmatrix},\begin{bmatrix}\boldsymbol{QQ}^{\top}+\boldsymbol{\Psi} & \boldsymbol{Q}\\ \boldsymbol{Q}^{\top} & \boldsymbol{I}\end{bmatrix}\right),$$

由此, 条件期望 $E(\boldsymbol{f}_i|\boldsymbol{x}_i)=\boldsymbol{Q}^{\top}(\boldsymbol{QQ}^{\top}+\boldsymbol{\Psi})^{-1}(\boldsymbol{x}_i-\boldsymbol{\mu})$ 的形式导致估计

$$\hat{\boldsymbol{f}}_i=\hat{\boldsymbol{Q}}^{\top}(\hat{\boldsymbol{Q}}\hat{\boldsymbol{Q}}^{\top}+\hat{\boldsymbol{\Psi}})^{-1}(\boldsymbol{x}_i-\bar{\boldsymbol{x}}).$$

针对因子数 m 不一定合适的问题, 用 $\boldsymbol{S}$ 替代 $\hat{\boldsymbol{Q}}\hat{\boldsymbol{Q}}^{\top}+\hat{\boldsymbol{\Psi}}$ 会更稳定:

$$\hat{\boldsymbol{f}}_i=\hat{\boldsymbol{Q}}^{\top}\boldsymbol{S}^{-1}(\boldsymbol{x}_i-\bar{\boldsymbol{x}}).$$

如果因子被正交矩阵 $\boldsymbol{T}$ 所旋转, 因子得分也应做相应的旋转, 即

$$\hat{\boldsymbol{f}}_i^{*}=\boldsymbol{T}^{\top}\hat{\boldsymbol{f}}_i.$$

6.3.7 建议和评论

在实际因子分析模型的估计中, 由于可选择的太多, 而且没有一种可以说是绝对最优的, 因此, 可以根据具体问题和数据做探索, 以使数据的因子角度稳定. 下面是几点具体建议.

(1) 基于数据的相关结构或者特征值的崖底碎石图来确定因子数目.

(2) 实施几种 (包括旋转的) 现成方法, 比较各种方法得到的载荷、共同度以及因子得分.

(3) 如果结果显著不同, 基于因子得分核对离群点, 并考虑改变因子数目.

根据数据驱动的思维, 可以把样本分成训练集和测试集来做交叉验证. 在训练集样本中用选择的方法估计因子模型, 并利用得到的参数针对测试集预测因子得分. 这个预测的因子得分应该和只用测试集得到的因子得分类似. 这个稳定性准则还可以包括载荷和共同度.

虽然因子分析和主成分分析使用类似的数学工具 (谱分解、投影等), 但它们有着根本的不同. 虽然在协方差阵的秩为 $m(\ll p)$ 时, 主成分分析可以类似于 m 个因子的因子分析, 但在实践中几乎不可能会有奇异的协方差阵. 主成分分析和因子分析之间最大的区别在于模型的哲学.

因子分析假定了固定数目的共同因子的严格结构, 而主成分分析按照重要性的递减方式来确定 p 个因子. 主成分分析的最重要的成分 (因子) 使得投影的方差最大. 因子分析的最重要因子是 (在旋转后) 给出最大解释的那一个, 其最重要因子的方向有时类似于第一主成分的方向, 但通常不一样. 主成分分析按照方差下降的方式为变量做线性变换以减少数据集的维数, 而因子分析用有固定有限数量的隐变量 (因子) 对数据做线性变换. 因子分析的目标是找到载荷 $\boldsymbol{Q}$ 和特殊方差 $\boldsymbol{\Psi}$.

实践中, 主成分分析方法单一, 结果唯一且明确. 因子分析模型包含了各种数值方法, 其多样性产生了各种研究及结果, 也使得模型的识别与评价特别困难. 基于数据而不是模型的视角是应该提倡的. 模型的采用应该基于客观的数据, 而不是主观的假定.

6.4 本章计算的 Python 代码

在 Python 中, 我们用机器学习模块 `sklearn.decomposition` 所包含的因子分析方法 `FactorAnalysis()`. 这个方法的选项和通常统计中的方法很不同, 因此结果也不一样, 但还是可以看出其特点和共性 (比如, 通过图6.8和图6.7的比较).

由于本章的各个例子的代码类似, 我们只就例5.1教师数据用 Python 做因子分析, 其他例子请读者自己练习.

首先输入需要的模块:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import FactorAnalysis
from sklearn import preprocessing
```

利用下面的代码输入数据, 实现标准化 (利用 `preprocessing.scale()` 函数), 并做因子分析 (事先确定因子数目 $m = 2$), 计算并打印出事先因子载荷.

```
w=pd.read_csv("full.aaup.csv")
u=w[w.columns[4:]]
u_scale=preprocessing.scale(u)

fa=FactorAnalysis(tol=1e-8,max_iter=10000)
fa.n_components=2
fa.fit(np.array(u_scale))
fa.components_
print pd.DataFrame(fa.components_,columns=u.columns)
```

输出的因子载荷为:

```
        ASF      ASA1      ASA2     ASALL       ACF      ACA1      ACA2  \
0  0.580978  0.500926  0.539414  0.570559  0.554143  0.477202  0.523710
1  0.790862  0.828997  0.779008  0.803704  0.813813  0.844064  0.789364

      ACALL        NF       NA1       NA2       NIN      NALL
0  0.548206  0.964038  0.962803  0.951251  0.482013  0.999998
1  0.821205  0.038151  0.003608 -0.016972 -0.188513 -0.000343
```

下面的代码计算因子得分 (利用函数 `fit_transform()`):

```
CA=np.array(w[w.columns[2]]=='CA')
name_CA=np.array(w[w.columns[1]][CA])
sc1=fa.fit_transform(u_scale)[CA,:]
```

然后画出载荷图和加利福尼亚州学校的得分, 这两个图显示在图6.8中.

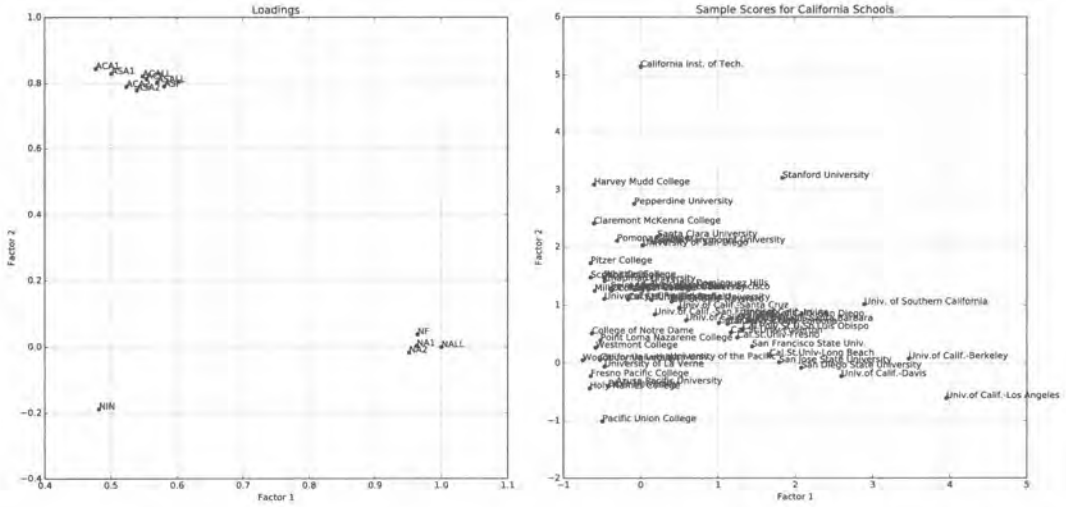

图 6.8 例5.1教师数据根据 Python 模块 sklearn 计算的因子分析的载荷图 (左) 及加利福尼亚州学校的得分 (右)

产生图6.8的代码如下:

```
fig=plt.figure(figsize=(10,8))
fig.add_subplot(121)
plt.scatter(fa.components_[0],fa.components_[1])
for i in range(len(u.columns)):
    plt.text(fa.components_[0,i],fa.components_[1,i],u.columns[i])
plt.grid(True)
plt.title('Loadings')
plt.xlabel('Factor 1')
plt.ylabel('Factor 2')
fig.add_subplot(122)
plt.scatter(sc1[:,0],sc1[:,1])
for i in range(sc1.shape[0]):
    plt.text(sc1[i,0],sc1[i,1],name_CA[i])
plt.title('Sample Scores for California Schools')
plt.xlabel('Factor 1')
plt.ylabel('Factor 2')
plt.grid(True)
plt.show()
```

6.5 习 题

1. 隐变量是假定的不可观测变量 (不能用一个可观测度量描述), 但可以通过一些可观测变量来体现. 例如, 一个人被描述成 "有效率", 可能是因为他总是按时完成任务, 及时回复他人的问询, 约会绝对准时, 能够尽早完成任务绝不拖后, 总是用最优的方式办事, 等等. 这个 "有效率" 就是一个隐变量, 是通过对许多有关现象的观测得到的.
   (1) 一个人 "是否幸福" 能够用回答 "是" 或 "不是" 反映吗? 试解释.

(2) 一个人 "是否随和" 是隐变量吗? 试解释.

(3) 一个孩子 "叛逆" 是隐变量吗? 试解释.

(4) 很多情绪变量是隐变量, 试举一些例子, 并且设计度量这些隐变量的方法.

(5) 举出一些不是隐变量的变量例子.

2. 和主成分分析比较, 因子分析在模型建立上使用了大量的主观假定.

(1) 请列举出这些对数据结构的主观假定.

(2) 在什么情况下, 这些假定会被违背?

(3) 这些假定中的任何一条能够被证明吗?

3. 和主成分分析比较, 因子分析在计算方法和理念上有很多人为的选择.

(1) 请列举出这些选择背后的考虑或隐含的假定.

(2) 不同的选择可能造成计算结果的不同, 有没有什么准则可以说明哪种选择最好?

(3) 因子个数的不同选择必定会使得不同的结果没有一致性. 比如, 在一种情况下得到的结果可能表明 m 应该是多少, 而在采取了这个结果的建议之后, 可能不会和前面的暗示一致. 你觉得这是数学问题或者工程问题吗? 是否应该以对实际问题本身的理解作为行动的出发点?

(4) 是不是任何数据都可以做出合适的因子分析?

(5) 是不是因子分析都可以得到合理的解释?

4. 用 Python 对例6.1WHO 数据做因子分析.

5. 用 Python 对例6.2豌豆数据做因子分析.

6. 对例6.2豌豆数据做主成分分析, 并且将其和该数据的因子分析做比较, 给出评论.

第 7 章　聚类分析

7.1　目的及基本概念

7.1.1 目的

通常, 我们希望通过一些变量 (特征) 将观测值个体分组. 比如, 根据学校的学生人数、师生比、教师工资、科研成果等将学校分组. 再比如, 根据气候、纬度、地理环境、土壤条件等对土地进行分组. 有时, 我们也需要通过观测值对变量 (特征) 分组. 如果数据的各个变量取值都是类似的数量型变量, 那么从原理和方法上对观测值分组与对变量分组没有多少区别, 它们可看作对数据矩阵的行分组或者列分组. 经典多元分析中的聚类大多讨论数量型变量的情况 (往往假定服从多元正态分布); 但当数据的变量有多种变量形式, 比如分类变量、定序变量及数量变量的混合, 对观测值分组和对变量分组就很不同了.

上面一段所用的 "分组" 属于常用词, 它在本章对应的术语为 "**聚类**"(clustering), 当然也可以用 "分类" 这样的常用词. 但在机器学习中, "**分类**"(classification) 作为术语实际上是指第4章属于**有指导学习**的分类或判别, 而聚类属于**无指导学习**, 下面具体说明.

(1) **作为有指导学习的分类:** 样本观测数据的某种类别区分为已知类别并且由 (称为因变量的) 目标变量标示, 利用这个因变量建立模型, 并利用这个模型对未知类别的观测值进行分类, 这称为**分类**, 分类在机器学习中属于**有指导学习**或**有监督学习**. 这种分类和回归问题类似, 仅有的不同是回归的因变量是数量变量 (quantitative variable), 而分类的因变量是定性变量 (qualitative variable)[1].

(2) **作为无指导学习的聚类:** 观测值不属于事先确定的类 (没有标明类别的目标变量), 只能利用目前所有的特征或变量来分类, 这种分类的过程称为**聚类**或**聚类分析** (cluster analysis), 聚类在机器学习中属于**无指导学习**或**无监督学习**. 聚类分析就是本章的内容.

在聚类分析中, 每一**类**可以称为**集群**、**组**、**簇**或**群**. 聚类的基本思想是根据观测值之间的距离远近, 把距离相近的观测值分为一类, 而且每一类与其他类尽可能保持较远的距离.

7.1.2 聚类和分类方法没有必然联系

一定要注意属于有指导学习的分类和属于无指导学习的聚类的目的并不一定相同, 它们各自涉及的类也不一定一样. 在分类中, 作为定性变量的因变量受到各个变量的影响并不均匀, 有些变量甚至可能和因变量无关; 在聚类中, 所有变量都被同等使用, 但没有其他 "因变量" 来 "指导". 比如一组数据既有学生各科的学习成绩, 又有学生的身高、体重等体检指标 (假定变量个数为 p), 每个变量都代表 p 维空间中的一维, 每个观测值为 p 维空间中的一个点. 那么:

[1]定性变量也称名义变量 (nominal variable) 或分类 (属性) 变量 (categorical variable).

- 如果用这些变量来聚类, 则所有 p 个坐标都同等地度量 p 维空间中各点之间的距离. 最终得到的类则是按照距离的远近来决定应该分多少类及哪些点属于一类的结果.
- 如果用这些变量来分类, 比如考虑关于学习成绩的分类, 那么该分类与身体指标关系不大; 如果考虑关于学生是否肥胖的分类, 则该分类和学习成绩关系不大. 一般来说, 分类一定有目的, 而且即使针对同一组数据, 不同目的所导致的最终分类结果也不会一样.

例 7.1. 说明分类和聚类区别的人造例子 (classclust.csv) 此例的目的是说明分类和聚类的区别. 数据中的 V1, V2 为自变量, 也就是用于聚类的变量, 而 V3 是用于分类指导的因变量 (有 0 和 1 两个水平). 代码包括读取数据. 我们既用决策树 (见4.6.1节) 做分类 (10 折交叉验证误差为零, 这里未给出交叉验证程序), 也用本章后面要讲的 K 均值聚类方法 (见7.3节) 做聚类, 最后利用分类结果和聚类结果生成两个图 (见图7.1的左右两图). 图7.1表示分类的类和由聚类得到的两类完全无关.

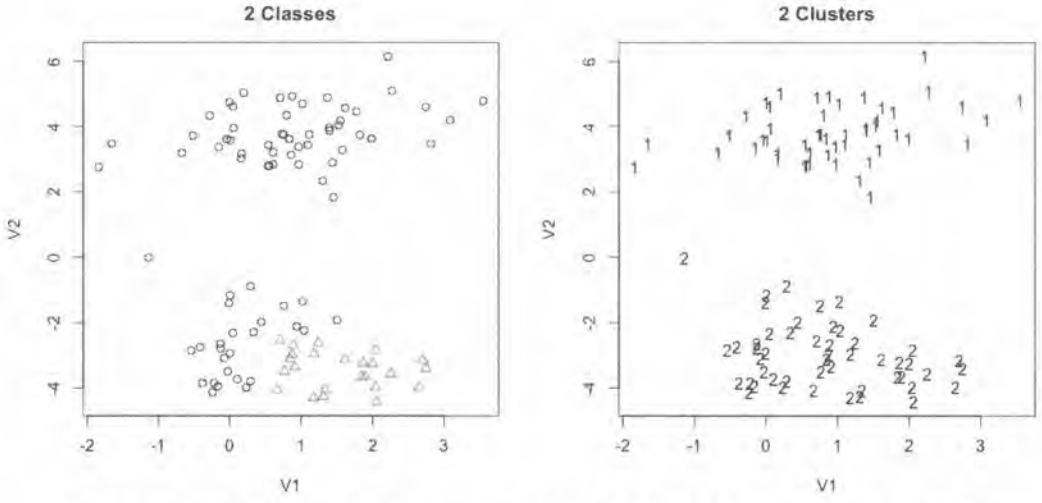

图 7.1 例7.1人造数据的分类 (左, 用圆圈和三角符号表示两类) 和聚类图 (右, 用 1 和 2 表示两类)

对例7.1读取数据、分类及计算距离、画图的代码为:

```
w=read.csv("classclust.csv")
library(rpart.plot)
(a=rpart(factor(V3)~.,w))
table(factor(w[,3]),predict(a,w,type="class"))
rpart.plot(a)
b=kmeans(w[,-3],2)
par(mar=c(4,4,3,2))
layout(t(1:2))
plot(w[,-3],pch=(w[,3]+1),col=w[,3]+1)
title("2 Classes")
plot(w[,-3],type="n");text(w[,-3],label=b$cluster)
title("2 Clusters")
```

7.1.3 距离

聚类分析的思想比较简单. 对于有 p 个变量的问题来说, 每个观测值都是 p 维空间的一个点. 因此, 如果的确存在若干不同的类别, 那么属于同一类的点在空间中的相互距离应该显著小于属于不同类的点之间的距离. 这就是聚类分析的基本思想.

距离或者不相似性 (dissimilarity) 在聚类中起着不可替代的作用. 没有距离就没有点或类的远近, 也就无从聚类. 但是, 针对距离有各种不同的定义 (和距离相反但作用相同的是相似性或亲密度). 下面介绍一些常用的关于连续的数量变量之间距离的概念.

1. 连续变量的点间距离

当然, 人们可能注意到, 在聚类之前, 首先需要定义两个点 $\boldsymbol{x} = (x_1, x_1, \ldots, x_p)^\top$ 与 $\boldsymbol{y} = (y_1, y_2, \ldots, y_p)^\top$ 之间的距离. 在连续变量之间, 有很多方式定义点间距离或者与其相反的度量亲密度 (或相似性), 常见的包括:

- 欧氏距离:
$$\|\boldsymbol{x} - \boldsymbol{y}\|_2 = \sqrt{\sum_{i=1}^{p}(x_i - y_i)^2}.$$
- 平方欧氏距离:
$$\|\boldsymbol{x} - \boldsymbol{y}\|_2^2 = \sum_{i=1}^{p}(x_i - y_i)^2.$$
- Manhattan 距离 (绝对距离):
$$\|\boldsymbol{x} - \boldsymbol{y}\|_1 = \sum_{i=1}^{p}|x_i - y_i|.$$
- Chebychev 距离 (最大距离):
$$\|\boldsymbol{x} - \boldsymbol{y}\|_\infty = \max_i |x_i - y_i|.$$
- Mahalanobis 距离:
$$\sqrt{(\boldsymbol{x} - \boldsymbol{y})^\top \boldsymbol{\Sigma} (\boldsymbol{x} - \boldsymbol{y})},$$
这里的 $\boldsymbol{\Sigma}$ 是协方差矩阵.
- Canberra 距离:
$$\sum_{i=1}^{p}\frac{|x_i - y_i|}{|x_i| + |y_i|},$$
- Jaccard 距离 (binary 距离的推广): $1 - J(\boldsymbol{x}, \boldsymbol{y})(x_i, y_i \geqslant 0, \forall i)$, 这里
$$J(\boldsymbol{x}, \boldsymbol{y}) = \frac{\sum_{i=1}^{p}\min(x_i, y_i)}{\max_i \min(x_i, y_i)}.$$

- Minkowski 距离:

$$\left(\sum_{i=1}^{p}(x_i-y_i)^q\right)^{\frac{1}{q}}.$$

- 夹角余弦 (一种亲密度):

$$\cos\theta_{\boldsymbol{xy}}=\frac{\sum_{i=1}^{p}x_iy_i}{\sqrt{\sum_{i=1}^{p}x_i^2\sum_{i=1}^{p}y_i^2}}.$$

- Pearson 线性相关距离: 定义为 1 减去 Pearson 线性相关系数 (一种亲密度):

$$1-r_{ij}=1-\frac{\sum_{i=1}^{p}(x_i-\bar{x})(y_i-\bar{y})}{\sqrt{\sum_{i=1}^{p}(x_i-\bar{x})^2\sum_{i=1}^{p}(y_i-\bar{y})^2}}.$$

- Eisen 余弦相关距离: (Eisen et al., 1998): Pearson 线性相关距离的特例, 这里用 0 代替 $\bar{x}$ 和 $\bar{y}$:

$$1-\left|\sum_{i=1}^{p}x_iy_i\right|\Big/\sqrt{\sum_{i=1}^{p}x_i^2\sum_{i=1}^{p}y_i^2}.$$

- Kendall 相关距离: 定义为 1 减去 Kendall 秩相关系数 (也称为 Kendall τ, 为一种亲密度), Kendall τ 是基于观测值中两组坐标同时增长或同时减少 (即 $(x_i-x_j)(y_i-y_j)>0$) 个数来计算的相关系数, Kendall 相关距离的具体计算步骤为, 按照大小顺序为 x 变量排序, 然后对每个 y_i 计算 $y_j>y_i$ (即协同 (concordant) 对) 的个数 c 及 $y_j<y_i$ (即不协同 (discordant) 对) 的个数 d, 于是 Kendall 相关距离定义为:

$$1-\frac{n_c-n_d}{\frac{1}{2}p(p-1)},$$

这里的 n_c 为协同对的总数, n_d 为不协同对的总数.

- Spearman 相关距离: 定义为 1 减去 Spearman 秩相关系数 (也称 Spearman ρ, 为一种亲密度), Spearman ρ 类似于 Pearson 相关系数, 只不过用观测值的秩代替坐标. Spearman 相关距离定义为:

$$1-\frac{\sum_{i=1}^{n}(x_i'-\bar{x'})(y_i'-\bar{y'})}{\sqrt{\sum_{i=1}^{n}(x_i'-\bar{x'})^2\sum_{i=1}^{n}(y_i'-\bar{y'})^2}},$$

式中, $x_i'=\mathrm{rank}(x_i)$ 及 $y_i'=\mathrm{rank}(y)$.

2. 混合变量的点间距离

在混合类型的变量情况下对观测值做聚类, 可考虑 Gower (1971) 的相似系数 (Gower's similarity coefficient). 对不同类型的变量聚类, 可考虑两个由程序包 `CluMix` 名字定义的方法:

(1) CluMix-ama 法 (关联度量法, association measures approach): 将不同的相似性度量组合的方法, 它基于水平 (范畴) 重新排序来度量多水平分类变量和其他变量之间的关联.

(2) CluMix-dcor 法 (距离相关法, distance correlation): 利用广义距离相关的概念所导出的相似性度量 (Lyons, 2013). 对有序和数量变量使用欧氏距离, 对定性变量使用离散距离.

对观测值聚类

记第 i 个观测值为 $\boldsymbol{x}_i = (x_{i1}, x_{i2}, ..., x_{ip})^\top$ $(i = 1, 2, \ldots, n)$. 记 $s_k(x_{ik}, x_{jk})$ 为基于第 k 个变量的观测值 $\boldsymbol{x}_i$ 和 $\boldsymbol{x}_j$ 之间的相似度, 定义为:

- 对定性变量:

$$s_k(x_{ik}, x_{jk}) = \begin{cases} 1, & \text{如果 } \boldsymbol{x}_{ik} \text{ 和 } \boldsymbol{x}_{jk} \text{ 相同;} \\ 0, & \text{如果 } \boldsymbol{x}_{ik} \text{ 和 } \boldsymbol{x}_{jk} \text{ 不同.} \end{cases}$$

- 对定量变量: $s_k(x_{ik}, x_{jk}) = 1 - |x_{ik} - x_{jk}|/R_k$, 这里的 R_k 是变量 k 的极差.
- 对定序变量:

$$s_k(x_{ik}, x_{jk}) = 1 - \frac{|r_k(x_{ik}) - r_k(x_{jk})|}{\max_m\{r_k(x_{mk}\} - \min_m\{r_k(x_{mk}\})},$$

这里的 $r_k(x_{mk})$ 是 x_{mk} 在所有 k 变量观测值中的秩.

定义观测值 $\boldsymbol{x}_i$ 和 $\boldsymbol{x}_j$ 之间的基于 p 个变量的相似度为:

$$s(\boldsymbol{x}_i, \boldsymbol{x}_j) = \frac{\sum_{k=1}^{p} s_k(x_{ik}, x_{jk})\delta(x_{ik}, x_{jk})w_k}{\sum_{k=1}^{p} \delta(x_{ik}, x_{jk})w_k},$$

这里的 $\delta(x_{ik}, x_{jk})$ 满足

$$\delta(x_{ik}, x_{jk}) = \begin{cases} 0, & \text{如果 } \boldsymbol{x}_i \text{ 或 } \boldsymbol{x}_j \text{ 有缺失值;} \\ 1, & \text{其他情况,} \end{cases}$$

权重 w_k 一般取 1, 但如果先验地认为变量 k 重要, 可以加更多的权重.

对变量聚类: CluMix-ama 法

这一部分的参考资料包括 Gonzalez and Nelson (1996) 与 Goktas and Isci (2011). 下面的指标用于度量变量 k 和变量 l 的相似度:

- 数量变量和数量 (或定序) 变量之间: 使用 Spearman 相关系数 (Spearman ρ) 的绝对

值, 即 $s_{kl}=|\rho_{\text{Spearman}}(x_k,x_l)|$.

- 定序变量和定序变量之间以及数量 (或定序) 变量和二分变量之间: 使用取绝对值的 Goodman-Kruskal γ 系数, 即 $s_{kl}=|(n_c-n_d)/(n_c+n_d)|$, 这里的 n_c 及 n_d 分别为变量 k 及变量 l 的协同对和不协同对的个数.
- 数量 (或定序) 变量 (X) 和分类变量 (Y) 之间: 根据数量 (或定序) 变量的情况把分类变量转换成定序变量, 为避免两个变量无关时过分运作的倾向, 在分类变量 Y 的每个水平对 X 做 Kruskal-Wallis 检验, 只有显著才能继续. 不显著时, 原始 Y 计算 Spearman ρ 或 Goodman-Kruskal γ 来显示各个水平的随机性 (Goodman and Kruskal, 1954).
- 分类 (或二分) 变量和分类 (或二分) 变量之间: 可以用二维表及 Pearson 列联表系数或 Cramer V 系数等, 更合适的方法可能和上面一样, 把分类变量转换成定序变量. 方法是使得二维表对角线频率最大化, 使用的方法包括计算 Goodman-Kruskal γ 及对原始变量做 χ^2 检验. 细节参见 Goodman (1979) 和 Siersma and Kreiner (2009).

对变量聚类: CluMix-dcor 法

这里的 "dcor" 为 Feuerverger (1993) 建议的距离协方差 (distance covariance) 的缩写, 距离协方差的概念还被推广到多元情况 (Székely et al., 2007; Székely and Rizzo, 2009). 下面是有关观测值间距离的定义:

- 如果第 k 个变量是分类变量:

$$d_k(x_{ik},x_{jk})=\begin{cases}0, & x_{ik}=x_{jk};\\ 1, & x_{ik}\neq x_{jk}.\end{cases}$$

- 如果第 k 个变量是数量变量:

$$d_k(x_{ik},x_{jk})=|x_{ik}-x_{jk}|.$$

- 如果第 k 个变量是定序变量, 令 $r_k(x_{ik})$ 表示观测值关于变量 k 的秩:

$$d_k(x_{ik},x_{jk})=|r_k(x_{ik})-r_k(x_{jk})|.$$

关于变量 k 的中心化距离定义为:

$$\tilde{d}_{ij}^{(k)}=d_k(x_{ik},x_{jk})-\frac{1}{n}\sum_{i=1}^{n}d_k(x_{ik},x_{jk})-\frac{1}{n}\sum_{j=1}^{n}d_k(x_{ik},x_{jk})+\frac{1}{n^2}\sum_{i=1}^{n}\sum_{j=1}^{n}d_k(x_{ik},x_{jk}).$$

样本形式的第 k 个和第 l 个变量之间的广义距离协方差 (Lyons, 2013) 定义为下面 $\widehat{U}_{kl}$ 的平方根:

$$\widehat{U}_{kl}=\frac{1}{n^2}\sum_{i=1}^{n}\sum_{j=1}^{n}\tilde{d}_{ij}^{(k)}\tilde{d}_{ij}^{(l)}.$$

由于 $\widehat{U}_{kl}$ 有偏, Székely and Rizzo (2013) 提出其修正版:

$$\widetilde{U}_{kl}=\frac{1}{n(n-3)}\left(\sum_{i=1}^{n}\sum_{j=1}^{n}\tilde{d}_{ij}^{(k*)}\tilde{d}_{ij}^{(l*)}-\frac{n}{n-2}\sum_{i=1}^{n}\tilde{d}_{ii}^{(k*)}\tilde{d}_{ii}^{(l*)}\right),$$

其中

$$\tilde{d}_{ii}^{(k*)}=\begin{cases}\dfrac{n}{n-1}\left(\tilde{d}_{ii}^{(k)}\right)-\dfrac{1}{n}d_k(x_{ik},x_{jk}), & i\neq j;\\[2ex] \dfrac{n}{n-1}\left[\dfrac{1}{n}\displaystyle\sum_{j=1}^{n}d_k(x_{ik},x_{jk})-\dfrac{1}{n^2}\sum_{i=1}^{n}\sum_{j=1}^{n}d_k(x_{ik},x_{jk})\right], & i=j.\end{cases}$$

标准化 $\widetilde{U}_{kl}$ 之后, 得到广义距离相关系数 (generalized distance correlation)

$$\widetilde{R}_{kl}=\operatorname{sign}\left(\widetilde{U}_{kl}\right)\sqrt{\frac{|\widetilde{U}_{kl}|}{\sqrt{|\widetilde{U}_{kk}||\widetilde{U}_{ll}|}}},$$

式中, $\widetilde{U}_{kk}$ 称为第 k 个变量的平方广义距离方差.

3. 类间距离

除了点之间的距离之外, 聚类分析的某些方法还需要定义类间距离, 由一个点组成的类是最基本的类; 如果每一类都由一个点组成, 那么点间的距离就是类间距离. 但是, 如果某一类包含不止一个点, 就要定义类间距离, 类间距离是基于点间距离定义的. 比如, 你可以把两类间最近点 (或最远点) 之间的距离作为类间距离, 也可以把两类间各个点间的平均距离作为两类的类间距离, 或者把两类各自的重心 (或中心) 间的距离作为两类间的距离, 等等. 类间距离也称为联系准则 (linkage criteria), 在分层抽样中需要应用类间距离的概念.

关于联系准则有一个简便的统一公式, 称为 Lance-Williams 公式 (Lance-Williams dissimilarity update formula). 下面我们把点也看成类, 统一把点 (类)i 与 j 之间的距离用 $D(i,j)$ 表示. 如果点 (或者类)i 与 j 聚合成一类 $i\cup j$, 记该类与类 (点)k 之间的距离为 $D(i\cup j,k)$. 该公式为:

$$D(i\cup j,k)=\alpha_i D(i,k)+\alpha_j D(j,k)+\beta D(i,j)+\gamma|D(i,k)-D(j,k)|,$$

这里, 参数 $\alpha_i,\alpha_j,\beta,\gamma$ 的不同取值的组合相应于不同的距离定义或联系准则, 再记 $|i|$ 为 i 中的元素数目, 记 $\boldsymbol{g}_i$(为一个 p 维向量) 为类 i 的中心或中心的初始值. 下面给出了这种公式及具体含义的对照:

- **最短距离法** (single-linkage). $\alpha_i=0.5,\beta=0,\gamma=-0.5$, 公式可化简为:

$$D(i\cup j,k)=\min(D(i,k),D(j,k)).$$

- **最长距离法** (complete-linkage). $\alpha_i = 0.5, \beta = 0, \gamma = 0.5$, 公式可化简为

$$D(i \cup j, k) = \max(D(i,k), D(j,k)).$$

- **类平均法** (mean or average linkage, UPGMA). $\alpha_i = |i|/(|i|+|j|), \beta = 0, \gamma = 0$.
- **加权类平均法** (McQuitty 法, WPGMA). $\alpha_i = 0.5, \beta = 0, \gamma = 0$.
- **中位数法** (WPGMC). $\alpha_i = 0.5, \beta = -0.25, \gamma = 0$; $\boldsymbol{g}_{i\cup j} = (\boldsymbol{g}_i + \boldsymbol{g}_j)/2$; $\boldsymbol{g}_i$ 和 $\boldsymbol{g}_j$ 的不相似度为 $\|\boldsymbol{g}_i - \boldsymbol{g}_j\|^2$.
- **中心法** (centroid linkage, UPGMC). $\alpha_i = |i|/(|i|+|j|), \beta = -|i||j|/(|i|+|j|)^2, \gamma = 0$. $\boldsymbol{g}_{i\cup j} = (|i|\boldsymbol{g}_i + |j|\boldsymbol{g}_j)/(|i|+|j|)$; $\boldsymbol{g}_i$ 和 $\boldsymbol{g}_j$ 的不相似度为 $\|\boldsymbol{g}_i - \boldsymbol{g}_j\|^2$.
- **Ward 法** (minimum variance, error sum of squares). $\alpha_i = (|i|+|k|)/(|i|+|j|+|k|), \beta = -|k|/(|i|+|j|+|k|), \gamma = 0$. $\boldsymbol{g}_{i\cup j} = (|i|\boldsymbol{g}_i + |j|\boldsymbol{g}_j)/(|i|+|j|)$; $\boldsymbol{g}_i$ 和 $\boldsymbol{g}_j$ 的不相似度为 $|i||j|\|\boldsymbol{g}_i - \boldsymbol{g}_j\|^2/(|i|+|j|)$.

Lance-Williams 公式在分层聚类的计算中使用起来很方便.

7.1.4 两个数据例子

首先看变量均为数量变量的例子.

例 7.2. 美国各州数据 (statex77.csv) 这是 R 自带 (程序包 datasets) 的名为 state.x77 的数据, 它是关于美国 50 个州的 8 个变量的数据. 变量包括 Population (1975 年 7 月 1 日估计的人口)、Income (1974 年人均收入)、Illiteracy (1970 年的文盲率)、Life Exp (1969 年到 1971 年的预期寿命)、Murder (1976 年每 10 万人的谋杀和非疏忽杀人数)、HS Grad (1970 年高中毕业百分比)、Frost (首府或大城市 1931 年到 1960 年温度降至零下的平均天数)、Area (土地面积, 单位为平方英里). 我们可以尝试用这组数据来看美国 50 个州的集群现象.

对于例7.2, 我们可以计算各种距离矩阵, 比如通过标准化的各种变量得到各州之间的欧式距离, 并且用图表示出来, 可用下面的代码 (显示距离的图在图7.2中), 这里用到的程序包为 factoextra.[2] 函数 get_dist() 中的选项 method 可以有 euclidean(默认)、maximum、manhattan、canberra、binary、minkowski、pearson、spearman 及 kendall. 用 R 自带的 dist 函数也可以产生距离矩阵, 但方法中没有 pearson、spearman 及 kendall 的选项.

```
library(factoextra)
d=get_dist(state.x77, method = "euclidean", stand = T)
fviz_dist(d,gradient = list(low = "#00AFBB", mid = "white",
    high = "#FC4E07"))
```

注意, 上面代码中的 d 不是 R 中的矩阵 (matrix 类), 而属于 dist 类, 必须用代码 as.matrix(d) 转换成矩阵来查看或输出具体数值, 这是 get_dist 和 dist 函数不同的地方.

[2] Alboukadel Kassambara and Fabian Mundt (2017). factoextra: Extract and Visualize the Results of Multivariate Data Analyses. R package version 1.0.5. https://CRAN.R-project.org/package=factoextra.

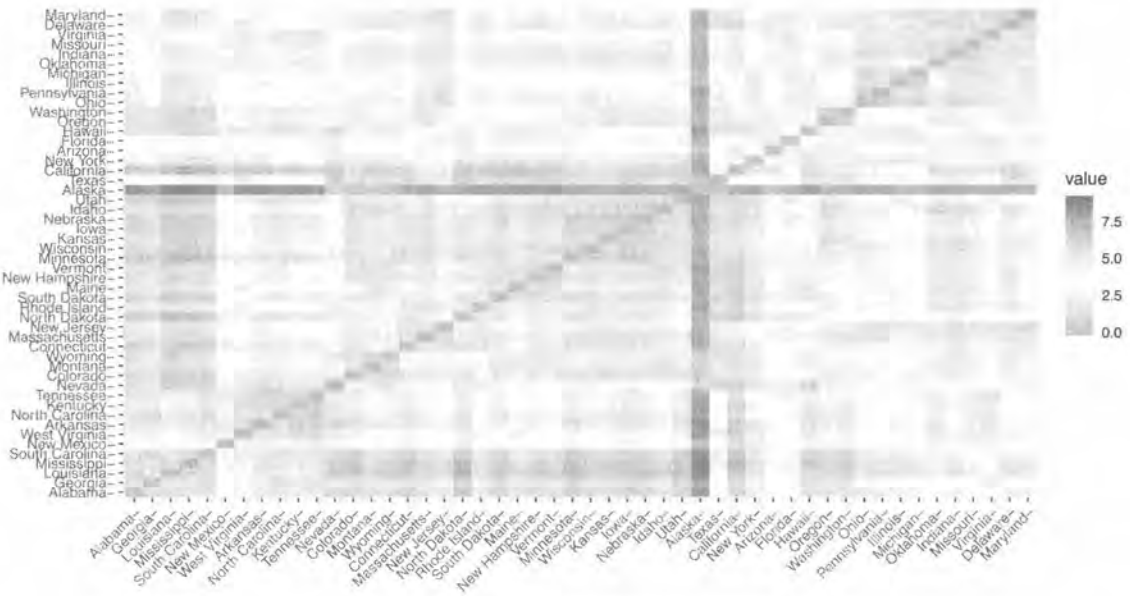

图 7.2 例7.2美国各州数据中各州在变量标准化之后的欧氏距离图

下面看一组多种变量混合的数据.

例 7.3. 花卉数据 (flower.csv) 这组数据由 Struyf et al. (1996) 提供, 由于其变量有数量变量、分类变量及定序变量, 所以被广泛引用, 包含并使用于诸如 `ClustOfVar`、`cluster` 及 `PCAmixdata` 的各程序包中. 该数据有 18 个观测值和 8 个变量, 这 8 个变量为: V1 表明植物冬天是否可以留在花园中 (分类变量, 哑元水平: "0", "1"), V2 表明植物是否需要处于阴影中 (分类变量, 哑元水平: "0", "1"), V3 表明是否块茎 (分类变量, 哑元水平: "0", "1"), V4 表明颜色 (分类变量, 哑元水平: "1"= 白色, "2"= 黄色, "3"= 粉红色, "4"= 红色, "5"= 蓝色), V5 表明土壤情况 (有序变量: "1"<"2"<"3", 代表 "干旱"<"正常"<"潮湿"), V6 表明偏好排名 (有序变量: "1"<"2"<···<"18"), V7 表明高度 (数量变量, 单位为厘米), V8 表明植株间距 (数量变量, 单位为厘米).

针对例7.3, 可以对观测值及变量分别画出距离图 (见图7.3), 图中还给出了后面要介绍的分层聚类的树状图. 这里, 我们使用了程序包 `CluMix`,[3] 代码如下:

```
library(CluMix)
distmap(flower, what="subjects")
distmap(flower, what="variables")
```

[3]M. Hummel, D. Edelmann and A. Kopp-Schneider (2017). CluMix: Clustering and Visualization of Mixed-Type Data. R package version 2.1. https://CRAN.R-project.org/package=CluMix.

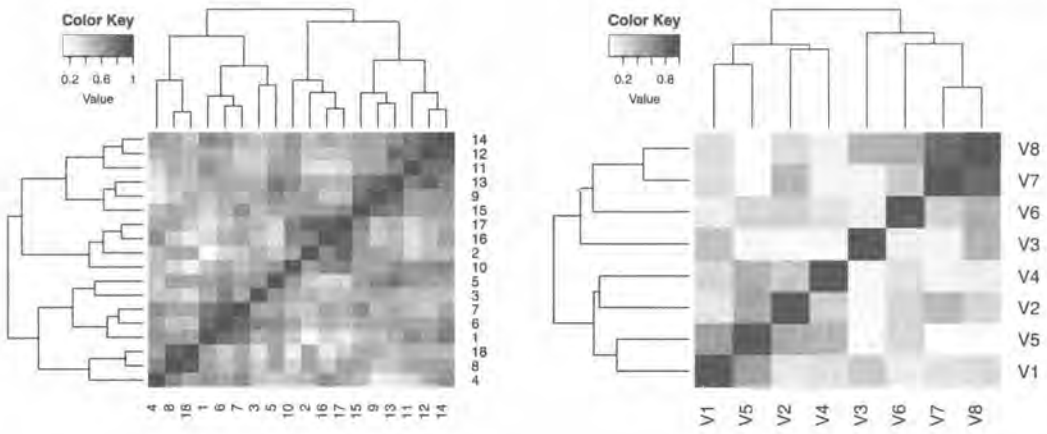

图 7.3 对例7.3花卉数据的观测值 (左) 及变量 (右) 分别画出距离图

7.1.5 集群倾向的度量

一个关于数据是否有集群倾向的度量为 Hopkins 统计量 (Lawson and Jurs, 1990), 它度量已给数据属于均匀分布 (完全随机) 的概率. 计算过程如下: 首先选择一定百分比 (比如 5%) 的 n 个数据点, 做上标记, 然后计算这些标记过的点与最近数据点的距离 (记为 $\{W_i\}$), 在数据空间中均匀地随机产生 n 个模拟的人工点, 再度量它们到最近的数据点的距离 (记为 $\{U_i\}$), 然后计算 Hopkins 统计量

$$H = \frac{\sum_{i=1}^{n} W_i}{\sum_{i=1}^{n} U_i + \sum_{i=1}^{n} W_i}, \tag{7.1.1}$$

如果数据的确是均匀分布的, 也就是说没有集群倾向, 那么 $\sum_{i=1}^{n} U_i$ 和 $\sum_{i=1}^{n} W_i$ 应该差不多, 这样 H 就应该接近 0.5; 如果有集群倾向, $\sum_{i=1}^{n} U_i$ 应该比 $\sum_{i=1}^{n} W_i$ 大得多, 即和 0.5 相比, H 应该小得多.

如果 $H < 0.25$, 则说明在显著性水平 10%时有集群倾向. 这里涉及的假设检验为:

H_0 : 数据集是均匀分布的 (没有集群) $\Leftrightarrow$ H_A : 数据集不是均匀分布的 (具有集群).

不过, 我们不必计算 p 值, 只要看 Hopkins 统计量是否足够小就行了.

注意: 目前的程序包 `clustertend`[4]中的函数 `hopkins()` 及程序包 `factoextra` 中的函数 `get_clust_tendency()` 在计算 Hopkins 统计量时是按照式 (7.1.1) 计算的, 但原始

[4]Luo YiLan and Zeng RuTong (2015). clustertend: Check the Clustering Tendency. R package version 1.4. https://CRAN.R-project.org/package=clustertend.

的 Lawson and Jurs (1990) 的 H 定义为:

$$H = \frac{\sum_{i=1}^{n} U_i}{\sum_{i=1}^{n} U_i + \sum_{i=1}^{n} W_i},$$

因此, 按照原始的定义, H 越接近 1, 则越有可能出现集群现象. 另外, 由于计算 Hopkins 统计量时的随机性, 如果使用不同的随机种子, 得到的统计量值也不尽相同 (如果使用函数 `get_clust_tendency()`, 则选项中有默认的随机种子, 可以更改; 而在函数 `hopkins()` 选项中没有随机种子, 必要的话自己通过函数 `set.seed()` 来设置).

我们来看例7.2美国各州数据集群倾向, 首先计算 Hopkins 统计量:

```
library(factoextra)
a=get_clust_tendency(scale(state.x77), nrow(state.x77)-1,
   graph = TRUE)
a$hopkins_stat;plot(a$plot)
```

得到 Hopkins 统计量的值为 0.2884082, 同时输出图形显示 (见图7.4的左图), 该图根据色谱显示出相似度 (similarity) 或不相似度 (dissimilarity).

作为对照, 我们生成与原数据范围有同样规模的均匀分布的数据, 并计算其 Hopkins 统计量:

```
rs=apply(state.x77, 2,function(x){runif(length(x), min(x), (max(x)))})
ra=get_clust_tendency(scale(rs), nrow(rs)-1, graph = TRUE)
ra$hopkins_stat;plot(ra$plot)
```

得到 Hopkins 统计量的值为 0.5169748, 同时输出图形显示 (见图7.4的右图), 该图显示出中间的色谱, 相对于 Hopkins 统计量在 0.5 附近.

无论从 Hopkins 统计量还是图7.4中, 都可以看出例7.2美国各州数据的确有集群倾向, 实行聚类分析可能会产生有意义的结果.

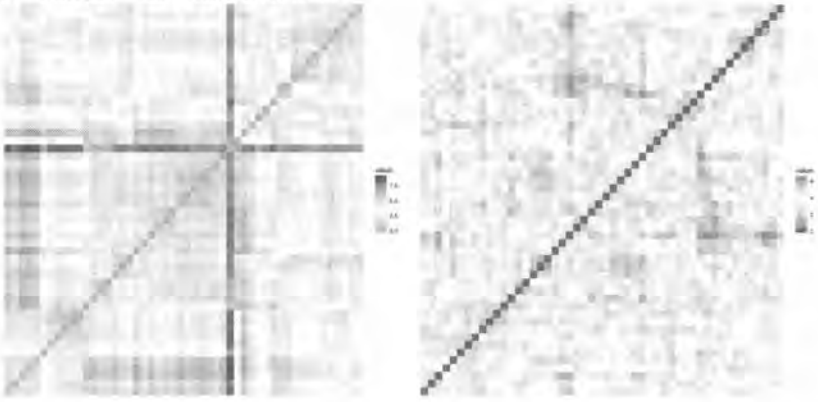

图 7.4 例7.2美国各州数据的集群倾向图形色谱显示, 左图为原始数据, 右图为随机数据

7.2 分层聚类

一种思想比较简单的聚类是分层聚类 (hierarchical clustering). 分层聚类有两种形式, 一种是聚合形式 (agglomerative), 其具体步骤如下:

(1) 首先把每一个观测值看成一类, 如果有 n 个观测值, 就有 n 类.
(2) 根据事先定义的点间距离, 把最近的两个点合并成一类, 这样就剩下 $n-1$ 类.
(3) 根据 Lance-Williams 公式, 根据选定的联系准则, 计算各个类间的距离, 再把最近的两类合并成一类, 剩下 $n-2$ 类.
(4) 如此下去, 每一步减少一类, 而且每次合并的两类之间的距离也比前一次更远.
(5) 最终在第 $n-1$ 次合并结束时, 只剩下包括全部观测值的一类.
(6) 一些软件可以针对上述合并过程 (包括类间距离) 画出树状图 (dendrogram), 人们可以根据直观或者某些准则来决定划分为几类.

聚合形式的分层聚类是我们要讨论的. 和后面要介绍的 K 均值聚类相比, 它的计算速度较慢.

另一种分层聚类形式是拆分形式 (divisive). 它和聚合形式相反, 一开始把所有的观测值看成一类, 然后将其分裂, 这种方法的计算速度比聚合形式还要慢, 我们不讨论这种形式的分层聚类.

7.2.1 对观测值分类: 例7.2美国各州数据

下面我们对例7.2美国各州数据的观测值 (州) 做分层聚类. 下面是代码 (这里, 函数 `dist` 默认的距离为欧氏距离, 选项 `method` 取 8 种不同的类间距离), 这里产生了 8 个分层聚类的树状图 (见图7.5).

```
m=c("ward.D", "ward.D2", "single", "complete",
  "average", "mcquitty", "median","centroid")
h=list();par(mfrow=c(2,4))
for(i in 1:length(m)){
  h[[i]]=hclust(dist(scale(state.x77)), method=m[i])
  plot(h[[i]] ,labels=rownames(state.x77),cex=.4,
    main=paste("Method: ",m[i]))
}
```

从图7.5可以看出, 根据 8 种不同类间距离所做的分层聚类的结果非常不同. 我们仅就第一种类间距离 (`ward.D`) 讨论, 使用该类间距离所产生的类比较均匀, 似乎分成 3 类比较合适, 如果分成 3 类, 则每一类的观测值名称 (州名) 及树状图 (见图7.6) 可以用下面代码得到 (利用前面计算输出的结果):

```
plot(h[[1]],cex=.8,main="Method: ward.D")
memb=identify(h[[1]], N=3)
```

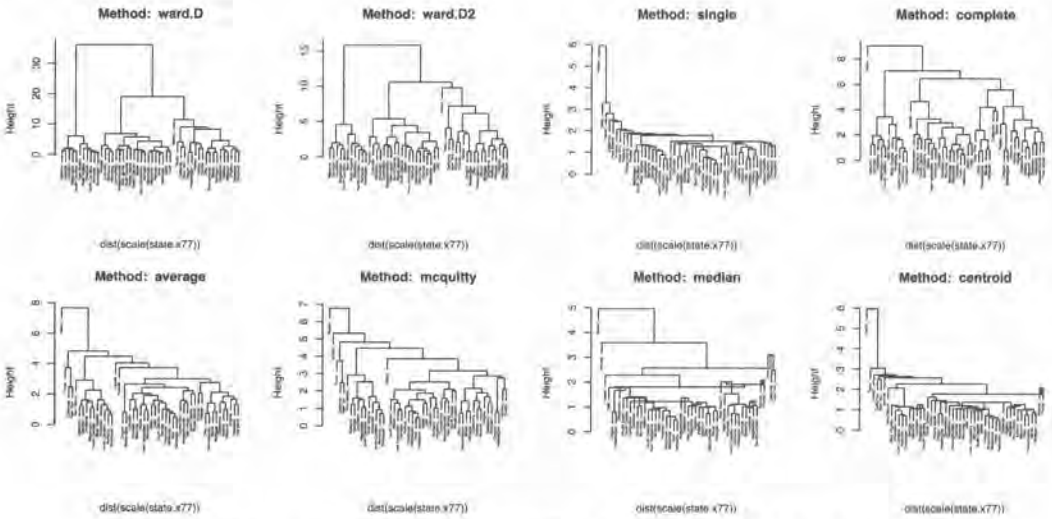

图 7.5 例7.2美国各州数据的相应于 8 种类间距离的分层聚类树状图

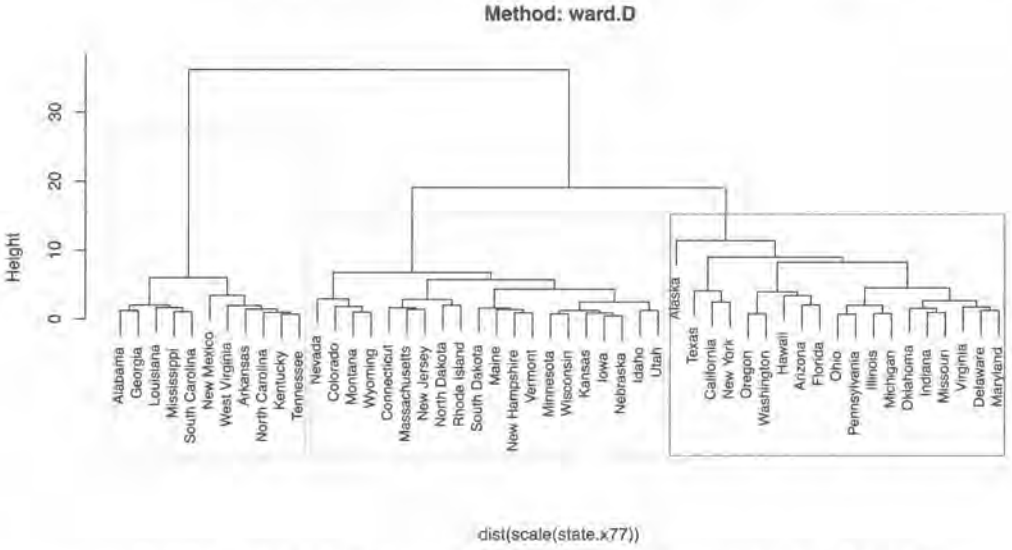

图 7.6 例7.2美国各州数据的相应于 1 种类间距离的分层聚类树状图及划分 3 类的结果

3 类州名的打印输出为:

```
> for(i in 1:3)
+   {print(paste("Class ", i));print(names(memb[[i]]))}
[1] "Class  1"
[1] "Alabama"        "Arkansas"       "Georgia"        "Kentucky"
[5] "Louisiana"      "Mississippi"    "New Mexico"  "North Carolina"
[9] "South Carolina" "Tennessee"      "West Virginia"
[1] "Class  2"
[1] "Colorado"      "Connecticut"   "Idaho"          "Iowa"
[5] "Kansas"        "Maine"         "Massachusetts" "Minnesota"
```

```
[9] "Montana"        "Nebraska"       "Nevada"         "New Hampshire"
[13] "New Jersey"    "North Dakota"   "Rhode Island"   "South Dakota"
[17] "Utah"          "Vermont"        "Wisconsin"      "Wyoming"
[1] "Class  3"
[1] "Alaska"        "Arizona"       "California"   "Delaware"   "Florida"
[6] "Hawaii"        "Illinois"      "Indiana"      "Maryland"   "Michigan"
[11] "Missouri"     "New York"      "Ohio"         "Oklahoma"     "Oregon"
[16] "Pennsylvania" "Texas"         "Virginia"      "Washington"
```

可能只有了解美国国情的人才有资格说这个聚类结果有没有道理.

下面, 我们再看例6.1的分层聚类, 使用下面代码得到 (见图7.7):

```
w=read.csv("who1.csv")
hw=hclust(dist(scale(w[,-1])))
plot(hw ,labels=w[,1],cex=.5,main="Who data hierarchical clustering")
```

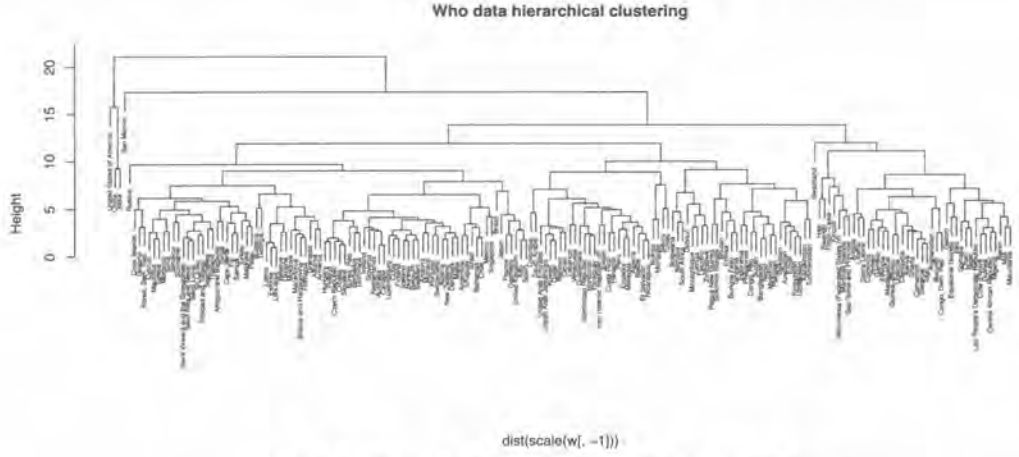

图 7.7 例6.1的相应于 1 种类间距离的分层聚类树状图

读者可能会认为图7.7很有意思. 我不想判断例6.1的数据应该分成多少类, 但这个图说明了不同的聚类数目都可能有道理. 更重要的是, 从图7.7可以看出哪些国家关于这个数据的变量相似, 这也是分层聚类结果的一个特点. 读者可以试试不同的类间距离得到的不同聚类结果, 看看如何解释更好.

通过重抽样来度量分层抽样中群体的重要性

R 中有一个程序包 pvclust[5], 用于评估分层聚类分析中的不确定性. 对于分层聚类中的每个分叉, 通过多尺度自助法重抽样来计算一个 "p 值". 一个集群的 p 值大小 $(0 < p < 1)$ 表示数据支持集群的强弱.

为了确定分层抽样聚类方法的类数, 程序包 pvclust 提供了关于集群重要性的两种类型的度量:

[5]Suzuki, R., and Shimodaira, H., (2015). pvclust: Hierarchical Clustering with P-Values via Multiscale Bootstrap Resampling. R package version 2.0-0. https://CRAN.R-project.org/package=pvclust.

(1) AU(近似无偏, approximately unbiased) p 值.

(2) BP(自助法概率, bootstrap probability) 值.

通过多尺度自助法重抽样来计算的 AU p 值比通过正态自助法重抽样计算的 BP 值的近似度更好. 具体细节参看 Sebastiani and Perls (2016).[6]

用下面代码中的函数 `pvclust` 做和前面 `hclust` 函数类似的聚类, 并且计算 AU p 值和 BP 值, 这些结果由函数 `plot.pvclust` (对于 `pvclust` 对象, 简写成 `plot`) 生成树状图. 函数 `pvrect` 找到具有相对较高/较低 p 值的集群, 在现有的树状图中显示这样的集群 (见图7.8). 图7.8的下面排列太密, 不易看清, 请读者尝试自己生成.

```
library(pvclust)
w=read.csv("who1.csv")
fit <- pvclust((scale(t(w[,-1]))), method.hclust="ward",
    method.dist="euclidean")
plot(fit,labels=w[,1],cex=.4)
pvrect(fit, alpha=.95) #选择AU p值大于0.95的群体
```

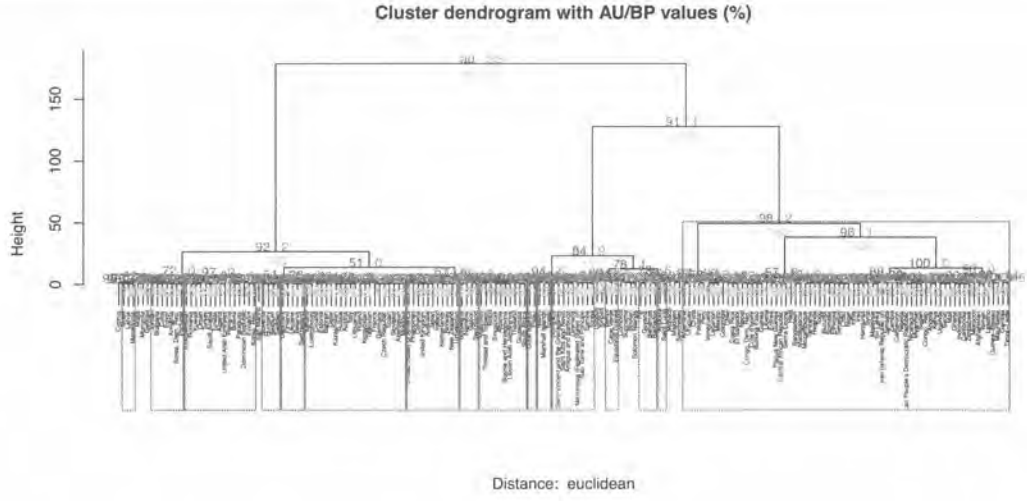

图 7.8 例6.1的包含 AU p 值和 BP 值的分层聚类树状图

7.2.2 对观测值聚类和对变量聚类: 例7.3花卉数据

例7.3的变量有三种类型: 定量变量、分类变量、定序变量, 因此无论对观测值还是对变量做分层聚类, 都不能简单地用 `hclust` 函数. 我们可以用程序包 `CluMix` 中的函数 `mix.heatmap` 同时产生对观测值和变量 (这里选的是距离相关方法) 的分层聚类 "热量图"(heatmap)(见图7.9):

```
library(CluMix)
mix.heatmap(flower, dist.variables.method="distcor",
    rowmar=7, legend.mat=TRUE)
```

[6]网页: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4976109/.

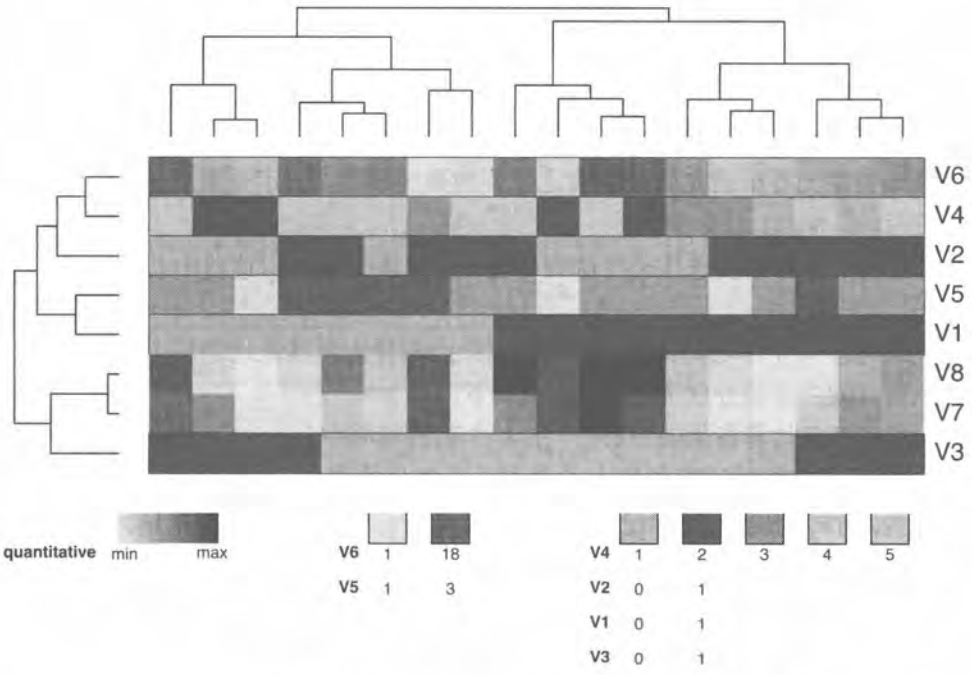

图 7.9 例7.3花卉数据的对观测值及变量的分层聚类树状图

使用下面代码产生例7.3的对观测值聚类 (左) 及两种对变量的分层聚类 (中、右) 树状图 (见图7.10):

```
layout(t(1:3))
plot(dendro.subjects(flower))
title("cluster for observations")
plot(dendro.variables(flower, method="associationMeasures"))
title("cluster for variables: association method")
plot(dendro.variables(flower, method="distcor"))
title("cluster for variables: distant correlation method")
```

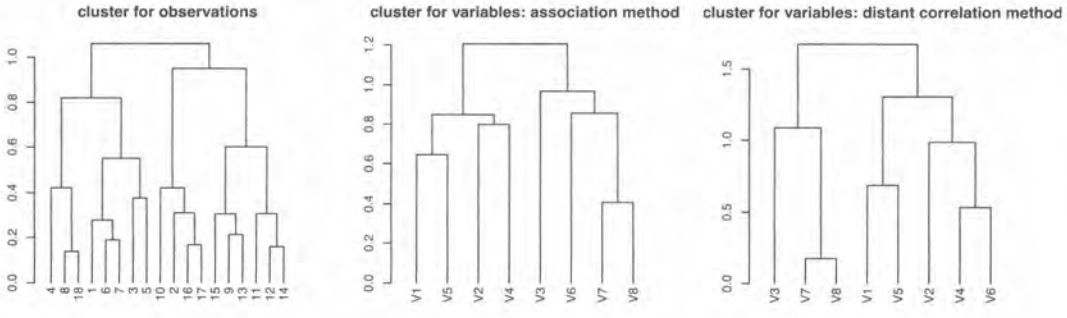

图 7.10 例7.3花卉数据的对观测值聚类 (左) 及两种对变量的分层聚类 (中、右) 树状图

7.3 K 均值聚类和基于密度的聚类

7.3.1 K 均值聚类的基本思想

K 均值聚类是最简单的无指导学习方法, 计算速度很快, 也称为快速聚类. 和分层聚类不同, K 均值聚类事先要根据先验经验确定需要的聚类数目. 假定要聚成 k 类, 那么根据下面步骤来进行:

(1) 在观测值空间中为每个类选定一个临时中心, 一共 k 个中心, 这些中心离得较远比较好.

(2) 计算每个点到这 k 个中心的距离, 然后把这个点划分为距其最近的那个中心所定义的类, 这样, 所有的点全部分成 k 类, 没有点剩下.

(3) 再在这 k 类中各自计算各类的中心, 产生 k 个新的临时中心.

(4) 重复上面的第2步和第3步, 直到这 k 个临时中心的位置收敛 (或到迭代次数限度).

更具体一些, 例如, 把第 q 类标记为 G_q $(q=1,2,\ldots,k)$, 记 G_q 的样本量为 n_q, 如果有 p 个变量, 总样本量为 $n=\sum_{q=1}^{k} n_q$, 观测值为 $n\times p$ 矩阵 (元素为 $\{x_{ij}\}$), 可以重新记属于 G_q 的观测值为 $x_{ij}^{(q)}(i=1,2,\ldots,n_q;\ j=1,2,\ldots,p)$. 这时, 第 q 类的中心坐标可以取均值 $\bar{\boldsymbol{x}}^{(q)}=(\bar{x}_1^{(q)},\bar{x}_2^{(q)},\ldots,\bar{x}_p^{(q)})^{\top}$, 这里 $\bar{x}_j^{(q)}=\frac{1}{n_q}\sum_{i=1}^{n_q} x_{ij}^{(q)}$, 再选择诸如平方欧氏距离, 则上述的 K 均值聚类可以在每个 G_q 中使得

$$\sum_{i=1}^{n_q}\sum_{j=1}^{p}(x_{ij}^{(q)}-\bar{x}_j^{(q)})^2$$

最小. 显然, k 个中心 (或临时中心) 都是 $k\times p$ 矩阵. 在软件中, 确定 k 之后, 初始的临时中心可以自己输入, 也可以由计算机生成.

下面用一个二维可视人造例子直观说明 K 均值聚类的过程.

例 7.4. 人造二维数据分类数据 (kmeansFig.csv) 该原始数据展示在图7.11的左上图中. 我们一开始随意设立三个中心 c_1、c_2、c_3, 然后根据上面的步骤进行迭代计算, 不断变化 c_1、c_2、c_3 的位置, 经过几次迭代, 划分的三类已经不再变化, 而且三个中心点也不再改变 (收敛). 可以看出, 在这个例子中, 一开始选的三个中心的位置实际上最终都属于同一类范围, 也就是说, 在这个例子中, 初始中心点的选择并不是重要的.

用函数 kmeans 计算例7.4人造数据的聚类. 读数据、使用 K 均值聚类 (取 $k=3$, 选项是 centers=3, 这时自动选取中心, 如果给选项 centers 以若干行坐标, 则其是若干初始中心的坐标)、在图中画出三类点及中心位置 c_1、c_2、c_3 的代码为 (这里不再给出图形, 因为会和图7.11的最后一个类似, 请读者自己产生):

```
w=read.csv("kmeansFig.csv")
a=kmeans(w,3) #确定分成3类
plot(w,pch=a$cluster)
text(a$center,expression(c[1],c[2],c[3]),col=2)
```

上面代码中的 a$cluster 对每个观测值给出了类别标记 (这里是 1、2、3), 而上面代码 a$center 给出了三个中心 ($c_1$、$c_2$、$c_3$) 的坐标.

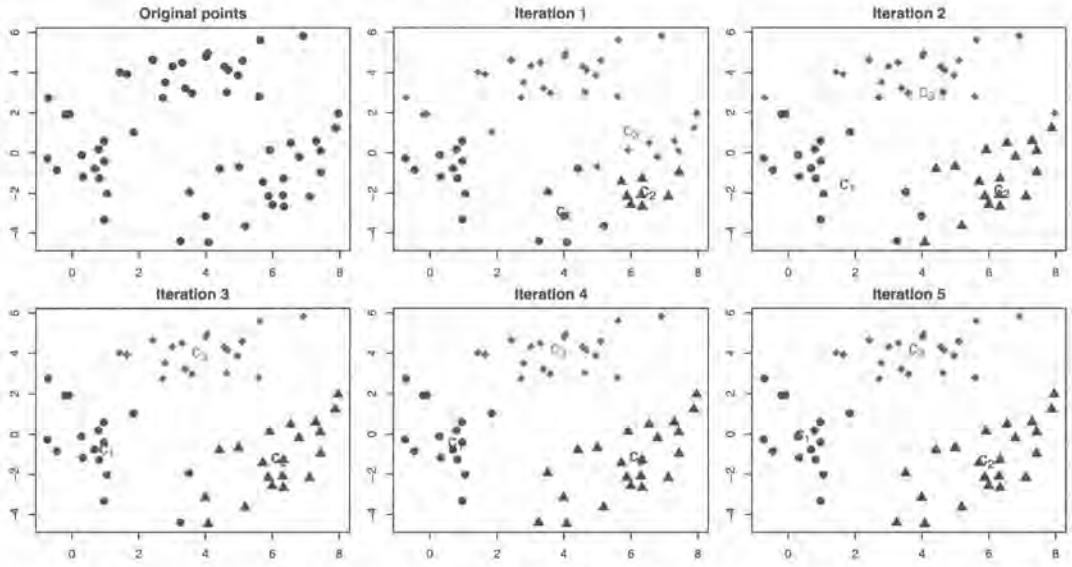

图 7.11 例7.4人造数据的 K 均值聚类 5 次迭代示意图 (每次迭代用实心的圆、菱形和三角形代表分的三类点)

分层 K 均值聚类

由于 K 均值聚类的结果有时对聚类中心的初始随机选择非常敏感. 有人提出**分层 K 均值聚类** (hierarchical K-means clustering), 它是分层聚类方法和 K 均值方法的混合方法解决方案. 其步骤为:

(1) 计算分层聚类.
(2) 通过分层聚类结果分成 k 类.
(3) 计算每一类的中心.
(4) 用上面得到的中心为 K 均值聚类的初始值, 实行 K 均值聚类.

分层 K 均值聚类方法很简单, 结果有时很好 (参看7.6.1节对例7.6种子数据分层 K 均值聚类的实践).

7.3.2 K 均值聚类中类别数目的确定

前面之所以把例7.4人造数据分成三类, 是因为数据简单, 只有二维, 从散点图中可以明显看见有三个集群 (有如作弊). 在一般情况下, k 应该取多少呢? 下面我们对此进行讨论.

通过类内平方和的变化判断类别数目. 假定数据有 i 个观测值及 p 个变量, 如果划分为 k 类, 第 q 类 ($q = 1, 2, \cdots, k$) 的数据记为 $x_{ij}^{(q)} (i = 1, 2, \ldots, n_q;\ j = 1, 2, \ldots, p)$, 这里 $\sum_{q=1}^{k} n_q = n$. 这时候, 第 q 类的平方和 (关于均值的均方误差) 为

$$SS^{(q)} = \sum_{i=1}^{n_q} \sum_{j=1}^{p} \left(x_{ij}^{(q)} - \bar{x}_j^{(q)} \right)^2,$$

这里 $\bar{x}_j^{(q)} = \frac{1}{n_q}\sum_{i=1}^{n_q} x_{ij}^q$. 于是类内总平方和为

$$WSS_k = \sum_{q=1}^{k} SS^{(q)} = \sum_{q=1}^{k}\sum_{i=1}^{n_q}\sum_{j=1}^{p}\left(x_{ij}^{(q)} - \bar{x}_j^{(q)}\right)^2.$$

一般来说, 如果类数增加, WSS_k 会逐渐减少, 如果类数增加到和观测值一样多, 每个点就是一类, TSS_k 就等于 0. 因此, 人们希望逐步增加 k, 看 WSS_k 的减少趋势, 如果减少到一定数量, WSS_k 减少变慢, 那么就停止增加 k. 这有些像在主成分分析中的崖底碎石图中寻找 L (如同胳膊肘) 形状的 "拐弯点", 因此也被称为肘方法或弯头法 (elbow method).

图7.12就是关于例7.4人造数据的 WSS_k 图, 这里 $k = 1, 2, \ldots, 10$. 从图7.12可以看出, 在 $k = 3$ 处对应拐弯点 (肘), 之后的 WSS_k 变化显著放缓. 按照这个图, 应该选择 3 类.

产生图7.12的代码如下:

```
w=read.csv("kmeansFig.csv")
wss=vector()
for (i in 1:10) wss[i-1] = kmeans(w,centers=i)$tot.withinss
plot(1:10, wss, type="b", xlab="K",
ylab=expression(WSS[K]),main="Within groups sum of squares")
```

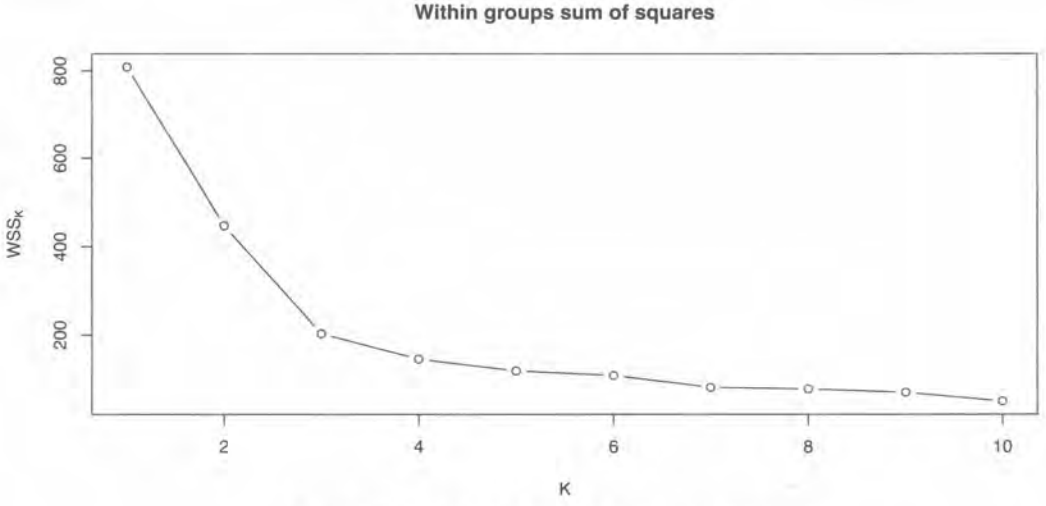

图 7.12 例7.4人造数据用不同的 k 所得到的类内平方和

也可以用程序包 factoextra 的函数 fviz_nbclust 来画出与图7.12类似的图形, 只不过它给出的类别数目为 4, 代码为 (请读者自己实现):

```
w=read.csv("kmeansFig.csv")
library(factoextra)
fviz_nbclust(w, kmeans, method = "wss") +
geom_vline(xintercept = 4, linetype = 2)+
labs(subtitle = "Elbow method")
```

我们在后面还将介绍多种聚类数目的选择方法.

7.3.3 基于密度聚类的思想 *

K 均值聚类适用于各类聚集于若干椭球 (椭圆) 形状的凸形区域中的数据, 而对于其他情况, 则可能失效. 请看下面一个演示性人工例子.

例 7.5. 基于密度聚类的人造二维数据 (3g1.csv) 这组二维数据显示在图7.13(左) 中, 看上去可以分成三类: 左下同心椭圆的心为一类, 外面的 (椭) 圆环为一类, 右上边的点云一类. 通过类内平方和的变化判断类别数目的图显示在图7.13(右) 中, 该图显示分成两类比较合适.

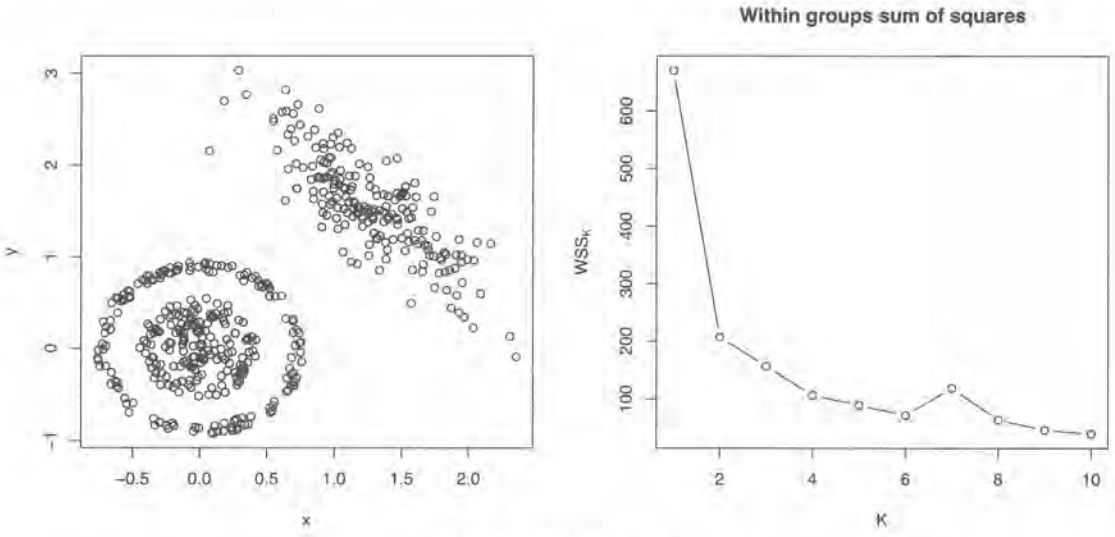

图 7.13 例7.5数据点图

基于密度的聚类 (density-based spatial clustering), 也称为 DBSCAN (基于密度的聚类算法, density-based spatial clustering and application with noise), 是一种基于密度的聚类算法 (Ester et al., 1996), 可以用于识别包含噪声和异常值的数据集中的任何形状的聚类.

基于密度的聚类不仅考察数据空间中的密集集群区域, 也考察不同群之间的低密度空隙 (包括其中的 "离群点" 或 "噪声"). DBSCAN 要确定一个集群的每个点在其给定半径的邻域内所必须包含的最少点数.

诸如 K 均值聚类以及将要介绍的 pam 聚类方法和分层聚类只适用于寻找椭球形或凸形集群. 也就是说, 它们只适用于结构紧凑、分离良好的集群. 当然, 所有聚类都会受到数据中存在的噪声和异常值的严重影响.

DBSCAN 必须理解若干个术语, 假定数据集 $D \in \mathbb{R}^p$ 有 n 个观测值, p 和 q 两点之间距离为 $d(p,q)$, 则定义:

(1) 点 p 的ϵ **邻域**定义为: $NEps(p) = \{q \in D | d(p,q) \leqslant \epsilon\}$.

(2) 关于 ϵ 和 $Minpts$(参数: 可到达的最少点数), 点 p 可以从点 q **直接密度可达** (directly density-reachable), 即 $p \in NEps(q)$ 以及$|NEps(q)| \geqslant Minpts$.

(3) 关于 ϵ 和 $Minpts$, 点 p 可以从点 q **密度可达** (density-reachable) 定义为: 如果存在一列点 $p_1, p_2, \ldots, p_n, p_1 = q, p_n = p$, 使得 p_{i+1} 为从 p_i 直接密度可达.

(4) 关于 ϵ 和 $Minpts$, 点 p 可以从点 q **密度连接** (density-connected) 定义为: 如果存在一个点 o, 使得 p 和 q 两点均为关于 ϵ 和 $Minpts$ 从点 o 密度可达.

(5) 关于 ϵ 和 $Minpts$, 集群 C 是 D 的一个非空子集, 满足下面两个条件:

(a) 对于任意 p 和 q, 如果 $p \in C$ 且 q 是从 p 密度可达的 (关于 ϵ 和 $Minpts$), 那么 $q \in C$, 称为**最大性**.

(b) 对于任意 $p, q \in C$, 点 p 可以从点 q **密度连接** (关于 ϵ 和 $Minpts$), 称为**连接性**.

(6) 如果 $G_1, G_2, \ldots, G_k$ 为数据 D 关于 ϵ 和 $Minpts_i (i = 1, 2, \ldots, k)$ 的类. **噪声** (noise) 定义为数据 D 中不属于任何 G_i 的点集 $(\{p \in D | p \notin G_i, \forall i\})$.

(7) 如果 $d(p, q) \leqslant \epsilon$ 且 q 是核心点 (核心点指聚类的初始点) 以及 $|NEps(p)| < Minpts$, 则 p 是一个边界点.

基于密度的聚类被定义为一组密度连接的点. DBSCAN 的工作原理为:

- 对于每个点 p, 计算 p 和其他点之间的距离. 在起点 p 的 ϵ 邻域内找到所有相邻点. 邻居数大于或等于 $Minpts$ 的每个点被标记为核心点或被访问过.
- 对于每个核心点, 如果尚未分配给群集, 则创建一个新的群集. 递归地查找所有密度连接点, 并将它们分配到与核心点相同的群集.
- 迭代数据集中剩余的未访问点.

那些不属于任何聚类的点被视为离群值或噪声.

下面对例7.5使用 DBSCAN 做聚类. 我们用程序包 `fpc`[7]中的函数 `dbscan`. 图7.14的上面三个图是不同 k 值 $(k = 2, 3, 4)$ 的 K 均值聚类, 而下面三个图是不同 $Minpts$ 值 $(Minpts = 1, 5, 7)$ 的 DBSCAN 聚类结果.

从图7.14上面三个图可以看出, K 均值聚类最好分成 2 类, 把左下的 "靶形" 图形组合分成一类, 右上的椭圆点云分为另一类; K 均值聚类分 3 类和 4 类的结果都没有意义.

在图7.14下面的三个图中, 当 $Minpts = 1$ 时, 预想的三类基本都识别了, 但在右上的点云周围有太多的 (大约 7 个)"异常点"; 当 $Minpts = 5$ 时, 聚类比较理想 (刚好分成 3 类); 而当 $Minpts = 7$ 时, 分成 5 类, 把椭圆环分成了 3 类. 因此, 在使用 DBSCAN 时, 参数的选择是很重要的, 对于简单的例7.5二维数据, 参数选择比较简单, 但对于多维的复杂数据, 问题就没有这么简单了. 当然, DBSCAN 聚类有很多推广, 比如, 参见 Liu et al. (2007)、Tsai and Liu (2006).

产生图7.14的代码为:

```
w=read.csv("3g1.csv")
library("fpc")
set.seed(1010)
par(mfrow=c(2,3))
for(i in c(2,3,4)){
  km <- kmeans(w, i)
```

[7]Christian Hennig (2015). fpc: Flexible Procedures for Clustering. R package version 2.1-10. https://CRAN.R-project.org/package=fpc.

```
  plot(w,pch=km$cluster,col=km$cluster)
  title(paste("k-means: k=", i))
}
for(i in c(1,5,7)){
  db <- fpc::dbscan(w, eps = 0.2, MinPts =i)
  plot(w,pch=db$cluster,col=db$cluster)
  title(paste("DBSCAN: Minpts=", i))
}
```

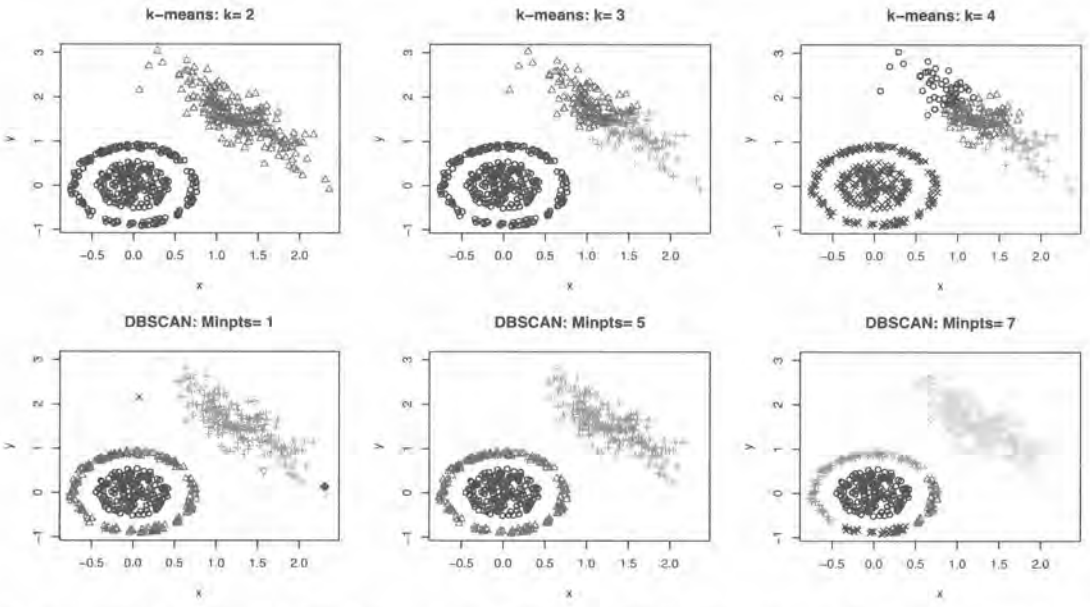

图 7.14 例7.5数据聚类：对于不同 k 值 $(k=2,3,4)$ 的 K 均值聚类及不同 $Minpts$ 值 $(Minpts=1,5,7)$ 的 DBSCAN 聚类结果，不同的类用不同的符号表示

7.4 基于模型的聚类 *

上面介绍的几种聚类分析对分布没有非常强的分布假定，最多希望各个类的点云是凸集，因此可以说是基于直观和比较随意的方法. 这里介绍的**基于模型的聚类** (model-based clustering) 则对数据的分布有数学假定.

基于模型的聚类假定数据分布为两个或更多的分量的混合，用分布密度的加权组合来描述. 通常假定为若干个正态分布的混合，也就是说，数据满足多元正态分布. 混合中的每个分量为一个集群.

7.4.1 直观描述

现直观地查看例7.4人造数据的二维数据，图7.15显示了例7.4数据点云的非参数密度估计示意图. 图中显示了大约 3 个分量，因此可以大致看成 3 个分量的分布组合.

产生图7.15的代码为:

```
w=read.csv("kmeansFig.csv")
library(KernSmooth)
est=bkde2D(w, bandwidth=rep(1,2)*.6)
contour(est$x1, est$x2, est$fhat, col = "blue",xlab="x",ylab="y")
points(w, col = "red", pch = 16)
```

当然, 图7.15并没有显示出二元正态分布的形态, 而且没有理由假定任何数据属于多元正态分布. 但由于估计多元正态分布的各个参数比估计其他分布的参数在数学及计算上要容易, 因此, 为方便起见, 人们喜欢用正态分布来近似数据的分布. 在基于模型的聚类方法中, 参数的估计很重要. 下面介绍参数估计的 E-M 算法及计算机实现.

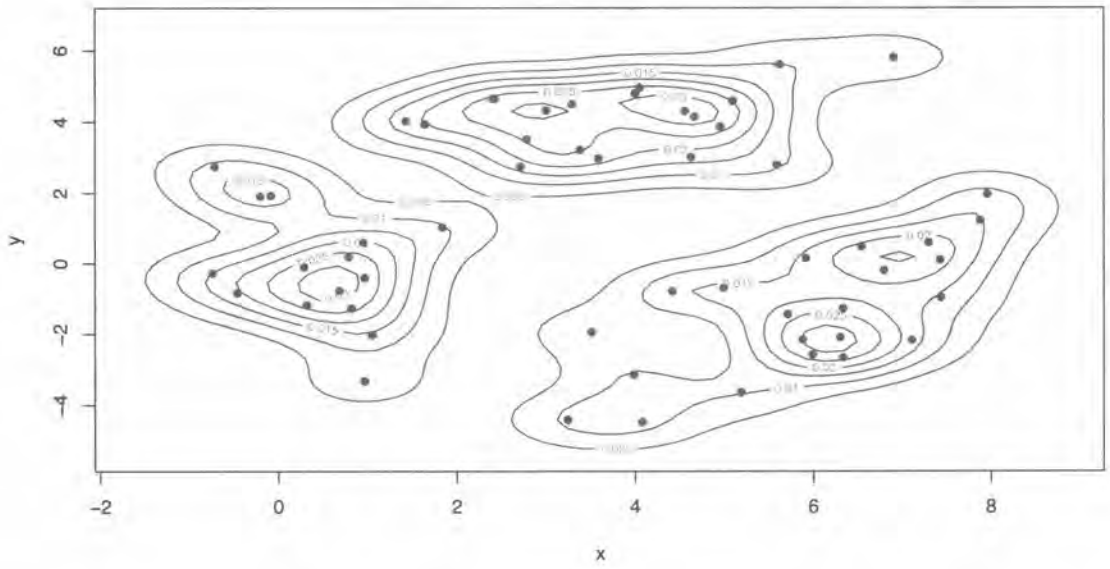

图 7.15 例7.4人造数据点云的非参数密度估计示意图

7.4.2 E-M 算法

记 p 维观测值为 $\boldsymbol{x}=(\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_n)^{\top}$. 假定联合分布为 K 个分量的混合, 而且它们的密度函数分别记为 $f_k(x,|\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k)$ (均值为 $\boldsymbol{\mu}_k$, 协方差矩阵为 $\boldsymbol{\Sigma}_k$ $(k=1,2,\ldots,K)$), 在正态分布的假定下, 混合模型的分布为:

$$
\begin{aligned}
f(x|\boldsymbol{\pi},\boldsymbol{\mu},\boldsymbol{\Sigma}) &= \prod_{i=1}^{n}\sum_{k=1}^{K}\pi_k f_k(x,|\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k) \\
&\propto \prod_{i=1}^{n}\sum_{k=1}^{K}\pi_k\,|\boldsymbol{\Sigma}_k|^{-\frac{1}{2}}\exp\left\{-\frac{1}{2}(\boldsymbol{x}_i-\boldsymbol{\mu}_k)'\boldsymbol{\Sigma}_k^{-1}(\boldsymbol{x}_i-\boldsymbol{\mu}_k)\right\}f_k(x,|\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k),
\end{aligned}
$$

式中的 π_k 定义如下:

$$\pi_k = P(\boldsymbol{x}_i \in \text{第 } k \text{ 个分量}),\ 0 < \pi_k < 1,\ \sum_{k=1}^{K} \pi_k = 1.$$

还定义

$$\zeta_{ik} = \begin{cases} 1, & \text{当第 } i \text{ 个观测值 } \boldsymbol{x}_k \text{ 属于第 } k \text{ 个分量;} \\ 0, & \text{当第 } i \text{ 个观测值 } \boldsymbol{x}_k \text{ 不属于第 } k \text{ 个分量,} \end{cases}$$

由于我们并不知道 ζ_{ik}, 所以把其视为缺失值. 下面考虑 E-M 算法来估计未知参数. 然后可以计算完全的似然函数 (complete loglikelihood) 以及完全的似然函数的期望 (expected complete log-likelihood), 它也称为 Q 函数:

$$\begin{aligned} \ell(\boldsymbol{\vartheta}; \boldsymbol{x}) = &-\frac{1}{2}\sum_{i=1}^{n}\sum_{k=1}^{K}\zeta_{ik}\left\{\log|\boldsymbol{\Sigma}_k| + (\boldsymbol{x}_i - \boldsymbol{\mu}_k)'\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}_i - \boldsymbol{\mu}_k)\right\} \\ &+ \sum_{i=1}^{n}\sum_{k=1}^{K}\zeta_{ik}\log\boldsymbol{\pi}_k - \frac{pn}{2}\log 2\pi, \\ Q(\boldsymbol{\vartheta}; \boldsymbol{x}) = &-\frac{1}{2}\sum_{i=1}^{n}\sum_{k=1}^{K}\pi_{ik}\left\{\log|\boldsymbol{\Sigma}_k| + (\boldsymbol{x}_i - \boldsymbol{\mu}_k)'\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}_i - \boldsymbol{\mu}_k)\right\} \\ &+ \sum_{i=1}^{n}\sum_{k=1}^{K}\pi_{ik}\log\boldsymbol{\pi}_k - \frac{pn}{2}\log 2\pi, \end{aligned}$$

式中, $\pi_{ik} = E(\zeta_{ik}|\boldsymbol{x}_i)$, 在 E 步骤中计算. 下面是 E 步骤和 M 步骤的迭代公式, 从认真选择的初始值开始, 不断交替重复进行下面两个步骤直到收敛为止.

E 步骤: 在第 s 步的迭代:

$$\pi_{ik}^{(s)} = \frac{\pi_k^{(s-1)}\phi\left(\boldsymbol{x}_i; \boldsymbol{\mu}_k^{(s-1)}, \boldsymbol{\Sigma}_k^{(s-1)}\right)}{\sum_{k'=1}^{K}\pi_{k'}^{(s-1)}\phi\left(\boldsymbol{x}_i; \boldsymbol{\mu}_{k'}^{(s-1)}, \boldsymbol{\Sigma}_{k'}^{(s-1)}\right)}.$$

M 步骤: 在第 s 步的迭代:

$$\pi_{k'}^{(s)} = \frac{1}{n}\sum_{i=1}^{n}\pi_{ik}^{(s)},\ \boldsymbol{\mu}_{k'}^{(s)} = \frac{\sum_{i=1}^{n}\pi_{ik}^{(s)}\boldsymbol{x}_k}{\sum_{i=1}^{n}\pi_{ik}^{(s)}},\ \boldsymbol{\Sigma}_{k'}^{(s)} = \frac{\sum_{i=1}^{n}\pi_{ik}^{(s)}(\boldsymbol{x}_i - \boldsymbol{\mu}_k^{(s)})(\boldsymbol{x}_i - \boldsymbol{\mu}_k^{(s)})'}{\sum_{i=1}^{n}\pi_{ik}^{(s)}}.$$

每一个分量都是以 $\boldsymbol{\mu}_k$ 为中心、以 $\boldsymbol{\Sigma}_k$ 为协方差的椭球体, 但 $\boldsymbol{\Sigma}_k$ 不相同, 可把 $\boldsymbol{\Sigma}_k$ 写成:

$$\boldsymbol{\Sigma}_k = \lambda_k \boldsymbol{D}_k \boldsymbol{A}_k \boldsymbol{D}_k',$$

这里的 λ_k 为一个标量, $\boldsymbol{D}_k$ 为 $\boldsymbol{\Sigma}_k$ 的特征向量组成的正交矩阵, $\boldsymbol{A}_k$ 为对角线矩阵, 对角线

元素和 $\boldsymbol{\Sigma}_k$ 的特征向量成比例.

$\boldsymbol{D}_k$ 确定了 $\boldsymbol{\Sigma}_k$ 的主成分方向, $\boldsymbol{A}_k$ 决定了密度的形状, λ_k 反映了椭球的大小 (与 $\lambda_k^p|\boldsymbol{A}_k|$ 成比例).

7.4.3 基于模型聚类的计算实现

在 R 中, 我们使用程序包 mclust[8]中的函数, 主要是函数 Mclust. Mclust 对协方差矩阵的各种参数化的识别使用三个字母 "E" (equal, 相等)、"V" (variable, 可变的)、"I" (coordinate axes, 坐标轴) 的有次序组合, 组合中的第一个字母表示大小, 第二个字母代表形状, 第三个字母显示在 p 维空间中的方向. 由这三个字母可以形成许多种有意义的组合. 比如 EII、VII、EEI、VEI、EVI、VVI、EEE、EVE、VEE、VVE、EEV、VEV、EVV、VVV 等. 比如, EEE 表示这 k 个集群有同样的大小、同样的形状和同样的方向; EII 表示 VVV; VEI 表示这 k 个集群有变化的大小、相同的形状及等于坐标轴的方向; VVV 表示这 k 个集群有变化的大小、变化的形状和变化的方向.

对于给定的 k, Mclust 对模型的选择使用 BIC (Bayesian Information Criterion):

$$\mathrm{BIC} = -2\log(L) + m\log(n),$$

这里的 L 为似然函数, m 是自由参数个数. 按照 BIC, 使得 BIC 最小的模型为首选的模型.

下面用 Mclust 来处理例7.4人造数据:

```
library(mclust)
w=read.csv("kmeansFig.csv")
res = Mclust(w)
summary(res)
plot(res,  what = c("BIC", "classification"))
```

得到下面的输出及 BIC 图 (见图7.16) 和聚类图 (见图7.17).

```
----------------------------------------------------
Gaussian finite mixture model fitted by EM algorithm
----------------------------------------------------

Mclust EEV (ellipsoidal, equal volume and shape) model with 3 components:

log.likelihood  n df       BIC      ICL
     -239.3084 57 13 -531.1766 -532.03

Clustering table:
 1  2  3
15 20 22
```

[8]Scrucca L., Fop M., Murphy T. B. and Raftery A. E. (2016) mclust 5: clustering, classification and density estimation using Gaussian finite mixture models *The R Journal* 8/1, pp. 205-233.

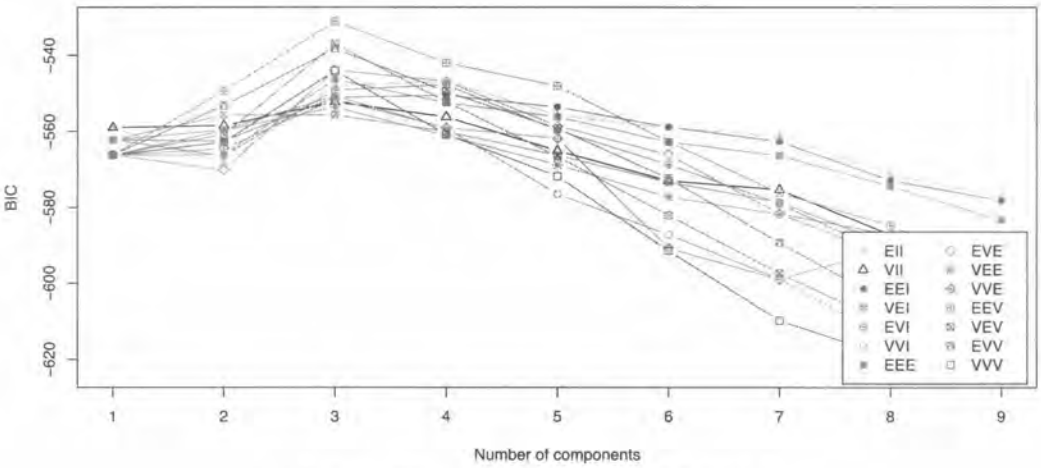

图 7.16 例7.4人造数据的基于模型聚类的 BIC 图

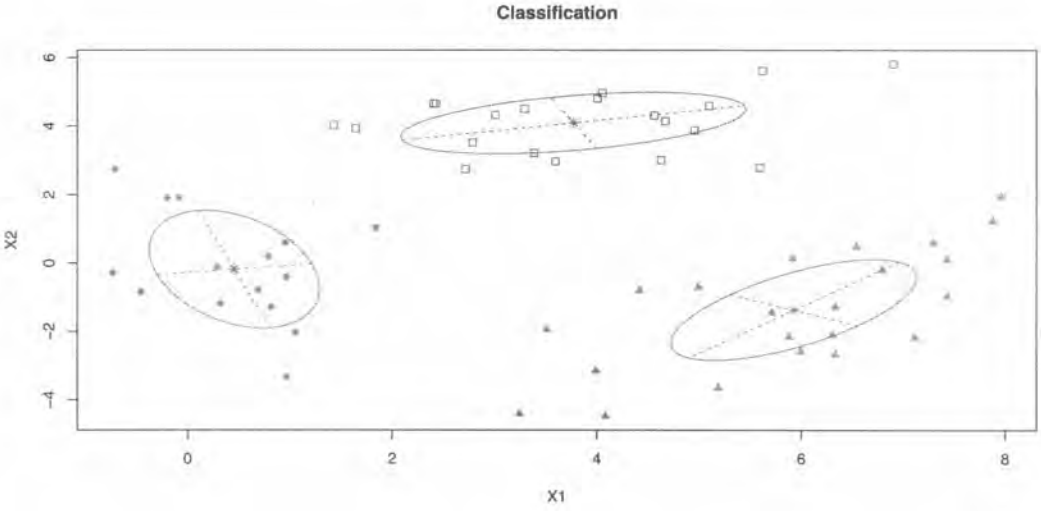

图 7.17 例7.4人造数据的基于模型聚类的结果示意

输出结果和图7.16显示, 最适合聚类数目为 $k = 3$ 的 EEV 模型, 这并不奇怪, 因为我们的数据就是这样人为造出来的.

下面用例6.1来做基于模型的聚类.

```
w=read.csv("who1.csv")
res = Mclust(scale(w[,-1]))
summary(res)
plot(res,  what = c("BIC", "classification"))
```

得到下面的输出及 BIC 图 (见图7.18).

```
----------------------------------------------------
Gaussian finite mixture model fitted by EM algorithm
----------------------------------------------------

Mclust VEV (ellipsoidal, equal shape) model with 2 components:

log.likelihood   n  df       BIC       ICL
     -1315.981 185 842 -7027.502 -7027.556

Clustering table:
 1   2
77 108
```

输出结果和图7.18显示, 最适合聚类数目似乎为 $k=2$ 的 VEV 模型, 但 $k=3$ 时似乎也差不多.

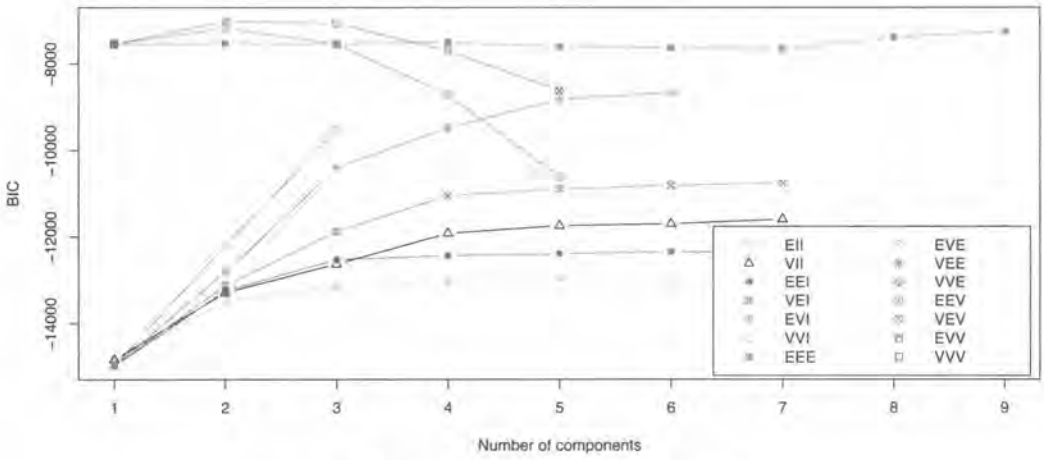

图 7.18 例6.1的基于模型聚类的 BIC 图

我们可以利用下面的代码来看输出的两类各包含多少国家 (这里不输出结果):

```
w[res$classification==1,1]
w[res$classification==2,1]
```

结果显示, 这种分类很没有说服力, 其中, 美国、中国和大部分较贫穷国家如朝鲜等分成一类, 而大部分欧州国家和日本分成另一类. 参照前面分层聚类结果 (见图7.7) 来看, 多分几类可能更有道理, 如果很难得到满意的结果, 则可能需要改换其他聚类方法. 读者可以用代码 `res = Mclust(scale(w[,-1]),G=3)` 来做实验, 分成 3 类也不一定合适. 这说明该数据不是很符合基于模型的聚类假定.

由于基于混合正态分布模型的假定较强, 而这些假定不易验证, 所以使用基于模型的聚类方法时一定要对相关应用领域的知识有所了解, 并且要结合其他聚类方法做综合分析, 互

相验证.

7.5 聚类数目的选择

前面 K 均值聚类一节介绍了通过类内平方和的变化判断类别数目的肘方法, 下面介绍一些其他方法.

7.5.1 Gap 方法 *

Gap 方法计算聚类度量的一个拟合优度度量, 即 Gap 统计量. 下面是其基本思想, 细节请看 Tibshirani, et al. (2001). 假定数据有 n 个观测值及 p 个变量, 记为 $\{x_{ij}\}$ $(i = 1, 2, ..., n;\ j = 1, 2, \ldots, p)$. 用 $d_{ii'}$ 表示第 i 个观测值和第 i' 个观测值之间的距离, 如果选择常用的欧氏距离, 则 $d_{ii'} = \sum_j (x_{ij} - x_{i'j})^2$. 如果把数据的观测值聚成 k 类, 各类的下标集合记为 $C_1, C_2, \ldots, C_k$, 而第 r 类的观测值有 n_r 个. 记在 C_r 中的所有点对的距离和为:

$$D_r = \sum_{i,i' \in C_r} d_{ii'},$$

设

$$W_k = \sum_{r=1}^{k} \frac{1}{n_r} D_r,$$

我们的目的是希望知道对于不同的 k, $\log(W_k)$ 相应于参照曲线 $E_n^*[\log(W_k)]$ 的距离有多远 (越远越好), 这里的 E^* 是对样本量 n 的参照分布的期望. 我们希望选择 k 来最大化下面定义的 Gap 统计量:

$$Gap_n(k) = E_n^*[\log(W_k)] - \log(W_k),$$

这里关于参照分布及如何选取使得 Gap 统计量最大的 k, 请参看 Tibshirani et al. (2001), 我们将使用程序包 `cluster`[9]的函数 `clusGap` 来确定聚类的个数. 在选项中可以选择聚类的方法、尝试类数、确定选取 k 所用的方法 (默认是 `firstSEmax`, 还有其他 4 个选项) 等等. 下面对例7.4人造数据通过函数 `clusGap` 来使用 Gap 方法 (使用默认的 `firstSEmax` 方法), 画图 (见图7.19) 的代码为:

```
library(cluster)
(z=clusGap(w, FUN=kmeans, 5))
plot(z, main = "Gap statistic")
```

上面代码的输出为:

```
Clustering k = 1,2,..., K.max (= 5): .. done
Bootstrapping, b = 1,2,..., B (= 100)  [one "." per sample]:
.................................................. 50
.................................................. 100
```

[9]Maechler, M., Rousseeuw, P., Struyf, A., Hubert, M., Hornik, K.(2017). cluster: Cluster Analysis Basics and Extensions. R package version 2.0.6.

```
Clustering Gap statistic ["clusGap"] from call:
clusGap(x = w, FUNcluster = kmeans, K.max = 5)
B=100 simulated reference sets, k = 1..5; spaceH0="scaledPCA"
--> Number of clusters (method 'firstSEmax', SE.factor=1): 3
        logW   E.logW          gap     SE.sim
[1,] 4.222013 4.215519 -0.006493606 0.04586778
[2,] 3.871177 3.905040  0.033862614 0.05456160
[3,] 3.472781 3.649537  0.176755594 0.06080989
[4,] 3.365399 3.451649  0.086249697 0.04932806
[5,] 3.197999 3.321451  0.123451853 0.05078347
```

输出表明, 该程序选择了 $k=3$. 在图7.19中选择 k 时, **需要找到曲线的最大值, 而且在其一个标准差之内没有其他的点**, 显然 $k=3$ 符合这个条件.

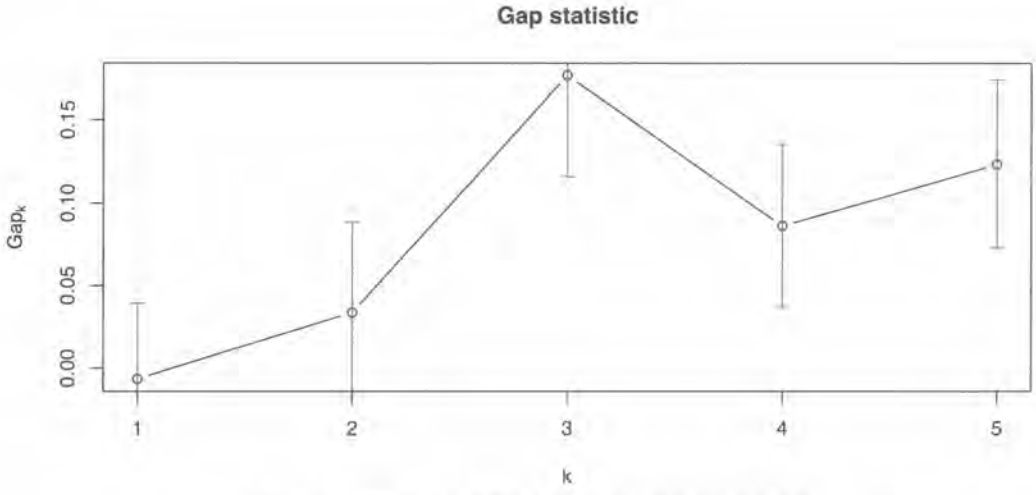

图 7.19 对例7.4人造数据使用 Gap 统计量来确定聚类数目

此外, 利用程序包 `factoextra` 的函数 `fviz_nbclust` 也可以画出和图7.19一样的图, 代码为:

```
library(factoextra)
fviz_nbclust(w, kmeans, nstart = 10,  method = "gap_stat", nboot = 50)+
labs(subtitle = "Gap statistic method")
```

这里不重复输出, 请读者自己体会.

7.5.2 轮廓法 *

轮廓 (silhouette) 法 (Rousseeuw, 1987) 可以产生每个对象在集群内位置的概要图形, 用以解释和验证集群内部一致性, 轮廓值是衡量一个对象相对于其他类来说与本类的相似度. 轮廓值的范围从 -1 到 $+1$, **轮廓值中较高的值表示该对象与自己的集群很好地匹配,**

但不与相邻集群相似. 如果大多数对象具有较高的值, 那么集群的配置是适当的. 如果许多点具有低值或负值, 则聚类个数可能不合适. 计算轮廓值可以用任何距离, 常用的是欧氏距离或曼哈顿距离.

令 $a(i)$ 为第 i 个观测值与本类的所有观测值的平均不相似度 (距离), 如果该类只有第 i 个观测值本身, 则 $a(i)$ 定义为 0. 对于任何其他集群 C, 令 $d(i,C)$ 为第 i 个观测值与 C 的所有观测值的平均不相似度. 令 $b(i) = \min_C d(i,C)$, 这是第 i 个观测值和其不相属的最近集群的距离. 于是, 轮廓宽度 $s(i)$ 定义为:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}.$$

下面用例7.4人造数据, 使用程序包 `cluster` 的函数 `pam` 来做聚类 (聚 3 类), 并且用函数 `silhouette` 计算轮廓宽度 $s(i)$, 最后逐点画出轮廓宽度图 (见图7.20). 具体代码为:

```
library(cluster)
pr3 <- pam(w, 3)
si <- silhouette(pr3)
plot(si,col=c("red","green","blue"))
```

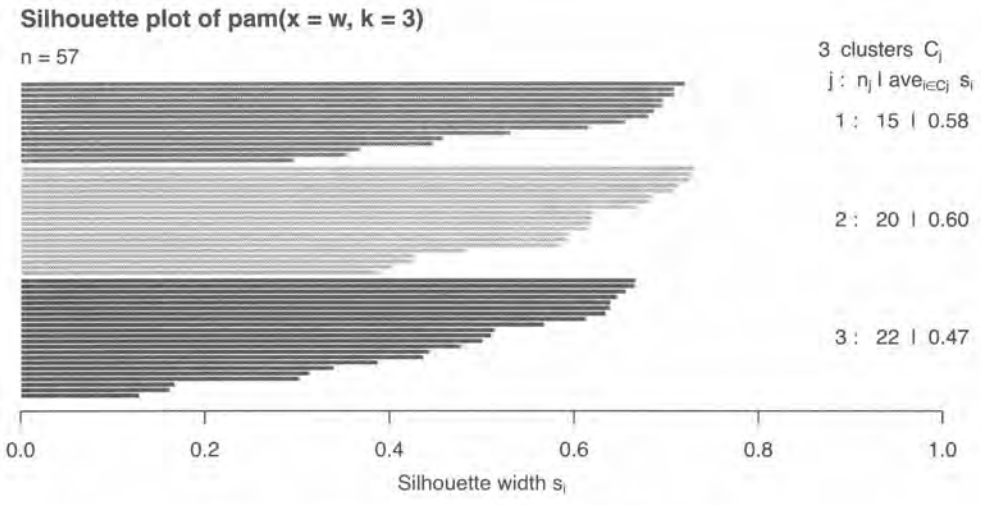

图 7.20 对例7.4人造数据显示每个观测值的轮廓宽度

由于例7.4是人造的, 这个结果应该比较令人满意. 但如果用其他数据或变换不同的聚类个数, 则结果可能不好.

轮廓法可以用来确定聚类个数, 仍以例7.4为例, 我们可以用程序包中 `factoextra` 中的函数 `fviz_nbclust` 来画出针对不同聚类个数的平均轮廓宽度图 (见图7.21), 代码为:

```
library(factoextra)
fviz_nbclust(w, kmeans, method = "silhouette")+
labs(subtitle = "Silhouette method")
```

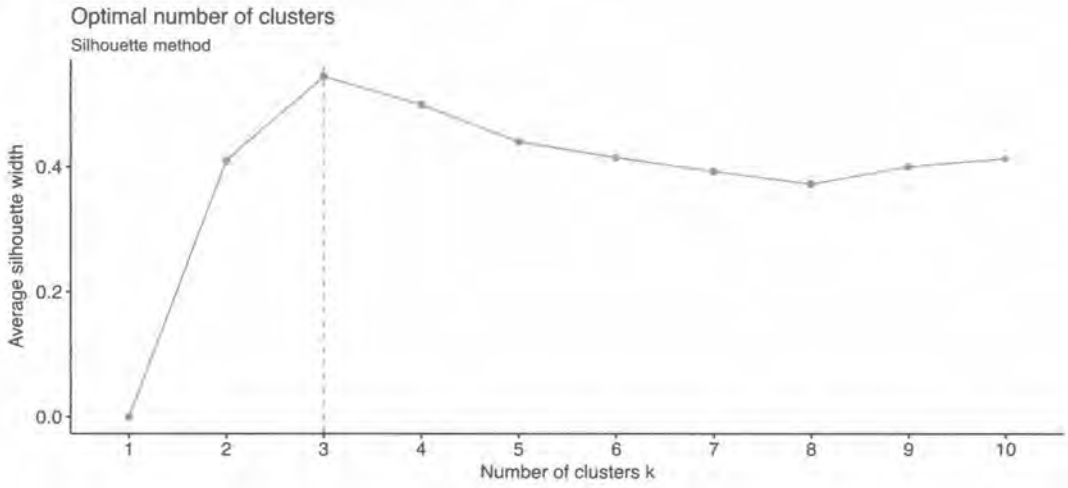

图 7.21 对例7.4人造数据利用不同聚类数目的平均轮廓宽度选择最优聚类个数

下面以例6.1为例确定类数, 并产生图7.22. 具体代码如下 (这里, 选项 hcut 是分层聚类法):

```
library(factoextra)
w=read.csv("who1.csv")
fviz_nbclust(scale(w[,-1]), hcut, method = "silhouette")+
labs(subtitle = "Silhouette method")
```

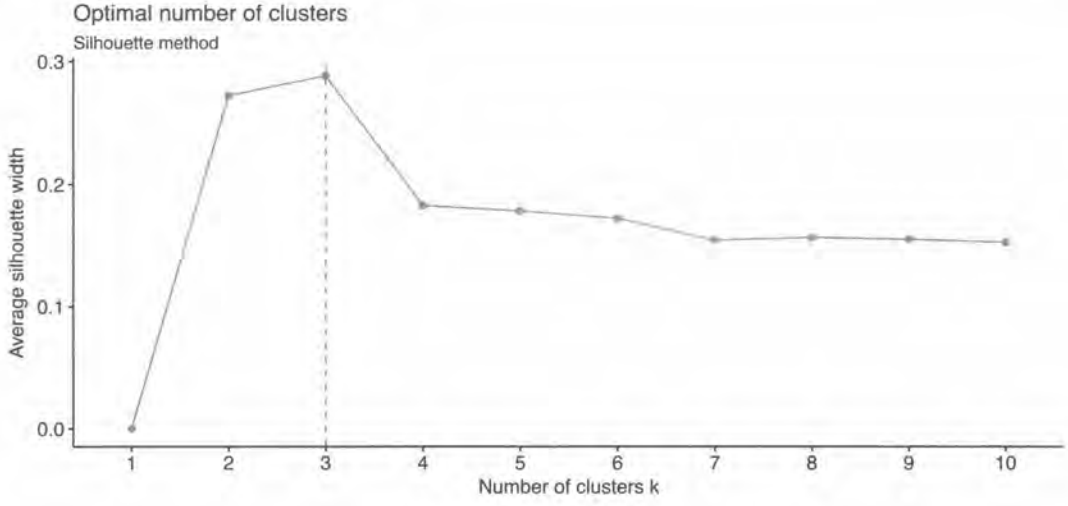

图 7.22 对例6.1利用不同聚类数目的平均轮廓宽度选择最优聚类个数 (3 个)

根据图7.22, 选择 3 个做分层聚类, 这里全部使用程序包 factoextra 的函数. 在代码中, 函数 hcut 用来做分层聚类 (3 类), 函数 fviz_dend 用来画出树状图 (见图7.23(上)), 函数 fviz_silhouette 用来画出轮廓宽度图 (见图7.23(左下)), 而函数 fviz_cluster 则在数据第一、第二主成分的得分图上画出聚类结果 (见图7.23(右下)).

```
w=read.csv("who1.csv")
w1=data.frame(w[,-1],row.names = w[,1])
res=hcut(scale(w1), k = 3, stand = TRUE)
fviz_dend(res, rect = TRUE,cex=.3,horiz = T)
fviz_silhouette(res)
fviz_cluster(res)
```

从聚类结果来看, 所有国家分成 3 类, 一类大体上是一些最穷困的国家, 另一类大体上是较富裕的国家, 还有一类只有三个国家: 中国、美国、印度. 具体国家名字可以用下面代码输出:

```
for(i in 1:3) print(w[res$cluster==i,-1])
```

请读者分析这样聚类是否合理. 按照图7.23(左下), 轮廓宽度连 0.5 都不到, 有些是负值, 这个指标并不好. 图7.23(右下) 显示, 两个大类的区间好像重合得较多. 但聚类是否合理, 最终还得以内行专家的观点为准.

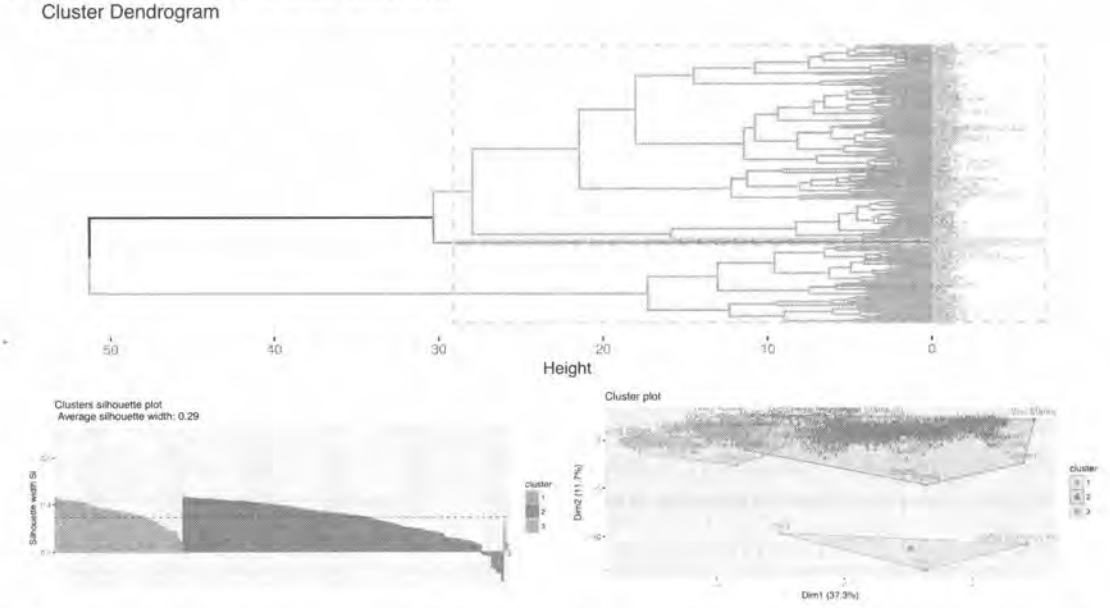

图 7.23 对例6.1做分层聚类的树状图 (上)、轮廓宽度图 (左下)、在数据第一、第二主成分的得分图上画出聚类结果 (右下)

7.5.3 计算聚类数目的不同软件程序包 NbClust 包含的方法

这里介绍的聚类数目的选择实际上不是一种方法, 而是集合在一个程序包中的一系列方法, 也包括前面的 Gap 统计量. 该程序包为 NbClust[10], 它提供了 30 个指数用于确定在

[10]Malika Charrad, Nadia Ghazzali, Veronique Boiteau, Azam Niknafs (2014). NbClust: An R Package for Determining the Relevant Number of Clusters in a Data Set. *Journal of Statistical Software*, 61(6), 1-36. URL http://www.jstatsoft.org/v61/i06/.

数据集中的最优聚类数目, 并根据不同的结果为用户提供最佳的聚类方案. 它不仅确定聚类数目, 而且根据选项实行具体的聚类, 包括前面的 K 均值聚类.

NbClust 提供的聚类基于的距离选择选项 distance 有:

```
"euclidean", "maximum", "manhattan", "canberra", "binary",
"minkowski", "NULL".
```

distance 的默认值为 euclidean(欧氏距离), 如果选了 NULL, 则必须在 diss 选择不相似矩阵 (diss 默认值为 NULL). NbClust 提供的方法有:

```
"ward.D", "ward.D2", "single", "complete", "average",
"mcquitty", "median", "centroid", "kmeans".
```

而判别数目的判别指标选项包括:

```
"kl", "ch", "hartigan", "ccc", "scott", "marriot",
"trcovw", "tracew", "friedman", "rubin", "cindex", "db",
"silhouette", "duda", "pseudot2", "beale", "ratkowsky",
"ball", "ptbiserial", "gap", "frey", "mcclain", "gamma",
"gplus", "tau", "dunn", "hubert", "sdindex", "dindex",
"sdbw", "all", "alllong".
```

其中, all 指标包括除 gap、 gamma、 gplus、 tau 之外的所有指数, 而 alllong 包括 all 所没有包括的所有指标.

我们就例6.1来说明这个程序包的应用. 下面是代码 (这里选了指标 all):

```
w=read.csv("who1.csv")
w1=data.frame(w[,-1],row.names = w[,1])
library(NbClust)
a<-NbClust(scale(w1), distance = "euclidean", min.nc=2,
max.nc=8,  method = "complete", index = "all")
```

输出为:

```
*******************************************************************
* Among all indices:
* 7 proposed 2 as the best number of clusters
* 2 proposed 3 as the best number of clusters
* 5 proposed 4 as the best number of clusters
* 6 proposed 5 as the best number of clusters
* 1 proposed 7 as the best number of clusters
* 3 proposed 8 as the best number of clusters

***** Conclusion *****
```

```
* According to the majority rule, the best number of clusters is  2
```

结果表明, 有 7 个方法认为聚类数目应该为 2, 有 6 个方法认为应该分成 5 类, 有 5 个方法建议聚成 4 类, 还有 3 个认为类数应该为 8.

程序包 fpc 的函数 pamk 包含的方法

程序包 fpc 的函数 pamk 通过程序包 cluster 的函数 pam 或 clara 围绕中心点 (meoid) 的聚类来利用轮廓法确定聚类数目. 这里的聚类方法主要是 k 中心点法 (k-medioids), 中心点类似于几何中位数 (geometric median) 但却是一个真实的观测值 (几何中位数不一定是真实值). 判别聚类数目的方法是通过前面介绍的平均轮廓宽度, 使用 Calinski-Harabasz 指数 (Calinski-Harabasz index, calinhara) 来确定. 另外, 关于聚类数目是否应该多于 1 的检验为 Duda-Hart 检验 (dudahart2), 细节请参看 Hennig and Liao (2013).

对于例6.1, 可使用下面代码来实现:

```
w=read.csv("who1.csv")
w1=data.frame(w[,-1],row.names = w[,1])
library(fpc)
pamk.best <- pamk(scale(w1))
cat("estimated number of clusters:", pamk.best$nc, "\n")
plot(pam(w1, pamk.best$nc))
```

输出的最优聚类个数的估计数目为 2. 同时, 还输出了前两个主成分的观测值得分的分类图和轮廓宽度图, 在这里就不显示了, 显然分成两类不那么合适.

INCA 指数

另一种方法是利用所谓的 INCA 指数, 具体方法参见 Arenas and Cuadras (2002) 与 Irigoien and Arenas (2008). 我们所用的程序包为 ICGE[11], 其主要思想是: 首先, 估计数据集中的实际集群的数量, 然后, 确定新观测值是否属于这些先前识别的集群之一, 或者属于 "异常的" 或 "非典型" 的范畴. 该方法基于任何一对观测值之间的距离或不相似度, 可以应用于任何类型的多变量数据 (包括连续、二元或多属性数据). INCA 指数基于的距离或相似度的概念就是前面介绍的 Gover(1971) 所定义的.

对于例6.1数据, 使用下面代码计算并点出图 (见图7.24) 来:

```
w=read.csv("who1.csv")
library(ICGE)
d=dist(scale(w[,-1]))
T=vector()
for (i in 2:10)
{part=pam(d, i)$clustering
```

[11]Itziar Irigoien and Concepcion Arenas (2014). ICGE: Estimation of number of clusters and identification of atypical units. R package version 0.3. https://CRAN.R-project.org/package=ICGE.

```
T[i]=INCAindex(d, part)$Total}
plot(T,  type="b", xlab="Number of clusters",
    ylab="INCA",  xlim=c(1.5,  10.5))
title("Percentage of objects well classified in the partition")
```

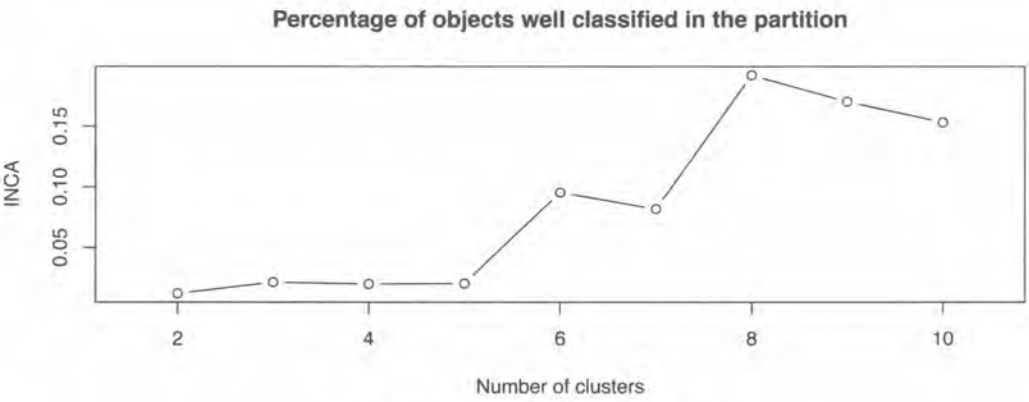

图 7.24 例6.1数据的 INCA 指数

图7.24显示, 最大值点在类数上取 8, 也就是说, 应该分成 8 类.

对例6.1数据做分层聚类, 使用下面代码可得到图7.25:

```
library("factoextra")
w=read.csv("who1.csv")
w1=data.frame(w[,-1],row.names = w[,1])
res=hcut(scale(w1), k = 8, stand = TRUE)
fviz_dend(res, rect = TRUE,cex=.3,horiz = T)
fviz_silhouette(res)
fviz_cluster(res)
```

下面输出各类的国家名字. 其中第 1 类是一些亚洲和非洲的中上等水平发展中国家, 第 2 类大都是东欧国家和由苏联解体后分化出去的国家, 第 3 类大都是中南美洲国家和阿拉伯国家, 第 4 类是美国之外的发达国家 (包括卫生发达的古巴), 第 5 类大都是南部非洲国家, 第 6 类有一些岛国和其他国家 (包括土耳其), 第 7 类是中国和印度, 第 8 类是美国. 这种分法似乎比分成 2 类或 3 类更好. 但人们也可以说, 每个国家都具有独特性, 还是一个国家为一类最好. 这个例子可以说明, 无论分成多少类, 都不能由统计学家来决定, 更重要的是, 一切结论应该以人们所关心的问题为导向.

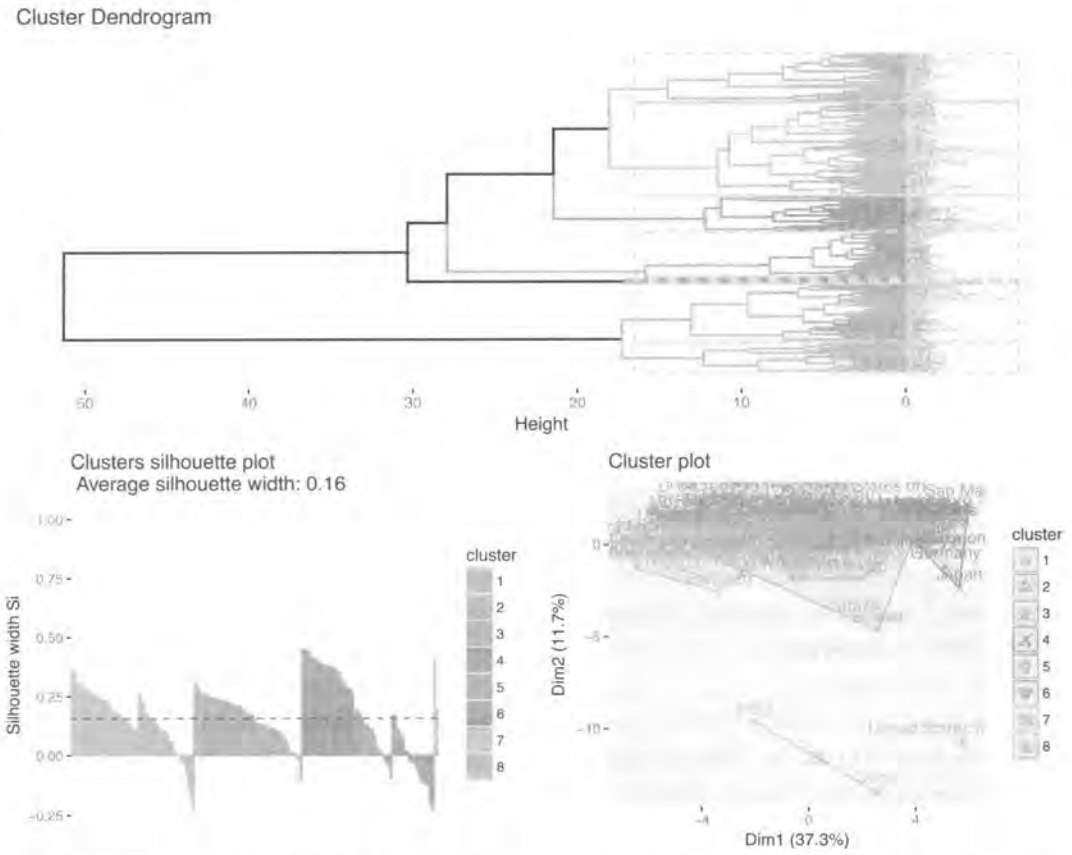

图 7.25 例6.1数据的分层聚类 (8 类) 树状图 (上), 轮廓宽度图 (左下) 及前两个主成分得分聚类图 (右下)

```
> for(i in 1:8) print(as.character(w[res$cluster==i,1]))
#以下是第1类 (34个)
[1] "Afghanistan" "Bangladesh"  "Benin"  "Burundi"  "Cambodia"
[6] "Cameroon" "Central African Republic"  "Chad"  "Comoros"
[10] "Congo, Dem. Rep." "Congo, Rep." "Cote d'Ivoire"
[13] "Equatorial Guinea" "Eritrea" "Gabon" "Ghana" "Guinea"
[18] "Guinea-Bissau" "Iraq" "Kenya"
[20] "Lao People's Democratic Republic" "Mali" "Mauritania"
[24] "Myanmar" "Nepal"  "Niger" "Nigeria" "Pakistan"  "Senegal"
[30] "Somalia" "Sudan" "Togo" "Uganda" "Yemen"
#以下是第2类 (28个)
[1] "Albania" "Armenia" "Azerbaijan" "Belarus"
[5] "Bosnia and Herzegovina" "Brazil" "Bulgaria" "Croatia"
[9] "Czech Republic" "Egypt" "Estonia" "Georgia" "Grenada" "Hungary"
[15] "Kazakhstan" "Kyrgyzstan" "Latvia" "Lithuania" "Macedonia"
[20] "Moldova" "Poland" "Romania" "Russia" "Slovakia" "Tajikistan"
[26] "Turkmenistan" "Ukraine" "Uzbekistan"
#以下是第3类 (54个)
```

```
 [1] "Algeria" "Antigua and Barbuda" "Argentina" "Bahamas" "Bahrain"
 [6] "Barbados" "Belize" "Chile" "Colombia" "Cook Islands" "Costa Rica"
[12] "Cyprus" "Dominica" "Dominican Republic" "Ecuador" "El Salvador"
[17] "Guatemala" "Honduras" "Indonesia" "Iran (Islamic Republic of)"
[21] "Israel" "Jamaica" "Jordan" "Korea, Rep." "Kuwait" "Lebanon"
[27] "Libyan Arab Jamahiriya" "Malaysia" "Mauritius" "Mexico"
[31] "Morocco" "Nicaragua" "Oman" "Panama" "Paraguay" "Peru"
[37] "Philippines" "Qatar" "Saint Kitts and Nevis" "Saint Lucia"
[41] "Saint Vincent and the Grenadines" "Saudi Arabia" "Seychelles"
[44] "Singapore" "Sri Lanka" "Suriname" "Syria" "Thailand" "Tonga"
[50] "Trinidad and Tobago" "Tunisia" "United Arab Emirates" "Uruguay"
[54] "Venezuela"
#以下是第4类 (27个)
[1] "Andorra" "Australia" "Austria" "Belgium" "Canada" "Cuba" "Denmark"
[8] "Finland" "France" "Germany" "Iceland" "Ireland" "Italy" "Japan"
[15] "Luxembourg" "Malta" "Monaco" "Netherlands" "New Zealand" "Norway"
[21] "Portugal" "San Marino" "Slovenia" "Spain" "Sweden" "Switzerland"
[27] "United Kingdom"
#以下是第5类 (18个)
[1] "Angola" "Botswana" "Burkina Faso" "Djibouti" "Ethiopia" "Gambia"
[7] "Lesotho" "Liberia" "Madagascar" "Mozambique" "Namibia" "Rwanda"
[13] "Sierra Leone" "South Africa" "Swaziland" "Tanzania" "Zambia"
[18] "Zimbabwe"
#以下是第6类 (21个)
[1] "Bhutan" "Bolivia" "Cape Verde" "Fiji" "Guyana" "Haiti" "Kiribati"
[8] "Korea, Dem. Rep." "Maldives" "Marshall Islands"
[11] "Micronesia (Federated States of)" "Mongolia" "Nauru" "Palau"
[15] "Papua New Guinea" "Samoa" "Sao Tome and Principe" "Solomon Islands"
[19] "Timor-Leste" "Turkey" "Tuvalu"
#以下是第7类 (2个)
[1] "China" "India"
#以下是第8类 (1个)
[1] "United States of America"
```

7.6 实例的计算

7.6.1 种子数据

例 7.6. 种子数据 (seeds.csv) 该数据[12]包含了来自 3 个不同品种 (Kama、Rosa、Canadian) 的小麦种子, 这 3 个品种的代码由哑元 1、2、3 代表, 形成第 8 个变量 ID. 每一个品种里有随机选择的 70 个样本个体, 共计 210 个观测值. 除了 ID 之外, 其他 7 个变量均为关于

[12]该数据提供者来自波兰卢布林若望保罗二世天主教大学的数学及计算机科学学院 (Institute of Mathematics and Computer Science, The John Paul II Catholic University of Lublin), 而且得到波兰卢布林科学院农业物理学研究所 (Institute of Agrophysics of the Polish Academy of Sciences in Lublin) 的支持.

种子度量的实质连续型数量变量. 它们分别为 area(面积)、perimeter(周长)、compactness (紧密度、等于 4π面积/周长^2)、length of kernel(核长)、width of kernel(核宽)、asymmetry coefficient (非对称系数)、length of kernel groove (内核槽长度). 该数据可以从网上下载.[13]

例7.6既可以做有指导学习的分类, 又可以做无指导学习的聚类. 本章当然考虑聚类, 我们不使用第 8 个变量 ID (已经知道的种类), 而希望通过其他 7 个变量把种子分成若干类.

例7.6种子数据的集群倾向和距离图

先看集群倾向, 首先通过计算 Hopkins 统计量并点出热量图 (见图7.26) 来得到 Hopkins 统计量的值为 0.1972587, 说明其有集群倾向.

```
library(factoextra)
w=read.csv("seeds.csv")
a=get_clust_tendency(scale(w[,-8]), nrow(w)-1, graph = TRUE)
a$hopkins_stat
library(CluMix)
hm=mix.heatmap(scale(w[,-8]),rowmar = 14)
```

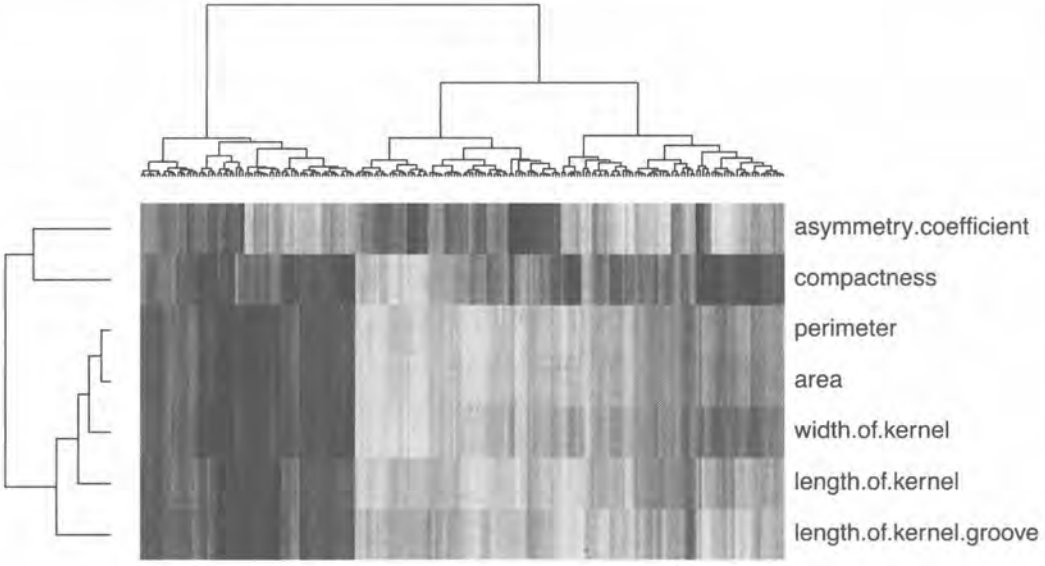

图 7.26 例7.6种子数据的热量图

例7.6种子数据聚类个数的确定

我们用 NbClust 函数来判断聚为几类.

```
library(NbClust)
a<-NbClust(scale(w[,-8]), distance = "euclidean", min.nc=2,
```

[13]网址: https://archive.ics.uci.edu/ml/datasets/seeds.

```
max.nc=8,  method = "complete", index = "all")
```

输出为:

```
* Among all indices:
* 7 proposed 2 as the best number of clusters
* 16 proposed 3 as the best number of clusters
* 1 proposed 8 as the best number of clusters

***** Conclusion *****

* According to the majority rule, the best number of clusters is  3
```

如此看来争议不大, 再用 INCA 指数, 结果也表明 3 类最合适 (请读者自己实现).

例7.6种子数据的分层聚类

分层聚类的计算很简单, 输入数据、实施对观测值分层聚类并画出树状图 (见图7.27) 的 R 代码如下:

```
w=read.csv("seeds.csv")
hh=hclust(dist(scale(w[,-8])), "complete")
plot(hh,labels=w[,8],cex=.6)
id=identify(hh)#手工分成3份
```

在计算时用了标准化的没有第 8 个变量的数据并使用欧氏距离 (代码第一行). 在图7.27中, 我们用已知的 ID 哑元变量 (1、2、3) 来标记观测值 (第三行的 `labels=w[,8]`), 这样就可以知道我们得到的聚类和已知的类别的异同. 上面四行代码 `identify(hh)` 使得我们可以手工决定选择结果的类别, 并且记录在 `id` 中.

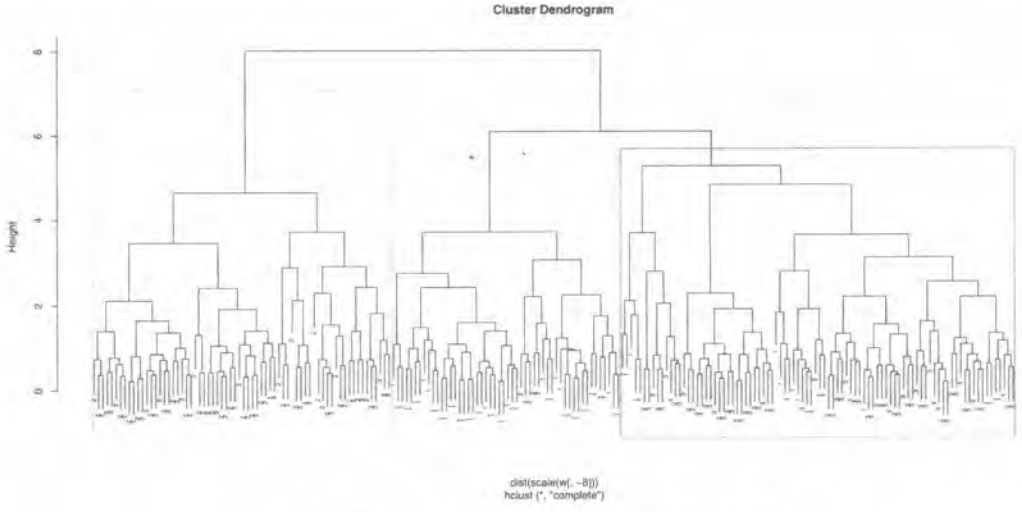

图 7.27 例7.6种子数据对观测值分层聚类的树状图

图7.27显示, 越靠上 (后) 的叉越长, 这表明在分层聚类中距离远的类后合并. 可以从图7.27看到手工选择导致的三个类 (先选的中间 "1" 多的类, 后选左右以 "2" 和 "3" 为主的类). 得到的第 1 个聚类中原来属于第 1 类的占 92.3%, 所得第 2 个聚类中, 原来属于第 2 类的占 97.1%, 而第 3 个聚类中, 原来属于第 3 类的占 77.8%. 在这个问题上, 用聚类代替分类总的 "准确率" 达到 0.8761905, 一共有 26 个观测值分到了错误的聚类.

这个结果是不是说明聚类和分类有类似的意义? 完全不是. 这一特殊问题, 即关于种子种类的分类和聚类结果类似的问题, 不能推广. 聚类和分类结果是否相似与我们的目标变量有关. 正如前面提到过的, **同样一组变量观测值的分类可能有很多种结果, 其依因变量 (即问题) 的性质而定, 但聚类结果仅仅依赖于聚类方法和聚类个数的选择. 毫无疑问的是, 用分类结果来 "验证" 聚类结果会产生误导.**

利用下面代码, 可得到前两个主成分的观测值得分的聚类情况 (见图7.28):

```
res=hcut(scale(w[,-8]), k = 3, stand = TRUE)
fviz_cluster(res)
```

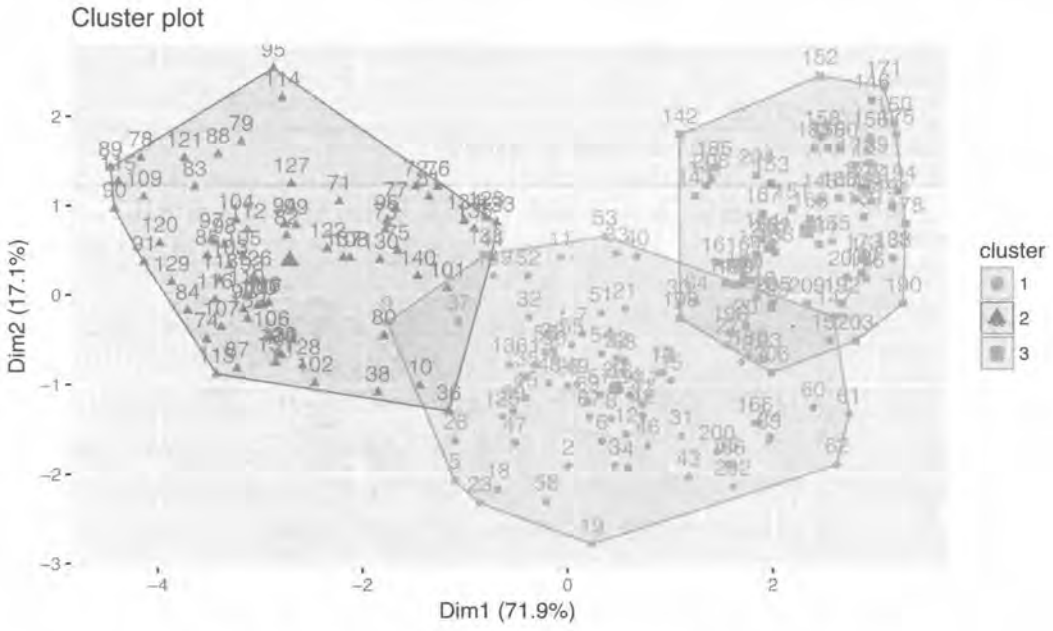

图 7.28 例7.6种子数据前两个主成分的观测值得分的 K 均值聚类情况

我们还可以对变量进行聚类, 产生树状图 (见图7.29) 的代码为:

```
wt=t(w[,-8])
ht=hclust(dist(scale(wt)), "complete")
plot(as.dendrogram(ht),horiz=TRUE)
```

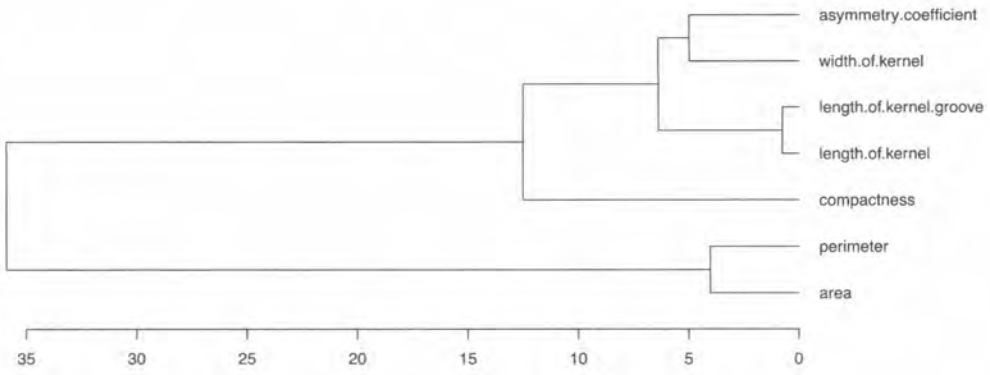

图 7.29 例7.6种子数据对变量分层聚类的树状图

例7.6种子数据的 K 均值聚类

关于例7.6的 K 均值聚类, 利用前面的结果, 我们选择聚类个数为 $k=3$, 用 R 自带的函数做聚类 (因为初始中心的选择是随机的, 我们使用随机种子以使得结果可重复), 并得到前两个主成分的观测值得分的聚类情况 (见图7.30):

```
set.seed(1010)
a=kmeans(scale(w[,-8]),3)
fviz_cluster(a, w[, -8], ellipse.type = "norm")
```

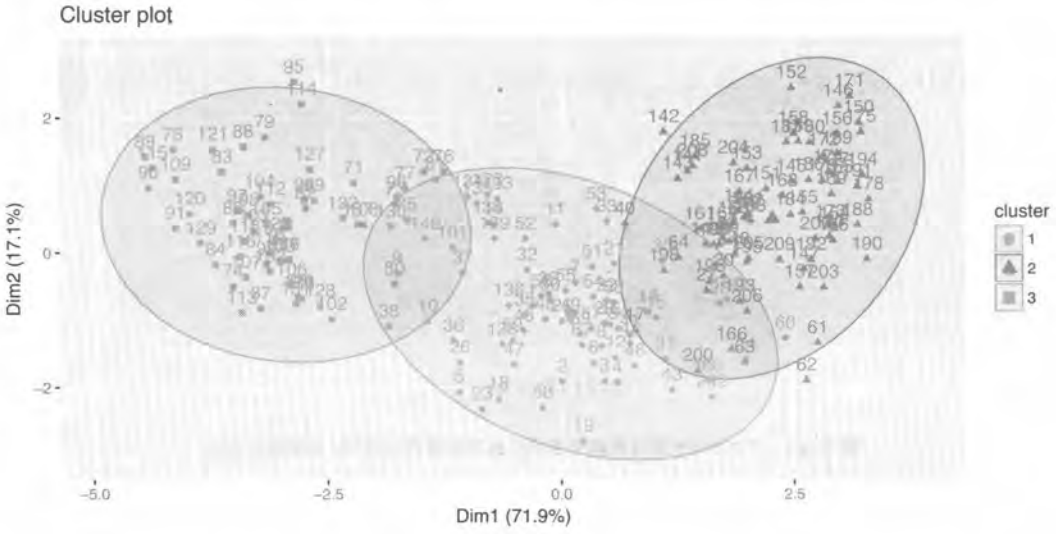

图 7.30 例7.6种子数据前两个主成分的观测值得分的 K 均值聚类情况

例7.6种子数据的分层 K 均值聚类

用分层 K 均值聚类方法, 得到前两个主成分的观测值得分的聚类情况 (见图7.31).

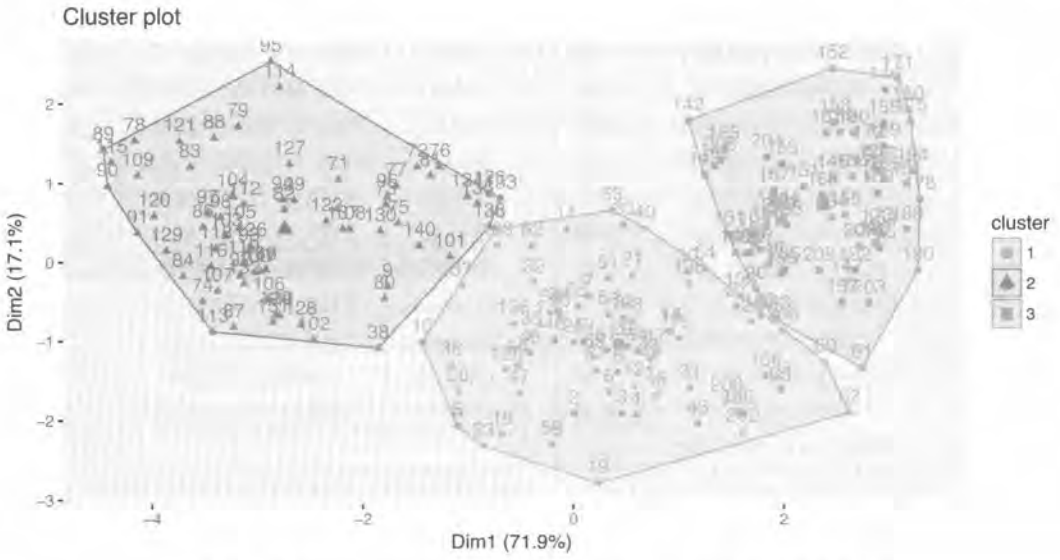

图 7.31 例7.6种子数据前两个主成分的观测值得分的分层 K 均值聚类情况

产生图7.31的代码为:

```
res.hk <-hkmeans(scale(w[,-8]), 3)
fviz_cluster(res.hk)
```

把图7.31和前面的单独分层聚类的图7.28及单独 K 均值聚类图7.30做比较, 似乎分层 K 均值聚类结果更好 (显然和有监督学习的分类结果无关).

7.6.2 胎心宫缩监护数据

例 7.7. 胎心宫缩监护 (cardiotocography, CTG) 数据 (CTG.xls) 这组数据[14]来源于 Frank and Asuncion (2010). 该数据有 2129 个观测值及 23 个变量, 包含了致命心律 (fetal heart rate, FHR) 的各种度量以及基于监护记录的由专家分类的宫缩 (uterine contraction, UC) 特征. 这些变量的情况列示于下表. 表中最后三个分类变量的水平为: Tendency(FHR 直方图的趋势) 有三个水平 (-1 = 左不对称, 0= 对称, 1= 右不对称); CLASS(FHR 分类代码) 用 1∼10 表示从平静睡眠到可疑的 10 种活动状况; NSP(胎儿状态分类代码) 有三个水平 (1= 正常, 2= 疑似, 3= 病态).

我们删去了 3 个有不少缺失值的观测值, 形成新的数据文件 ctg.naomit.csv. 如果考虑分类问题, 我们可以用前面 22 个变量作为自变量来预测作为因变量的 (最后一个变量)NSP 的类别. 但这里用聚类, 我们舍弃第 23 个变量 NSP, 只用前面 22 个变量来聚类.

[14]该数据可从网站https://archive.ics.uci.edu/ml/datasets/cardiotocography下载.

变量名	描述	性质
LB	FHR 基线 (心跳数/分)	定量
AC	每秒钟加速数目	定量
FM	每秒钟胎儿动作数目	定量
UC	每秒钟子宫收缩数目	定量
DL	每秒钟轻微减速数目	定量
DS	每秒钟严重减速数目	定量
DP	每秒钟持续减速数目	定量
ASTV	短期变异的时间百分比	定量
MSTV	短期变异的平均值	定量
ALTV	长期变异的时间百分比	定量
MLTV	长期变异的平均值	定量
Width	FHR 直方图的宽度	定量

变量名	描述	性质
Min	FHR 直方图的最小值	定量
Max	FHR 直方图的最大值	定量
Nmax	FHR 直方图的峰值数	定量
Nzeros	FHR 直方图的零值数	定量
Mode	FHR 直方图的众数	定量
Mean	FHR 直方图的均值	定量
Median	FHR 直方图的中位数	定量
Variance	FHR 直方图的方差	定量
Tendency	FHR 直方图的趋势	分类
CLASS	FHR 分类代码	分类
NSP	胎儿状态分类代码	分类

例7.7胎心宫缩监护数据的热量图

下面代码画出例7.7数据关于观测值和变量两方面的距离的热量图 (见图7.32):

```
w=read.csv("CTG.NAOMIT.csv")
w[,21:23]=apply(w[,21:23],2,factor)
w[,1:20]=scale(w[,1:20])
library(CluMix)
mix.heatmap(w[,-23], rowmar=7, dist.variables.method ="distcor")
```

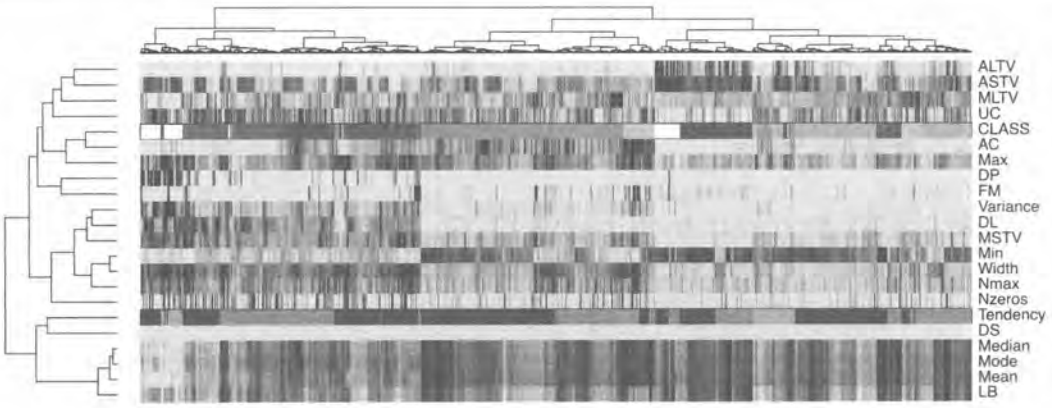

图 7.32 例7.7胎心宫缩监护数据的热量图

例7.7胎心宫缩监护数据的分层聚类

可以对观测值和变量均做分层聚类, 并画出图 (分别为图7.33的左、右两图). 由于观测值太多, 图7.33左图的标签看不清楚. 这两个图实际上和热量图中的树状图一致.

```
plot(dendro.subjects(w[,-23]))
plot(dendro.variables(w[,-23], method="distcor"))
```

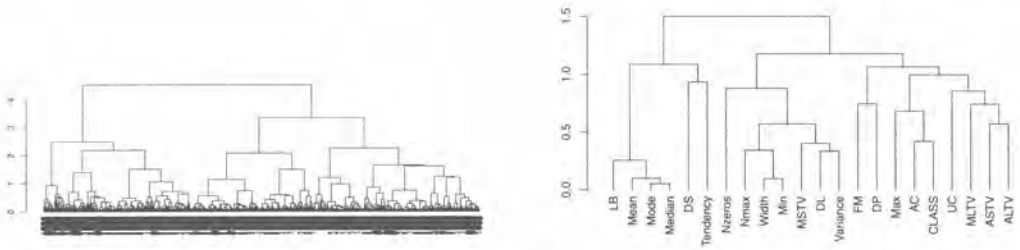

图 7.33 例7.7胎心宫缩监护数据对观测值和变量均做分层聚类的树状图

例7.7胎心宫缩监护数据的分层聚类和分类的比较

使用下面代码, 可以得到对观测值聚成 3 类的结果, 以及和原来因变量 NSP 的类的比较:

```
library(cluster)
cluster <- daisy(w[,-23], metric = c("gower"),
  stand = FALSE, type = list())
try <- agnes(cluster,3)
rhc=cutree(try,k=3)
table(w$NSP,rhc)[,c(2,1,3)]
```

得到:

```
  rhc
     2    1    3
1 1647    0    8
2  293    1    1
3   63    7  106
```

显然, 如果要用聚类来挑战分类, 那么上面的混杂矩阵算是 "最优" 的了, 也就是说, 聚类的第 2 类相当于因变量 NSP 的 "1", 聚类的第 1 类相当于 NSP 的 "2", 聚类的第 3 类相当于 NSP 的 "3". 按照这样的分法, 错判率为 17.5%, 相当大, 而 NSP 第 "2" 类的错判率为 99.66%. 如果用某些分类的方法, 交叉验证的误差可以仅为 1.3%左右. 如果改变因变量, 则分类与聚类的结果差距可能会更大.

之所以人们喜欢把分类和聚类两个完全不同的问题做比较, 可能是因为很多作者喜欢对著名的鸢尾花数据 (R 自带的名为 `iris` 的数据) 做分类和聚类的比较. 例7.6的种子数据也有鸢尾花数据的特性, 它们很相似. 希望前面针对例7.6数据把分类和聚类做比较不会引起误解.

7.6.3 例5.1教师数据的聚类

例5.1的热量图

下面考虑例5.1教师数据的聚类问题. 我们先产生热量图 (见图7.34), 它包括了对观测值及变量做分层聚类的树状图.

```
w=read.csv("full.aaup.csv")
w1=w[,-(1:4)]
library(CluMix)
mix.heatmap(scale(w1), rowmar=7)
```

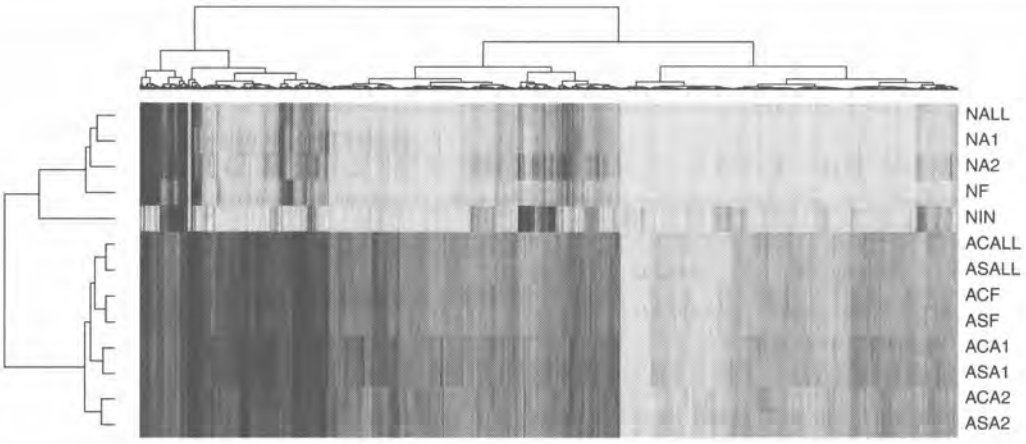

图 7.34 例5.1数据的热量图

例5.1对观测值的聚类个数

利用各种方法来判断对观测值的聚类个数:

```
library(NbClust)
res<-NbClust(scale(w1), distance = "euclidean", min.nc=2,
   max.nc=8,  method = "complete", index = "all")
```

得到的多数方法支持聚 3 类.

```
*******************************************************************
* Among all indices:
* 6 proposed 2 as the best number of clusters
* 7 proposed 3 as the best number of clusters
* 2 proposed 4 as the best number of clusters
* 5 proposed 7 as the best number of clusters
* 2 proposed 8 as the best number of clusters

***** Conclusion *****
```

```
* According to the majority rule, the best number of clusters is  3
```

例5.1对观测值的分层聚类

用分层聚类把例5.1数据的观测值分 3 类时的前两个主成分得分 (见图7.35) 的画图代码为:

```
library(factoextra)
ds=hcut(w1, k = 3, stand = TRUE)
fviz_cluster(ds)
```

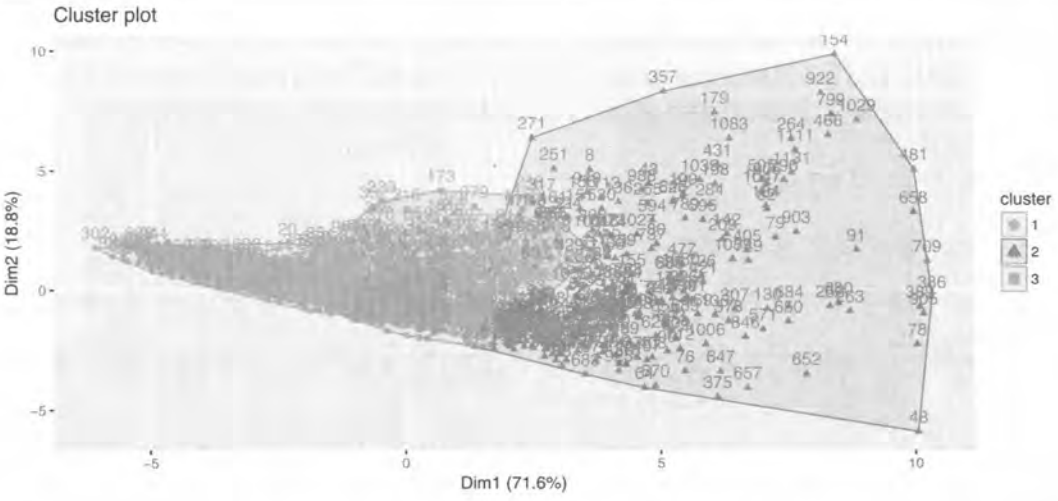

图 7.35 例5.1数据对观测值分层聚类 (3 类) 的前两个主成分得分图

用代码 `for(i in 1:3) print(w[ds$cluster==i,2])` 可以得到分成 3 类后每一类有什么学校的结果, 请读者自己实现, 并且进行讨论.

例5.1对变量的分层聚类

用分层聚类把例5.1数据的变量分 2 类时的前两个主成分得分及树状图的画图 (见图7.36) 及输出各类变量名字的代码为:

```
dv=hcut(t(w1), k = 2, stand = TRUE)
fviz_cluster(dv)
for(i in 1:2) print(names(w1)[dv$cluster==i])
fviz_dend(dv, k = 2, k_colors = c("#1B9E77", "#D95F02"))
```

两类变量名字的输出为:

```
#第1类
[1] "ASF"   "ASA1"  "ASA2"  "ASALL"
[5] "ACF"   "ACA1"  "ACA2"  "ACALL"
#第2类
```

```
[1] "NF"   "NA1"  "NA2"  "NIN"  "NALL"
```

根据图7.36的左图可以看出, 把变量分成两类是很合适的.

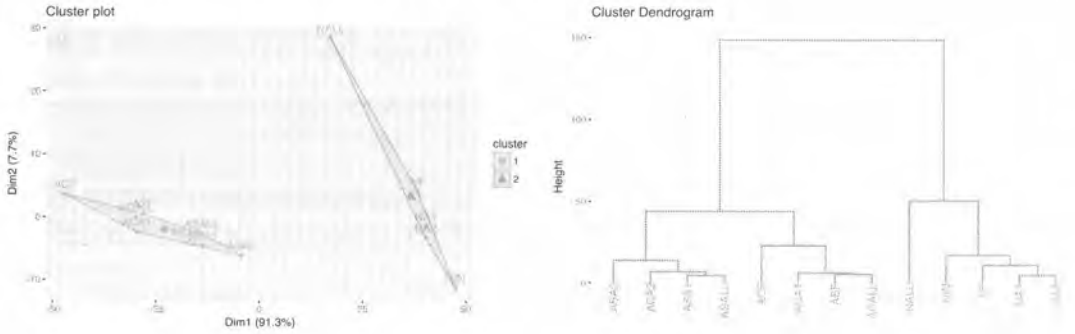

图 7.36 例5.1数据对变量分层聚类 (3 类) 的前两个主成分得分图 (左) 及树状图 (右)

7.7 图像色彩的聚类

7.7.1 对图像色彩聚类的例子

例 7.8. 梵 · 高油画 (VG.jpg) 这是梵 · 高油画《有房舍和农夫的景色》的图片 (见图7.37).

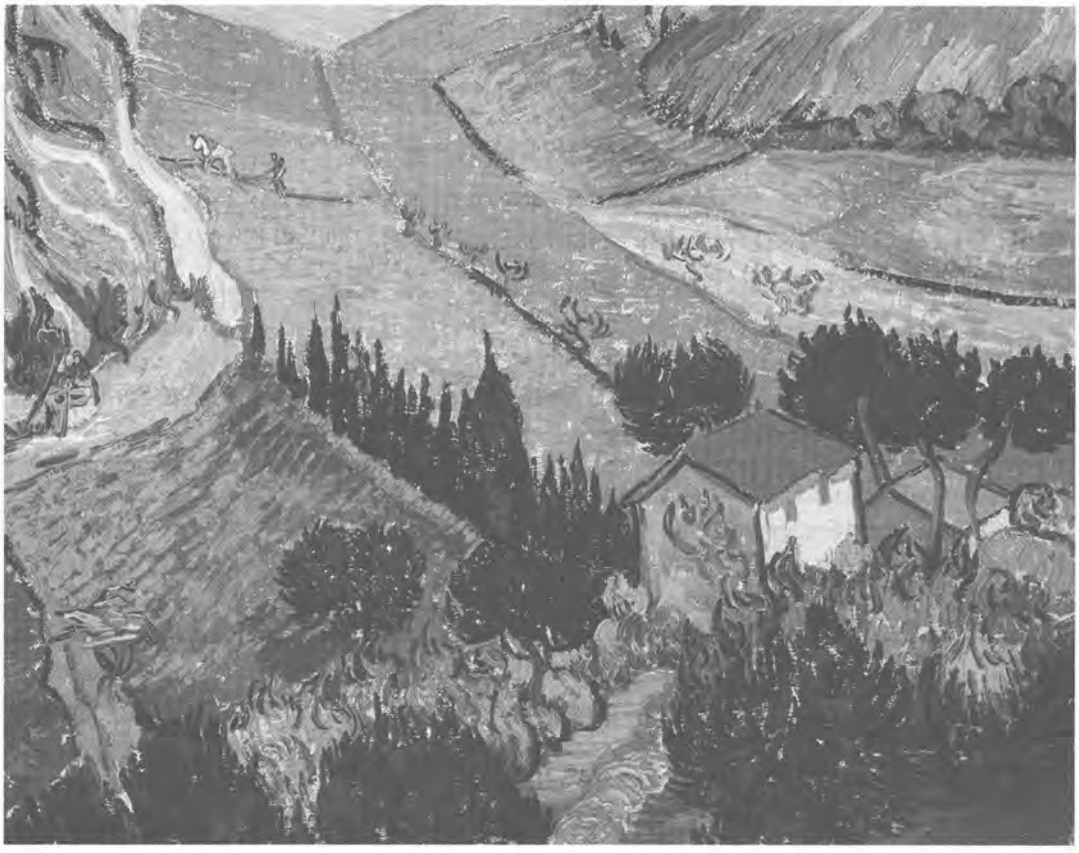

图 7.37 梵 · 高《有房舍和农夫的景色》原图

下面我们用画家梵 · 高 (Vincent Willem van Gogh) (1853—1890) 在 1889 年画的油画《有房舍和农夫的景色》(Landscape with house and ploughman) 来展示在颜色上做 K 均值聚类的效果. 原图是彩色的, 在这里用黑白图的展示效果可能不会令人满意, 但读者可以用自己选择的图片来再现这里的聚类.

对颜色进行 K 均值聚类就是把一些相似的颜色组合起来 (调和起来), 形成 k 种颜色, 然后用这 k 种颜色来形成图像. 如果只有两种颜色, 则就有黑白画的效果 (这两种颜色不太可能是 "黑" 和 "白", 而是由原图像聚出来的两种颜色), 在图中不会出现这 k 种颜色之外的颜色.

这里对颜色进行 K 均值聚类所取的 k 分别等于 241632. 显然, 当 $k=2$ 时, 只有两种颜色, 如水墨画一样, 随着 k 的增加, 画面趋于原画的色彩 (见图7.38)(注意: 本书为黑白印刷, 彩色无法显示).

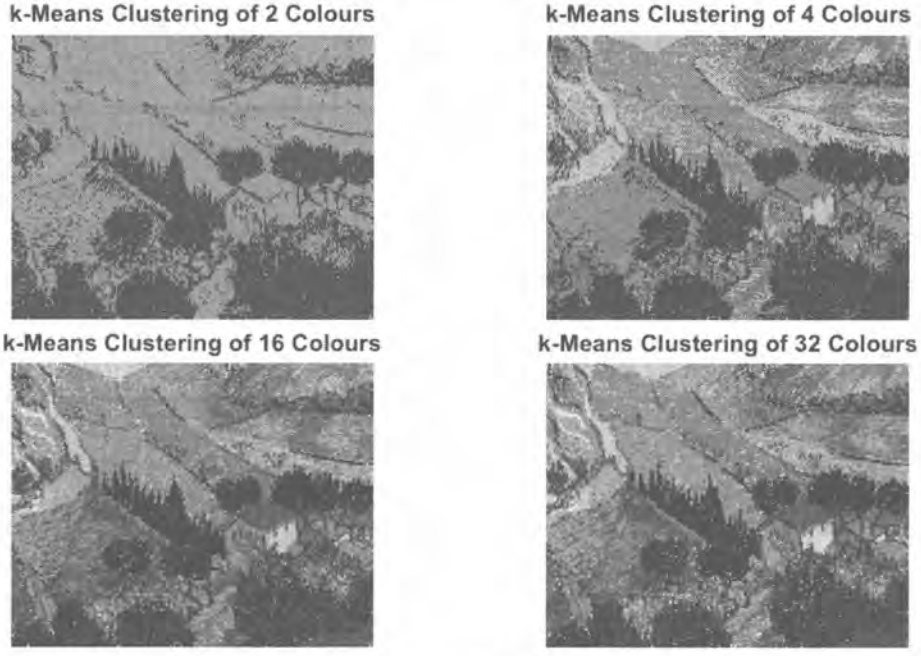

图 7.38 梵 · 高《有房舍和农夫的景色》油画关于颜色的 K 均值聚类 (k =2、4、16、32: 按照左上、右上、左下、右下的次序)

产生图7.38的代码为:

```
library(jpeg)
VG=readJPEG("VG.jpg")
imgDm=dim(VG)
VG_rgb <- data.frame(
x = rep(1:imgDm[2], each = imgDm[1]),
y = rep(imgDm[1]:1, imgDm[2]),
R = as.vector(VG[,,1]),
G = as.vector(VG[,,2]),
B = as.vector(VG[,,3]))
```

```
par(mfrow=c(2,2))
for (k in c(2,4,16,32)){
VG_km=kmeans(VG_rgb[, 3:5], centers = k)
kc=rgb(VG_km$centers[VG_km$cluster,])
plot(y~x, data=VG_rgb, col=kc,asp=1,pch = ".",
axes=F,xlab="",ylab="",
main=paste("k-Means Clustering of", k, "Colours"))
}
```

原画颜色聚类后的这 4 幅图画分别用了 2、4、16 和 32 种颜色组合 (也称为调色板), 这些颜色是什么呢? 图7.39展示了这 4 个调色板, 图中每种颜色上面的数字为相应颜色的十六进制表示形式 (hex triplet 或 hexadecimal representation of the colours)(注意: 本书为黑白印刷, 彩色无法显示). 在这里我们用了程序包 `scales` [15], 相应的代码为:

```
library(scales)
par(mfrow=c(2,2))
for (k in c(2,4,16,32)){
  VG_km = kmeans(VG_rgb[, 3:5], centers = k)
  show_col(rgb(VG_km$centers))
}
```

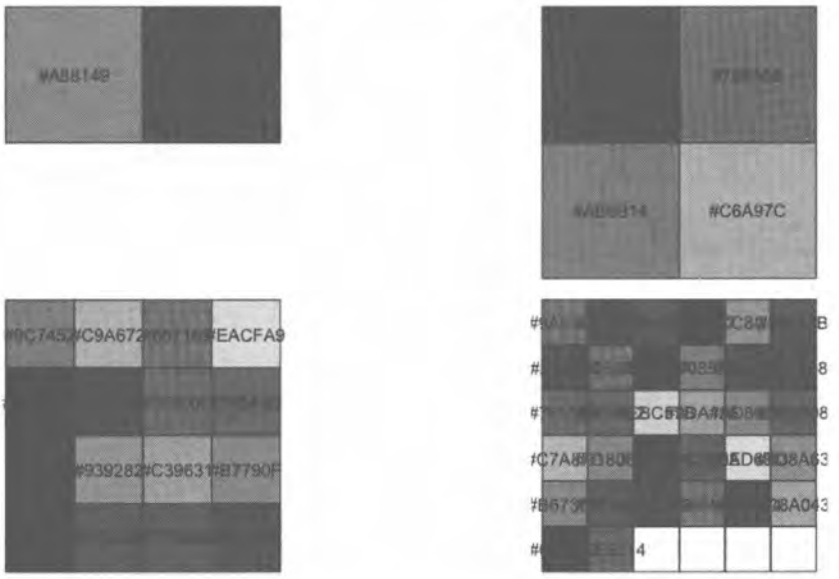

图 7.39 梵 · 高《有房舍和农夫的景色》油画聚类后的 4 个画面的调色板 (分别有 2、4、16、32 种颜色: 按照左上、右上、左下、右下次序)

[15]Hadley Wickham (2017). scales: Scale Functions for Visualization. R package version 0.5.0. https://CRAN.R-project.org/package=scales.

7.7.2 颜色的十六进制表示形式 *

各种颜色可以用 R(红)、G(绿)、B(蓝) 三种颜色调成 (注意, 这里的绿色实际上是柠檬色). 目前, 有很多种颜色标识系统, 主要有:

- RGB 或 RGBA 在 [0,1] 中的浮点数组系统 (比如 (0.1, 0.2, 0.5) 或 (0.1, 0.2, 0.5, 0.3), 表示各种颜色的比例).
- 十六位进制 RGB 或 RGBA 字符串 (如 '#0F0F0F' 或 '#0F0F0F0F').
- 在 [0,1] 中包括灰度级 (例如 "0.5") 的浮点值的字符串表示.
- 代表不同颜色的字符表 {'b', 'g', 'r', 'c', 'm', 'y', 'k', 'w'} 之一.
- X11/CSS4 颜色名字.
- 来自 xkcd 色彩调查的名字, 以 'xkcd' 为前缀 (例如 'xkcd: sky blue').
- 代表不同颜色的字符表:

```
'tab:blue', 'tab:orange', 'tab:green', 'tab:red', tab:purple', 'tab:brown',
'tab:pink', 'tab:gray', 'tab:olive', 'tab:cyan'
```

其中之一是 "T10" 类别调色板中的 Tableau 颜色 (默认的颜色循环).

上面的各种形式均在各种计算机程序中使用, 其中第二种为十六进制表示形式, 用 "#" 后面加上 6 个字符的形式表示各种颜色. 其中, 前 2 位数表示红色的强度, 中间 2 位表示绿色的强度, 最后 2 位表示蓝色的强度. 强度由 16 进制的两位数字表示. 十六进制和十进制的对照如下:

16 位数	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
10 位数	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

在十六进制的两位数中, 最大的是 FF, 最小的是 00. 因此, 所有颜色都最强的白光为 #FFFFFF, 最弱的黑色为 #000000, 红色为 #FF0000, 柠檬色为 #00FF00, 绿色为 #008000, 蓝色为 #0000FF, 黄色为 #FFFF00, 等等. 显然, 一共有 $16^6 = 16777216$ 种颜色.

例 7.9. 拉斐尔画 (Greek.jpg) 这是拉斐尔 (Raphael) 画的《雅典学院》(见图7.40的左上图).

图7.40对图片《雅典学院》(左上是原图片[16]) 做 $k = 2$ 的 K 均值聚类, 形成由两种颜色 (具体到这个图像是 #5B432B 和 #BEAB86) 组成的图 (右上), 然后把一种颜色转换成白色 (#FFFFFF), 另一种转换成黑色 (#000000), 因此形成下面左右两个黑白反转图片 (一个类似于另一个的底片)(注意: 本书为黑白印刷, 彩色无法显示).

产生图7.40原图 (左上) 之外的三个图的代码为:

```
img=readJPEG("Greek.jpg")
imgDm <- dim(img);imgDm
img_rgb <- data.frame(
```

[16]该图为意大利文艺复兴时期的画家拉斐尔 (Raphael) 在 1509—1511 年间创作的, 名为《雅典学院》, 英文为 "The School of Athens"(意大利文为: *Scuola di Atene*). 该画在梵蒂冈的使徒宫殿里名为 *Stanze di Raffaello* 的房间中被当作装饰画.

```
  x = rep(1:imgDm[2], each = imgDm[1]),
  y = rep(imgDm[1]:1, imgDm[2]),
  R = as.vector(img[,,1]),
  G = as.vector(img[,,2]),
  B = as.vector(img[,,3]))
  img_km = kmeans(img_rgb[, 3:5], centers = 2)
kc = rgb(img_km$centers[img_km$cluster,])
ukc=unique(kc)
kc1=kc;kc1[kc1==ukc[1]]="#000000";kc1[kc1==ukc[2]]="#ffffff"
kc2=kc;kc2[kc2==ukc[1]]="#ffffff";kc2[kc2==ukc[2]]="#000000"
plot(y~x,img_rgb,col=kc,asp=1,pch=".",axes=F,xlab="",ylab="")
plot(y~x,img_rgb,col=kc1,asp=1,pch=".",axes=F,xlab="",ylab="")
plot(y~x,img_rgb,col=kc2,asp=1,pch=".",axes=F,xlab="",ylab="")
```

图 7.40 左上图为《雅典学院》原图，右上图为把颜色聚两类的结果，左下图把两类颜色换成黑白，右下图为左下图黑白颜色的反转

7.8 本章计算的 Python 代码

首先，为分层聚类和 K 均值聚类导入必要的模块:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from __future__ import division #为了整数除法得到浮点值
from sklearn.cluster import AgglomerativeClustering, KMeans
```

```
import scipy.cluster.hierarchy
from scipy.cluster.hierarchy import dendrogram, linkage,
    set_link_color_palette
import seaborn as sns
%matplotlib inline
```

7.8.1 例7.2美国各州数据的聚类

例7.2美国各州数据的描述和热量图

可以使用下面代码描述数据, 并用模块 seaborn 的函数 snsclustermap 得到例7.2美国各州数据的热量图 (见图7.41).

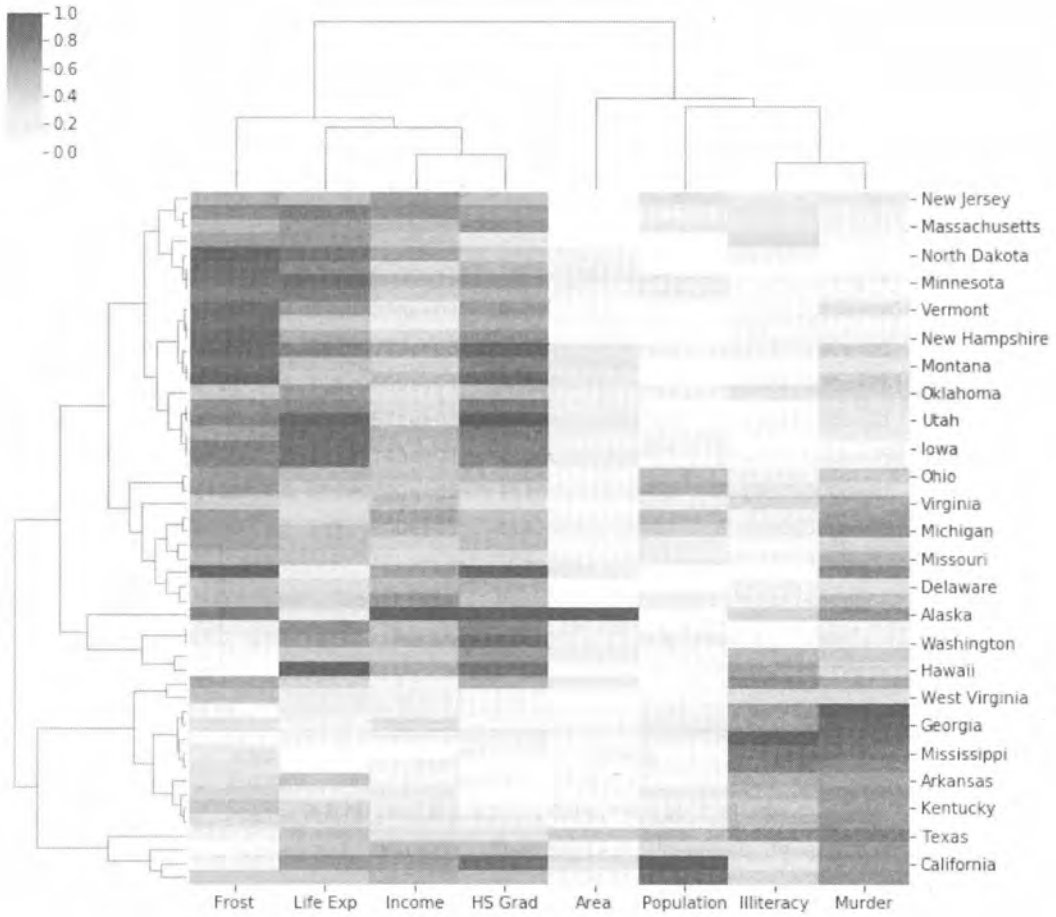

图 7.41 例7.2美国各州数据的热量图

描述数据并产生热量图 (见图7.41) 的代码为:

```
w=pd.read_csv("statex77.csv")
u=w[w.columns[1:]]
u.index=w[w.columns[0]] #把w的第一列变成index
del u.index.name #删去index的名字
u.head()
sns.clustermap(u, metric="correlation", method="complete", cmap="Blues",
    standard_scale=1)
```

上面代码 u.head() 得到的输出截图为:

	Population	Income	Illiteracy	Life Exp	Murder	HS Grad	Frost	Area
Alabama	3615	3624	2.1	69.05	15.1	41.3	20	50708
Alaska	365	6315	1.5	69.31	11.3	66.7	152	566432
Arizona	2212	4530	1.8	70.55	7.8	58.1	15	113417
Arkansas	2110	3378	1.9	70.66	10.1	39.9	65	51945
California	21198	5114	1.1	71.71	10.3	62.6	20	156361

例7.2美国各州数据的分层聚类

其实, 上面产生的图7.41已经包括对观测值和变量的分层聚类, 我们再在这里演示使用模块 scipy.cluster.hierarchy 的函数 dendrogram 和 linkage 做分层聚类, 并画树状图 (见图7.42).

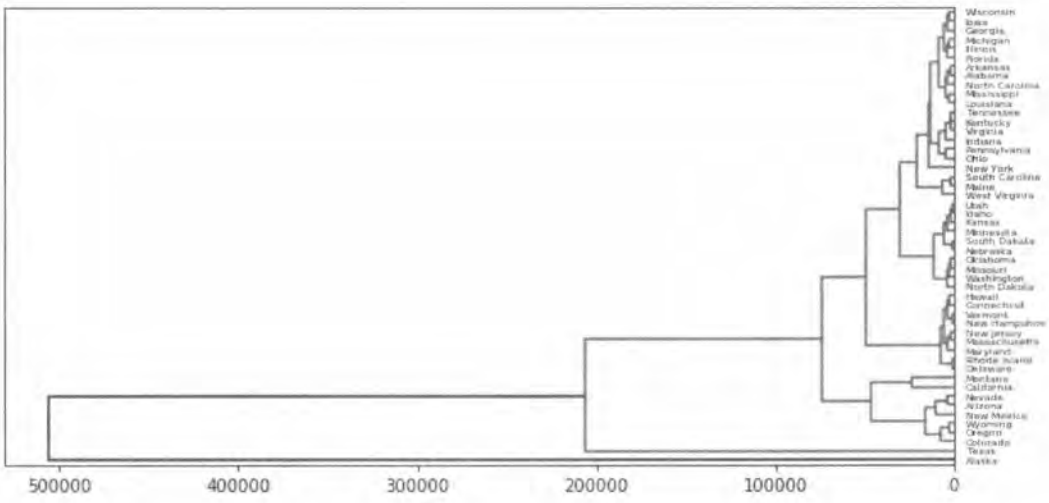

图 7.42 例7.2美国各州数据分层聚类的树状图

```
Z= linkage(u, method='average', metric='euclidean')
set_link_color_palette(['m', 'c', 'y', 'k'])
fig = plt.figure(figsize=(10, 5), dpi=72)
dendrogram(Z, labels=u.index, leaf_rotation=0, orientation="left",
leaf_font_size=6,color_threshold=None,above_threshold_color='blue')
```

```
set_link_color_palette(None)
#plt.show()
```

例7.2美国各州数据的 K 均值聚类

K 均值聚类很简单, 但需要确定 k 等于多少. 当然, 由于选择初始中心有随机性, 不同次数运行同一个程序的结果可能会不同. 如果确定随机种子, 按理说应该可以产生同样的结果, 但实践表明, 使用模块 `random` 中的 `random.seed` 函数不成功, 如果使用 `np.random.seed` 函数, 则可以产生同样的结果. 下面是把例7.2数据聚成 4 类的代码 (打印 4 个中心坐标及 4 类的州名):

```
np.random.seed(8888)
cent,label=scipy.cluster.vq.kmeans2(u, 4, iter=20, thresh=1e-05,
    minit='random', missing='warn', check_finite=True)
print("centroid:\n",cent)
for i in range(4):
    print(u.index[label==i])
```

下面输出的是 4 个中心坐标及各类所包含的州的名字:

```
centroid:
 [[ 5.51483333e+03   4.22925000e+03   1.34166667e+00   7.03629167e+01
    8.81666667e+00   4.82833333e+01   9.07083333e+01   5.01750833e+04]
 [ 6.30100000e+03   5.25150000e+03   1.85000000e+00   7.01050000e+01
    1.17500000e+01   5.70500000e+01   9.35000000e+01   4.14283000e+05]
 [ 2.96664286e+03   4.50078571e+03   8.28571429e-01   7.15457143e+01
    5.95714286e+00   6.02857143e+01   1.23500000e+02   1.01579000e+05]
 [ 2.58300000e+03   4.67740000e+03   1.10000000e+00   7.13370000e+01
    5.04000000e+00   5.38500000e+01   1.13000000e+02   8.19200000e+03]]
Index(['Alabama', 'Arkansas', 'Florida', 'Georgia', 'Illinois', 'Indiana',
       'Iowa', 'Kentucky', 'Louisiana', 'Maine', 'Michigan', 'Mississippi',
       'Missouri', 'New York', 'North Carolina', 'North Dakota', 'Ohio',
       'Oklahoma', 'Pennsylvania', 'South Carolina', 'Tennessee', 'Virginia',
       'Washington', 'Wisconsin'],
      dtype='object')
Index(['Alaska', 'Texas'], dtype='object')
Index(['Arizona', 'California', 'Colorado', 'Idaho', 'Kansas', 'Minnesota',
       'Montana', 'Nebraska', 'Nevada', 'New Mexico', 'Oregon', 'South Dakota',
       'Utah', 'Wyoming'],
      dtype='object')
Index(['Connecticut', 'Delaware', 'Hawaii', 'Maryland', 'Massachusetts',
       'New Hampshire', 'New Jersey', 'Rhode Island', 'Vermont',
       'West Virginia'],
```

```
        dtype='object')
```

7.8.2 例7.6种子数据的聚类

例7.6种子数据的描述

读入数据, 并做描述:

```
w=pd.read_csv("seeds.csv")
wnm=w.columns[:7] #除去ID
u=w[wnm]
u.describe()
```

得到:

	area	perimeter	compactness	length of kernel	width of kernel	asymmetry coefficient	length of kernel groove
count	210.000000	210.000000	210.000000	210.000000	210.000000	210.000000	210.000000
mean	14.847524	14.559286	0.870999	5.628533	3.258605	3.700201	5.408071
std	2.909699	1.305959	0.023629	0.443063	0.377714	1.503557	0.491480
min	10.590000	12.410000	0.808100	4.899000	2.630000	0.765100	4.519000
25%	12.270000	13.450000	0.856900	5.262250	2.944000	2.561500	5.045000
50%	14.355000	14.320000	0.873450	5.523500	3.237000	3.599000	5.223000
75%	17.305000	15.715000	0.887775	5.979750	3.561750	4.768750	5.877000
max	21.180000	17.250000	0.918300	6.675000	4.033000	8.456000	6.550000

计算相关系数, 并画相关图 (见图7.43).

```
cor = u.corr()
sns.heatmap(cor, square = True)
```

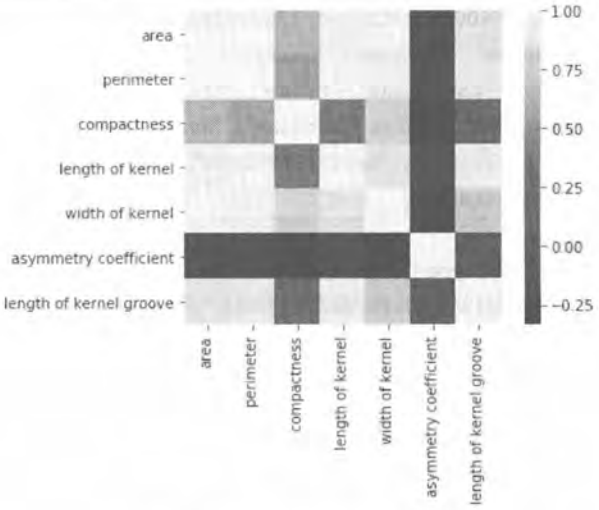

图 7.43 例7.6种子数据的相关图

例7.6种子数据分层聚类

下面使用模块 seaborn 的函数 snsclustermap 产生例7.6种子数据的热量图 (见图7.44). 在画图过程中, 实际上已经分别对观测值和变量做分层聚类.

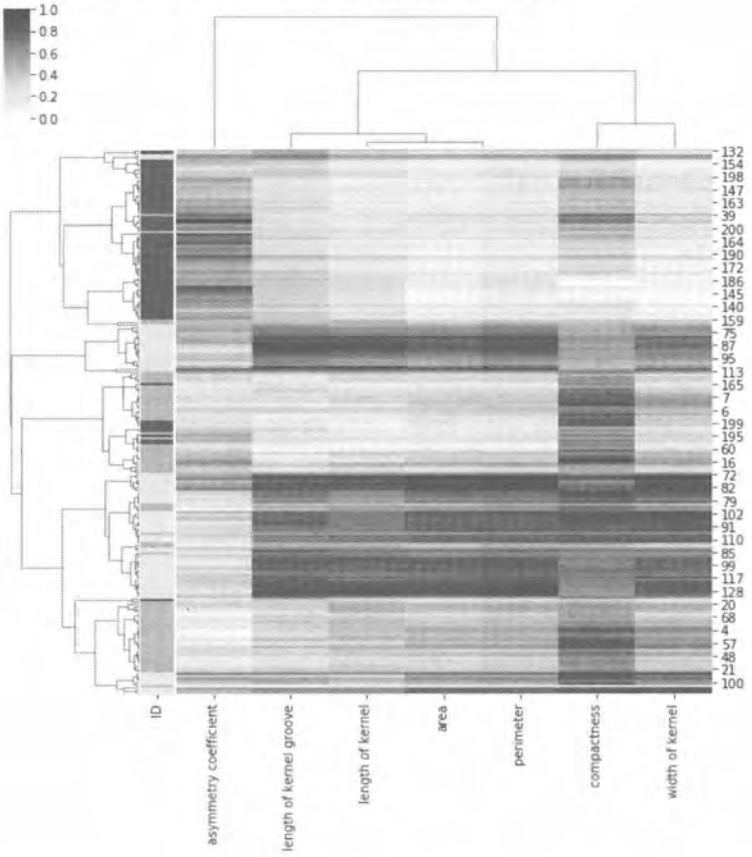

图 7.44 例7.6种子数据的热量图

```
my_palette = dict(zip(w.ID.unique(), ["orange","yellow","brown"]))
row_colors = w.ID.map(my_palette)

sns.clustermap(u, metric="correlation", method="complete", cmap="Blues",
  standard_scale=1, row_colors=row_colors)
```

首先, 使用模块 scipy.cluster.hierarchy 的函数 dendrogram 和 linkage 做分层聚类并画树状图 (见图7.45) 的代码为:

```
Z = linkage(u, 'ward')

fig = plt.figure(figsize=(20, 10), dpi=72)
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('sample index')
```

```
plt.ylabel('distance (Ward)')
dendrogram(Z, labels=u.index, leaf_rotation=90)
my_palette = plt.cm.get_cmap("Accent", 3)

w['ID']=pd.Categorical(w['ID'])
my_color=w['ID'].cat.codes

ax = plt.gca()
xlbls = ax.get_ymajorticklabels()
num=-1
for lbl in xlbls:
    num+=1
    val=my_color[num]
    lbl.set_color(my_palette(val))
```

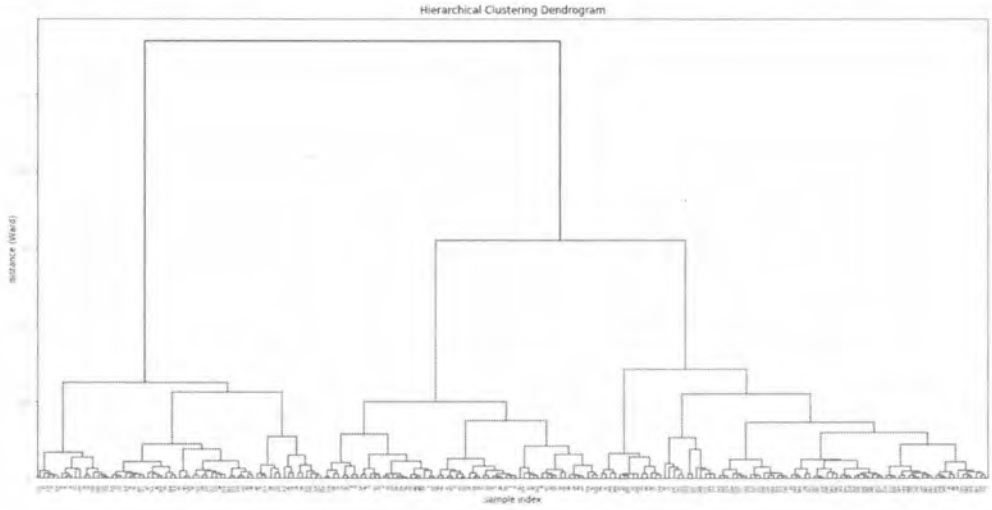

图 7.45 例7.6种子数据分层聚类树状图

现在, 改用模块 sklearn.cluster 的函数 AgglomerativeClustering 做分层聚类:

```
hc=AgglomerativeClustering(n_clusters=3, linkage='complete')
hc.fit(u)
hcl=hc.labels_.astype(np.int)
print ('1st original group','\n', hcl[:71])
print ('2nd original group','\n', hcl[71:141])
print ('3rd original group','\n', hcl[141:211])
```

得到对 210 个观测值的聚类, 并且验证原先的三个类别是否与聚类结果吻合:

```
1st original group
[2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 2 2 0 2 2 2 0 2 2 0 0 2 2 0 2 2 2 2 2 2
2 2 0 0 0 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 0 0 0 0 0 0 2 2 2 0 2]
```

```
2nd original group
[2 2 1 2 2 2 1 1 2 2 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 2 1 1 1 1 1 1 2
1 1 1 1 1 1 1 1 1 1 1 1 2 2 1 2 1 1 1 1 2 1 1 2 2 2 2 2 2 2 2 0]
3rd original group
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

虽然聚类结果和分类结果一般来说没有多少关系, 但在这个输出中, 它们似乎有些关系: 输出表明, 在原先第一群中 (ID=1), 大部分属于标记为 "2" 的群; 在原先第二群中 (ID=2), 大部分属于标记为 "1" 的群; 在原先第三群中 (ID=3), 大部分属于标记为 "0" 的群. 看来, 如果用聚类代替分类, 正确率居然有 80%. 当然, 如果用有监督学习的分类, 误差可能会小得多.

例7.6种子数据的 K 均值聚类

用模块 `sklearn.cluster` 的函数 `KMeans` 做 K 均值聚类的代码为:

```
kc=KMeans(n_clusters=3,random_state=1010)
kc.fit(u)
kcl=kc.predict(u)
print ('1st original group','\n', kcl[:71])
print ('2nd original group','\n', kcl[71:141])
print ('3rd original group','\n', kcl[141:211])
```

得到对 210 个观测值的聚类, 并且验证原先的三个类别是否和聚类结果吻合:

```
1st original group
[2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 2 1 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2
0 2 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 2 2 2 2 2 1 0]
2nd original group
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 2 0 0 0 0 0 0 2 2 2 2 0 2 2 2 2 1]
3rd original group
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1]
```

输出表明, 在原先第一群中 (ID=1), 大部分属于标记为 "2" 的群; 在原先第二群中 (ID=2), 大部分属于标记为 "0" 的群; 在原先第三群中 (ID=3), 大部分属于标记为 "1" 的群. 如果用聚类代替分类的话, 正确率居然达到 88.5%. 当然, 这里聚类和分类的结果比较类似并不是必然的, 不排除原先种子的分类就是用类似于聚类的思想得到的可能.

7.8.3 图像色彩的聚类

例 7.10. 雷诺阿画 (dance.jpg) 这是雷诺阿画的《红磨坊街的露天舞会》(见图7.46).

这里用一幅油画——法国印象画派的著名画家、雕塑家皮埃尔-奥古斯特·雷诺阿 (Pierre-

Auguste Renoir)(1841-1919) 1876 年画的《红磨坊街的露天舞会》(Bal du moulin de la Galette)(见图7.46) 来做色彩聚类的练习.

图 7.46 雷诺阿画作《红磨坊街的露天舞会》

我们试图把原图的色彩种类通过 K 均值聚类聚成 2、8、16、32 种, 产生的四个图像显示在图7.47中 (注意: 本书为黑白印刷, 彩色无法显示).

产生图7.47的代码如下:

```
from sklearn.cluster import MiniBatchKMeans
from PIL import Image
dance = Image.open("Dance.jpg")
d=np.array(dance)
d = d / 255.0
d = d.reshape(np.prod(d.shape[:2]), 3)
plt.figure(figsize=(26,22))
fig.subplots_adjust(wspace=0)
pp=np.array([2,8,16,32])
for i in range(4):
    K=220+i+1
    plt.subplot(K)
    k = MiniBatchKMeans(pp[i]);k.fit(d)
    new = k.cluster_centers_[k.predict(d)]
    dd = new.reshape(np.array(dance).shape)
```

```
imshow(dd)
```

图 7.47 雷诺阿《红磨坊街的露天舞会》的色彩种类通过 K 均值聚类聚成 2、8、16、32 种 (依次按从左到右的顺序) 的效果

7.9 习 题

1. 聚类中的点间距离和类间距离有很多种不同的选择, 考虑下面的问题:
   (1) 这些选择是数学推导的结果, 还是主观创造的结果?
   (2) 能不能说点间距离和类间距离的不同搭配有的好, 有的不好? 应该怎样叙述和解释这个问题?
2. 对于不同类型变量 (比如, 定性、定量、定序等变量) 混合的数据, 距离定义繁多且复杂, 你觉得有可能创造出新的度量方法吗? 试着做一做.
3. 本章介绍了许多聚类方法, 其中很多是通过某些奇特的低维 (可视) 反例想出来的, 你认为这些基于低维数据创造出来的方法可以推广到高维情况吗?
4. 在分层聚类中, 不同的选项会得到不同的聚类结果; 同一个分层聚类可以通过选择不同的聚类数目来产生不同的结果. 考虑下面的问题:
   (1) 你能说这些不同的结果有的合适, 有的不合适吗? 你能够证明吗?
   (2) 是不是不同的聚类结果体现了数据的不同特征?
   (3) 不同的聚类结果依赖于不同的变量选择, 变量选择是数学问题, 还是人的主观意志?

5. 没有因变量的 (属于无指导学习或无监督学习) 聚类分析和有因变量的 (属于有指导学习) 分类变量有什么区别? 请举几个例子说明.
6. 如果某一个物种的类别是用聚类分析确定的, 那么关于该物种的数据就会有因变量 (类别), 之后再对这类数据做分类, 结果是不是和聚类结果相似呢?
7. 请使用 R 和其他软件自带的鸢尾花 (iris) 数据. 该数据有 150 个观测值及 5 个变量, 其中 4 个表示鸢尾花物理尺寸: Sepal.Length、Sepal.Width、Petal.Length、Petal.Width (萼片长度、萼片宽度、花瓣长度、花瓣宽度), 它们在分类中作为自变量; 另一个是种类 (Species), 它作为因变量 (有三个水平: setosa、versicolor、virginica, 各有 50 个观测值).
   (1) 用全部 4 个自变量 (除了因变量 Species 的其他变量) 做分层聚类和 K 均值聚类, 解释结果.
   (2) 交替使用 3 个自变量 ($\binom{4}{3} = 4$ 种可能搭配) 做分层聚类和 K 均值聚类, 解释结果.
   (3) 交替使用 2 个自变量 ($\binom{4}{2} = 6$ 种可能搭配) 做分层聚类和 K 均值聚类, 解释结果.
   (4) 想象一下人们喜欢用鸢尾花数据将聚类和分类做比较的原因.
8. 回答下面问题:
   (1) 聚类分析是数学吗? 请解释.
   (2) 聚类分析可能有标准答案吗? 请解释.
   (3) 聚类分析是统计推断吗? 请解释.
   (4) 聚类分析的结果是具有确定性的吗?
   (5) 聚类分析的结果究竟应该用于什么目的?
9. 对例6.1做各种分层聚类 (选择不同的点间距离和类间距离组合, 并选择不同的聚类数目), 对各种结果做出你的解释.
10. 利用颜色聚类的方法把彩色图片转换成有两种颜色的图片 (比如, 黑白图片及其负片). 实际上, 你可以试着利用聚类把一彩色图片转换成有 k 个颜色的图片, 并把聚出来的 k 个颜色改成其他颜色以得到一些意想不到的效果.

第 8 章　典型相关分析

8.1　基本内容

8.1.1 问题的描述和例子

典型相关分析 (canonical correlation analysis, CCA) 用于识别和测量两组变量之间的关联. 当谈到两组变量之间的关系, 人们会想到多因变量的多元回归, 但是, 在因变量相关且自变量是数量变量的情况下, 典型相关性就是更合适的选择.

通常, 相关分析依赖于描述变量的坐标系. 这意味着, 即使在两组多元变量之间有很强的线性关系, 也有可能在一个坐标系中看不到其相关, 而它们在另一个坐标系中会显得很相关. 典型相关分析试图寻找对于相关分析最优的坐标系 (或者基向量).

典型相关分析的具体目标为确定一组**典型变量** (canonical variates)、这组变量是成对出现的, 各对变量正交; 每一对中的两个变量分别为两个变量集合的线性组合, 分别代表两个变量集合, 而且与其他线性组合相比, 这两个线性组合最相关. 典型相关分析解释了集合内部和集合之间的变化和关系.

典型相关分析和主成分分析之间的异同

典型相关分析是与主成分分析类似的多元探索性统计方法. 两种方法有相同的数学工具和背景 (矩阵代数和特征值分析), 均可以通过类似的图形表示来说明结果. 典型相关分析的主要目的是探索在相同实验单元上观察到的**两组**定量变量之间的样本相关性, 而主成分分析仅仅处理一个数据集以通过原始变量的线性组合来降低维度.

典型相关分析和因子分析之间的异同

典型相关分析所得到的各典型变量对是不可观测的, 勉强可以称为隐变量或潜变量, 但没有任何数学结构来描述它们, 这和主成分分析较接近. 因子分析的因子也是不可观测的隐变量, 但因子分析只考虑一组变量的代表 (即因子), 并以此降维; 典型相关分析则涉及两组变量的代表, 并研究它们之间的相关性, 这里也涉及对降维的考虑.

为了说明典型相关分析的基本概念, 首先引入一个例子.

例 8.1. 橄榄油数据 (oliveoil.csv) 该数据由从一组传感器中获得的关于 16 种橄榄油的 5 个属性以及 6 个物理化学质量参数的 11 个变量组成, 这 16 种油中的前 5 种产自希腊, 中间 5 种产自意大利, 最后 6 种产自西班牙. 该数据来自 Massart et al (1998).[1] 该数据包括由传感器获得的 5 个变量 Acidity、Peroxide、K232、K270、DK, 以及 6 个物理化学性质变量为 yellow、green、brown、glossy、transp、syrup.

[1] 该数据为 R 程序包 pls 所带, 可以通过该程序包得到.

考虑例8.1, 研究人员可能对橄榄油由传感器得到的一组变量与由物理化学性质组成的另一组变量如何相关感兴趣. 此外, 还要考虑需要几维的典型变量来理解这两组变量的关系才合适 (这和主成分分析选择主成分的个数类似).

如果用 $\boldsymbol{X}=(X_1,X_2,\ldots,X_5)^\top$ 及 $\boldsymbol{Y}=(Y_1,Y_2,\ldots,Y_6)^\top$ 表示例8.1数据的两组随机变量的向量, 那么作为隐变量的典型变量 V_i 和 W_i $(i=1,2,\ldots,5)$(图中只选择了 $i=1,2$ 的情况) 与这两组变量的关系可以用图8.1表示.

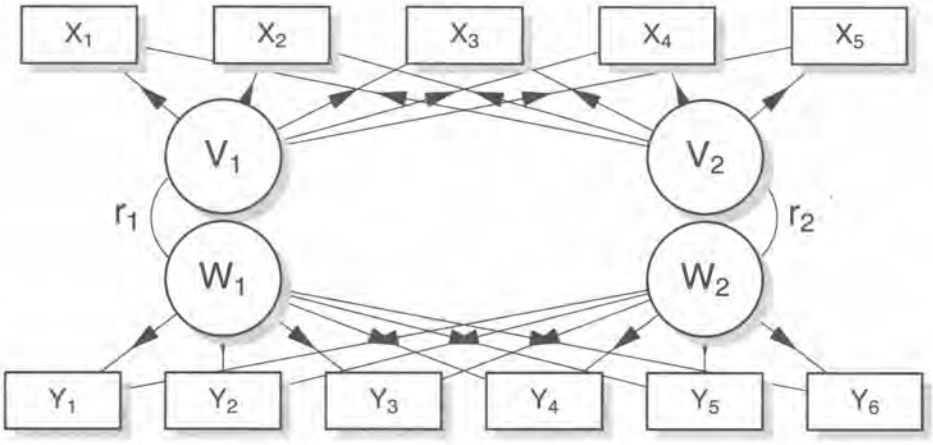

图 8.1 典型相关分析示意图

下面通过图8.1来认识典型相关分析的一些概念. 按照典型相关分析, 隐变量 V_i 和 W_i 分别是各自的可观察变量 $\boldsymbol{X}$ 和 $\boldsymbol{Y}$ 的线性组合, 它们之间的直线箭头所代表的关系称为**典型函数系数** (canonical function coefficients), 这些系数可以标准化或非标准化形式出现. $\boldsymbol{X}$ 的每个变量和每个 V (或者 $\boldsymbol{Y}$ 的每个变量和每个 W) 之间的相关系数称为**典型结构系数** (canonical structure coefficient) 或**典型载荷** (canonical loading); 每个可观察变量与对立的典型变量之间, 即 $\boldsymbol{X}$ 的每个变量和每个 W (或者 $\boldsymbol{Y}$ 的每个变量和每个 V) 之间的相关系数称为**典型交叉载荷** (canonical cross-loading). V_i 和 W_i 的相关系数 $r_i(i=1,2)$ 为**典型相关系数**; (V_i,W_i) 称为第 i **典型变量对** (canonical variate pair), 比如 (V_1,W_1) 为第一典型变量对, (V_2,W_2) 为第二典型变量对, 等等. 典型变量对的个数和两组变量中少的变量数目一样, 对于例8.1橄榄油数据, 最多只有 5 对.

对于例8.1橄榄油数据, 给出相关系数及两两散点图 (见图8.2), 产生图8.2的代码为:

```
w=read.csv("oliveoil.csv")
library(GGally)
ggpairs(w)
```

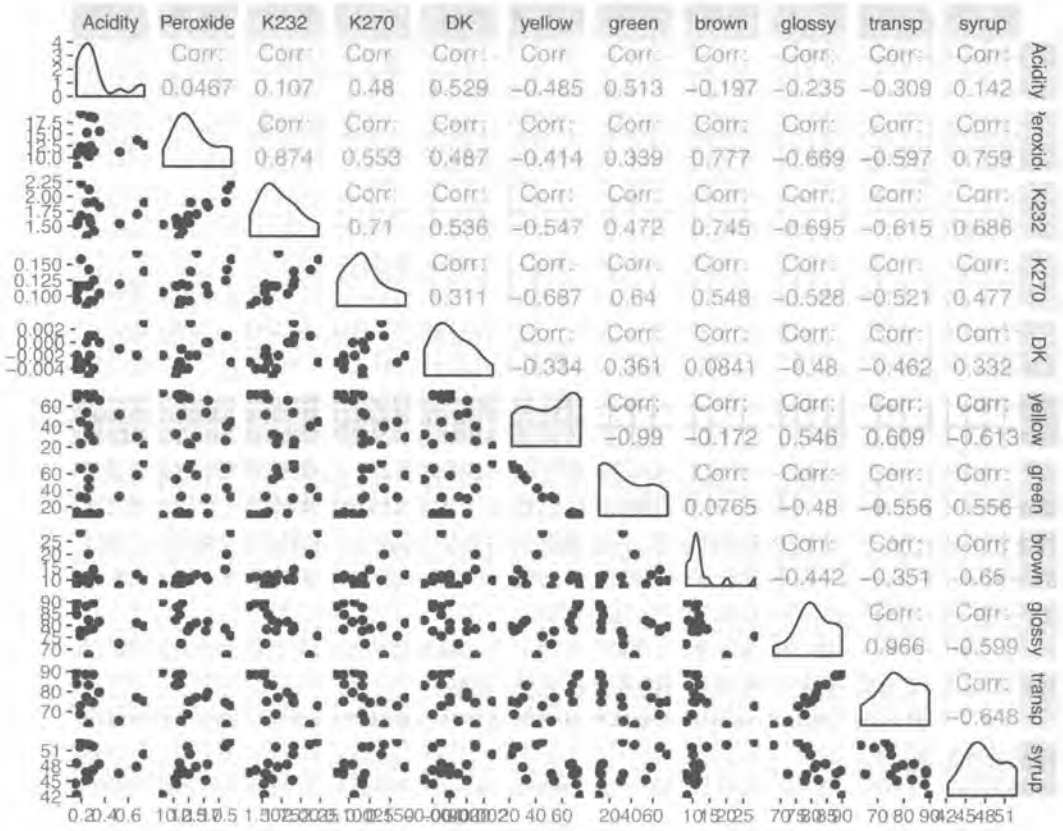

图 8.2 例8.1橄榄油数据变量的相关系数及两两散点图

图8.2展示的相关是两组变量中单个变量之间通常的相关, 而典型相关可以揭示这些简单相关不能反映的很多信息. 下面就通过例8.1橄榄油数据引入典型相关分析的各种概念.

8.1.2 典型相关及特征值问题

标准数理统计教科书涉及单独两个变量之间的线性相关, 而这里的**典型相关分析**则是研究两组变量之间的相关问题. 典型相关分析针对每组变量找到一组基, 而这两组基关于相关最优. 也就是说, 在这两组基上, 变量间的相关阵是对角线型的, 而且对角线上的相关是最大化的.

典型相关分析在变量的仿射变换下是不变的, 而通常的相关分析则依赖于基. 下面按照 Hotelling (1936) 的思路来介绍这个概念.

假定两组随机变量的向量形式为: $\boldsymbol{X} = (X_1, X_2, \ldots, X_p)^\top$ 及 $\boldsymbol{Y} = (Y_1, Y_2, \ldots, Y_q)^\top$, 它们的均值都为 0. 再把矩阵 $[\boldsymbol{X}^\top, \boldsymbol{Y}^\top]^\top$ 的协方差矩阵划分为下面的子矩阵块:

$$\boldsymbol{\Sigma} = \begin{bmatrix} \boldsymbol{\Sigma}_{XX} & \boldsymbol{\Sigma}_{XY} \\ \boldsymbol{\Sigma}_{YX} & \boldsymbol{\Sigma}_{YY} \end{bmatrix},$$

这里子矩阵 $\boldsymbol{\Sigma}_{XX} \equiv \mathrm{Cov}(\boldsymbol{X}, \boldsymbol{X})$ 是 $p \times p$ 维, $\boldsymbol{\Sigma}_{YY} \equiv \mathrm{Cov}(\boldsymbol{Y}, \boldsymbol{Y})$ 是 $q \times q$ 维, $\boldsymbol{\Sigma}_{XY} = \boldsymbol{\Sigma}_{YX}^\top \equiv \mathrm{Cov}(\boldsymbol{X}, \boldsymbol{Y})$ 是 $p \times q$ 维.

我们的目的是寻找基 (系数或权重) $\boldsymbol{a}=(a_1,a_2,\ldots,a_p)^\top$ 及 $\boldsymbol{b}=(b_1,b_2,\ldots,b_q)^\top$, 使得新的综合变量, 即典型变量

$$V=\boldsymbol{X}^\top\boldsymbol{a} \text{ 及 } W=\boldsymbol{Y}^\top\boldsymbol{b},$$

最大化 V 和 W 的相关系数:

$$\rho(\boldsymbol{a},\boldsymbol{b})=\frac{\boldsymbol{a}^\top\boldsymbol{\Sigma}_{XY}\boldsymbol{b}}{\sqrt{\boldsymbol{a}^\top\boldsymbol{\Sigma}_{XX}\boldsymbol{a}}\sqrt{\boldsymbol{b}^\top\boldsymbol{\Sigma}_{YY}\boldsymbol{b}}}.$$

显然, 这里有尺度不变性, 即对于任意 $c>0$, $\rho(c\boldsymbol{a},\boldsymbol{b})=\rho(\boldsymbol{a},c\boldsymbol{b})=\rho(\boldsymbol{a},\boldsymbol{b})$.

注意

这里假定各个变量的均值等于 0, 对中心化的数据都能满足. 这里所列的协方差阵及其子矩阵都是随机变量的理论协方差阵, 在实际应用中, 这些理论协方差阵或相关阵往往都用样本协方差阵及样本相关阵代替 (在不会混淆的情况下, 一般免去 "样本" 二字) 以得到标准化的结果. 标准化数据的协方差矩阵等于 (无论是否标准化) 数据的相关矩阵. 对于例8.1橄榄油数据, $\boldsymbol{\Sigma}_{XX}$ 是 5×5 维, $\boldsymbol{\Sigma}_{YY}$ 是 6×6 维, $\boldsymbol{\Sigma}_{XY}=\boldsymbol{\Sigma}_{YX}^\top$ 是 5×6 维.

这里的变量 V 和 W 就是图8.1中的隐变量, 它们是各自代表的变量组的线性组合; 这里要寻找的基 $\boldsymbol{a}$ 和 $\boldsymbol{b}$ 就是使得 V 和 W 最相关的线性组合系数.

由于对数据做典型相关分析时的软件都使用和特征值问题相关的方法, 下面是针对该方法合理性的数学推导.

有关的特征值问题解的推导

下面的推导过程请不要死记硬背, 要弄清楚结论是什么. 后面8.3节还介绍了许多看似不同的推导过程, 但它们的基本思想没有多少区别, 而且会得出完全一样的结论.

假定矩阵 $\boldsymbol{\Sigma}_{xy}$ 的秩为 k. 相关的特征值问题除了求出最大特征值 (记为 λ_1 个) 及相应的特征向量 (目标记为 $\boldsymbol{a}$, 实际结果为 $\boldsymbol{a}_1$) 之外, 会有另外的 $r-1$ 个特征值解 $(2\leqslant r\leqslant k)$, 所有这些特征向量称为典型特征向量. 由于尺度不变性, 可以加上下面的约束:

$$\boldsymbol{a}^\top\boldsymbol{\Sigma}_{XX}\boldsymbol{a}=1,\quad \boldsymbol{b}^\top\boldsymbol{\Sigma}_{YY}\boldsymbol{b}=1,$$

以及不同的特征列向量在下面意义下 (加权) 正交:

$$\boldsymbol{a}_i^\top\boldsymbol{\Sigma}_{XX}\boldsymbol{a}_j=0,\ \boldsymbol{b}_i^\top\boldsymbol{\Sigma}_{XX}\boldsymbol{b}_j=0\ (2<i\neq j\leqslant r\leqslant k),$$

在这种约束下, $\rho(\boldsymbol{a},\boldsymbol{b})=\boldsymbol{a}^\top\boldsymbol{\Sigma}_{XY}\boldsymbol{b}$, 我们可以求解

$$(\boldsymbol{a},\boldsymbol{b})=\arg\max_{\boldsymbol{a},\boldsymbol{b}}\rho(\boldsymbol{a},\boldsymbol{b})=\arg\max_{\boldsymbol{a},\boldsymbol{b}}\boldsymbol{a}^\top\boldsymbol{\Sigma}_{XY}\boldsymbol{b}. \tag{8.1.1}$$

满足这个条件的 $\boldsymbol{a}$ 和 $\boldsymbol{b}$ 为下面 Lagrange 乘数法的解:

$$\phi=\boldsymbol{a}^\top\boldsymbol{\Sigma}_{XY}\boldsymbol{b}-\frac{\lambda}{2}(\boldsymbol{a}^\top\boldsymbol{\Sigma}_{XX}\boldsymbol{a}-1)-\frac{\nu}{2}(\boldsymbol{b}^\top\boldsymbol{\Sigma}_{YY}\boldsymbol{b}-1),$$

这里的 λ 和 ν 为 Lagrange 乘数. 对 ϕ 求偏导数并令其等于 0:

$$\frac{\partial \phi}{\partial \boldsymbol{a}} = \boldsymbol{\Sigma}_{XY}\boldsymbol{b} - \lambda \boldsymbol{\Sigma}_{XX}\boldsymbol{a} = 0, \tag{8.1.2}$$

$$\frac{\partial \phi}{\partial \boldsymbol{b}} = \boldsymbol{\Sigma}_{XY}^{\top}\boldsymbol{a} - \nu \boldsymbol{\Sigma}_{YY}\boldsymbol{b} = 0, \tag{8.1.3}$$

上面两式分别左乘 $\boldsymbol{a}^{\top}$ 和 $\boldsymbol{b}^{\top}$, 利用条件 $\boldsymbol{a}^{\top}\boldsymbol{\Sigma}_{XX}\boldsymbol{a} = 1$ 及 $\boldsymbol{b}^{\top}\boldsymbol{\Sigma}_{YY}\boldsymbol{b} = 1$ 得到 $\lambda = \nu = \boldsymbol{a}^{\top}\boldsymbol{\Sigma}_{XY}\boldsymbol{b}$. 这时, 式 (8.1.2) 和式 (8.1.3) 可以写成:

$$\begin{bmatrix} -\lambda \boldsymbol{\Sigma}_{XX} & \boldsymbol{\Sigma}_{XY} \\ \boldsymbol{\Sigma}_{YX} & -\lambda \boldsymbol{\Sigma}_{YY} \end{bmatrix} \begin{bmatrix} \boldsymbol{a} \\ \boldsymbol{b} \end{bmatrix} = \boldsymbol{0}, \tag{8.1.4}$$

因为 $\lambda = \mathrm{Cor}(W, Y) > 0$ 是方程 (8.1.4) 的一个解, 因此式 (8.1.4) 左边矩阵的行列式必须等于 0:

$$\begin{vmatrix} -\lambda \boldsymbol{\Sigma}_{XX} & \boldsymbol{\Sigma}_{XY} \\ \boldsymbol{\Sigma}_{YX} & -\lambda \boldsymbol{\Sigma}_{YY} \end{vmatrix} = 0. \tag{8.1.5}$$

此外, 式 (8.1.4) 左边的矩阵可以写成:

$$\begin{bmatrix} -\lambda \boldsymbol{\Sigma}_{XX} & \boldsymbol{\Sigma}_{XY} \\ \boldsymbol{\Sigma}_{YX} & -\lambda \boldsymbol{\Sigma}_{YY} \end{bmatrix} = \begin{bmatrix} \boldsymbol{\Sigma}_{XX} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{1} \end{bmatrix} \begin{bmatrix} -\lambda \boldsymbol{I} & \boldsymbol{\Sigma}_{XX}^{-1}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1} \\ \boldsymbol{\Sigma}_{YX} & -\lambda \boldsymbol{I} \end{bmatrix} \begin{bmatrix} \boldsymbol{1} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{\Sigma}_{YY} \end{bmatrix}.$$

记 $\boldsymbol{A} = \boldsymbol{\Sigma}_{XX}^{-1}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1}\boldsymbol{\Sigma}_{YX}$, 于是行列式方程 (8.1.5) 成为

$$|\lambda^2 \boldsymbol{I} - \boldsymbol{A}| = 0. \tag{8.1.6}$$

对称地, 如果记 $\boldsymbol{B} = \boldsymbol{\Sigma}_{YY}^{-1}\boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{XX}^{-1}\boldsymbol{\Sigma}_{XY}$, 有类似的方程

$$|\lambda^2 \boldsymbol{I} - \boldsymbol{B}| = 0. \tag{8.1.7}$$

但是, 必须注意到有约束条件 $\boldsymbol{a}^{\top}\boldsymbol{\Sigma}_{XX}\boldsymbol{a} = 1$ 及 $\boldsymbol{b}^{\top}\boldsymbol{\Sigma}_{YY}\boldsymbol{b} = 1$. 我们做变换, $\boldsymbol{a} = \boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{c}$ 及 $\boldsymbol{b} = \boldsymbol{\Sigma}_{YY}^{-1/2}\boldsymbol{d}$ (关于实对称矩阵的指数幂请参看2.2.1节), 约束条件导致新的约束条件

$$\boldsymbol{c}^{\top}\boldsymbol{c} = \boldsymbol{d}^{\top}\boldsymbol{d} = 1,$$

及正交性

$$\boldsymbol{c}_i^{\top}\boldsymbol{c}_j = 0,\ \boldsymbol{d}_i^{\top}\boldsymbol{d}_j = 0\ (2 < i \neq j \leqslant r \leqslant k).$$

这里的 $\boldsymbol{c}$ 和 $\boldsymbol{d}$ 都是单位向量. 这样变换之后, 式 (8.1.5) 变成

$$\begin{vmatrix} -\lambda \boldsymbol{\Sigma}_{XX}\boldsymbol{\Sigma}_{XX}^{-1/2} & \boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1/2} \\ \boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{XX}^{-1/2} & -\lambda \boldsymbol{\Sigma}_{YY}\boldsymbol{\Sigma}_{YY}^{-1/2} \end{vmatrix} = 0. \tag{8.1.8}$$

于是, 类似于 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 导致式 (8.1.6) 和式 (8.1.7) 的过程, 可以定义 (如前面定义 $\boldsymbol{A}$ 和 $\boldsymbol{B}$):

$$\begin{aligned}\boldsymbol{N}_1 \equiv& \boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1}\boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{XX}^{-1/2};\\ \boldsymbol{N}_2 \equiv& \boldsymbol{\Sigma}_{YY}^{-1/2}\boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{XX}^{-1}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1/2},\end{aligned}$$

这样就有特征方程

$$|\lambda^2\boldsymbol{I}-\boldsymbol{N}_1|=0;\ |\lambda^2\boldsymbol{I}-\boldsymbol{N}_2|=0,$$

问题于是归结为解特征值问题

$$\boldsymbol{N}_1\boldsymbol{c}=\lambda^2\boldsymbol{c},\ \boldsymbol{N}_2\boldsymbol{d}=\lambda^2\boldsymbol{d},$$

而且两个问题各自不同的单位特征向量 ($\boldsymbol{c}$ 或 $\boldsymbol{d}$) 正交. 可以直接解矩阵 $\boldsymbol{N}_1$, $\boldsymbol{N}_2$ 的特征值问题, 这很容易. 另一种方法是做奇异值分解. 令 $\boldsymbol{K}=\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1/2}$, 由于 $\boldsymbol{N}_1=\boldsymbol{K}\boldsymbol{K}^\top$ ($\boldsymbol{N}_2=\boldsymbol{K}^\top\boldsymbol{K}$), 根据2.2.2节, 矩阵 $\boldsymbol{K}$ 的奇异值分解为:

$$\boldsymbol{K}=\boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{V}^\top,$$

这里 $\boldsymbol{U}$ 的列向量为 $\boldsymbol{N}_1$ 的特征向量 $\boldsymbol{c}_1,\boldsymbol{c}_2,\ldots,\boldsymbol{c}_k$, 而 $\boldsymbol{V}$ 的列向量为 $\boldsymbol{N}_2$ 的特征向量 $\boldsymbol{d}_1,\boldsymbol{d}_2,\ldots,\boldsymbol{d}_k$, $\boldsymbol{\Lambda}$ 为对角线矩阵, 其对角线元素为 $\lambda_1,\lambda_2,\ldots,\lambda_k$.

原问题式 (8.1.1) 的解向量矩阵于是为 $\boldsymbol{a}=\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{c},\boldsymbol{b}=\boldsymbol{\Sigma}_{YY}^{-1/2}\boldsymbol{d}$. 假定 $k=p<q$, 记 $p\times p$ 矩阵 $\boldsymbol{a}=(\boldsymbol{a}_1,\boldsymbol{a}_2,\ldots,\boldsymbol{a}_k)$ 以及 $q\times p$ 矩阵 $\boldsymbol{b}=(\boldsymbol{b}_1,\boldsymbol{b}_2,\ldots,\boldsymbol{b}_k)$(注意, 只有 k 个非零特征值). 对于 $i=1,2,\ldots,k$, 向量 $\boldsymbol{a}_i,\boldsymbol{b}_i$ 分别称为变量 $\boldsymbol{X}$ 和 $\boldsymbol{Y}$ 的**第 i 个典型相关向量**; 变量 $\boldsymbol{a}_i^\top\boldsymbol{X}$ 和 $\boldsymbol{b}_i^\top\boldsymbol{Y}$ 称为**第 i 个典型相关变量**, 也可称为X **记分**和Y **记分** (score); λ_i 称为**第 i 个典型相关系数**.

有关的特征值问题解的计算

注意软件输出的符号: 在多数软件输出的代码符号中, 是用 `X` 而不是我们前面所用的 V 来代表与 $\boldsymbol{X}$ 变量相联系的典型变量, 用 `Y` 而不是我们前面所用的 W 来代表与 $\boldsymbol{Y}$ 变量相联系的典型变量.

对于例8.1橄榄油数据, 求前面公式中的 $\boldsymbol{\Sigma},\boldsymbol{\Sigma}_{XX},\boldsymbol{\Sigma}_{YY},\boldsymbol{\Sigma}_{XY},\boldsymbol{\Sigma}_{YX},\ \boldsymbol{\Sigma}_{XX}^{-1/2},\boldsymbol{\Sigma}_{YY}^{-1/2}$ 以及 $\boldsymbol{K}$, 并且对 $\boldsymbol{K}$ 做奇异值分解得到典型相关向量 (即典型相关变量的系数) 矩阵 $\boldsymbol{a}$, $\boldsymbol{b}$ 及典型相关系数 λ_i 的计算代码为:

```
w=scale(read.csv("oliveoil.csv"))
S=cor(w)
Sxx=S[1:5,1:5];Syy=S[6:11,6:11]
Sxy=S[1:5,6:11];Syx=t(Sxy)
"%^%" <- function(x, n)
  with(eigen(x), vectors %*% (values^n * t(vectors)))
Sxx12=Sxx%^%(-1/2)
```

```
Syy12=Syy%^%(-1/2)
K=Sxx12%*%Sxy%*%Syy12
svd.k=svd(K)
c=svd.k$u;d=svd.k$v;lambda=svd.k$d
a=Sxx12%*%c;b=Syy12%*%d
print(list(xcoef=a,ycoef=b, cor=lambda))
```

得到典型相关向量矩阵 $\boldsymbol{a}$ ($xcoef)、$\boldsymbol{b}$ ($ycoef) 及典型相关系数 ($cor):

```
$xcoef
            [,1]       [,2]       [,3]       [,4]       [,5]
[1,]  0.40422238  1.2062037  0.3233250  1.4151168  0.2696271
[2,]  0.23004392  0.2914776 -0.8815697 -0.4727886  1.8081386
[3,]  1.07359187  1.5212306  0.4148186  1.2822569 -2.2022848
[4,] -0.05856915 -1.9739419  0.3809729 -0.9721128  0.5958676
[5,] -0.89532037 -1.4242593 -0.7713769 -0.3412694 -0.1440288

$ycoef
            [,1]       [,2]       [,3]       [,4]       [,5]
[1,] -4.80823821 -7.5691575 -5.9136126 -1.9686540  1.1970795
[2,] -4.16218282 -8.0906438 -5.0134632 -1.4478035  0.8366659
[3,]  0.51861972 -1.4950580  0.3315819 -1.1868987  0.3585611
[4,]  0.22875263 -0.9121876  2.9026630 -1.0259242  3.7252937
[5,] -0.04323115  1.5784883 -2.3258187  0.8384109 -3.6881515
[6,] -0.20249924  1.3325465 -1.3210546  0.5505988  0.3559670

$cor
[1] 0.9764811 0.8397163 0.8231290 0.5730970 0.2858563
```

这里的 5 个典型相关系数分别为 0.9765、0.8397、0.8231、0.5731、0.2859, 而在 $\boldsymbol{X}$ 和 $\boldsymbol{Y}$ 之间的两两相关系数最大的才只有 0.7774. 这说明了典型相关分析的基本含义.

注意, 对于例8.1橄榄油数据, 我们用 scale 函数把变量标准化了, 我们得到的系数也称为标准化典型系数 (standardized canonical coefficients). 如果我们不标准化数据, 而且不用样本相关阵而是用样本协方差阵 (用 R 函数 cov, 而不是 cor), 那么得到的就是一般的典型相关系数. 这种系数由于量纲不同而变化较大 (标准差很不同), 但只要非标准化典型系数列乘以相应变量的标准差, 这种系数则可以转换成标准化典型系数; 反之, 如果标准化系数矩阵的列除以相应变量的标准差, 这种系数则可以转换成非标准化典型系数. 当然, 非标准化典型系数大小的意义不大.

把例8.1橄榄油数据的标准化典型系数 (我们程序的代码为 a) 转换成非标准化系数的代码及结果为:

```
> X1=read.csv("oliveoil.csv")[,1:5]
> sqrt(1/diag(cov(X1)))*a
          [,1]       [,2]      [,3]      [,4]     [,5]
[1,]   2.28933    6.83138    1.8312    8.0146   1.5270
[2,]   0.06877    0.08713   -0.2635   -0.1413   0.5405
[3,]   4.31628    6.11597    1.6677    5.1552  -8.8541
[4,]  -2.47051  -83.26295   16.0698  -41.0047  25.1343
[5,] -400.39944 -636.94814 -344.9703 -152.6203 -64.4116
```

对于 $i=1,2,\ldots,k$, $\boldsymbol{a}_i^\top\boldsymbol{X}$ 和 $\boldsymbol{b}_i^\top\boldsymbol{Y}$ 称为第 i 个典型相关变量, 它们也称为记分 (score). 它们形成两个 $n\times k$ 矩阵, 即 $\boldsymbol{Xa}$ 和 $\boldsymbol{Yb}$. 对于例8.1橄榄油数据, 这是两个 16×5 矩阵. 计算两个记分矩阵的代码为:

```
X.score=X%*%a; Y.score=Y%*%b
```

8.1.3 载荷

各个原始变量和典型相关变量之间的相关系数称为**典型载荷** (canonical loading), 也就是对记分 $\boldsymbol{Xa}$ 和 $\boldsymbol{Yb}$ 与原始变量 $\boldsymbol{X}$ 和 $\boldsymbol{Y}$ 两两做相关, 一共得到 4 个相关矩阵. 对例8.1橄榄油数据的载荷计算代码如下:

```
corr.X.x=cor(X,X.score); corr.Y.x=cor(Y,X.score)
corr.X.y=cor(X,Y.score); corr.Y.y=cor(Y,Y.score)
print(list(corr.X.x=corr.X.x,corr.Y.x=corr.Y.x,
corr.X.y=corr.X.y,corr.Y.y=corr.Y.y))
```

得到 corr.X.x ($\boldsymbol{X}$ 和 $\boldsymbol{Xa}$ 相关阵)、corr.Y.x ($\boldsymbol{Y}$ 和 $\boldsymbol{Xa}$ 相关阵)、corr.X.y ($\boldsymbol{X}$ 和 $\boldsymbol{Yb}$ 相关阵)、corr.Y.y ($\boldsymbol{Y}$ 和 $\boldsymbol{Yb}$ 相关阵):

```
$corr.X.x
                [,1]       [,2]       [,3]        [,4]       [,5]
Acidity   0.02815200 -0.3174072  0.1012506 0.883538243  0.3279766
Peroxide  0.71917203 -0.1081867 -0.6683947 0.009837019  0.1556889
K232      0.79678169 -0.2600853 -0.4635335 0.147059513 -0.2469915
K270      0.74670584 -0.5966497  0.1027786 0.249583283  0.1165516
DK       -0.01283871 -0.4439427 -0.6885779 0.561444723 -0.1157168

$corr.Y.x
             [,1]       [,2]        [,3]       [,4]        [,5]
yellow -0.5392768  0.29315313 -0.02304563 -0.4108508 -0.03592752
green   0.4312430 -0.34135525  0.02807317  0.4263194  0.03949126
brown   0.7911537 -0.08004001 -0.29626072 -0.2530374  0.02732249
glossy -0.5346421  0.18992806  0.39512267 -0.2300659  0.01229651
transp -0.4782324  0.20384845  0.32937189 -0.2796669 -0.05197383
```

```
syrup   0.6432597  0.02151368 -0.41253091  0.1445997  0.13633609

$corr.X.y
                [,1]        [,2]        [,3]        [,4]        [,5]
Acidity   0.02748989 -0.26653198  0.08334232 0.506353079  0.09375418
Peroxide  0.70225787 -0.09084613 -0.55017501 0.005637566  0.04450466
K232      0.77804223 -0.21839784 -0.38154785 0.084279360 -0.07060407
K270      0.72914411 -0.50101650  0.08460002 0.143035420  0.03331702
DK       -0.01253676 -0.37278592 -0.56678841 0.321762263 -0.03307837

$corr.Y.y
             [,1]        [,2]        [,3]       [,4]        [,5]
yellow -0.5522655  0.34910971 -0.02799760 -0.7168958 -0.12568383
green   0.4416296 -0.40651257  0.03410543  0.7438870  0.13815073
brown   0.8102090 -0.09531791 -0.35992017 -0.4415263  0.09558122
glossy -0.5475192  0.22618121  0.48002522 -0.4014432  0.04301642
transp -0.4897508  0.24275871  0.40014616 -0.4879922 -0.18181802
syrup   0.6587529  0.02562017 -0.50117409  0.2523128  0.47693925
```

从这些输出中可以看出: 在 X 变量中, 与第一个 X.score 最相关的就是 X.score 主要代表的 Peroxide、K232、K270; 在 Y 变量中, 与第一个 Y.score 最相关的是 brown (仅有的一个). 从所有输出中都可以看出, 所有的关系都是通过 X 变量的这三个变量及 Y 变量的这一个变量进行的, 其他变量之间的相关程度都不太大. 如果不做典型相关分析, 单凭变量之间的两两相关矩阵是不大容易得到这个结论的, 这体现了典型相关分析的重要意义.

8.1.4 典型变量的选择

崖底碎石图

通过解特征值问题可以得到 k ($\leqslant \min p, q$) 对典型变量, 它们都是隐变量. 但是, 和因子分析中的因子类似, 这些典型变量并不一定都显著. 对于到底哪些显著这个问题, 可以用不同的方式展示, 首先可以画崖底碎石图 (见图8.3), 代码为:

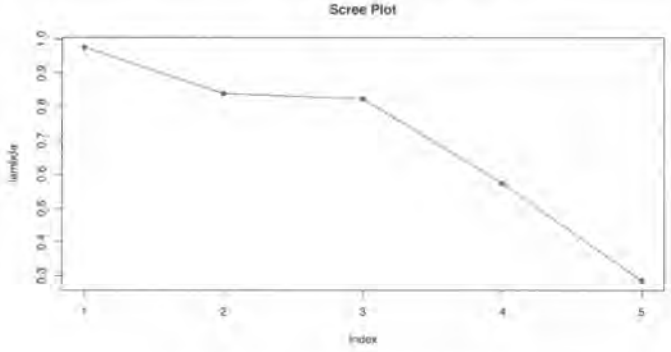

图 8.3 例8.1橄榄油数据崖底碎石图

```
plot(lambda,type = "o",main="Scree Plot")
```

从崖底碎石图来看, 前三个特征值之后才下降, 似乎降维并不明显.

直观上, 还可以用典型相关变量对的记分散点图来确定. 图8.4就是对例8.1前 4 对记分的散点图, 可以看出, 第二对及第三对变量之间的相关性虽不如第一对那么强, 但似乎差别不太大, 一直到第四对才开始下降, 这和崖底碎石图是一致的 (因为基于同样的相关机制).

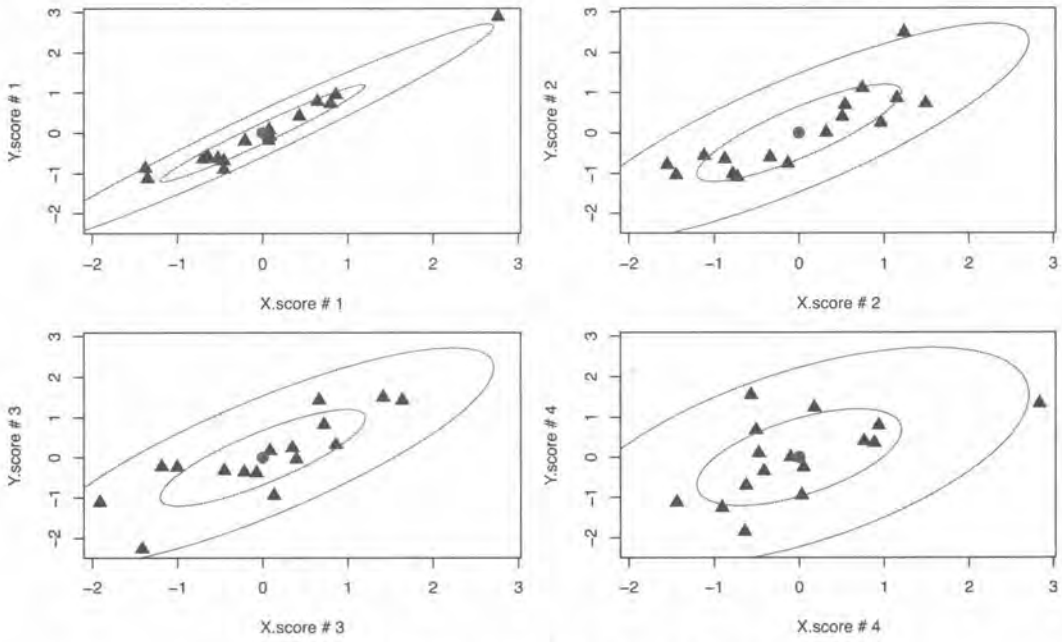

图 8.4 例8.1橄榄油数据前 4 对典型相关变量记分点图

产生图8.4的代码为:

```
par(mfrow=c(2,2))
for(i in 1:4){
  plot(X.score[,i],Y.score[,i],pch=17,cex=1.5,col=4,
    ylim=range(Y.score),xlim=range(X.score),
    xlab=paste("X.score #", i),ylab=paste("Y.score #", i))
  dataEllipse(X.score[,i],Y.score[,i],add = T,lwd=1,grid=F)
}
```

假设检验

除了直观的描述之外, 我们还可以用形式上的检验. 这种检验之一是关于典型相关的 Wilks Λ 检验 (Wilk's Lambda test).

典型相关系数为 $\lambda_1, \lambda_2, \ldots, \lambda_k$, Wilks 检验: $H_0 : \lambda_j = \cdots = \lambda_k = 0$. 这是对第 j 个到

第 k 个相关系数是否集体显著的检验. Wilks 检验统计量为:

$$\Lambda_j = \prod_{\ell=j}^{k}(1+\lambda_\ell)^{-1}, \quad j = 1, 2, \ldots, k.$$

关于 Wilks 检验统计量的分布有两种近似, 都需要原始数据变量的正态分布假定或者大样本 (样本量 n 被认为很大时) 假定. 一个统计量为:

$$X^2 = -\ln(\Lambda_j)(n-(p+q+3)/2), \quad j = 1, 2, \ldots, k,$$

它在正态分布和大样本的假定下有近似的 χ^2 分布, 自由度为 $(p-j+1)(q-j+1)$. 另一个统计量为:

$$F_j = \frac{1-\Lambda_j^{1/s_j}}{\Lambda^{1/s_j}} \frac{df_{2j}}{df_{1j}},$$

它在正态分布和大样本的假定下有近似的 F 分布, 自由度为 df_{1j}, df_{2j}. 上式中,

$$\begin{aligned} df_{1j} &= u_j v_j, \\ df_{2j} &= ks_j - \tfrac{u_j v_j}{2} + 1, \\ s_j &= \sqrt{\tfrac{u_j^2 v_j^2 - 4}{u_j^2 + v_j^2 - 5}}, \end{aligned}$$

式中, $u_j = p-j+1$, $v_j = q-j+1$.

除了 Wilks 检验之外, 还有下面的一些类似检验:

- Pillai-Bartlett 迹检验 (Pillai-Bartlett trace test):

$$\Lambda_{\text{Pillai}} = \sum_{\ell=1}^{k}(\lambda_\ell/(1+\lambda_\ell)).$$

- Lawley-Hotelling 迹检验 (Lawley-Hotelling trace test):

$$\Lambda_{\text{LH}} = \sum_{\ell=1}^{k}(\lambda_\ell).$$

- Roy 最大根检验 (Roy's greatest root test):

$$\Lambda_{\text{Roy}} = \max_{\ell}(\lambda_\ell).$$

对于例8.1橄榄油数据, 我们也可以做这些检验. 注意, 除了做 Roy 最大根检验, 软件还输出几个 p 值, 第一个包含所有的典型相关系数 $(\lambda_1, \lambda_2, \ldots,)$, 而第二个则没有 λ_1, 只有 $\lambda_2, \lambda_3, \ldots,$ 第三个则没有 λ_1, λ_2, 如此下去便可知道哪些典型变量是重要的. 我们的检验使用了程序包 CCP[2]中的 `p.asym` 函数和 `plt.asym` 函数. 针对例8.1橄榄油数据的代码为:

[2]Uwe Menzel (2012). CCP: Significance Tests for Canonical Correlation Analysis (CCA). R package version

```
stats=c("Wilks","Hotelling","Pillai","Roy")
ctest=function(lambda=lambda,stat=stat){
  res=p.asym(lambda, 16, 5, 6, tstat = stat)
  print(res)
  if (stat!="Roy")
    for(i in 1:2)
      plt.asym(res,rhostart=i)
}
par(mfrow=c(3,2))
for(i in 1:4)ctest(lambda=lambda,stats[i])
```

得到下面的 (部分) 输出和图8.5(图标题中的 rho 即为我们的 λ).

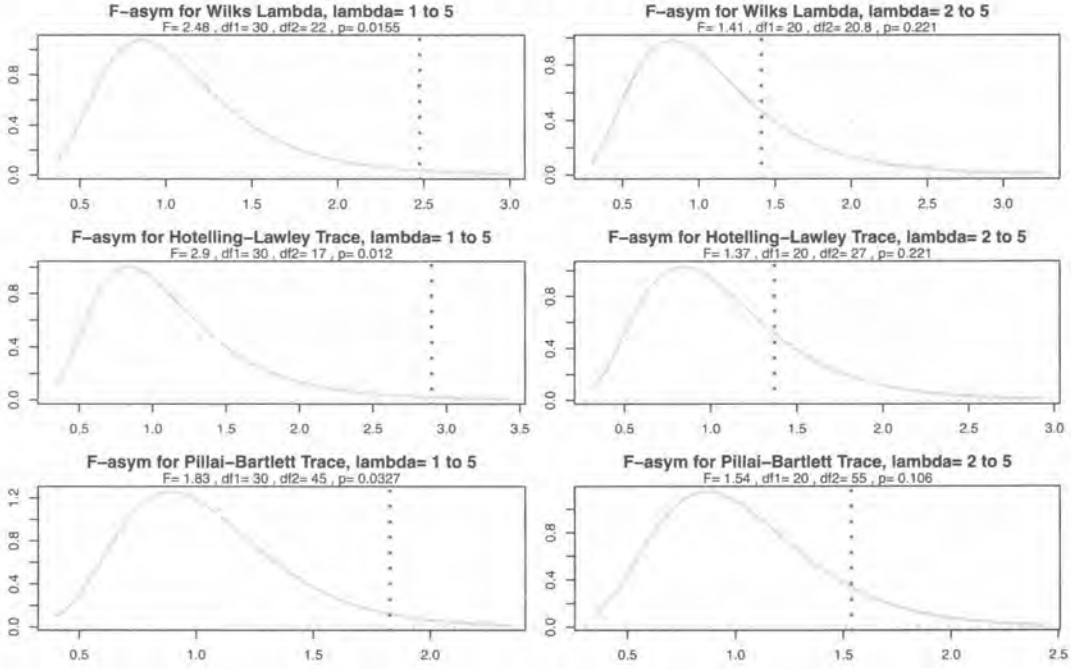

图 8.5 例8.1橄榄油数据的 Wilks、Hotelling、Pillai 检验分布图

```
 Wilks' Lambda, using F-approximation (Rao's F):
                 stat    approx df1      df2    p.value
1 to 5:  0.002725758 2.4761091  30 22.00000 0.01548473
2 to 5:  0.058637707 1.4092725  20 20.84962 0.22130513
3 to 5:  0.198855166 1.3188851  12 18.81176 0.28623002
4 to 5:  0.616684148 0.7290961   6 16.00000 0.63306931
5 to 5:  0.918286170 0.4004332   2  9.00000 0.68139854

 Hotelling-Lawley Trace, using F-approximation:
```

1.1. https://CRAN.R-project.org/package=CCP.

```
                stat    approx df1 df2    p.value
1 to 5:  25.58291688 2.8993972  30  17 0.01201576
2 to 5:   5.07047871 1.3690293  20  27 0.22066520
3 to 5:   2.67922802 1.6521906  12  37 0.11922623
4 to 5:   0.57805564 0.9056205   6  47 0.49906070
5 to 5:   0.08898515 0.5072153   2  57 0.60486214

 Pillai-Bartlett Trace, using F-approximation:
                stat    approx df1 df2    p.value
1 to 5:  2.74633406 1.8279111  30  45 0.03269236
2 to 5:  1.79281880 1.5372539  20  55 0.10575182
3 to 5:  1.08769526 1.5059365  12  65 0.14490922
4 to 5:  0.41015395 1.1170145   6  75 0.36064928
5 to 5:  0.08171383 0.7061073   2  85 0.49643350

 Roy's Largest Root, using F-approximation:
              stat   approx df1 df2      p.value
1 to 1:  0.9535153 30.76866   6   9 1.665276e-05
```

从这些检验的结果可以看出, 第一对典型相关变量是显著的, 剩下的远没有那么显著. **这三个检验结果是一致的, 与我们从崖底碎石图 (见图8.3) 及成对变量图 (见图8.4) 所得到的印象不大一样.**

8.2 实例计算: 例5.1教师数据

典型相关分析的过程和直接使用 R 程序包的代码

这里, 我们还是用例5.1教师数据来计算. 该数据的变量为: FICE (联邦 ID 代码)、College (学校名称)、State (州)、Type (类型: I、IIA 或 IIB)、ASF (正教授平均工资)、ASA1 (副教授平均工资)、ASA2 (助理教授平均工资)、ASALL (所有级别平均工资)、ACF (正教授平均补贴)、ACA1 (副教授平均补贴)、ACA2 (助理教授平均补贴)、ACALL (所有级别平均补贴)、NF (正教授数目)、NA1 (副教授数目)、NA2 (助理教授数目)、NIN (助教人数)、NALL (所有级别教授数目). 其中, 除了前 4 个变量之外、其余 13 个变量中有 8 个以字母 "A" 开头的变量是关于收入的, 而后面 5 个以 "N" 开头的变量是关于人数的.

在我们的分析中, 用关于收入的 8 个变量形成第一组, 命名为 X, 而用关于人数的 5 个变量形成第二组, 命名为 Y. 读入数据, 并绘出各个变量之间显示相关系数的散点图 (见图8.6) 的具体 R 代码为:

```
w=read.csv("full.aaup.csv")
X=w[,5:12];Y=w[,13:17]
library(GGally)
ggpairs(w[,-c(1:4)])
```

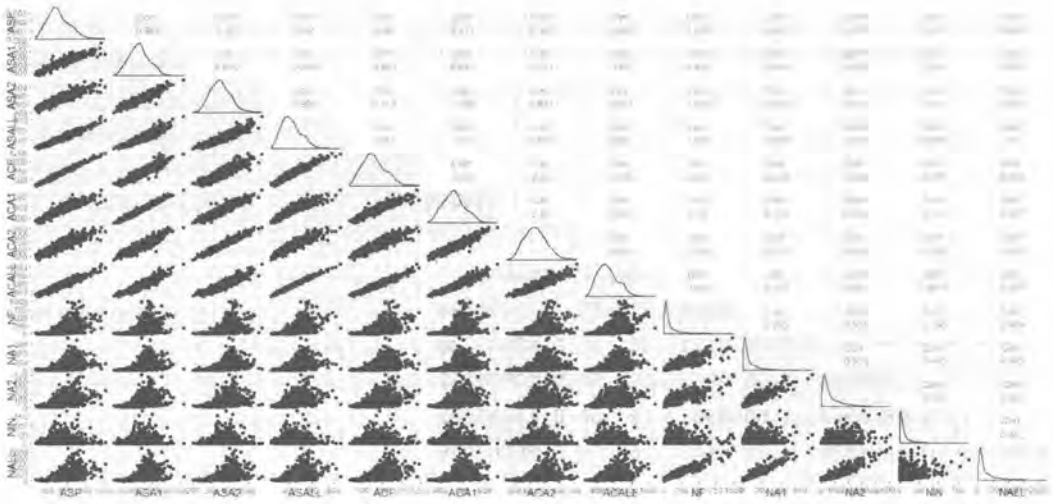

图 8.6 例5.1教师数据变量间的相关系数散点图

图8.6所包含的信息也可以用 R 的基本函数代码得到:

```
pairs(w[,-(1:4)])#不显示图
cor(w[,-(1:4)])
```

输出为:

```
        ASF ASA1 ASA2 ASALL  ACF ACA1 ACA2 ACALL   NF  NA1  NA2   NIN NALL
ASF    1.00 0.95 0.92 0.970 0.99 0.93 0.90 0.959 0.58 0.56 0.55 0.151 0.58
ASA1   0.95 1.00 0.94 0.945 0.94 0.98 0.93 0.940 0.49 0.50 0.48 0.137 0.50
ASA2   0.92 0.94 1.00 0.932 0.91 0.93 0.97 0.924 0.53 0.53 0.51 0.150 0.54
ASALL  0.97 0.94 0.93 1.000 0.96 0.93 0.92 0.989 0.61 0.54 0.51 0.078 0.57
ACF    0.99 0.94 0.91 0.965 1.00 0.95 0.92 0.972 0.55 0.54 0.52 0.131 0.55
ACA1   0.93 0.98 0.93 0.932 0.95 1.00 0.95 0.952 0.47 0.47 0.46 0.114 0.48
ACA2   0.90 0.93 0.97 0.918 0.92 0.95 1.00 0.938 0.52 0.52 0.50 0.138 0.52
ACALL  0.96 0.94 0.92 0.989 0.97 0.95 0.94 1.000 0.58 0.52 0.49 0.067 0.55
NF     0.58 0.49 0.53 0.606 0.55 0.47 0.52 0.583 1.00 0.89 0.86 0.356 0.96
NA1    0.56 0.50 0.53 0.543 0.54 0.47 0.52 0.524 0.89 1.00 0.92 0.453 0.96
NA2    0.55 0.48 0.51 0.509 0.52 0.46 0.50 0.487 0.86 0.92 1.00 0.536 0.95
NIN    0.15 0.14 0.15 0.078 0.13 0.11 0.14 0.067 0.36 0.45 0.54 1.000 0.48
NALL   0.58 0.50 0.54 0.570 0.55 0.48 0.52 0.548 0.96 0.96 0.95 0.482 1.00
```

可以看出, X 和 Y 各自变量的相关系数较高, 但 X 和 Y 之间的各个变量的相关系数都不高 (特别是助理教授人数 NIN 与各个 X 中变量的相关系数最大才为 0.1515).

我们下面用程序包 candisc 中的 cancor() 函数做典型相关分析 (基础包 stats 也有同名的 cancor() 函数):

```
library(candisc)
ccxy <- candisc::cancor(X, Y, set.names=c("X", "Y"))
ccxy
```

首先得到两个表格, 其中一个是 Wilks 检验:

```
Test of H0: The canonical correlations in the
current row and all that follow are zero

     CanR  WilksL       F df1    df2  p.value
1 0.78056 0.23178 49.8793  40 5006.8 0.000000
2 0.61359 0.59321 23.0663  28 4144.2 0.000000
3 0.15896 0.95141  3.2111  18 3253.2 0.000005
4 0.13848 0.97607  2.8044  10 2302.0 0.001857
5 0.06960 0.99516  1.4017   4 1152.0 0.231236
```

输出的第一列 CanR 是典型相关系数, 前两个典型相关系数为 0.78056 和 0.61359, 而原先不同变量组的最大两两变量的相关系数才为 0.60598. 因此, 典型相关分析还是比较有效的. 这些相关系数是 Wilks 检验统计量的基础.

在 Wilks 检验 (利用 F 分布的近似) 的输出中, 第一行显著意味着 5 个相关系数不都为零, 第二行显著意味着 2 ~ 5 个相关系数不都为零, 第三行显著意味着 3 ~ 5 个相关系数不都为零, 如此下去, 第五行不显著意味着第 5 个相关系数可能为零. 当然, 很难说清楚 p 值等于多少算显著, 因为检验的正态性及大样本的条件都无法验证. 至少前两个相关系数看起来不应该为零.

下面用代码 ccxy$coef 输出典型系数, 它描述了典型变量和原始变量的关系:

```
> ccxy$coef
$X
              Xcan1        Xcan2        Xcan3       Xcan4       Xcan5
ASF     0.016963852 -0.040067168  0.007297465 -0.11470545 -0.02401978
ASA1    0.025901456 -0.006295195 -0.014253894 -0.04787580 -0.04910495
ASA2    0.028768407  0.002510302 -0.107072276 -0.12205171  0.03424754
ASALL -0.072916653  0.030961194  0.117418253  0.29975319  0.03426430
ACF    -0.002550845  0.014985051  0.004173380  0.08806083  0.03635531
ACA1   -0.007620937  0.011870568 -0.001097879  0.06565083  0.03563339
ACA2   -0.015154427 -0.016994619  0.079665939  0.08684778 -0.05039470
ACALL   0.025373305 -0.005705357 -0.097659915 -0.24902536 -0.03259419

$Y
              Ycan1        Ycan2        Ycan3       Ycan4       Ycan5
NF    -0.027192753  0.015395283  0.005369037  0.04220860 -0.01350677
NA1   -0.010254915  0.008026918 -0.031742233  0.03793421 -0.02752996
NA2   -0.005381446 -0.003136049  0.009393578  0.06567833  0.01513126
```

```
NIN   0.003907590  0.009239250  0.029536323  0.02838972 -0.06000367
NALL  0.014290459 -0.011278153  0.003483731 -0.04434300  0.01137371
```

注意，很多人会对这些系数做如下解释：对于第二部分 $Y，在“其他变量不变”时，变量 NALL 增加一个单位会导致第一个典型变量 (Ycan) 增加 0.01429. 这种解释仅仅在这些变量都不相关时才有意义，在本例中，尤其是同组变量之间的各个变量大都非常相关，不可能出现一个变量改变而“其他变量不变”的情况，这种解释毫无意义. 在对回归系数的解释中，人们也往往犯同样的错误.

上面输出的系数 ($coef) 不是原始变量和典型变量的相关系数，相关系数为**典型载荷** (canonical loading)，可以用 ccxy$structure 得到. 这是两个典型变量记分 (score) 和各个变量之间的相关系数，一共有四组：

```
> ccxy$structure
$X.xscores
           Xcan1      Xcan2      Xcan3     Xcan4       Xcan5
ASF   -0.4544346 -0.7886903 -0.3218030 0.1638125  0.14373111
ASA1  -0.3637877 -0.6993803 -0.4786783 0.3143550 -0.03274614
ASA2  -0.3959974 -0.7454075 -0.4778059 0.1696141 -0.15077754
ASALL -0.6166055 -0.6557395 -0.3755615 0.1763660  0.04163205
ACF   -0.4487152 -0.7431428 -0.4021356 0.1669924  0.16001081
ACA1  -0.3685637 -0.6518970 -0.5496345 0.2891407 -0.02409015
ACA2  -0.4041268 -0.7091942 -0.4951086 0.1657195 -0.20137943
ACALL -0.5990892 -0.6258006 -0.4439802 0.1535103  0.03304767

$Y.xscores
           Xcan1      Xcan2        Xcan3        Xcan4        Xcan5
NF   -0.5294912 -0.4369932  0.013260070 -0.009432029 -0.010091534
NA1  -0.2967652 -0.5303820 -0.033275059  0.002200141 -0.017634057
NA2  -0.2087843 -0.5816665  0.011413201  0.020126856 -0.004125484
NIN   0.1833883 -0.3333179  0.072379740  0.011119991 -0.045950518
NALL -0.3654652 -0.5309606  0.004423195 -0.006130446 -0.011899622

$X.yscores
           Ycan1      Ycan2       Ycan3      Ycan4       Ycan5
ASF   -0.3547135 -0.4839301 -0.05115449 0.02268452  0.010003023
ASA1  -0.2839582 -0.4291306 -0.07609173 0.04353143 -0.002278980
ASA2  -0.3090998 -0.4573723 -0.07595305 0.02348791 -0.010493423
ASALL -0.4812976 -0.4023532 -0.05970005 0.02442291  0.002897399
ACF   -0.3502492 -0.4559827 -0.06392434 0.02312487  0.011136016
ACA1  -0.2876861 -0.3999955 -0.08737107 0.04003978 -0.001676564
ACA2  -0.3154453 -0.4351523 -0.07870353 0.02294860 -0.014015081
ACALL -0.4676252 -0.3839831 -0.07057604 0.02125788  0.002299966
```

```
$Y.yscores
          Ycan1      Ycan2      Ycan3      Ycan4      Ycan5
NF   -0.6783478 -0.7121944  0.08341653 -0.06811185 -0.14500290
NA1  -0.3801952 -0.8643958 -0.20932694  0.01588795 -0.25337965
NA2  -0.2674801 -0.9479773  0.07179823  0.14534278 -0.05927811
NIN   0.2349445 -0.5432285  0.45532690  0.08030119 -0.66025227
NALL -0.4682089 -0.8653388  0.02782546 -0.04427001 -0.17098289
```

前面三组典型变量在其他组中是最相关的. 从数字上看不如从图形上看直观, 图8.7显示了观测值在前三组典型变量中的记分的点图. 图8.7显示第三组记分基本上不相关.

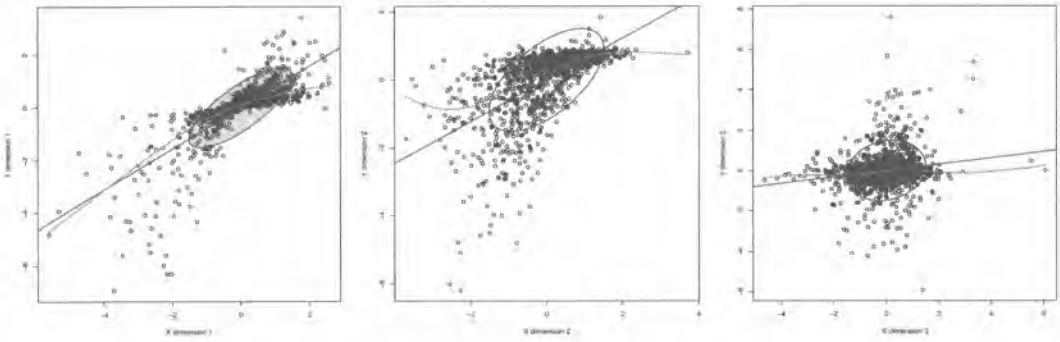

图 8.7 例5.1教师数据的观测值相应于前三组典型变量的记分图

图8.7是用下面代码产生的:

```
par(mfrow=c(1,3))
plot(ccxy, smooth=TRUE, id.n=3, ellipse.args=list(fill=TRUE))
plot(ccxy, which=2, smooth=TRUE)
plot(ccxy, which=3, smooth=TRUE)
par(mfrow=c(1,1))
```

图8.7所给予的观测值记分事实上可以用代码 `scores(ccxy,type='both')` 得到, 它是由两部分组成的 `list`, 如果令 `SC=scores(ccxy,type='both')`, 那么图8.7的左图可以由代码

```
plot(SC$X[,1],SC$Y[,1])
```

产生, 而中图可由代码

```
plot(SC$X[,2],SC$Y[,2])
```

产生, 等等. 图8.7中的直线和曲线分别是两个变量之间的线性及非参数 (loess) 回归拟合线.

通过下面代码还可以在第一、二个典型变量坐标中显示前面输出中四组相关的情况 (见图8.8).

```
par(mfrow=c(2,2))
for(i in 1:4){
  t=ccxy$structure[[i]];rg=range(t)+c(-.1,.1)
  plot(t[,1:2],type="n",xlim=rg,ylim=rg)
  text(t[,1:2],rownames(t))
  arrows(0,0,t[,1],t[,2],col=4,angle = 5)
  abline(h=0,lty=2);abline(v=0,lty=2)
}
```

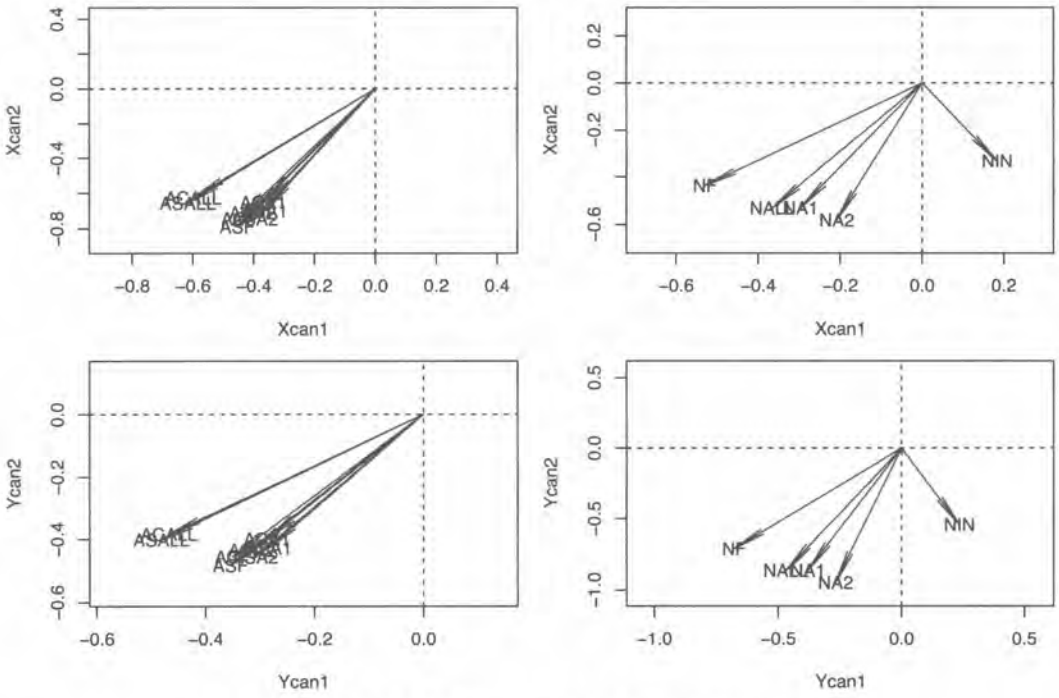

图 8.8 例5.1教师数据在第一、二个典型变量坐标中显示四组相关的情况

8.3 典型相关分析的不同角度推导 *

不同的作者对典型相关分析的出发点和方法的看法不大一样, 当然它们最终都涉及共同的特征值问题. 其实, 这些推导的思路都差不多, 但由于使用的数学语言不一样, 利用的现成结果的比例也不一样, 看上去大相径庭. 下面列举一些不同的途径, 相信读者可以看出来它们实际上异曲同工.

8.3.1 完全利用特征值问题的性质

8.1.2节求极值的问题式 (8.1.1) 等价于 $\max_{\boldsymbol{a},\boldsymbol{b}}(\boldsymbol{a}^\top \boldsymbol{\Sigma}_{XY}\boldsymbol{b})^2$. 首先, 固定 $\boldsymbol{a}$, 关于 $\boldsymbol{b}$ 求最大化, 即在 $\boldsymbol{b}^\top \Sigma_{YY}\boldsymbol{b} = 1$ 的约束下, 解

$$\max_{\boldsymbol{b}}(\boldsymbol{a}^\top \boldsymbol{\Sigma}_{XY}\boldsymbol{b})^2 = \max_{\boldsymbol{b}} \boldsymbol{b}^\top \boldsymbol{\Sigma}_{YX}\boldsymbol{a}\boldsymbol{a}^\top \boldsymbol{\Sigma}_{XY}\boldsymbol{b},$$

这是在 $\boldsymbol{b}^\top\boldsymbol{\Sigma}_{YY}\boldsymbol{b}=1$ 约束下的矩阵 $\boldsymbol{\Sigma}_{YX}\boldsymbol{a}\boldsymbol{a}^\top\boldsymbol{\Sigma}_{XY}$ 的广义特征值问题, 它等价于矩阵 $\boldsymbol{\Sigma}_{YY}^{-1}\boldsymbol{\Sigma}_{YX}\boldsymbol{a}\boldsymbol{a}^\top\boldsymbol{\Sigma}_{XY}$ 的特征值问题. 根据2.1.12节关于特征值的性质, 该特征值等于

$$\boldsymbol{a}^\top\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1}\boldsymbol{\Sigma}_{YX}\boldsymbol{a}. \tag{8.3.1}$$

下面, 关于 $\boldsymbol{a}$ 求式 (8.3.1) 的最大值, 约束还是式 (8.1.1) 中的约束, 令 $\boldsymbol{c}=\boldsymbol{\Sigma}_{XX}\boldsymbol{a}, \boldsymbol{d}=\boldsymbol{\Sigma}_{YY}\boldsymbol{b}$, 这样, 上述问题就成为在约束条件 $\boldsymbol{c}^\top\boldsymbol{c}=1$ 及 $\boldsymbol{c}_i^\top\boldsymbol{c}=1$ $(2\leqslant r\leqslant k)$ 下, 解

$$\max_{\boldsymbol{a}}\boldsymbol{a}^\top\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1}\boldsymbol{\Sigma}_{YX}\boldsymbol{a}=\max_{\boldsymbol{c}}\boldsymbol{c}^\top\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1}\boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{11}^{-1/2}\boldsymbol{c}=\max_{\boldsymbol{c}}\boldsymbol{c}^\top\boldsymbol{N}_1\boldsymbol{c}, \tag{8.3.2}$$

这里和8.1.2节一样, 定义

$$\boldsymbol{N}_1\equiv\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1}\boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{XX}^{-1/2},$$

对称地, 也定义

$$\boldsymbol{N}_2\equiv\boldsymbol{\Sigma}_{YY}^{-1/2}\boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{XX}^{-1}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1/2}.$$

根据2.3节, 问题式 (8.3.2) 的最大值为 $\boldsymbol{N}_1$ 的最大特征值, 记为 λ_1^2 (因为这是原来问题的平方), 而解 $\boldsymbol{c}$ 为相应于最大特征值的特征向量 $\boldsymbol{c}=\boldsymbol{c}_1$ (注意, $\boldsymbol{c}_1^\top\boldsymbol{N}_1\boldsymbol{c}_1=\boldsymbol{c}_1^\top\lambda_1\boldsymbol{c}_1=\lambda_1\boldsymbol{c}_1^\top\boldsymbol{c}=\lambda_1$). 对称地, 问题式 (8.3.2) 的最大值也是 $\boldsymbol{N}_2$ 的最大特征值, 即 λ_1^2, 而解 $\boldsymbol{d}$ 为相应于最大特征值的特征向量 $\boldsymbol{d}=\boldsymbol{d}_1$.

关于上面特征值问题的解完全和8.1.2节相同.

8.3.2 利用 Cauchy-Schwarz 不等式

8.1.2节给出的问题的解可以从不同的角度得到. 比如, 做变换[3]

$$\boldsymbol{c}=\boldsymbol{\Sigma}_{XX}^{1/2}\boldsymbol{a},\quad \boldsymbol{d}=\boldsymbol{\Sigma}_{YY}^{1/2}\boldsymbol{b},$$

于是, 相关系数为:

$$\rho(\boldsymbol{c},\boldsymbol{d})=\frac{\boldsymbol{c}^\top\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1/2}\boldsymbol{d}}{\sqrt{\boldsymbol{c}^\top\boldsymbol{c}}\sqrt{\boldsymbol{d}^\top\boldsymbol{d}}}.$$

根据 Cauchy-Schwarz 不等式 [4], 有

$$\boldsymbol{c}^\top\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1/2}\boldsymbol{d}\leqslant(\boldsymbol{c}^\top\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1}\boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{c})^{1/2}(\boldsymbol{d}^\top\boldsymbol{d})^{1/2},$$

这样, 使其相关系数最大的问题归结到二次型的极大值所关联的对称矩阵

$$\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1}\boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{XX}^{-1/2} \tag{8.3.3}$$

的特征值问题. 记该矩阵的非零特征值为 $\lambda_1^2\geqslant\lambda_2^2\geqslant\cdots\geqslant\lambda_k^2$ $(k\leqslant\min(p,q)$ (这些特征值

[3]对于 $\alpha\in\mathbb{R}$, 求对称矩阵 $\boldsymbol{M}$ 的 α 次幂 $\boldsymbol{M}^\alpha$: 做 Jordan 分解 (奇异值分解)$\boldsymbol{M}=\boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{U}^\top$, 则 $\boldsymbol{M}^\alpha=\boldsymbol{U}\boldsymbol{\Lambda}^\alpha\boldsymbol{U}^\top$; 这里 $\boldsymbol{\Lambda}^\alpha$ 为 $\boldsymbol{\Lambda}$ 的对角线元素取 α 次幂.

[4]对于向量 $\boldsymbol{u}$ 和 $\boldsymbol{v}$, $|\boldsymbol{u}^\top\boldsymbol{v}|\leqslant\|\boldsymbol{u}\|\|\boldsymbol{v}\|=(\boldsymbol{u}^\top\boldsymbol{u})^{1/2}(\boldsymbol{v}^\top\boldsymbol{v})^{1/2}$, 在这个问题中, $\boldsymbol{u}=\boldsymbol{c}^\top\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1/2}$, $\boldsymbol{v}=\boldsymbol{d}$.

和前面8.1.2节相同), 而相应的单位特征向量为 $\boldsymbol{c}_1, \boldsymbol{c}_2, \ldots, \boldsymbol{c}_k$ (这里 $\boldsymbol{c}_i^{\top}\boldsymbol{c}_i = 1$). 最大特征值的平方根 λ_1 为所求的最大相关系数. 这些特征值等于矩阵

$$\boldsymbol{\Sigma}_{YY}^{-1/2}\boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{XX}^{-1}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1/2} \tag{8.3.4}$$

的非零特征值, 记矩阵 (8.3.4) 的单位特征向量为 $\boldsymbol{d}_1, \boldsymbol{d}_2, \ldots, \boldsymbol{d}_k$ (这里 $\boldsymbol{d}_i^{\top}\boldsymbol{d}_i = 1$).

8.3.3 利用奇异值分解

下面的推导实际上省略了前面推导的许多叙述, 利用了较多的已有结论.

前面做了变换:

$$\begin{aligned}\boldsymbol{c} &= \boldsymbol{\Sigma}_{XX}^{1/2}\boldsymbol{a},\\ \boldsymbol{d} &= \boldsymbol{\Sigma}_{YY}^{1/2}\boldsymbol{b},\end{aligned}$$

或者

$$\begin{aligned}\boldsymbol{a}^{\top} &= \boldsymbol{c}^{\top}\boldsymbol{\Sigma}_{XX}^{-1/2},\\ \boldsymbol{b} &= \boldsymbol{\Sigma}_{YY}^{-1/2}\boldsymbol{d},\end{aligned}$$

在约束 $\boldsymbol{a}^{\top}\boldsymbol{\Sigma}_{XX}\boldsymbol{a} = 1$, $\boldsymbol{b}^{\top}\boldsymbol{\Sigma}_{YY}\boldsymbol{b} = 1$ 下求 $\boldsymbol{a}$ 和 $\boldsymbol{b}$, 解最大化问题 $\max_{\boldsymbol{a},\boldsymbol{b}} \boldsymbol{a}^{\top}\boldsymbol{\Sigma}_{XY}\boldsymbol{b}$ 就成为解下面的最大化问题:

$$\max_{\boldsymbol{c},\boldsymbol{d}} \boldsymbol{c}^{\top}\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1/2}\boldsymbol{d},$$

这里的 $\boldsymbol{c}$ 和 $\boldsymbol{d}$ 均为单位向量 ($\boldsymbol{c}^{\top}\boldsymbol{c} = 1, \boldsymbol{d}^{\top}\boldsymbol{d} = 1$). 令

$$\boldsymbol{K} = \boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1/2},$$

则 $\boldsymbol{K}\boldsymbol{K}^{\top}$ 和 $\boldsymbol{K}^{\top}\boldsymbol{K}$ 分别等于式 (8.3.3) 和式 (8.3.4) 的矩阵, 而那里的特征值问题就可以通过奇异值分解来得到结果.

令 $\boldsymbol{K}$ 的奇异值分解为 $\boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{V}^{\top}$, 则

$$\boldsymbol{\Lambda} = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_k \end{bmatrix},$$

这里的 k ($\leqslant \min(p, q)$) 为非零特征值的个数, 而 $\boldsymbol{U}$ 的列向量为相应于各个特征值的特征向量 $\boldsymbol{c}_1, \boldsymbol{c}_2, \ldots, \boldsymbol{c}_k$, $\boldsymbol{V}$ 的列向量为相应于各个特征值的特征向量 $\boldsymbol{d}_1, \boldsymbol{d}_2, \ldots, \boldsymbol{d}_k$.

8.3.4 利用广义特征值问题及 Cholesky 分解

问题式 (8.1.1) 等价于广义特征值问题:

$$\arg\max_{\boldsymbol{c},\boldsymbol{d}}\left(\frac{\mathrm{tr}(\boldsymbol{c}^\top\boldsymbol{\Sigma}_{XY}\boldsymbol{d})}{\sqrt{\mathrm{tr}(\boldsymbol{c}^\top\boldsymbol{\Sigma}_{XX}\boldsymbol{c})\mathrm{tr}(\boldsymbol{d}^\top\boldsymbol{\Sigma}_{YY}\boldsymbol{d})}}\right)$$

做 Cholesky 分解:

$$\begin{aligned}\boldsymbol{\Sigma}_{XX}&=\boldsymbol{L}_x\boldsymbol{L}_x^\top,\\ \boldsymbol{\Sigma}_{YY}&=\boldsymbol{L}_y\boldsymbol{L}_y^\top,\end{aligned}$$

对三角矩阵 $\boldsymbol{L}_x$ 及 $\boldsymbol{L}_y$ 求逆, 得到 $\boldsymbol{L}_x^{-1}$ 及 $\boldsymbol{L}_y^{-1}$, 它们相当于上面的 $\boldsymbol{\Sigma}_{XX}^{-1/2}$ 和 $\boldsymbol{\Sigma}_{YY}^{-1/2}$. 令

$$\boldsymbol{K}^*\equiv\boldsymbol{L}_x^{-1}\boldsymbol{\Sigma}_{XY}(\boldsymbol{L}_y^{-1})^\top,$$

它实际上相当于上面的 $\boldsymbol{K}$. 然后, 对 $\boldsymbol{K}^*$ 做奇异值分解, 问题就和前面一样了. 参看2.1.13节和2.2.4节.

注意: 在 R 中, 使用函数 `chol()` 的 Cholesky 分解得到的是上三角矩阵.

8.3.5 各种公式的总结

由于典型相关分析有若干等价的特征值问题的结论, 我们把几种计算方法的结论和各种指标之间的关系列举如下. 注意, 为了符号简洁, 下面所示的特征向量和特征向量矩阵都用同样的符号表示 (如 $\boldsymbol{\alpha},\boldsymbol{\beta},\boldsymbol{a},\boldsymbol{b},\boldsymbol{c},\boldsymbol{d}$), 而计算结果得到的是由 k ($\leqslant\min(p,q)$) 个特征值所对应的 k 个特征向量所组成的特征向量矩阵, 而且特征向量成比例也是指相应于同一个特征值的特征向量之间的关系. 相信读者不会混淆.

我们在前面定义了下面的一些矩阵:

$$\begin{aligned}\boldsymbol{A}=&\boldsymbol{\Sigma}_{XX}^{-1}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1}\boldsymbol{\Sigma}_{YX};\\ \boldsymbol{B}=&\boldsymbol{\Sigma}_{YY}^{-1}\boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{XX}^{-1}\boldsymbol{\Sigma}_{XY};\\ \boldsymbol{K}=&\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1/2};\\ \boldsymbol{N}_1\equiv&\boldsymbol{K}\boldsymbol{K}^\top=\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1}\boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{XX}^{-1/2};\\ \boldsymbol{N}_2\equiv&\boldsymbol{K}^\top\boldsymbol{K}=\boldsymbol{\Sigma}_{YY}^{-1/2}\boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{XX}^{-1}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1/2}.\end{aligned}$$

我们有下面结论:

- $\boldsymbol{A},\boldsymbol{B},\boldsymbol{N}_1,\boldsymbol{N}_2$ 有相同的非零特征值 $\lambda_1^2\geqslant\lambda_2^2\geqslant\cdots\geqslant\lambda_k^2$, 记 $\boldsymbol{A},\boldsymbol{B}$ 的特征向量矩阵分别为 $\boldsymbol{\alpha},\boldsymbol{\beta}$, 它们的列向量都是单位特征向量; $\boldsymbol{N}_1,\boldsymbol{N}_2$ 的特征向量矩阵分别为 $\boldsymbol{c},\boldsymbol{d}$, 而且 $\boldsymbol{c}^\top\boldsymbol{c}=\boldsymbol{I}$, $\boldsymbol{d}^\top\boldsymbol{d}=\boldsymbol{I}$.

- $\boldsymbol{a}=\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{c}, \boldsymbol{b}=\boldsymbol{\Sigma}_{YY}^{-1/2}\boldsymbol{d}$ 为前面问题式 (8.1.1) 的解, 而且

$$\boldsymbol{a}^{\top}\boldsymbol{\Sigma}_{XX}\boldsymbol{a}=\boldsymbol{I},\ \ \boldsymbol{b}^{\top}\boldsymbol{\Sigma}_{YY}\boldsymbol{b}=\boldsymbol{I}.$$

- 按列成比例: $\boldsymbol{\alpha}\propto\boldsymbol{\Sigma}_{XX}^{-1}\boldsymbol{\Sigma}_{XY}\boldsymbol{\beta},\ \boldsymbol{\beta}\propto\boldsymbol{\Sigma}_{YY}^{-1}\boldsymbol{\Sigma}_{YX}\boldsymbol{\alpha}$.
- 按列成比例: $\boldsymbol{c}\propto\boldsymbol{K}\boldsymbol{d},\ \ \boldsymbol{d}\propto\boldsymbol{K}^{\top}\boldsymbol{c}$.
- 按列成比例: $\boldsymbol{a}\propto\boldsymbol{\alpha},\ \ \boldsymbol{b}\propto\boldsymbol{\beta}$.
- $\boldsymbol{K}=\boldsymbol{\Sigma}_{XX}^{-1/2}\boldsymbol{\Sigma}_{XY}\boldsymbol{\Sigma}_{YY}^{-1/2}$ 的奇异值分解为:

$$\boldsymbol{K}=\boldsymbol{c}\begin{bmatrix}\lambda_1 & 0 & \cdots & 0\\ 0 & \lambda_2 & \cdots & 0\\ \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & \cdots & \lambda_k\end{bmatrix}\boldsymbol{d}^{\top}.$$

8.4 本章计算的 Python 代码

首先输入需要的模块及数据 (例5.1的教师数据), 和前面一样把和收入有关的一组变量 (以字母 "A" 开头) 称为 X, 而把与人数有关的一组变量 (以字母 "N" 开头) 称为 Y:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cross_decomposition import CCA

w=pd.read_csv("full.aaup.csv")
u=w[w.columns[4:]]
X=np.array(u[u.columns[:8]])
Y=np.array(u[u.columns[8:]])
```

下面引用典型相关分析的函数, 并进行拟合. 函数 CCA() 的变元均取默认值, 仅仅第一个变元 (n_components) 取 3, 这意味着只输出 3 对典型变量.

```
cc=CCA(3)
cc.fit(X,Y)
```

拟合之后, 得到的结果 (比如权重、载荷、观测值得分等) 可以用下面代码得到 (我们不展示输出):

```
print (cc.x_weights_)
print (cc.y_weights_)
print (cc.x_loadings_)
print (cc.y_loadings_)
print (cc.x_scores_)
```

```
print (cc.y_scores_)
```

产生类似于图8.7的图8.9的代码为:

```
fig=plt.figure(figsize=(30,10))
for i in range(3):
    fig.add_subplot(1,3,i+1)
    plt.scatter(cc.x_scores_[:,i],cc.y_scores_[:,i])
plt.show()
```

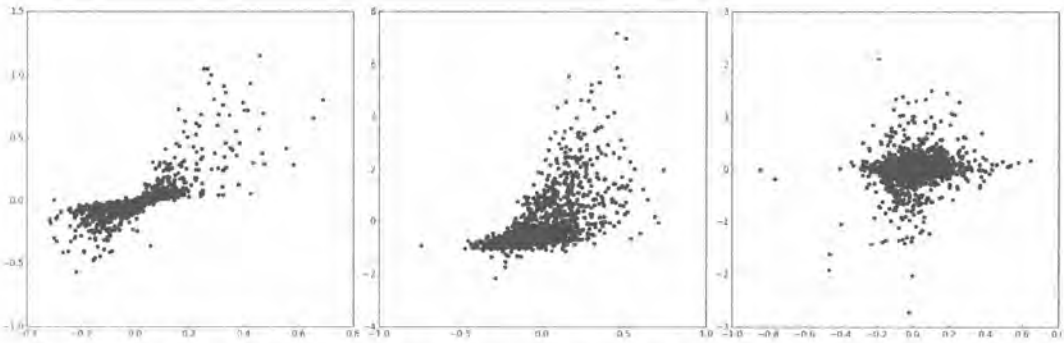

图 8.9 用 Python 产生的例5.1教师数据的观测值相应于前三组典型变量的记分图

图8.9和图8.7类似, 但可能因为算法的不同使得所得到的记分相差一个 (或正或负) 因子.

8.5 习 题

1. 请回答下面问题:
   (1) 典型相关分析所选择的少数典型变量对被假定代表各自变量组的所有信息, 还是部分信息? 从这一点来看, 它更像主成分分析还是因子分析?
   (2) 典型相关分析提供的信息和两组变量的两两相关得到的信息有什么不同? 由此说明典型相关分析的目的.
   (3) 典型相关分析的结论完全是由数学推导得到的吗? 其中有没有人为主观的决策? 如果有, 在哪里?
   (4) 对于典型相关分析本身有没有意义, 事先可以确定吗?
   (5) 对于典型相关分析典型变量的个数, 事先可以确定吗?
2. 有很多典型相关分析解的公式推导途径, 它们的数学本质完全是一样的.
   (1) 请用一个数据 (比如例8.1橄榄油数据) 根据公式编制程序来验证这些结果的等价性.
   (2) 请逐项比较用公式和方便的直接计算程序得到的结果 (这会是一个很好的编程训练).
3. 请说明典型相关分析和因子分析在模型上的异同.

第 9 章　对应分析

9.1　基本内容和应用

对应分析 (correspondence analysis, CA) 属于可视化探索性数据分析, 可描述二维和多维列联表数据, 在低维情况下可以得到数据的直观印象. 但由于对应分析不涉及分布及检验等经典统计 "要素", 因此有些多元统计书不予介绍.

在二维列联表情况下, 两个变量的对应反映在对各种组合的计数汇总上, 需要找出一个变量的某些水平和另一变量的某些水平是否相关联.

对应分析在概念上类似于主成分分析, 但在传统上多应用于分类变量数据, 特别是列联表显示的计数. 但由于其是描述性的, 只要意义清楚, 其他表格也可以应用.

二元对应分析涉及的数学并不复杂, 输出也就是一张图; 多元对应分析稍微不同, 但原理类似. 我们通过下面的例子来理解其意义.

例 9.1. 收入数据 (IncomeCA.csv) 这是程序包 `arules`[1]所带的数据, 原数据有 8993 个观测值及 14 个变量, 这里只取没有缺失值的 6876 个观测值以及 3 个变量, 并且合并了少数变量的水平. 我们取的变量为 income (收入, 水平为: `[0,10)`, `[10,20)`, `[20,30)`, `[30,50)`, `[50,75)`, `75+`, 单位: 千美元); education (教育, 水平为: `C<=3` (大学肄业), `CG` (大学毕业), `grade<=11` (最多 11 年级), `graduate` (研究生), `HSG` (中学毕业)); language.in.home (家中所用语言, 水平为: `english`, `spanish`, `other`).

9.2　二元对应分析

从例9.1收入数据中可以得到二维和三维的列联表, 下面利用程序包`ca`[2]的函数`ca()` 输出一个关于变量 education (教育) 和 income (收入) 的二维列联表:

```
w=read.csv("IncomeCA.csv")#读入数据
(ie=xtabs(~education+income,w))#生成二维表
library(ca)
(res=ca(ie))
library(factoextra)
fviz_ca_biplot(res,map ="rowprincipal",arrow = c(TRUE, TRUE))
```

下面展示二维列联表:

[1]Michael Hahsler, Bettina Gruen and Kurt Hornik (2011). arules: Mining Association Rules and Frequent Itemsets. R package version 1.0-7.

[2]Nenadic, O., Greenacre, M. (2007) Correspondence Analysis in R, with two- and three-dimensional graphics: The ca package. *Journal of Statistical Software*, 20(3):1-13.

```
                    income
education   [0,10) [10,20) [20,30) [30,50) [50,75) 75+
  C<=3         294      409      457      625      415 207
  CG            57      123      214      358      269 186
  grade<=11    634      116       66       80       39  28
  graduate      29       61       80      216      189 245
  HSG          241      325      328      351      157  77
```

二维列联表还可以直观显示 (见图9.1), 图中的圆圈大小和相应列联表格子中的计数成比例.

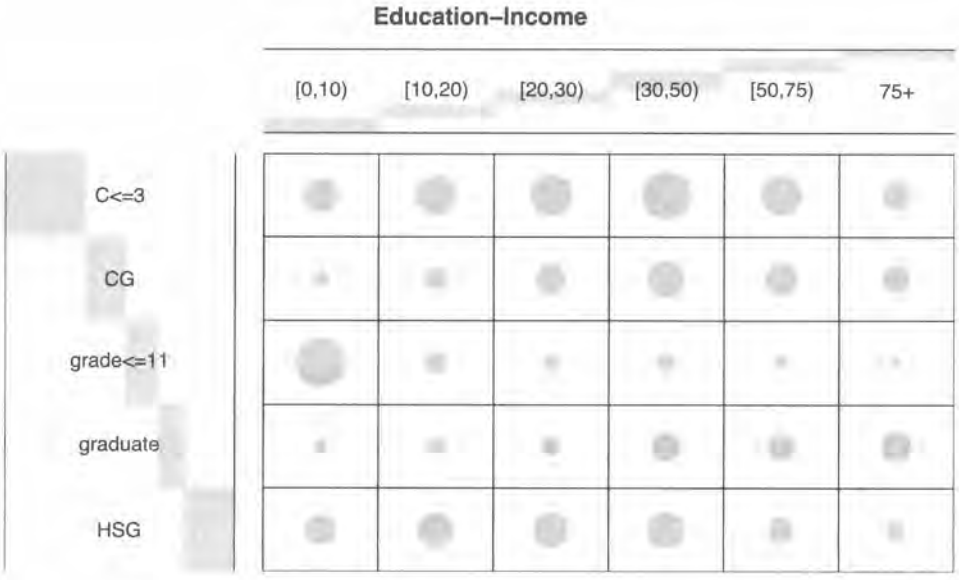

图 9.1 例9.1收入数据的收入和教育列联表图显示

产生图9.1的代码为:

```
library("gplots")
balloonplot(t(ie), main ="Education-Income", xlab ="", ylab="",
label = FALSE, show.margins = FALSE)
```

对应分析的主要输出在图9.2中. 在图9.2中, 不同的教育程度和收入水平都用点来代表, 而且显示相应的向量. 一个变量各个水平与另一个变量各个水平的向量之间的夹角大小代表它们的关联程度. 可以看出, education 较高 (低) 的向量和 income 较高 (低) 的向量之间的夹角比较小, 这说明教育程度和收入有很大关系. 描述列联表行列变量关系的直观图就是对应分析的主要产品.

图9.2实际上是两个图叠加而成的, 一个图显示代表列联表行变量 (education) 的一组点, 而另一个图为代表列变量 (income) 的一组点. 每个图中的点的二维坐标是如何获得的?

是依照什么原理获得的? 这就是本章 "对应分析" 所要介绍的.

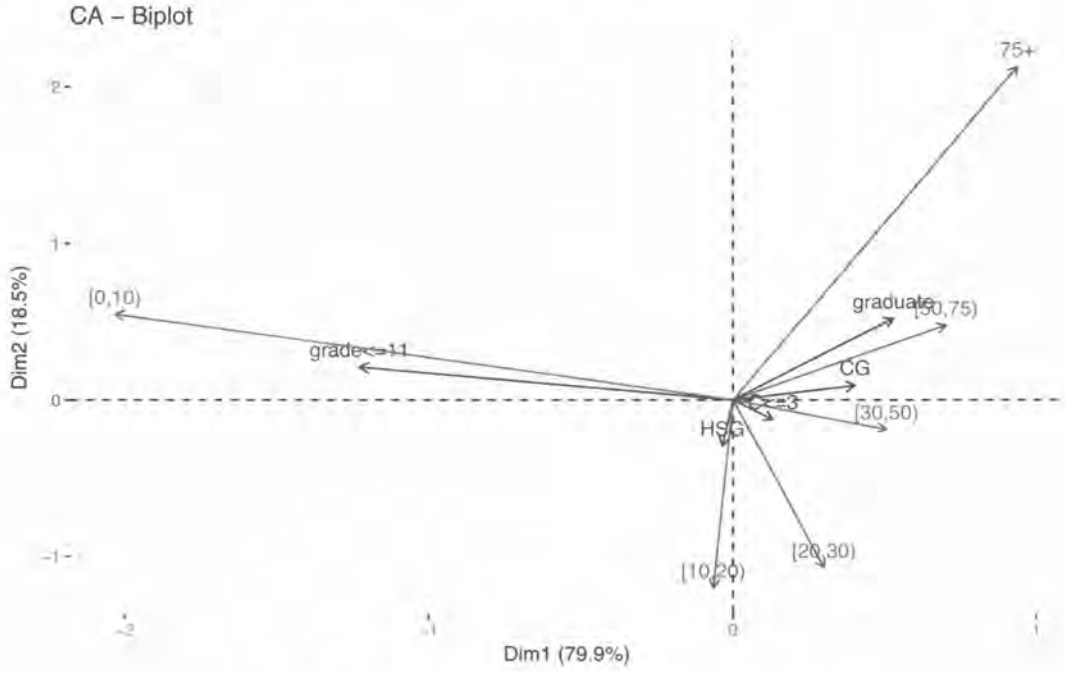

图 9.2 例9.1收入数据的收入和教育列联表的对应分析结果

图9.2的两组点的坐标在下面分别关于行 (Rows) 和列 (Columns) 输出 (R 代码ca(ie)) 的最后两行 (Dim. 1 和Dim. 2). 这里输出的其余部分会在后面介绍.

```
> (res=ca(ie))
 Principal inertias (eigenvalues):
           1        2        3        4
Value      0.278327 0.064449 0.004965 0.00041
Percentage 79.94%   18.51%   1.43%    0.12%

 Rows:
             C<=3        CG grade<=11 graduate        HSG
Mass     0.350058 0.175538  0.140052 0.119255  0.215096
ChiDist  0.185170 0.424743  1.243758 0.749238  0.311586
Inertia  0.012003 0.031668  0.216652 0.066945  0.020883
Dim. 1   0.248140 0.761222 -2.323910 1.000649 -0.066717
Dim. 2  -0.492235 0.360306  0.821981 2.056243 -1.168200

 Columns:
            [0,10)   [10,20)   [20,30)   [30,50)  [50,75)      75+
Mass     0.182519  0.150378  0.166521  0.237056 0.155468 0.108057
ChiDist  1.080121  0.325442  0.316602  0.273542 0.403919 0.741997
Inertia  0.212938  0.015927  0.016692  0.017738 0.025365 0.059492
```

```
Dim. 1  -2.030031 -0.064339  0.299790  0.505172 0.701745 0.938575
Dim. 2   0.549432 -1.203396 -1.074263 -0.190400 0.478748 2.131057
```

9.2.1 二元对应分析的数学原理

二维列联表回顾

行	列 1	列 2	⋯	列 J	行总计	行边际频率 $\boldsymbol{r}$
1	n_{11}	n_{12}	⋯	n_{1J}	n_{1+}	$p_{1+}=\frac{n_{1+}}{n_{++}}$
2	n_{21}	n_{22}	⋯	n_{2J}	n_{2+}	$p_{2+}=\frac{n_{2+}}{n_{++}}$
⋮	⋮	⋮		⋮	⋮	⋮
I	n_{I1}	n_{I2}	⋯	n_{IJ}	n_{I+}	$p_{2+}=\frac{n_{I+}}{n_{++}}$
列总计	n_{+1}	n_{+2}	⋯	n_{+J}	n_{++}	
列边际频率 $\boldsymbol{c}$	$p_{+1}=\frac{n_{+1}}{n_{++}}$	$p_{+2}=\frac{n_{+2}}{n_{++}}$	⋯	$p_{+J}=\frac{n_{+J}}{n_{++}}$		p_{++}

关于列联表独立性和齐性的 χ^2 检验统计量:

$$\chi^2=\sum_{i,j}\frac{(O_{ij}-E_{ij})^2}{E_{ij}}=\sum_{i,j}\frac{\left(n_{ij}-n\frac{n_{i+}}{n}\frac{n_{+j}}{n}\right)^2}{n\frac{n_{i+}}{n}\frac{n_{+j}}{n}}=n\sum_{i,j}\frac{\left(\frac{n_{ij}}{n}-\frac{n_{i+}}{n}\frac{n_{+j}}{n}\right)^2}{\frac{n_{i+}}{n}\frac{n_{+j}}{n}}=n\sum_{i,j}\left(\frac{p_{ij}-p_{i+}p_{+j}}{\sqrt{p_{i+}p_{+j}}}\right)^2$$

对二元列联表数据, 令列联表为 $I\times J$ 矩阵 $\boldsymbol{N}=\{n_{ij}\}$, 总计数为 $n_{++}=\sum_{i=1}^{I}\sum_{j=1}^{J}n_{ij}$. 使其从计数变成频率: $\boldsymbol{P}=\{p_{ij}\}=\boldsymbol{N}/n_{++}$. 令 $p_{i+}=\sum_{j=1}^{J}p_{ij}$ $(i=1,2,\ldots,I)$ 及 $p_{+j}=\sum_{i=1}^{I}p_{ij}$ $(j=1,2,\ldots,J)$ 分别代表行和列的边际频率, 并记行边际频率为 $\boldsymbol{r}=\{p_{i+}\}$, $\boldsymbol{D}_r=\mathrm{diag}(\boldsymbol{r})$, 以及列边际频率 $\boldsymbol{c}=\{p_{+j}\}$, $\boldsymbol{D}_c=\mathrm{diag}(\boldsymbol{c})$, 然后计算 $\boldsymbol{P}$ 的标准化残差: $\boldsymbol{S}=\boldsymbol{D}_r^{-1/2}(\boldsymbol{P}-\boldsymbol{r}\boldsymbol{c}^{\top})\boldsymbol{D}_c^{-1/2}$. 显然 $\boldsymbol{S}=\{s_{ij}\}$ 的元素为:

$$s_{ij}=\frac{p_{ij}-p_{i+}p_{+j}}{\sqrt{p_{i+}p_{+j}}}.$$

$n\sum_{i=1}^{I}\sum_{j=1}^{J}s_{ij}^2$ 就是熟知的关于二维列联表的关于独立性 χ^2 检验的统计量. 在行列独立的零假设下, 有渐近的 $\chi^2_{(I-1)(J-1)}$ 分布.

记 $\boldsymbol{x}=\{x_i\}$ 为 I 维 (未知) 向量, 称为**行记分** (row score), $\boldsymbol{y}=\{y_j\}$ 为 J 维 (未知) 向量, 称为**列记分** (column score). 那么满足下面条件的 $(\alpha,\boldsymbol{x},\boldsymbol{y})$ 称为**对应分析问题**$C_0(\boldsymbol{S})$ 的解:

$$\alpha x_i=\sum_{j=1}^{m}s_{ij}y_j,\quad i=1,2,\ldots,I; \tag{9.2.1}$$

$$\alpha y_j=\sum_{i=1}^{n}s_{ij}x_i,\quad j=1,2,\ldots,J. \tag{9.2.2}$$

该式的意义为: **行记分 $\{x_i\}$ 与列记分 $\{y_j\}$ 的加权均值成比例, 列记分 $\{y_j\}$ 则与行记**

分 $\{x_i\}$ 的加权均值成比例. 这里的权重就是 $\{s_{ij}\}$. 数值 α 代表行列记分的相关 (在典型相关的意义上). 对该问题解的数学推导也很简单.

式 (9.2.1) 和式 (9.2.2) 的矩阵形式分别为:

$$\alpha \boldsymbol{x} = \boldsymbol{S}\boldsymbol{y}$$

和

$$\alpha \boldsymbol{y} = \boldsymbol{S}^{\top}\boldsymbol{x}.$$

把上面两式中的 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 互相替代, 就得到下面等价的一对特征值问题:

$$\begin{aligned}\alpha^2\boldsymbol{x} =& \boldsymbol{S}\boldsymbol{S}^{\top}\boldsymbol{x},\\ \alpha^2\boldsymbol{y} =& \boldsymbol{S}^{\top}\boldsymbol{S}\boldsymbol{y}.\end{aligned}$$

根据线性代数 (参看2.1.12节) 来看, 这是解两个有相同非零特征值方程的特征值问题. 也就是说, $\boldsymbol{S}\boldsymbol{S}^{\top}$ 和 $\boldsymbol{S}^{\top}\boldsymbol{S}$ 有相同的非零特征值. 如果有 k ($\leqslant \min\{(I-1),(J-1)\}$) 个非零特征值 $\alpha_1^2 \geqslant \alpha_2^2 \geqslant \cdots \geqslant \alpha_k^2$, 而 $\boldsymbol{S}\boldsymbol{S}^{\top}$ 和 $\boldsymbol{S}^{\top}\boldsymbol{S}$ 的相应于这些特征值的特征向量分别为 $I\times k$ 维和 $J\times k$ 维矩阵, 可以用奇异值分解的方法来解这个特征值问题 (参看2.2.2节):

$$\boldsymbol{S} = \boldsymbol{U}\boldsymbol{D}_{\alpha}\boldsymbol{V}^{\top},$$

这里的 $\boldsymbol{U}$ 就是相应于 $\boldsymbol{S}\boldsymbol{S}^{\top}$ 的特征向量矩阵, 而 $\boldsymbol{V}$ 是相应于 $\boldsymbol{S}^{\top}\boldsymbol{S}$ 的特征向量矩阵, D_{α} 则是对角线矩阵 $\mathrm{diag}(\alpha_1,\alpha_2,\ldots,\alpha_{\min(I,J)})$, 其对角线元素的平方 $(\alpha_1^2,\alpha_2^2,\ldots,\alpha_k^2)$ 为 $\boldsymbol{S}\boldsymbol{S}^{\top}$ 和 $\boldsymbol{S}^{\top}\boldsymbol{S}$ 的非零特征值. 图9.2的两对坐标分别由 $\boldsymbol{U}$ 和 $\boldsymbol{V}$ 的标准化坐标 (standard coordinates)

$$\boldsymbol{X} = \boldsymbol{D}_r^{-1/2}\boldsymbol{U} \quad 和 \quad \boldsymbol{Y} = \boldsymbol{D}_c^{-1/2}\boldsymbol{V}$$

的前两列组成, 分别是前面行 (Rows) 和列 (Columns) 输出的最后两行 (Dim. 1 和Dim. 2). 前面输出 Principal inertias (eigenvalues) 下面两行中的第一行 (Value) 就是特征值 $\alpha_1^2,\alpha_2^2,\ldots,\alpha_k^2$, 而第二行 (Percentage) 则是这些特征值的比例 $\alpha_i/(\sum_i \alpha_i)$. 行 (Rows) 和列 (Columns) 输出的 Inertia 分别是 $\boldsymbol{S}$ 元素平方后所得矩阵 $\{s_{ij}^2\}$ 的行总和及列总和. 行 (Rows) 和列 (Columns) 输出的 Mass 分别是行和列的边际频率 $\boldsymbol{c}$ 和 $\boldsymbol{r}$. 此外, 一些文献称 $\boldsymbol{S}$ 矩阵元素的平方和 $\sum_i\sum_j s_{ij}^2 = \sum_i \alpha_i^2$ 为惯量 (inertial), 并分别称 $\boldsymbol{F} = \boldsymbol{X}\boldsymbol{D}_{\alpha}$ 及 $\boldsymbol{G} = \boldsymbol{Y}\boldsymbol{D}_{\alpha}$ 为行和列的主坐标 (principal coordinates).

容易验证: $\boldsymbol{X}^{\top}\boldsymbol{D}_r\boldsymbol{X} = \boldsymbol{Y}^{\top}\boldsymbol{D}_c\boldsymbol{Y} = \boldsymbol{I}$ 及 $\boldsymbol{F}^{\top}\boldsymbol{D}_r\boldsymbol{F} = \boldsymbol{G}^{\top}\boldsymbol{D}_c\boldsymbol{G} = \boldsymbol{D}_{\alpha}^2$.

9.2.2 对应分析行列相关的显著性 *

有两种判别对应分析行列之间相关的显著性的方法, 一种是迹方法, 即计算作为行列相关系数的迹 (特征根之和) 的平方根, Bendixen (1995) 及 Healey (2013) 认为如果它大于 0.2, 则认为相关显著. 例9.1收入数据的迹的平方根为 0.59004, 因此可以认为其是相关的. 对例9.1数据 (根据前面结果) 计算的代码为:

```
library("factoextra")
eig=get_eigenvalue(res);tr=sum(eig$eig);(ccor=sqrt(tr))
```

另一种检验统计量是迹乘以样本量, 它有自由度 $(I-1)(J-1)$ 的渐近 χ^2 分布. 对例9.1收入数据 (根据前面结果) 检验的代码为:

```
ch2=tr*sum(as.matrix(ie))
df=(nrow(ie) - 1)*(ncol(ie) - 1)
(pval=pchisq(ch2, df = df, lower.tail = FALSE))
```

所得到的 p 值为 0, 看来相关也是显著的.

9.2.3 两个解释图形

崖底碎石图

和主成分分析及典型相关分析等方法类似, 可以做出崖底碎石图 (见图9.3). 因为对于随机的数据来说, 按行或按列排列的特征值的期望应该是 $1/(I-1)$ 或 $1/(J-1)$, 对于例9.1收入数据, 随机矩阵的崖底碎石图不应该超过 $\max(1/(I-1), 1/(J-1) = 25\%$ (图中虚线), 当然, 这里早已超过.

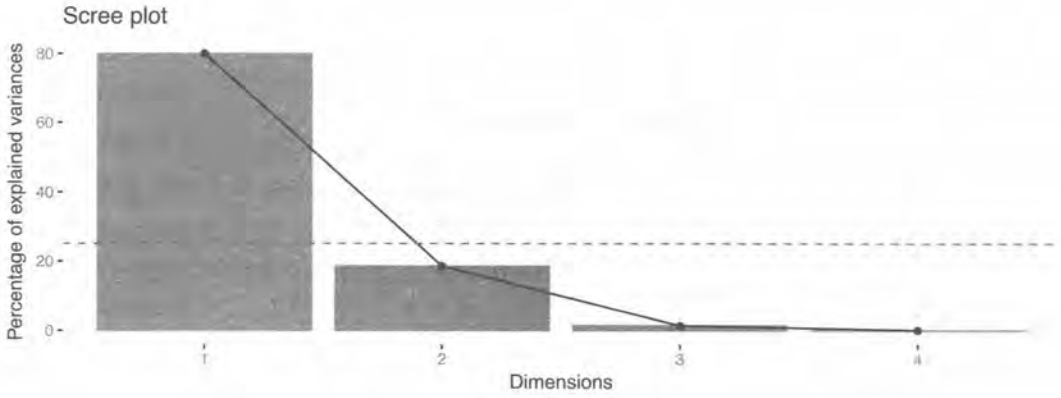

图 9.3 例9.1收入数据的收入和教育列联表的对应分析特征值的崖底碎石图

产生图9.3的代码为:

```
fviz_screeplot(res) +
   geom_hline(yintercept=25, linetype=2, color="red")
```

行列变量对各维的贡献图

行列变量对各维的贡献 (按照百分比) 可以做成图 (见图9.4) 来展示, 代码为:

```
row=get_ca_row(res);col=get_ca_col(res)
library("corrplot")
```

```
corrplot(row$contrib, is.corr=FALSE)
corrplot(col$contrib, is.corr=FALSE)
```

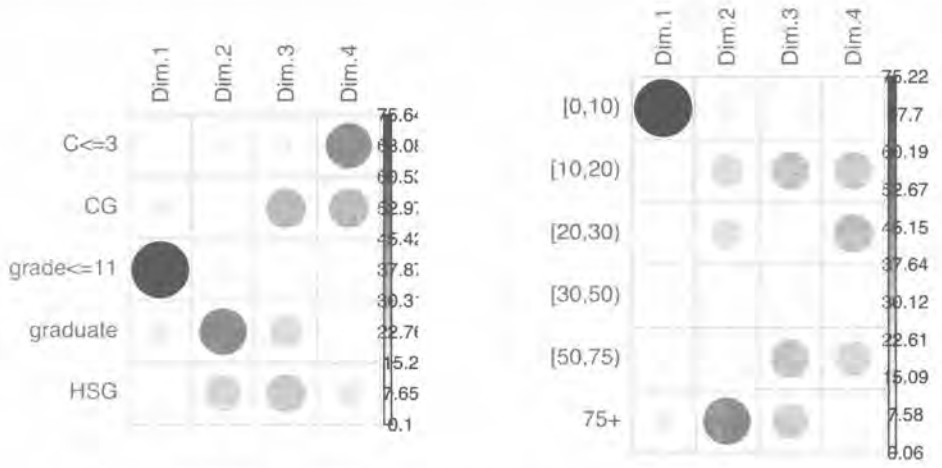

图 9.4 例9.1收入数据的收入和教育列联表的对应分析行列变量对各维的贡献

从图9.4中很容易看出哪些变量对行列变量的影响大 (主要是颜色深、尺寸大的变量).

9.3 多元及联合对应分析

9.3.1 例9.1收入数据多元及联合对应分析实践

多元对应分析 (multiple correspondence analysis, MCA) 和联合对应分析 (joint correspondence analysis, JCA) 的思想与二元对应分析类似, 虽然在数学上稍微有些烦琐 (参看9.3.2节), 但在实践上却很简单. 为此, 在例9.1收入数据中增加一个变量 language.in.home, 这样加上二元对应分析用过的 income 和 education 之后就有三个变量了. 先看这个三维列联表的数字形式:

```
> w=read.csv("IncomeCA.csv")#读入数据
> z=xtabs(~education+income+language.in.home,w)
> ftable(z, row.vars =c(1,3), col.vars = "income")
                          income [0,10) [10,20) [20,30) [30,50) [50,75) 75+
education language.in.home
C<=3      english                266     381     431     595     393 201
          other                   14       8      10      12      10   5
          spanish                 14      20      16      18      12   1
CG        english                 49     109     201     331     261 173
          other                    5       9      10      12       3   5
          spanish                  3       5       3      15       5   8
grade<=11 english                524      86      49      68      33  23
          other                   33       2       3       1       2   4
          spanish                 77      28      14      11       4   1
```

```
graduate  english   28   56   77   208   184 238
          other      0    2    1    7     3   4
          spanish    1    3    2    1     2   3
HSG       english  200  290  293  313   146  70
          other     10    7    8    4     3   5
          spanish   31   28   27   34     8   2
```

用下面语句做基于 Burt 矩阵 (参看9.3.2节的定义) 的多元对应分析 (MCA) 及联合对应分析 (JCA), 并且分别画出图来 (见图9.5):

```
library(ca)
(MCA=mjca(w,lambda="Burt"))
(JCA=mjca(w,lambda="JCA"))
par(mfrow=c(1,2))
plot(MCA)
plot(JCA)
```

输出的格式和二元对应分析类似, 只不过都在一个表中, 下面的Dim. 1 和Dim. 2 是所有三个变量各个水平在图 (见图9.5) 中的坐标.

```
> (MCA=mjca(w,lambda="Burt"))

 Eigenvalues:
           1        2        3        4        5        6        7
Value      0.283503 0.177046 0.130311 0.119677 0.113453 0.109628 0.108481
Percentage 21.62%   13.5%    9.94%    9.13%    8.65%    8.36%    8.27%
           8        9        10       11
Value      0.093848 0.088885 0.061756 0.02441
Percentage 7.16%    6.78%    4.71%    1.86%


 Columns:
       income:[0,10) income:[10,20) income:[20,30) income:[30,50)
Mass        0.060840       0.050126       0.055507       0.079019
ChiDist     1.377803       1.386369       1.304581       1.048305
Inertia     0.115494       0.096343       0.094469       0.086837
Dim. 1      2.288614       0.175033      -0.305077      -0.568797
Dim. 2     -0.873356       1.501287       1.290031       0.286134
       income:[50,75) income:75+ language.in.home:english
Mass         0.051823   0.036019                 0.304295
ChiDist      1.368025   1.715698                 0.182161
Inertia      0.096986   0.106026                 0.010097
Dim. 1      -0.869097  -1.140886                -0.167524
Dim. 2      -0.542333  -2.449526                -0.030102
```

```
        language.in.home:other language.in.home:spanish education:C<=3
Mass                 0.009793                 0.019246       0.116686
ChiDist              3.328734                 2.383355       0.796393
Inertia              0.108506                 0.109323       0.074007
Dim. 1               1.002260                 2.138774      -0.344076
Dim. 2              -0.961212                 0.965028       0.549178
        education:CG education:grade<=11 education:graduate education:HSG
Mass        0.058513            0.046684           0.039752      0.071699
ChiDist     1.276773            1.615139           1.631398      1.120016
Inertia     0.095384            0.121784           0.105798      0.089941
Dim. 1     -0.865067            2.681408          -1.232690      0.203474
Dim. 2     -0.399911           -1.178984          -2.324375      1.488960
> (JCA=mjca(w,lambda="JCA"))

 Eigenvalues:
           1        2        3        4        5
Value      0.142992 0.030341 0.000958 0.000262 8.8e-05
Percentage NA%      NA%      NA%      NA%      NA%


 Columns:
        income:[0,10) income:[10,20) income:[20,30) income:[30,50)
Mass         0.060840       0.050126       0.055507       0.079019
ChiDist      1.377803       1.386369       1.304581       1.048305
Inertia      0.115494       0.096343       0.094469       0.086837
Dim. 1       2.086945       0.079787      -0.319212      -0.520630
Dim. 2       0.935682      -1.625295      -1.427785      -0.306480
        income:[50,75) income:75+ language.in.home:english
Mass          0.051823   0.036019                 0.304295
ChiDist       1.368025   1.715698                 0.182161
Inertia       0.096986   0.106026                 0.010097
Dim. 1       -0.739792  -0.937622                -0.095213
Dim. 2        0.590148   2.704956                 0.027259
        language.in.home:other language.in.home:spanish education:C<=3
Mass                 0.009793                 0.019246       0.116686
ChiDist              3.328734                 2.383355       0.796393
Inertia              0.108506                 0.109323       0.074007
Dim. 1               0.549107                 1.226030      -0.373961
Dim. 2               0.433618                -0.651626      -0.494846
        education:CG education:grade<=11 education:graduate education:HSG
Mass        0.058513            0.046684           0.039752      0.071699
ChiDist     1.276773            1.615139           1.631398      1.120016
Inertia     0.095384            0.121784           0.105798      0.089941
Dim. 1     -1.016205            3.113531          -1.425384      0.200924
```

```
Dim. 2      0.412662          0.871556          2.238431      -1.339967
```

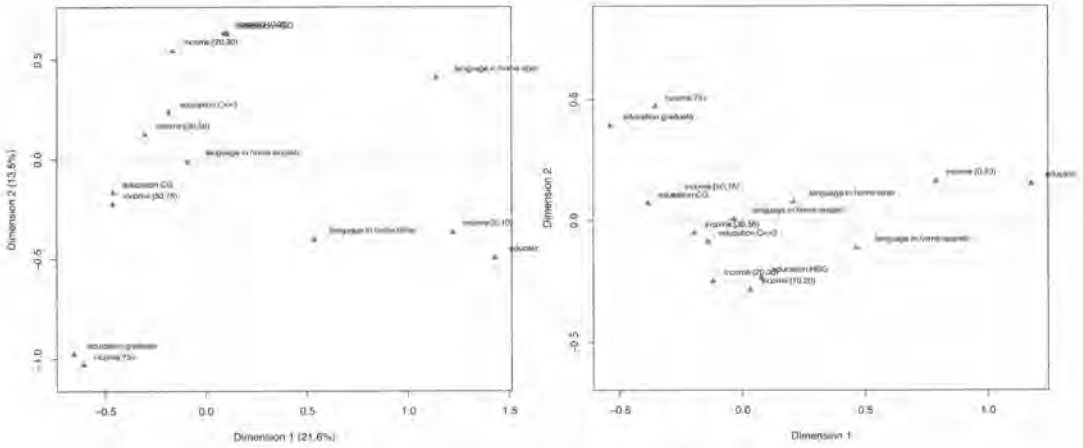

图 9.5 例9.1收入数据的多元对应分析 (左) 和联合对应分析点图 (右)

根据二元对应分析的计算输出格式及对图形的理解, 相信读者能够很容易解释上面计算机的输出及图9.5所包含的多元对应分析的意义.

9.3.2 多元对应分析方法的数学 *

二元对应分析涉及的数学在前面已经比较详尽地介绍了. 从二元到**多元对应分析**的推广基本上有两种方式, 一种基于指标矩阵 (indicator matrix), 另一种基于 Burt 矩阵. 联合对应分析方法有所不同. 细节请参看 Greenacre (1984, 1994).

9.3.3 基于指标矩阵和 Burt 矩阵的方法

假定数据有 n 个观测值及 Q 个分类变量, 每个分类变量有 J_q $(q = 1, 2, \ldots, Q)$ 个水平. 那么, 每个变量可形成一个 $n \times J_q$ 阶的包含 0 和 1 的矩阵 $\boldsymbol{Z}_q$, 其第 ij 个元素如果等于 1, 则表示这个变量第 i 个观测值取水平 j, 否则为 0. 这样的 Z_q 称为 (第 q 个变量的) 指标矩阵.

对于二元情况, 列联表为 $\boldsymbol{Z}_1^\top \boldsymbol{Z}_2$, 令 $\boldsymbol{P}_{ij} = (1/n)\boldsymbol{Z}_i^\top \boldsymbol{Z}_j$ $(i, j = 1, 2)$, 称为**对应矩阵**, 而且记对角阵 (边际频率) 为 $\boldsymbol{D}_i = \boldsymbol{P}_{ii}$ $(i = 1, 2)$. 如果考虑尺度变量 $\boldsymbol{s}_1, \boldsymbol{s}_2$, 则尺度化的指标矩阵为 $\boldsymbol{Z}_i\boldsymbol{s}_i$ $(i = 1, 2)$. 如果中心化, 则有: $\mathbf{1}^\top \boldsymbol{Z}_i\boldsymbol{s}_i = \mathbf{0}$ $(i = 1, 2)$. 如此, 这两个变量的协方差 s_{12} 和方差 $s_i^2 = s_{ii}$ $(i = 1, 2)$ 分别为:

$$
\begin{aligned}
s_{ij} &= (1/n)\boldsymbol{s}_i^\top \boldsymbol{Z}_i^\top \boldsymbol{Z}_j \boldsymbol{s}_j = \boldsymbol{s}_i^\top \boldsymbol{P}_{ij} \boldsymbol{s}_j, \ i = 1, 2; \\
s_i^2 &= (1/n)\boldsymbol{s}_i^\top \boldsymbol{Z}_i^\top \boldsymbol{Z}_i \boldsymbol{s}_i = \boldsymbol{s}_i^\top \boldsymbol{D}_i \boldsymbol{s}_i, \ i = 1, 2.
\end{aligned}
$$

相关系数为:

$$
r = \frac{s_{12}}{s_1 s_2} = \frac{\boldsymbol{s}_1^\top \boldsymbol{P}_{12} \boldsymbol{s}_2}{\sqrt{\boldsymbol{s}_1^\top \boldsymbol{D}_1 \boldsymbol{s}_1 \boldsymbol{s}_2^\top \boldsymbol{D}_2 \boldsymbol{s}_2}}.
$$

实际上, 尺度 $\boldsymbol{s}_i$ $(i=1,2)$ 未知, 我们希望找到适当的尺度, 使得 $\boldsymbol{Z}_1$ 和 $\boldsymbol{Z}_2$ 的典型相关最大. 做奇异值分解:

$$\boldsymbol{D}_1^{-1/2}\boldsymbol{P}_{12}\boldsymbol{D}_2^{-1/2}=\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\top}, \tag{9.3.1}$$

式中左右的正交矩阵 $\boldsymbol{U}$ 和 $\boldsymbol{V}$ 分别用它们的一对列向量 $\boldsymbol{u},\boldsymbol{v}$ 代替, 并左右分别乘 $\boldsymbol{u}^{\top}$ 和 $\boldsymbol{v}$, 得到

$$\boldsymbol{u}\boldsymbol{D}_1^{-1/2}\boldsymbol{P}_{12}\boldsymbol{D}_2^{-1/2}\boldsymbol{v}=\sigma.$$

因此, 如果令 $\boldsymbol{s}_1=\boldsymbol{D}_1^{-1/2}\boldsymbol{u}$ 及 $\boldsymbol{s}_2=\boldsymbol{D}_2^{-1/2}\boldsymbol{v}$, 则有 $\boldsymbol{s}_1^{\top}\boldsymbol{P}_{12}\boldsymbol{s}_2=\sigma$, 即协方差. 由于 $\boldsymbol{U}$ 和 $\boldsymbol{V}$ 的正交性, $\boldsymbol{s}_1^{\top}\boldsymbol{D}_1\boldsymbol{s}_1=\boldsymbol{s}_2^{\top}\boldsymbol{D}_2\boldsymbol{s}_2=1$. 我们将 $\boldsymbol{P}_{12}$ 中心化, 并做奇异值分解:

$$\boldsymbol{D}_1^{-1/2}(\boldsymbol{P}_{12}-\boldsymbol{P}_{12}\boldsymbol{1}\boldsymbol{1}^{\top}\boldsymbol{P}_{12}^{\top})\boldsymbol{D}_2^{-1/2}=\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\top}, \tag{9.3.2}$$

这个公式和上一节 $\boldsymbol{S}$ 的分解完全一样, 结果和上一节的二元对应分析相同.

注意, 没有中心化的分解 (9.3.1) 和中心化的分解 (9.3.2) 所得到的非平凡奇异值相同, 但前者的 $\boldsymbol{\Sigma}$ 多一个等于 1 的平凡奇异值, 相应于等于 $\boldsymbol{1}$ 的 $\boldsymbol{s}_1$ 和 $\boldsymbol{s}_2$, 并相应于 $\boldsymbol{U}$ 和 $\boldsymbol{V}$ 的第一列. 在中心化之后, 这个问题就不存在了. 最大 (非奇异) 奇异值在式 (9.3.1) 中是第二个, 而在式 (9.3.2) 中是第一个. 在式 (9.3.2) 中的最大奇异值 $\sigma_1(\sigma_1\geqslant\sigma_2\geqslant\cdots\geqslant\sigma_k)$ 称为典型相关系数.

显然, 使得两个变量的相关最大等价于其中任意一个变量和它们的平均值的相关最大. 如此, 我们考虑 $\boldsymbol{Z}=[\boldsymbol{Z}_1\ \boldsymbol{Z}_2]$, 而某种尺度标准化的平均为:

$$\frac{1}{2}(\boldsymbol{Z}_1\boldsymbol{s}_1+\boldsymbol{Z}_2\boldsymbol{s}_2)=\frac{1}{2}[\boldsymbol{Z}_1\ \boldsymbol{Z}_2]\begin{bmatrix}\boldsymbol{s}_1\\ \boldsymbol{s}_2\end{bmatrix}.$$

下面定义后面常用的一些符号:

$$\boldsymbol{D}=\begin{bmatrix}\boldsymbol{D}_1 & \boldsymbol{0}\\ \boldsymbol{0} & \boldsymbol{D}_2\end{bmatrix},\quad \boldsymbol{s}=\begin{bmatrix}\boldsymbol{s}_1\\ \boldsymbol{s}_2\end{bmatrix},\quad \boldsymbol{C}=\boldsymbol{Z}^{\top}\boldsymbol{Z}=\begin{bmatrix}\boldsymbol{Z}_1^{\top}\boldsymbol{Z}_1 & \boldsymbol{Z}_1^{\top}\boldsymbol{Z}_2\\ \boldsymbol{Z}_2^{\top}\boldsymbol{Z}_1 & \boldsymbol{Z}_2^{\top}\boldsymbol{Z}_2\end{bmatrix}=n\begin{bmatrix}\boldsymbol{D}_1 & \boldsymbol{P}_{12}\\ \boldsymbol{P}_{21} & \boldsymbol{D}_2\end{bmatrix},$$

这里的 $\boldsymbol{C}=\boldsymbol{Z}^{\top}\boldsymbol{Z}$ 称为 Burt 矩阵.

现在考虑的对应矩阵为 $\frac{1}{2n}\boldsymbol{Z}$, 其行边际频率为 $\frac{1}{n}\boldsymbol{1}$, 列边际频率组成对角线矩阵 $\boldsymbol{D}$ 的元素. 做奇异值分解:

$$\frac{\sqrt{n}}{2n}\boldsymbol{Z}\boldsymbol{D}^{-1/2}=\boldsymbol{U}\boldsymbol{\Gamma}\boldsymbol{V}^{\top},$$

其对称特征值公式为:

$$\left(\frac{\sqrt{n}}{2n}\boldsymbol{Z}\boldsymbol{D}^{-1/2}\right)^{\top}\left(\frac{\sqrt{n}}{2n}\boldsymbol{Z}\boldsymbol{D}^{-1/2}\right)=\frac{1}{4n}\boldsymbol{D}^{-1/2}\boldsymbol{Z}^{\top}\boldsymbol{Z}\boldsymbol{D}^{-1/2}=\boldsymbol{V}\boldsymbol{\Gamma}^2\boldsymbol{V}^{\top}.$$

令 $\boldsymbol{\Lambda}=\boldsymbol{\Gamma}^2$, 有

$$\frac{1}{4n}\boldsymbol{D}^{-1/2}\boldsymbol{C}\boldsymbol{D}^{-1/2}=\boldsymbol{V}\boldsymbol{\Gamma}^2\boldsymbol{V}^{\top}=\boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^{\top}. \tag{9.3.3}$$

显然, 对于某一个 $\boldsymbol{V}$ 的列向量 $\boldsymbol{v}$ (可分解成分别代表行和列的两部分 $\boldsymbol{v}_1$ 和 $\boldsymbol{v}_2$), 如果令

$$\boldsymbol{s}=\boldsymbol{D}^{-1/2}\boldsymbol{v},$$

即

$$\begin{bmatrix}\boldsymbol{s}_1\\ \boldsymbol{s}_2\end{bmatrix}=\begin{bmatrix}\boldsymbol{D}_1^{-1/2} & \boldsymbol{0}\\ \boldsymbol{0} & \boldsymbol{D}_2^{-1/2}\end{bmatrix}\begin{bmatrix}\boldsymbol{v}_1\\ \boldsymbol{v}_2\end{bmatrix},$$

有

$$\begin{aligned}\frac{1}{4n}\boldsymbol{s}^{\top}\boldsymbol{C}\boldsymbol{s}&=\frac{1}{4n}\begin{bmatrix}\boldsymbol{s}_1\\ \boldsymbol{s}_2\end{bmatrix}^{\top}\begin{bmatrix}\boldsymbol{Z}_1^{\top}\boldsymbol{Z}_1 & \boldsymbol{Z}_1^{\top}\boldsymbol{Z}_2\\ \boldsymbol{Z}_2^{\top}\boldsymbol{Z}_1 & \boldsymbol{Z}_2^{\top}\boldsymbol{Z}_2\end{bmatrix}\begin{bmatrix}\boldsymbol{s}_1\\ \boldsymbol{s}_2\end{bmatrix}\\ &=\frac{1}{4}\begin{bmatrix}\boldsymbol{s}_1\\ \boldsymbol{s}_2\end{bmatrix}^{\top}\begin{bmatrix}\boldsymbol{D}_1 & \boldsymbol{P}_{12}\\ \boldsymbol{P}_{21} & \boldsymbol{D}_2\end{bmatrix}\begin{bmatrix}\boldsymbol{s}_1\\ \boldsymbol{s}_2\end{bmatrix}=\gamma^2=\lambda,\end{aligned}$$

这在 λ 取最大特征值 $\lambda_1=\gamma_1^2$ 时 ($\lambda_1\geqslant\lambda_2\geqslant\cdots\geqslant\lambda_k$) 达到最大. 因此, 这里的解 $\boldsymbol{s}_1,\boldsymbol{s}_2$ 和做简单二元对应分析时相同. 与二元对应分析不同的是, 那里的最大相关是 σ_1, 而现在等于 $(1+\sigma_1)/2$, 也就是 $\lambda_1=(1+\sigma_1)/2$ 为前面所说的一个变量和两个变量平均之间相关的平方. λ_1 也是 $\boldsymbol{C}$ 的奇异值, 等于 $\boldsymbol{C}$ 的主惯量的平方根. 由分解 (9.3.3) 得到的标准正交矩阵 $\boldsymbol{V}$ 的向量 $\boldsymbol{v}$ 所确定的向量 $\boldsymbol{s}$ 满足 $\boldsymbol{s}^{\top}\boldsymbol{D}\boldsymbol{s}=1$.

现在可以推广到 $Q(>2)$ 个分类变量的情况, 这时令 $\boldsymbol{Z}=[\boldsymbol{Z}_1,\boldsymbol{Z}_1,\ldots,\boldsymbol{Z}_Q]$, Burt 矩阵和边际频率矩阵为:

$$\boldsymbol{C}=\begin{bmatrix}\boldsymbol{Z}_1^{\top}\boldsymbol{Z}_1 & \boldsymbol{Z}_1^{\top}\boldsymbol{Z}_2 & \cdots & \boldsymbol{Z}_1^{\top}\boldsymbol{Z}_Q\\ \boldsymbol{Z}_2^{\top}\boldsymbol{Z}_1 & \boldsymbol{Z}_2^{\top}\boldsymbol{Z}_2 & \cdots & \boldsymbol{Z}_2^{\top}\boldsymbol{Z}_Q\\ \vdots & \vdots & \ddots & \vdots\\ \boldsymbol{Z}_Q^{\top}\boldsymbol{Z}_1 & \boldsymbol{Z}_Q^{\top}\boldsymbol{Z}_2 & \cdots & \boldsymbol{Z}_Q^{\top}\boldsymbol{Z}_Q\end{bmatrix},\quad \boldsymbol{D}=\begin{bmatrix}\boldsymbol{D}_1 & \boldsymbol{0} & \cdots & \boldsymbol{0}\\ \boldsymbol{0} & \boldsymbol{D}_2 & \cdots & \boldsymbol{0}\\ \vdots & \vdots & \ddots & \vdots\\ \boldsymbol{0} & \boldsymbol{0} & \cdots & \boldsymbol{D}_Q\end{bmatrix}.$$

如果第 q 个变量有 J_q 个水平, 记 $J=\sum_{q=1}^{Q}J_q$, 则 $\boldsymbol{Z}$ 为 $n\times J$ 矩阵, $\boldsymbol{C}$ 为 $J\times J$ 矩阵. 现在考虑的对应矩阵为 $\frac{1}{Qn}\boldsymbol{Z}$, 其行边际频率为 $\frac{1}{n}\boldsymbol{1}$, 列边际频率组成对角线矩阵 $\boldsymbol{D}$ 的元素. 做奇异值分解:

$$\frac{\sqrt{n}}{Qn}\boldsymbol{Z}\boldsymbol{D}^{-1/2}=\boldsymbol{U}\boldsymbol{\Gamma}\boldsymbol{V}^{\top}.$$

为了去掉平凡解, 可把上面被分解的矩阵换成相应的中心化矩阵:

$$\sqrt{n}\left(\frac{1}{Qn}\boldsymbol{Z}-\frac{1}{n}\boldsymbol{1}\boldsymbol{1}^{\top}\boldsymbol{D}\right)\boldsymbol{D}^{-1/2}.$$

非中心化的 $\boldsymbol{C}$ 的分解为

$$\boldsymbol{D}^{-1/2}\frac{1}{Q^2n}\boldsymbol{C}\boldsymbol{D}^{-1/2}=\boldsymbol{U}\boldsymbol{\Gamma}^2\boldsymbol{V}^{\top}=\boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^{\top}.$$

为了去掉平凡解, 可把上面被分解的矩阵换成相应的中心化矩阵:

$$\boldsymbol{D}^{-1/2}\left(\frac{1}{Q^2 n}\boldsymbol{C}-\boldsymbol{D}\mathbf{1}\mathbf{1}^{\top}\boldsymbol{D}\right)\boldsymbol{D}^{-1/2}.$$

如果尺度参数 $\boldsymbol{s}=(\boldsymbol{s}_1^{\top},\boldsymbol{s}_2^{\top},\ldots,\boldsymbol{s}_Q^{\top})^{\top}$, 则它为奇异向量所定义: $\boldsymbol{s}=\boldsymbol{D}^{-1/2}\boldsymbol{v}$, 这里的 $\boldsymbol{v}$ 是 $\boldsymbol{V}$ 的第一列.

9.3.4 从 Burt 矩阵到联合对应分析

联合对应分析是基于 Burt 矩阵的. 回顾前面小节中基于 Burt 矩阵 $\boldsymbol{C}=\{c_{ij}\}$ 做多元对应分析的步骤, 我们有:

(1) 得到 $\boldsymbol{P}=\{p_{ij}\}=\frac{1}{Qn}\boldsymbol{C}$, 并且计算行总数, 即边际频率 $\boldsymbol{r}=\{r_i\}=\boldsymbol{P}\mathbf{1}$(根据对称性, 等于列边际频率), 即 $\boldsymbol{D}=\mathrm{diag}(\boldsymbol{r})$.

(2) 做特征值分解 (等价于奇异值分解)

$$\boldsymbol{D}^{-1/2}\left(\frac{1}{Q^2 n}\boldsymbol{C}-\boldsymbol{D}\mathbf{1}\mathbf{1}^{\top}\boldsymbol{D}\right)\boldsymbol{D}^{-1/2}=\boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^{\top}.$$

可以选择用调整的惯量 $\lambda^{\mathrm{adj}}=\left(\frac{Q}{Q-1}\right)^2\left(\lambda_s-\frac{1}{Q}\right)$ 来代替原来的惯量 λ_s.

(3) 第 s 维的第 i 行 (列) 标准坐标为 $\{a_{is}\}$, 这里 $\boldsymbol{A}=\{a_{ij}\}=\boldsymbol{D}^{-1/2}\boldsymbol{V}$.

(4) 第 s 维的第 i 行 (列) 的主坐标为 $f_{is}=a_{is}\lambda_s$, 这里 $F=\boldsymbol{A}\boldsymbol{\Lambda}$.

在多元对应分析中, 由于 Burt 矩阵的对角线子矩阵块 $(\boldsymbol{Z}_i^{\top}\boldsymbol{Z}_i)$ 扮演了主要的角色, 使得对角线之外的信息缺失. 联合对应分析利用迭代加权最小二乘法来不断更新其对角线子矩阵. 它事先确定解的维度 S^*, 然后对维度 $1,2,\ldots,S^*$ 做多元对应分析以实现 $\boldsymbol{C}$ 对角线矩阵的更新. 其过程如下:

(1) 令 $\boldsymbol{C}^*=\{c_{ij}^*\}=\{c_{ij}\}$.

(2) 对 $\hat{\boldsymbol{C}}^*$ 实施多元对应分析, 得到解:

$$\hat{\boldsymbol{C}}=\{\hat{c}_{ij}\}\equiv nr_i r_j\left(1+\sum_{s=1}^{S^*}\lambda_s a_{is}a_{js}\right).$$

(3) 用 $\hat{C}$ 代替 $\boldsymbol{C}^*$, 回到步骤 2, 直到收敛.

9.4 本章计算的 Python 代码

9.4.1 二元对应分析

这里没有用专门的模块, 完全按照公式计算. 输入模块和数据整理 (把分类变量变成哑元变量) 的代码如下:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
v= pd.read_csv('incomeCA.csv',index_col=False)
id=v.columns[np.array([0,2])]
w=v[id]
Z=w
for i in range(2):
    Z=pd.concat([Z,pd.get_dummies(w[id[i]])],axis=1)
```

用下面语句按照前面公式做二元对应分析:

```
z=np.array(Z[Z.columns[2:]])
P=z[:,:6].T.dot(z[:,6:])/z.shape[0]
r=np.sum(P,axis=1);c=np.sum(P,axis=0)
Dr=np.diag(np.sqrt(r))
Dc=np.diag(np.sqrt(c))
r.shape=len(r),1
c.shape=len(c),1
S=np.dot(Dr.dot(P-r.dot(c.T)),Dc)
U,s,V=np.linalg.svd(S)
X=Dr.dot(U)
Y=Dc.dot(V.T)
```

下面给出例9.1收入数据的二元对应分析图 (见图9.6).

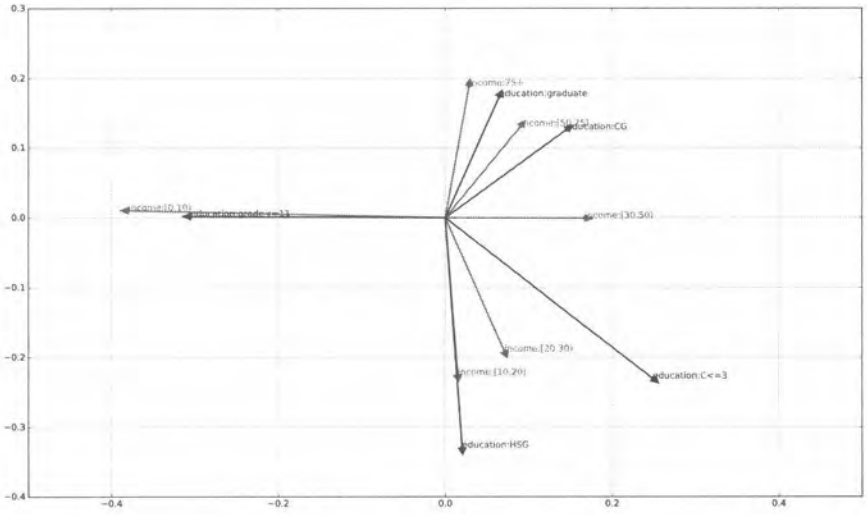

图 9.6 例9.1收入数据的二元对应分析图

产生图9.6的代码为:

```
T1=id[0]+":"+Z.columns[2:8]
T2=id[1]+":"+Z.columns[8:14]
```

```
plt.xlim(-0.5,0.5)
plt.scatter(X[:,0],X[:,1],color='r')
for i in range(6):
    plt.text(X[i,0],X[i,1],T1[i],color='r')
    plt.arrow(0,0, X[i,0],X[i,1],color='r',head_width=0.01,head_length=0.01)
plt.scatter(Y[:,0],Y[:,1],color='b')
for i in range(5):
    plt.text(Y[i,0],Y[i,1],T2[i],color='b')
    plt.arrow(0,0, Y[i,0],Y[i,1],color='b',head_width=0.01,head_length=0.01)
plt.grid()
plt.show()
```

9.4.2 多元对应分析

和前面 R 软件分析类似, 选择三个变量, 但这里没有用专门的模块, 完全按照公式计算, 直到奇异值分解为止. 具体代码如下:

```
v= pd.read_csv('income0.csv',index_col=False)
id=v.columns[np.array([0,4,13])]
w=v[id]
Z=w
for i in range(3):
    Z=pd.concat([Z,pd.get_dummies(w[id[i]])],axis=1)

z=np.array(Z[Z.columns[3:]])
Z1=z[:,:6];Z2=z[:,6:11];Z3=z[:,11:]
Z01=np.concatenate((Z1.T.dot(Z1),Z1.T.dot(Z2),Z1.T.dot(Z3)),axis=1)
Z02=np.concatenate((Z2.T.dot(Z1),Z2.T.dot(Z2),Z2.T.dot(Z3)),axis=1)
Z03=np.concatenate((Z3.T.dot(Z1),Z3.T.dot(Z2),Z3.T.dot(Z3)),axis=1)
C=np.concatenate((Z01,Z02,Z03),axis=0)
Q=3;q=np.array([6,5,3])
n=sum(q);d=np.sum(C/Q/n,axis=0);D=np.diag(np.sqrt(d))
d=d.reshape(len(d),1)
U,s,V=np.linalg.svd((D.dot(C/Q/n-d.dot(d.T))).dot(D))
```

结果的矩阵 $\boldsymbol{U}, \boldsymbol{s}, \boldsymbol{V}$ 可以用下面代码打印出来 (因为篇幅长, 这里未提供):

```
print("U:\n",U,"\n","s:\n",s,"\n","V:\n",V)
```

9.5 习 题

1. 对应分析虽然用了不少数学工具, 但仍被认为是具有描述性的, 为什么? 在本书中, 哪些章的内容是属于描述性的?

2. 描述性统计是相对于推断统计 (参数估计、假设检验等经典统计) 而言的, 但实际上所有推断统计都是大量的人为假定和主观决策的结合. 你对这句话的看法如何? 举例说明.
3. 本书中的哪些内容和对应分析很相似?
4. R 中提供了一个名为`HairEyeColor` 的数据, 是三维数组形式, 变量有 Hair(头发颜色, 水平: Black、Brown、Red、Blond)、Eye(眼睛颜色, 水平: Brown、Blue、Hazel、Green)、Sex(性别, 水平: Male、Female). 请用头发颜色和眼睛颜色形成的列联表做二元对应分析.

第 10 章　多维尺度变换

10.1　目 的

多维尺度变换 (multidimensional scaling, MDS) 也称**多维标度**或**多维缩放**, 是一组对象之间的距离或不相似度的直观表示. 最典型的对象是地理位置, 当然也可以是观点、颜色、面孔, 或任何种类的实体或抽象的概念.

尺度变换 (scaling) 来自心理测量学, 用来测量人们对某个问题的态度的强弱程度, 比如, 对个人目前精神压力打分, 可以从 0 分打到 9 分 (10 个整数). 根据打分情况可以把人分类, 看哪些人比较接近, 哪些人相距较远. 当然, 这只是一维数据, 实际上绝对不可能仅仅对一个问题打分, 这就是多维问题了. 地理坐标是 2 维的, 可以在地图上标明位置以度量远近, 但是如果根据城市间的距离 (比如公路距离) 再重新产生 2 维图, 此时的位置就有可能和地图表明的位置不一样了, 然而可以寄希望于重新产生的 2 维图的距离尽可能和原先的距离一致. 多维数据不易在图中直观展示, 但我们希望可以按照各个对象的距离或差距对多维数据做出度量, 并且以较低维形式 (主要是 2 维或 3 维) 使这些对象的距离或差距可视化.

例 10.1. 城市距离数据　美国 9 个城市之间的道路距离. [1]

下面是例10.1数据中包含的美国 9 个城市之间的道路距离:

	Atlanta	Chicago	Denver	Houston	LosAngeles	Miami	NewYork	SanFrancisco	Seattle
Chicago	587								
Denver	1212	920							
Houston	701	940	879						
LosAngeles	1936	1745	831	1374					
Miami	604	1188	1726	968	2339				
NewYork	748	713	1631	1420	2451	1092			
SanFrancisco	2139	1858	949	1645	347	2594	2571		
Seattle	2182	1737	1021	1891	959	2734	2408	678	
Washington.DC	543	597	1494	1220	2300	923	205	2442	2329

我们希望通过上面的距离矩阵重新获得这些城市的 2 维位置, 通过 MDS 方法, 可以得到下面的图10.1, 画图时, 如果下面代码针对坐标`x`, `y` 不加负号, 所产生的图的东西南北方向会和实际方位相反, 但无论方向如何, 城市间距离看起来都和实际差不太多. 符号并不重要, 最多和把地图倒过来看或透过光从反面看一样. 在这个例子中, 基于城市距离重建地图有较好的效果同美国的公路距离与实际距离差不太多有关. 当然, 更重要的是原先的距离是真实的, 而城市是可以画在图上的真实的点, 否则根本无法比较.

图10.1是用 R 的默认函数`cmdscale` 做的, 代码为:

[1] 来自 *The Cambridge Encyclopaedia* 的表格, 在 R 中以 `UScitiesD` 名字提供.

```
loc=cmdscale(UScitiesD)
x=-loc[, 1];y=-loc[, 2]#负号是为了保持习惯上的"上北下南左西右东"
plot(x, y,type="n",xlab="",ylab="",asp=1,main="US cities")
text(x, y, rownames(loc), cex = 1)
```

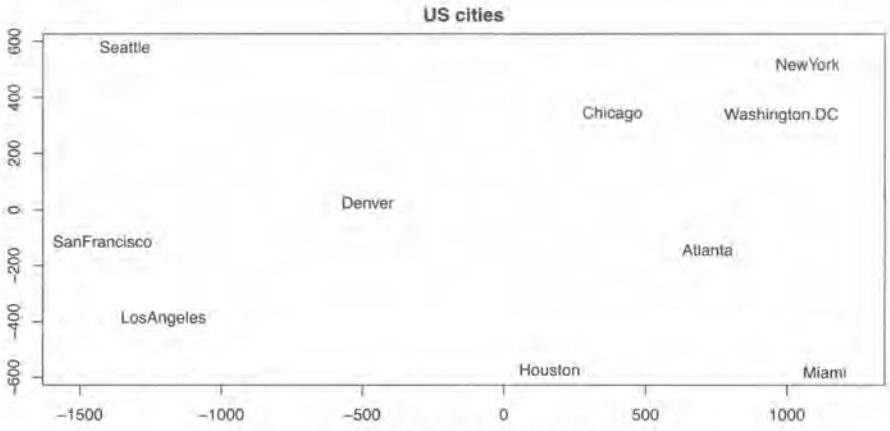

图 10.1 例10.1数据降维后各城市的 2 维位置图

在多维尺度变换的术语中, 为便于标记, 我们会使用缩写 MDS 来表示多维尺度变换. 此外, 除了**距离** (distance) 之外, 常用的词还有**相似度** (similarities) 或**不相似度** (dissimilarities) 及**邻近度** (proximities), 其实它们都是可以用某些方法互相转换的.

多维尺度变换是**一组**可视性方法, 它把多元数据中的相似性或不相似性度量投影到低维空间. 严格来说, 主成分分析是多维尺度变换的一个最简单的特例.

10.2 多维尺度变换分类

根据选择的度量和获得的相似性 (或不相似) 矩阵, 可以将 MDS 分为两种主要方法: **度量的 MDS** (metric multidimensional scaling, mMDS) 和**非度量的 MDS** (nonmetric multidimensional scaling, nMDS). 这依分析的矩阵是否基于度量距离而定.

度量多维尺度变换 (mMDS) 也称为**主坐标分析** (principal coordinate analysis) 或经典尺度变换 (classical scaling), 它将距离矩阵转换为一组坐标, 以便使从这些坐标导出的降维后的欧氏距离尽可能接近原始距离. 使用 mMDS 的优势在于, 在大多数情况下, 对象之间的关系可以在欧几里德空间中完全表示. 对于只知道对象之间次序的情况, 则用非度量多维尺度变换 (nMDS) 处理, 它试图通过投影重新产生这些秩而不是距离.

10.3 距离函数

两个点 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 之间的距离 (或不相似度) $d(\boldsymbol{x},\boldsymbol{y})$ 可以在任何空间定义, 在数学上距离函数称为**度量** (metric), 具有度量的空间称为**度量空间**. 距离函数在数学上必须满足下列条件:

- 非负性: $d(\boldsymbol{x},\boldsymbol{y}) \geqslant 0$, 而且 $d(\boldsymbol{x},\boldsymbol{y}) = 0 \iff \boldsymbol{x} = \boldsymbol{y}$.
- 对称性: $d(\boldsymbol{x},\boldsymbol{y}) = d(\boldsymbol{y},\boldsymbol{x})$.

- 三角不等式: $d(\boldsymbol{x},\boldsymbol{z}) \leqslant d(\boldsymbol{x},\boldsymbol{y}) + d(\boldsymbol{y},\boldsymbol{z})$.

给定任何距离 (或不相似度), MDS 试图把其解释成低维的欧氏距离.[2]

相似度与距离或不相似度相反, 用 $\boldsymbol{S} = \{s_{ij}\}$ 表示相似度矩阵, 那么所需要的条件为: $\boldsymbol{S}$ 为对称的, 而且对于任何 i, j, 有 $s_{ij} \leqslant s_{ii}$ 及 $s_{ij} \leqslant s_{jj}$. 如果记 d_{ij} 为不相似度, 那么可以做如下转换:

$$d_{ij} = \sqrt{s_{ii} + s_{jj} - 2s_{ij}}.$$

10.4 mMDS 的目的

记原始的距离 (或不相似度) 矩阵为 $\boldsymbol{\Delta}_{(n\times n)} = \{\delta_{ij}\}$, 这里的 $\boldsymbol{\Delta}$ 是对称、非负及中空的 (hollow) 矩阵, 中空意味着对角线元素为 0 ($\delta_{ii} = 0,\ \forall i$), 显然 $\boldsymbol{\Delta}$ 满足上面距离函数的条件, 但不要求是欧氏距离. 我们希望在低维 (比如 p 维) 欧氏空间找到 n 个点 $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$, 使得 $\{\delta_{ij}\}$ 和由矩阵 $\boldsymbol{X}_{(p\times n)} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n]$ 形成的距离阵 $d(\boldsymbol{X}) \equiv \boldsymbol{D}_{(n\times n)} = \{d_{ij}\}$ 的元素 $d_{ij} = d_{ij}(\boldsymbol{X})$ 与 δ_{ij} 尽可能接近: $d_{ij} \approx \delta_{ij}$; 或者说, $\{d_{ij}\}$ 是 $\{\delta_{ij}\}$ 的某个函数或估计值, 可记为 $d_{ij} = \hat{\delta}_{ij} = f(\delta_{ij})$. 这里的符号 $d_{ij}(\boldsymbol{X})$ 定义为 $\boldsymbol{X}$ 的两个列向量 $\boldsymbol{x}_i$ 和 $\boldsymbol{x}_j$ 之间的欧氏距离

$$d_{ij}(\boldsymbol{X}) = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2 = \sqrt{\sum_{k=1}^{p}(x_{ki} - x_{kj})^2},$$

这里的矩阵 $\boldsymbol{X}$ 的元素称为对象的**构形** (configurations). MDS 对每个对象做尺度变换到 p 维空间, 使得在新空间的点间距离尽可能保持和原空间中的距离接近.

以什么标准来衡量 δ_{ij} 和它的 "预测值" d_{ij} 的接近程度呢? 这就需要选择损失函数, 以及从损失函数导出的拟合优度. 最自然想到的就是误差平方和或者加权误差平方和. 在 MDS 中, 这些拟合优度称为**应力** (strain, stress). 在 MDS 中, 常用的应力有:

- 通常 mMDS 用的应力 (也可以取其平方根):

$$E = \sum_{i<j}(\delta_{ij} - d_{ij})^2.$$

- Kruskal 应力 (也可以取其平方根):

$$E_{Kruskal\,1} = \frac{\sum_{i<j}(\delta_{ij} - d_{ij})^2}{\sum_{i<j}\delta_{ij}^2}.$$

- Sammon 应力:

$$E_{Sammon} = \frac{1}{\sum_{\ell<k}\delta_{\ell k}} \frac{\sum_{i<j}(\delta_{ij} - d_{ij})^2}{\sum_{i<j}\delta_{ij}}.$$

[2]欧氏距离定义为 $d(\boldsymbol{x},\boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|_2 \equiv \sqrt{\sum_k (x_{ik} - y_{ik})^2}$; 一般地, 所谓 Lp 距离 (p-norm distance) 定义为 $\|\boldsymbol{x} - \boldsymbol{y}\|_p \equiv (\sum_k |x_{ik} - y_{ik}|^p)^{1/p}$, 欧氏距离称为 L2 距离.

- Kruskal 应力的一般形式:

$$E_{Kruskal} = \sum_{i<j} w_{ij}(\delta_{ij} - d_{ij})^2,$$

这里的权重矩阵 $\boldsymbol{W}_{(n\times n)} = \{w_{ij}\}$ 假定为非负、对称及中空的已知矩阵. 不失一般性, 可以假定权重满足下列条件:

$$\sum_{i<j} w_{ij}\delta_{ij}^2 = n(n-1)/2,$$

而且根据 De Leeuw (1977), 还假定 $\boldsymbol{W}$ 是不可约的 (irreducible), 也就是说, 问题不会进一步分成独立的小问题.

10.5 mMDS 方法的原理

mMDS 方法考虑 $\boldsymbol{\Delta}$ 属于数学上的距离 (度量) 矩阵的情况, 也就是说, 两点之间的空间大小可以度量, 这相对于两点之间只有次序的情况.

由于构形 $\boldsymbol{X}$ 定义的欧氏距离 $d_{ij}(\boldsymbol{X}) = \|\boldsymbol{x_i} - \boldsymbol{x_j}\|_2 = \|(\boldsymbol{x_i} - \boldsymbol{c}) - (\boldsymbol{x_j} - \boldsymbol{c})\|_2$, 因此, 对于任意常数向量 $\boldsymbol{c}$, $\boldsymbol{X} - c$ 也是解, 也就是说构形的解不唯一. 因此, 我们仅仅寻找满足

$$\sum_{k=1}^{n} x_{ik} = 1, \quad \forall i,$$

条件的中心化构形.

因此, 建立平方距离矩阵 $\boldsymbol{\Delta}^{(2)} = \{\delta_{ij}^2\}$, 并且对 $\boldsymbol{\Delta}^{(2)}$ 做双中心化:

$$\boldsymbol{B} = -\frac{1}{2}\left(\boldsymbol{I} - \frac{1}{n}\mathbf{1}\mathbf{1}^\top\right)\boldsymbol{\Delta}^{(2)}\left(\boldsymbol{I} - \frac{1}{n}\mathbf{1}\mathbf{1}^\top\right).$$

可以证明: $\boldsymbol{\Delta}$ 是欧氏距离当且仅当 $\boldsymbol{B}$ 为半正定的.

对半正定的 $\boldsymbol{B}$ 做特征值分解:

$$\boldsymbol{B} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}\boldsymbol{\Gamma}^\top,$$

令构形 $\boldsymbol{X}$ 满足 $\boldsymbol{B} = \boldsymbol{X}^\top\boldsymbol{X}$, 即 $\boldsymbol{X} = \boldsymbol{\Lambda}^{1/2}\boldsymbol{\Gamma}^\top$, 在矩阵 $\boldsymbol{\Gamma}$ 的行向量所张成的特征空间之中, 选择和 p 个最大特征值相应的 $\boldsymbol{\Lambda}$ 子矩阵 $\boldsymbol{\Lambda}_{p\times p}$ 及相应的 $\boldsymbol{\Gamma}$ 矩阵的 p 列 $\boldsymbol{\Gamma}_p$:

$$\boldsymbol{X}_{(p)} = \boldsymbol{\Lambda}_p^{1/2}\boldsymbol{\Gamma}_p^\top.$$

这样, 我们的降维就完成了. 这些 $\boldsymbol{X}_{(p)}$ 列之间的欧氏距离就是原先距离的近似. 显然, 在 mMDS 中, d_{ij} 和 δ_{ij} 间有线性关系.

例10.1美国城市之间道路距离 (见图10.1) 就是 mMDS 的一个例子, 上面计算过程的每一步都写在下面的函数中, 这也就是 R 函数`cmdscale` 所做的. 对于这个例子, 这两个函数

是等价的 (当然, 函数cmdscale 有更多的功能):

```
mmds=function(d,k=2){
    d=as.matrix(d);n=nrow(d)
    I=diag(n);one=matrix(1,n,1)
    C=I-0.25*one%*%t(one)
    B=-0.5*C%*%d%*%C
    a=eigen(B); lambda=diag(a$values)
    "%^%" <- function(x, n)
        with(eigen(x), vectors %*% (values^n * t(vectors)))
    Xp=t(lambda[1:k,1:k]%^%(1/2)%*%t(a$vectors[,1:k]))
    row.names(Xp)=row.names(d)
    return(Xp)}
```

10.6 nMDS 的基本思维

在原始距离不是欧氏距离的情况下, 不相似性是被秩或排序度量的, 在排序中相继对象之间的空间大小不是人们感兴趣的, 或者是不得而知的. 比如, 学校或企业排名中的第 1、2 名之间的距离可能根本无法度量. 因此, 我们所需要的原始度量 δ_{ij} 的近似的函数 $f(\delta_{ij})$ 不一定能够得到, 这时我们仅仅希望近似值 $d^*_{ij} = f(\delta_{ij})$ 能够保持 δ_{ij} 的秩:

$$\delta_{ij} < \delta_{\ell k} \iff f(\delta_{ij}) \leqslant f(\delta_{\ell k}) \iff d^*_{ij} \leqslant d^*_{\ell k},$$

因此, 我们只能称 d^*_{ij} 为**差距** (disparity) 而不是距离. 相似度可以很容易转换成不相似度, 比如 s_{ij} 是相似度, 那么不相似度可以定义为 $\delta_{ij} = c - s_{ij}$, 这里 $c > \max_{i,j}(s_{ij})$, 对于不同的 c 及采用不同的应力, 结果可能会不同.

下面的数据包含美国各州与教育有关的一些统计数字. 用这种数据建立的各州距离显然不是真正的数学度量.

例 10.2. 教育数据 (States.csv) 这是一个 R 程序包carData 自带的名为States 的数据, 该数据包含美国各州及华盛顿特区与教育有关的一些统计数字, 该数据有 51 行 8 列, 其变量为: region (地区), 包括 ESC (中部东南)、PAC (太平洋地区)、MTN (洛基山地区)、WSC (中部西南)、NE (新英格兰)、SA (南大西洋地区)、ENC (中部东北)、WNC (中部西北)、MA (中部大西洋地区); pop (人口, 单位: 千人); SATV (高中毕业生 SAT 作文的州平均成绩); SATM (高中毕业生 SAT 数学的州平均成绩); percent (高中毕业生参加 SAT 考试的百分比); dollars (州公共教育投入, 单位: 千美元/学生); pay (教师工资的州平均数, 单位: 千美元). 各州的名字在原数据行名字 (row.names) 中, 在数据文件 States.csv 的第一列. 我们不用 region 变量.

Sammon 非线性投影

Sammon 投影 (Sammon, 1969) 是非线性的, 因为它不能得到如解特征值那样的线性形式. 其目标是使得 Sammon 应力 E_{Sammon} 最小, 最小化可以采取梯度下降法或者其他

的迭代方法. 但是, 人们必须根据经验确定迭代次数, 而且有可能不收敛, 当然可以用其他方法得到初始构形 (比如第一主成分). Sammon 投影把很多应力放在精确地重塑小距离上.

保序 Kruskal MDS 方法

保序 Kruskal MDS 方法 (isotone Kruskal MDS) 使用 mMDS(或其他方法) 通过单调函数 f 得到初始值 $f(\delta_{ij})$, 然后通过调整点 d_{ij}^* 及函数 f 使 Kruskal 应力 (或其平方根) 最小, 得到 d_{ij}^*:

$$d_{ij}^* = \arg\min_{f,\, d*_{ij}} \sum_{i<j} \frac{\left(f(\delta_{ij}) - d_{ij}^*\right)^2}{\sum_{i<j} f(\delta_{ij})^2}.$$

关于保序 Kruskal 及其他 MDS 的细节请参见 Cox and Cox (2001).

对例10.2教育数据做 mMDS、Sammon 投影及 Kruskal MDS

我们对例10.2教育数据做 mMDS、Sammon 投影及 Kruskal MDS, 并作图 (见图10.2), 这里使用了程序包MASS 中的函数sammon 和isoMDS.[3] 程序代码为:

```
library(MASS)
data("States",package = "carData")
df=States[,-1]
par(mar=c(1,1,3,1))
layout(t(1:3))
df.scal <- cmdscale(dist(df), k = 2, eig = T)
plot(df.scal$points, type = "n",
xlim=range(df.scal$points[,1])+c(-2000,1000),
ylim=range(df.scal$points[,2]),
xlab = "",main = "mMDS",tick=0,yaxt="n")
text(df.scal$points, labels = row.names(States),
    col = 1, cex = 0.8)

df.sam <- sammon(dist(df))
plot(df.sam$points, type = "n",
xlim=range(df.sam$points[,1])+c(-2000,1000),
ylim=range(df.sam$points[,2]),
xlab = "",main = "Sammon's mapping",tick=0,yaxt="n")
text(df.sam$points, labels = row.names(States),
    col = 1, cex = 0.8)

df.iso <- isoMDS(dist(df))
plot(df.iso$points, type = "n",
xlim=range(df.iso$points[,1])+c(-2000,1000),
ylim=range(df.iso$points[,2]),
```

[3]Venables, W. N. & Ripley, B. D. (2002) *Modern Applied Statistics with S*. Fourth Edition. Springer, New York. ISBN 0-387-95457-0.

```
xlab = "",main = "Kruscal MDS",tick=0,yaxt="n")
text(df.iso$points, labels = row.names(States),
    col = 1, cex = 0.8)
```

得到的三个图显示了各个州基于这些统计数字的差距. 看起来, 三个图的差别不是很大.

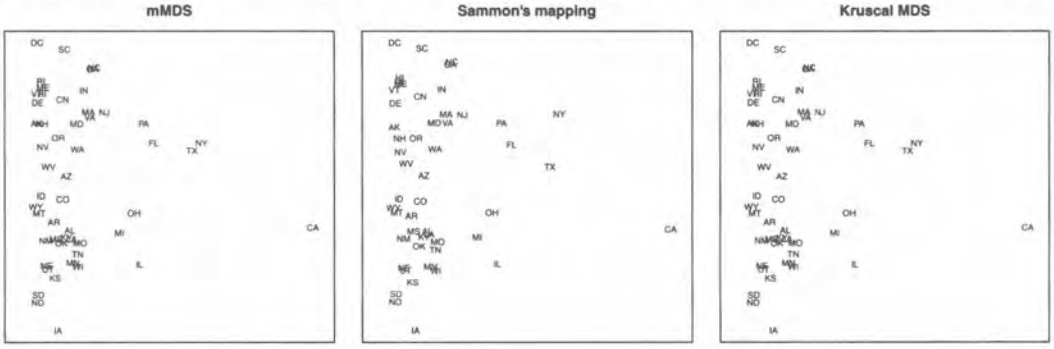

图 10.2 对例10.2教育数据做 mMDS(左)、Sammon 投影 (中) 及 Kruskal MDS(右)

10.7 本章计算的 Python 代码

首先输入必要的模块:

```
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
from matplotlib.collections import LineCollection
from sklearn import manifold
from sklearn.metrics import euclidean_distances
from sklearn.decomposition import PCA
```

读入数据并计算距离:

```
w=pd.read_csv("States.csv",index_col=0)#州名字在数据文件第一列
w=w.iloc[:,1:]#去掉地区名字列
X=np.array(w)
d= euclidean_distances(w)
```

计算 mMDS 及 nMDS:

```
seed=1010
mds = manifold.MDS(n_components=2, max_iter=3000, eps=1e-9,
    random_state=seed,dissimilarity="precomputed", n_jobs=1)
pos = mds.fit(d).embedding_
nmds = manifold.MDS(n_components=2,metric=False, max_iter=3000,
    eps=1e-12, dissimilarity="precomputed",random_state=seed,
```

```
    n_jobs=1,n_init=1)
npos = nmds.fit_transform(d, init=pos)
```

由于图像挤在一起, 做些尺度变换及旋转:

```
# 改变尺度:
pos *= np.sqrt((X ** 2).sum()) / np.sqrt((pos ** 2).sum())
npos *= np.sqrt((X ** 2).sum()) / np.sqrt((npos ** 2).sum())
# 利用主成分分析变换:
clf = PCA(n_components=2)
X = clf.fit_transform(X)
pos = clf.fit_transform(pos)
npos = clf.fit_transform(npos)
```

点出原始数据的主成分得分、mMDS 及 nMDS 图 (见图10.3):

```
fig = plt.figure(figsize=(12,4))
ax = fig.add_subplot(1,3, 1)
s = 10 #符号大小
plt.scatter(X[:, 0], X[:, 1], color='navy',marker='o',s=s,lw=0)
plt.title('True position')
for i in np.arange(len(npos[:,0])):
plt.text(X[i, 0],X[i, 1],w.index[i])
ax = fig.add_subplot(1,3, 2)
plt.scatter(pos[:,0],pos[:,1],color='turquoise',marker='^',s=s,lw=0)
for i in np.arange(len(pos[:,0])):
plt.text(pos[i, 0],pos[i, 1],w.index[i])
plt.title('mMDS')
ax = fig.add_subplot(1,3, 3)
plt.scatter(npos[:,0],npos[:,1],color='darkorange',marker='s',s=s,lw=0)
for i in np.arange(len(npos[:,0])):
plt.text(npos[i, 0],npos[i, 1],w.index[i])
plt.title('nMDS')
plt.show()
```

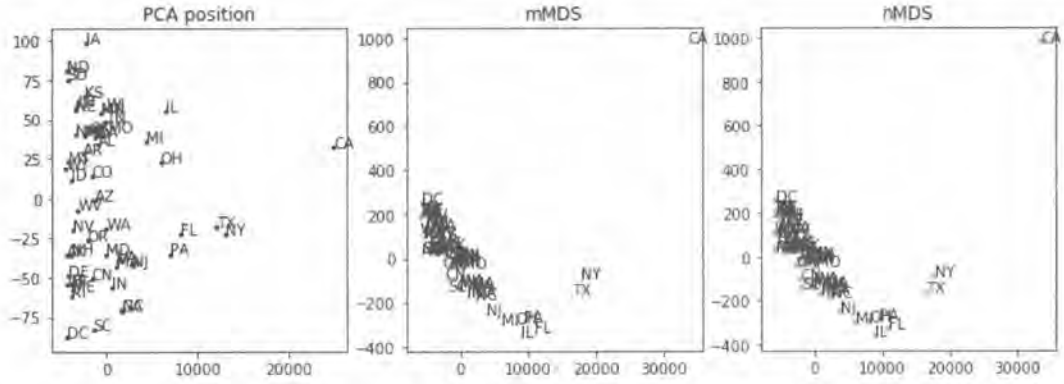

图 10.3 例10.2教育数据：原始数据的主成分得分（左）、mMDS(中) 及 nMDS(右)

10.8 习 题

1. 考虑例5.1的教师数据.
   (1) 对一个州的学校, 比如加利福尼亚州 (代号: "CA"), 做观测值的 mMDS 及 nMDS, 并且作图.
   (2) 对变量做 mMDS 及 nMDS, 并且作图. 为此, 你必须对数据做转置.
2. 用 R 自带数据swiss 做 mMDS 及 nMDS, 并且作图.

附录 1 R 简介——通过运行来领悟

简 介

R 软件 (R Development Core Team, 2011) 用的是 S 语言. 对 R 软件编程的熟悉有助于学习其他快速计算及处理各种数据的语言, 比如 C++、FORTRAN、Python、Java、Hadoop、Spark、NoSQL、SQL 等等, 这是因为编程理念具有相似性, 这对应对因快速处理庞大的数据集而要面临的巨大计算量有所裨益, 而熟悉一些傻瓜式商业软件对学习这些语言没有任何好处.

R 软件是免费的自由软件, 它的代码大多公开, 可以修改, 十分透明和方便. 国外新出版的大量统计方法专著都附带 R 程序. R 软件有强大的帮助系统, 其子程序称为函数. 所有函数都有详细说明, 包括变元的性质、缺省值是什么、输出值是什么、方法的大概说明以及参考文献和作者地址. 大多数函数都有例子, 把这些例子的代码复制并粘贴到 R 界面就可以立即得到结果, 学习有关函数十分方便.

反映新方法的各种程序包 (package) 可以从 R 网站下载, 更方便的是联网时可以通过 R 软件菜单的 "程序包"—"安装程序包" 选项直接下载程序包.

软件必须在使用中学, 仅仅从软件手册中学习是不可取的, 正如仅仅用字典和语法书来学习外语不可能成功. 笔者用过众多的编程软件, 没有一个是从课堂或者手册中学的, 全部是在分析数据的实践中学会的. 笔者见到 R 软件时, 已至耳顺之年, 但在一天内即可基本掌握, 几天内就可以熟练编程和无障碍达到数据分析目的.

安装和运行小贴士

- 登录 R 网站 (`http://www.r-project.org/`)[4], 根据说明从你所选择的镜像网站, 下载并安装 R 的所有基本元素.
- 向左边变元赋值可以用 "=" 或者 "<-", 还可以用 "->" 向右赋值.
- 运行时可以在提示码 ">" 后逐行输入指令. 如果回车之后出现 "+" 号, 则说明你的语句不完整 (得在 "+" 号后面继续输入) 或者已输入的语句有错误.
- 每一行可以输入多个语句, 之间用半角分号 ";" 分隔.
- 所有代码中的标点符号都用半角格式 (基本 ASCII 码). R 的代码对字母的大小写敏感. 变量名字、定性变量的水平以及外部文件路径和名字都可以用中文.
- 提倡使用工作目录. 可以在 "文件"—"改变工作目录" 菜单确定工作目录, 也可以用诸如`setwd("D:/工作")` 的代码建立工作目录. 有了工作目录, 输入存取数据及脚本文件的命令时就不用键入路径了.

[4]网上搜索 "R" 即可得到网址.

- 不一定非得键入你的程序, 可以粘贴, 也可以打开或新建以 R 为扩展名的文件 (或其他文本文件) 作为运行脚本. 在脚本中可以用Ctrl+R 来执行 (计算) 光标所在行的命令, 或者仅运行光标选中的部分. 使用脚本文件是应该提倡的, 它可以使你对你的操作有一个完整的书面记录, 对发现错误及更改程序十分方便.
- 出现的图形可以用Ctrl+W 或Ctrl+C 来复制并粘贴 (前者像素高), 或者通过菜单保存成所需的文件格式.
- 输入代码history(n), 会为你找回 n 行你输入过的代码 (无论对错).
- 如果在运行时点击Esc, 则会终止运行.
- 在运行完毕时会被问 "是否保存工作空间映像?" 如果选择 "保存", 下次运行时这次的运行结果还会重新载入内存, 不用重复计算, 缺点是占用空间. 如果已经有脚本, 而且运算量不大, 一般都不必保存. 如果你点击了 "保存", 又没有输入文件名, 这些结果会放在所设或默认的工作目录下名为.RData 的文件中, 你可以随时找到并删除它.
- 注意, 从 PPT 或 Word 文档之类的非文本文件中复制并粘贴到 R 上的代码, 很可能存在由这些软件自动变换的首字母大写、可见及不可见的格式符号或者左右引号等造成的 R 无法执行的问题. 此外, 不要使用全角标点符号 (例如中文标点).
- R 中有很多常用的数学函数、统计函数以及其他函数. 可以通过在 R 的帮助菜单中选择 "手册 (PDF 文件)", 在该手册的附录中找到各种常用函数的内容.
- 在 R 界面, 你可以用问号加函数名 (或数据名) 的方式来得到该函数或数据的细节, 比如用 "?lm" 可以得到关于线性模型函数 "lm" 的各种细节. 另外, 如果想查看在MASS 程序包中的稳健线性模型 "rlm", 可以在已经打开该程序包的情况下 (用library(MASS) 打开, 用detach(package:MASS) 关闭), 用 "?rlm" 得到该函数的细节. 如果MASS 没有打开[5], 或者不知道rlm 在哪个程序包, 可以用 "??rlm" 来得到其位置 (条件是你有这个程序包). 如果对名字不清楚, 但知道部分字符, 比如 "lm", 可以用 "apropos("lm")" 得到所有包含 "lm" 字符的函数和数据.
- 如果想知道某个程序包中有哪些函数或数据, 可以在 R 的帮助菜单上选择 "Html 帮助", 再选择 "Packages", 即可找到你的 R 上装载的所有程序包. 这个 "Html 帮助" 很方便, 可以链接到许多帮助 (包括手册等).
- 有一些简化的函数, 如加、减、乘、除、乘方 (+, -, *, /, ^) 等, 可以用诸如 "?"+"" 的命令得到帮助 (不能用 "?+").
- 你还可以写关于代码的注释: 任何在 "#" 号后面作为注释的代码或文字都不会参与运行.
- 你可能会遇到无法运行过去已经成功运行的一些代码, 或者得到不同结果的现象. 原因往往是这些程序包经过更新后, 一些函数选项 (甚至函数名称和代码) 已经改变, 这说明 R 软件的更新和成长是很快的. 解决的办法是查看该函数, 或者查看提供有关函数的程序包来探索究竟.
- 一个名为 "RStudio" 可以自由下载的软件能更方便地用几个窗口来展示 R 的执行、运行历史、脚本文件、数据细节等.

[5]通常, 为节省内存以及避免变量名称混杂, 应该在需要时打开相应的程序包, 不需要时关闭.

- 网页https://vincentarelbundock.github.io/Rdatasets/datasets.html提供大量 R 的各个程序包所带的数据, 可以由此搜寻所需要的数据.

动 手

如果你不愿意弄湿泳衣, 即使你的教练是世界游泳冠军, 或者你在教室里听了几百小时的课, 你也永远学不会游泳. 如果你不开口, 即使你熟记了字典中所有英文单词的音标, 或者你完全明白英语语法, 你也永远学不会说英语.

软件当然要在使用中学. R 软件资源丰富, 功能非常强大, 我们不可能也没有必要把每一个细节都弄明白, 软件中的很多功能很少用到, 或者是因为不知道, 或者是因为没有需求, 或者是因为有替代方法. 我们小时候都有读书的经验, 那时能看懂多少就看懂多少, 很少查字典, 后来长大了, 在开始学外语时, 由于大量单词不懂才会针对每个不认识的单词查字典. 实际上, 学外语时, 在有一定单词量的情况下, 能猜就不查字典可能是更好的学习方式.

下面提供了一些作者为练习编写的代码, 如果一次全部运行, 用不了一分钟, 但希望读者在每运行一行之后思考一下. 大多数人能够在一两天内将这些代码完全弄懂. 如果在学习各种统计方法时不断实践, R 语言就会成为你自己的语言.

实 践

最初几步:

```
x=1:100#把1,2,...,100这个整数向量赋值到x
(x=1:100) #同上, 只不过显示出来
sample(x,20) #从1,2,...,100中随机不放回地抽取20个值作为样本
set.seed(0);sample(1:10,3)#先设随机种子再抽样
#从1,2,...,200000中随机不放回地抽取10000个值作为样本:
z=sample(1:200000,10000)
z[1:10]#方括号中为向量z的下标
y=c(1,3,7,3,4,2)
z[y]#以y为下标的z的元素值
(z=sample(x,100,rep=T))#从x有放回地随机抽取100个值作为样本
(z1=unique(z))
length(z1)#z中不同元素的个数
xz=setdiff(x,z) #x和z之间的不同元素--集合差
sort(union(xz,z))#对xz及z的并的元素从小到大排序
setequal(union(xz,z),x) #xz及z的并的元素与x是否一样
intersect(1:10,7:50) #两个数据的交
sample(1:100,20,prob=1:100)#从1,2,...,100中不等概率随机抽样,
#上一语句各数字被抽到的概率与其值大小成比例
```

一些简单运算:

```
pi*10^2 #能够用?"*"来看基本算术运算方法, pi是圆周率
"*"(pi,"^"(10,2)) #和上面一样, 有些烦琐, 是吧? 没有人这么用
pi*(1:10)^-2.3#可以对向量求指数幂
x = pi * 10^2 ; print(x)
(x=pi *10^2) #赋值带打印
pi^(1:5) #指数也可以是向量
print(x, digits= 12)#输出x的12位数字
```

关于 R 对象的类型等:

```
x=pi*10^2
class(x) #x的class
typeof(x) #x的type
class(cars)#cars是一个R中自带的数据
typeof(cars) #cars的type
names(cars)#cars数据的变量名字
summary(cars) #cars的汇总
head(cars)#cars的头几行数据, 和cars[1:6,]相同
tail(cars) #cars的最后几行数据
str(cars)#也是汇总
row.names(cars) #行名字
attributes(cars)#cars的一些信息
class(dist~speed)#公式形式,"~"左边是因变量,右边是自变量
plot(dist~speed,cars)#两个变量的散点图
plot(cars$speed,cars$dist) #同上
```

包括简单自变量为定量变量及定性变量的回归:

```
ncol(cars);nrow(cars) #cars的行列数
dim(cars) #cars的维数
lm(dist ~ speed, data = cars)#以dist为因变量,speed为自变量做OLS
cars$qspeed =cut(cars$speed, breaks=quantile(cars$speed),
    include.lowest = TRUE) #增加定性变量qspeed, 四分位点为分割点
names(cars) #数据cars多了一个变量
cars[3]#第三个变量的值, 和cars[,3]类似
table(cars[3])#列表
is.factor(cars$qspeed)
plot(dist ~ qspeed, data = cars)#点出箱线图
#拟合线性模型(简单最小二乘回归):
(a=lm(dist ~ qspeed, data = cars))
summary(a)#回归结果(包括一些检验)
```

简单样本描述统计量:

```
x <- round(runif(20,0,20), digits=2)#四舍五入
summary(x) #汇总
min(x);max(x) #极值, 与range(x)类似
median(x)  # 中位数(median)
mean(x)    # 均值(mean)
var(x)     #方差(variance)
sd(x)      # 标准差(standard deviation),为方差的平方根
sqrt(var(x)) #平方根
rank(x)    # 秩(rank)
order(x)#升幂排列的x的下标
order(x,decreasing = T)#降幂排列的x的下标
x[order(x)] #和sort(x)相同
sort(x)     #同上: 升幂排列的x
sort(x,decreasing=T)#sort(x,dec=T) 降幂排列的x
sum(x);length(x)#元素和及向量元素个数
round(x) #四舍五入,等于round(x,0),而round(x,5)为留到小数点后5位
fivenum(x)  # 五数汇总, quantile
quantile(x) # 分位点 quantile (different convention)有多种定义
quantile(x, c(0,.33,.66,1))
mad(x) # "median average distance":
cummax(x)#累积最大值
cummin(x)#累积最小值
cumprod(x)#累积积
cor(x,sin(x/20)) #线性相关系数 (correlation)
```

简单图形:

```
x=rnorm(200)#将200个随机正态数赋值到x
hist(x, col = "light blue")#直方图(histogram)
rug(x) #在直方图下面加上实际点的大小
stem(x)#茎叶图
x <- rnorm(500)
y <- x + rnorm(500) #构造一个线性关系
plot(y~ x) #散点图
a=lm(y~x) #做回归
abline(a,col="red")#或者abline(lm(y~x),col="red")散点图加拟合线
print("Hello World!")
paste("x 的最小值= ", min(x)) #打印
demo(graphics)#演示画图(点击Enter来切换)
```

复数运算和求函数极值:

```
(2+4i)^-3.5+(2i+4.5)*(-1.7-2.3i)/((2.6-7i)*(-4+5.1i))#复数运算
#下面构造一个10维复向量，实部和虚部均为10个标准正态样本点：
(z <-complex(real=rnorm(10), imaginary =rnorm(10)))
complex(re=rnorm(3),im=rnorm(3))#3维复向量
Re(z) #实部
Im(z) #虚部
Mod(z) #模
Arg(z) #辐角
choose(3,2) #组合
factorial(6)#排列6!
#解方程：
f=function(x) x^3-2*x-1
uniroot(f,c(0,2))#迭代求根
#如果知道根为极值
f=function(x) x^2+2*x+1 #定义一个二次函数
optimize(f,c(-2,2))#在区间(-2,2)内求极值
```

字符型向量和因子型变量:

```
a=factor(letters[1:10]);a #letters:小写字母向量,LETTERS:大写
a[3]="w" #不行！会给出警告
a=as.character(a) #转换一下
a[3]="w" #可以了
a;factor(a) #两种不同的类型
#定性变量的水平：
levels(factor(a))
sex=sample(0:1,10,r=T)
sex=factor(sex);levels(sex)
#改变因子的水平：
levels(sex)=c("Male","Female");levels(sex)
#确定水平次序：
sex=ordered(sex,c("Female","Male"));sex
levels(sex)
```

数据输入输出:

```
x=scan()#屏幕输入，可键入或粘贴，多行输入在空行后按Enter键
1.5 2.6 3.7 2.1 8.9 12 -1.2 -4

x=c(1.5,2.6,3.7,2.1,8.9,12,-1.2,-4)#等价于上面代码
w=read.table(file.choose(),header=T)#从列表中选择有变量名的数据
setwd("f:/mydata")#建立工作路径
(x=rnorm(20))#给x赋值20个标准正态数据值
#(注:有常见分布的随机数、分布函数、密度函数及分位数函数)
```

```
write(x,"test.txt")#把数据写入文件(路径要对)
y=scan("test.txt");y#扫描文件数值数据到y
y=iris;y[1:5,];str(y) #iris是R自带数据
write.table(y,"test.txt",row.names=F)#把数据写入文本文件
w=read.table("test.txt",header=T)#读带有变量名的数据
str(w) #汇总
write.csv(y,"test.csv")#把数据写入csv文件
v=read.csv("test.csv")#读入csv数据文件
str(v) #汇总
data=read.table("clipboard")#读入剪贴板的数据
```

序列等:

```
(z=seq(-1,10,length=100))#从-1到10等间隔的100个数组成的序列
z=seq(-1,10,len=100)#和上面写法等价
(z=seq(10,-1,-0.1)) #10到-1间隔为-0.1的序列
(x=rep(1:3,3))  #三次重复1:3
(x=rep(3:5,1:3)) #自己看, 这又是什么呢?
x=rep(c(1,10),c(4,5))
w=c(1,3,x,z);w[3]#把数据(包括向量)组合(combine)成一个向量
x=rep(0,10);z=1:3;x+z #向量加法(如果长度不同, R给出警告和结果)
x*z #向量乘法
rev(x)#颠倒次序
z=c("no cat","has ","nine","tails") #字符向量
z[1]=="no cat" #双等号为逻辑等式
z=1:5
z[7]=8;z #什么结果? 注:NA为缺失值(not available)
z=NULL
z[c(1,3,5)]=1:3;
z
rnorm(10)[c(2,5)]
z[-c(1,3)]#去掉第1、3元素
z=sample(1:100,10);z
which(z==max(z))#给出最大值的下标
```

矩阵:

```
x=sample(1:100,12);x #抽样
all(x>0);all(x!=0);any(x>0);(1:10)[x>0]#逻辑符号的应用
diff(x) #差分
diff(x,lag=2) #差分
x=matrix(1:20,4,5);x #矩阵的构造
x=matrix(1:20,4,5,byrow=T);x#矩阵的构造, 按行排列
t(x) #矩阵转置
```

```
x=matrix(sample(1:100,20),4,5)
2*x
x+5
y=matrix(sample(1:100,20),5,4)
x+t(y) #矩阵之间相加
(z=x%*%y) #矩阵乘法
z1=solve(z) #用solve(a,b)可以解方程ax=b
z1%*%z #应该是单位向量，但浮点运算不可能得到干净的0
round(z1%*%z,14)  #四舍五入
b=solve(z,1:4); b #解联立方程
```

矩阵 (续):

```
nrow(x);ncol(x);dim(x)#行列数目
x=matrix(rnorm(24),4,6)
x[c(2,1),]#第2行和第1行
x[,c(1,3)] #第1列和第3列
x[2,1] #第[2,1]元素
x[x[,1]>0,1] #第1列大于0的元素
sum(x[,1]>0) #第1列大于0的元素的个数
sum(x[,1]<=0) #第1列不大于0的元素的个数
x[,-c(1,3)]#没有第1、3列的x
diag(x)  #x的对角线元素
diag(1:5) #以1:5为对角线元素,其他元素为0的对角线矩阵
diag(5) #5维单位矩阵
x[-2,-c(1,3)]#没有第2行，第1、3列的x
x[x[,1]>0&x[,3]<=1,1]#第1列>0并且第3列<=1的第1列元素
x[x[,2]>0|x[,1]<.51,1]#第1列<.51或者第2列>0的第1列元素
x[!x[,2]<.51,1]#第1列中相应于第2列>=.51的元素
apply(x,1,mean)#对行(第一维)求均值
apply(x,2,sum)#对列(第二维)求和
x=matrix(rnorm(24),4,6)
x[lower.tri(x)]=0;x #得到上三角阵，
#为得到下三角阵，用x[upper.tri(x)]=0)
```

高维数组:

```
x=array(runif(24),c(4,3,2));x
#上面用24个服从均匀分布的样本点构造4乘3乘2的三维数组
is.matrix(x)
dim(x)#得到维数(4,3,2)
is.matrix(x[1,,])#部分三维数组是矩阵
x=array(1:24,c(4,3,2))
x[c(1,3),,]
```

```
x=array(1:24,c(4,3,2))
apply(x,1,mean)   #可以对部分维做求均值运算
apply(x,1:2,sum)  #可以对部分维做求和运算
apply(x,c(1,3),prod) #可以对部分维做求乘积运算
```

矩阵与向量之间的运算:

```
x=matrix(1:20,5,4) #5乘4矩阵
sweep(x,1,1:5,"*")#把向量1:5的每个元素乘到每一行
sweep(x,2,1:4,"+")#把向量1:4的每个元素加到每一列
x*1:5
#下面把x标准化,即每一元素减去该列均值,除以该列标准差
(x=matrix(sample(1:100,24),6,4));(x1=scale(x))
(x2=scale(x,scale=F))#自己观察并总结结果
(x3=scale(x,center=F)) #自己观察并总结结果
round(apply(x1,2,mean),14) #自己观察并总结结果
apply(x1,2,sd)#自己观察并总结结果
round(apply(x2,2,mean),14);apply(x2,2,sd)#自己观察并总结结果
round(apply(x3,2,mean),14);apply(x3,2,sd)#自己观察并总结结果
```

缺失值及数据的合并:

```
airquality #有缺失值(NA)的R自带数据
complete.cases(airquality)#判断每行有没有缺失值
which(complete.cases(airquality)==F) #有缺失值的行号
sum(complete.cases(airquality)) #完整观测值的个数
na.omit(airquality) #删去缺失值的数据
#附加, 横或竖合并数据: append,cbind,rbind
x=1:10;x[12]=3
(x1=append(x,77,after=5))
cbind(1:5,rnorm(5))
rbind(1:5,rnorm(5))
cbind(1:3,4:6);rbind(1:3,4:6) #去掉矩阵重复的行
(x=rbind(1:5,runif(5),runif(5),1:5,7:11))
x[!duplicated(x),]
unique(x)
```

关于 list:

```
#list可以是任何对象(包括list本身)的集合
z=list(1:3,Tom=c(1:2,a=list("R",letters[1:5]),w="hi!"))
z[[1]];z[[2]]
z$T
z$T$a2
```

```
z$T[[3]]
z$T$w
for (i in z){print(i)}
```

条形图和表;

```
x=scan()#30个顾客在5个品牌中的挑选
3 3 3 4 1 4 2 1 3 2 5 3 1 2 5 2 3 4 2 2 5 3 1 4 2 2 4 3 5 2

barplot(x) #不合题意的图
table(x) #制表
barplot(table(x)) #正确的图
barplot(table(x)/length(x)) #比例图(和上图形状一样)
table(x)/length(x)
```

形成表格:

```
library(MASS)#载入程序包MASS
quine #MASS所带数据
attach(quine)#把数据变量的名字放入内存
#下面语句产生从该数据得到的各种表格
table(Age)
table(Sex, Age); tab=xtabs(~ Sex + Age, quine); unclass(tab)
tapply(Days, Age, mean)
tapply(Days, list(Sex, Age), mean)
detach(quine) #attach的逆运行
```

如何写函数:

```
#下面这个函数是按照定义(编程简单, 但效率不高)求n以内的素数
ss=function(n=100){z=2;
for (i in 2:n)if(any(i%%2:(i-1)==0)==F)z=c(z,i);return(z) }
fix(ss) #用来修改任何函数或编写一个新函数
ss() #计算100以内的素数
t1=Sys.time() #记录时间点
ss(10000) #计算10000以内的素数
Sys.time()-t1 #计算费了多少时间
system.time(ss(10000))#计算执行ss(10000)所用时间
#函数可以不写return,这时最后一个值为return的值
#为了输出多个值最好使用list输出
```

画图:

```
x=seq(-3,3,len=20);y=dnorm(x)#产生数据
w= data.frame(x,y)#合并x,成为数据w
par(mfcol=c(2,2))#准备画四个图的地方
plot(y ~ x, w,main="正态密度函数")
plot(y ~ x,w,type="l", main="正态密度函数")
plot(y ~ x,w,type="o", main="正态密度函数")
plot(y ~ x,w,type="b",main="正态密度函数")
par(mfcol=c(1,1))#取消par(mfcol=c(2,2))
```

色彩和符号等的调整:

```
plot(1,1,xlim=c(1,7.5),ylim=c(0,5),type="n") #画出框架
#在plot命令后面追加点(如要追加线可用lines函数):
points(1:7,rep(4.5,7),cex=seq(1,4,l=7),col=1:7, pch=0:6)
text(1:7,rep(3.5,7),labels=paste(0:6,letters[1:7]),
    cex=seq(1,4,l=7),col=1:7)#在指定位置加文字
points(1:7,rep(2,7), pch=(0:6)+7)#点出符号7到13
text((1:7)+0.25, rep(2,7), paste((0:6)+7))#加符号号码
points(1:7,rep(1,7), pch=(0:6)+14) #点出符号14到20
text((1:7)+0.25, rep(1,7), paste((0:6)+14)) #加符号号码
#关于符号形状、大小、颜色以及其他画图选项的说明可用"?par"来查看
```

如何得到函数源代码

对于一般的函数, 比如线性模型 lm, 可以用语句edit(lm) 来查看源代码, 或者用语句 mylm=edit(lm) 改写源代码. 对于某些函数, 比如函数 mean, 如果用 edit(mean) 则会出现下面结果:

```
function (x, ...)
UseMethod("mean")
```

这时, 你要用methods(mean) 语句来查看, 并得到下面结果:

```
> methods(mean)
[1] mean,ANY-method          mean,Matrix-method
[3] mean,sparseMatrix-method mean,sparseVector-method
[5] mean.Date                mean.default
[7] mean.difftime            mean.POSIXct
[9] mean.POSIXlt
see '?methods' for accessing help and source code
```

这时, 你可以用代码getAnywhere(mean.default) 得到源代码, 下面是输出:

```
A single object matching 'mean.default' was found
It was found in the following places
  package:base
  registered S3 method for mean from namespace base
  namespace:base
with value

function (x, trim = 0, na.rm = FALSE, ...)
{
   if (!is.numeric(x) && !is.complex(x) && !is.logical(x)) {
      warning("argument is not numeric or logical: returning NA")
      return(NA_real_)
   }
   if (na.rm)
      x <- x[!is.na(x)]
   if (!is.numeric(trim) || length(trim) != 1L)
      stop("'trim' must be numeric of length one")
   n <- length(x)
   if (trim > 0 && n) {
      if (is.complex(x))
         stop("trimmed means are not defined for complex data")
      if (anyNA(x))
         return(NA_real_)
      if (trim >= 0.5)
         return(stats::median(x, na.rm = FALSE))
      lo <- floor(n * trim) + 1
      hi <- n + 1 - lo
      x <- sort.int(x, partial = unique(c(lo, hi)))[lo:hi]
   }
   .Internal(mean(x))
}
<bytecode: 0x1198b8e00>
<environment: namespace:base>
```

附录 2 Python 简介——通过运行来领悟

引 言

一些人说 Python 比 R 好学, 而另一些人正相反, 觉得 R 更易掌握. 其实, 熟悉编程语言的人学哪一个都很快. 它们的区别大体如下. 由于有统一的志愿团队管理, R 的语法相对比较一致, 安装程序包很简单, 而且很容易找到帮助和支持, 但由于 R 主要用于数据分析, 所以一些对统计不那么熟悉的人可能会觉得太专业了. Python 则是一个通用软件, 比 C++ 容易学, 功能不差, 改进的 Cython 运行速度也非常快. 但是, Python 没有统一的团队管理, 针对不同 Python 版本的模块非常多. 因此对于不同的计算机操作系统, 不同版本的 Python、不同的模块, 安装过程多种多样, 首先遇到的就是安装问题. 另外, R 软件的基本语言 (即下载 R 之后所装的基本程序包) 本身就可以应付相当复杂的统计运算, 而相比之下 Python 统计模型没有那么多, 做一些统计分析不如 R 那么方便, 但由其基本语法产生的成千上万的模块使得它几乎可以做任何想做的事情.

在大数据时代的数据分析中, 最重要的不是掌握一两种编程语言, 而是掌握一种泛型编程能力, 有了这种能力, 语言之间的不同不会引发太多的烦恼.

由于 Python 是个应用广泛的通用软件, 这里只能介绍其中和数据分析有关的部分简单操作. 如果读者有疑问, 记住最好的老师是网络上的信息.

下面通过运行各种语句来领悟简单的语法, 我们尽量不做更多的解释.

安 装

安装

初学者可以使用 Anaconda 下载 Python Navigator [6], 以获得 Jupyter、RStudio、Visual Studio Code、IPython and Spyder 等软件界面, 可以选择你认为方便的方式运行 Python 程序. 使用 Anaconda 的好处是其包含了常用的模块 Numpy、Pandas、Matplotlib, 而且安装其他一些模块 (比如 Sklearn) 也比较方便.

这里不可能给出太多的安装细节, 因为这些都可能会发生变化, 相信读者会在网上找到需要的各种线索、提示和帮助. 下面对 Python 的介绍是基于 Anaconda 的 notebook 运行 Python3 的实践.

运行 Notebook

安装完 Anaconda 之后, 就可以运行 Notebook 了. 在 Mac 机或 Windows 中可以通过点击 Anaconda 图标, 然后选中 Notebook 或其他运行界面, 或者通过终端键入

[6] 网址: https://www.anaconda.com/distribution/.

cd Python Work 到达你的工作目录, 然后键入 jupyter notebook 在默认浏览器产生一个工作界面 (称为 "Home"). 如果你已经有文件, 则会有以书本图标开头的列表, 你的文件名以 .ipynb 为扩展名. 如果没有现成的, 可创建新的文件, 点击右上角 New 并选择 Python3, 则产生一个没有名字的 (默认是 Untitled) 以 .ipynb 为扩展名的文件 (自动存在你的工作目录中) 的一页, 文件名可以随时任意地更改.

在你的文件页中会出现 In [ ]: 标记, 可以在其右边的框中输入代码, 得到的结果会出现在代码 (代码所在的框称为 "Cell") 下面. 一个 Cell 中可有一群代码, 可以在其上下增加 Cell, 也可以合并或拆分 Cell, 相信读者会很快掌握这些小技巧.

你可以先键入:

```
3*' Python is easy!'
```

用 CTrlEnter+ 输出:

```
' Python is easy! Python is easy! Python is easy!'
```

实际上, 这行代码等价于 print(3*' Python is easy!'). 在一个 Cell 中, 如果有可以输出的几条语句, 则只输出有 print 的行及最后一行代码 (无论有没有 print) 的结果.

在 Python 中, 也可以一行输入几个简单 (不分行的) 命令, 用分号分隔. 要注意, Python 和 R 的代码都是分大小写的. Python 与 R 的注释一样, 在 # 号后面的符号不会被当成代码执行.

当前工作目录是在存取文件, 输入输出模块时只需输入文件或模块名称, 而不需输入路径的目录. 查看工作目录和改变工作目录的代码为:

```
import os
print (os.getcwd()) #查看目录
os.chdir('D:/Python work') #Windows系统中改变工作目录
os.chdir('/users/Python work') #OSx系统中改变工作目录
```

若要查看你某个目录下 (比如 " /users/work/") 的某种文件 (比如以 " .csv" 结尾的文件) 的路径名、文件名及大小, 可以用下面语句:

```
import os
from os.path import join
for (dirname, dirs, files) in os.walk('/users/work/'):
    for filename in files:
        if filename.endswith('.csv') :
            thefile = os.path.join(dirname,filename)
            print (thefile,os.path.getsize(thefile))
```

基本模块的编程

熟悉 R 的人首先不习惯的可能是 Python 中的向量、矩阵、列表或其他多元素对象的下标是从 0 开始的, 请输入下面代码并看输出:

```
y=[[1,2],[1,2,3],['ss','swa','stick']]
y[2],y[2][:2],y[1][1:]
```

从 0 开始的下标也有方便的地方, 比如下标 [:3] 实际上是左闭右开的整数区间 0,1,2, 类似地, [3:7] 是 3,4,5,6. 这样, 以首尾相接的形式 [:3], [3:7], [7:10] 实际上覆盖了从 0 到 9 的所有下标; 而在 R 中, 这种下标应该写成 [1:2], [3:6], [7:9], 中间的端点由于是闭区间, 没有重合. 请试运行下面语句, 一些首尾相接的下标区间可以得到完整的下标群:

```
x='A poet can survive everything but a misprint.'
x[:10]+x[10:20]+x[20:30]+x[30:40]+x[40:]
```

关于 append、extend 和 pop:

```
x=[[1,2],[3,5,7],'Oscar Wilde']
y=['save','the world']
x.append(y);print(x)
x.extend(y);print(x)
x.pop();print(x)
x.pop(2);print(x)
```

整数和浮点运算:

```
print (2**0.5,2.0**(1/2),2**(1/2.))
print( 4/3,4./3 )
```

关于 remove 和 del:

```
x=[0,1,4,23]
x.remove(4);print(x)
del x[0];print (x, type(x))
```

关于 tuple:

```
x =(0,12,345,67,8,9,'we','they')
print (type(x),x[-4:-1])
```

关于 range、 xrange 及一些打印格式:

```
x=range(2,11,2)
print ('x={}, list(x)={}'.format(x,list(x)))
print ('type of x is {}'.format(type(x)))
```

关于 dictionary(字典) 类型 (注意打印的次序与原来不一致):

```
data = {'age': 34, 'Children' : [1,2], 1: 'apple','zip': 'NA'}
print (type(data))
print ('age=',data['age'])
data['age'] = '99'
data['name'] = 'abc'
print (data)
```

一些集合运算:

```
x=set(['we','you','he','I','they']);y=set(['I','we','us'])
x.add('all');print (x,type(x),len(x))
set.add(x,'none');print (x)
print ('set.difference(x,y)=', set.difference(x,y))
print ('set.union(x,y)=',set.union(x,y))
print ('set.intersection(x,y)=',set.intersection(x,y))
x.remove('none')
print ('x=',x,'\n','y=', y)
```

用 id 函数来确定变量的存储位置 (是不是等同):

```
x=1;y=x;print (x,y,id(x),id(y))
x=2.0;print (x,y,id(x),id(y))
x = [1, 2, 3];y = x;y[0] = 10
print (x,y,id(x),id(y))
x = [1, 2, 3];y = x[:]
print (x,y,id(x)==id(y),id(x[0])==id(y[0]))
print (id(x[1])==id(y[1]),id(x[2])==id(y[2]))
```

函数的简单定义 (包括 lambda 函数) 及应用:

```
def f(x): return x**2-x
g=lambda x: max(x**2,x**3)
print (list(map(lambda x: x**2+1-abs(x), [1.2,5.7,23.6,6])))
print (f(10),g(-3.4))
print (list(range(-10,10,2)),'\n', list(filter(lambda x: x>0,range(-10,10,2))))
```

一般函数的定义 (注意在 Python 中, 函数、类、条件和循环等语句后面有冒号 ":", 而随后的行要缩进首先确定数目的若干空格 (和 R 中的花括号作用类似):

```
from random import *
def RandomHappy():
    if randint(1,100)>50:
        x='happy'
    else:
        x='unhappy'
    if randint(1,100)>50:
        y='happy'
    else:
        y='unhappy'
    if x=='happy' and y=='happy':
        print ('You both are happy')
    elif x!=y:
        print ('One of you is happy')
    else:
        print ('Both are unhappy')

RandomHappy() #执行函数
```

循环语句和条件语句, 比如:

```
# 例1
for line in open("UN.txt"):
    for word in line.split():
        if word.endswith('er'):
            print (word)

# 例2
with open('UN.txt') as f:
    lines=f.readlines()
lines[1:20]

# 例3
x='Just a word'
for i in  x:
    print (i)

# 例4
for i in  x.split():
    print (i,len(i))

# 例5
for i in [-1,4,2,27,-34]:
    if i>0 and i<15:
```

```
        print (i,i**2+i/.5)
    elif i<0 and abs(i)>5:
        print (abs(i))
    else:
        print (4.5**i)
```

一个关于 list 的例子:

```
x = range(5)
y = []
for i in range(len(x)):
    if float(i/2)==i/2:
        y.append(x[i]**2)
print ('y', y)
z=[x[i]**2 for i in range(len(x)) if float(i/2)==i/2]
print ('z',z)
```

Numpy 模块

在 Numpy 模块中, 首先输入这个模块, 比如用 `import numpy`, 这样, 凡是该模块的命令 (比如 `array`) 都要加上 `numpy` 成为 `numpy.array`. 如果嫌字母太多, 可以简写, 在输入时敲入 `import numpy as np`. 这样, `numpy.array` 就成为 `np.array`.

数据文件的存取:

```
import numpy as np
x = np.random.randn(25,5)
np.savetxt('tabs.txt',x)#存成制表符分隔的文件
np.savetxt('commas.csv',x,delimiter=',')#存成逗号分隔的文件(如csv)
u = np.loadtxt('commas.csv',delimiter=',')#读取逗号分隔文件
v = np.loadtxt('tabs.txt')#读取逗号分隔文件
```

矩阵和数组:

```
import numpy as np
y = np.array([[[1,4,7],[2,5,8]],[[3,6,9],[10,100,1000]]])
print (y)
print (np.shape(y))
print (type(y),y.dtype)
print (y[1,0,0],y[0,1,:])
```

整形和浮点型数组 (向量) 运算:

```
import numpy as np
u = [0, 1, 2];v=[5,2,7]
u=np.array(u);v=np.array(v)
print (u.shape,v.shape)
print (u+v,u/v,np.dot(u,v))
u = [0.0, 1, 2];v=[5,2,7]
u=np.array(u);v=np.array(v)
print (u+v,u/v)
print (v/3, v/3.,v/float(3),(v-2.5)**2)
```

向量和矩阵的维数转换和矩阵乘法的运算. 这里列出一些等价的做法, 请逐条执行和比较:

```
x=np.arange(3,5,.5)
y=np.arange(4)
print (x,y,x+y,x*y) #向量计算
print (x[:,np.newaxis].dot(y[np.newaxis,:]))
print (np.shape(x),np.shape(y))
print (np.shape(x[:,np.newaxis]),np.shape(y[np.newaxis,:]))
print (np.dot(x.reshape(4,1),y.reshape(1,4)))
x.shape=4,1;y.shape=1,4
print (x.dot(y))
print (np.dot(x,y))
print (np.dot(x.T,y.T), x.T.dot(y.T))#x.T是x的转置
print (x.reshape(2,2).dot(np.reshape(y,(2,2))))
x=[[2,3],[7,5]]
z = np.asmatrix(x)
print (z, type(z))
print (z.transpose() * z )
print (z.T*z== z.T.dot(z),z.transpose()*z==z.T*z)
print (np.ndim(z),z.shape)
```

分别按照列 (axis=0: 竖向) 或行 (axis=1: 横向) 合并矩阵, 和 R 的 rbind 及 cbind 类似:

```
x = np.array([[1.0,2.0],[3.0,4.0]])
y = np.array([[5.0,6.0],[7.0,8.0]])
z = np.concatenate((x,y),axis = 0)
z1 = np.concatenate((x,y),axis = 1)
print (z,"\n" ,z1,"\n",z.transpose()*z1)
z = np.vstack((x,y)) # Same as z = concatenate((x,y),axis = 0)
z1 = np.hstack((x,y))
print (z,"\n",z1)
```

数组的赋值:

```
print (np.ones((2,2,3)),np.zeros((2,2,3)),np.empty((2,2,3)))
x=np.random.randn(20).reshape(2,2,5);print (x)
x=np.random.randn(20).reshape(4,5)
x[0,:]=np.pi
print (x)
x[0:2,0:2]=0
print (x)
x[:,4]=np.arange(4)
print (x)
x[1:3,2:4]=np.array([[1,2],[3,4]])
print (x)
```

关于行列序列的定义, 这里的 np.c_[0:10:2] 是从 0 到 10, 间隔为 2 的列 (c) 序列, 而 np.r_[1:5:4j] 是从 1 到 5, 等间隔长度为 4 的行 (r) 序列:

```
print (np.c_[0:10:2],np.c_[0:10:2].shape)
print (np.c_[1:5:4j],np.c_[1:5:4j].shape)
print (np.r_[1:5:4j],np.r_[1:5:4j].shape)
```

网格及按照网格抽取数组 (矩阵) 的子数组:

```
print (np.ogrid[0:3,0:2:.5],'\n',np.mgrid[0:3,0:2:.5])
print (np.ogrid[0:3:3j,0:2:5j],'\n',np.mgrid[0:3:3j,0:2:5j])
x = np.reshape(np.arange(25.0),(5,5))
print ('x=\n',x)
print ('np.ix_(np.arange(2,4),[0,1,2])=\n',np.ix_(np.arange(2,4),[0,1,2]))
print ('ix_([2,3],[0,1,2])=\n',np.ix_([2,3],[0,1,2]))
print ('x[np.ix_(np.arange(2,4),[0,1,2])]=\n',
x[np.ix_(np.arange(2,4),[0,1,2])]) # Rows 2 & 3, cols 0, 1 and 2
print ('x[ix_([3,0],[1,4,2])]=\n', x[np.ix_([3,0],[1,4,2])])
print ('x[2:4,:3]=\n',x[2:4,:3])# Same, standard slice
print ('x[ix_([0,3],[0,1,4])]=\n',x[np.ix_([0,3],[0,1,4])])
```

舍入、加减乘除、差分、指数对数等各种对向量和数组的数学运算:

```
x = np.random.randn(3)
print ('np.round(x,2)={},np.round(x, 4)={}'.format(np.round(x,2),np.round(x, 4)))
print ('np.around(np.pi,4)=', np.around(np.pi,4))
print ('np.around(x,3)=', np.around(x,3))

print ('x.round(3)={},np.floor(x)={}'.format(x.round(3),np.floor(x)))
print ('np.ceil(x)={}, np.sum(x)={},'.format(np.ceil(x), np.sum(x)))
print ('np.cumsum(x)={},np.prod(x)={}'.format(np.cumsum(x),np.prod(x)))
```

```
print (',np.cumprod(x)={},np.diff(x)={}'.format(np.cumprod(x),np.diff(x)))

x= np.random.randn(3,4)
print ('x={},np.diff(x)={}'.format( x,np.diff(x)))
print ('np.diff(x,axis=0)=',np.diff(x,axis=0))
print ('np.diff(x,axis=1)=',np.diff(x,axis=1))
print ('np.diff(x,2,1)=', np.diff(x,2,1))
print ('np.sign(x)={}, np.exp(x)={}'.format(np.sign(x),np.exp(x)))
print ('np.log(np.abs(x))={},x.max()={}'.format(np.log(np.abs(x)),x.max()))
print (',x.max(1)={},,np.argmin(x,0)={}'.format(x.max(1),np.argmin(x,0)))
print ('np.max(x,0)={},np.argmax(x,0)={}'.format(np.max(x,0),np.argmax(x,0)))
print ('x.argmin(0)={},x[x.argmax(1)]={}'.format(x.argmin(0),x[:,x.argmax(1)]))
```

一些函数的操作:

```
x = np.repeat(np.random.randn(3),(2))
print (x)
print (np.unique(x))
y,ind = (np.unique(x, True))
print ('y={},ind={},x[ind]={},x.flat[ind]={}'.format(y,ind,x[ind],x.flat[ind]))

x = np.arange(10.0)
y = np.arange(5.0,15.0)
print ('np.in1d(x,y)=', np.in1d(x,y))
print ('np.intersect1d(x,y)=', np.intersect1d(x,y))
print ('np.union1d(x,y)=', np.union1d(x,y))
print ('np.setdiff1d(x,y)=' , np.setdiff1d(x,y))
print ('np.setxor1d(x,y)=',np.setxor1d(x,y))
x=np.random.randn(4,2)
print (x,'\n','\n',np.sort(x,1),'\n',np.sort(x,axis=None))
print ('np.sort(x,0)',np.sort(x,0))
print ('x.sort(0)',x.sort(axis=0) )
x=np.random.randn(3)
x[0]=np.nan #赋缺失值
print ('x{}\nsum(x)={}\nnp.nansum(x)={}'.format(x,sum(x),np.nansum(x)))
print ('np.nansum(x)/np.nanmax(x)=', np.nansum(x)/np.nanmax(x))
```

分割数组:

```
x = np.reshape(np.arange(24),(4,6))
y = np.array(np.vsplit(x,2))
z = np.array(np.hsplit(x,3))
print ('x={}\ny={}\nz={}'.format(x,y,z))
print (x.shape,y.shape,z.shape)
```

```
print (np.delete(x,1,axis=0)) #删除x第1行
print (np.delete(x,[2,3],axis=1)) #删除x第2,3列
print (x.flat[:], x.flat[:4]) #把x变成向量
```

矩阵的对角线元素与对角线矩阵:

```
x = np.array([[10,2,7],[3,5,4],[45,76,100],[30,2,0]])#same as R
y=np.diag(x) #对角线元素
print ('x={}\ny={}'.format(x,y))
print ('np.diag(y)=\n',np.diag(y)) #由向量形成对角线方阵
print ('np.triu(x)=\n' ,np.triu(x)) #x上三角阵
print ('np.tril(x)=\n',np.tril(x))#x下三角阵
```

一些随机数的产生:

```
print(np.random.randn(2,3))#随机标准正态2x3矩阵
#给定均值矩阵和标准差矩阵的随机正态矩阵:
print (np.random.normal([[1,0,3],[3,2,1]],[[1,1,2],[2,1,1]]))
print (np.random.normal((2,3),(3,1)))#均值为2,3标准差为3,1的2个随机正态数
print (np.random.uniform(2,3))#均匀U[2,3]随机数
np.random.seed(1010)#随机种子
print (np.random.random(10))#10个随机数(0~1之间)
print (np.random.randint(20,100))#20到100之间的随机整数
print (np.random.randint(20,100,10))#20到100之间的10个随机整数
print (np.random.choice(np.arange(-10,10,3)))#从序列随机选一个
x=np.arange(10);np.random.shuffle(x);print (x)
```

一些线性代数运算:

```
import numpy as np
x=np.random.randn(3,4)
print (x)
u,s,v= np.linalg.svd(x)#奇异值分解
Z=np.array([[1,-2j],[2j,5]])
print ('Cholesky:', np.linalg.cholesky(Z))#Cholesky分解
print ('x={}\nu={}\ndiag(s)={}\nv={}'.format(x,u,np.diag(s),v))
print (np.linalg.cond(x))#条件数
x=np.random.randn(3,3)
print (np.linalg.slogdet(x))#行列式的对数(及符号:1.为正-1.为负)
print (np.linalg.det(x)) #行列式
y=np.random.randn(3)
print (np.linalg.solve(x,y)) #解联立方程
X = np.random.randn(100,2)
y = np.random.randn(100)
```

```
beta, SSR, rank, sv= np.linalg.lstsq(X,y,rcond=None)#最小二乘法
print ('beta={}\nSSR={}\nrank={}\nsv={}'.format(beta, SSR, rank, sv))
#cov(x)方阵的特征值问题解:
va,ve=np.linalg.eig(np.cov(x))
print ('eigen value={}\neigen vectors={}'.format(va,ve))
x = np.array([[1,.5],[.5,1]])
print ('x inverse=', np.linalg.inv(x))#矩阵的逆
x = np.asmatrix(x)
print ('x inverse=', np.asmatrix(x)**(-1)) #注意使用**(-1)的限制
z = np.kron(np.eye(3),np.ones((2,2)))#单位阵和全1矩阵的Kronecker积
print ('z={},z.shape={}'.format(z,z.shape))
print ('trace(Z)={}, rank(Z)={}'.format(np.trace(z),np.linalg.matrix_rank(z)))
```

关于日期:

```
import datetime as dt
yr, mo, dd = 2016, 8, 30
print ('dt.date(yr, mo, dd)=',dt.date(yr, mo, dd))
hr, mm, ss, ms= 10, 32, 10, 11
print ('dt.time(hr, mm, ss, ms)=',dt.time(hr, mm, ss, ms))
print (dt.datetime(yr, mo, dd, hr, mm, ss, ms))
d1 = dt.datetime(yr, mo, dd, hr, mm, ss, ms)
d2 = dt.datetime(yr + 1, mo, dd, hr, mm, ss, ms)
print ('d2-d1', d2-d1 )
print (np.datetime64('2016'))
print (np.datetime64('2016-08'))
print (np.datetime64('2016-08-30'))
print (np.datetime64('2016-08-30T12:00')) # Time
print (np.datetime64('2016-08-30T12:00:01')) # Seconds
print (np.datetime64('2016-08-30T12:00:01.123456789')) # Nanoseconds
print (np.datetime64('2016-08-30T00','h'))
print (np.datetime64('2016-08-30T00','s'))
print (np.datetime64('2016-08-30T00','ms'))
print (np.datetime64('2016-08-30','W'))#Upcase!
dates = np.array(['2016-09-01','2017-09-02'],dtype='datetime64')
print (dates)
print (dates[0])
```

Pandas 模块

产生一个数据框 (类似于 R 的), 并存入 csv 及 Excel 文件 (指定 Sheet).

```
import pandas as pd
np.random.seed(1010)
w=pd.DataFrame(np.random.randn(10,5),columns=['X1','X2','X3','X4','Y'])
v=pd.DataFrame(np.random.randn(20,4),columns=['X1','X2','X3','Y'])
w.to_csv('Test.csv',index=False)
writer=pd.ExcelWriter('Test1.xlsx')
v.to_excel(writer,'sheet1',index=False)
w.to_excel(writer,'sheet2')
```

从 csv 及 Excel 文件 (指定 Sheet) 中读入数据:

```
W=pd.read_csv('Test.csv')
V=pd.read_excel('Test1.xlsx','sheet2')
U=pd.read_table('Test.csv',sep=',')
print ('V.head()=\n',V.head())#前5行
print ('U.head(2)=\n',U.head(2))#前2行
print ('U.tail(3)=\n',U.tail(3))#最后3行
print ('U.size={}\nU.columns={}'.format(U.size, U.columns))
U.describe() #简单汇总统计量
```

一个例子 (diamonds.csv):

```
diamonds=pd.read_csv("diamonds.csv")
print (diamonds.head())
print (diamonds.describe())
print ('diamonds.columns=',diamonds.columns)
print ('sample size=', len(diamonds)) #样本量
cut=diamonds.groupby("cut") #按照变量cut的各水平分群
print ('cut.median()=\n',cut.median())
print ('Cross table=\n',pd.crosstab(diamonds.cut, diamonds.color))
```

Matplotlib 模块

输入模块. 一般在 `plt.show` 之后显示独立图形, 可以对独立图形做一些编辑. 如果想在输出结果中看到 "插图", 则可用 `%matplotlib inline` 语句, 但没有独立图形那么方便.

```
#如果输入下一行代码, 则会产生输出结果之间的插图(不是独立的图)
#%matplotlib inline
import matplotlib.pyplot as plt
```

最简单的图:

```
y = np.random.randn(100)
plt.plot(y)
plt.plot(y,'g--')
plt.title('Random number')
plt.xlabel('Index')
plt.ylabel('y')
plt.show()
```

几张图:

```
import scipy.stats as stats
fig = plt.figure(figsize=(15,10))
ax = fig.add_subplot(2, 3, 1)#2x3图形阵
y = 50*np.exp(.0004 + np.cumsum(.01*np.random.randn(100)))
plt.plot(y)
plt.xlabel('time ($\tau$)')
plt.ylabel('Price',fontsize=16)
plt.title('Random walk: $d\ln p_t = \mu dt + \sigma dW_t$',fontsize=16)

y = np.random.rand(5)
x = np.arange(5)
ax = fig.add_subplot(2, 3, 5)
colors = ['#FF0000','#FFFF00','#00FF00','#00FFFF','#0000FF']
plt.barh(x, y, height = 0.5, color = colors, \
edgecolor = '#000000', linewidth = 5)
ax.set_title('Bar plot')

y = np.random.rand(5)
y = y / sum(y)
y[y < .05] = .05
ax = fig.add_subplot(2, 3, 3)
plt.pie(y)
ax.set_title('Pie plot')

z = np.random.randn(100, 2)
z[:, 1] = 0.5 * z[:, 0] + np.sqrt(0.5) * z[:, 1]
x = z[:, 0]
y = z[:, 1]
ax = fig.add_subplot(2, 3, 4)
plt.scatter(x, y)
ax.set_title('Scatter plot')

ax = fig.add_subplot(2, 3, 2)
x = np.random.randn(100)
```

```
ax.hist(x, bins=30, label='Empirical')
xlim = ax.get_xlim()
ylim = ax.get_ylim()
pdfx = np.linspace(xlim[0], xlim[1], 200)
pdfy = stats.norm.pdf(pdfx)
pdfy = pdfy / pdfy.max() * ylim[1]
plt.plot(pdfx, pdfy,'r-',label='PDF')
ax.set_ylim((ylim[0], 1.2 * ylim[1]))
plt.legend()
plt.title('Histogram')

ax = fig.add_subplot(2, 3, 6)
x = np.cumsum(np.random.randn(100,4), axis = 0)
plt.plot(x[:,0],'b-',label = 'Series 1')
plt.plot(x[:,1],'g-.',label = 'Series 2')
plt.plot(x[:,2],'r:',label = 'Series 3')
plt.plot(x[:,3],'h--',label = 'Series 4')
plt.legend()
plt.title('Random lines')
plt.show()
```

参考文献

[1] Aldenderfer, S., and Blashfield, K. (1985) *Cluster Ananlysis*. SAGE Publ.

[2] Arenas, C. and Cuadras, C.M. (2002) Some recent statistical methods based on distances. *Contributions to Science*, 2(2), 183-191.

[3] Bendixen, M. (1995), Compositional perceptual mapping using chi-squared tree analysis and Correspondence Analysis. *Journal of Marketing Management*, 11, 571-581.

[4] Breiman, L. (2001), Statistical Modeling: The Two Cultures, *Statistical Science* Vol. 16, No. 3, 199–231.

[5] Breiman, L. (2001), Random Forests, *Machine Learning* 45(1), 5-32.

[6] Cox, T. F. and Cox, M. A. A. (2001) *Multidimensional Scaling*. Second Edition. Chapman & Hall/CRC.

[7] De Leeuw, J. (1977). Applications of Convex Analysis to Multidimensional Scaling. In J. Barra, F. Brodeau, G. Romier, B. van Cutsem (eds.). *Recent Developments in Statistics*, pp. 133 – 145. North Holland Publishing Company, Amsterdam, The Netherlands.

[8] De Leeuw, J, Heiser, W.J. (1977). Convergence of Correction Matrix Algorithms for Multidi- mensional Scaling. In J Lingoes (ed.), *Geometric Representations of Relational Data*, pp. 735–752. Mathesis Press, Ann Arbor, Michigan.

[9] De Leeuw J, Heiser W.J. (1980). Multidimensional Scaling with Restrictions on the Con- figuration. In P Krishnaiah (ed.), *Multivariate Analysis*, Volume V, pp. 501–522. North Holland Publishing Company, Amsterdam, The Netherlands.

[10] DeSarbo, W. S. and Cron, W. L. (1988) A maximum likelihood methodology for clusterwise linear regression. *Journal of Classification*, 5, 249-282.

[11] Eisen, M.B., Spellman, P.T., Brown, P.O, and Botstein, D. (1998) Cluster analysis and display of genome-wide expression patterns. *Proc Natl Acad Sci U S A*. 1998 Dec 8;95(25):14863-8.

[12] Ester, M., Kriegel, H-P. Sander, J. and Xu, X. (1996). A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. Institute for Computer Science, University of Munich. *Proceedings of 2nd International Conference on Knowledge Discovery and Data Mining (KDD-96).*

[13] Fang, Y. and Wang, J. (2012) Selection of the number of clusters via the bootstrap method. *Computational Statistics and Data Analysis*, 56, 468-477.

[14] Feuerverger, A. (1993) A consistent test for bivariate dependence. *International Statistical Review*, 61(3): 419–433.

[15] Frank, A. & Asuncion, A. (2010) UCI machine learning repository [http://archive.ics.uci.edu/ml]. Irvine, CA: University of California, School of Information and Computer Science.

[16] Goktas, A., and Isci, O. (2011) A Comparison of the Most Commonly Used Measures of Association for Doubly Ordered Square Contingency Tables via Simulation. *Advances in Methodology & Statistics*, 8(1): p17.

[17] Gonzalez, R., and Nelson, TO. (1996) Measuring ordinal association in situations that contain tied scores. *Psychol Bull*, 119(1): 159–165. pmid:8559859

[18] Goodman, L. A. (1979) Simple models for the analysis of association in cross-classifications having ordered categories. *Journal of the American Statistical Association*, 74(367): 537–552.

[19] Goodman, L. A., and Kruskal, W. H. (1954) Measures of association for cross classifications. *Journal of the American Statistical Association*, 49: 732–764.

[20] Graham, R. L., Lnuth, D. E., and Patashnik, O. (1988) *Concrete Mathematics.* Addison-Wesley, Reading MA.

[21] Gower J. (1971) A general coefficient of similarity and some of its properties. *Biometrics*, 27: 857–871.

[22] Greenacre, M.J. (1984). *Theory and Applications of Correspondence Analysis.* London: Academic Press.

[23] Greenacre, M.J. (1994). Multiple and joint correspondence analysis. In M.Greenacre and J. Blasius (eds), *Correspondence Analysis in the Social Sciences*, pp. 141-161. London: Academic Press.

[24] Healey, J.F. (2013), *The Essentials of Statistics: A Tool for Social Research*, 3rd ed., Belmont, Wadsworth.

[25] Henderson and Velleman (1981), Building multiple regression models interactively. *Biometrics*, 37, 391–411.

[26] Hennig, C. (2002) Fixed point clusters for linear regression: computation and comparison, *Journal of Classification*, 19, 249-276.

[27] Hennig, C. (2005) Fuzzy and Crisp Mahalanobis Fixed Point Clusters, in Baier, D., Decker, R., and Schmidt-Thieme, L. (eds.): *Data Analysis and Decision Support*. Springer, Heidelberg, 47-56.

[28] Hennig, C. (2007) Cluster-wise assessment of cluster stability. *Computational Statistics and Data Analysis*, 52, 258-271.

[29] Hennig, C. (2008) Dissolution point and isolation robustness: robustness criteria for general cluster analysis methods. *Journal of Multivariate Analysis*, 99, 1154-1176.

[30] Hennig, C. (2010) Methods for merging Gaussian mixture components. *Advances in Data Analysis and Classification*, 4, 3-34.

[31] Hennig, C. and Christlieb, N. (2002) Validating visual clusters in large datasets: Fixed point clusters of spectral features. *Computational Statistics and Data Analysis*. 40, 723-739.

[32] Hennig, C. and Liao, T. (2013) How to find an appropriate clustering for mixed-type variables with application to socio-economic stratification. *Journal of the Royal Statistical Society*, Series C Applied Statistics, 62, 309-369.

[33] Hotelling, H. (1936) Relations between two sets of variants. *Biometrika*, 28, 321-377.

[34] Irigoien, I. and Arenas, C. (2008) INCA: New statistic for estimating the number of clusters and identifying atypical units. *Statistics in Medicine*, 27(15): 2948–2973.

[35] Lawson, R G., and Jurs, Peter C. (1990). New Index for Clustering Tendency and Its Application to Chemical Problems. *Journal of Chemical Information and Computer Sciences*, 30 (1): 36–41.

[36] Liu, P., Zhou, D., and Wu, N. (2007) VDBSCAN: Varied Density Based Spatial Clustering of Applications with Noise. *International Conference on Service Systems and Service Management (ICSSSM)*, pp.1-4.

[37] Lyons R. (2013) Distance covariance in metric spaces. *The Annals of Probability*, 41(5): 3284–3305.

[38] Massart, D. L., Vandeginste, B. G. M., Buydens, L. M. C., de Jong, S., Lewi, P. J., Smeyers-Verbeke, J. (1998) *Handbook of Chemometrics and Qualimetrics*: Part B. Elsevier. Tables 35.1 and 35.4.

[39] Megiddo, N., and Supowit, K. (1984) On the complexity of some common geometric locations problems. *SIAM J. on Comp*, Vol. 13(1). P. 182–196.

[40] Rousseeuw, P.J. (1987) Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *J. Comput. Appl. Math.*, 20, 53–65.

[41] Sammon, J. W. (1969) A non-linear mapping for data structure analysis. *IEEE Transactions on Computers*, C-18, 401–409.

[42] Sebastiani, P., and Perls, T. T. (2016) Detection of Significant Groups in Hierarchical Clustering by Resampling, Front Genet. 2016; 7: 144. Published online 2016 Aug 8. doi: 10.3389/fgene.2016.00144, PMCID: PMC4976109.

[43] Siersma, V., and Kreiner, S. (2009) A coefficient of association between categorical variables with partial or tentative ordering of categories. *Sociological Methods Research*, 38(2): 265–286.

[44] Spaeth, H. (1983) *Cluster-Formation und Analyse*. Oldenbourg Verlag.

[45] Struyf, A., Hubert, M., and Rousseeuw, P. J., (1996): Clustering in an Object-Oriented Environment. *Journal of Statistical Software*, 1. `http://www.stat.ucla.edu/journals/jss/`.

[46] Székely, G. J., Rizzo, M. L., and Bakirov, N. K. (2007) Measuring and testing independence by correlation of distances. *The Annals of Statistics*, 35(6): 2769–2794. View Article PubMed/NCBI Google Scholar.

[47] Székely, G. J., and Rizzo, M, L. (2009) Brownian distance covariance. *The Annals of Applied Statistics*, 3(4): 1236–1265.

[48] Székely, G. J., and Rizzo, M. L. (2013) The distance correlation t-test of independence in high dimension. *Journal of Multivariate Analysis*, 117: 193–213.

[49] Tibshirani, R., Walther, G. and Hastie, T. (2001). Estimating the number of data clusters via the Gap statistic. *Journal of the Royal Statistical Society*, B, 63, 411–423.

[50] Tyler, D. E., Critchley, F., Duembgen, L., Oja, H. (2009) Invariant coordinate selection (with discussion). *Journal of the Royal Statistical Society*, Series B, 549-592.

[51] Tibshirani, R. and Walther, G. (2005) Cluster validation by prediction strength. *Journal of Computational and Graphical Statistics*, 14, 511-528.

[52] Tsai, C-F. and Liu, C-W. (2006) KIDBSCAN: A new efficient data clustering algorithm. *ICAISC*, pp. 702-711.

[53] Venables, W. N. & Ripley, B. D. (2002) *Modern Applied Statistics with S.* Fourth Edition. Springer, New York.

[54] Vannahme, I.M. (1996) *Clusteralgorithmen zur Mathematischen Simulation von Bausparkollektiven: doctoral thesis.* Cologne: Univ. of Cologne.

[55] 吴喜之. 应用回归与分类——基于 R. 北京: 中国人民大学出版社, 2016.

教师教学服务说明

中国人民大学出版社工商管理分社以出版经典、高品质的工商管理、财务会计、统计、市场营销、人力资源管理、运营管理、物流管理、旅游管理等领域的各层次教材为宗旨。

为了更好地为一线教师服务，近年来工商管理分社着力建设了一批数字化、立体化的网络教学资源。教师可以通过以下方式获得免费下载教学资源的权限：

在中国人民大学出版社网站 www.crup.com.cn 进行注册，注册后进入“会员中心”，在左侧点击“我的教师认证”，填写相关信息，提交后等待审核。我们将在一个工作日内为您开通相关资源的下载权限。

如您急需教学资源或需要其他帮助，请在工作时间与我们联络：

中国人民大学出版社　工商管理分社
联系电话：010-62515735，82501048，62515782，62515987
电子邮箱：rdcbsjg@crup.com.cn
通讯地址：北京市海淀区中关村大街甲 59 号文化大厦 1501 室（100872）

[General Information]
书名=14565844
SS号=14565844